

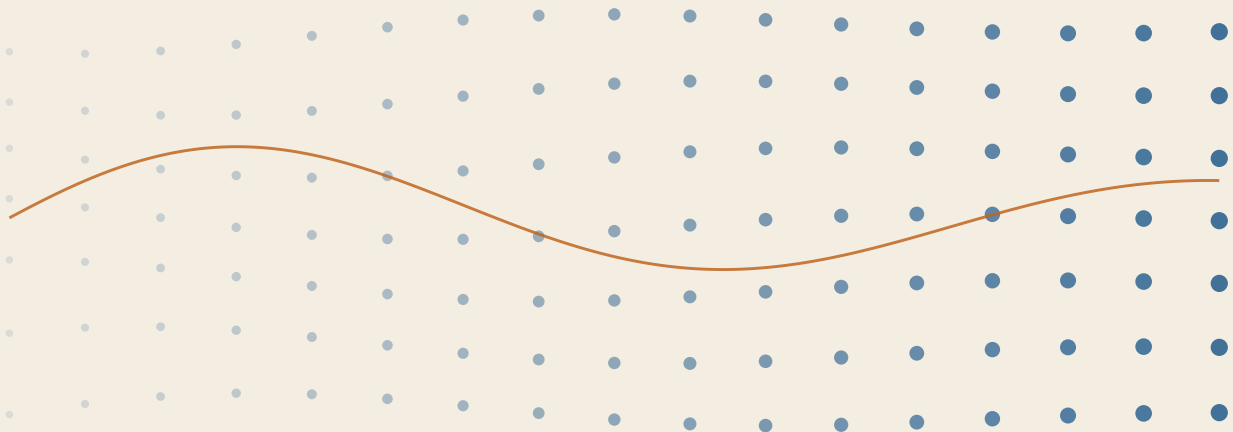
CATATAN KULIAH MASTER ARTIFICIAL INTELLIGENCE DI LINZ

Belajar Mesin Belajar

*dari data dan neuron sampai sinyal,
molekul, dan fluida*

Eigenzayn

Edisi pertama, Oktober 2026



Belajar Mesin Belajar

Catatan kuliah Master Artificial Intelligence di Linz

Isi buku ini ditulis ulang dengan kata-kata sendiri dari kuliah yang diikuti, lalu diperluas dengan bacaan di luar kuliah: bagian perkembangan riset di akhir bab, beberapa bab yang disusun dari literatur, dan lampiran. Slide, naskah, dan soal ujian asli tetap milik para pengajarnya dan tidak disertakan. Semua rujukan di daftar pustaka diperiksa terhadap Crossref, arXiv, atau halaman hak cipta bukunya. Laporan, tanggal, dan angka peristiwa terbaru diperiksa dengan sumber resminya pada saat penulisan.

Disusun dengan $\text{\LaTeX}$.

Prakata

Pada semester musim dingin 2024/25 saya mulai belajar kecerdasan buatan di Linz. Semester pertama langsung padat: computer vision, jaringan saraf, dan LSTM yang diajarkan oleh penemunya sendiri. Setiap kuliah meninggalkan setumpuk slide. Dua tahun kemudian, map di laptop berisi ratusan PDF.

Program master ini mensyaratkan 120 ECTS untuk lulus. Saya mengambil lebih dari itu. Rasa ingin tahu membawa saya dari fondasi machine learning ke radar dan kendali, dari genom dan protein ke simulasi fluida, dari teori belajar statistik ke informasi kuantum, sampai ke kuliah tentang hukum, psikologi, dan cara berbicara tentang AI kepada publik. Karena itulah buku ini mencakup bidang yang luas. Cerita AI terasa tidak lengkap tanpa robot yang bergerak, molekul yang dirancang mesin, simulasi fisika yang dipercepat, dan manusia yang hidup bersama semua itu.

Slide bagus untuk mengikuti kuliah, tetapi kurang enak dibaca ulang. Kalimatnya terpotong, rumus muncul tanpa cerita, dan hubungan antar mata kuliah jarang terlihat. Padahal hubungan itulah yang paling menarik. Konvolusi di kuliah computer vision ternyata saudara dekat stensil beda hingga di kuliah mekanika fluida. Model difusi di kuliah machine learning lanjut ternyata bersandar pada persamaan Fokker–Planck dari fisika statistik. Kalman filter di kuliah model probabilistik ternyata cara yang sama yang dipakai ahli cuaca untuk menggabungkan simulasi dengan pengukuran, dan teorema yang sama tentang estimasi muncul lagi di radar dan di pengukuran kuantum.

Buku ini adalah catatan perjalanan itu. Saya menulis ulang isi kuliah dengan bahasa sendiri, menambahkan gambaran yang dulu membantu saya memahaminya, dan menurunkan rumus-rumus pentingnya langkah demi langkah, seperti yang dulu saya kerjakan di kuliah. Tetapi buku ini juga lebih dari catatan kuliah. Bidang ini bergerak lebih cepat dari slide yang mengajarkannya, sehingga hampir setiap bab ditutup dengan bagian *Perkembangan riset* yang merangkum arah penelitian beberapa tahun terakhir. Beberapa bab, seperti bab tentang keselamatan AI, disusun dari bacaan di luar kuliah, dan dua lampiran disediakan untuk pembaca yang ingin menyegarkan bekal matematikanya atau mulai meneliti sendiri.

Gaya penulisannya sengaja dibuat seperti catatan yang dibaca, bukan buku latihan. Setiap gagasan besar diberi satu gambaran dari kehidupan sehari-hari sebelum rumusnya muncul. Analogi tidak pernah sempurna, dan kalau sebuah analogi mulai menyesatkan, teks akan mengatakannya. Contoh hitungan kecil ditulis di dalam paragraf, supaya pembaca bisa mengikutinya dengan pensil kalau mau. Istilah teknis dibiarkan dalam bahasa Inggris dan dicetak miring saat pertama muncul, karena istilah itulah yang dipakai di kuliah dan di makalah. Bab Peta Perjalanan berikut memberi saran cara membaca buku ini, baik bagi yang ingin mengikuti setiap penurunan maupun bagi yang ingin langsung melompat ke topik yang paling menarik baginya.

Setiap bab diakhiri catatan singkat tentang mata kuliah asal dan bacaan utamanya. Slide dan soal asli tetap milik para pengajarnya dan tidak disertakan. Yang ada di sini adalah pemahaman saya atas materi itu, jadi kekeliruan yang tersisa adalah kekeliruan saya.

Linz, Oktober 2026

Daftar Isi

Prakata	iii
0 Peta Perjalanan	1
I Fondasi	6
1 Belajar dari Data	7
2 Memilih Model dengan Statistik dan Bayes	16
3 Teori Belajar Statistik	22
4 Optimasi: Turun Gunung dalam Kabut	27
5 AI Klasik: Mencari, Merencanakan, Memutuskan	35
6 Belajar Tanpa Label	43
II Deep Learning dan Persepsi	49
7 Deep Learning: Teknik Dasar	50
8 Deep Learning dalam Praktik	59
9 Mengingat: dari RNN ke LSTM dan Transformer	64
10 Bahasa: dari Kantong Kata ke Model Bahasa Besar	73
11 Melihat: Computer Vision	79
12 Arsitektur Modern dan Model Generatif	89
13 Geometri dan Simetri	96
14 Bernalar dalam Ketidakpastian	106
15 Dari Estimasi ke Difusi	114
16 Belajar dari Ganjaran: Reinforcement Learning	124
17 Membuka Kotak Hitam: AI dan Visualisasi	133

III Sinyal, Sensor, dan Sistem	140
18 Menangkap Sinyal: Estimasi, Filter, dan Radar	141
19 Mengendalikan Sistem	151
20 Komputasi yang Menghilang	160
21 Robot dan Kendaraan Otonom	168
IV AI untuk Ilmu Hayati	176
22 Membaca Kode Kehidupan	177
23 Molekul, Obat, dan Data Medis	185
24 Bentuk Protein	195
25 Melihat ke Dalam Tubuh	202
V AI untuk Simulasi	209
26 Fluida yang Ingin Kita Tiru	210
27 Dari Persamaan ke Solver	216
28 Fisika Komputasi	223
29 Physics-Informed Neural Networks	230
30 Belajar Menyelesaikan: Neural Operator dan Simulator	236
31 Menemukan dan Memampatkan Dinamika	242
32 Model Hibrida: Fisika dan Data Bersama	247
33 Recurrence CFD: Simulasi dari Ingatan	253
VI AI, Manusia, dan Masa Depan	259
34 Belajar dari Jejak Pengguna	260
35 AI, Hukum, dan Manusia	267
36 Kecerdasan Umum, Superintelligence, dan Keselamatan AI	273
37 Informasi Kuantum dan Machine Learning Kuantum	278

38 Ke Mana Arah AI	283
A Glosarium	290
B Bekal Matematika	300
C Meneliti, Menerbitkan, dan Mematenkan	304
Bibliografi	308
Indeks	351

0 Peta Perjalanan

Sebelum masuk ke materi, ada baiknya melihat peta besarnya dulu: mata kuliah apa saja yang dilalui, dalam urutan apa, dan bagaimana semuanya tersusun menjadi buku ini.

0.1 Empat semester

Tabel 1 menyusun mata kuliah menurut semester ketika saya mengikutinya. Urutan bab di buku tidak persis mengikuti urutan waktu. Bab disusun menurut alur gagasan: fondasi lebih dulu, lalu deep learning, lalu simulasi. Tabel ini berguna bagi pembaca yang ingin membaca buku ini sambil mengikuti kuliah serupa.

Bab-bab tentang *scientific machine learning* (Bab 29 sampai 32) tidak berasal dari satu mata kuliah. Isinya dirangkai dari beberapa kuliah, terutama machine learning lanjut, metode numerik fluida, dan fisika komputasi, ditambah bacaan selama mengerjakan tugas akhir.

Program master ini mensyaratkan 120 ECTS untuk lulus, dan tabel di atas memuat lebih banyak dari itu. Mata kuliah yang melebihi syarat kelulusan tetap masuk ke buku, sebab merekalah yang membuat ceritanya utuh: robot yang bergerak, molekul yang dirancang dengan bantuan mesin, sinyal radar yang harus dipisahkan dari derau, rekomendasi yang kita terima setiap hari, dan pertanyaan hukum serta psikologis yang muncul ketika semua itu bertemu manusia.

Buku ini terbagi menjadi enam bagian. Bagian I berisi fondasi: apa artinya belajar dari data, bagaimana statistik dan Bayes membantu memilih model, apa yang bisa dijamin teori belajar, bagaimana mencari parameter terbaik, bagaimana AI klasik mencari dan merencanakan, dan bagaimana menemukan struktur tanpa label. Bagian II membahas deep learning dan persepsi: teknik dasar dan praktiknya, ingatan dari RNN sampai xLSTM, bahasa, penglihatan, model generatif, simetri, penalaran probabilistik, model difusi, reinforcement learning, dan explainable AI. Bagian III turun ke dunia sinyal dan sistem fisik: estimasi dan filter, radar, kendali, komputasi pervasif, dan robot. Bagian IV masuk ke ilmu hayati: genom, molekul dan obat, protein, dan citra medis. Bagian V membahas AI untuk simulasi fisika, bidang tugas akhir saya. Bagian VI menutup dengan sistem rekomendasi, hukum dan manusia, keselamatan AI, informasi kuantum, dan perkembangan AI terbaru. Tiga lampiran melengkapinya: glosarium, bekal matematika, dan panduan singkat untuk meneliti, menerbitkan, dan mematenkan.

Hampir setiap bab punya tiga lapis. Lapis pertama adalah isi kuliah, ditulis ulang dengan penurunan dan contoh hitungan. Lapis kedua adalah bagian “di luar kelas” yang menghubungkan materi dengan bidang lain di buku ini. Lapis ketiga adalah bagian *Perkembangan riset*, yang merangkum makalah-makalah beberapa tahun terakhir.

Semua makalah di lapis ketiga diperiksa terhadap catatan resminya, dan isinya ditulis berdasarkan abstrak yang benar-benar dibaca, bukan dari ingatan.

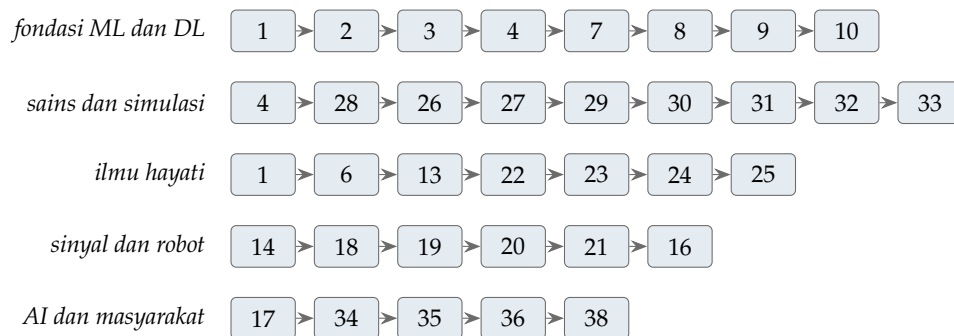
0.2 Cara membaca buku ini

Buku ini bisa dibaca dengan dua cara, dan keduanya sama sahnya.

Pembaca yang ingin mengikuti setiap penurunan akan terbantu oleh tiga bekal: kalkulus, terutama turunan dan aturan rantai; aljabar linear, terutama perkalian matriks, nilai eigen, dan dekomposisi nilai singular; serta peluang dan statistik dasar, termasuk ekspektasi, varians, distribusi Gauss, dan teorema Bayes. Semuanya dirangkum di Lampiran B, yang bisa dibaca lebih dulu atau dibuka setiap kali sebuah rumus terasa asing. Rumus di buku ini jarang muncul tanpa diturunkan, dan contoh hitungannya sengaja dibuat kecil supaya bisa diperiksa dengan pensil.

Pembaca yang mencari wawasan tidak perlu berhenti di setiap rumus. Paragraf di sekitar persamaan selalu menjelaskan maknanya dengan kata-kata, sehingga persamaan bisa dilewati tanpa kehilangan alur. Bab-bab tentang AI dan manusia, keselamatan AI, dan perkembangan terbaru (Bab 35, 36, dan 38) hampir tidak membutuhkan matematika sama sekali, dan bisa langsung dibaca kapan saja. Begitu juga bab-bab penerapan, seperti ilmu hayati atau robot, yang bisa dibaca sebagai cerita tentang apa yang dilakukan AI di bidang-bidang itu.

Bagi yang ingin membaca sebagian saja, Gambar 1 menyarankan lima jalur. Setiap jalur bisa diikuti sendiri, dan bab-bab yang dilewati selalu bisa dikunjungi belakangan lewat rujukan silang di dalam teks dan indeks di akhir buku.



GAMBAR 1. Lima jalur baca. Angka adalah nomor bab. Jalur terbawah hampir tidak membutuhkan matematika.

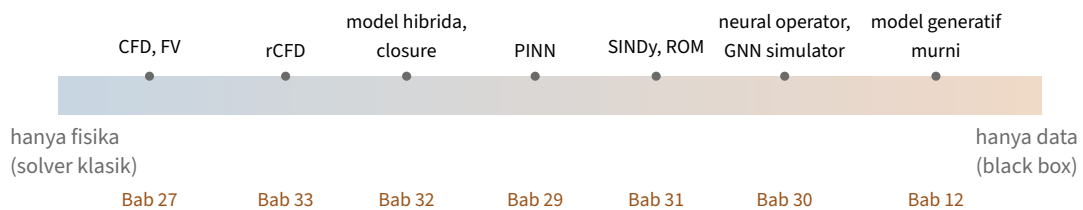
0.3 Mengapa simulasi

Simulasi fisika seperti *computational fluid dynamics* (CFD) menyelesaikan persamaan diferensial parsial di jutaan sel. Hasilnya akurat, tetapi mahal: satu kasus bisa berjalan sehari-hari di kluster komputer. Machine learning menawarkan jalan pintas. Ia belajar dari hasil simulasi sebelumnya, lalu menebak hasil baru dalam hitungan detik. Masalahnya, model yang hanya belajar dari data tidak tahu apa-apa tentang kekekalan massa atau energi. Ia bisa menghasilkan aliran yang terlihat meyakinkan tetapi

melanggar fisika.

Saya suka membayangkan perbedaan ini seperti dua cara memasak. Juru masak pertama mengikuti resep secara persis: takaran, suhu, waktu. Hasilnya bisa diulang, tetapi lambat, karena setiap langkah harus ditimbang. Juru masak kedua memasak dengan lidah. Ia mencicipi, mengingat rasa masakan sebelumnya, lalu menebak. Cepat, tetapi kadang meleset jauh. Juru masak terbaik memakai keduanya: resep menjaga agar masakan tidak gagal total, lidah membuatnya cepat dan luwes. Solver CFD adalah resep, machine learning adalah lidah, dan *physics-informed machine learning* adalah usaha memakai keduanya sekaligus.

Di antara dua ujung itu terbentang spektrum yang lebar. Gambar 26.2 menempatkan pendekatan-pendekatan yang akan kita temui di sepanjang buku.



GAMBAR 2. Spektrum pendekatan simulasi, dari solver berbasis persamaan sampai model berbasis data. Semakin ke kanan, semakin cepat, tetapi semakin sedikit jaminan fisiknya.

Literatur memakai beberapa nama untuk wilayah ini: *scientific machine learning*, *physics-informed machine learning* (Karniadakis et al., 2021), atau *physics-based deep learning* (Thuerey et al., 2021). Tinjauan yang khusus membahas mekanika fluida ditulis oleh Brunton et al. (2020), Vinuesa and Brunton (2022), dan Lino et al. (2023). Di buku ini saya memakai istilah yang lebih sederhana: AI untuk simulasi.

0.4 Tiga pintu masuk fisika

Karniadakis et al. (2021) membedakan tiga cara memasukkan pengetahuan fisika ke dalam model pembelajaran. Pembagian ini akan muncul berulang kali, jadi ada gunanya dikenal sejak awal.

Pintu pertama adalah data. Kalau sebuah model dilatih dengan cukup banyak hasil simulasi yang benar, ia ikut meniru pola fisika yang terkandung di dalamnya. Inilah yang disebut *observational bias*, dan contohnya adalah neural operator yang dilatih dari ribuan solusi persamaan (Bab 30). Pintu kedua adalah arsitektur. Model bisa dirancang agar otomatis mematuhi sifat tertentu, misalnya tetap berlaku ketika sistemnya diputar atau digeser. Konvolusi, yang dengan sendirinya menghormati pergeseran (Bab 7), dan jaringan yang menghormati rotasi (Bab 13) termasuk di sini. Ini *inductive bias*. Pintu ketiga adalah fungsi loss. Kita menambahkan suku yang menghukum setiap pelanggaran persamaan fisika, sehingga model belajar mematuhihinya selama pelatihan. Ini *learning bias*, dan contoh paling terkenalnya adalah PINN (Bab 29).

Setiap kali bertemu model baru di bab-bab berikutnya, ada tiga pertanyaan yang layak diajukan. Lewat pintu mana fisika masuk: data, arsitektur, atau loss? Apa yang

terjadi kalau model dipakai di luar jangkauan data latihnya, misalnya untuk waktu yang lebih panjang atau geometri yang berbeda? Dan berapa biaya melatihnya dibanding biaya menjalankan solver klasik? Tiga pertanyaan ini akan menemani kita sampai akhir.

0.5 Notasi

Buku ini memakai notasi yang umum di kuliah. Vektor dicetak tebal ($\mathbf{x}$), matriks huruf besar tebal ($\mathbf{W}$), skalar huruf miring (x). Data latih ditulis $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$, dan parameter model ditulis $\mathbf{w}$ atau $\boldsymbol{\theta}$. Ekspektasi ditulis $\mathbb{E}[\cdot]$, distribusi normal $\mathcal{N}(\mu, \sigma^2)$, dan divergensi Kullback–Leibler $\text{KL}(q \parallel p)$. Pada bab-bab fluida, $\mathbf{u}$ berarti kecepatan, p tekanan, ρ massa jenis, ν viskositas kinematik, dan T temperatur.

TABEL 1. Mata kuliah menurut semester dan bab yang membahasnya.

Semester	Mata kuliah	Bab
Musim dingin 2024/25	Deep Learning and Neural Nets I	7
	LSTM and Recurrent Neural Nets	9
	Computer Vision	11
	Machine Learning: Supervised Techniques	1
	Control Systems	19
	Production Automation Systems	19
	Optimum and Adaptive Signal Processing Systems	18
	AI in Society, AI and Law I	35
	Patent and Literature Searching	C
Musim panas 2025	Deep Learning and Neural Nets II	12, 8
	Deep Reinforcement Learning	16
	Machine Learning: Unsupervised Techniques	6
	Machine Learning and Pattern Classification	6
	Statistical Learning Theory	3
	Introduction to Autonomous Vehicles	21
	Introduction to Robotic Systems	21
	Learning from User-generated Data	34
	Introduction to Genetics; Genome Analysis and Transcriptionomics	22
	Structural Bioinformatics	24
	AI in Life Sciences	23
	Deep Learning for Medical Imaging	25
	Robopsychology; Technology and Society; AI and Law II	35
	Quantum Information	37
Musim dingin 2025/26	Machine Learning: Advanced Techniques	15
	Probabilistic Models	14
	AI and Visualization	17
	Machine Learning: Basic Techniques	1, 2
	Reinforcement Learning	16
	Numerical Methods in Fluid Mechanics	26, 27
	Computational Physics I	28
	Optimum and Adaptive Signal Processing Systems (latihan)	18
	Radar System Engineering (latihan)	18
	Pervasive Computing: Systems and Environments; Design and Development	20
	KS Operations Research	5
	Natural Language Processing	10
	Communicating AI	35
Musim panas 2026	Introduction to AI	5
	Deep Learning: Geometric Techniques	13
	Numerical Optimization for AI	4
	Tugas akhir: recurrence CFD	33

Bagian I

Fondasi

1 Belajar dari Data

Kerangka di bab ini tidak datang dari satu kuliah. Ia muncul pertama kali di kuliah supervised learning pada semester pertama, lalu kembali di kuliah unsupervised, di kuliah dasar-dasar machine learning, dan di bagian pembuka kuliah geometric deep learning. Karena dipakai oleh begitu banyak kuliah, saya menaruhnya di depan.

Semua yang ada di buku ini, dari LSTM sampai PINN, berdiri di atas satu gagasan sederhana: kita punya contoh, dan kita ingin aturan yang juga berlaku untuk contoh yang belum pernah dilihat. Bab ini membahas gagasan itu secara jujur. Apa yang sebenarnya kita minimalkan? Mengapa model yang sempurna di data latih bisa gagal di data baru? Dan mengapa dimensi tinggi membuat semuanya lebih sulit?

1.1 Tiga cara belajar

Bayangkan tiga murid. Murid pertama belajar dengan kunci jawaban: setiap soal latihan punya jawaban benar, dan ia menyesuaikan cara berpikirnya sampai jawabannya cocok. Murid kedua hanya diberi setumpuk soal tanpa jawaban. Ia mengelompokkan soal yang mirip dan menemukan polanya sendiri. Murid ketiga tidak diberi soal sama sekali. Ia mencoba sesuatu, lalu gurunya hanya berkata “bagus” atau “kurang”, kadang baru setelah banyak langkah.

Murid pertama mewakili *supervised learning*, di mana setiap input x datang bersama target y . Murid kedua mewakili *unsupervised learning*, di mana hanya ada x dan kita mencari strukturnya: kelompok, arah utama, atau bentuk distribusinya. Murid ketiga mewakili *reinforcement learning*, tempat sebuah agen bertindak, menerima *reward*, dan harus menebak tindakan mana yang pantas dipuji (Bab 16). Di antara ketiganya ada bentuk campuran. *Self-supervised learning*, misalnya, membuat target dari data itu sendiri, seperti menebak kata yang ditutup dalam sebuah kalimat. Hampir semua model bahasa besar dilatih dengan cara ini. Bab ini berfokus pada murid pertama, karena kerangka berpikirnya dipakai di mana-mana.

1.2 Kerangka supervised learning

Kita menganggap data berasal dari sebuah distribusi gabungan $p(x, y)$ yang tidak kita ketahui. Yang kita miliki hanyalah sampel $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ yang diambil secara independen dari distribusi itu, asumsi yang biasa disebut *i.i.d.* Kita lalu memilih sebuah model $f(x; w)$ dari suatu *model class* $\mathcal{F}$, dan sebuah *loss function* $L(y, f(x))$ yang mengukur seberapa buruk tebakan kita. Dua besaran kemudian menjadi pusat

perhatian:

$$R(f) = \mathbb{E}_{(x,y) \sim p} [L(y, f(x))] = \int L(y, f(x)) p(x, y) dx dy, \quad (1.1)$$

$$\hat{R}_N(f) = \frac{1}{N} \sum_{n=1}^N L(y_n, f(x_n)). \quad (1.2)$$

Yang pertama disebut *generalization error* atau risiko sejati. Yang kedua disebut *empirical risk* atau *training error*. Yang sebenarnya kita inginkan adalah $R(f)$ kecil, tetapi yang bisa kita hitung hanya $\hat{R}_N(f)$. *Empirical risk minimization* (ERM) memilih $\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_N(f)$ dan berharap $R(\hat{f})$ ikut kecil. Hampir seluruh teori generalisasi pada dasarnya bertanya kapan harapan ini masuk akal.

Seandainya kita mengetahui $p(x, y)$ dengan sempurna, ada prediktor terbaik yang mungkin, yaitu *Bayes-optimal predictor* f^* , dengan risiko $R^* = R(f^*)$ yang disebut *Bayes risk*. Untuk loss kuadrat, prediktor itu adalah rata-rata bersyarat $f^*(x) = \mathbb{E}[y | x]$. Alasannya mudah dilihat. Tetapkan satu x dan cari bilangan c yang meminimalkan $\mathbb{E}[(y - c)^2 | x]$. Turunan terhadap c adalah $-2 \mathbb{E}[y - c | x]$, yang bernilai nol tepat ketika $c = \mathbb{E}[y | x]$. Kalau loss diganti dengan $|y - c|$, jawaban optimalnya berubah menjadi median bersyarat. Jadi pilihan loss menentukan statistik apa yang akan ditiru oleh model. Untuk klasifikasi dengan loss 0–1, prediktor Bayes memilih kelas dengan peluang bersyarat terbesar. Bayes risk umumnya tidak nol, karena data sendiri berisik: dua pasien dengan hasil laboratorium yang sama bisa saja punya diagnosis yang berbeda.

1.3 Dari asumsi noise ke fungsi loss

Kuliah dasar-dasar machine learning menekankan satu hal yang sering terlewat: fungsi loss bukan pilihan sembarang. Ia lahir dari asumsi tentang bagaimana data dibangkitkan. Misalkan $y = f(x; w) + \varepsilon$ dengan noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. Likelihood satu titik data adalah

$$p(y_n | x_n, w) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_n - f(x_n; w))^2}{2\sigma^2}\right).$$

Karena data i.i.d., likelihood seluruh data adalah hasil kali, dan negatif log-likelihoodnya menjadi

$$-\log p(\mathcal{D} | w) = \frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - f(x_n; w))^2 + \frac{N}{2} \log(2\pi\sigma^2). \quad (1.3)$$

Suku kedua tidak bergantung pada w . Maka *maximum likelihood* dengan noise Gauss sama persis dengan meminimalkan *mean squared error*. Dengan jalan yang sama, noise Laplace menghasilkan loss mutlak $|y - f|$, keluaran Bernoulli dengan peluang $\sigma(f)$ menghasilkan *binary cross-entropy*, dan keluaran kategorikal dengan softmax menghasilkan *cross-entropy* (Tabel 1.1).

Kasus Bernoulli layak diturunkan juga, karena loss ini dipakai di hampir semua pengklasifikasi. Misalkan label $y \in \{0, 1\}$ dan model mengeluarkan peluang $p = \sigma(f(x; w))$ bahwa $y = 1$. Peluang satu label bisa ditulis dalam satu rumus, $p(y | x) =$

$p^y(1-p)^{1-y}$, karena untuk $y = 1$ rumus itu menjadi p dan untuk $y = 0$ menjadi $1 - p$. Negatif log-likelihood seluruh data adalah

$$-\log p(\mathcal{D} | \mathbf{w}) = -\sum_{n=1}^N \left[y_n \log p_n + (1 - y_n) \log(1 - p_n) \right], \quad (1.4)$$

dan itulah binary cross-entropy. Sebagai contoh, kalau model memberi peluang 0,9 untuk contoh yang labelnya satu, loss-nya $-\ln 0,9 \approx 0,105$. Kalau peluangnya hanya 0,1, loss-nya $-\ln 0,1 \approx 2,30$, lebih dari dua puluh kali lebih besar. Loss ini menghukum keyakinan yang salah dengan keras sekali, dan sifat itulah yang membuat model terdorong memberi peluang yang terkalibrasi.

TABEL 1.1. Asumsi noise dan loss yang dihasilkannya.

Asumsi pada $y \mathbf{x}$	Loss dari $-\log$ likelihood	Sifat
Gauss $\mathcal{N}(f, \sigma^2)$	kuadrat, $(y - f)^2$	peka terhadap pencilan
Laplace	mutlak, $ y - f $	lebih tahan pencilan
Bernoulli, $p = \sigma(f)$	binary cross-entropy	klasifikasi dua kelas
Kategorikal, $p = \text{softmax}(f)$	cross-entropy	klasifikasi banyak kelas

Timbangan dapur memberi gambaran yang pas. Kalau timbangan Anda kadang meleset sedikit ke atas atau ke bawah secara acak, rata-rata dari beberapa kali menimbang adalah tebakan yang baik, dan itulah loss kuadrat. Tetapi kalau sesekali seekor kucing melompat ke atas timbangan, satu hasil bisa meleset jauh sekali. Rata-rata akan ikut tertarik oleh angka itu, sedangkan median tidak. Noise berekor tebal seperti Laplace menghasilkan loss mutlak, yang tidak terlalu peduli pada satu pencilan.

1.4 Prior sebagai regularisasi

Maximum likelihood hanya mendengarkan data. Kalau datanya sedikit, model mudah percaya pada kebetulan. Pendekatan Bayesian menambahkan *prior* $p(\mathbf{w})$, yaitu keyakinan awal tentang parameter sebelum melihat data. Teorema Bayes kemudian memberi *posterior*

$$p(\mathbf{w} | \mathcal{D}) = \frac{p(\mathcal{D} | \mathbf{w}) p(\mathbf{w})}{p(\mathcal{D})},$$

dan estimasi *maximum a posteriori* (MAP) memaksimalkannya. Dalam bentuk logaritma,

$$\hat{\mathbf{w}}_{\text{MAP}} = \arg \min_{\mathbf{w}} \left[-\log p(\mathcal{D} | \mathbf{w}) - \log p(\mathbf{w}) \right]. \quad (1.5)$$

Dengan prior Gauss $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1} \mathbf{I})$, suku kedua menjadi $\frac{\alpha}{2} \|\mathbf{w}\|_2^2$. Itulah *weight decay*, atau *ridge regression* dalam bahasa statistik. Dengan prior Laplace, suku kedua menjadi $\lambda \|\mathbf{w}\|_1$, yang dikenal sebagai lasso (Tibshirani, 1996). Lasso punya sifat yang menarik: ia cenderung membuat banyak bobot bernilai tepat nol. Sifat ini nanti menjadi inti metode SINDy untuk menemukan persamaan dari data (Bab 31).

Dalam simulasi, kita sering tahu banyak hal sebelum melihat data. Aliran harus kekal massa, temperatur dalam kelvin tidak boleh negatif, dan solusi harus simetris

kalau geometrinya simetris. Semua itu bisa dibaca sebagai prior yang kuat. PINN (Bab 29) memasukkan persamaan diferensial ke dalam loss dengan cara yang mirip suku $-\log p(\mathbf{w})$ di atas, hanya saja “prior”-nya menyangkut fungsi keluaran, bukan bobot.

1.5 Regresi linear sampai tuntas

Semua gagasan di atas bisa dilihat bekerja pada model paling sederhana. Regresi linear memodelkan $\hat{y} = \mathbf{w}^\top \mathbf{x}$, dengan komponen konstan disisipkan ke dalam $\mathbf{x}$ sebagai fitur bernilai satu. Untuk N contoh yang disusun sebagai baris matriks $\mathbf{X}$ dan target $\mathbf{y}$, loss kuadrat rata-rata adalah $\frac{1}{N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$. Gradiennya terhadap $\mathbf{w}$ adalah $-\frac{2}{N} \mathbf{X}^\top (\mathbf{y} - \mathbf{X}\mathbf{w})$, dan menyamakannya dengan nol memberi *normal equations*

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y}, \quad \hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (1.6)$$

Namanya berasal dari tafsiran geometrisnya. Syarat $\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}) = \mathbf{0}$ berarti sisa $\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}$ tegak lurus, *normal*, terhadap setiap kolom $\mathbf{X}$. Prediksi $\mathbf{X}\hat{\mathbf{w}}$ adalah proyeksi ortogonal $\mathbf{y}$ ke ruang yang direntang fitur-fitur, titik terdekat yang bisa dicapai model.

Untuk satu fitur ditambah konstanta, rumusnya menyederhana menjadi kemiringan $\hat{w}_1 = S_{xy}/S_{xx}$ dan titik potong $\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$, dengan $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y})$ dan $S_{xx} = \sum (x_i - \bar{x})^2$. Ambil tiga titik (1, 2), (2, 3), dan (3, 5). Rata-ratanya $\bar{x} = 2$ dan $\bar{y} = 10/3$. Simpangan x adalah $(-1, 0, 1)$ dan simpangan y adalah $(-4/3, -1/3, 5/3)$, sehingga $S_{xy} = 4/3 + 0 + 5/3 = 3$ dan $S_{xx} = 2$. Kemiringannya 1,5 dan titik potongnya $10/3 - 3 = 1/3$. Garis $\hat{y} = 1/3 + 1,5x$ meramal 1,83, 3,33, dan 4,83, dengan sisa 0,17, -0,33, dan 0,17. Jumlah sisanya nol, dan jumlah sisa dikali x juga nol, persis seperti yang diminta normal equations.

Ridge regression menambahkan $\lambda \|\mathbf{w}\|^2$, dan dengan langkah yang sama solusinya menjadi $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$. Penambahan λ pada diagonal punya dua efek. Secara statistik, ia menarik bobot ke nol, sesuai prior Gauss dari Subbab 1.4. Secara numerik, ia membuat matriks yang dibalik selalu bisa dibalik dan kondisinya lebih baik. Kalau dua fitur hampir identik, $\mathbf{X}^\top \mathbf{X}$ hampir singular, dan regresi biasa bisa memberi bobot +1000 dan -999 yang saling menghapus. Ridge memilih 0,5 dan 0,5.

1.6 Dari regresi ke klasifikasi

Untuk label biner, regresi logistik memodelkan peluang kelas positif sebagai $p = \sigma(\mathbf{w}^\top \mathbf{x})$ dengan $\sigma(a) = 1/(1 + e^{-a})$. Loss-nya adalah binary cross-entropy yang diturunkan di atas. Gradiennya punya bentuk yang sangat rapi. Turunan sigmoid adalah $\sigma'(a) = \sigma(a)(1 - \sigma(a))$, dan turunan $-y \ln p - (1 - y) \ln(1 - p)$ terhadap p adalah $-y/p + (1 - y)/(1 - p)$. Mengalikan keduanya lewat aturan rantai,

$$\frac{\partial \mathcal{L}}{\partial a} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p} \right) p(1 - p) = -y(1 - p) + (1 - y)p = p - y, \quad (1.7)$$

sehingga $\nabla_{\mathbf{w}} \mathcal{L} = (p - y)\mathbf{x}$. Gradiennya adalah galat prediksi dikali input, bentuk yang sama dengan regresi linear. Kalau model yakin $p = 0,9$ untuk contoh yang ternyata

negatif, gradiennya $0,9x$, dorongan kuat untuk memperbaiki. Kalau ia sudah benar dengan $p = 0,99$ untuk contoh positif, dorongannya hanya $-0,01x$. Berbeda dengan regresi linear, tidak ada solusi tertutup, karena p bergantung nonlinear pada w , sehingga regresi logistik dilatih dengan metode iteratif dari Bab 4. Batas keputusannya, tempat $p = 0,5$, adalah bidang $w^\top x = 0$: regresi logistik adalah pengklasifikasi linear dengan keluaran peluang.

1.7 Mengukur generalisasi

Risiko sejati (1.1) tidak bisa dihitung, jadi kita memperkirakannya. Cara paling sederhana adalah menyisihkan sebagian data yang tidak pernah disentuh selama pelatihan, lalu menghitung loss rata-rata di sana. Karena titik-titik uji itu independen dari model yang sudah jadi, rata-ratanya adalah estimator tak bias untuk $R(\hat{f})$.

Kesulitan muncul karena kita juga harus memilih *hyperparameter*: jumlah lapisan, laju belajar, kekuatan regularisasi. Kalau data uji dipakai untuk memilih, ia ikut bocor ke dalam model dan estimasinya menjadi terlalu optimistis. Karena itu data biasanya dibagi tiga: data latih, data validasi untuk memilih hyperparameter, dan data uji yang disentuh sekali saja di akhir. Kalau data sedikit, *k-fold cross-validation* membagi data menjadi k bagian dan melatih model k kali, setiap kali dengan satu bagian sebagai validasi. Rata-rata skornya menjadi estimasi generalisasi, dengan biaya k kali pelatihan. Perlu diingat bahwa cross-validation menghasilkan estimasi kinerja, bukan satu model. Model akhir biasanya dilatih ulang pada seluruh data latih.

Ada beberapa kebocoran yang sering terjadi tanpa disadari. Menormalisasi seluruh data, termasuk data uji, sebelum dibagi berarti statistik data uji ikut masuk ke pelatihan. Rata-rata dan simpangan baku seharusnya dihitung dari data latih saja. Untuk deret waktu atau hasil simulasi yang berurutan, membagi data secara acak juga menyesatkan. Titik-titik yang berdekatan dalam waktu hampir identik, sehingga ujian menjadi terlalu mudah. Data seperti ini sebaiknya dibagi menurut waktu atau menurut kasus simulasi.

1.8 Bias, varians, dan dua bentuk kurva

Ambil loss kuadrat dan anggap $y = g(x) + \varepsilon$ dengan $\mathbb{E}[\varepsilon] = 0$ dan $\text{Var}[\varepsilon] = \sigma^2$. Model $\hat{f}_{\mathcal{D}}$ bergantung pada data latih $\mathcal{D}$ yang acak. Untuk satu titik x , ekspektasi galatnya terhadap semua kemungkinan data latih dan noise terurai menjadi tiga suku (Geman et al., 1992):

$$\mathbb{E}[(y - \hat{f}_{\mathcal{D}}(x))^2] = \underbrace{(g(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)])^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)])^2]}_{\text{variens}} + \underbrace{\sigma^2}_{\text{noise}}. \quad (1.8)$$

Bias mengukur seberapa jauh rata-rata model dari kebenaran. Varians mengukur seberapa banyak model berubah kalau data latihnya diganti. Noise tidak bisa dikurangi oleh model apa pun.

Penurunannya hanya memakai satu trik: tambahkan dan kurangkan besaran yang sama. Tulis singkat $\hat{f} = \hat{f}_{\mathcal{D}}(x)$, $\bar{f} = \mathbb{E}_{\mathcal{D}}[\hat{f}]$, dan $g = g(x)$. Karena $y = g + \varepsilon$,

$$y - \hat{f} = \varepsilon + (g - \bar{f}) + (\bar{f} - \hat{f}).$$

Kuadratkan, lalu ambil ekspektasi. Noise ε berasal dari titik uji dan independen dari data latih, sehingga semua suku silang yang memuat ε hilang karena $\mathbb{E}[\varepsilon] = 0$. Suku silang $2(g - \bar{f})\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}]$ juga hilang, karena $g - \bar{f}$ tidak acak dan $\mathbb{E}_{\mathcal{D}}[\hat{f}] = \bar{f}$. Yang tersisa adalah tiga kuadrat: $\mathbb{E}[\varepsilon^2] = \sigma^2$, $(g - \bar{f})^2$, dan $\mathbb{E}_{\mathcal{D}}[(\hat{f} - \bar{f})^2]$, persis tiga suku di atas.

Contoh angka membuatnya konkret. Misalkan nilai sebenarnya di sebuah titik $g = 2$, noise bervarians $\sigma^2 = 0,25$, dan kita melatih model yang sama pada tiga himpunan data latih berbeda, yang memberi prediksi 1,6, 2,0, dan 2,1 di titik itu. Rata-rata prediksinya $\bar{f} = 1,9$, sehingga bias kuadratnya $(2 - 1,9)^2 = 0,01$. Variansnya $\frac{1}{3}[(1,6 - 1,9)^2 + (2,0 - 1,9)^2 + (2,1 - 1,9)^2] = \frac{1}{3}(0,09 + 0,01 + 0,04) \approx 0,047$. Galat kuadrat harapannya $0,01 + 0,047 + 0,25 \approx 0,31$, dan di sini noise adalah bagian terbesar. Model yang lebih rumit bisa saja menurunkan bias, tetapi tidak akan menyentuh 0,25 itu.

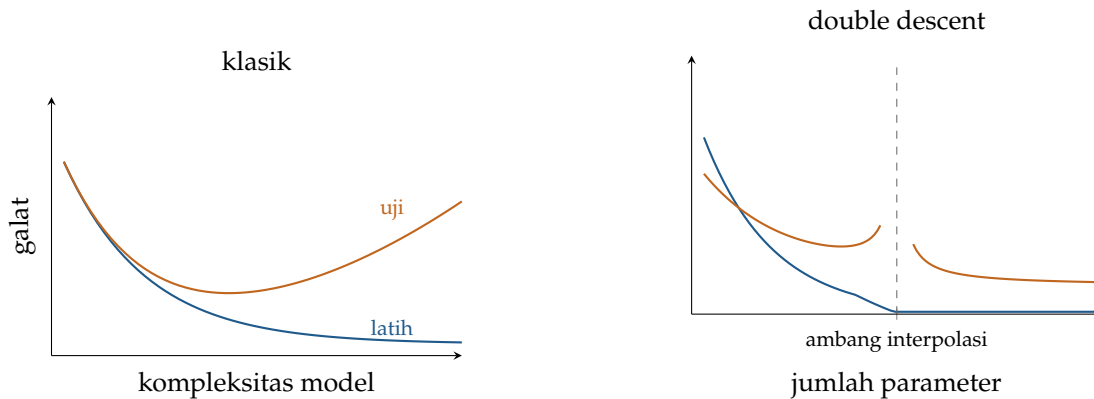
Papan dart membantu membayangkannya. Pemain dengan bias kecil dan varians kecil mengumpulkan semua anak panah di tengah. Pemain dengan bias besar tetapi varians kecil mengumpulkannya rapat di pojok kiri atas: konsisten, tetapi salah arah. Pemain dengan bias kecil tetapi varians besar menyebar ke mana-mana, meski rata-ratanya tepat di tengah. Model yang terlalu sederhana mirip pemain kedua. Model yang terlalu rumit mirip pemain ketiga, karena setiap set data latih membuatnya melempar ke tempat yang berbeda.

Gambaran klasik mengatakan bahwa ketika kompleksitas model bertambah, bias turun dan varians naik, sehingga galat uji membentuk huruf U dengan titik terbaik di tengah. Gambaran ini benar untuk banyak model klasik, tetapi jaringan saraf modern sering punya parameter jauh lebih banyak daripada jumlah data, dan tetap generalisasi dengan baik. Belkin et al. (2019) menunjukkan bahwa setelah *interpolation threshold*, yaitu titik ketika model baru saja mampu menghafal semua data latih, galat uji bisa turun lagi. Kurva lengkapnya disebut *double descent* (Gambar 1.1). Sebelumnya, Zhang et al. (2016) memperlihatkan bahwa jaringan yang sama bisa menghafal label acak dengan sempurna. Artinya, kapasitas mentah tidak menjelaskan generalisasi. Yang menentukan adalah solusi mana yang dipilih oleh algoritme pelatihan di antara banyak solusi yang sama-sama menghafal data. Stochastic gradient descent tampaknya cenderung memilih solusi yang halus, sebuah topik yang muncul lagi di Bab 4.

Kuliah geometric deep learning dan teori pembelajaran statistik memakai cara lain untuk memecah galat, yang lebih umum. Misalkan f^* prediktor Bayes, $f_{\mathcal{F}}$ prediktor terbaik di dalam model class, $\hat{f}$ hasil ERM, dan $\tilde{f}$ hasil algoritme optimasi yang sungguh kita jalankan. Maka

$$R(\tilde{f}) - R^* = \underbrace{R(f_{\mathcal{F}}) - R^*}_{\text{aproksimasi}} + \underbrace{R(\hat{f}) - R(f_{\mathcal{F}})}_{\text{estimasi}} + \underbrace{R(\tilde{f}) - R(\hat{f})}_{\text{optimasi}}. \quad (1.9)$$

Model class yang lebih besar mengecilkan galat aproksimasi, tetapi bisa membesarkan galat estimasi, karena semakin banyak fungsi yang kebetulan cocok dengan data. Galat optimasi muncul karena kita hampir tidak pernah benar-benar menemukan minimum global.



GAMBAR 1.1. Kiri, kurva U klasik. Kanan, double descent: setelah model mampu menghafal semua data latih, galat uji bisa turun lagi. Kedua kurva bersifat skematik.

1.9 Kutukan dimensi

Ada satu hal yang membuat machine learning di dimensi tinggi berbeda secara mendasar dari intuisi kita di dimensi rendah. Namanya *curse of dimensionality*. Ambil kubus satuan $[0, 1]^d$ dan kubus kecil di tengahnya dengan sisi 0,9. Fraksi volume di dalam kubus kecil adalah $0,9^d$. Untuk $d = 1$ hasilnya 0,9. Untuk $d = 10$ hasilnya sekitar 0,35. Untuk $d = 100$ hasilnya kira-kira $2,7 \times 10^{-5}$. Hampir seluruh volume kubus berdimensi seratus berada di kulit tipis dekat permukaannya. Titik acak hampir selalu dekat ke tepi, dan hampir semua pasangan titik punya jarak yang mirip.

Akibatnya, metode lokal seperti k -nearest neighbour kehilangan makna. Untuk menutup ruang berdimensi d dengan ketelitian ϵ , kita butuh sekitar ϵ^{-d} titik. Kuliah geometric deep learning menyatakannya secara formal: untuk kelas fungsi Lipschitz di d dimensi, galat estimasi terbaik turun hanya seperti $N^{-1/d}$. Dengan $d = 100$, menggandakan jumlah data hampir tidak membantu.

Mencari kunci yang jatuh di lorong panjang masih mudah. Di lantai sebuah aula, butuh waktu jauh lebih lama. Di gedung bertingkat yang penuh ruangan, kita mulai putus asa. Sekarang bayangkan gedung dengan seratus arah yang berbeda: sebanyak apa pun senter yang dibawa, sebagian besar ruang tetap gelap.

Lalu mengapa deep learning bisa bekerja pada gambar berukuran jutaan piksel? Ada dua jawaban yang akan muncul berkali-kali di buku ini. Jawaban pertama adalah *manifold hypothesis*: data nyata tidak memenuhi seluruh ruang. Gambar wajah manusia menempati semacam permukaan berdimensi jauh lebih rendah di dalam ruang piksel. Kalau model bisa menemukan permukaan itu, dimensi efektif masalahnya kecil. Jawaban kedua adalah simetri. Kita tahu bahwa kucing tetap kucing kalau digeser sedikit ke kiri. Kalau arsitektur dibuat invarian atau ekuivarian terhadap simetri seperti ini, ruang hipotesis menyusut drastis tanpa membuang solusi yang benar. Itulah gagasan pokok Bronstein et al. (2021), yang kita bahas di Bab 13.

1.10 Tidak ada makan siang gratis

Teorema *no free lunch* (Wolpert, 1996) menyatakan bahwa kalau dirata-ratakan atas semua kemungkinan masalah, tidak ada algoritme belajar yang lebih baik dari yang lain. Kedengarannya pesimistis, tetapi pesannya praktis: keberhasilan selalu datang dari asumsi yang cocok dengan masalahnya. Konvolusi bekerja untuk gambar karena gambar punya struktur lokal dan simetri geser. PINN bekerja untuk masalah yang persamaannya kita ketahui. Tanpa asumsi, tidak ada generalisasi.

1.11 Mengukur kinerja klasifikasi

Untuk klasifikasi dua kelas, hasil prediksi disusun dalam *confusion matrix* yang berisi *true positive* (TP), *false positive* (FP), *true negative* (TN), dan *false negative* (FN). Dari keempatnya diperoleh

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad \text{specificity} = \frac{TN}{TN + FP}.$$

Misalkan sebuah tes menandai 50 dari 1000 orang sebagai positif, dan 40 di antaranya memang sakit, sementara total orang yang sakit ada 60. Maka $TP = 40$, $FP = 10$, dan $FN = 20$, sehingga $\text{precision} = 40/50 = 0,80$ dan $\text{recall} = 40/60 \approx 0,67$. Precision menjawab “kalau tes bilang positif, seberapa bisa dipercaya?”, recall menjawab “dari semua yang sakit, berapa yang tertangkap?”. Keduanya sering dirangkum dengan rata-rata harmonik, $F_1 = 2 \cdot \frac{0,80 \cdot 0,67}{0,80 + 0,67} \approx 0,73$. Rata-rata harmonik dipakai karena ia tertarik ke nilai yang lebih kecil: model tidak bisa menutupi recall yang buruk dengan precision yang sempurna.

Akurasi bisa menipu kalau kelasnya tidak seimbang. Pada data dengan 99 persen kelas negatif, model yang selalu menjawab “negatif” punya akurasi 99 persen dan recall nol. Kurva ROC memplot recall terhadap $1 - \text{specificity}$ untuk semua ambang keputusan. Luas di bawahnya, AUC, sama dengan peluang model memberi skor lebih tinggi pada satu contoh positif acak dibanding satu contoh negatif acak.

Tafsiran peluang AUC memberi cara menghitungnya langsung. Misalkan model memberi skor 0,9 dan 0,6 untuk dua contoh positif, serta 0,7, 0,4, dan 0,2 untuk tiga contoh negatif. Ada $2 \times 3 = 6$ pasangan positif–negatif. Skor 0,9 lebih tinggi dari ketiga skor negatif. Skor 0,6 lebih tinggi dari 0,4 dan 0,2, tetapi kalah dari 0,7. Dari enam pasangan, lima diurutkan dengan benar, sehingga $\text{AUC} = 5/6 \approx 0,83$. Nilai 0,5 berarti urutan acak, dan nilai satu berarti setiap positif diberi skor lebih tinggi dari setiap negatif. AUC hanya menilai urutan, bukan kalibrasi: mengalikan semua skor dengan setengah tidak mengubah AUC sama sekali, walau peluang yang dikeluarkan model menjadi salah semua.

1.12 Generalisasi di luar kelas

Pertanyaan-pertanyaan di bab ini akan muncul lagi di setiap bidang yang dibahas buku ini, dengan wajah yang berbeda-beda.

Yang pertama adalah *distribution shift*: data saat model dipakai tidak berasal dari distribusi yang sama dengan data latih. Model citra medis yang dilatih di satu rumah

sakit bisa gagal di rumah sakit lain dengan pemindai berbeda (Bab 25). Model pengenalan aktivitas yang dilatih pada segelintir orang bisa gagal pada orang dengan gaya gerak lain (Bab 20). Model pengganti simulasi yang dilatih pada bilangan Reynolds antara 100 dan 1000 belum tentu benar pada 5000 (Bab 30).

Yang kedua adalah kejujuran pembagian data. Di kimia, molekul dari satu seri senyawa hampir identik, sehingga pembagian acak membuat model tampak jauh lebih baik dari kenyataannya (Bab 23). Di sensor yang dikenakan, jendela-jendela dari orang yang sama berkorelasi kuat. Di simulasi, potret berurutan dari satu kasus hampir sama. Pola umumnya: bagilah data menurut satuan yang benar-benar baru saat model dipakai, entah itu pasien, orang, kerangka molekul, atau kasus simulasi.

Yang ketiga adalah mahalnnya data. Satu label ahli radiologi, satu pengukuran assay, atau satu simulasi CFD penuh bisa memakan waktu dan biaya besar. Di sinilah prior, dalam bentuk simetri, pengetahuan domain, atau hukum fisika, menjadi berharga: ia mengurangi data yang dibutuhkan dan menjaga model tetap masuk akal di luar daerah yang pernah dilihatnya.

1.13 Perkembangan riset

Pertanyaan paling dasar di bab ini, mengapa model yang mampu menghafal data latih tetap bisa generalisasi, masih menjadi bidang riset yang aktif. Kurva double descent (Belkin et al., 2019) memperlihatkan bahwa galat uji bisa turun lagi setelah model melewati titik ketika ia tepat bisa menghafal data. Fenomena yang lebih mengejutkan disebut *grokking* (Power et al., 2022). Pada himpunan data kecil yang dibangkitkan secara algoritmik, misalnya tabel penjumlahan modulo sebuah bilangan prima, jaringan mula-mula menghafal data latih dengan akurasi uji setara tebakan acak, lalu, jauh setelah titik overfitting, tiba-tiba mencapai generalisasi sempurna. Pembelajaran ternyata bisa terus berlangsung di bawah permukaan walau loss latih sudah nol. Data yang lebih sedikit membutuhkan optimasi yang lebih lama sebelum lompatan itu terjadi.

Arah lain menyangkut ketidakpastian. Model yang mengeluarkan satu angka atau satu label tidak memberi tahu seberapa jauh angka itu bisa dipercaya. *Conformal prediction* (Angelopoulos and Bates, 2021) menawarkan jaminan yang tidak bergantung pada model maupun distribusi data. Resepnya sederhana. Sisihkan himpunan kalibrasi berisi n contoh, hitung skor ketidaksesuaian untuk setiap contoh, misalnya galat mutlak $|y_i - \hat{y}_i|$ untuk regresi, lalu ambil kuantil ke- $\lceil (n+1)(1-\alpha) \rceil / n$ dari skor-skor itu sebagai $\hat{q}$. Untuk contoh baru, interval $[\hat{y} - \hat{q}, \hat{y} + \hat{q}]$ memuat jawaban yang benar dengan peluang paling sedikit $1 - \alpha$. Dengan seribu contoh kalibrasi dan $\alpha = 0,1$, $\hat{q}$ adalah skor terbesar ke-901. Jaminannya hanya membutuhkan satu asumsi: contoh kalibrasi dan contoh baru bisa ditukar urutannya tanpa mengubah distribusi bersamanya. Karena itu, conformal prediction bisa dibungkuskan pada model apa pun yang sudah dilatih, dari jaringan saraf di citra medis sampai model pengganti di simulasi.

CATATAN SUMBER. Bab ini merangkai materi dari Machine Learning: Basic Techniques, Supervised dan Unsupervised Techniques, Statistical Learning Theory, serta bagian pembuka Deep Learning: Geometric Techniques. Bacaan pendamping yang baik adalah Bishop (2006), Hastie et al. (2009), dan Bach (2024).

2 Memilih Model dengan Statistik dan Bayes

Pada musim dingin 2025/26 saya mengikuti kuliah Machine Learning: Basic Techniques dari Sepp Hochreiter, kuliah yang namanya “dasar” tetapi dalam perjalanan saya datang setelah sebagian besar kuliah lanjut. Naskahnya dibuka dengan satu klaim yang terus diulang sampai halaman terakhir: inti machine learning adalah *model selection*. Memilih arsitektur, memilih fungsi loss, memilih regularisasi, bahkan memilih cara meringkas data sebelum memodelkannya, semuanya adalah memilih model, dan setiap pilihan itu menyimpan asumsi tentang dunia. Bab ini mengikuti alur naskah itu. Ia melengkapi Bab 1 dengan sisi statistik yang lebih klasik, yang sering terlewat ketika orang langsung melompat ke jaringan saraf.

2.1 Deduktif dan induktif

Naskah kuliah membedakan dua cara membangun model. Model deduktif diturunkan dari prinsip pertama: hukum Newton, persamaan Navier–Stokes, aturan yang ditulis ahli. Model induktif dibangun dari contoh: kita memilih sebuah keluarga model, lalu data menentukan anggota keluarga mana yang dipakai. Machine learning adalah pembangunan model induktif. Bab-bab tentang simulasi di bagian akhir buku ini pada dasarnya adalah cerita tentang mempertemukan kedua cara itu.

Keluarga model sendiri bisa parametrik atau nonparametrik. Model parametrik punya jumlah parameter tetap, berapa pun datanya, seperti regresi linear atau jaringan saraf berukuran tertentu. Model nonparametrik tumbuh bersama data, seperti k -nearest neighbour yang menyimpan semua contoh atau penaksir kerapatan kernel di bawah. Model parametrik kuat ketika asumsinya benar dan berbahaya ketika salah. Model nonparametrik lebih lentur, dengan harga butuh data lebih banyak.

2.2 Meringkas data sebelum memodelkan

Naskah kuliah menghabiskan satu bab penuh untuk hal yang tampak sepele: meringkas dan menggambar data. Contohnya data bunga iris milik Anderson dan data microarray dari beberapa jaringan tubuh. Alasannya sederhana. Ringkasan yang salah memberi gambaran yang salah, dan model yang dibangun di atas gambaran yang salah tidak akan menjadi benar karena arsitekturnya canggih.

2.2.1 Pusat dan sebaran

Rata-rata, median, dan modus sama-sama mengukur pusat, tetapi bereaksi berbeda terhadap pencilan. Lima gaji bulanan, dalam juta rupiah, 3, 4, 4, 5, dan 100, punya rata-rata 23,2 dan median 4. Satu angka ekstrem menarik rata-rata jauh dari tempat sebagian besar data berada, sedangkan median nyaris tidak terganggu. Median disebut

robust: setengah data harus dirusak sebelum median bisa digeser sembarang jauh, sedangkan untuk rata-rata, satu titik saja sudah cukup.

Untuk sebaran, varians sampel bisa dihitung dengan pembagi N atau $N - 1$. Pembagi N memberi penaksir yang bias. Alasannya, simpangan dihitung terhadap rata-rata sampel, bukan rata-rata sejati, dan rata-rata sampel selalu terletak tepat di tengah data itu sendiri, sehingga jumlah kuadrat simpangannya selalu sedikit lebih kecil dari seharusnya. Menghitung nilai harapannya memberi $\mathbb{E}\left[\frac{1}{N} \sum (x_i - \bar{x})^2\right] = \frac{N-1}{N} \sigma^2$, dan membagi dengan $N - 1$ menghapus bias itu. Tetapi naskah kuliah menunjukkan sesuatu yang lebih halus. Untuk data Gauss, penaksir yang bias justru punya galat kuadrat rata-rata yang lebih kecil, karena variansnya sendiri lebih kecil, dan pengurangan varians itu lebih besar dari tambahan bias kuadratnya. Tak bias tidak sama dengan terbaik. Inilah tawar-menawar bias dan varians dari Bab 1, dalam bentuk yang paling kecil.

Pilihan antara rata-rata dan median juga bergantung pada distribusinya. Kalau data berdistribusi Gauss, varians rata-rata sampel adalah σ^2/N , sedangkan varians median sampel kira-kira $\frac{\pi}{2} \sigma^2/N$, sekitar 57 persen lebih besar. Untuk data Gauss, rata-rata adalah penaksir pusat yang lebih baik. Kalau data berdistribusi Laplace, yang ekornya lebih tebal, keadaannya terbalik: varians median sampel hanya $\sigma^2/(2N)$, separuh varians rata-rata. Tidak ada penaksir yang terbaik untuk semua keadaan. Yang ada hanya penaksir yang cocok untuk asumsi tertentu.

2.2.2 Menggambar data

Boxplot meringkas satu variabel dengan lima angka. Kotaknya membentang dari kuartil pertama ke kuartil ketiga, garis di dalamnya adalah median, dan kumis memanjang sampai titik data terjauh yang masih berada dalam 1,5 kali jarak antarkuartil dari kotak. Titik di luar itu digambar sendiri sebagai calon pencilan. *Boxplot* per kelas pada data iris langsung memperlihatkan bahwa panjang mahkota bunga memisahkan spesies jauh lebih baik dari panjang kelopaknya, sebelum satu model pun dilatih.

Histogram bergantung pada lebar dan letak kotak-kotaknya, sehingga bentuknya bisa berubah hanya karena kotaknya digeser. Penaksir kepadatan kernel menghaluskannya dengan meletakkan sebuah kernel, biasanya Gauss, di setiap titik data dan menjumlahkannya,

$$\hat{p}(x) = \frac{1}{Nh} \sum_{i=1}^N K\left(\frac{x - x_i}{h}\right). \quad (2.1)$$

Lebar pita h memainkan peran yang sudah kita kenal. Pita yang terlalu sempit membuat setiap titik data menjadi puncak sendiri, *overfitting*. Pita yang terlalu lebar meratakan dua puncak menjadi satu, *underfitting*. *Violin plot* menggabungkan *boxplot* dengan kepadatan kernel yang dicerminkan, sehingga bentuk distribusi dan ringkasan lima angkanya terlihat sekaligus.

2.2.3 Korelasi dan bahaya angka tunggal

Koefisien korelasi Pearson mengukur seberapa linear hubungan dua variabel. Untuk menguji apakah korelasi sampel r dari N pasangan berbeda nyata dari nol, dengan

asumsi kedua variabel berdistribusi normal, dipakai statistik

$$t = r \sqrt{\frac{N-2}{1-r^2}}, \quad (2.2)$$

yang mengikuti distribusi Student- t dengan $N-2$ derajat kebebasan. Naskah kuliah memakai kuartet Anscombe (Anscombe, 1973), empat himpunan data berisi sebelas titik yang punya rata-rata, varians, korelasi, dan garis regresi yang hampir sama persis. Untuk himpunan pertama, $r \approx 0,816$ dan $t = 0,816\sqrt{9/0,334} \approx 4,24$, dengan nilai p satu sisi sekitar 0,001. Tetapi ketika digambar, keempatnya sama sekali berbeda: satu awan titik yang wajar, satu kurva lengkung yang sempurna, satu garis lurus dengan satu pencilan, dan satu kolom titik dengan satu titik jauh yang sendirian menciptakan seluruh korelasinya. Angka ringkasan yang sama bisa berasal dari dunia yang jauh berbeda. Gambarlah data sebelum mempercayai angkanya.

2.3 Loss sebagai asumsi noise

Di Bab 1 kita menurunkan galat kuadrat dari asumsi noise Gauss. Naskah kuliah melanjutkannya ke noise lain. Kalau noise berdistribusi Laplace, $p(\epsilon) \propto e^{-\beta|\epsilon|}$, log likelihood negatifnya sebanding dengan $\sum |y_i - \hat{y}_i|$, galat mutlak. Secara umum, noise dengan kerapatan $\propto e^{-\beta|\epsilon|^r}$ menghasilkan galat Minkowski $\sum |y_i - \hat{y}_i|^r$. Pilihan r adalah pilihan seberapa tebal ekor noise yang kita percayai.

Hubungan antara loss dan penaksir pusat bisa diturunkan dalam beberapa baris. Cari satu konstanta c yang mewakili data $y_1, \dots, y_N$. Untuk galat kuadrat, turunan $\sum (y_i - c)^2$ terhadap c adalah $-2 \sum (y_i - c)$, yang nol tepat ketika c adalah rata-rata. Untuk galat mutlak, turunan $\sum |y_i - c|$ adalah banyaknya titik di bawah c dikurangi banyaknya titik di atas c , karena setiap titik menyumbang $+1$ atau -1 . Turunan itu nol ketika titik di bawah dan di atas sama banyak, yaitu ketika c adalah median. Galat kuadrat menghasilkan rata-rata, galat mutlak menghasilkan median, dan kita sudah melihat bahwa median tahan terhadap pencilan. Jadi memilih loss L1 untuk regresi berarti memilih model yang tidak mudah ditarik oleh beberapa pengukuran yang rusak. Untuk data gaji di atas, model L2 akan meramal 23,2, model L1 meramal 4.

2.4 Batas bawah galat: klasifikasi Bayes

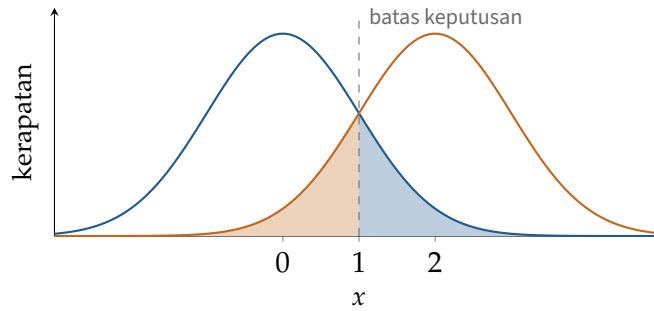
Seberapa baik sebuah pengklasifikasi bisa bekerja? Kalau distribusi data diketahui, jawabannya bisa dihitung. Pengklasifikasi yang memilih kelas dengan peluang posterior tertinggi, $\arg \max_y p(y | x)$, mencapai risiko serendah mungkin, dan risiko itu disebut galat Bayes. Tidak ada model, berapa pun besarnya, yang bisa lebih baik dari ini, karena di daerah tempat kedua kelas tumpang tindih, informasi untuk membedakannya memang tidak ada.

Naskah kuliah menghitungnya untuk dua kelas yang masing-masing berdistribusi Gauss. Log rasio kedua posterior adalah

$$\ln \frac{p(y = 1 | x)}{p(y = -1 | x)} = -\frac{1}{2}(x - \mu_1)^\top \Sigma_1^{-1}(x - \mu_1) + \frac{1}{2}(x - \mu_{-1})^\top \Sigma_{-1}^{-1}(x - \mu_{-1}) + \text{konstanta}. \quad (2.3)$$

Kalau kedua kovarians sama, suku kuadrat $x^T \Sigma^{-1} x$ dari kedua kelas saling menghapus, dan yang tersisa linear dalam x , dengan arah $\Sigma^{-1}(\mu_1 - \mu_{-1})$, arah yang sama dengan analisis diskriminan Fisher di Bab 6. Batas keputusannya adalah bidang datar. Kalau kovariansnya berbeda, suku kuadrat tidak terhapus, dan batasnya menjadi permukaan kuadrat, misalnya hiperbola atau elips, seperti yang digambarkan naskah kuliah. Model yang dipilih untuk batas keputusan, linear atau kuadrat, adalah asumsi tentang kovarians kelas.

Dalam satu dimensi, galat Bayes bisa dihitung dengan tangan (Gambar 2.1). Dua kelas dengan peluang sama, $\mathcal{N}(0, 1)$ dan $\mathcal{N}(2, 1)$, punya batas keputusan di tengah, $x = 1$. Kesalahan terjadi ketika contoh dari kelas pertama jatuh di atas 1 atau contoh dari kelas kedua jatuh di bawah 1. Masing-masing berpeluang $\Phi(-1) \approx 0,159$, dengan Φ fungsi distribusi kumulatif normal baku, sehingga galat Bayes sekitar 16 persen. Kalau jarak kedua rata-rata dua kali lipat, galatnya turun menjadi $\Phi(-2) \approx 2,3$ persen. Pengklasifikasi yang dilaporkan mencapai akurasi 95 persen pada data dengan galat Bayes 16 persen pasti sedang mengukur sesuatu yang salah, misalnya data uji yang bocor.



GAMBAR 2.1. Galat Bayes untuk dua kelas Gauss dengan rata-rata 0 dan 2. Daerah yang diarsir adalah contoh yang pasti salah diklasifikasikan oleh pengklasifikasi terbaik sekalipun.

2.5 Memilih model dengan cara Bayes

Bab terakhir naskah kuliah membawa pemilihan model ke kerangka Bayes sepenuhnya. Parameter w diberi prior, data memberi likelihood, dan teorema Bayes memberi posterior,

$$p(w | \mathcal{D}) = \frac{p(\mathcal{D} | w) p(w)}{p(\mathcal{D})}, \quad p(\mathcal{D}) = \int p(\mathcal{D} | w) p(w) dw. \quad (2.4)$$

Penyebutnya, peluang data setelah semua parameter yang mungkin dirata-ratakan, disebut *evidence*. Ia tidak berperan dalam mencari w , tetapi menjadi tokoh utama ketika yang dipilih adalah modelnya sendiri.

2.5.1 MAP dan error bar

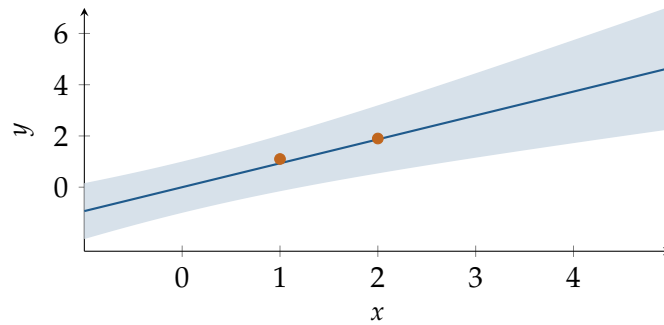
Memaksimalkan posterior memberi penaksir MAP, yang untuk noise Gauss dan prior Gauss tidak lain adalah galat kuadrat ditambah weight decay (Bab 1). Tetapi posterior

memberi lebih dari satu titik. Ia juga memberi ketidakpastian, dan ketidakpastian itu bisa diteruskan ke prediksi. Kalau keluaran model dilinearkan di sekitar w_{map} dan posterior didekati dengan Gauss yang kovariansnya kebalikan Hessian $\mathbf{H}$, naskah kuliah menurunkan varians prediksi

$$\sigma^2(x) = \frac{1}{\beta} + \mathbf{g}^\top \mathbf{H}^{-1} \mathbf{g}, \quad (2.5)$$

dengan $1/\beta$ varians noise dan $\mathbf{g}$ gradien keluaran terhadap parameter di x . Suku pertama adalah noise yang tidak bisa dihindari. Suku kedua adalah ketidakpastian tentang parameter, dan ia membesar di tempat model peka terhadap parameter yang kurang ditentukan data.

Contoh dengan satu parameter membuatnya jelas. Model $y = wx$ dengan prior $w \sim \mathcal{N}(0, 1/\alpha)$, $\alpha = 1$, noise dengan $\beta = 4$ (simpangan baku 0,5), dan dua titik data di $x = 1$ dan $x = 2$. Posterior w adalah Gauss dengan presisi $\alpha + \beta \sum x_i^2 = 1 + 4 \cdot 5 = 21$, sehingga variansnya $1/21 \approx 0,048$. Di sini $\mathbf{g} = x$, sehingga varians prediksi $\sigma^2(x) = 0,25 + x^2/21$. Di $x = 1$, simpangan baku prediksi $\sqrt{0,298} \approx 0,55$, hampir hanya noise. Di $x = 3$, di luar data, nilainya $\sqrt{0,679} \approx 0,82$. Pita ketidakpastian melebar ketika model diminta berbicara jauh dari tempat ia pernah melihat data (Gambar 2.2), sifat yang persis kita inginkan dari model mana pun yang dipakai untuk mengambil keputusan.



GAMBAR 2.2. Prediksi model $y = wx$ dengan pita dua simpangan baku dari kerangka Bayes. Pita tersempit di sekitar data dan melebar ke arah yang jauh dari data, karena ketidakpastian parameter dikalikan x .

2.5.2 Evidence dan pisau Occam

Hyperparameter seperti α dan β juga bisa dipilih dengan Bayes: pilih nilai yang memaksimalkan evidence $p(\mathcal{D} \mid \alpha, \beta)$. Kerangka ini, yang dikembangkan MacKay (1992) dan dibahas rinci di naskah kuliah, disebut *evidence framework*. Untuk model linear dengan noise dan prior Gauss, log evidence punya bentuk tertutup yang berisi galat latih, suku regularisasi, dan $-\frac{1}{2} \ln |\mathbf{H}|$, sehingga tidak perlu data validasi terpisah.

Evidence juga membandingkan model yang berbeda, dan cara kerjanya mengandung pisau Occam yang otomatis. Kalau posterior memuncak tajam, evidence kira-kira sama dengan likelihood terbaik dikali rasio lebar posterior terhadap lebar prior:

$$p(\mathcal{D} \mid \mathcal{M}) \approx p(\mathcal{D} \mid w_{\text{map}}, \mathcal{M}) \cdot \frac{\Delta w_{\text{posterior}}}{\Delta w_{\text{prior}}}. \quad (2.6)$$

Faktor kedua, faktor Occam, selalu lebih kecil dari satu, dan semakin kecil kalau model punya banyak parameter yang harus disempitkan dari prior yang lebar. Misalkan satu parameter punya prior selebar 10 dan posterior selebar 0,1. Faktor Occam-nya 0,01, setara dengan menghukum log evidence sebesar 4,6. Model yang lebih rumit harus memperbaiki log likelihood terbaiknya lebih dari 4,6 untuk setiap parameter seperti itu sebelum ia dipilih. Model yang terlalu sederhana kalah karena likelihood-nya buruk, model yang terlalu rumit kalah karena faktor Occam-nya, dan evidence memilih di tengah-tengah tanpa pernah menyentuh data uji.

Kalau posterior tidak berbentuk Gauss, integralnya didekati dengan sampling, misalnya dengan MCMC dari Bab 14. Naskah kuliah juga mencatat satu jebakan untuk jaringan saraf: posteriornya punya banyak puncak yang setara, karena menukar dua neuron tersembunyi beserta bobotnya tidak mengubah fungsi jaringan, sehingga perkiraan evidence perlu dikoreksi dengan faktor jumlah permutasi.

2.6 Di luar kelas

Gagasan-gagasan klasik di bab ini muncul kembali di banyak tempat dalam buku. Loss yang tahan pencilan, seperti galat mutlak, berguna setiap kali sebagian pengukuran bisa rusak, dari sensor yang dikenakan sampai citra medis. Ringkasan robust seperti median dan normalisasi kuantil dipakai pada data microarray (Bab 22). Error bar dari posterior adalah dasar Gaussian process (Bab 15), dan pita yang melebar jauh dari data adalah sifat yang kita harapkan dari model pengganti simulasi, yang justru paling berbahaya ketika diam-diam berekstrapolasi. Dan kuartet Anscombe adalah pengingat yang pantas ditempel di dinding setiap laboratorium: gambarlah datanya.

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Machine Learning: Basic Techniques (pilihan model, ringkasan dan visualisasi data, teori peluang, risiko minimum untuk klasifikasi Gauss, loss dan asumsi noise, teknik Bayes untuk prior, error bar, evidence, dan perbandingan model, serta lampiran tentang rata-rata dan median distribusi simetris). Bacaan pendamping: Bishop (2006) dan MacKay (1992).

3 Teori Belajar Statistik

Kuliah Statistical Learning Theory and Applications dari Werner Zellinger saya ikuti pada musim panas 2025. Naskahnya pendek, bukunya tebal: rujukan utamanya adalah *Learning Theory from First Principles* (Bach, 2024). Pertanyaan yang diajukan di halaman pertama naskah sederhana sekali. Kalau data tak terhingga, model apa yang terbaik, dan berapa galat terbaiknya? Kapan sebuah algoritme belajar dijamin semakin baik dengan bertambahnya data? Dan apakah belajar mungkin tanpa asumsi apa pun tentang data? Bab ini mengikuti jawaban-jawabannya. Ia lebih formal dari Bab 1, tetapi setiap definisi punya alasan yang konkret.

3.1 Merumuskan belajar

Data adalah n pasangan (x_i, y_i) yang ditarik independen dari distribusi P yang tidak diketahui. Algoritme belajar adalah fungsi yang memetakan himpunan data ke sebuah model f . Loss $\ell(z, y)$ mengukur kerugian memakai prediksi z ketika jawabannya y . Risiko model adalah loss harapan atas distribusi data, $R(f) = \mathbb{E}_{(x,y) \sim P}[\ell(f(x), y)]$, dan risiko empiris adalah rata-ratanya di data, $\hat{R}(f) = \frac{1}{n} \sum_i \ell(f(x_i), y_i)$.

Model terbaik yang mungkin bisa ditulis tanpa melihat data sama sekali. Karena $R(f) = \mathbb{E}_x[\mathbb{E}[\ell(f(x), y) \mid x]]$, risiko diminimalkan dengan memilih, untuk setiap x secara terpisah, prediksi z yang meminimalkan loss harapan bersyarat. Fungsi yang melakukannya disebut *risk minimizer* f^* , dan risikonya, R^* , adalah risiko minimal. Untuk regresi dengan loss kuadrat, $\mathbb{E}[(z - y)^2 \mid x]$ adalah parabola dalam z yang minimum di rata-rata bersyarat, sehingga $f^*(x) = \mathbb{E}[y \mid x]$, sama dengan yang kita turunkan untuk satu konstanta di Bab 2. Untuk klasifikasi biner dengan loss 0–1, prediksi $z = 1$ salah dengan peluang $P(y = 0 \mid x)$ dan prediksi $z = 0$ salah dengan peluang $P(y = 1 \mid x)$, sehingga yang terbaik adalah memilih kelas dengan peluang bersyarat lebih besar, dan

$$R^* = \mathbb{E}_x \left[\min \{P(y = 1 \mid x), 1 - P(y = 1 \mid x)\} \right]. \quad (3.1)$$

Itulah galat Bayes dari Bab 2, dalam bahasa yang lebih umum. Kalau P diketahui, belajar menjadi trivial. Seluruh kesulitan berasal dari kenyataan bahwa P hanya diketahui lewat sampel.

3.2 Dua sumber galat

Algoritme memilih model dari sebuah kelas $\mathcal{F}$, misalnya semua fungsi linear atau semua jaringan berukuran tertentu. Galat berlebih model yang dipilih, $\hat{f}$, terhadap

yang terbaik yang mungkin bisa dipecah menjadi dua:

$$R(\hat{f}) - R^* = \underbrace{\left(R(\hat{f}) - \inf_{f \in \mathcal{F}} R(f) \right)}_{\text{galat estimasi}} + \underbrace{\left(\inf_{f \in \mathcal{F}} R(f) - R^* \right)}_{\text{galat aproksimasi}}. \quad (3.2)$$

Galat aproksimasi adalah harga memilih kelas yang terlalu sempit: kalau fungsi sebenarnya melengkung dan kita hanya mengizinkan garis lurus, tidak ada jumlah data yang bisa menolong. Galat ini tidak bergantung pada data. Galat estimasi adalah harga memilih dari data yang terbatas: semakin kaya kelasnya, semakin mudah kita memilih anggota yang kebetulan cocok dengan noise di data latih. Galat ini mengecil dengan bertambahnya data dan membesar dengan bertambahnya kapasitas kelas. Kelas yang lebih kaya menurunkan yang pertama dan menaikkan yang kedua, bentuk umum dari tawar-menawar bias dan varians di Bab 1. Kapasitas dikendalikan dengan mengurangi parameter atau dengan menambahkan penalti norma, seperti weight decay.

3.3 Kualitas sebuah algoritme

Kuliah membedakan beberapa cara menilai algoritme. Algoritme *konsisten* kalau risiko harapan model yang dihasilkannya mendekati R^* ketika n menuju tak hingga. Versi PAC, *probably approximately correct*, meminta lebih: dengan peluang paling sedikit $1 - \delta_n$, galat berlebihnya tidak lebih dari ϵ_n , dengan kedua batas mengecil ke nol. Algoritme *universally consistent* konsisten untuk setiap distribusi dalam sebuah kelas yang besar. Risiko *minimax*,

$$\inf_{\mathcal{A}} \sup_{P \in \mathcal{P}} \mathbb{E}_{\text{data} \sim P^n} [R(\mathcal{A}(\text{data}))] - R^*, \quad (3.3)$$

menilai algoritme terbaik pada distribusi terburuknya. Salah satu tujuan utama teori belajar adalah algoritme yang mencapai laju penurunan risiko minimax yang optimal.

3.4 Tidak ada makan siang gratis

Teorema *no free lunch* dalam kuliah menjawab pertanyaan ketiga di awal bab. Untuk klasifikasi biner dengan loss 0–1, kalau kelas distribusi yang diizinkan adalah *semua* distribusi pada ruang input yang tak terhingga, maka untuk setiap algoritme dan setiap ukuran data, ada distribusi yang membuat galat berlebih harapannya paling sedikit $\frac{1}{2}$. Gagasan pembuktiannya bisa diceritakan tanpa rumus. Pilih distribusi yang tersebar di banyak sekali titik, sehingga n contoh hampir pasti tidak pernah menyentuh titik-titik uji. Lalu beri label pada titik-titik yang tidak terlihat itu secara acak. Apa pun yang dilakukan algoritme di titik yang belum pernah ia lihat, rata-ratanya ia benar separuh kali, padahal label itu sebenarnya tertentu, sehingga $R^* = 0$.

Pesannya bukan bahwa belajar mustahil, melainkan bahwa belajar *tanpa asumsi* mustahil. Asumsi tentang distribusi, misalnya bahwa fungsi targetnya mulus, bahwa ia hanya bergantung pada sedikit arah, atau bahwa ia hampir linear, adalah apa yang membuat generalisasi mungkin. Inductive bias di Bab 0 adalah nama lain dari asumsi itu.

Kuliah merangkum “perlombaan ilmiah untuk adaptivitas” dengan beberapa laju konvergensi. Kalau fungsi target hanya diketahui Lipschitz di $\mathbb{R}^d$, galat turun seperti $n^{-1/d}$. Untuk $d = 10$, mengurangi galat setengahnya menuntut 2^{10} , sekitar seribu kali lebih banyak data. Inilah kutukan dimensi dalam bentuk yang paling telanjang. Kalau target punya m turunan yang terbatas, laju membaik menjadi $n^{-m/d}$, dan jaringan saraf bisa mencapai laju $n^{-m/k}$ kalau target sebenarnya hanya bergantung pada proyeksi berdimensi k dari input, sesuatu yang menurut naskah kuliah tidak bisa dicapai metode kernel. Algoritme yang baik adalah algoritme yang menyesuaikan diri dengan struktur yang kebetulan ada, tanpa harus diberi tahu sebelumnya.

3.5 Least squares di bawah kaca pembesar

Kuliah memakai least squares sebagai laboratorium, karena semua efek penting, tawar-menawar bias dan varians, kutukan dimensi, dan efek regularisasi, bisa dihitung secara eksak di sana. Modelnya linear dalam parameter, $f_{\theta}(x) = \theta^{\top} \phi(x)$, dengan peta fitur ϕ yang boleh nonlinear dalam x . Estimator OLS $\hat{\theta} = (\Phi^{\top} \Phi)^{-1} \Phi^{\top} y$ diturunkan di Bab 1, dan prediksinya adalah proyeksi ortogonal y ke ruang kolom matriks fitur Φ .

Analisisnya paling bersih dalam *fixed design*: input dianggap tetap, misalnya titik-titik grid, dan hanya output yang acak, $y_i = \theta^{*\top} \phi(x_i) + \epsilon_i$ dengan noise independen berrata-rata nol dan varians σ^2 . Risiko diukur pada input yang sama. Dengan $\hat{\Sigma} = \frac{1}{n} \Phi^{\top} \Phi$, galat berlebih sebuah parameter adalah $(\theta - \theta^*)^{\top} \hat{\Sigma} (\theta - \theta^*)$. OLS tak bias, $\mathbb{E}[\hat{\theta}] = \theta^*$, dan kovariansnya $\frac{\sigma^2}{n} \hat{\Sigma}^{-1}$. Galat berlebih harapannya adalah jejak kovarians itu ditimbang dengan $\hat{\Sigma}$:

$$\mathbb{E}[R(\hat{\theta})] - R^* = \text{tr}\left(\hat{\Sigma} \cdot \frac{\sigma^2}{n} \hat{\Sigma}^{-1}\right) = \frac{\sigma^2 d}{n}. \quad (3.4)$$

Rumus sederhana ini memuat banyak pelajaran. Galat turun seperti $1/n$, laju terbaik yang mungkin. Tetapi ia sebanding dengan jumlah parameter d . Dengan $d = 50$, $n = 1000$, dan $\sigma^2 = 1$, galat berlebihnya 0,05. Dengan $d = 500$, sepuluh kali lebih besar. Kalau d mendekati n , model bisa menghafal data latih, dan untuk $d = n$ dengan matriks yang bisa dibalik, prediksinya tepat sama dengan data, noise sekalipun.

3.6 Ridge dan derajat kebebasan

Ridge regression menambahkan $\lambda \|\theta\|^2$, dengan solusi $\hat{\theta}_{\lambda} = \frac{1}{n} (\hat{\Sigma} + \lambda I)^{-1} \Phi^{\top} y$. Naskah kuliah memberi galat berlebih harapannya secara eksak sebagai jumlah dua suku:

$$\mathbb{E}[R(\hat{\theta}_{\lambda})] - R^* = \lambda^2 \theta^{*\top} (\hat{\Sigma} + \lambda I)^{-2} \hat{\Sigma} \theta^* + \frac{\sigma^2}{n} \text{tr}(\hat{\Sigma}^2 (\hat{\Sigma} + \lambda I)^{-2}). \quad (3.5)$$

Suku pertama adalah bias kuadrat: naik dengan λ , karena parameter ditarik semakin kuat ke nol, dan tidak bergantung pada n . Suku kedua adalah varians: turun dengan λ , mendekati $\sigma^2 d/n$ ketika λ menuju nol, dan menghilang ketika λ besar. Jejak di suku kedua, $\text{df}(\lambda) = \sum_i \lambda_i^2 / (\lambda_i + \lambda)^2$ dengan λ_i nilai eigen $\hat{\Sigma}$, disebut *derajat kebebasan* efektif.

Derajat kebebasan menjelaskan apa yang sebenarnya dilakukan regularisasi. Arah dengan nilai eigen jauh lebih besar dari λ dihitung hampir penuh, arah dengan nilai eigen jauh lebih kecil hampir tidak dihitung. Untuk tiga arah dengan nilai eigen 4, 1, dan 0,01,

dan $\lambda = 0,1$, sumbangannya $16/16,81 \approx 0,95$, $1/1,21 \approx 0,83$, dan $0,0001/0,0121 \approx 0,01$, sehingga $df \approx 1,79$. Model punya tiga parameter, tetapi variansnya seperti model dengan kurang dari dua parameter, karena arah yang hampir tidak didukung data praktis dimatikan.

Dengan pilihan λ yang tepat, naskah kuliah menunjukkan bahwa galat berlebih ridge bisa dibatasi oleh sebuah suku yang turun seperti $1/\sqrt{n}$ dan sama sekali tidak bergantung pada d , hanya pada norma θ^* , besar fitur, dan σ . Bandingkan dengan OLS: OLS turun lebih cepat, seperti $1/n$, tetapi dengan konstanta yang tumbuh dengan d . Ridge turun lebih lambat dengan konstanta yang tidak bergantung pada d . Untuk model dengan banyak parameter dibanding data, keadaan normal di deep learning, yang kedua jauh lebih menarik. Dalam praktik, λ tidak dipilih dengan rumus karena σ dan θ^* tidak diketahui, melainkan dengan validasi silang, yang menurut kuliah menikmati laju optimal yang sama.

3.7 Kernel

Bagaimana kalau peta fitur ϕ berdimensi besar sekali, bahkan tak hingga? Teorema representer menjawab bahwa untuk minimisasi risiko empiris dengan penalti norma di ruang Hilbert, solusi optimalnya selalu bisa ditulis sebagai kombinasi fitur dari titik-titik data, $\theta = \sum_i \alpha_i \phi(x_i)$, apa pun loss-nya, asalkan penaltinya naik dengan norma. Pembuktiannya pendek. Pecah θ menjadi komponen di dalam rentang $\phi(x_i)$ dan komponen tegak lurus terhadapnya. Komponen tegak lurus tidak mengubah satu pun prediksi di data latih, karena hasil kali dalamnya dengan setiap $\phi(x_i)$ nol, tetapi menambah norma. Jadi solusi optimal tidak punya komponen tegak lurus.

Akibatnya, semua yang dibutuhkan hanya hasil kali dalam antar fitur, $k(x, x') = \phi(x)^\top \phi(x')$, yang disusun dalam matriks kernel K . Masalah di ruang berdimensi tak hingga menjadi masalah dengan n variabel. Untuk loss kuadrat, kernel ridge regression memberi $\alpha = (K + n\lambda I)^{-1}y$ dan prediksi $\hat{f}(x) = \sum_i \alpha_i k(x_i, x)$, bentuk yang sama dengan rata-rata posterior Gaussian process di Bab 15. Teorema Aronszajn menjamin bahwa setiap fungsi k yang selalu menghasilkan matriks semidefinit positif adalah hasil kali dalam di suatu ruang fitur, sehingga kita cukup merancang kernel, tanpa pernah menulis ϕ . Inilah *kernel trick*.

Contoh kecil memperlihatkan fitur tersembunyi di balik kernel. Untuk $x = (x_1, x_2)$, kernel polinomial $k(x, x') = (x^\top x')^2$ bisa diuraikan menjadi $(x_1 x'_1 + x_2 x'_2)^2 = x_1^2 x_1'^2 + 2x_1 x_2 x'_1 x'_2 + x_2^2 x_2'^2$, yaitu hasil kali dalam dari fitur $\phi(x) = (x_1^2, \sqrt{2}x_1 x_2, x_2^2)$. Menghitung kernel hanya butuh satu hasil kali dalam dua dimensi lalu dikuadratkan, sedangkan fiturnya tiga dimensi. Untuk pangkat dan dimensi yang lebih tinggi, selisihnya meledak. Kernel Gauss, $k(x, x') = \exp(-\|x - x'\|^2)$, berkorespondensi dengan ruang fitur berdimensi tak hingga, dan tetap dihitung dengan satu jarak.

3.8 Di luar kelas

Teori di bab ini terdengar jauh dari praktik, tetapi ia menjawab pertanyaan yang muncul di seluruh buku. Mengapa ukuran data penting untuk model pengganti simulasi

berdimensi tinggi? Laju $n^{-1/d}$ menjawabnya, dan juga menjelaskan mengapa struktur fisika yang mengurangi dimensi efektif begitu berharga (Bab 29 dan 30). Mengapa jaringan saraf dengan parameter jauh lebih banyak dari data tetap bisa generalisasi? Derajat kebebasan efektif dan regularisasi implisit memberi sebagian jawabannya, dan double descent di Bab 1 adalah kepingan lainnya. Dan teorema no free lunch adalah pengingat yang rendah hati: setiap keberhasilan sebuah metode adalah keberhasilan asumsinya.

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Statistical Learning Theory and Applications (pembelajaran terawasi dan risiko, minimisasi risiko empiris, konsistensi dan risiko minimax, teorema no free lunch, analisis least squares dan ridge dalam fixed design, teorema representer dan kernel). Rujukan utamanya adalah Bach (2024).

4 Optimasi: Turun Gunung dalam Kabut

Optimizer sudah saya pakai sejak tugas deep learning di semester pertama, tetapi teorinya baru dibahas secara sistematis di semester terakhir, di kuliah optimasi numerik: mengapa gradient descent bisa lambat, kapan ia pasti konvergen, dan apa yang dibeli metode Newton dengan biaya yang mahal. Bab ini menggabungkan kedua sisi itu.

Bab sebelumnya berakhir dengan perintah “minimalkan risiko empiris”. Sekarang kita membahas cara melakukannya. Hampir semua model di buku ini, dari regresi linear sampai PINN, dilatih dengan varian dari satu gagasan: ikuti arah turunan.

Bayangkan Anda berdiri di lereng gunung yang tertutup kabut tebal. Anda ingin turun ke lembah, tetapi hanya bisa melihat tanah di sekitar kaki. Strategi paling masuk akal adalah merasakan ke arah mana tanah paling curam menurun, melangkah ke sana, lalu mengulangnya. Inilah gradient descent. Persoalannya ada pada panjang langkah. Langkah yang terlalu kecil membuat Anda baru sampai besok pagi. Langkah yang terlalu besar membuat Anda melompati lembah dan mendarat di lereng seberang, bahkan lebih tinggi dari tempat semula.

4.1 Masalah dan syarat optimalitas

Kita mencari $\mathbf{x}^* = \arg \min_{\mathbf{x} \in \mathbb{R}^N} f(\mathbf{x})$ untuk fungsi f yang cukup mulus. Dua syarat klasik membantu mengenali minimum. Kalau $\mathbf{x}^*$ minimum lokal, gradiennya pasti nol, $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Sebaliknya, kalau gradiennya nol dan Hessian $\nabla^2 f(\mathbf{x}^*)$ definit positif, maka $\mathbf{x}^*$ pasti minimum lokal yang tegas. Titik dengan gradien nol disebut *stationary point*, dan ia bisa berupa minimum, maksimum, atau *saddle point*. Di dimensi tinggi, saddle point jauh lebih banyak daripada minimum, karena untuk menjadi minimum semua nilai eigen Hessian harus positif sekaligus.

Fungsi f disebut *convex* kalau ruas garis yang menghubungkan dua titik di grafiknya selalu berada di atas grafik itu, yaitu $f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$ untuk setiap $\lambda \in [0, 1]$. Untuk fungsi konveks, setiap minimum lokal adalah minimum global. Regresi linear dengan loss kuadrat, regresi logistik, dan lasso semuanya konveks. Jaringan saraf tidak. Meski begitu, teori konveks tetap berguna, karena di sekitar sebuah minimum hampir semua fungsi mulus terlihat seperti mangkuk kuadrat.

Fungsi uji klasik yang tidak konveks adalah fungsi Rosenbrock (Rosenbrock, 1960), $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$. Minimumnya di $(1, 1)$, tersembunyi di dasar lembah sempit berbentuk pisang. Menemukan lembahnya mudah. Yang sulit adalah berjalan menyusuri lembah itu sampai ke titik terendahnya.

4.2 Gradient descent

Aturan dasarnya adalah

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k), \quad (4.1)$$

dengan $\alpha_k > 0$ panjang langkah atau *learning rate*. Arah negatif gradien dipilih karena ekspansi Taylor orde satu, $f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h}$, paling cepat turun kalau $\mathbf{h}$ searah $-\nabla f(\mathbf{x})$.

Kapan langkah-langkah ini dijamin menuju minimum? Anggap gradien f bersifat Lipschitz dengan konstanta L , artinya $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$. Konstanta L mengukur seberapa cepat kemiringan bisa berubah. Dengan langkah tetap $\alpha = 1/L$, setiap iterasi menjamin

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \frac{1}{2L} \|\nabla f(\mathbf{x}_k)\|^2,$$

jadi selama gradien belum nol, nilai fungsi pasti turun.

Ketaksamaan ini, yang di naskah kuliah disebut *descent lemma*, bisa diturunkan dalam dua baris. Gradien yang Lipschitz berarti fungsi tidak bisa melengkung ke atas lebih cepat dari sebuah parabola dengan kelengkungan L , sehingga untuk setiap $\mathbf{h}$,

$$f(\mathbf{x} + \mathbf{h}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \mathbf{h} + \frac{L}{2} \|\mathbf{h}\|^2.$$

Ruas kanan adalah parabola yang selalu berada di atas fungsi. Masukkan langkah gradient descent, $\mathbf{h} = -\alpha \nabla f(\mathbf{x})$, sehingga ruas kanan menjadi $f(\mathbf{x}) - \alpha(1 - \frac{L\alpha}{2}) \|\nabla f(\mathbf{x})\|^2$. Penurunan terbesar didapat dengan memaksimalkan $\alpha(1 - L\alpha/2)$, yang terjadi pada $\alpha = 1/L$, dan nilainya $1/(2L)$. Dari sini juga terlihat batas atas langkah: kalau $\alpha \geq 2/L$, faktor $1 - L\alpha/2$ menjadi nol atau negatif, dan jaminan penurunannya hilang.

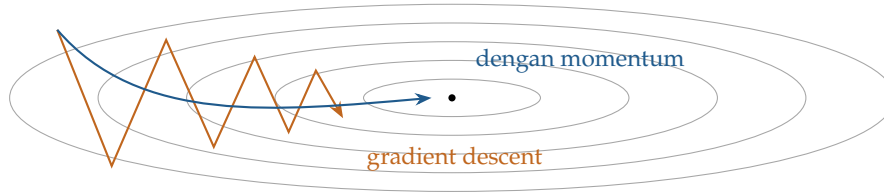
Kalau f juga *strongly convex* dengan konstanta μ , galat menyusut secara geometris dengan laju yang ditentukan oleh *condition number* $\kappa = L/\mu$.

Contoh kecil memperlihatkan arti condition number. Ambil $f(x_1, x_2) = \frac{1}{2}(x_1^2 + 100x_2^2)$. Gradiennya $(x_1, 100x_2)$, sehingga $L = 100$, $\mu = 1$, dan $\kappa = 100$. Dengan langkah tetap α , kedua koordinat berkembang terpisah: x_1 dikalikan $(1 - \alpha)$ dan x_2 dikalikan $(1 - 100\alpha)$ di setiap langkah. Supaya x_2 tidak meledak, kita butuh $|1 - 100\alpha| < 1$, yang berarti $\alpha < 0,02$. Tetapi dengan langkah sekecil itu, x_1 hanya menyusut dengan faktor sekitar 0,98. Bahkan dengan langkah terbaik, $\alpha = 2/(L + \mu)$, faktor terburuknya $(\kappa - 1)/(\kappa + 1) = 99/101$. Untuk mengecilkan galat sejuta kali, dibutuhkan kira-kira $\ln(10^{-6})/\ln(99/101) \approx 690$ langkah. Metode Newton menyelesaikan masalah kuadrat yang sama dalam satu langkah.

Itulah inti masalah optimasi dalam praktik. Gradient descent lambat bukan karena fungsinya rumit, melainkan karena skala arah-arahnya berbeda jauh. Pada lembah yang lonjong, lintasannya berzig-zag menyeberangi lembah alih-alih berjalan menyusurnya (Gambar 4.1).

4.3 Line search

Daripada menebak α sekali untuk selamanya, kita bisa mencarinya di setiap iterasi. *Line search* mencoba beberapa panjang langkah sepanjang arah turun $\mathbf{d}_k$ dan menerima yang



GAMBAR 4.1. Pada lembah lonjong, gradient descent berzig-zag. Momentum meredam gerak bolak-balik dan mempercepat gerak sepanjang lembah. Gambar skematik.

pertama dianggap cukup baik. Ukuran “cukup baik” yang paling umum adalah syarat Armijo (Armijo, 1966),

$$f(\mathbf{x}_k + \alpha \mathbf{d}_k) \leq f(\mathbf{x}_k) + c \alpha \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k, \quad 0 < c < 1,$$

yang meminta agar penurunan nyata setidaknya sebagian kecil dari penurunan yang dijanjikan oleh garis singgung. Prosedur *backtracking* memulai dari $\alpha = 1$ lalu terus mengalikannya dengan sebuah faktor $\beta \in (0, 1)$ sampai syarat itu terpenuhi.

Syarat Armijo saja bisa dipenuhi oleh langkah yang kecil sekali, karena langkah kecil hampir selalu menurunkan fungsi sedikit. Syarat Wolfe menambahkan syarat kelengkungan, $\nabla f(\mathbf{x}_k + \alpha \mathbf{d}_k)^\top \mathbf{d}_k \geq c_2 \nabla f(\mathbf{x}_k)^\top \mathbf{d}_k$ dengan $c < c_2 < 1$: di titik baru, kemiringan sepanjang arah langkah harus sudah jauh lebih landai daripada di titik awal. Langkah yang terlalu pendek, yang masih berada di bagian curam, ditolak. Kedua syarat bersama menjepit panjang langkah dari dua sisi.

Contoh dengan angka membantu. Ambil $f(x) = x^2$ di $x_0 = 1$ dengan arah $\mathbf{d} = -f'(1) = -2$ dan $c = 10^{-4}$. Langkah $\alpha = 1$ membawa ke $x = -1$, dengan $f = 1$, tidak turun sama sekali, sehingga ditolak. Dengan $\beta = \frac{1}{2}$, langkah $\alpha = \frac{1}{2}$ membawa tepat ke $x = 0$, dengan $f = 0$. Syarat Armijo meminta $f \leq 1 - 10^{-4} \cdot \frac{1}{2} \cdot 4 \approx 0,9998$, dan terpenuhi. Backtracking berhenti setelah satu kali mengecilkan langkah, dan kebetulan mendarat tepat di minimum. Di deep learning, line search jarang dipakai, karena setiap evaluasi f berarti melewati seluruh data. Gagasannya muncul kembali dalam L-BFGS.

4.4 Metode Newton dan kerabatnya

Metode Newton memanfaatkan kelengkungan. Ia meminimalkan model kuadrat lokal $f(\mathbf{x}_k) + \nabla f^\top \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \nabla^2 f \mathbf{h}$, yang menghasilkan

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\nabla^2 f(\mathbf{x}_k)]^{-1} \nabla f(\mathbf{x}_k). \quad (4.2)$$

Di dekat minimum yang Hessian-nya definit positif, Newton konvergen secara kuadratik: jumlah digit yang benar kira-kira berlipat dua di setiap iterasi. Ada tiga hal yang membatasinya. Hessian berukuran $N \times N$, sehingga untuk jaringan dengan ratusan juta parameter, menyimpannya saja sudah mustahil. Jauh dari minimum, Hessian bisa tidak definit dan langkah Newton justru menanjak. Kuliah menyebut perbaikannya *Hessian modification*, yaitu menambahkan $\lambda \mathbf{I}$ sampai matriksnya definit positif. Terakhir, Newton tertarik ke titik mana pun yang gradiennya nol, termasuk saddle point.

Metode *quasi-Newton* seperti BFGS, dan versi hemat memorinya L-BFGS, membangun aproksimasi Hessian dari selisih gradien beberapa iterasi terakhir (Nocedal and Wright, 2006). L-BFGS populer untuk masalah deterministik berukuran sedang, termasuk tahap akhir pelatihan PINN. Gagasannya bertumpu pada *secant condition*: aproksimasi Hessian yang baru, B_{k+1} , harus memetakan langkah terakhir $s_k = x_{k+1} - x_k$ ke perubahan gradien yang teramati, $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, yaitu $B_{k+1}s_k = y_k$. Dalam satu dimensi, syarat ini tidak lain adalah turunan kedua yang ditaksir dengan beda hingga dari dua gradien. BFGS memilih, di antara semua matriks yang memenuhi syarat itu, yang paling dekat dengan aproksimasi sebelumnya, dan L-BFGS hanya menyimpan sepuluh sampai dua puluh pasangan (s, y) terakhir alih-alih matriks penuh.

Kuliah deep learning juga membahas dua cara lain memanfaatkan kelengkungan. Metode *conjugate gradient* memilih arah-arah yang saling “konjugat” terhadap Hessian, sehingga langkah di satu arah tidak merusak kemajuan di arah-arah sebelumnya. Untuk fungsi kuadrat berdimensi N , ia mencapai minimum eksak dalam paling banyak N langkah, dan dalam praktik galatnya menyusut dengan faktor sekitar $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ per langkah. Metode *trust region* membalik urutan pertanyaan line search. Alih-alih memilih arah lalu panjang langkah, ia menetapkan jari-jari kepercayaan, mencari langkah terbaik untuk model kuadrat di dalam jari-jari itu, lalu memperbesar jari-jarinya kalau model ternyata meramal penurunan dengan baik dan memperkecilnya kalau tidak.

4.5 Stochastic gradient descent

Dalam machine learning, f adalah rata-rata loss atas N contoh, jadi menghitung gradien penuh berarti melewati seluruh data. *Stochastic gradient descent* (SGD) menggantinya dengan gradien dari sebuah *mini-batch* acak berukuran B :

$$w_{k+1} = w_k - \alpha_k \frac{1}{B} \sum_{n \in \mathcal{B}_k} \nabla_w L(y_n, f(x_n; w_k)). \quad (4.3)$$

Gradien mini-batch adalah estimator tak bias dari gradien penuh, tetapi berisik. Teorinya berakar pada *stochastic approximation* (Robbins and Monro, 1951): agar iterasi konvergen, panjang langkah harus mengecil dengan syarat $\sum_k \alpha_k = \infty$ dan $\sum_k \alpha_k^2 < \infty$. Syarat pertama menjamin kita masih bisa berjalan sejauh apa pun, syarat kedua menjamin noise akhirnya teredam.

Untuk mengetahui harga rata-rata beras di seluruh kota, Anda bisa mendatangi semua warung, atau bertanya ke sepuluh warung yang dipilih acak. Jawaban sepuluh warung tidak tepat, tetapi didapat jauh lebih cepat. Kalau survei kecil itu diulang terus dan tebakan disesuaikan sedikit demi sedikit, Anda sampai ke angka yang benar lebih cepat daripada menunggu satu survei besar. SGD bekerja dengan cara yang sama.

Noise SGD ternyata bukan sekadar kekurangan. Ia membuat iterasi terus bergoyang dan sulit bertahan di minimum yang sempit. Hochreiter and Schmidhuber (1997b) berargumen bahwa minimum yang datar generalisasi lebih baik, karena perubahan kecil pada bobot tidak banyak mengubah loss, sehingga bobotnya bisa dideskripsikan dengan presisi lebih rendah dan modelnya efektif lebih sederhana. Keskar et al. (2016)

mengamati bahwa batch yang besar cenderung berakhir di minimum tajam dengan generalisasi lebih buruk. Kuliah deep learning menyebut efek ini *self-regularization* oleh SGD.

4.6 Momentum dan metode adaptif

Metode *heavy ball* dari Polyak (1964) menambahkan ingatan tentang arah sebelumnya,

$$\mathbf{v}_{k+1} = \beta \mathbf{v}_k - \alpha \nabla f(\mathbf{x}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_{k+1}, \quad (4.4)$$

dengan β biasanya sekitar 0,9. Komponen gradien yang berganti tanda setiap langkah saling meniadakan di dalam $\mathbf{v}$, sedangkan komponen yang konsisten menumpuk. Gradient descent biasa mirip orang yang berhenti total di setiap langkah lalu melihat lagi ke mana arah turun. Momentum mirip bola yang menggelinding: ia membawa kecepatannya, tidak langsung berbalik karena satu gundukan kecil, dan melaju lebih cepat di lereng yang panjang. Zig-zag di Gambar 4.1 teredam dengan cara ini.

Berapa besar keuntungannya? Untuk fungsi kuadrat dengan condition number κ , *heavy ball* dengan α dan β yang dipilih optimal menyusutkan galat dengan faktor sekitar $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ per langkah, bukan $(\kappa - 1)/(\kappa + 1)$. Untuk $\kappa = 100$ dari contoh di atas, faktornya 9/11 alih-alih 99/101, dan jumlah langkah untuk mengecilkan galat sejuta kali turun dari sekitar 690 menjadi sekitar $\ln(10^{-6})/\ln(9/11) \approx 69$. Momentum mengganti κ dengan akarnya, sama seperti conjugate gradient.

Masalah condition number juga bisa diserang dengan memberi setiap parameter laju belajarnya sendiri. AdaGrad membagi langkah dengan akar jumlah kuadrat gradien masa lalu, RMSProp memakai rata-rata bergerak agar pembagi itu tidak terus membesar, dan Adam (Kingma and Ba, 2014) menggabungkan momentum dengan RMSProp:

$$\begin{aligned} \mathbf{m}_k &= \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k, & \mathbf{v}_k &= \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2, \\ \hat{\mathbf{m}}_k &= \frac{\mathbf{m}_k}{1 - \beta_1^k}, & \hat{\mathbf{v}}_k &= \frac{\mathbf{v}_k}{1 - \beta_2^k}, & \mathbf{w}_{k+1} &= \mathbf{w}_k - \alpha \frac{\hat{\mathbf{m}}_k}{\sqrt{\hat{\mathbf{v}}_k} + \epsilon}. \end{aligned} \quad (4.5)$$

Semua operasinya per elemen, dengan nilai bawaan $\beta_1 = 0,9$, $\beta_2 = 0,999$, dan $\epsilon = 10^{-8}$. Faktor $1 - \beta^k$ disebut *bias correction*, dan asalnya mudah ditelusuri. Membuka rekursi $\mathbf{m}_k$ dari $\mathbf{m}_0 = \mathbf{0}$ memberi $\mathbf{m}_k = (1 - \beta_1) \sum_{j=1}^k \beta_1^{k-j} \mathbf{g}_j$. Kalau gradien punya ekspektasi tetap $\mathbb{E}[\mathbf{g}]$, maka

$$\mathbb{E}[\mathbf{m}_k] = (1 - \beta_1)(1 + \beta_1 + \cdots + \beta_1^{k-1})\mathbb{E}[\mathbf{g}] = (1 - \beta_1^k)\mathbb{E}[\mathbf{g}],$$

karena deret geometri berhingga $\sum_{j=0}^{k-1} \beta_1^j = (1 - \beta_1^k)/(1 - \beta_1)$. Jadi $\mathbf{m}_k$ bias ke bawah dengan faktor tepat $1 - \beta_1^k$, dan membaginya dengan faktor itu menghapus bias. Pada langkah pertama dengan $\beta_1 = 0,9$, $\mathbf{m}_1 = 0,1 \mathbf{g}_1$, sepuluh kali terlalu kecil, dan koreksi mengembalikannya ke $\mathbf{g}_1$. Untuk $\beta_2 = 0,999$, faktornya pada langkah pertama 0,001, sehingga tanpa koreksi langkah awal Adam akan besar sekali dan tidak menentu. Setelah beberapa ribu langkah, kedua faktor mendekati satu dan koreksinya tidak berpengaruh lagi.

Beberapa salah paham tentang Adam cukup umum. Adam tidak selalu lebih baik dari SGD. Untuk beberapa tugas visi, SGD dengan momentum dan jadwal laju belajar yang baik masih menghasilkan generalisasi yang lebih baik. Menambahkan suku L2 ke loss lalu memakai Adam juga tidak sama dengan weight decay, karena suku itu ikut dibagi $\sqrt{v}$ sehingga efeknya tidak seragam antar parameter. AdamW memisahkan weight decay dari langkah adaptif untuk mengatasi hal ini (Loshchilov and Hutter, 2017).

Laju belajar yang tetap pun jarang optimal. Pola yang umum adalah *warm-up*, yaitu mulai kecil dan naik selama beberapa ratus langkah, lalu menurun perlahan, misalnya mengikuti kurva kosinus. Warm-up mencegah langkah awal yang terlalu agresif ketika statistik Adam belum stabil.

Untuk jaringan rekuren, kadang gradien tiba-tiba membesar ribuan kali karena perkalian berulang melewati banyak langkah waktu (Bab 9). Satu langkah dengan gradien seperti itu bisa melempar bobot jauh dari daerah yang baik. *Gradient clipping* menskalakan ulang gradien kalau normanya melebihi ambang τ : $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, \tau/\|\mathbf{g}\|)$. Arahnya dipertahankan, hanya panjangnya dibatasi. Gradien dengan norma 50 dan ambang 5 diperkecil sepuluh kali. Langkah ini murah dan menjadi bagian standar pelatihan model bahasa.

Seberapa besar mini-batch yang sebaiknya dipakai? Karena gradien mini-batch adalah rata-rata B gradien yang hampir independen, variansnya menyusut seperti $1/B$, dan simpangan bakunya seperti $1/\sqrt{B}$. Batch empat kali lebih besar hanya mengurangi noise separuhnya, padahal biayanya empat kali lipat. Aturan praktis yang sering dipakai, menaikkan laju belajar sebanding dengan ukuran batch, berasal dari pengamatan bahwa yang menentukan dinamika SGD adalah perbandingan laju belajar terhadap ukuran batch, setidaknya selama batch belum terlalu besar.

4.7 Optimasi dengan kendala

Banyak masalah punya kendala: parameter harus positif, jumlah peluang harus satu, anggaran tidak boleh terlampaui. Untuk kendala persamaan $h(\mathbf{x}) = 0$, metode pengali Lagrange menyatakan bahwa di optimum, gradien fungsi tujuan harus sejajar dengan gradien kendala, $\nabla f(\mathbf{x}^*) = \lambda \nabla h(\mathbf{x}^*)$. Alasannya geometris: kalau ∇f punya komponen yang menyinggung permukaan kendala, kita masih bisa bergeser sepanjang permukaan itu dan menurunkan f .

Contoh paling sederhana: minimalkan $x^2 + y^2$ dengan syarat $x + y = 1$, yaitu titik terdekat ke asal di sebuah garis. Syarat Lagrange memberi $(2x, 2y) = \lambda(1, 1)$, sehingga $x = y$, dan dari kendala $x = y = \frac{1}{2}$, dengan $\lambda = 1$. Pengali Lagrange punya arti: ia adalah laju perubahan nilai optimum ketika kendala dilonggarkan. Kalau syaratnya menjadi $x + y = 1 + \varepsilon$, nilai optimumnya $(1 + \varepsilon)^2/2 \approx \frac{1}{2} + \varepsilon$, naik dengan laju $\lambda = 1$. Inilah harga bayangan yang sama dengan variabel dual dalam program linear (Bab 5). Untuk kendala pertidaksamaan, syarat Karush–Kuhn–Tucker menambahkan bahwa pengali harus nonnegatif dan nol untuk kendala yang tidak aktif. Di deep learning, kendala biasanya ditangani dengan cara yang lebih kasar: suku penalti di loss, atau reparameterisasi, misalnya menulis parameter positif sebagai eksponen dari parameter

bebas.

4.8 Lanskap loss jaringan dalam

Lanskap loss jaringan dalam ternyata berbeda dari intuisi dimensi rendah. Untuk fungsi acak di dimensi tinggi, sebagian besar titik kritis dengan loss tinggi adalah saddle point, dan minimum lokal cenderung punya nilai loss yang mirip satu sama lain. Pada jaringan yang sangat lebar, ada banyak sekali solusi yang sama-sama mencapai loss latih hampir nol, dan solusi-solusi itu sering terhubung oleh lembah dengan loss rendah. Pertanyaan pentingnya pun bergeser, dari “apakah kita menemukan minimum global?” menjadi “minimum mana yang dipilih, dan apakah ia generalisasi?”.

4.9 Optimasi di luar kelas

Contoh mangkuk lonjong di atas bukan sekadar latihan. Masalah condition number yang sama muncul di banyak tempat dalam buku ini dengan nama yang berbeda. Filter adaptif LMS dalam pemrosesan sinyal adalah SGD dengan mini-batch satu sampel, dan kecepatan konvergensinya ditentukan oleh sebaran nilai eigen matriks autokorelasi input (Bab 18). Algoritme RLS, yang memperbarui invers matriks itu, adalah kerabat metode Newton. Faktorisasi matriks dalam sistem rekomendasi dilatih dengan SGD (Bab 34), dan model molekul dilatih dengan Adam seperti jaringan lain.

Di simulasi, loss sebuah PINN adalah jumlah beberapa suku: residu persamaan diferensial, syarat batas, syarat awal, dan data. Suku-suku ini punya skala dan kelengkungan yang jauh berbeda, sehingga condition number masalahnya buruk sekali. Wang et al. (2021) menunjukkan bahwa gradien suku residu bisa mendominasi suku batas, sehingga jaringan memenuhi persamaan di dalam domain tetapi melupakan syarat batasnya. Resep yang sering dipakai adalah memulai dengan Adam untuk keluar dari daerah awal yang buruk, melanjutkan dengan L-BFGS untuk turun tajam ke minimum, dan menyeimbangkan bobot suku-suku loss secara adaptif (Bab 29).

4.10 Perkembangan riset

Teori di awal bab ini meramalkan bahwa gradient descent dengan langkah η hanya stabil kalau kelengkungan terbesar L lebih kecil dari $2/\eta$. Cohen et al. (2021) mengamati sesuatu yang aneh pada pelatihan jaringan saraf dengan gradien penuh. Kelengkungan terbesar Hessian naik selama pelatihan sampai menyentuh $2/\eta$, lalu berhenti di sana dan melayang tepat di atas batas itu. Loss naik-turun dalam jangka pendek, tetapi tetap turun dalam jangka panjang. Rezim ini dinamai *edge of stability*. Jaringan seolah menyesuaikan lanskapnya sendiri dengan langkah yang dipakai, sesuatu yang tidak diramalkan analisis klasik dengan L tetap.

Optimizer baru terus bermunculan. Lion (Chen et al., 2023) ditemukan bukan oleh manusia, melainkan oleh pencarian program otomatis atas ruang algoritme optimasi. Hasilnya ternyata sederhana sekali: hanya menyimpan momentum, lalu memperbarui setiap parameter dengan tanda dari campuran momentum dan gradien,

$w \leftarrow w - \eta \text{sign}(c)$. Semua parameter bergerak dengan besar langkah yang sama, dan memorinya separuh Adam karena tidak menyimpan rata-rata kuadrat gradien. Muon mengambil sudut pandang matriks: pembaruan untuk satu matriks bobot dibuat ortogonal sebelum diterapkan, sehingga semua arah singular bergerak sama cepat. Liu et al. (2025) melaporkan bahwa dengan weight decay dan penskalaan per parameter yang tepat, Muon mencapai efisiensi komputasi sekitar dua kali AdamW dalam pelatihan model bahasa besar. Schedule-Free (Defazio et al., 2024) menghapus jadwal laju belajar sama sekali dengan teori yang menyatukan penjadwalan dan perata-rataan iterasi, dan tidak menambah hyperparameter. Ketiga contoh ini memperlihatkan bahwa pertanyaan lama di bab ini, bagaimana mengatasi condition number dan noise, masih menghasilkan jawaban baru.

CATATAN SUMBER. Teori konvergensi, line search, dan metode Newton mengikuti naskah kuliah Numerical Optimization for AI. Bagian SGD, momentum, dan optimizer adaptif mengikuti bab metode belajar di kuliah deep learning dasar. Bacaan lanjut: Nocedal and Wright (2006) untuk optimasi klasik dan Bottou et al. (2018) untuk optimasi skala besar dalam machine learning.

5 AI Klasik: Mencari, Merencanakan, Memutuskan

Kuliah pengantar AI baru saya ikuti di semester terakhir, pada musim panas 2026, setelah hampir semua kuliah deep learning selesai. Dengan urutan terbalik seperti itu, search, planning, game playing, dan representasi pengetahuan tidak tampil sebagai masa lalu yang usang, melainkan sebagai fondasi yang masih dipakai di dalam sistem modern, dari AlphaGo sampai agen berbasis model bahasa. Setahun sebelumnya, kuliah operations research memberi sisi lain dari cerita yang sama: mencari keputusan terbaik di antara pilihan yang terhitung tetapi banyak sekali.

Sebelum jaringan saraf mendominasi, AI berarti program yang bernalar secara eksplisit: mencari jalan di ruang kemungkinan, menyusun rencana langkah demi langkah, dan menarik kesimpulan dari fakta. Bab ini membahas gagasan-gagasan itu, karena tanpa mereka sulit memahami mengapa sistem AI paling kuat hari ini menggabungkan pembelajaran dengan pencarian.

5.1 Dari Turing ke Dartmouth

TURING (1950) mengganti pertanyaan “bisakah mesin berpikir?” dengan permainan tiruan: kalau seorang penanya tidak bisa membedakan mesin dari manusia lewat percakapan tertulis, apa alasan kita menolak menyebut mesin itu cerdas? Istilah *artificial intelligence* sendiri lahir dari proposal lokakarya musim panas di Dartmouth pada 1956. Sejak itu, sejarah AI bergerak dalam gelombang optimisme dan kekecewaan. Sistem pakar pada 1980-an menyimpan pengetahuan manusia dalam ribuan aturan jika-maka, lalu tersandung pada sulitnya menulis semua aturan itu dengan tangan. Gelombang statistik dan kemudian deep learning menggeser fokus dari aturan yang ditulis ke pola yang dipelajari (Russell and Norvig, 2021).

Kuliah juga menyinggung perdebatan filosofisnya. Searle (1980) mengajukan eksperimen pikiran ruang Cina. Seseorang yang tidak mengerti bahasa Cina duduk di dalam ruangan dan menjawab pesan dalam bahasa Cina dengan mengikuti buku aturan yang lengkap. Dari luar, jawabannya sempurna. Dari dalam, tidak ada pemahaman sama sekali. Searle berargumen bahwa memanipulasi simbol tidak sama dengan memahami. Perdebatan antara AI lemah, yang hanya meniru perilaku cerdas, dan AI kuat, yang sungguh memahami, belum selesai sampai sekarang, dan model bahasa besar membuatnya kembali hangat.

5.2 Pencarian

Banyak masalah bisa dirumuskan sebagai pencarian: ada keadaan awal, sekumpulan aksi, fungsi transisi yang mengatakan ke mana sebuah aksi membawa kita, uji tujuan, dan biaya setiap langkah. Teka-teki geser delapan angka, rute navigasi, dan penyusunan jadwal semuanya cocok dengan bentuk ini. Solusinya adalah jalur dari keadaan awal ke keadaan tujuan.

Pencarian tanpa informasi tidak tahu apa-apa tentang di mana tujuan berada. *Breadth-first search* menjelajah lapis demi lapis dan menjamin jalur terpendek dalam jumlah langkah, tetapi memorinya tumbuh eksponensial. *Depth-first search* masuk sedalam mungkin lebih dulu, hemat memori, tetapi bisa tersesat di cabang yang tak berujung. *Uniform-cost search* selalu memperluas simpul dengan biaya kumulatif terkecil, sehingga menemukan jalur termurah. Kalau versi ini dijalankan dengan antrean prioritas pada graf dengan biaya positif, ia sama dengan algoritme Dijkstra (Dijkstra, 1959), yang juga dibahas di kuliah operations research untuk mencari rute terpendek di jaringan jalan.

Mengapa memori menjadi masalah? Kalau setiap keadaan punya b penerus dan solusi terdekat ada di kedalaman d , breadth-first search menyimpan sekitar b^d simpul. Dengan $b = 10$ dan $d = 6$, itu sejuta simpul. Dengan $d = 12$, satu triliun. Pertumbuhan eksponensial seperti ini adalah alasan mengapa pencarian buta tidak pernah cukup untuk masalah nyata, dan mengapa pengetahuan tentang arah tujuan berharga.

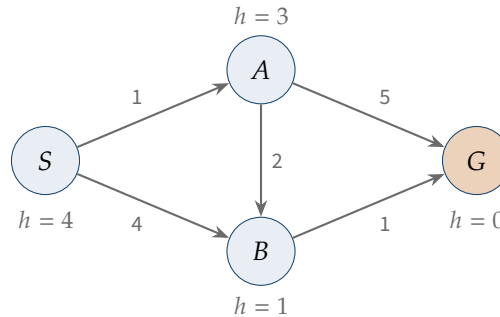
Pencarian dengan informasi memakai *heuristic* $h(n)$, perkiraan biaya dari simpul n ke tujuan. Algoritme A* (Hart et al., 1968) memperluas simpul dengan nilai

$$f(n) = g(n) + h(n) \quad (5.1)$$

terkecil, dengan $g(n)$ biaya yang sudah ditempuh. Kalau heuristik tidak pernah melebihi biaya sebenarnya, yaitu *admissible*, A* dijamin menemukan jalur optimal. Pada teka-teki delapan angka, jumlah ubin yang salah tempat adalah heuristik *admissible*, dan jumlah jarak Manhattan setiap ubin ke posisi seharusnya juga *admissible* tetapi lebih informatif, karena selalu sama atau lebih besar. Heuristik yang lebih informatif memangkas jauh lebih banyak simpul.

Contoh kecil di Gambar 5.1 memperlihatkan cara kerjanya. Dari S , dua simpul bisa dicapai: A dengan $f = 1 + 3 = 4$ dan B dengan $f = 4 + 1 = 5$. A* memperluas A lebih dulu. Dari A , simpul B kini bisa dicapai dengan biaya $g = 3$, sehingga $f = 3 + 1 = 4$, lebih baik dari jalur langsung tadi, dan G bisa dicapai dengan $g = 6$, $f = 6$. Berikutnya B diperluas, dan dari sana G dicapai dengan $g = 4$, $f = 4$. Simpul dengan f terkecil sekarang adalah G sendiri, sehingga pencarian berhenti dengan jalur $S \rightarrow A \rightarrow B \rightarrow G$ berbiaya 4. Jalur $S \rightarrow A \rightarrow G$ yang tampak lebih pendek dalam jumlah langkah tidak pernah dipilih.

Bukti optimalitasnya pendek. Misalkan biaya optimal C^* , dan A* hendak mengembalikan tujuan G_2 dengan biaya $g(G_2) > C^*$. Di jalur optimal pasti masih ada simpul n yang belum diperluas. Karena heuristiknya *admissible*, $f(n) = g(n) + h(n) \leq C^*$. Sementara itu, $f(G_2) = g(G_2) > C^*$ karena h bernilai nol di tujuan. Jadi $f(n) < f(G_2)$, dan A* pasti memperluas n sebelum G_2 . Mengulang argumen ini sepanjang jalur optimal memperlihatkan bahwa tujuan yang optimal selalu dicapai lebih dulu.



GAMBAR 5.1. Graf kecil untuk A*. Angka pada panah adalah biaya, h adalah heuristik admissible. Jalur optimal $S \rightarrow A \rightarrow B \rightarrow G$ berbiaya 4.

A* mirip pengemudi yang memilih belokan berdasarkan dua hal sekaligus: jarak yang sudah ditempuh di odometer, dan jarak garis lurus yang tersisa ke tujuan. Karena garis lurus tidak pernah lebih panjang dari jalan sebenarnya, ia tidak akan tertipu untuk mengabaikan rute yang sebenarnya baik.

Ketika jalurnya tidak penting dan yang dicari hanya keadaan terbaik, misalnya susunan jadwal yang paling sedikit bentrok, *local search* lebih cocok. Hill climbing terus bergerak ke tetangga yang lebih baik sampai tidak ada lagi, dan sering terjebak di puncak lokal. Simulated annealing (Kirkpatrick et al., 1983) kadang menerima langkah yang memburuk dengan peluang yang menurun perlahan, seperti logam yang didinginkan pelan-pelan agar atomnya menemukan susunan berenergi rendah. Tabu search (Glover, 1989) mengingat langkah-langkah terakhir dan melarang kembali ke sana, sehingga tidak berputar-putar di sekitar puncak yang sama.

Kuliah operations research membahas contoh paling terkenal untuk local search: *travelling salesman problem*, mencari rute terpendek yang mengunjungi setiap kota tepat sekali. Jumlah rute untuk n kota adalah $(n-1)!/2$. Untuk sepuluh kota, itu 181 440 rute, masih bisa dicoba semua. Untuk dua puluh kota, sekitar 6×10^{16} . Heuristik konstruksi, seperti selalu pergi ke kota terdekat atau menyisipkan kota di tempat yang paling murah, membuat rute awal yang lumayan. Langkah *2-opt* lalu memperbaikinya: ambil dua sisi rute, putuskan, dan sambung ulang dengan cara lain kalau itu memperpendek rute. Setiap rute yang saling bersilangan bisa diperbaiki dengan cara ini, karena dua sisi yang bersilangan selalu lebih panjang dari dua sisi yang tidak, menurut ketaksamaan segitiga. Masalah pengiriman barang dengan banyak kendaraan berkapasitas terbatas, yang dibahas di kuliah dengan contoh perusahaan selai rumahan, dibangun di atas gagasan yang sama.

5.3 Bermain melawan lawan

Dalam permainan dua pemain, keadaan berikutnya juga ditentukan oleh lawan yang berusaha mengalahkan kita. Algoritme *minimax* mengasumsikan lawan bermain sempurna: di giliran kita pilih langkah dengan nilai maksimum, di giliran lawan anggap ia memilih nilai minimum. Contoh dari Russell and Norvig (2021) memperlihatkankannya. Misalkan kita punya tiga langkah, dan lawan menjawab setiap langkah dengan tiga

kemungkinan yang nilainya (3, 12, 8), (2, 4, 6), dan (14, 5, 2). Lawan akan memilih minimum di setiap cabang, yaitu 3, 2, dan 2, sehingga kita memilih cabang pertama dengan nilai 3.

Alpha-beta pruning (Knuth and Moore, 1975) memperoleh jawaban yang sama tanpa melihat semua daun. Setelah cabang pertama memberi nilai 3, kita tahu kita bisa mendapat setidaknya 3. Di cabang kedua, begitu kita melihat daun bernilai 2, kita tahu lawan akan memberi paling banyak 2 di cabang itu, sehingga sisa daunnya tidak perlu diperiksa. Dengan urutan langkah yang baik, alpha-beta bisa menggandakan kedalaman pencarian dengan waktu yang sama. Alasannya, dengan urutan terbaik, jumlah daun yang perlu diperiksa turun dari b^m menjadi sekitar $b^{m/2}$ untuk pohon berkedalaman m , dan $b^{m/2}$ dengan waktu yang sama memungkinkan kedalaman m dua kali lipat.

Untuk permainan dengan cabang banyak seperti Go, minimax tidak mungkin dijalankan sampai akhir permainan. *Monte Carlo tree search* membangun pohon pencarian secara bertahap dengan memainkan banyak permainan acak sampai selesai, lalu memusatkan pencarian pada langkah yang menjanjikan. Aturan UCT (Kocsis and Szepesvári, 2006) menyeimbangkan eksploitasi langkah yang sejauh ini terbaik dengan eksplorasi langkah yang belum banyak dicoba, mengikuti teori bandit. Di setiap simpul, langkah yang dipilih adalah yang memaksimalkan

$$\bar{X}_j + c \sqrt{\frac{\ln N}{n_j}}, \quad (5.2)$$

dengan $\bar{X}_j$ rata-rata hasil permainan setelah langkah j , n_j berapa kali langkah itu dicoba, dan N total kunjungan simpul. Suku kedua adalah bonus keingintahuan. Langkah yang jarang dicoba mendapat bonus besar, dan bonus itu mengecil seiring langkah itu dicoba lagi. Misalkan setelah seratus simulasi, langkah pertama dicoba 80 kali dengan tingkat menang 0,55 dan langkah kedua 20 kali dengan 0,50. Dengan $c = \sqrt{2}$, skornya $0,55 + 1,41\sqrt{4,6/80} \approx 0,89$ dan $0,50 + 1,41\sqrt{4,6/20} \approx 1,18$. Langkah kedua dicoba lagi, karena ketidakpastiannya masih besar. AlphaGo (Silver et al., 2016) menggabungkan MCTS dengan jaringan saraf yang menilai posisi dan menyarankan langkah, lalu AlphaZero (Silver et al., 2018) mempelajari catur, shogi, dan Go hanya dari bermain melawan dirinya sendiri. Kuliah menekankan pelajaran yang lebih umum: pembelajaran memberi intuisi, pencarian memberi ketelitian, dan sistem terkuat memakai keduanya.

Kuliah machine learning in games menunjukkan bahwa pembelajaran dalam permainan sudah jauh lebih tua dari AlphaGo. MENACE pada 1960-an adalah “mesin” dari kotak-kotak korek api berisi manik-manik berwarna yang belajar bermain tic-tac-toe: manik untuk langkah yang membawa kemenangan ditambah, manik untuk langkah yang membawa kekalahan diambil. TD-Gammon (Tesauro, 1995) melatih jaringan saraf untuk menilai posisi backgammon hanya dari bermain melawan dirinya sendiri dengan temporal difference learning (Bab 16), dan mencapai tingkat pemain manusia terbaik, sepuluh tahun sebelum deep learning populer. Kuliah juga mencatat bahwa teknik yang sama dipakai di luar permainan, misalnya untuk mengatur lift dan menjadwalkan pekerjaan di pabrik.

5.4 Perencanaan

Perencanaan adalah pencarian dengan bahasa yang lebih kaya. Alih-alih keadaan sebagai kotak hitam, keadaan dijelaskan dengan fakta-fakta logis, dan setiap aksi punya prasyarat dan efek. Bahasa STRIPS (Fikes and Nilsson, 1971), dan penerusnya PDDL, menulis aksi seperti “pindahkan balok A dari meja ke atas balok B” dengan prasyarat bahwa A dan B kosong di atasnya, dan efek bahwa A sekarang di atas B. Representasi ini memungkinkan heuristik umum yang tidak bergantung pada domain, misalnya dengan menyelesaikan versi masalah yang dilonggarkan, di mana efek negatif aksi diabaikan. Panjang rencana untuk masalah yang dilonggarkan memberi perkiraan biaya untuk masalah aslinya.

5.5 Pengetahuan dan logika

Representasi pengetahuan menyimpan fakta tentang dunia dalam bentuk yang bisa dipakai untuk bernalar. Logika proposisi bekerja dengan pernyataan benar atau salah, sedangkan logika predikat orde pertama menambahkan objek, relasi, dan kuantor, sehingga bisa menyatakan “setiap manusia fana”. Mesin inferensi menurunkan fakta baru dari fakta lama, misalnya dengan resolusi atau dengan rantai aturan maju dan mundur. Contoh kecil memperlihatkan resolusi (Robinson, 1965). Basis pengetahuan berisi “kalau hujan, jalan basah”, “kalau jalan basah, jalan licin”, dan “hujan”. Dalam bentuk klausa, aturan pertama menjadi $\neg H \vee B$ dan aturan kedua $\neg B \vee L$. Untuk membuktikan L , tambahkan negasinya, $\neg L$, lalu cari kontradiksi. Resolusi $\neg B \vee L$ dengan $\neg L$ menghasilkan $\neg B$. Resolusi $\neg B$ dengan $\neg H \vee B$ menghasilkan $\neg H$. Resolusi $\neg H$ dengan fakta H menghasilkan klausa kosong, yaitu kontradiksi. Jadi L pasti benar. Bahasa pemrograman logika seperti Prolog dibangun di atas prosedur ini.

Ontologi dan graf pengetahuan menyusun konsep ke dalam hierarki dan relasi. Menurut definisi yang dikutip di kuliah, ontologi adalah spesifikasi eksplisit dari sebuah konseptualisasi: kosakata bersama untuk sebuah domain beserta makna istilah-istilahnya, dan bentuk modernnya dipakai mesin pencari untuk menjawab pertanyaan faktual secara langsung. Kekuatan pendekatan simbolik adalah ketelitian dan kemampuan menjelaskan penalarannya. Kelemahannya adalah kerapuhan di hadapan dunia nyata yang berisik dan tidak lengkap, persis di tempat machine learning unggul.

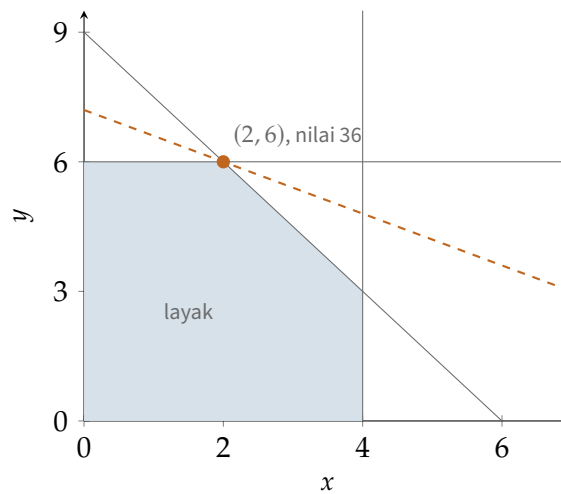
5.6 Keputusan optimal: operations research

Kuliah operations research membahas keputusan yang bisa dirumuskan sebagai optimasi dengan kendala. Bentuk paling penting adalah *linear programming*: maksimalkan fungsi linear dengan kendala pertidaksamaan linear. Contoh klasik dari Hillier and Lieberman (2015) adalah pabrik yang membuat dua produk dengan keuntungan 3 dan 5 per unit, dan tiga pabrik pendukung dengan kapasitas terbatas:

$$\max 3x + 5y \quad \text{dengan} \quad x \leq 4, \quad 2y \leq 12, \quad 3x + 2y \leq 18, \quad x, y \geq 0. \quad (5.3)$$

Daerah yang memenuhi semua kendala adalah sebuah poligon, dan karena fungsi tujuannya linear, optimumnya selalu berada di salah satu titik sudut. Di sini titik

terbaiknya $(x, y) = (2, 6)$ dengan keuntungan 36. Metode simpleks berjalan dari satu titik sudut ke titik sudut tetangga yang lebih baik sampai tidak ada lagi perbaikan (Gambar 5.2).



GAMBAR 5.2. Program linear dua variabel. Daerah layak (biru) dibatasi tiga kendala. Garis putus-putus adalah garis keuntungan $3x + 5y = 36$, yang menyentuh daerah layak tepat di titik sudut $(2, 6)$.

Setiap program linear punya pasangan, yaitu program dual, yang variabelnya bisa dibaca sebagai harga bayangan setiap sumber daya: berapa keuntungan bertambah kalau kapasitas satu pabrik dinaikkan satu unit. Untuk contoh di atas, dualnya adalah

$$\min 4u_1 + 12u_2 + 18u_3 \quad \text{dengan} \quad u_1 + 3u_3 \geq 3, \quad 2u_2 + 2u_3 \geq 5, \quad u_i \geq 0, \quad (5.4)$$

setiap kendala dual berasal dari satu produk dan setiap variabel dual dari satu pabrik. Solusinya $(u_1, u_2, u_3) = (0; 1,5; 1)$ dengan nilai $12 \cdot 1,5 + 18 \cdot 1 = 36$, sama dengan nilai primal. Pabrik pertama punya harga bayangan nol: kapasitasnya tidak terpakai penuh di optimum ($x = 2 < 4$), sehingga menambahnya tidak berguna. Satu jam tambahan di pabrik kedua bernilai 1,5, dan di pabrik ketiga bernilai satu. Informasi seperti ini sering lebih berguna bagi manajer daripada solusinya sendiri. Pada optimum, nilai primal dan dual selalu sama, dan teori dualitas ini menjadi dasar banyak algoritme optimasi modern, termasuk support vector machine di Bab 6.

Kalau variabel harus bilangan bulat, misalnya jumlah truk, masalahnya menjadi *integer programming*, yang jauh lebih sulit. *Branch and bound* memecah masalah menjadi cabang dengan menambahkan kendala seperti $x \leq 2$ atau $x \geq 3$, memakai solusi relaksasi linear sebagai batas, dan membuang cabang yang batasnya sudah lebih buruk dari solusi terbaik yang ditemukan. *Dynamic programming* (Bellman, 1957) memecah keputusan berurutan menjadi submasalah dengan prinsip optimalitas: bagian akhir dari kebijakan optimal juga optimal untuk keadaan tempat bagian itu dimulai. Prinsip yang sama menjadi persamaan Bellman di reinforcement learning (Bab 16). Untuk masalah yang terlalu besar, heuristik seperti simulated annealing dan tabu search kembali menjadi pilihan, dan analisis keputusan menambahkan ketidakpastian dengan pohon keputusan dan nilai harapan.

Contoh analisis keputusan di kuliah, yang diambil dari Hillier and Lieberman (2015), adalah perusahaan yang memiliki tanah yang mungkin mengandung minyak, dengan peluang satu banding empat. Mengebor sendiri memberi keuntungan 700 ribu dolar kalau ada minyak dan rugi 100 ribu kalau kering. Menjual tanah memberi 90 ribu pasti. Nilai harapan mengebor $0,25 \cdot 700 - 0,75 \cdot 100 = 100$ ribu, lebih tinggi dari menjual. Berapa nilai informasi sempurna? Kalau kita tahu sebelumnya apakah ada minyak, kita akan mengebor kalau ada dan menjual kalau tidak, dengan nilai harapan $0,25 \cdot 700 + 0,75 \cdot 90 = 242,5$ ribu. Selisihnya, 142,5 ribu, adalah batas atas yang masuk akal untuk dibayar demi informasi, misalnya survei seismik. Survei yang tidak sempurna bernilai lebih kecil, dan teorema Bayes (Bab 14) dipakai untuk memperbarui peluang setelah hasil survei diketahui. Pohon keputusannya diselesaikan dari belakang, mengambil nilai harapan di simpul peluang dan maksimum di simpul keputusan, persis seperti minimax yang lawannya diganti alam.

5.7 Mengapa semua ini masih penting

Sistem AI modern jarang murni simbolik atau murni statistik. AlphaZero adalah pencarian pohon yang dipandu jaringan saraf. Model bahasa yang menyelesaikan soal matematika sering menghasilkan beberapa kemungkinan langkah lalu memilih yang terbaik, sebuah bentuk pencarian. Robot gudang memakai A* untuk rute dan pembelajaran untuk persepsi. Dalam simulasi teknik, keputusan tentang desain dan jadwal operasi sering diselesaikan dengan program linear atau integer yang dibangun di atas prediksi dari model pengganti. Pembelajaran dan penalaran tidak bersaing. Keduanya saling melengkapi.

5.8 Perkembangan riset

Gagasan memadukan pembelajaran dengan pencarian terus berkembang. MuZero (Schrittwieser et al., 2020) melanjutkan AlphaZero tanpa diberi aturan permainannya. Ia mempelajari model internal yang meramalkan nilai, policy, dan ganjaran, lalu menjalankan MCTS di dalam model itu, dan bekerja untuk Go, catur, shogi, sekaligus permainan Atari. Pencarian tidak lagi butuh simulator yang ditulis manusia.

AlphaDev (Mankowitz et al., 2023) merumuskan penulisan rutin pengurutan sebagai permainan satu pemain: setiap langkah menambahkan satu instruksi assembly, dan ganjarannya adalah kebenaran dan kecepatan program. Agen deep RL menemukan rutin pengurutan kecil yang lebih cepat dari rutin buatan manusia, dan rutin itu dimasukkan ke pustaka standar C++ LLVM. FunSearch (Romera-Paredes et al., 2024) memasang model bahasa dengan evaluator otomatis dalam lingkaran evolusi. Model bahasa mengusulkan program, evaluator menilainya, dan program terbaik dipakai sebagai contoh untuk usulan berikutnya. Cara ini menemukan konstruksi baru untuk masalah cap set dalam kombinatorika dan heuristik baru untuk bin packing. Yang dicari bukan jawabannya, melainkan program yang menghasilkan jawaban, sehingga hasilnya bisa dibaca dan diperiksa manusia.

Di sisi lain, perencanaan klasik menjadi alat ukur bagi model bahasa. PlanBench

(Valmeekam et al., 2022) memakai domain dari kompetisi perencanaan seperti dunia balok di bagian perencanaan di atas untuk menguji apakah model bahasa benar-benar merencanakan atau hanya mengingat pola dari pengetahuan umum. Hasilnya, kinerja model bahasa pada pembuatan rencana masih jauh dari memadai. Pelajaran dari bab ini kembali relevan: pemeriksa yang eksak, seperti validator rencana atau evaluator program, membuat keluaran model bahasa bisa dipercaya.

CATATAN SUMBER. Bab ini mengikuti kuliah Introduction to AI (sejarah, search, game playing, planning, representasi pengetahuan, machine learning untuk permainan, filsafat AI) dan kuliah KS Operations Research (linear programming dan dualitas, integer programming, dynamic programming, heuristik, analisis keputusan). Buku rujukan utamanya adalah Russell and Norvig (2021) dan Hillier and Lieberman (2015).

6 Belajar Tanpa Label

Pada musim panas 2025 saya mengikuti dua kuliah yang saling melengkapi. Kuliah unsupervised learning dari Sepp Hochreiter memakai naskah panjang yang bergerak dari PCA dan ICA sampai biclustering, metode yang lahir dari analisis data ekspresi gen. Kuliah machine learning and pattern classification memberi sisi praktisnya: pengklasifikasi dasar, seleksi fitur, dan cara membangun sistem pengenalan pola dari awal sampai akhir.

Sebagian besar data di dunia tidak berlabel. Rekaman sensor, foto di ponsel, dan hasil pengukuran ekspresi ribuan gen datang tanpa keterangan tentang apa isinya. Unsupervised learning mencari struktur di dalam data seperti itu: arah-arahan yang penting, sumber-sumber tersembunyi, kelompok-kelompok alami, atau peta berdimensi rendah yang mempertahankan tetangga. Hasilnya sering menjadi langkah pertama sebelum model supervised dibangun.

6.1 Dua cara melihat PCA

PCA sudah muncul di Bab 11 sebagai eigenfaces. Kuliah unsupervised menekankan bahwa ada dua cara setara untuk menurunkannya. Cara pertama mencari arah v dengan varians proyeksi terbesar, $\max_{\|v\|=1} v^T \Sigma v$, yang jawabannya vektor eigen kovarians dengan nilai eigen terbesar. Cara kedua mencari subruang berdimensi q yang meminimalkan galat rekonstruksi kuadrat rata-rata ketika data diproyeksikan ke subruang itu lalu dikembalikan. Kedua masalah punya jawaban yang sama, karena varians total tetap, dan varians yang tidak tertangkap oleh proyeksi adalah tepat galat rekonstruksinya.

Penurunan cara pertama memakai pengali Lagrange dari Bab 4. Kita memaksimalkan $v^T \Sigma v$ dengan kendala $v^T v = 1$. Menyamakan gradien fungsi tujuan dengan λ kali gradien kendala memberi $2\Sigma v = 2\lambda v$, yaitu $\Sigma v = \lambda v$: setiap titik stasioner adalah vektor eigen. Varians proyeksi pada vektor eigen itu $v^T \Sigma v = \lambda$, sehingga yang terbesar adalah vektor eigen dengan nilai eigen terbesar.

Contoh dua dimensi membuatnya konkret. Untuk data dengan kovarians $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, nilai eigennya 3 dan 1, dengan vektor eigen $(1, 1)/\sqrt{2}$ dan $(1, -1)/\sqrt{2}$. Data membentang paling lebar sepanjang diagonal, dan komponen utama pertama menangkap $3/(3+1) = 75$ persen varians. Memproyeksikan ke satu dimensi itu membuang seperempat varians, dan galat rekonstruksi rata-ratanya tepat sama dengan nilai eigen yang dibuang, yaitu satu.

PCA unik hanya sampai tanda setiap komponen, dan kalau dua nilai eigen sama, arah-arahnya bisa diputar bebas di dalam subruang mereka. Komponen utama tidak

berkorelasi satu sama lain, tetapi tidak berarti independen. PCA juga linear. Untuk struktur yang melengkung, *kernel PCA* (Schölkopf et al., 1998) memetakan data secara implisit ke ruang fitur berdimensi tinggi lewat sebuah kernel, lalu menjalankan PCA di sana. Yang dibutuhkan hanya matriks kernel antar titik data, bukan pemetaannya secara eksplisit.

6.2 Memisahkan sumber: ICA

Di sebuah pesta, dua orang berbicara bersamaan, dan dua mikrofon di sudut ruangan merekam campuran suara keduanya dengan proporsi yang berbeda. Bisakah kita mendapatkan kembali suara masing-masing orang tanpa tahu proporsi campurannya? Inilah *cocktail party problem*, dan *independent component analysis* (Hyvärinen and Oja, 2000) menjawabnya. Modelnya $x = As$, dengan s sumber yang saling independen dan A matriks campuran yang tidak diketahui. Kita mencari matriks pemisah W sehingga $y = Wx$ sedekat mungkin dengan independen.

Kuncinya adalah teorema limit pusat. Campuran dari beberapa sumber independen cenderung lebih mirip Gauss daripada sumber-sumber aslinya. Jadi kita bisa membalik logikanya: cari arah proyeksi yang hasilnya paling *tidak* Gauss. Ketidak-Gaussian diukur dengan kurtosis atau negentropi. Kurtosis berlebih, $\mathbb{E}[y^4]/\mathbb{E}[y^2]^2 - 3$, bernilai nol untuk distribusi Gauss, negatif untuk distribusi yang “rata” seperti distribusi seragam $(-1,2)$, dan positif untuk distribusi yang runcing dengan ekor tebal seperti distribusi Laplace $(+3)$. Suara manusia cenderung runcing: kebanyakan waktu hampir diam, sesekali keras. Campuran dua suara sudah lebih dekat ke Gauss, dan kurtosisnya lebih kecil. Memaksimalkan nilai mutlak kurtosis berarti mencari arah yang paling mirip satu suara saja. Langkah praktisnya dimulai dengan *whitening*, yaitu mentransformasi data agar kovariansnya menjadi matriks identitas. Setelah itu, yang tersisa hanya mencari rotasi, dan algoritme seperti FastICA atau Infomax menemukannya. ICA tidak bisa memulihkan urutan dan skala sumber, dan ia gagal kalau lebih dari satu sumber berdistribusi Gauss, karena campuran Gauss yang diputar tetap Gauss.

Perbedaan dengan PCA mudah dilihat pada sinyal suara. PCA menemukan arah varians terbesar, yang biasanya masih campuran kedua pembicara. ICA menemukan arah yang membuat setiap keluaran terdengar seperti satu orang.

6.3 Analisis faktor

Analisis faktor mirip probabilistic PCA (Bab 15), $x = Wz + \mu + \epsilon$, dengan satu perbedaan penting: noise ϵ punya kovarians diagonal Ψ yang boleh berbeda untuk setiap variabel. Setiap variabel teramati punya “noise pribadinya” sendiri, dan faktor laten z menjelaskan korelasi di antara mereka. Model ini tidak punya solusi tertutup, sehingga dilatih dengan EM. Dalam data biologis, di mana setiap gen atau setiap probe punya tingkat noise yang berbeda, asumsi ini jauh lebih masuk akal daripada noise yang sama untuk semua.

6.4 Peta berdimensi rendah

Kadang yang dicari bukan sumber, melainkan peta: gambaran dua dimensi dari data berdimensi tinggi yang mempertahankan sebanyak mungkin hubungan kedekatan. Kuliah membahas sederet metode dengan filosofi yang berbeda.

Multidimensional scaling mencari titik-titik di bidang yang jarak-jaraknya sedekat mungkin dengan jarak asli. *Non-negative matrix factorization* menguraikan data menjadi bagian-bagian yang hanya boleh dijumlahkan, tidak boleh dikurangkan, sehingga komponennya sering bisa ditafsirkan sebagai bagian-bagian nyata, misalnya bagian wajah atau topik dalam teks. *Locally linear embedding* (Roweis and Saul, 2000) menganggap setiap titik bisa direkonstruksi secara linear dari tetangganya, lalu mencari koordinat rendah dimensi yang mempertahankan bobot rekonstruksi itu. Isomap (Tenenbaum et al., 2000) mengganti jarak lurus dengan jarak di sepanjang graf tetangga, sebuah pendekatan jarak geodesik, sehingga data yang tergulung seperti gulungan kue bisa dibentangkan. *Self-organizing map* (Kohonen, 1982) menempatkan grid neuron di ruang data dan menariknya ke arah data sambil menjaga neuron yang bertetangga di grid tetap berdekatan. Metode yang paling populer untuk visualisasi saat ini adalah t-SNE dan UMAP (McInnes et al., 2018), yang mencocokkan peluang ketetanggaan di ruang asli dengan di ruang peta.

Peta seperti ini menyenangkan dilihat, tetapi mudah disalahbaca. Jarak antar kelompok di peta t-SNE tidak bermakna kuantitatif, dan ukuran kelompok bisa diperbesar atau diperkecil oleh algoritmenya. Kuliah AI and Visualization kembali ke masalah ini di Bab 17.

6.5 Pengelompokan

6.5.1 *k-means*

Algoritme *k-means* (Lloyd, 1982) bergantian antara dua langkah: tetapkan setiap titik ke pusat terdekat, lalu pindahkan setiap pusat ke rata-rata titik-titik anggotanya. Setiap langkah tidak menaikkan jumlah kuadrat jarak titik ke pusatnya, sehingga algoritme pasti berhenti, meski belum tentu di solusi terbaik. Ambil empat titik di garis bilangan, 1, 2, 10, dan 11, dengan pusat awal 1 dan 2. Pada langkah pertama, titik 1 masuk ke pusat pertama, sedangkan 2, 10, dan 11 masuk ke pusat kedua, yang lalu pindah ke rata-ratanya, $23/3 \approx 7,67$. Pada langkah kedua, titik 2 sekarang lebih dekat ke pusat pertama, sehingga kelompoknya menjadi $\{1, 2\}$ dan $\{10, 11\}$, dengan pusat 1,5 dan 10,5. Langkah berikutnya tidak mengubah apa pun. Kalau pusat awalnya buruk, hasil akhirnya bisa buruk juga, sehingga *k-means* biasanya dijalankan beberapa kali dari awal acak yang berbeda.

6.5.2 *Model campuran dan EM*

k-means adalah versi keras dari model campuran Gauss. Pada model campuran, setiap titik punya peluang menjadi anggota setiap kelompok, dan kelompoknya boleh lonjong dengan kovarians sendiri. Algoritme EM bergantian antara menghitung peluang keanggotaan (langkah E) dan memperbarui rata-rata, kovarians, dan bobot setiap

komponen dengan rata-rata berbobot (langkah M):

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}, \quad \boldsymbol{\mu}_k = \frac{\sum_n \gamma_{nk} \mathbf{x}_n}{\sum_n \gamma_{nk}}, \quad \pi_k = \frac{1}{N} \sum_n \gamma_{nk}. \quad (6.1)$$

Langkah E adalah teorema Bayes: peluang titik n berasal dari komponen k sebanding dengan bobot komponen itu dikali seberapa mungkin titik itu dihasilkannya. Langkah M adalah estimasi maksimum likelihood biasa, hanya dengan setiap titik dihitung sebagian untuk setiap komponen. Setiap putaran EM tidak pernah menurunkan likelihood, alasan yang sama dengan batas bawah ELBO di Bab 15. Ketika kovarians semua komponen dibuat sama dan kecil sekali, keanggotaan menjadi nol atau satu, dan EM berubah menjadi k -means.

Berapa banyak kelompok yang sebaiknya dipakai? Jumlah kuadrat jarak ke pusat selalu turun kalau k ditambah, sampai nol ketika setiap titik menjadi kelompoknya sendiri, sehingga ukuran itu tidak bisa dipakai langsung. Metode siku mencari titik di mana penurunannya tiba-tiba melandai. Koefisien silhouette membandingkan, untuk setiap titik, jarak rata-rata ke anggota kelompoknya sendiri, a , dengan jarak rata-rata ke kelompok terdekat lainnya, b , sebagai $(b - a)/\max(a, b)$. Nilainya mendekati satu kalau titik itu jelas berada di kelompok yang benar. Untuk contoh empat titik tadi, titik 1 punya $a = 1$ dan $b = (9 + 10)/2 = 9,5$, sehingga silhouette-nya $8,5/9,5 \approx 0,89$. Untuk model campuran, kriteria informasi seperti BIC menimbang likelihood terhadap jumlah parameter.

6.5.3 Hierarki dan kemiripan

Pengelompokan hierarkis tidak memilih jumlah kelompok di awal. Versi aglomeratif mulai dengan setiap titik sebagai kelompoknya sendiri, lalu berulang kali menggabungkan dua kelompok terdekat. Arti “terdekat” menentukan bentuk hasilnya. *Single linkage* memakai pasangan titik terdekat dan cenderung menghasilkan rantai panjang, *complete linkage* memakai pasangan terjauh dan menghasilkan kelompok yang padat, dan *average linkage* berada di antaranya. Hasilnya adalah dendrogram yang bisa dipotong di ketinggian mana pun. Metode berbasis kemiripan, termasuk pengelompokan spektral, memandang data sebagai graf kemiripan dan memotongnya di tempat ikatan paling lemah, sebuah ide yang sudah kita temui pada segmentasi gambar.

6.5.4 Biclustering

Data ekspresi gen berbentuk matriks: baris adalah gen, kolom adalah sampel jaringan atau kondisi. Sekelompok gen sering hanya bekerja sama di sebagian sampel, misalnya hanya di jaringan tumor tertentu. Mengelompokkan seluruh baris atau seluruh kolom akan melewati pola ini. *Biclustering* mencari submatriks, yaitu sekelompok baris yang berperilaku mirip pada sekelompok kolom. FABIA (Hochreiter et al., 2010), yang dikembangkan di Linz, merumuskannya sebagai model faktor dengan prior yang jarang sekali (*sparse*), sehingga setiap faktor hanya melibatkan sedikit gen dan sedikit sampel. Inilah salah satu titik pertemuan pertama antara machine learning dan ilmu hayati di buku ini, dan kita akan kembali ke data seperti ini di Bab 23.

6.6 Dari fitur ke pengklasifikasi

Kuliah pattern classification membahas jalur lengkap sebuah sistem pengenalan pola: ambil sinyal mentah, ubah menjadi fitur, pilih fitur yang berguna, latih pengklasifikasi, lalu evaluasi dengan jujur. Contoh yang dipakai banyak berasal dari audio, seperti mendeteksi kejadian akustik dalam rekaman.

Pengklasifikasi dasar yang dibahas punya karakter yang berbeda-beda. k -nearest neighbour tidak belajar apa pun dan hanya mengingat data latih, lalu meminjam label tetangga terdekat. Naive Bayes menganggap semua fitur independen bersyarat terhadap kelas, asumsi yang hampir selalu salah tetapi sering bekerja dengan baik. Analisis diskriminan linear (FISHER, 1936) mencari arah proyeksi yang memisahkan rata-rata kelas sejauh mungkin relatif terhadap sebaran di dalam kelas. Untuk dua kelas, memaksimalkan rasio $(w^\top(\mu_1 - \mu_2))^2 / (w^\top S_W w)$ memberi arah $w \propto S_W^{-1}(\mu_1 - \mu_2)$, dengan S_W matriks sebaran di dalam kelas. Kalau sebaran itu bulat, arahnya sekadar garis yang menghubungkan kedua rata-rata. Kalau sebaran lonjong, arahnya dimiringkan menjauhi arah sebaran yang lebar, karena perbedaan rata-rata di arah itu mudah tertutup noise. Pohon keputusan memotong ruang fitur dengan pertanyaan ya atau tidak yang berurutan. Pertanyaan di setiap simpul dipilih yang paling mengurangi ketidakmurnian label, misalnya entropi $H = -\sum_c p_c \log_2 p_c$. Simpul dengan delapan contoh positif dan delapan negatif punya entropi satu bit. Kalau sebuah pertanyaan membaginya menjadi (7, 1) dan (1, 7), entropi setiap anak sekitar 0,54 bit, sehingga perolehan informasinya sekitar 0,46 bit. Pertanyaan yang membagi menjadi (4, 4) dan (4, 4) tidak memberi informasi sama sekali. Pohon tunggal mudah overfitting, dan random forest (Breiman, 2001) merata-ratakan banyak pohon yang dilatih pada sampel dan fitur acak. Support vector machine (Cortes and Vapnik, 1995) mencari bidang pemisah dengan margin terlebar, dan dengan kernel ia bisa memisahkan kelas yang tidak terpisah secara linear. Ide marginnya adalah regularisasi dalam bentuk geometris: di antara semua bidang yang memisahkan data latih, pilih yang paling jauh dari titik mana pun. Jarak sebuah titik ke bidang $w^\top x + b = 0$ adalah $|w^\top x + b| / \|w\|$. Kalau skala w diatur sehingga titik-titik terdekat memenuhi $y_n(w^\top x_n + b) = 1$, lebar margin di kedua sisi menjadi $2 / \|w\|$. Memaksimalkan margin sama dengan meminimalkan $\frac{1}{2} \|w\|^2$ dengan kendala $y_n(w^\top x_n + b) \geq 1$ untuk semua n , masalah kuadrat dengan kendala yang diselesaikan lewat bentuk dualnya (Bab 4). Di bentuk dual, data hanya muncul sebagai hasil kali dalam $x_n^\top x_m$, dan mengganti hasil kali itu dengan kernel memberi SVM nonlinear tanpa pernah menghitung pemetaan fiturnya.

Seleksi fitur bisa dilakukan dengan tiga cara. Metode filter menilai setiap fitur sendiri-sendiri, misalnya dari korelasinya dengan label. Metode wrapper mencoba berbagai himpunan fitur dengan melatih model sungguhan. Metode embedded memilih fitur sebagai bagian dari pelatihan, seperti lasso yang mendorong bobot fitur yang tidak berguna ke nol. Kuliah juga memperkenalkan jaringan saraf dengan cara pandang praktis: kapan lapisan dense cukup, kapan konvolusi lebih cocok, dan bagaimana transfer learning serta pretraining self-supervised mengurangi kebutuhan data berlabel.

6.7 Di luar kelas

Metode tanpa label muncul di hampir setiap bidang yang disentuh buku ini. Dalam ilmu hayati, pengelompokan profil ekspresi gen memisahkan subtype kanker yang tidak terlihat di bawah mikroskop, dan biclustering menemukan modul gen yang aktif bersama. Dalam pemrosesan sinyal, ICA memisahkan sinyal jantung ibu dan janin dari elektroda yang sama, dan PCA memampatkan data sensor sebelum dikirim. Dalam simulasi, PCA dengan nama proper orthogonal decomposition mereduksi medan aliran berjuta sel menjadi beberapa puluh mode (Bab 31). Dalam sistem rekomendasi, faktorisasi matriks yang mirip analisis faktor menemukan selera tersembunyi pengguna (Bab 34). Di semua tempat itu, pertanyaannya sama: struktur apa yang tersembunyi di balik angka-angka ini?

6.8 Perkembangan riset

Batas antara unsupervised dan supervised learning kini kabur karena *self-supervised learning*: label dibuat dari data itu sendiri. SimCLR (Chen et al., 2020) melatih encoder agar dua versi augmentasi dari gambar yang sama berdekatan dan versi dari gambar lain berjauhan, loss kontrastif yang sama dengan CLIP di Bab 23. Studi sistematisnya menemukan bahwa susunan augmentasi, satu proyeksi nonlinear tambahan, dan batch yang besar adalah kunci keberhasilannya. Masked autoencoder (He et al., 2021) mengambil jalan lain: sembunyikan tiga perempat potongan gambar secara acak, lalu latih jaringan merekonstruksi piksel yang hilang. Karena encoder hanya melihat potongan yang terlihat, pelatihannya juga tiga kali lebih cepat.

I-JEPA (Assran et al., 2023) berargumen bahwa merekonstruksi piksel membuang kapasitas untuk detail yang tidak penting. Ia meramalkan *representasi* dari blok-blok yang disembunyikan, bukan pikselnya, dari satu blok konteks, sehingga yang dipelajari lebih semantik. DINOv2 (Oquab et al., 2023) menunjukkan bahwa dengan data yang dikurasi dengan cermat dan model miliaran parameter, fitur self-supervised bisa dipakai langsung untuk banyak tugas tanpa penyetelan ulang. Analisis faktor dan PCA di bab ini mencari struktur laten dengan model linear. Metode-metode ini mencari struktur yang sama dengan jaringan dalam, dan menjadi titik awal model fondasi di bidang visi, citra medis, dan sinyal sensor.

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Machine Learning: Unsupervised Techniques (PCA dan kernel PCA, ICA, analisis faktor, metode proyeksi, clustering, biclustering) dan kuliah Machine Learning and Pattern Classification (pengklasifikasi dasar, seleksi dan konstruksi fitur, jaringan saraf dalam praktik). Bacaan pendamping: Bishop (2006), Hastie et al. (2009), dan Duda et al. (2001).

Bagian II

Deep Learning dan Persepsi

7 Deep Learning: Teknik Dasar

Ini kuliah deep learning pertama saya di Linz, pada semester musim dingin 2024/25. Naskah kuliahnya lebih mirip buku daripada slide, lengkap dengan sejarah, notasi yang rapi, dan bab khusus tentang inisialisasi dan normalisasi.

Bab ini membangun jaringan saraf dari satu neuron sampai jaringan konvolusi. Empat pertanyaan menjadi benang merahnya: apa yang bisa dihitung oleh sebuah jaringan, bagaimana gradiennya dihitung, bagaimana sinyal dijaga agar tidak mati atau meledak di jaringan yang dalam, dan bagaimana mencegah jaringan sekadar menghafal.

7.1 Sejarah singkat

McCulloch and Pitts (1943) memodelkan neuron sebagai unit ambang: jumlahkan input yang berbobot, lalu keluarkan satu kalau jumlahnya melewati ambang. Rosenblatt (1958) membuat *perceptron*, yang bisa mempelajari bobotnya sendiri. Antusiasme sempat surut ketika menjadi jelas bahwa satu lapisan perceptron tidak bisa menyelesaikan masalah sesederhana XOR. Algoritme backpropagation untuk jaringan berlapis kemudian dipopulerkan oleh Rumelhart et al. (1986), meski gagasannya sudah muncul lebih awal. Lecun et al. (1998) memakai jaringan konvolusi untuk membaca tulisan tangan, dan ledakan modern dimulai ketika jaringan konvolusi besar yang dilatih di GPU memenangkan kompetisi ImageNet (Krizhevsky et al., 2017). Sejarah yang lebih lengkap, termasuk peran Linz melalui LSTM, ditulis oleh Schmidhuber (2015).

7.2 Dari satu neuron ke jaringan berlapis

Satu neuron menghitung *pre-activation* $s = \mathbf{w}^\top \mathbf{x} + b$, lalu *activation* $a = f(s)$. Kalau f fungsi tangga, kita mendapat perceptron. Kalau f sigmoid $\sigma(s) = 1/(1 + e^{-s})$, keluarannya bisa dibaca sebagai peluang, dan model ini tidak lain adalah regresi logistik. Dalam kedua kasus, batas keputusannya, $\mathbf{w}^\top \mathbf{x} + b = 0$, selalu berupa bidang datar.

Mengapa satu bidang datar tidak cukup untuk XOR bisa dibuktikan dalam beberapa baris. Titik (0,0) dan (1,1) berlabel nol, sedangkan (0,1) dan (1,0) berlabel satu. Misalkan ada garis $w_1x_1 + w_2x_2 + b = 0$ yang memisahnya, dengan label satu di sisi positif. Titik (0,0) memberi $b < 0$. Titik (0,1) dan (1,0) memberi $w_2 + b > 0$ dan $w_1 + b > 0$. Kalau keduanya dijumlahkan, diperoleh $w_1 + w_2 + b > -b > 0$. Padahal (1,1) berlabel nol, sehingga seharusnya $w_1 + w_2 + b < 0$. Kontradiksi. Satu lapisan tersembunyi dengan dua neuron sudah cukup untuk menyelesaikannya.

Untuk data yang memang bisa dipisahkan garis, aturan belajar perceptron (Rosenblatt, 1958) sederhana: setiap kali sebuah contoh $(\mathbf{x}, y)$ dengan $y \in \{-1, +1\}$ salah diklasifikasikan, tambahkan $y\mathbf{x}$ ke vektor bobot. Teorema konvergensi perceptron, yang dibuktikan di naskah kuliah, menjamin bahwa aturan ini berhenti setelah jumlah

kesalahan yang terbatas. Kalau semua input punya norma paling besar R dan ada pemisah dengan margin γ , jumlah koreksinya paling banyak $(R/\gamma)^2$. Ide pembuktiannya: setiap koreksi menaikkan hasil kali dalam bobot dengan pemisah ideal paling sedikit γ , sedangkan kuadrat norma bobot hanya naik paling banyak R^2 , sehingga setelah k koreksi, $k\gamma \leq \|w\| \leq R\sqrt{k}$, yang hanya mungkin kalau $k \leq (R/\gamma)^2$. Dengan $R = 2$ dan $\gamma = 0,5$, paling banyak 16 koreksi. Margin yang tipis berarti belajar yang lama, gagasan margin yang juga menjadi inti support vector machine di Bab 6.

Jaringan *feed-forward* dengan L lapisan menumpuk transformasi yang sama berkali-kali. Dengan notasi kuliah,

$$s^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(s^{(l)}), \quad a^{(0)} = x, \quad \hat{y} = a^{(L)}. \quad (7.1)$$

Tanpa fungsi aktivasi yang nonlinear, tumpukan lapisan ini hanyalah satu perkalian matriks besar. Dengan nonlinearitas, *universal approximation theorem* (Cybenko, 1989; Hornik et al., 1989) menjamin bahwa satu lapisan tersembunyi dengan cukup banyak neuron bisa mendekati fungsi kontinu apa pun pada domain yang kompak. Teorema ini tidak mengatakan berapa banyak neuron yang dibutuhkan, dan juga tidak menjamin gradient descent akan menemukannya. Dalam praktik, jaringan yang dalam memakai parameter jauh lebih hemat daripada jaringan yang lebar tetapi dangkal.

Cara saya membayangkannya adalah origami. Lapisan dengan aktivasi ReLU melipat ruang input sepanjang beberapa garis lipatan. Lapisan berikutnya melipat hasil lipatan itu sekali lagi. Dengan beberapa kali lipatan, selembar kertas bisa berubah menjadi bentuk yang rumit, dan jumlah bagian datar yang terbentuk bertambah jauh lebih cepat dengan menambah lipatan baru daripada dengan memperlebar kertas.

7.3 Backpropagation

Untuk melatih jaringan dengan gradient descent, kita membutuhkan $\partial \mathcal{L} / \partial W^{(l)}$ untuk semua lapisan. Backpropagation adalah aturan rantai yang dihitung dengan urutan yang cerdas: dari belakang ke depan, sambil menyimpan hasil antara. Definisikan *delta* lapisan l sebagai $\delta^{(l)} = \partial \mathcal{L} / \partial s^{(l)}$. Maka

$$\delta^{(L)} = f'(s^{(L)}) \odot \frac{\partial \mathcal{L}}{\partial a^{(L)}}, \quad (7.2)$$

$$\delta^{(l)} = f'(s^{(l)}) \odot (W^{(l+1)})^\top \delta^{(l+1)}, \quad (7.3)$$

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^\top, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}. \quad (7.4)$$

Bayangkan sebuah pabrik yang menghasilkan produk cacat. Manajer di ujung jalur produksi melihat seberapa besar cacatnya, lalu membagi kesalahan itu ke para pekerja di stasiun sebelumnya, sebanding dengan seberapa besar sumbangan mereka pada produk akhir (bobot $W^\top$) dan seberapa peka mereka saat itu (f'). Setiap pekerja meneruskan bagian kesalahannya ke stasiun sebelumnya dengan cara yang sama. Tidak ada yang dihitung dua kali, dan itulah sebabnya backprop mudah.

Contoh dengan angka membuatnya konkret. Ambil input $x = 1$, satu neuron tersembunyi $a_1 = \sigma(w_1 x)$ dengan $w_1 = 0,5$, keluaran linear $\hat{y} = w_2 a_1$ dengan $w_2 = 2$,

target $y = 1$, dan loss $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$. Lintasan maju memberi $s_1 = 0,5$, $a_1 = \sigma(0,5) = 0,6225$, $\hat{y} = 1,2449$, dan $\mathcal{L} = 0,0300$. Delta keluaran adalah $\delta_2 = \hat{y} - y = 0,2449$, sehingga $\partial\mathcal{L}/\partial w_2 = \delta_2 a_1 = 0,1525$. Untuk lapisan tersembunyi, $\sigma'(s_1) = a_1(1 - a_1) = 0,2350$, sehingga $\delta_1 = 0,2350 \times w_2 \times \delta_2 = 0,1151$ dan $\partial\mathcal{L}/\partial w_1 = \delta_1 x = 0,1151$. Pengecekan dengan beda hingga selalu baik dilakukan saat menulis backprop sendiri: naikan w_1 sebesar $0,001$, dan loss seharusnya naik kira-kira $0,001 \times 0,1151$.

Kerangka kerja modern tidak menurunkan rumus delta secara manual. Mereka memakai *automatic differentiation* (Baydin et al., 2015). Program dipecah menjadi operasi dasar yang turunannya diketahui, lalu turunan dirangkai dengan salah satu dari dua cara. *Forward mode* membawa turunan terhadap satu input maju bersama perhitungan, cocok kalau input sedikit dan output banyak. *Reverse mode* mencatat semua operasi lalu berjalan mundur, cocok kalau outputnya sedikit, misalnya satu nilai loss, dan inputnya banyak, misalnya jutaan bobot. Backpropagation adalah reverse mode yang diterapkan pada jaringan saraf. Biayanya hanya beberapa kali biaya satu lintasan maju, berapa pun jumlah parameternya.

Autodiff tidak hanya bisa menghitung turunan loss terhadap bobot. Ia juga bisa menghitung turunan keluaran jaringan terhadap *input*. Kalau sebuah jaringan $u_\theta(x, t)$ memodelkan temperatur, autodiff memberi $\partial u_\theta / \partial t$ dan $\partial^2 u_\theta / \partial x^2$ secara eksak, tanpa grid dan tanpa beda hingga. Kemampuan sederhana inilah yang membuat PINN mungkin: residu persamaan panas $\partial_t u - \kappa \partial_{xx} u$ bisa dihitung langsung di titik mana pun (Bab 29).

Kuliah juga memperkenalkan dua objek turunan yang lebih umum. Jacobian $\mathbf{J} = \partial \hat{\mathbf{y}} / \partial \mathbf{x}$ mengukur kepekaan setiap keluaran terhadap setiap input, dan backprop pada dasarnya mengalikan Jacobian lapisan-lapisan dari kanan ke kiri dengan sebuah vektor. Hessian $\mathbf{H} = \partial^2 \mathcal{L} / \partial \mathbf{w} \partial \mathbf{w}^\top$ mengukur kelengkungan loss dan menentukan condition number masalah optimasi (Bab 4). Hasil kali Hessian dengan sebuah vektor bisa dihitung tanpa pernah membentuk Hessian, cukup dengan dua lintasan autodiff.

7.3.1 Backpropagation dengan angka

Satu lintasan maju dan mundur pada jaringan sekecil mungkin membuat rumus-rumus di atas terasa nyata. Ambil satu input $x = 1$, satu neuron tersembunyi dengan bobot $w_1 = 0,5$ dan aktivasi ReLU, satu keluaran linear dengan bobot $w_2 = 2$, target $y = 2$, dan loss $\frac{1}{2}(\hat{y} - y)^2$. Lintasan maju: $s_1 = w_1 x = 0,5$, $h = \text{ReLU}(0,5) = 0,5$, $\hat{y} = w_2 h = 1$, dan loss $\frac{1}{2}(1 - 2)^2 = 0,5$.

Lintasan mundur dimulai dari keluaran. Turunan loss terhadap $\hat{y}$ adalah $\hat{y} - y = -1$. Karena $\hat{y} = w_2 h$, gradien untuk w_2 adalah $-1 \cdot h = -0,5$, dan delta yang dikirim ke neuron tersembunyi adalah $-1 \cdot w_2 = -2$. ReLU meneruskan delta tanpa perubahan karena $s_1 > 0$, sehingga gradien untuk w_1 adalah $-2 \cdot x = -2$. Dengan laju belajar $0,1$, bobot baru menjadi $w_2 = 2,05$ dan $w_1 = 0,7$. Lintasan maju berikutnya memberi $\hat{y} = 2,05 \cdot 0,7 \approx 1,44$, lebih dekat ke target. Perhatikan bahwa w_1 mendapat gradien empat kali lebih besar dari w_2 , karena ia dikalikan bobot keluaran yang besar. Ketidakseimbangan skala seperti ini, yang menumpuk di jaringan dalam, adalah alasan inisialisasi dan normalisasi di bagian berikut begitu penting.

7.4 Fungsi aktivasi

Sigmoid dan tanh adalah fungsi aktivasi klasik, tetapi keduanya jenuh di kedua sisi: untuk input yang besar, turunannya hampir nol, dan turunan sigmoid tidak pernah melebihi seperempat. ReLU, $\max(0, s)$, murah dan tidak jenuh untuk input positif, tetapi neuron bisa “mati” kalau pre-activation-nya selalu negatif. ELU memperhalus bagian negatif dengan $\alpha(e^s - 1)$ sehingga rata-rata keluarannya lebih dekat ke nol. SELU adalah ELU yang diskalakan dengan konstanta tertentu, $\lambda \approx 1,0507$ dan $\alpha \approx 1,6733$, dan kita akan melihat sebentar lagi mengapa angka-angka itu istimewa. Transformer umumnya memakai GELU atau SiLU, yang mulus di sekitar nol.

Pasangan aktivasi keluaran dan loss ditentukan oleh jenis tugas, sesuai Tabel 1.1: keluaran linear dengan MSE untuk regresi, sigmoid dengan binary cross-entropy untuk dua kelas, dan softmax dengan cross-entropy untuk banyak kelas. Pasangan ini dipilih karena delta keluarannya menjadi sederhana sekali. Untuk softmax dengan cross-entropy, misalnya, $\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{y}$.

Hasil ini layak diturunkan sekali, karena ia muncul di setiap pengklasifikasi. Dengan $\hat{y}_k = e^{s_k} / \sum_j e^{s_j}$ dan label one-hot $\mathbf{y}$, loss-nya $\mathcal{L} = -\sum_k y_k \log \hat{y}_k = -\sum_k y_k s_k + \log \sum_j e^{s_j}$, karena $\sum_k y_k = 1$. Turunan suku pertama terhadap s_i adalah $-y_i$. Turunan suku kedua adalah $e^{s_i} / \sum_j e^{s_j} = \hat{y}_i$. Jadi $\partial \mathcal{L} / \partial s_i = \hat{y}_i - y_i$. Turunan softmax yang rumit, dengan semua suku silangnya, saling menghapus dengan logaritma dari cross-entropy. Sebagai contoh, kalau tiga kelas memberi $\hat{\mathbf{y}} = (0,7; 0,2; 0,1)$ dan kelas yang benar adalah yang pertama, deltanya $(-0,3; 0,2; 0,1)$: skor kelas yang benar didorong naik, skor kelas lain didorong turun sebanding dengan peluang yang keliru diberikan kepadanya.

7.5 Menjaga sinyal tetap hidup

Di jaringan yang dalam, sinyal maju dan gradien mundur melewati banyak perkalian matriks. Kalau setiap lapisan sedikit mengecilkan sinyal, setelah lima puluh lapisan sinyalnya hilang. Kalau setiap lapisan sedikit membesarkannya, sinyal meledak. Permainan bisik berantai memberi gambaran yang tepat. Kalau setiap orang berbisik sedikit lebih pelan dari yang ia dengar, pesan lenyap setelah dua puluh orang. Kalau setiap orang berbicara sedikit lebih keras, orang di ujung barisan sudah berteriak. Inisialisasi yang baik membuat setiap orang mengulang pesan dengan volume yang sama.

Kuliah membahas hal ini dengan *mean field theory*, yaitu melacak rata-rata dan varians aktivasi dari lapisan ke lapisan. Ambil satu pre-activation, $s_i = \sum_{j=1}^J w_{ij} a_j$. Kalau bobot diambil i.i.d. dengan rata-rata nol dan varians $\text{Var}(w)$, independen dari aktivasi, maka setiap suku punya rata-rata nol dan varians $\mathbb{E}[w^2 a^2] = \text{Var}(w) \mathbb{E}[a^2]$. Varians jumlah J suku independen adalah jumlah variansnya, sehingga

$$\text{Var}(s_i) = J \text{Var}(w) \mathbb{E}[a^2]. \quad (7.5)$$

Agar varians tidak berubah dari lapisan ke lapisan, kita butuh $J \text{Var}(w) \approx 1$ untuk aktivasi yang hampir linear di sekitar nol, yaitu inisialisasi LeCun, $\text{Var}(w) = 1/J$. Glorot meratakan kebutuhan arah maju dan mundur menjadi $\text{Var}(w) = 2/(J + I)$, dengan I jumlah

output. Untuk ReLU, $a = \max(0, s)$. Kalau s simetris di sekitar nol, separuh nilainya dipotong menjadi nol dan separuh lainnya dibiarkan, sehingga $\mathbb{E}[a^2] = \frac{1}{2}\mathbb{E}[s^2] = \frac{1}{2}\text{Var}(s)$. Masukkan ke rumus di atas: varians terjaga kalau $J \text{Var}(w) \cdot \frac{1}{2} = 1$, yaitu $\text{Var}(w) = 2/J$ (He et al., 2015). Untuk lapisan dengan 512 input, simpangan baku bobotnya $\sqrt{2/512} = 0,0625$. Dengan $\sqrt{1/512} \approx 0,044$ yang cocok untuk tanh, varians sinyal di jaringan ReLU akan menyusut separuh di setiap lapisan, dan setelah dua puluh lapisan tinggal sekitar sepersejuta.

Inisialisasi menjaga sinyal di awal pelatihan, sedangkan normalisasi menjaganya terus-menerus. *Batch normalization* (Ioffe and Szegedy, 2015) menormalkan setiap pre-activation dengan rata-rata dan simpangan baku di dalam mini-batch, lalu memberi skala γ dan geser β yang bisa dipelajari:

$$\hat{s} = \frac{s - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y = \gamma \hat{s} + \beta. \quad (7.6)$$

Saat inferensi, statistik batch diganti rata-rata bergerak yang dikumpulkan selama pelatihan. *Layer normalization* melakukan hal yang serupa, tetapi rata-ratanya dihitung atas neuron dalam satu contoh, bukan atas contoh dalam satu batch. Karena tidak bergantung pada ukuran batch, layer norm menjadi standar di jaringan rekuren dan transformer.

Pendekatan yang khas Linz adalah *self-normalizing networks* (Klambauer et al., 2017). Dengan aktivasi SELU dan inisialisasi yang tepat, pemetaan pasangan (rata-rata, varians) dari satu lapisan ke lapisan berikutnya punya titik tetap yang stabil dan menarik di $(0, 1)$. Kalau aktivasi menyimpang dari titik itu, lapisan berikutnya menariknya kembali. Jaringan menormalkan dirinya sendiri tanpa lapisan normalisasi tambahan, dan konstanta λ serta α pada SELU dipilih tepat agar titik tetap itu ada.

7.6 Regularisasi

Jaringan modern punya parameter jauh lebih banyak daripada jumlah contoh. Tanpa pengekanan, ia bisa menghafal. Kuliah membahas beberapa cara mengekangnya. Weight decay, suku $\frac{\lambda}{2}\|w\|^2$ pada loss, setara dengan prior Gauss pada bobot (Bab 1). Menambahkan noise pada neuron atau bobot selama pelatihan memaksa jaringan tahan terhadap gangguan kecil, dan untuk model linear, noise Gauss pada input setara dengan regularisasi L2. Dropout mematikan setiap neuron dengan peluang p selama pelatihan (Hinton et al., 2012), sehingga jaringan tidak bisa bergantung pada satu neuron tertentu. Saat inferensi semua neuron aktif dan keluaran diskalakan, dan dropout bisa dibaca sebagai melatih ensemble amat besar dari sub-jaringan yang berbagi bobot. Early stopping menghentikan pelatihan ketika galat validasi mulai naik. Data augmentation memperbanyak data dengan transformasi yang tidak mengubah label, seperti menggeser, memutar, atau mengubah kecerahan. Ada juga cara yang mengubah ukuran jaringan selama pelatihan, yaitu menambah neuron (*growing*) atau membuang bobot yang kurang penting (*pruning*), serta pencarian eksplisit akan minimum yang datar (Hochreiter and Schmidhuber, 1997b).

Bobot mana yang aman dibuang? Jawaban naif adalah bobot yang paling kecil. Naskah kuliah membahas jawaban yang lebih teliti dari metode *Optimal Brain Damage*. Di sekitar minimum, gradien nol, sehingga ekspansi Taylor orde dua (Lampiran B) menyatakan bahwa menghapus bobot w_i , yaitu mengubahnya sebesar $-w_i$, menaikkan loss kira-kira sebesar

$$s_i = \frac{1}{2} h_{ii} w_i^2, \quad (7.7)$$

dengan h_{ii} elemen diagonal Hessian. Bobot $w = 0,1$ di arah yang melengkung, $h = 50$, punya saliency 0,25. Bobot $w = 1$ di arah yang hampir datar, $h = 0,1$, hanya 0,05. Bobot yang *lebih besar* justru lebih aman dibuang, karena loss hampir tidak peduli pada arah itu. *Optimal Brain Surgeon* memakai Hessian penuh dan sekaligus menghitung bagaimana bobot-bobot lain harus digeser untuk menutupi bobot yang dibuang. Gagasan kelengkungan yang sama menghubungkan pruning dengan minimum datar: bobot di arah yang datar bisa diubah tanpa merugikan, sehingga bisa disimpan dengan presisi rendah atau dibuang sama sekali.

Dua kesalahan praktis sering terjadi di sini. Yang pertama adalah lupa mematikan dropout dan lupa memakai statistik batch norm hasil pelatihan saat evaluasi. Di PyTorch, inilah beda antara `model.train()` dan `model.eval()`. Yang kedua adalah augmentasi yang diam-diam mengubah label. Membalik angka 6 bisa menjadikannya 9. Pada data fisika, memutar medan kecepatan tanpa ikut memutar arah vektor-vektor kecepatannya juga menghasilkan data yang salah.

Ada detail kecil dalam dropout yang sering membingungkan. Kalau setiap neuron dimatikan dengan peluang p , nilai harapan keluarannya selama pelatihan hanya $(1 - p)$ kali nilai aslinya. Saat inferensi semua neuron aktif, sehingga lapisan berikutnya akan menerima input yang lebih besar dari yang biasa ia lihat. Implementasi modern memakai *inverted dropout*: selama pelatihan, neuron yang selamat dikalikan $1/(1 - p)$. Dengan $p = 0,5$, neuron yang tidak dimatikan dikali dua, sehingga nilai harapannya tetap sama dengan aslinya, dan saat inferensi tidak perlu ada penskalaan sama sekali.

7.7 Normalisasi dan sambungan residual

Batch normalization (Ioffe and Szegedy, 2015) menstandarkan setiap fitur dengan statistik mini-batch, lalu mengembalikan skala dan geser yang dipelajari:

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta. \quad (7.8)$$

Untuk mini-batch dengan nilai (1, 2, 3, 6), rata-ratanya 3, variansnya 3,5, dan nilai yang distandarkan sekitar $(-1,07; -0,53; 0; 1,60)$. Parameter γ dan β memastikan jaringan tetap bisa mewakili fungsi yang sama seperti tanpa normalisasi kalau itu memang yang terbaik. Saat inferensi, statistik batch diganti rata-rata bergerak dari pelatihan, sumber kesalahan `model.eval()` yang disebut di atas. Layer normalization menghitung statistik yang sama di sepanjang fitur untuk setiap contoh, bukan di sepanjang batch, sehingga tidak bergantung pada ukuran batch dan menjadi pilihan standar di transformer.

Sambungan residual (He et al., 2016) menulis keluaran sebuah blok sebagai $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l)$. Turunannya terhadap input blok adalah $\mathbf{I} + \partial F / \partial \mathbf{h}_l$. Melewati L blok, delta

dikalikan dengan hasil kali faktor-faktor seperti ini, dan suku identitas memastikan selalu ada jalur yang dilewati gradien tanpa diperkecil. Itulah gagasan constant error carousel dari LSTM (Bab 9), kali ini di sepanjang kedalaman, bukan di sepanjang waktu. Blok residual juga bisa dibaca sebagai satu langkah Euler dari persamaan diferensial $\dot{\mathbf{h}} = F(\mathbf{h})$, sudut pandang yang melahirkan neural ODE di Bab 15.

7.8 Jaringan konvolusi

Gambar berukuran 256×256 dengan tiga kanal punya hampir dua ratus ribu input. Lapisan fully-connected ke seribu neuron akan membutuhkan dua ratus juta bobot. Jaringan konvolusi memangkas angka ini dengan dua asumsi: pola penting bersifat lokal, dan pola yang sama bisa muncul di mana saja. Satu *kernel* kecil, misalnya 3×3 , digeser ke seluruh gambar,

$$(\mathbf{a} * \mathbf{k})_{ij} = \sum_{u,v} a_{i+u, j+v} k_{uv}, \quad (7.9)$$

dan bobot yang sama dipakai di semua posisi (*weight sharing*). Secara matematis, operasi ini adalah *cross-correlation*, karena konvolusi sejati membalik kernelnya lebih dulu. Karena kernel dipelajari, perbedaan itu tidak penting.

Bayangkan Anda mencari satu tanda tangan di sebuah dokumen besar dengan kaca pembesar kecil. Anda menggeser kaca pembesar ke seluruh halaman dan mencatat seberapa mirip isi di bawahnya dengan tanda tangan yang dicari. Anda tidak butuh kaca pembesar yang berbeda untuk setiap posisi. Setiap kernel konvolusi adalah satu kaca pembesar yang mencari satu jenis pola.

Untuk input berukuran I , kernel K , padding P , dan stride S , ukuran keluarannya adalah

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1, \quad (7.10)$$

dan satu lapisan dengan C_{in} kanal masuk serta C_{out} kanal keluar punya $K^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ parameter, tidak bergantung pada ukuran gambar. Rumus ukuran keluaran mudah diturunkan dengan menghitung posisi. Setelah padding, panjang input menjadi $I + 2P$. Kernel sepanjang K pertama kali diletakkan di posisi paling kiri, dan ruang geraknya sampai mentok di kanan adalah $I + 2P - K$ piksel. Dengan langkah S , banyaknya langkah yang muat adalah $\lfloor (I + 2P - K)/S \rfloor$, ditambah satu untuk posisi awal. Jumlah parameternya berasal dari satu kernel $K \times K$ untuk setiap pasangan kanal masuk dan kanal keluar, ditambah satu bias per kanal keluar. LeNet memberi contoh klasik (Lecun et al., 1998). Input 32×32 dengan kernel 5×5 tanpa padding menghasilkan keluaran 28×28 . Dengan enam kernel, lapisan pertama punya $5 \cdot 5 \cdot 6 + 6 = 156$ parameter, dan pooling 2×2 dengan stride dua mengecilkan peta fitur menjadi 14×14 .

Pooling mengambil maksimum atau rata-rata di jendela kecil. Ia mengecilkan peta fitur dan memberi sedikit ketahanan terhadap pergeseran kecil. *Receptive field* sebuah neuron adalah daerah input yang memengaruhinya, dan setiap lapisan konvolusi atau pooling memperluasnya. Lapisan awal melihat tepi dan tekstur, lapisan tengah melihat bagian objek, lapisan akhir melihat objek utuh. Kalau kernel lapisan pertama sebuah

jaringan terlatih divisualisasikan, hasilnya hampir selalu detektor tepi dan bercak warna, mirip sel-sel di korteks visual primer.

Karena konvolusi linear, backprop melaluinya juga berupa konvolusi. Gradien terhadap input adalah konvolusi delta dengan kernel yang dibalik, operasi yang disebut *transposed convolution*. Gradien terhadap kernel adalah korelasi antara input dan delta yang dijumlahkan atas semua posisi, karena kernel yang sama dipakai di mana-mana. Transposed convolution juga dipakai sebagai lapisan untuk memperbesar resolusi, misalnya di bagian decoder U-Net (Bab 12).

Sifat terpenting konvolusi adalah ini: kalau input digeser, keluarannya ikut bergeser dengan cara yang sama. Untuk operator geser T , berlaku $(Ta) * k = T(a * k)$. Sifat ini disebut *translation equivariance*. Pooling global di ujung jaringan mengubahnya menjadi *invariance*: label tidak berubah kalau objeknya digeser. Bab 13 akan menunjukkan bahwa konvolusi adalah satu-satunya operator linear yang punya sifat ini, lalu menggeneralisasikannya ke rotasi, permutasi, dan graf.

7.9 Konvolusi di mana-mana

Konvolusi jauh lebih tua dari jaringan saraf, dan hampir setiap bidang di buku ini punya versinya sendiri. Dalam pemrosesan sinyal, filter FIR yang meratakan atau menajamkan sinyal adalah konvolusi satu dimensi dengan kernel yang dirancang insinyur (Bab 18). Dalam pengolahan citra klasik, filter Gauss dan Sobel adalah kernel yang dirancang dengan tangan (Bab 11). Dalam genomik, jaringan konvolusi satu dimensi yang digeser sepanjang urutan DNA mempelajari motif pendek tempat protein menempel, persis seperti kaca pembesar yang mencari tanda tangan (Bab 22). Dalam sensor yang dikenakan, konvolusi pada sinyal akselerometer mempelajari pola gerak (Bab 20).

Dalam simulasi, hubungannya langsung terasa bagi siapa pun yang pernah menulis solver numerik. Kernel

$$k = \frac{1}{h^2} \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

adalah stensil lima titik untuk Laplacian $\nabla^2 u$ pada grid berjarak h . Kernel $\frac{1}{2h}(-1, 0, 1)$ adalah turunan pusat. Dengan kata lain, solver beda hingga pada grid teratur adalah jaringan konvolusi dengan kernel yang sudah ditentukan oleh fisika. Jaringan konvolusi yang dilatih pada data simulasi bisa dipandang sebagai solver yang mempelajari stensilnya sendiri. Pandangan ini menjelaskan mengapa CNN dan U-Net menjadi pilihan pertama untuk model pengganti pada grid teratur, dan mengapa mesh yang tidak teratur membutuhkan jaringan graf (Bab 13 dan 30).

7.10 Perkembangan riset

Bahkan blok paling dasar di bab ini masih dipertanyakan. Kolmogorov–Arnold Networks (Liu et al., 2024) terinspirasi oleh teorema representasi Kolmogorov–Arnold, yang menyatakan bahwa fungsi kontinu banyak variabel bisa ditulis sebagai jumlah

dan komposisi fungsi satu variabel. Alih-alih bobot linear di sisi dan aktivasi tetap di simpul, KAN memasang fungsi aktivasi yang dipelajari, berupa spline, di setiap sisi, dan tidak punya bobot linear sama sekali. Pada masalah fitting data dan PDE yang kecil, KAN yang jauh lebih kecil dilaporkan setara atau lebih akurat dari MLP yang lebih besar, dan fungsi-fungsi di sisinya bisa divisualisasikan, bahkan diganti rumus simbolik. Apakah keunggulan ini bertahan pada skala besar masih diperdebatkan.

Masalah praktis lain adalah inisialisasi dan laju belajar ketika lebar jaringan berubah. Analisis varians di bab ini menjaga sinyal maju dan mundur tetap hidup. Yang et al. (2022) melangkah lebih jauh dengan Maximal Update Parametrization: skala inisialisasi dan laju belajar setiap lapisan diatur sedemikian rupa sehingga besarnya pembaruan fitur tidak bergantung pada lebar. Akibatnya, hyperparameter terbaik dari model kecil bisa dipindahkan langsung ke model besar tanpa penyetelan ulang. Mereka melaporkan bahwa hyperparameter dari model 40 juta parameter, dipindahkan ke model 6,7 miliar parameter, mengungguli angka yang dipublikasikan untuk GPT-3 berukuran sama, dengan biaya penyetelan hanya sekitar tujuh persen dari biaya pretraining.

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Deep Learning and Neural Nets I, yang kini bernama Deep Learning: Basic Techniques: sejarah, notasi, arsitektur dasar, generalisasi, jaringan konvolusi, metode belajar, regularisasi, inisialisasi, dan normalisasi. Bacaan pendamping: Goodfellow et al. (2016), Prince (2023), dan ulasan LeCun et al. (2015).

8 Deep Learning dalam Praktik

Teori deep learning saya dapat di semester pertama. Cara melatih jaringan dengan benar, dan cara mencari tahu mengapa sebuah jaringan tidak mau belajar, lebih banyak datang dari latihan-latihan kuliah Deep Learning and Neural Networks II. Enam tugasnya bergerak dari dasar PyTorch, jaringan visi dan transfer learning, pemantauan eksperimen dan CNN lanjutan, autoencoder dan VAE, contoh adversarial dan GAN, sampai normalizing flows. Arsitektur-arsitekturnya sudah dibahas di Bab 12. Bab ini mengumpulkan sisi lainnya: apa yang terjadi di dalam kerangka kerja ketika kita menulis satu baris `loss.backward()`, dan kebiasaan kecil yang memisahkan eksperimen yang berhasil dari yang membuang waktu sehari-hari.

8.1 Graf komputasi dan autograd

Kuliah membuka dengan sejarah singkat kerangka kerja deep learning. Generasi awal seperti Theano dan TensorFlow 1 memakai graf statis: seluruh komputasi didefinisikan dulu sebagai graf simbolik, lalu dijalankan berkali-kali. PyTorch (Paszke et al., 2019) memakai graf dinamis, yang dibangun ulang setiap kali kode dijalankan (*define-by-run*), sehingga kontrol alur Python biasa, seperti percabangan dan perulangan, bisa dipakai di dalam model. TensorFlow 2 mengambil jalan serupa dengan *gradient tape* yang merekam operasi selama lintasan maju.

Di balik layar, setiap tensor yang dihasilkan operasi menyimpan rujukan ke operasi yang menghasilkannya dan ke input-input operasi itu. Ketika `backward()` dipanggil pada loss skalar, kerangka kerja menelusuri graf ini mundur dan menerapkan aturan rantai, persis reverse-mode autodiff di Bab 7. Tiga perilaku sering membingungkan pemula, dan ketiganya langsung masuk akal dari gambaran ini. Gradien *diakumulasikan* ke atribut `.grad`, tidak ditimpa, sehingga `optimizer.zero_grad()` harus dipanggil sebelum setiap lintasan mundur. Kalau lupa, langkah kedua memakai jumlah gradien langkah pertama dan kedua. Memanggil `.detach()` memotong graf, sehingga tidak ada gradien yang mengalir melewati tensor itu, berguna misalnya untuk target network di DQN (Bab 16). Dan blok `torch.no_grad()` menghentikan perekaman graf sama sekali, sehingga evaluasi tidak memakan memori untuk menyimpan aktivasi yang tidak akan diturunkan.

Satu putaran pelatihan, dengan urutan yang menjadi bahan soal di kuliah, berbentuk seperti ini:

```
model.train()
for x, y in loader:
    x, y = x.to(device), y.to(device)
    optimizer.zero_grad()      # bersihkan gradien lama
```

```

loss = loss_fn(model(x), y) # lintasan maju, graf direkam
loss.backward()             # lintasan mundur, gradien diisi
optimizer.step()            # perbarui parameter

```

Untuk evaluasi, `model.eval()` mematikan dropout dan membuat batch normalization memakai statistik bergerak, dan `torch.no_grad()` membungkus lintasan maju. Data dibaca oleh Dataset, yang tahu cara mengambil satu contoh, dan DataLoader, yang menyusunnya menjadi mini-batch, mengacak urutannya, dan memuatnya secara paralel dengan beberapa proses pekerja agar GPU tidak menunggu.

8.2 Resep men-debug jaringan

Jaringan yang tidak belajar jarang memberi pesan galat. Ia hanya menghasilkan kurva loss yang datar atau angka yang tidak masuk akal. Beberapa pemeriksaan murah menangkap sebagian besar masalah.

Pemeriksaan pertama adalah loss awal. Untuk klasifikasi K kelas dengan softmax dan cross-entropy, jaringan yang baru diinisialisasi seharusnya memberi peluang hampir seragam, $1/K$ untuk setiap kelas, sehingga loss awalnya mendekati $-\ln(1/K) = \ln K$. Untuk sepuluh kelas, itu sekitar 2,30. Kalau loss awal jauh lebih besar, misalnya 30, logit awalnya terlalu besar dan jaringan sudah yakin pada jawaban yang salah sebelum belajar apa pun, tanda inisialisasi atau normalisasi input yang keliru.

Pemeriksaan kedua adalah *overfit* satu batch kecil. Ambil beberapa contoh saja dan latih berulang-ulang pada contoh yang sama. Jaringan dengan kapasitas apa pun seharusnya bisa menghafalnya sampai loss mendekati nol. Kalau tidak bisa, masalahnya ada di kode, misalnya label yang tidak sesuai dengan input, gradien yang tidak mengalir karena detach yang salah tempat, atau laju belajar yang keliru, bukan di jumlah data atau arsitektur.

Pemeriksaan ketiga adalah laju belajar. Naikkan laju belajar secara eksponensial dari kecil ke amat besar selama beberapa ratus langkah, dan catat loss di setiap langkah. Loss mula-mula hampir tidak berubah, lalu turun cepat, lalu meledak. Laju belajar yang baik biasanya sedikit di bawah titik loss turun paling curam, jauh sebelum ledakan. Kurva ini menghubungkan praktik dengan teori di Bab 4: titik ledakan adalah tempat langkah melampaui batas $2/L$.

Pemeriksaan terakhir adalah melihat apa yang dilihat jaringan. Zeiler and Fergus (2014) memproyeksikan aktivasi lapisan dalam kembali ke ruang piksel dengan jaringan dekonvolusi, memakai indeks lokasi maksimum dari pooling untuk “membuka” pooling dan transposed convolution untuk membalik konvolusi. Lapisan awal ternyata merespons tepi dan warna, lapisan tengah tekstur, dan lapisan akhir bagian objek. Tugas kuliah meminta kami mereplikasi teknik ini dengan hanya mempertahankan beberapa aktivasi terbesar di setiap peta fitur. Gambar seperti ini sering langsung memperlihatkan bahwa jaringan memperhatikan latar belakang alih-alih objek, sama dengan pelajaran Grad-CAM di Bab 17.

8.3 Memori dan presisi

Ketika model atau batch tidak muat di memori GPU, dua teknik sering dipakai. *Gradient accumulation* memecah satu batch besar menjadi beberapa batch kecil, menjalankan lintasan maju dan mundur untuk masing-masing tanpa memanggil `optimizer.step()`, lalu memperbarui parameter sekali di akhir. Karena gradien diakumulasi, hasilnya sama dengan satu batch besar, asal loss setiap batch kecil dibagi dengan jumlah batch kecil. Batch 256 bisa dijalankan sebagai empat batch 64, dengan setiap loss dikalikan 1/4. Pengecualiannya adalah batch normalization, yang tetap melihat statistik batch kecil.

Mixed precision (Micikevicius et al., 2017) menghitung sebagian besar operasi dalam bilangan floating point 16 bit, yang memakan separuh memori dan jauh lebih cepat di GPU modern, sambil menyimpan salinan utama bobot dalam 32 bit. Masalahnya ada pada rentang angka. Format setengah presisi standar hanya bisa mewakili bilangan normal dari sekitar 6×10^{-5} sampai 65 504. Gradien yang lebih kecil dari batas bawah, yang umum di jaringan dalam, menjadi nol. Obatnya adalah *loss scaling*: kalikan loss dengan faktor besar, misalnya 1024, sebelum lintasan mundur, sehingga semua gradien ikut membesar 1024 kali dan keluar dari daerah yang hilang, lalu bagi kembali sebelum memperbarui bobot. Faktornya bisa disesuaikan otomatis: diperkecil kalau muncul angka tak hingga, diperbesar perlahan kalau tidak.

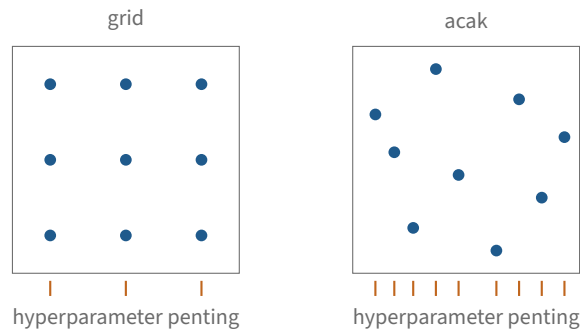
8.4 Mencari hyperparameter

Laju belajar, ukuran batch, weight decay, jumlah lapisan, dan peluang dropout harus dipilih sebelum pelatihan. Kuliah membahas beberapa strategi. Pencarian manual mengandalkan intuisi. Pencarian grid mencoba semua kombinasi dari beberapa nilai untuk setiap hyperparameter. Pencarian acak menarik kombinasi secara acak.

Bergstra and Bengio (2012) memberi argumen sederhana mengapa pencarian acak biasanya lebih baik dari grid. Misalkan dari dua hyperparameter hanya satu yang benar-benar berpengaruh, keadaan yang ternyata umum. Grid 3×3 menghabiskan sembilan percobaan tetapi hanya mencoba tiga nilai berbeda dari hyperparameter yang penting. Sembilan percobaan acak mencoba sembilan nilai berbeda (Gambar 8.1). Ada juga argumen peluang yang tidak bergantung pada dimensi. Kalau kita ingin setidaknya satu percobaan jatuh di lima persen konfigurasi terbaik, peluang n percobaan acak semuanya gagal adalah $0,95^n$. Dengan 60 percobaan, peluang berhasil $1 - 0,95^{60} \approx 95$ persen, berapa pun jumlah hyperparameter-nya.

Bayesian optimization (Snoek et al., 2012) memakai hasil percobaan sebelumnya untuk memilih percobaan berikutnya. Sebuah model pengganti, biasanya Gaussian process dari Bab 15, memodelkan skor validasi sebagai fungsi hyperparameter, lengkap dengan ketidakpastiannya. Titik berikutnya dipilih dengan memaksimalkan sebuah fungsi akuisisi. Yang paling umum adalah *expected improvement*: kalau skor terbaik sejauh ini f^* dan model meramal skor di titik x berdistribusi $\mathcal{N}(\mu, \sigma^2)$, perbaikan harapan atas f^* adalah

$$\text{EI}(x) = (\mu - f^*) \Phi(z) + \sigma \varphi(z), \quad z = \frac{\mu - f^*}{\sigma}, \quad (8.1)$$



GAMBAR 8.1. Sembilan percobaan dengan grid dan dengan pencarian acak. Kalau hanya sumbu mendatar yang berpengaruh, grid hanya menguji tiga nilainya, sedangkan pencarian acak menguji sembilan.

dengan Φ dan φ fungsi distribusi dan kerapatan normal baku. Suku pertama menghargai titik yang diramal bagus, suku kedua menghargai titik yang tidak pasti, keseimbangan eksplorasi dan eksploitasi yang sama dengan bandit di Bab 5. Dua titik dengan μ sama dengan f^* punya $z = 0$, sehingga $EI = \sigma\varphi(0) \approx 0,40\sigma$: titik yang lebih tidak pasti lebih menarik untuk dicoba. Neural architecture search memperluas gagasan ini ke struktur jaringan itu sendiri, dengan harga komputasi yang jauh lebih besar.

Untuk semua strategi, pemantauan eksperimen menjadi kebutuhan, bukan kemewahan. Kuliah memakai alat pencatat eksperimen seperti Weights & Biases: setiap percobaan mencatat hyperparameter, kurva loss latih dan validasi, dan versi kode. Tanpa catatan seperti itu, eksperimen terbaik minggu lalu tidak bisa diulang.

8.5 Transfer learning lebih dekat

Bab 12 menyebut resep dasar transfer learning: bekukan lapisan yang sudah dilatih, latih ulang kepalanya. Di PyTorch, membekukan berarti mematikan `requires_grad` pada parameter yang tidak ingin diubah, sehingga autograd tidak menghitung gradiennya dan optimizer tidak menyentuhnya. Gambar input juga harus disesuaikan dengan apa yang dilihat jaringan saat pretraining, misalnya diperbesar ke 224×224 dan dinormalisasi dengan rata-rata dan simpangan baku ImageNet.

Lapisan mana yang sebaiknya dibekukan? Yosinski et al. (2014) mengukur seberapa mudah fitur di setiap lapisan dipindahkan dari satu tugas ke tugas lain. Fitur lapisan awal bersifat umum dan hampir selalu bisa dipakai ulang. Fitur lapisan akhir semakin spesifik pada tugas asal. Mereka juga menemukan bahwa memindahkan fitur lalu menyetel ulang seluruh jaringan memberi generalisasi yang lebih baik daripada melatih dari nol, bahkan ketika data tugas baru cukup banyak. Dalam praktik, semakin sedikit data dan semakin mirip tugasnya, semakin banyak lapisan yang dibekukan. Semakin banyak data dan semakin berbeda tugasnya, semakin banyak lapisan yang disetel ulang, biasanya dengan laju belajar yang lebih kecil untuk lapisan awal.

8.6 Mengulang hasil

Hasil deep learning sulit diulang persis. Bobot awal, urutan data, dropout, dan augmentasi semuanya acak, dan beberapa operasi GPU tidak deterministik karena urutan penjumlahan floating point yang berubah-ubah memberi hasil yang sedikit berbeda. Menetapkan benih acak untuk semua sumber keacakan membuat satu proses bisa diulang. Tetapi benih yang berbeda tetap memberi hasil yang berbeda, kadang cukup jauh. Karena itu kebiasaan yang jujur adalah menjalankan beberapa benih dan melaporkan rata-rata beserta sebarannya. Selisih dua metode yang lebih kecil dari sebaran antar-benih belum bisa disebut perbaikan.

8.7 Di luar kelas

Kebiasaan di bab ini berlaku di semua bab penerapan. Pemeriksaan loss awal dan overfit satu batch sama bergunanya untuk PINN (Bab 29) dan simulator graf (Bab 30) seperti untuk klasifikasi gambar. Mixed precision memungkinkan model fondasi cuaca dan protein dilatih dalam waktu yang wajar. Bayesian optimization dipakai bukan hanya untuk hyperparameter jaringan, tetapi juga untuk mengkalibrasi parameter simulasi yang mahal, di mana setiap evaluasi berarti satu simulasi penuh. Dan laporan dengan beberapa benih dan pita sebaran adalah pengingat yang sama dengan error bar di Bab 2: angka tanpa ketidakpastian hanya separuh cerita.

CATATAN SUMBER. Bab ini mengikuti tugas dan materi kuliah Deep Learning and Neural Networks II (PyTorch dan autograd, jaringan visi dan transfer learning, pemantauan eksperimen dan pencarian hyperparameter, CNN lanjutan, transposed convolution dan visualisasi, autoencoder dan VAE, contoh adversarial dan GAN, normalizing flows). Makalah pendukung: Paszke et al. (2019), Bergstra and Bengio (2012), Snoek et al. (2012), Yosinski et al. (2014), Zeiler and Fergus (2014), dan Micikevicius et al. (2017).

9 Mengingat: dari RNN ke LSTM dan Transformer

Di semester pertama, kuliah LSTM diberikan langsung oleh Sepp Hochreiter, yang menganalisis masalah vanishing gradient pada 1991 dan bersama Jürgen Schmidhuber menerbitkan LSTM pada 1997. Kuliahnya tidak berhenti di sejarah. Bagian akhirnya membahas xLSTM, versi modern yang dikembangkan di Linz untuk bersaing dengan transformer. Setahun kemudian, kuliah NLP melengkapi sisi lain ceritanya: embedding kata dan model bahasa besar.

Banyak data datang berurutan: kata dalam kalimat, detak jantung, harga saham, langkah waktu sebuah simulasi. Untuk data seperti ini, model harus punya ingatan. Bab ini mengikuti evolusi ingatan buatan, dari jaringan rekuren sederhana yang cepat lupa, ke LSTM yang menjaga ingatan dengan gerbang, ke transformer yang melihat seluruh masa lalu sekaligus, lalu kembali lagi ke ingatan rekuren versi modern.

9.1 Jaringan rekuren

Jaringan feed-forward memproses setiap input sendiri-sendiri. *Recurrent neural network* (RNN) membawa sebuah keadaan tersembunyi $\mathbf{h}_t$ dari satu langkah ke langkah berikutnya:

$$\mathbf{h}_t = f(\mathbf{W}\mathbf{x}_t + \mathbf{R}\mathbf{h}_{t-1} + \mathbf{b}), \quad \hat{\mathbf{y}}_t = g(\mathbf{V}\mathbf{h}_t). \quad (9.1)$$

Matriks $\mathbf{R}$ adalah bobot rekuren. Bobot yang sama dipakai di setiap langkah waktu, persis seperti kernel konvolusi dipakai di setiap posisi gambar. Saat membaca novel, kita tidak mengingat setiap kata. Kita membawa ringkasan berjalan di kepala: siapa tokohnya, apa masalahnya, apa yang baru saja terjadi. Setiap kalimat baru memperbarui ringkasan itu. Keadaan tersembunyi $\mathbf{h}_t$ adalah ringkasan berjalan tersebut.

Kuliah membahas beberapa varian klasik. Jaringan Jordan mengumpankan kembali keluarannya, jaringan Elman mengumpankan kembali lapisan tersembunyinya, jaringan rekuren penuh menghubungkan semua unit, jaringan NARX memakai input dan keluaran yang tertunda, dan *time-delay neural network* hanya melihat jendela waktu yang tetap.

Untuk melatih RNN, *backpropagation through time* (BPTT) membuka jaringan menjadi jaringan feed-forward yang dalam, satu lapisan untuk setiap langkah waktu, lalu menjalankan backprop biasa. Karena bobotnya dipakai bersama, gradien untuk $\mathbf{R}$ adalah jumlah kontribusi dari semua langkah. Untuk deret yang panjang, BPTT dipotong setelah k langkah supaya memori dan waktu tetap terbatas. Alternatifnya adalah *real-time recurrent learning* (RTRL), yang membawa turunan keadaan terhadap semua bobot maju bersama waktu. Gradien tersedia di setiap langkah tanpa menyimpan masa lalu,

tetapi harganya mahal: untuk jaringan rekuren penuh dengan n unit, biayanya tumbuh seperti n^4 per langkah. Dalam bahasa Bab 7, RTRL adalah forward mode, sedangkan BPTT adalah reverse mode.

9.2 Gradien yang hilang dan meledak

Masalah berikut mengubah sejarah jaringan saraf. Delta di langkah $t - k$ bergantung pada delta di langkah t melalui perkalian k buah Jacobian:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{\tau=t-k+1}^t \frac{\partial \mathbf{h}_\tau}{\partial \mathbf{h}_{\tau-1}} = \prod_{\tau} \text{diag}(f'(s_\tau)) \mathbf{R}. \quad (9.2)$$

Kalau norma setiap faktor lebih kecil dari satu, hasil kalinya menyusut secara eksponensial. Inilah *vanishing gradient*. Kalau lebih besar dari satu, ia meledak. Hochreiter menganalisis gejala ini pada 1991 dalam tesis diplomasnya (Hochreiter, 1991), dan Bengio et al. (1994) menunjukkan secara independen bahwa ketergantungan jangka panjang memang sulit dipelajari dengan gradient descent.

Seberapa cepat gradien hilang? Ambil RNN skalar $h_t = \tanh(r h_{t-1} + w x_t)$ dengan $r = 0,5$. Karena turunan tanh tidak pernah melebihi satu, setiap langkah mengalikan gradien dengan paling banyak 0,5. Setelah dua puluh langkah, faktornya paling besar $0,5^{20}$, kurang dari satu per sejuta. Informasi dari dua puluh langkah sebelumnya praktis tidak lagi memengaruhi perubahan bobot. Dengan $r = 1,5$ dan unit yang bekerja di daerah linearnya, faktor yang sama bisa mencapai sekitar 3300.

Gradien yang meledak relatif mudah ditangani dengan *gradient clipping*: kalau norma gradien melewati batas tertentu, skalakan turun. Gradien yang hilang lebih sulit, karena tidak ada sinyal yang bisa diperbesar. Solusinya harus datang dari arsitektur.

9.3 LSTM

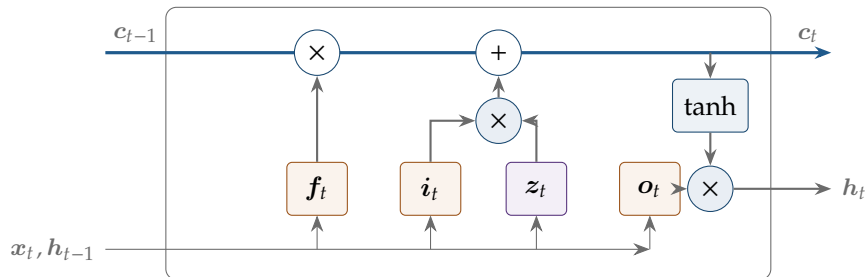
Hochreiter and Schmidhuber (1997a) memecahkan masalah ini dengan dua gagasan yang di kuliah disebut sebagai inti LSTM: *constant error carousel* dan *gating*.

Gagasan pertama sederhana. Bayangkan satu unit ingatan c_t yang diperbarui dengan penjumlahan, bukan perkalian, yaitu $c_t = c_{t-1} + (\text{sesuatu yang baru})$. Turunannya $\partial c_t / \partial c_{t-1}$ bernilai tepat satu. Error yang mengalir mundur lewat c tidak menyusut dan tidak membesar. Ia berputar tanpa berubah, seperti penumpang di komidi putar.

Penjumlahan tanpa kendali akan mengisi ingatan dengan sampah. Karena itu LSTM memberi gerbang yang dipelajari. Gerbang input memutuskan apa yang boleh masuk, dan gerbang output memutuskan apa yang boleh dibaca. Gerbang lupa (*forget gate*) ditambahkan kemudian oleh Gers et al. (2000) agar sel bisa mengosongkan dirinya. Versi yang kini dipakai di mana-mana berbentuk

$$\begin{aligned} z_t &= \tanh(\mathbf{W}_z \mathbf{x}_t + \mathbf{R}_z \mathbf{h}_{t-1} + \mathbf{b}_z), & i_t &= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{R}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \\ f_t &= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{R}_f \mathbf{h}_{t-1} + \mathbf{b}_f), & o_t &= \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{R}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \\ c_t &= f_t \odot c_{t-1} + i_t \odot z_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned} \quad (9.3)$$

Di sini z_t adalah input sel, i_t , f_t , dan o_t adalah gerbang input, lupa, dan output, dan c_t adalah keadaan sel. Sepanjang jalur sel, $\partial c_t / \partial c_{t-1} = f_t$. Selama gerbang lupa dekat satu, error mengalir hampir tanpa hambatan, dan jaringan sendiri yang memutuskan kapan harus melupakan. Gambar 9.1 menggambarkan aliran di dalam satu sel.



GAMBAR 9.1. Satu sel LSTM. Jalur atas (biru) adalah constant error carousel: keadaan sel hanya dikalikan gerbang lupa dan ditambah isi baru, sehingga error bisa mengalir mundur tanpa melewati nonlinearitas.

Satu langkah dengan angka membantu. Misalkan sel skalar menyimpan $c_{t-1} = 2$, dan pada langkah ini gerbang lupa $f_t = 0,9$, gerbang input $i_t = 0,1$, input sel $z_t = -1$, dan gerbang output $o_t = 0,5$. Keadaan baru $c_t = 0,9 \cdot 2 + 0,1 \cdot (-1) = 1,7$, dan keluaran $h_t = 0,5 \tanh(1,7) \approx 0,47$. Ingatan berubah sedikit saja. Kalau gerbang lupa tetap 0,9 selama dua puluh langkah tanpa input baru, sisa ingatan tinggal $0,9^{20} \approx 0,12$ dari semula, dan gradien sepanjang jalur sel menyusut dengan faktor yang sama. Dengan gerbang lupa 0,99, sisanya 0,82. Bandingkan dengan RNN di atas yang menyusut lebih dari sejuta kali dalam dua puluh langkah. Bedanya, faktor susutnya kini dipelajari jaringan sendiri dan bisa dibuat dekat ke satu.

Saya membayangkan keadaan sel sebagai buku catatan. Gerbang input adalah stabilo: hanya hal yang ditandai yang ditulis ke buku. Gerbang lupa adalah penghapus, karena kadang halaman lama perlu dibersihkan. Gerbang output adalah keputusan tentang halaman mana yang dibacakan sekarang. Tulisan di buku tidak memudar dengan sendirinya. Ia hanya berubah kalau ditulis atau dihapus.

Gambaran ini juga membantu menghindari beberapa salah paham. LSTM tidak menghilangkan semua masalah gradien. Jalur sel memang aman, tetapi jalur yang melewati gerbang dan h masih bisa mengalami vanishing. LSTM membuat belajar jangka panjang menjadi mungkin, bukan otomatis. Bias gerbang lupa juga sering diinisialisasi positif, misalnya satu, supaya di awal pelatihan sel cenderung mengingat. Dan pembagian peran tanh dan sigmoid bukan kebetulan: gerbang memakai sigmoid karena nilainya harus berada di antara nol dan satu, seperti keran, sedangkan input sel memakai tanh karena isinya boleh positif atau negatif.

LSTM mendominasi pemrosesan deret selama dua dekade, dari pengenalan suara di ponsel sampai terjemahan mesin. Kuliah juga menampilkan contoh di luar bahasa, seperti klasifikasi protein dari urutan asam amino dan analisis interval detak jantung.

9.4 Attention dan transformer

LSTM memampatkan seluruh masa lalu ke dalam vektor berukuran tetap. Untuk menerjemahkan kalimat yang panjang, pemampatan ini menjadi leher botol. Bahdanau et al. (2014) membiarkan dekoder melihat kembali semua keadaan enkoder dan memilih yang relevan. Mekanisme ini disebut *attention*. Transformer (Vaswani et al., 2017) kemudian membuang rekurensi sama sekali dan hanya memakai attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad Q = XW_Q, K = XW_K, V = XW_V. \quad (9.4)$$

Baris ke- i hasilnya adalah rata-rata berbobot dari semua *value*, dengan bobot dari kemiripan *query* ke- i dengan setiap *key*.

Perpustakaan adalah gambaran yang pas. Anda datang dengan sebuah pertanyaan, yaitu query. Setiap buku punya label di punggungnya, yaitu key, dan isi, yaitu value. Anda membandingkan pertanyaan dengan semua label, lalu membaca isi buku dengan perhatian yang sebanding dengan kecocokannya. Buku yang labelnya cocok sekali dibaca serius, yang tidak cocok hanya dilirik. Jawaban Anda adalah campuran dari isi buku-buku itu.

Sebagai contoh kecil, ambil query $q = (1, 0)$, dua key $k_1 = (1, 0)$ dan $k_2 = (0, 1)$, value $v_1 = 10$ dan $v_2 = 0$, serta $d_k = 2$. Skornya $q \cdot k_1 / \sqrt{2} \approx 0,707$ dan 0. Softmax mengubahnya menjadi bobot 0,670 dan 0,330, karena $e^{0,707} \approx 2,028$ dibagi $2,028 + 1$. Keluarannya 6,70. Pembagian dengan $\sqrt{d_k}$ menjaga skor tidak terlalu besar saat dimensinya tinggi, supaya softmax tidak jenuh menjadi pilihan tunggal yang keras.

Mengapa tepat $\sqrt{d_k}$? Anggap komponen q_i dan k_i saling independen dengan rata-rata nol dan varians satu. Hasil kali titik $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ adalah jumlah d_k suku yang masing-masing punya rata-rata nol dan varians $\mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 1$, sehingga variansnya d_k dan simpangan bakunya $\sqrt{d_k}$. Untuk $d_k = 64$, skor tipikal berukuran sekitar delapan. Selisih delapan pada input softmax berarti rasio bobot $e^8 \approx 3000$, sehingga hampir semua bobot jatuh ke satu kunci dan gradien ke kunci-kunci lain praktis nol. Membagi dengan $\sqrt{d_k}$ mengembalikan simpangan baku skor ke satu, berapa pun dimensinya.

Satu blok transformer berisi *multi-head attention*, yaitu beberapa attention paralel dengan proyeksi yang berbeda yang hasilnya digabung, lalu jaringan feed-forward yang bekerja per posisi. Keduanya dibungkus koneksi residual dan layer normalization. Karena attention tidak peduli urutan, informasi posisi ditambahkan lewat *positional encoding*. Versi asli memakai gelombang sinus dan kosinus dengan frekuensi berbeda untuk setiap dimensi, $\text{PE}(p, 2i) = \sin(p/10000^{2i/d})$ dan $\text{PE}(p, 2i+1) = \cos(p/10000^{2i/d})$, seperti jarum-jarum jam yang berputar dengan kecepatan berbeda: jarum cepat membedakan posisi yang bersebelahan, jarum lambat membedakan posisi yang berjauhan. Karena $\sin(a+b)$ dan $\cos(a+b)$ adalah kombinasi linear dari $\sin a$ dan $\cos a$, posisi $p+k$ bisa diperoleh dari posisi p dengan sebuah rotasi yang hanya bergantung pada k , sehingga attention mudah mempelajari posisi relatif. Banyak model sekarang memakai gagasan ini secara langsung dengan memutar query dan key menurut posisinya. Untuk model bahasa yang menulis dari kiri ke kanan, sebuah *causal mask* mencegah setiap posisi melihat masa depan.

Kelebihan transformer adalah semua posisi diproses paralel selama pelatihan, dan dua token mana pun hanya berjarak satu langkah attention. Kekurangannya, yang ditekankan di kuliah, adalah biaya attention yang tumbuh kuadratik terhadap panjang deret, serta keharusan menyimpan key dan value dari semua token sebelumnya ketika model membangkitkan teks.

Ramsauer et al. (2020), juga dari Linz, menunjukkan bahwa aturan update jaringan Hopfield modern dengan keadaan kontinu,

$$\xi^{\text{baru}} = X \text{softmax}(\beta X^\top \xi), \quad (9.5)$$

sama dengan attention. Kolom-kolom X adalah pola yang disimpan dan ξ adalah query. Biasanya satu langkah update sudah cukup untuk mengambil pola yang paling mirip. Dari sudut pandang ini, attention adalah cara membaca dari sebuah ingatan asosiatif yang kapasitasnya sangat besar.

9.5 Dari bahasa ke vektor

Agar bisa masuk ke jaringan rekuren atau transformer, kata harus lebih dulu diubah menjadi vektor. Jalan dari kantong kata, model bahasa n -gram, dan perplexity, ke word embedding seperti word2vec (Mikolov et al., 2013) dan GloVe, sampai model bahasa besar seperti BERT (Devlin et al., 2018) dan GPT (Brown et al., 2020), dibahas tersendiri di Bab 10. Untuk bab ini cukup satu gambaran: setiap token menjadi vektor padat berdimensi beberapa ratus, kata-kata yang muncul dalam konteks serupa mendapat vektor yang berdekatan, dan deret vektor itulah yang diproses oleh arsitektur-arsitektur di atas.

9.6 Rekurensi yang kembali

Biaya kuadratik transformer mendorong pencarian arsitektur dengan biaya linear. *State space model* memakai rekurensi linear

$$h_t = \bar{A} h_{t-1} + \bar{B} x_t, \quad y_t = C h_t, \quad (9.6)$$

yang bisa dihitung paralel justru karena linear. Mamba (Gu and Dao, 2023) membuat $\bar{B}$, C , dan langkah diskretisasinya bergantung pada input, sehingga model bisa memutuskan apa yang disimpan dan apa yang diabaikan, mirip gerbang LSTM. Rekurensi ini tidak lain adalah diskretisasi persamaan diferensial biasa yang linear, $\dot{h} = Ah + Bx$, sebuah objek yang akan kita temui lagi di Bab 15 dan 31.

9.7 xLSTM: LSTM untuk era model bahasa besar

9.8 xLSTM: LSTM untuk era model bahasa besar

Kuliah ditutup dengan xLSTM (Beck et al., 2024), yang dikembangkan di Linz pada 2024. Pertanyaan yang diajukan slide terakhir kuliah sederhana: seberapa jauh LSTM bisa melangkah kalau diperbesar ke miliaran parameter dengan teknik model bahasa

modern, dan kalau keterbatasan-keterbatasannya yang sudah dikenal diperbaiki? Slide menyebut tiga keterbatasan itu secara terang-terangan, masing-masing dengan satu tugas uji yang memperlihatkankannya.

9.8.1 Tiga keterbatasan LSTM

Keterbatasan pertama adalah ketidakmampuan merevisi keputusan penyimpanan. Tugas ujinya adalah pencarian tetangga terdekat: diberikan sebuah vektor acuan, baca deret vektor satu per satu, dan di akhir deret keluarkan nilai yang melekat pada vektor yang paling mirip dengan acuan. Setiap kali datang vektor yang lebih mirip, isi ingatan harus diganti seluruhnya. Gerbang input sigmoid paling besar bernilai satu, sehingga isi baru paling banyak bisa ditambahkan dengan bobot yang sama dengan isi lama, dan menghapus isi lama membutuhkan gerbang lupa yang tepat nol pada saat yang tepat.

Keterbatasan kedua adalah kapasitas penyimpanan. Satu sel LSTM menyimpan satu skalar. Tugas ujinya adalah prediksi token yang jarang muncul. Slide memperlihatkan perplexity pada Wikitext-103 yang dipisah menurut frekuensi token: token yang jarang tidak bisa dipelajari dari data latih karena terlalu jarang dilihat, sehingga harus diingat dari konteks, dan di situlah LSTM tertinggal. Keterbatasan ketiga adalah pelatihan yang tidak bisa diparalelkan, karena setiap langkah bergantung pada h_{t-1} melalui bobot rekuren.

9.8.2 sLSTM: exponential gating

Jawaban untuk keterbatasan pertama adalah *exponential gating*. Gerbang input menjadi $i_t = \exp(\tilde{i}_t)$, sehingga bisa bernilai jauh lebih besar dari satu, dan sebuah keadaan normalisasi mengikuti jumlah semua bobot yang pernah masuk:

$$c_t = f_t \odot c_{t-1} + i_t \odot z_t, \quad n_t = f_t \odot n_{t-1} + i_t, \quad h_t = o_t \odot \frac{c_t}{n_t}. \quad (9.7)$$

Keluaran adalah rata-rata berbobot dari semua input yang pernah disimpan. Contoh dengan angka memperlihatkan efeknya. Misalkan sel menyimpan nilai v_1 dengan bobot satu, jadi $c = v_1$ dan $n = 1$. Datang vektor yang jauh lebih cocok, dan jaringan memberi $\tilde{i} = 5$, sehingga $i = e^5 \approx 148$, dengan gerbang lupa satu. Maka $c = v_1 + 148 v_2$ dan $n = 149$, dan keluarannya $c/n \approx 0,007 v_1 + 0,993 v_2$. Isi lama praktis tertimpa tanpa perlu dihapus lebih dulu. Dengan gerbang sigmoid, yang paling besar satu, hasil terbaik dalam situasi yang sama adalah campuran setengah-setengah.

Eksponensial mudah meluap, sehingga xLSTM menambahkan keadaan penstabil $m_t = \max(\log f_t + m_{t-1}, \log i_t)$ dan memakai $\exp(\tilde{i}_t - m_t)$ serta $\exp(\log f_t + m_{t-1} - m_t)$ sebagai pengganti gerbang aslinya. Karena faktor e^{-m_t} muncul di pembilang dan penyebut c_t/n_t , keluarannya tidak berubah, hanya angkanya tetap dalam rentang yang aman. Varian ini disebut sLSTM. Ia tetap punya koneksi rekuren dari h_{t-1} , yang di slide disebut *memory mixing*: sel-sel saling memengaruhi lewat bobot rekuren, dibagi menjadi beberapa kepala dengan matriks blok-diagonal. Memory mixing inilah yang membuat sLSTM mampu melacak keadaan, tetapi juga yang membuatnya tidak bisa diparalelkan. Kapasitas ingatannya, menurut slide, sebanding dengan m/d untuk m sel dan dimensi tersembunyi d , dan tidak bergantung pada panjang deret.

9.8.3 mLSTM: ingatan berbentuk matriks

Jawaban untuk keterbatasan kedua dan ketiga adalah mLSTM, yang mengganti sel skalar dengan sebuah matriks dan memakai aturan update kovarians:

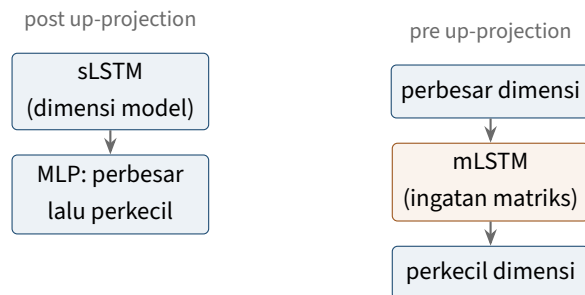
$$C_t = f_t C_{t-1} + i_t v_t k_t^\top, \quad n_t = f_t n_{t-1} + i_t k_t, \quad \tilde{h}_t = \frac{C_t q_t}{\max\{|n_t^\top q_t|, 1\}}. \quad (9.8)$$

Bentuknya adalah ingatan asosiatif: setiap langkah menyimpan pasangan key dan value sebagai matriks berpangkat satu, dan membaca berarti mengalikan matriks dengan query. Kalau key yang tersimpan kira-kira saling ortogonal dan ternormalisasi, query yang sama dengan key k_j mengambil kembali $C k_j \approx i_j v_j$, karena sumbangan pasangan lain hampir nol. Slide menyebut kapasitas matriks $d \times d$ ini sekitar $0,14 d$ pasangan, angka yang sama dengan kapasitas klasik jaringan Hopfield, dan sekali lagi tidak bergantung pada panjang deret. Tugas ujinya adalah *multi-query associative recall*: deret berisi pasangan key dan value acak dari kosakata besar, lalu di akhir model ditanya value dari beberapa key sekaligus.

Pada mLSTM, gerbang, key, value, dan query dihitung dari input saja, tidak dari h_{t-1} . Tanpa memory mixing, seluruh deret bisa diproses paralel selama pelatihan seperti attention linear, dan dijalankan secara rekuren saat inferensi. Harganya, mLSTM kehilangan kemampuan melacak keadaan yang dimiliki sLSTM.

9.8.4 Blok, arsitektur, dan hasil

Kedua sel dibungkus dalam blok residual yang berbeda (Gambar 9.2). sLSTM memakai blok *post up-projection* yang dikenal dari transformer: sel bekerja di dimensi model, lalu jaringan feed-forward memperbesar dan mengecilkan dimensinya. mLSTM memakai blok *pre up-projection* yang dikenal dari state space model: input diperbesar lebih dulu agar ingatan matriksnya berdimensi tinggi, dengan konvolusi pendek untuk query dan key, aktivasi Swish, gerbang output eksternal, dan koneksi lompat yang dipelajari. Arsitektur xLSTM menumpuk blok-blok ini secara residual, dan notasi xLSTM[$a:b$] menyatakan perbandingan jumlah blok mLSTM terhadap sLSTM.



GAMBAR 9.2. Dua jenis blok xLSTM. Blok sLSTM mengikuti pola transformer, dengan pemrosesan di dimensi model lalu MLP. Blok mLSTM mengikuti pola state space model: dimensi diperbesar dulu agar ingatan matriksnya lapang. Keduanya ditumpuk dengan koneksi residual.

Hasil yang dilaporkan di slide mencakup empat hal. Pada bahasa formal yang membutuhkan pelacakan keadaan, model dengan sLSTM mampu menyelesaikan tugas

yang gagal dikerjakan transformer dan state space model. Pada multi-query associative recall, mLSTM mengingat jauh lebih banyak pasangan dari model rekuren lain. Pada pemodelan bahasa dengan 300 miliar token SlimPajama, xLSTM bersaing dengan transformer dan state space model berukuran sama, dan model 1,3 miliar parameter yang dilatih dengan konteks 2048 token tetap stabil ketika diuji sampai 16 384 token. Kurva hukum penskalaannya (Bab 38) menunjukkan bahwa keunggulan itu bertahan ketika model diperbesar. Slide mencatat bahwa model 2,7 miliar parameter dengan perbandingan 7:1 masih kurang dilatih, karena konvergensinya awalnya lebih lambat.

Slide juga jujur tentang keterbatasan xLSTM sendiri. Memory mixing membuat sLSTM tidak bisa diparalelkan, dan kernel CUDA cepatnya masih sekitar 1,5 kali lebih lambat dari mLSTM paralel. Kernel mLSTM saat itu belum dioptimalkan, sekitar empat kali lebih lambat dari FlashAttention atau scan Mamba. Ingatan matriks mahal secara komputasi. Inisialisasi gerbang lupa harus dipilih hati-hati. Kapasitas ingatan terbatas pada $d \times d$, meski tidak bermasalah sampai konteks 16 ribu token. Dan arsitektur serta hyperparameternya belum dioptimalkan. Rencana yang ditulis di slide penutup adalah membangun model yang lebih besar, mencari bidang penerapan tempat xLSTM unggul, dan menjelajahi gagasan-gagasan yang belum diuji.

9.8.5 Sesudah kuliah

Rencana di slide terakhir itu kemudian berjalan. Vision-LSTM (Alkin et al., 2024a) memakai tumpukan blok mLSTM sebagai tulang punggung visi komputer. Gambar dipotong menjadi patch seperti pada vision transformer, dan karena xLSTM membaca dari satu arah, blok ganjil membaca patch baris demi baris dari kiri atas ke kanan bawah, sedangkan blok genap membaca dari kanan bawah ke kiri atas. Biayanya linear terhadap jumlah patch, sehingga menarik untuk gambar beresolusi tinggi seperti citra medis (Bab 25) dan medan simulasi fisika (Bab 30).

xLSTM 7B (Beck et al., 2025) memperbesar arsitekturnya menjadi model bahasa tujuh miliar parameter dengan optimasi khusus untuk inferensi. Kinerjanya pada tugas-tugas hilir dilaporkan sebanding dengan model lain berukuran sama, dengan inferensi yang lebih cepat dan lebih hemat daripada model berbasis Llama dan Mamba. Alasannya terletak pada struktur yang sudah kita bahas: ingatannya berukuran tetap, sehingga tidak ada cache key dan value yang membengkak seiring panjang konteks. Sifat ini penting justru di era model yang “berpikir” panjang saat menjawab (Bab 38), ketika jumlah token yang dibangkitkan, bukan ukuran model, menjadi biaya utama.

Bidang penerapan lain yang menonjol adalah peramalan deret waktu. TiRex (Auer et al., 2025) memakai xLSTM sebagai model peramalan *zero-shot*: nilai-nilai masa lalu sebuah deret dimasukkan sebagai konteks, dan model meramal masa depannya tanpa dilatih ulang untuk deret itu, mirip in-context learning pada model bahasa. Argumen makalahnya menarik benang dari awal bab ini. Transformer dan model rekuren yang bisa diparalelkan unggul dalam in-context learning, tetapi kehilangan kemampuan melacak keadaan, sedangkan LSTM klasik melacak keadaan dengan baik tetapi lemah dalam in-context learning. xLSTM, dengan sLSTM untuk pelacakan keadaan, memberi keduanya. Pada tolok ukur GiftEval dan Chronos-ZS, TiRex dilaporkan mengungguli

model peramalan yang jauh lebih besar, baik untuk ramalan jangka pendek maupun jangka panjang.

9.9 Deret di luar bahasa

Model deret yang lahir dari bahasa dipakai jauh di luar teks. Kuliah LSTM sendiri sudah menunjukkan klasifikasi protein dari urutan asam amino dan analisis detak jantung. Dalam genomik, urutan DNA dibaca oleh model yang sama dengan model bahasa, dan base calling pada pengurutan nanopore memakai jaringan rekuren untuk mengubah sinyal arus listrik menjadi huruf (Bab 22). Dalam kimia, string SMILES dibangkitkan huruf demi huruf oleh LSTM untuk merancang molekul baru (Bab 23). Dalam sensor yang dikenakan, deret akselerometer dikenali sebagai aktivitas (Bab 20), dan di sistem rekomendasi, urutan lagu yang baru didengar menjadi konteks untuk lagu berikutnya (Bab 34).

Simulasi yang berjalan dalam waktu juga sebuah deret: keadaan medan di langkah t menentukan keadaan di langkah $t + 1$. Model deret dipakai di sini dengan beberapa cara. Yang paling umum adalah memproyeksikan medan aliran ke beberapa mode POD (Bab 31), lalu membiarkan LSTM atau transformer mempelajari evolusi koefisien mode-mode itu, pendekatan yang dibahas dalam ulasan Brunton et al. (2020). Ingatan juga berguna ketika sensor hanya mengukur sebagian keadaan, karena masa lalu membantu menebak bagian yang tersembunyi. Transformer bahkan tidak harus berjalan sepanjang waktu. Ia bisa memperlakukan titik-titik mesh atau partikel sebagai token. Universal Physics Transformers (Alkin et al., 2024b), dari kelompok riset di Linz, memakai gagasan ini untuk neural operator yang tidak terikat pada grid (Bab 30). Di semua pendekatan itu, masalah yang selalu muncul adalah galat yang menumpuk ketika model menebak berulang-ulang dari tebakannya sendiri. Masalah ini kita bahas di Bab 30.

CATATAN SUMBER. Bab ini mengikuti slide LSTM and Recurrent Neural Nets dari Sepp Hochreiter (RNN, BPTT, RTRL, vanishing gradient, LSTM, attention, Mamba, xLSTM) dan slide Natural Language Processing (representasi kata, n-gram, embedding, model bahasa besar). Bacaan utama: Hochreiter and Schmidhuber (1997a), Vaswani et al. (2017), Beck et al. (2024), dan Gu and Dao (2023).

10 Bahasa: dari Kantong Kata ke Model Bahasa Besar

Kuliah natural language processing saya ikuti pada musim dingin 2025/26 di Institute of Computational Perception. Sepuluh pertemuannya bergerak dari pra-pemrosesan teks dan representasi dokumen, representasi kata dan embedding, jaringan saraf, model bahasa n -gram, embedding kata dan kalimat yang dipelajari, attention dan transformer, NLP multimodal, sampai model bahasa besar. Tiga tugas kelompoknya mengikuti alur yang sama: mengklasifikasikan dokumen dengan metode machine learning standar, menjelajahi word embedding, lalu mengklasifikasikan cerita pendek ke banyak label sekaligus dengan PyTorch dan BERT.

Transformer dan model bahasa besar sudah muncul di Bab 9 dan Bab 38. Bab ini mengisi jalan panjang di antaranya: bagaimana teks diubah menjadi angka, dan bagaimana gagasan-gagasan sederhana tentang menghitung kata perlahan tumbuh menjadi model yang kini dipakai jutaan orang.

10.1 Membersihkan teks

Sebelum apa pun, teks harus diseragamkan. Huruf besar dikecilkan, sehingga “Linz” dan “linz” menjadi satu kata, walau kadang dengan harga ambiguitas, seperti *General Motors* dan *general motors*. Angka dan tanggal sering diganti token khusus, karena kalau setiap angka menjadi kata tersendiri, kosakata meledak. Kata majemuk dalam bahasa seperti Jerman perlu dipecah, *Halsschlagader* menjadi leher, pukulan, dan pembuluh. Kata-kata yang umum, *stop words* seperti “yang”, “dan”, “di”, sering dibuang untuk pendekatan kantong kata, walau dalam frasa seperti “to be or not to be” justru merekalah isinya.

Variasi bentuk kata dikurangi dengan dua cara. *Stemming* memotong imbuhan dengan aturan kasar. Algoritme Porter (Porter, 1980), yang paling umum untuk bahasa Inggris, memakai aturan seperti *sses* menjadi *ss*, *ies* menjadi *i*, dan *ational* menjadi *ate*, sehingga *caresses* menjadi *caress*, *ponies* menjadi *poni*, dan *relational* menjadi *relate*. Hasilnya tidak harus kata yang sungguhan. *Lemmatization* memakai kamus untuk mengembalikan bentuk dasar yang benar, misalnya *am*, *are*, dan *is* menjadi *be*. Lebih teliti, tetapi lebih lambat, dan tidak berdaya untuk kata yang tidak ada di kamus. Bahasa Indonesia, dengan imbuhan seperti *me-*, *-kan*, dan *per-an*, menghadapi masalah yang sama dalam bentuk yang lebih kaya.

10.2 Kantong kata

Representasi paling sederhana untuk sebuah dokumen adalah *bag of words*: vektor sepanjang kosakata yang berisi berapa kali setiap kata muncul, dengan urutan dibuang sama sekali. “Anjing menggigit orang” dan “orang menggigit anjing” menjadi vektor yang sama. Meski kasar, representasi ini, ditambah pembobotan TF-IDF dari Bab 34, sudah cukup untuk banyak tugas klasifikasi dokumen. Tugas pertama kuliah membangun pengklasifikasi sentimen seperti ini dengan regresi logistik dan pengklasifikasi standar dari Bab 6.

Masalah utamanya adalah *sparsity*. Kosakata berisi puluhan ribu kata, sedangkan satu kalimat hanya memakai belasan, sehingga vektornya hampir seluruhnya nol. Lebih parah lagi, setiap kata adalah dimensi yang berdiri sendiri. “Bagus” dan “baik” sama jauhnya dengan “bagus” dan “buruk”. Model tidak bisa memindahkan apa yang ia pelajari tentang satu kata ke kata yang bermakna serupa.

10.3 Makna dari tetangga

Jalan keluarnya datang dari linguistik. Firth menulis pada 1957 bahwa kita mengenal sebuah kata dari teman-temannya. Kuliah memakai contoh kata yang hampir tidak dikenal, *tesgüino*. Dari kalimat-kalimat di sekitarnya, bahwa ia diminum, difermentasi dari jagung, dan dibuat untuk upacara, kita menyimpulkan bahwa ia sejenis minuman beralkohol, tanpa pernah melihat definisinya. Dua kata bermakna mirip kalau mereka sering muncul di tengah kata-kata yang sama. Inilah *distributional semantics*.

Gagasan ini dihitung dengan matriks kata-konteks: baris untuk setiap kata, kolom untuk setiap kata konteks, dan isinya berapa kali keduanya muncul berdekatan, misalnya dalam jendela beberapa kata. Hitungan mentah didominasi kata-kata umum, sehingga diganti dengan *pointwise mutual information*,

$$\text{PMI}(w, c) = \log_2 \frac{p(w, c)}{p(w)p(c)}, \quad (10.1)$$

yang bertanya seberapa sering dua kata muncul bersama dibanding yang diharapkan kalau keduanya independen. Misalkan dalam seribu pasangan kata-konteks, pasangan “kopi”–“seduh” muncul 10 kali, “kopi” muncul 50 kali sebagai kata, dan “seduh” 40 kali sebagai konteks. Peluang bersamanya 0,01, sedangkan hasil kali peluang terpisahnya $0,05 \times 0,04 = 0,002$. PMI-nya $\log_2 5 \approx 2,3$: kedua kata muncul bersama lima kali lebih sering dari kebetulan. Nilai negatif sulit ditaksir dengan andal dari data terbatas, sehingga biasanya dipotong ke nol, *positive PMI*. Kuliah juga mencatat kelemahan PMI: ia terlalu menghargai kata yang jarang, karena satu kebetulan saja bisa memberi rasio yang besar.

Matriks PPMI lebar sekali dan jarang. Memampatkannya dengan SVD (Lampiran B) dan mengambil beberapa ratus dimensi pertama memberi vektor padat untuk setiap kata, sebuah *word embedding*. GloVe (Pennington et al., 2014) merumuskan hal serupa sebagai masalah regresi: cari vektor kata w_i dan vektor konteks $\tilde{w}_j$ beserta bias sehingga

$$w_i^\top \tilde{w}_j + b_i + \tilde{b}_j \approx \log X_{ij}, \quad (10.2)$$

dengan X_{ij} hitungan kemunculan bersama, diminimalkan dengan galat kuadrat berbobot. Bobotnya meredam pasangan yang sangat sering dan mengabaikan pasangan yang tidak pernah muncul. Hasil kali dalam yang mendekati logaritma hitungan bersama, setelah bias memperhitungkan frekuensi masing-masing kata, pada dasarnya adalah PMI. Dari sinilah sifat analogi yang terkenal muncul: vektor dari “pria” ke “wanita” kira-kira sejajar dengan vektor dari “raja” ke “ratu”. Tugas kedua kuliah menjelajahi sifat-sifat ini: kemiripan kosinus, tetangga terdekat, dan analogi.

10.4 Model bahasa

Model bahasa memberi peluang pada deret kata. Dengan aturan rantai, peluang sebuah kalimat adalah hasil kali peluang setiap kata diberikan kata-kata sebelumnya. Model n -gram menyederhanakannya dengan asumsi Markov: setiap kata hanya bergantung pada $n - 1$ kata sebelumnya, dan peluangnya ditaksir dari hitungan. Di Bab 9 kita melihat contoh bigram “nasi goreng”: kalau “nasi” muncul 200 kali dan “nasi goreng” 50 kali, peluangnya 0,25.

Masalahnya adalah pasangan yang tidak pernah muncul di korpus, yang mendapat peluang nol, sehingga seluruh kalimat yang memuatnya juga berpeluang nol. *Laplace smoothing* menambahkan satu ke setiap hitungan,

$$P_{\text{Lap}}(w_t | w_{t-1}) = \frac{\text{hitung}(w_{t-1}, w_t) + 1}{\text{hitung}(w_{t-1}) + V}, \quad (10.3)$$

dengan V ukuran kosakata. Untuk kosakata sepuluh ribu kata, peluang “goreng” setelah “nasi” turun dari 0,25 menjadi $51/10\,200 = 0,005$. Penghalusan ini terlalu kasar: hampir seluruh massa peluang dipindahkan ke sembilan ribu lebih kelanjutan yang tidak pernah terlihat. Cara yang lebih baik adalah *backoff*, yang turun ke n -gram yang lebih pendek kalau hitungan n -gram panjang tidak ada, atau *interpolation*, yang mencampur taksiran bigram dan unigram dengan bobot λ .

Kinerja model bahasa diukur dengan *perplexity*, kebalikan rata-rata geometris peluang per kata pada data uji, $PP = P(w_1, \dots, w_N)^{-1/N}$. Kalau model memberi peluang 0,1, 0,2, dan 0,05 pada tiga kata sebuah kalimat, hasil kalinya 0,001, rata-rata geometrisnya 0,1, dan perplexity-nya 10: model sama bingungnya dengan memilih seragam di antara sepuluh kata. Model yang menebak seragam dari seluruh kosakata punya perplexity V . Perplexity adalah eksponen dari cross-entropy rata-rata, sehingga menurunkannya sama dengan menurunkan loss pelatihan di Bab 1.

Model bahasa neural (Bengio et al., 2003) mengganti tabel hitungan dengan jaringan. Setiap kata konteks dipetakan ke embedding, embedding-embedding itu digabung, dilewatkan ke lapisan tersembunyi, lalu ke softmax atas seluruh kosakata. Dengan kosakata sepuluh ribu, embedding berdimensi 100, tiga kata konteks, dan 200 neuron tersembunyi, tabel embedding berisi satu juta parameter, lapisan tersembunyi $300 \times 200 = 60\,000$, dan lapisan keluaran $200 \times 10\,000 = 2$ juta. Lapisan keluaran yang paling mahal, dan masalah yang sama muncul lagi pada model besar. Keuntungannya, kata-kata yang mirip mendapat embedding yang mirip, sehingga apa yang dipelajari dari “kopi panas” ikut membantu “teh panas”, sesuatu yang tidak bisa dilakukan tabel hitungan.

Loss-nya adalah negative log likelihood setelah softmax, dan kuliah menurunkan gradiennya terhadap logit: $\mathbf{p} - \mathbf{y}$, peluang yang diramalkan dikurangi label one-hot, bentuk yang sama dengan regresi logistik di Bab 1. Softmax menciptakan persaingan di antara semua kata. Menaikkan logit satu kata otomatis menurunkan peluang semua kata lain.

10.5 word2vec dan embedding kalimat

word2vec (Mikolov et al., 2013) menyederhanakan model bahasa neural menjadi tugas yang hanya bertujuan menghasilkan embedding. Skip-gram menebak kata-kata konteks dari satu kata pusat. Softmax atas seluruh kosakata terlalu mahal, sehingga dipakai *negative sampling*: tugas klasifikasi banyak kelas diubah menjadi klasifikasi biner. Untuk pasangan sungguhan antara kata pusat c dan kata konteks o , serta k kata acak $n_1, \dots, n_k$ yang ditarik dari distribusi derau,

$$\mathcal{L} = -\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) - \sum_{i=1}^k \log \sigma(-\mathbf{u}_{n_i}^\top \mathbf{v}_c). \quad (10.4)$$

Suku pertama menaikkan skor pasangan sungguhan, suku kedua menurunkan skor pasangan palsu. Kalau skor pasangan sungguhan 2, suku pertamanya $-\ln \sigma(2) \approx 0,13$. Kalau satu kata derau punya skor 1, sumbangannya $-\ln \sigma(-1) \approx 1,31$, hukuman besar yang mendorong skor itu turun. Kata-kata derau biasanya ditarik dengan peluang sebanding dengan frekuensinya dipangkatkan $3/4$, agar kata yang umum tidak terlalu mendominasi dan kata langka sesekali ikut terpilih.

Embedding kata belum tentu menjadi embedding kalimat yang baik. Merata-ratakan vektor kata dalam kalimat, cara paling sederhana, membuang urutan dan memberi bobot yang sama pada kata “yang” dan kata “gempa”. Pembobotan dengan TF-IDF memperbaikinya sedikit. Kuliah membahas cara yang lebih baik, yaitu melatih model langsung dengan tujuan tingkat kalimat, misalnya meramalkan kata yang dihilangkan dari kalimat memakai representasi seluruh kalimat lainnya, sehingga embedding yang dihasilkan memang dirancang untuk mewakili kalimat utuh.

10.6 Model bahasa besar

Kuliah membagi model bahasa besar yang dilatih sebelumnya ke dalam tiga keluarga. Model encoder seperti BERT (Devlin et al., 2018) melihat seluruh kalimat sekaligus dan menghasilkan embedding kontekstual: kata “bank” di “bank sungai” dan di “bank sentral” mendapat vektor yang berbeda. Model decoder seperti GPT (Brown et al., 2020) menulis dari kiri ke kanan. Model encoder–decoder menggabungkan keduanya untuk tugas seperti terjemahan.

BERT dilatih dengan *masked language model*: sebagian token dalam kalimat ditutup, dan model menebak token yang ditutup dari konteks di kedua sisinya. Contoh di kuliah: “Jim made [MASK] for his girlfriend and [MASK] was very proud”, dengan jawaban *spaghetti* dan *he*. Kosakatanya sekitar 30 ribu token, karena kata dipotong menjadi potongan-potongan dengan WordPiece, saudara dekat byte-pair encoding di

Bab 38. BERT-Base punya 12 lapisan, dimensi tersembunyi 768, dan sekitar 110 juta parameter. Slide kuliah memberi pembandingan yang menarik: tabel embedding statis seperti word2vec dengan kosakata 200 ribu kata dan vektor berdimensi 768 sudah berisi sekitar 153 juta parameter, lebih banyak dari seluruh BERT-Base. Tokenisasi subkata bukan hanya mengatasi kata yang tidak dikenal, tetapi juga jauh lebih hemat.

Untuk menyetel BERT ke tugas klasifikasi, embedding keluaran dari token khusus [CLS] di awal kalimat dilewatkan ke lapisan klasifikasi kecil, dan seluruh jaringan disetel ulang dengan laju belajar kecil. Untuk tugas pasangan kalimat, seperti menilai apakah dua kalimat bermakna sama, keduanya digabung dengan token pemisah [SEP]. Tugas ketiga kuliah memakai resep ini untuk mengklasifikasikan cerita pendek. Di situ muncul perbedaan antara *multi-class* dan *multi-label*. Kalau setiap cerita punya tepat satu genre, keluaran memakai softmax dan cross-entropy, sehingga peluang semua kelas berjumlah satu. Kalau sebuah cerita bisa sekaligus lucu, sedih, dan misterius, setiap label mendapat sigmoid sendiri dengan binary cross-entropy, dan setiap label diputuskan dengan ambangnya sendiri. Memakai softmax untuk masalah multi-label memaksa label-label bersaing, padahal mereka tidak saling meniadakan.

10.7 Bahasa bersama modalitas lain

NLP multimodal menggabungkan teks dengan gambar, suara, video, atau data sensor. Pada *visual question answering* (Agrawal et al., 2015), model menerima sebuah gambar dan pertanyaan tentangnya, misalnya “berapa banyak orang di foto ini?”, dan harus menjawab dalam bahasa. Pembuatan keterangan gambar berjalan ke arah sebaliknya. Kuncinya adalah ruang embedding bersama, tempat teks dan gambar yang bermakna sama berdekatan, gagasan yang sama dengan CLIP di Bab 23. Kuliah juga menyebut penerapan seperti kendaraan otonom yang memadukan bahasa dengan lidar dan kamera, dan komputasi afektif yang membaca emosi dari wajah, suara, dan kata.

10.8 Biaya dan bias

Kuliah tidak berhenti pada kemampuan. Model besar membutuhkan energi yang besar untuk dilatih, dan kefasihannya bisa menutupi kenyataan bahwa ia tidak selalu memahami apa yang ditulisnya, argumen yang diangkat Bender et al. (2021). Bias sosial juga terbawa dari data. Contoh dari kuliah adalah terjemahan mesin dari bahasa dengan kata ganti netral gender, seperti bahasa Turki atau bahasa Indonesia, ke bahasa Inggris: “dia seorang dokter” dan “dia seorang perawat” bisa diterjemahkan menjadi *he* dan *she* menurut stereotip, padahal kalimat aslinya tidak menyebut gender sama sekali. Mesin pencari gambar untuk kata “CEO” dan “perawat” memperlihatkan pola yang sama. Bias embedding kata yang dibahas Bolukbasi et al. (2016) adalah akar dari masalah ini, dan persoalan yang lebih luas dibahas di Bab 35.

10.9 Perkembangan riset

Banyak perkembangan model bahasa besar sudah dibahas di Bab 38: hukum penskalaan, penyelarasan dengan preferensi manusia, model yang bernalar, dan agen. Untuk NLP sebagai bidang, satu pergeseran besar adalah bahwa banyak tugas yang dulu punya model khusus, seperti klasifikasi sentimen, ekstraksi entitas, atau terjemahan, kini dikerjakan oleh satu model umum yang diberi instruksi dalam bahasa biasa. Tugas-tugas kuliah ini, dari kantong kata sampai BERT yang disetel, menjadi cara terbaik untuk memahami apa yang sebenarnya terjadi di dalam model umum itu, dan untuk menilai kapan model kecil yang khusus masih lebih tepat, lebih murah, dan lebih mudah diperiksa.

CATATAN SUMBER. Bab ini mengikuti kuliah dan latihan Natural Language Processing (pra-pemrosesan dan kantong kata, representasi kata, PMI dan GloVe, jaringan saraf dan softmax, model bahasa n -gram, embedding kata dan kalimat yang dipelajari, attention dan transformer, NLP multimodal, model bahasa besar). Buku rujukan yang disebut kuliah antara lain buku Jurafsky dan Martin tentang pemrosesan ucapan dan bahasa.

11 Melihat: Computer Vision

Computer vision adalah salah satu kuliah pertama saya, di semester musim dingin 2024/25. Kuliahnya bergerak dari fisika lensa sampai NeRF dalam sebelas pertemuan, dan setiap beberapa minggu ada tugas praktikum: pengolahan citra dasar, ekstraksi fitur dan segmentasi, optical flow dan deteksi objek, lalu rekonstruksi tiga dimensi. Di bagian optical flow, untuk pertama kalinya di buku ini, gambar dan aliran fluida bertemu.

Bagi manusia, melihat terasa tanpa usaha. Bagi komputer, gambar hanyalah tabel angka. Bab ini menyusuri jalan dari cahaya yang masuk ke lensa sampai mesin yang bisa merekonstruksi dunia tiga dimensi. Di sepanjang jalan akan terlihat bahwa banyak alat klasik, seperti konvolusi, transformasi Fourier, dan kuadrat terkecil, tetap hidup di dalam jaringan saraf modern.

11.1 Dari cahaya ke piksel

Kamera paling sederhana adalah kotak gelap dengan lubang kecil. Setiap titik di dunia memancarkan cahaya ke segala arah, dan hanya satu berkas yang lolos lewat lubang itu, sehingga terbentuk gambar terbalik yang tajam. Kelemahannya, lubang kecil berarti sedikit cahaya. Lensa mengumpulkan banyak berkas dan memfokuskannya ke satu titik. Model lensa tipis menghubungkan panjang fokus f , jarak benda z_o , dan jarak bayangan z_i :

$$\frac{1}{f} = \frac{1}{z_o} + \frac{1}{z_i}. \quad (11.1)$$

Hanya benda pada satu jarak yang tepat fokus. Benda lain membentuk lingkaran kabur, dan f -number $N = f/D$, dengan D diameter bukaan, menentukan seberapa cepat kekaburan itu tumbuh. Bukaan kecil memberi *depth of field* yang lebar, dengan harga gambar yang lebih gelap.

Persamaan lensa tipis langsung memberi angka yang bisa dirasakan. Untuk lensa $f = 50$ mm dan benda sejauh dua meter, $1/z_i = 1/50 - 1/2000 = 39/2000$, sehingga sensor harus berada 51,3 mm di belakang lensa. Untuk benda sejauh sepuluh meter, jaraknya 50,25 mm. Memindahkan fokus dari sepuluh meter ke dua meter berarti menggeser lensa sekitar satu milimeter, dan benda di tak hingga difokuskan tepat pada jarak f . Lensa $f/2$ dengan panjang fokus 50 mm punya bukaan selebar 25 mm. Menutup bukaan ke $f/4$ membuat diameter bukaan setengahnya, sehingga lingkaran kabur untuk setiap benda yang tidak fokus juga setengahnya, dan rentang jarak yang tampak tajam melebar. Harganya, cahaya yang masuk tinggal seperempat, karena luas bukaan sebanding dengan kuadrat diameternya.

11.1.1 Medan cahaya dan bukaan sintetis

Kuliah ini diberikan oleh kelompok Oliver Bimber, yang banyak meneliti *light field*. Kamera biasa merekam berapa banyak cahaya yang jatuh di setiap piksel, tetapi tidak dari arah mana cahaya itu datang. Medan cahaya merekam keduanya: intensitas setiap berkas sebagai fungsi empat koordinat, dua untuk posisi di bidang lensa (u, v) dan dua untuk posisi di bidang sensor (s, t) (Levoy and Hanrahan, 1996). Medan cahaya bisa direkam dengan larik banyak kamera, dengan susunan lensa mikro di depan sensor, atau dengan satu kamera yang dipindah-pindahkan.

Dengan medan cahaya, fokus bisa dipilih setelah pemotretan. Gambar dari setiap sudut pandang digeser sebanding dengan disparitas yang sesuai untuk kedalaman tertentu, lalu semuanya dirata-ratakan. Benda pada kedalaman itu jatuh di tempat yang sama di semua gambar dan tampak tajam. Benda pada kedalaman lain jatuh di tempat yang berbeda-beda dan menjadi kabur. Hasilnya setara dengan foto dari lensa raksasa seukuran seluruh larik kamera, yang disebut *synthetic aperture*. Sifat ini punya akibat yang tidak terduga: penghalang di depan bidang fokus, seperti dedaunan, ikut dirata-ratakan dan memudar, sedangkan benda di belakangnya, yang di setiap gambar hanya terlihat sebagian lewat celah yang berbeda, muncul kembali.

Airborne optical sectioning (Kurmi et al., 2018), dari kelompok yang sama di Linz, membawa gagasan ini ke drone. Drone terbang sambil memotret hutan dari banyak posisi, dan gambar-gambar itu digabung menjadi satu bukaan sintetis selebar puluhan meter yang difokuskan ke permukaan tanah, sehingga daun-daun di atasnya memudar. Makalahnya menempatkan pendekatan ini sebagai alternatif yang lebih murah dari lidar dan yang tidak bergantung pada pencocokan titik seperti fotogrametri. Penerapan lanjutannya mencakup pencarian dan penyelamatan orang di bawah kanopi hutan (Schedl et al., 2020). Kuliah juga menunjukkan penerapan lain dari tumpukan fokus, misalnya memperkirakan kedalaman dari seberapa kabur setiap bagian gambar (*depth from defocus*) dengan fitur dari jaringan saraf.

Untuk komputasi, kita kembali ke model lubang jarum. Titik (X, Y, Z) dalam koordinat kamera jatuh di bidang gambar pada $x = fX/Z$ dan $y = fY/Z$. Pembagian dengan Z membuat proyeksi ini tidak linear, dan trik *homogeneous coordinates* mengembalikannya ke bentuk perkalian matriks dengan menambahkan satu koordinat:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} [\mathbf{R} \mid \mathbf{t}] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}. \quad (11.2)$$

Matriks pertama, $\mathbf{K}$, berisi sifat kamera sendiri: panjang fokus dalam piksel, titik pusat, dan kemiringan. Rotasi $\mathbf{R}$ dan translasi $\mathbf{t}$ menyatakan posisi kamera di dunia. Lensa nyata menambahkan distorsi radial, $x_d = x(1 + k_1 r^2 + k_2 r^4)$, yang membuat garis lurus tampak melengkung di tepi gambar. *Camera calibration* memperkirakan semua parameter ini, biasanya dari beberapa foto papan catur yang posisi sudut-sudutnya diketahui.

Proyeksi perspektif mirip bayangan tangan di dinding saat kita bermain wayang dengan senter. Tangan yang dekat ke senter membuat bayangan besar, yang jauh

membuat bayangan kecil. Dari bayangannya saja, kita tidak bisa tahu apakah itu tangan kecil di dekat senter atau tangan besar di dekat dinding. Kedalaman hilang saat proyeksi, dan sebagian besar kuliah ini adalah usaha untuk mendapatkannya kembali.

Sensor digital menambahkan lapisan lain. Ada noise foton dan noise elektronik, ada kuantisasi ke delapan atau dua belas bit, dan ada filter warna Bayer yang membuat setiap piksel hanya mengukur satu warna. Dua warna lainnya ditebak dari piksel tetangga dalam proses yang disebut *demosaiing*.

11.2 Pengolahan citra

Gambar bisa diperlakukan sebagai fungsi $I(x, y)$, dan banyak operasi dasarnya berupa konvolusi dengan kernel kecil (Bab 7). Filter kotak atau Gauss meratakan gambar dan menekan noise. Turunan, misalnya kernel Sobel, menonjolkan tepi. Laplacian, $\nabla^2 I = I_{xx} + I_{yy}$, menonjolkan perubahan tajam ke segala arah. Besar dan arah gradien ∇I menjadi bahan dasar deteksi tepi.

Banyak sifat filter lebih mudah dipahami di domain frekuensi. *Convolution theorem* menyatakan bahwa konvolusi di ruang sama dengan perkalian di frekuensi, $\mathcal{F}\{I * k\} = \mathcal{F}\{I\} \cdot \mathcal{F}\{k\}$. Filter Gauss tetap berbentuk Gauss di domain frekuensi, sehingga ia meredam frekuensi tinggi dengan halus. Filter kotak berubah menjadi fungsi sinc yang bergelombang, sehingga meninggalkan artefak. Spektrum gambar alami umumnya turun kira-kira seperti $1/f$, artinya sebagian besar energinya berada di frekuensi rendah.

Teorema yang sama menjelaskan mengapa *deconvolution* sulit. Kalau gambar kabur $B = I * k + n$ dibagi di domain frekuensi, $\mathcal{F}\{I\} \approx \mathcal{F}\{B\}/\mathcal{F}\{k\}$, noise ikut terbagi oleh bilangan yang hampir nol di frekuensi tempat kernel lemah, dan hasilnya meledak. Solusinya adalah regularisasi, misalnya filter Wiener. Tema yang sama akan muncul lagi pada masalah invers dalam simulasi.

Kernel Sobel untuk arah horizontal adalah

$$k_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad (11.3)$$

sebuah beda pusat di arah x yang dihaluskan dengan bobot $(1, 2, 1)$ di arah y . Terapkan pada potongan 3×3 yang kolom kirinya bernilai 10 dan dua kolom kanannya bernilai 50, tepi tegak antara gelap dan terang. Hasilnya $(-10 + 50) + 2(-10 + 50) + (-10 + 50) = 160$. Pada daerah yang rata, hasilnya nol. Kernel transposnya, k_y , menangkap tepi mendatar, dan dari kedua hasil diperoleh besar gradien $\sqrt{g_x^2 + g_y^2}$ dan arahnya $\text{atan2}(g_y, g_x)$.

Detektor tepi Canny (Canny, 1986) membangun di atas gradien ini dengan empat langkah. Gambar dihaluskan dengan Gauss supaya noise tidak dianggap tepi. Besar dan arah gradien dihitung. Tepi ditipiskan dengan hanya mempertahankan piksel yang gradiennya maksimum lokal sepanjang arah gradien, sehingga tepi setebal satu piksel. Terakhir, dua ambang dipakai: piksel di atas ambang tinggi pasti tepi, piksel di bawah ambang rendah pasti bukan, dan piksel di antaranya diterima hanya kalau tersambung dengan tepi yang pasti. Ambang ganda ini, yang disebut histeresis, menjaga garis tepi tetap utuh walau di beberapa tempat kontrasnya melemah.

Objek bisa tampak besar atau kecil. *Gaussian pyramid* membuat salinan gambar yang berulang kali dihaluskan lalu diperkecil setengahnya, sehingga algoritme yang bekerja pada satu ukuran bisa dijalankan di setiap tingkat piramida untuk menangkap semua skala. Piramida akan muncul lagi di optical flow, di U-Net, dan di solver multigrid.

11.3 Machine learning untuk visi

Gambar wajah berukuran 100×100 piksel adalah sebuah titik di ruang berdimensi sepuluh ribu, tetapi wajah tidak memenuhi seluruh ruang itu. *Principal component analysis* (Hotelling, 1933; Pearson, 1901) mencari arah dengan varians terbesar. Untuk data terpusat $\mathbf{X} \in \mathbb{R}^{N \times D}$, arah-arrah itu adalah vektor eigen matriks kovarians $\Sigma = \frac{1}{N} \mathbf{X}^T \mathbf{X}$ dengan nilai eigen terbesar. Turk and Pentland (1991) menyebut vektor eigen wajah sebagai *eigenfaces*. Setiap wajah bisa didekati sebagai rata-rata ditambah kombinasi beberapa puluh eigenface, dan jumlah komponen dipilih dari fraksi varians yang dijelaskan, $\sum_{k \leq K} \lambda_k / \sum_k \lambda_k$.

Bayangkan memotret sekumpulan balon panjang yang melayang miring. Dari satu sudut, balon-balon tampak sebagai titik-titik yang bertumpuk. Dari sudut lain, panjangnya terlihat jelas dan balon-balon tampak tersebar. PCA memilih sudut pandang tempat data paling tersebar, karena di sanalah perbedaan antar objek paling terlihat. PCA akan muncul lagi dengan nama lain di Bab 31, yaitu *proper orthogonal decomposition*, alat utama untuk mereduksi medan aliran.

Selama dua dekade, visi komputer bergantung pada fitur yang dirancang manusia. Contoh paling terkenal adalah SIFT (Lowe, 2004). SIFT mencari titik khas sebagai ekstrem lokal dari *difference of Gaussians* di seluruh skala, sehingga titik yang sama tetap ditemukan meski gambarnya diperbesar. Di sekitar setiap titik, ia menghitung histogram arah gradien relatif terhadap arah dominan, sehingga deskriptornya tahan terhadap rotasi. Hasilnya adalah vektor 128 dimensi yang bisa dicocokkan antar foto.

AlexNet (Krizhevsky et al., 2017) mengubah keadaan. Jaringan konvolusi yang dilatih pada ImageNet mempelajari fiturnya sendiri: kernel lapisan pertama menjadi detektor tepi dan warna, dan lapisan yang lebih dalam menjadi detektor bagian-bagian objek. Fitur dari jaringan terlatih seperti VGG atau ResNet ternyata berguna jauh di luar klasifikasi. Kuliah menunjukkan contohnya untuk fusi gambar dan estimasi kedalaman dari fokus.

11.4 Segmentasi

Segmentasi memberi label pada setiap piksel. *Semantic segmentation* memberi kelas seperti jalan, mobil, atau pohon, sedangkan *instance segmentation* juga membedakan mobil yang satu dari mobil yang lain.

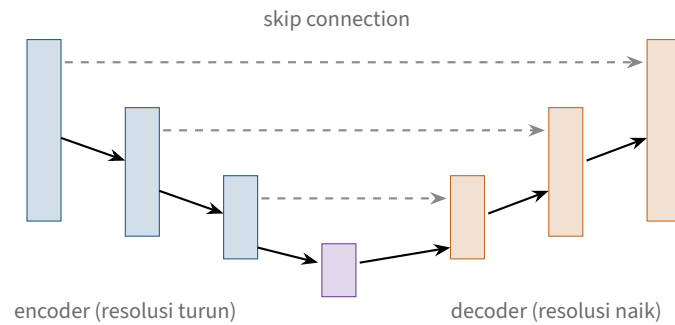
Cara klasik memandang segmentasi sebagai pengelompokan. *k-means* (Lloyd, 1982) mengelompokkan piksel berdasarkan warna dan posisi. *Mean shift* (Comaniciu and Meer, 2002) tidak perlu jumlah kelompok di awal. Ia memandang piksel sebagai sampel dari sebuah kerapatan, ditaksir dengan kernel seperti di Bab 2, lalu menggeser setiap titik berulang-ulang ke rata-rata berbobot tetangganya. Setiap langkah naik ke arah gradien

kerapatan, sehingga titik-titik berkumpul di puncak-puncak kerapatan, dan semua piksel yang berakhir di puncak yang sama membentuk satu segmen. Lebar kernel kembali menjadi kenop yang menentukan berapa banyak segmen yang muncul. Pendekatan graf menganggap piksel sebagai simpul dengan bobot kemiripan, lalu memotong graf di tempat ikatannya paling lemah, dan pemotongan ini bisa didekati dengan vektor eigen Laplacian graf, konsep yang muncul lagi di Bab 13. Pemotongan paling murah, yang hanya menjumlahkan bobot sisi yang diputus, cenderung memisahkan satu piksel yang terisolasi, karena memotong sedikit sisi selalu murah. Shi and Malik (2000) mengusulkan *normalized cut*, yang membagi biaya potongan dengan total ikatan setiap bagian ke seluruh graf,

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}, \quad (11.4)$$

sehingga potongan yang menghasilkan bagian yang amat kecil mendapat hukuman besar. Mencari Ncut minimum secara eksak sulit, tetapi versi yang dilonggarkan menjadi masalah nilai eigen umum $(D - W)y = \lambda Dy$, dengan W matriks bobot dan D matriks diagonal derajat. Vektor eigen dengan nilai eigen terkecil kedua memberi partisinya: piksel dengan nilai positif di satu sisi, negatif di sisi lain. *Hough transform* mendeteksi bentuk parametrik seperti garis: setiap titik tepi “memilih” semua garis $\rho = x \cos \theta + y \sin \theta$ yang melewatinya, dan garis yang nyata mengumpulkan suara terbanyak.

Cara modern memakai jaringan encoder dan decoder. Encoder mengecilkan resolusi sambil menangkap konteks, decoder memperbesarnya kembali dengan transposed convolution. U-Net (Ronneberger et al., 2015) menambahkan *skip connection* dari setiap tingkat encoder ke tingkat decoder yang sesuai, sehingga detail halus tidak hilang (Gambar 11.1). Model fondasi seperti Segment Anything (Kirillov et al., 2023) kini bisa memotong objek apa pun dari petunjuk sederhana seperti satu klik.



GAMBAR 11.1. Skema U-Net. Encoder mengecilkan resolusi, decoder memperbesarnya lagi, dan skip connection membawa detail dari setiap tingkat encoder ke decoder.

11.5 Optical flow

Video adalah deret gambar. *Optical flow* adalah medan vektor (u, v) yang menyatakan ke mana setiap piksel bergerak dari satu bingkai ke bingkai berikutnya. Asumsi

dasarnya adalah *brightness constancy*: titik yang bergerak mempertahankan kecerahannya, $I(x + u, y + v, t + 1) = I(x, y, t)$. Untuk gerak kecil, ruas kiri bisa diuraikan dengan Taylor orde satu, $I(x + u, y + v, t + 1) \approx I(x, y, t) + I_x u + I_y v + I_t$, dengan I_x , I_y , dan I_t turunan parsial kecerahan. Mengurangkan $I(x, y, t)$ dari kedua ruas memberi persamaan optical flow,

$$I_x u + I_y v + I_t = 0. \quad (11.5)$$

Ini satu persamaan dengan dua bilangan yang tidak diketahui. Di setiap piksel, kita hanya bisa mengetahui komponen gerak yang searah gradien kecerahan. Masalah ini disebut *aperture problem*. Contoh angka membuatnya jelas. Misalkan di sebuah piksel $I_x = 2$, $I_y = 0$, dan $I_t = -4$. Persamaannya menjadi $2u - 4 = 0$, sehingga $u = 2$, sedangkan v bisa bernilai berapa saja. Tepi vertikal hanya memberi tahu gerak horizontal. Untuk menentukan v , kita butuh piksel lain yang gradiennya berarah berbeda, misalnya di sebuah sudut.

Tiang pembatas jalan yang bergaris miring dan berputar memberi ilusi yang sama. Kalau dilihat lewat lubang kecil, garis-garisnya tampak bergerak naik, padahal tiangnya berputar ke samping. Melalui celah sempit, hanya gerak yang tegak lurus terhadap garis yang terlihat.

Ada dua cara klasik untuk menutup kekurangan informasi ini. Lucas–Kanade menganggap semua piksel dalam sebuah jendela kecil bergerak sama, lalu menyelesaikan masalah kuadrat terkecil

$$\begin{pmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = - \begin{pmatrix} \sum I_x I_t \\ \sum I_y I_t \end{pmatrix}. \quad (11.6)$$

Asal persamaan ini sederhana. Setiap piksel p di jendela memberi satu persamaan $I_x(p)u + I_y(p)v = -I_t(p)$. Dengan banyak piksel, kita punya lebih banyak persamaan daripada bilangan yang dicari, dan kita meminimalkan jumlah kuadrat sisanya, $E(u, v) = \sum_p (I_x(p)u + I_y(p)v + I_t(p))^2$. Menurunkan terhadap u dan v lalu menyamakannya dengan nol memberi dua persamaan normal, yang ditulis dalam bentuk matriks di atas. Matriks di ruas kiri hanya bisa dibalik kalau jendela berisi gradien ke berbagai arah, seperti di sudut dan tekstur. Di sepanjang tepi lurus, semua gradien sejajar, matriksnya berpangkat satu, dan aperture problem muncul kembali. Nilai eigen matriks yang sama juga dipakai detektor sudut Harris untuk mencari titik yang baik untuk dilacak. Untuk gerak lebih dari satu piksel, Taylor orde satu tidak berlaku lagi, dan solusinya adalah piramida: hitung flow kasar di resolusi rendah, lalu perhalus tingkat demi tingkat. Horn–Schunck (Horn and Schunck, 1981) mencari medan flow yang padat dengan meminimalkan

$$E(u, v) = \iint (I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2) dx dy. \quad (11.7)$$

Suku pertama memaksa kecocokan dengan data, suku kedua memaksa medan yang halus. Ini regularisasi dalam bentuk paling murni: data yang tidak lengkap dilengkapi dengan prior.

Jaringan saraf kemudian belajar optical flow langsung dari data. FlowNet (Fischer et al., 2015) memakai CNN encoder dan decoder, dan varian korelasinya membandingkan

fitur dua bingkai secara eksplisit. RAFT (Teed and Deng, 2020) membangun volume korelasi untuk semua pasangan piksel lalu memperbaiki flow secara iteratif dengan sebuah unit rekuren.

Hubungan dengan fluida di sini langsung. Di laboratorium, aliran diberi partikel kecil dan disinari laser, lalu dua foto berurutan dibandingkan untuk mendapatkan medan kecepatan. Teknik ini disebut *particle image velocimetry*, dan secara matematis ia adalah optical flow. Hubungannya bahkan lebih dalam. Untuk zat pewarna yang terbawa aliran tak termampatkan, kekekalan konsentrasi c berbunyi $\partial_t c + \mathbf{u} \cdot \nabla c = 0$, dan itu adalah Persamaan (11.5) dengan I diganti c . Suku kehalusan dalam Persamaan (11.7) bisa diganti syarat fisika seperti $\nabla \cdot \mathbf{u} = 0$. Raissi et al. (2020) membawa gagasan ini sampai ujungnya: dari video konsentrasi zat pewarna saja, sebuah PINN menyimpulkan medan kecepatan dan tekanan yang tidak pernah diukur, dengan memaksakan persamaan Navier–Stokes di dalam loss (Bab 29).

11.6 Deteksi objek

Deteksi menjawab dua pertanyaan sekaligus: apa, dan di mana. Keluarannya berupa kotak pembatas beserta kelasnya. Tumpang tindih antara kotak tebakan A dan kotak benar B diukur dengan *intersection over union*, $|A \cap B|/|A \cup B|$.

Pendekatan dua tahap dimulai dengan R-CNN, yang mengusulkan sekitar dua ribu wilayah dengan *selective search* lalu mengklasifikasi masing-masing dengan CNN. Fast R-CNN (Girshick, 2015) menghitung peta fitur sekali untuk seluruh gambar dan memotong wilayah dari peta itu. Faster R-CNN (Ren et al., 2015) mengganti selective search dengan jaringan pengusul wilayah yang dipelajari, dan menambahkan cabang mask ke arsitektur ini menghasilkan instance segmentation. Pendekatan satu tahap seperti YOLO (Redmon et al., 2015) membagi gambar menjadi grid dan membiarkan setiap sel langsung memprediksi kotak dan kelas dalam satu lintasan. Jauh lebih cepat, cocok untuk waktu nyata.

Contoh dengan angka: kotak tebakan dari $(0, 0)$ sampai $(4, 4)$ dan kotak benar dari $(2, 2)$ sampai $(6, 6)$ masing-masing berluas 16, dan irisannya kotak 2×2 berluas 4. Gabungannya $16 + 16 - 4 = 28$, sehingga $\text{IoU} = 4/28 \approx 0,14$. Walau kedua kotak tampak bersinggungan cukup banyak, deteksi ini akan dianggap salah dengan ambang yang lazim 0,5. Untuk mencapai IoU 0,5, dua kotak berukuran sama yang bergeser sepanjang diagonal hanya boleh bergeser sekitar seperlima sisinya di setiap arah.

Banyak kotak yang tumpang tindih untuk objek yang sama dibersihkan dengan *non-maximum suppression*: simpan kotak dengan skor tertinggi, buang kotak lain yang tumpang tindihnya dengan kotak itu melewati ambang. Kinerja diukur dengan kurva presisi terhadap recall, dan luas di bawahnya, *average precision*, dirata-ratakan atas semua kelas.

11.7 Geometri banyak pandang

Satu kamera kehilangan kedalaman, tetapi dua kamera bisa mendapatkannya kembali, persis seperti dua mata kita (Hartley and Zisserman, 2004). Titik x di gambar kiri bisa

berasal dari titik mana pun di sepanjang sinar dari pusat kamera kiri. Dilihat dari kamera kanan, sinar itu tampak sebagai garis, yang disebut *epipolar line*, dan pasangan titik di gambar kanan pasti terletak di garis tersebut. Secara aljabar, $\mathbf{x}'^T \mathbf{F} \mathbf{x} = 0$, dengan $\mathbf{F}$ matriks fundamental 3×3 berpangkat dua. Untuk kamera terkalibrasi, matriks padanannya adalah matriks esensial $\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R}$. Pencarian pasangan titik berubah dari pencarian dua dimensi menjadi satu dimensi.

Setelah *rectification*, garis epipolar menjadi horizontal. Pergeseran horizontal titik yang sama antara kedua gambar disebut *disparity* d , dan segitiga sebangun memberi

$$Z = \frac{f B}{d}, \quad (11.8)$$

dengan f panjang fokus dalam piksel dan B jarak antar kamera. Penurunannya hanya memakai proyeksi lubang jarum. Letakkan kamera kiri di titik asal dan kamera kanan bergeser sejauh B di sumbu x . Titik (X, Z) jatuh di gambar kiri pada $x_L = fX/Z$ dan di gambar kanan pada $x_R = f(X - B)/Z$. Selisihnya, $d = x_L - x_R = fB/Z$, tidak lagi bergantung pada X , dan membalik persamaan itu memberi $Z = fB/d$. Kamera stereo dengan $f = 700$ piksel dan $B = 0,12$ meter yang melihat titik dengan disparity dua puluh piksel menempatkan titik itu pada 4,2 meter. Kalau disparity salah satu piksel saja, kedalamannya menjadi 4,0 atau 4,4 meter. Semakin jauh objeknya, semakin kecil disparity-nya, dan semakin besar galat relatif kedalamannya.

Structure from motion memperluas gagasan ini ke banyak foto dari kamera yang bergerak: cocokkan fitur antar foto, perkirakan pose kamera, triangulasi titik-titik tiga dimensi, lalu perhalus semuanya bersama-sama.

11.7.1 Homografi dan RANSAC

Kalau kamera memotret bidang datar, atau berputar di tempat tanpa bergeser, dua gambar dihubungkan oleh *homography*, sebuah matriks 3×3 $\mathbf{H}$ dalam koordinat homogen, $\tilde{\mathbf{x}}' \sim \mathbf{H} \tilde{\mathbf{x}}$. Matriks ini punya delapan derajat kebebasan, karena skalanya tidak penting, dan setiap pasangan titik yang bersesuaian memberi dua persamaan, sehingga empat pasangan sudah cukup untuk menentukannya. Homografi dipakai untuk menjahit foto panorama, meluruskan dokumen yang difoto miring, dan mengkalibrasi kamera dengan papan catur.

Masalahnya, pasangan titik yang ditemukan dengan pencocokan fitur seperti SIFT selalu mengandung pasangan yang salah. Kuadrat terkecil biasa akan ditarik jauh oleh satu pasangan yang salah saja. RANSAC (Fischler and Bolles, 1981) membalik caranya: ambil sampel minimal secara acak, empat pasangan untuk homografi, hitung model dari sampel itu, lalu hitung berapa banyak pasangan lain yang setuju dengan model itu dalam toleransi tertentu. Ulangi, dan simpan model dengan pendukung terbanyak. Berapa kali harus diulang? Kalau fraksi pasangan yang benar adalah w , peluang satu sampel berisi s pasangan benar semua adalah w^s , dan peluang N sampel semuanya gagal adalah $(1 - w^s)^N$. Supaya peluang berhasil paling sedikit p ,

$$N \geq \frac{\log(1 - p)}{\log(1 - w^s)}. \quad (11.9)$$

Dengan separuh pasangan benar, $s = 4$, dan $p = 0,99$, peluang satu sampel bersih $1/16$, dan dibutuhkan sekitar 72 percobaan. Kalau hanya sepertiga pasangan yang benar, angka itu naik menjadi sekitar 370. Ukuran sampel minimal menentukan: untuk model dengan delapan pasangan, seperti matriks fundamental dengan algoritme delapan titik, separuh pasangan benar sudah menuntut sekitar 1200 percobaan.

11.8 Visi tiga dimensi

Dunia tiga dimensi bisa direpresentasikan dengan banyak cara. Peta kedalaman memberi satu angka per piksel. Grid voxel sederhana, tetapi memorinya tumbuh kubik. Awan titik hemat, tetapi tidak punya permukaan. Mesh punya permukaan eksplisit, tetapi topologinya sulit diubah. Permukaan implisit $f(x) = 0$ menyimpan bentuk sebagai himpunan nol sebuah fungsi. Memprediksi kedalaman dari satu gambar mengandung ambiguitas skala, karena miniatur yang dekat dan bangunan nyata yang jauh bisa menghasilkan gambar yang sama. Karena itu galat sering diukur dengan ukuran yang tidak menghukum faktor skala global.

Kuliah ditutup dengan NeRF (Mildenhall et al., 2020). Sebuah MLP menerima posisi x dan arah pandang d , lalu mengeluarkan kerapatan $\sigma(x)$ dan warna $c(x, d)$. Warna satu piksel dihitung dengan *volume rendering* sepanjang sinar $r(t) = o + td$,

$$C(r) = \int_{t_n}^{t_f} T(t) \sigma(r(t)) c(r(t), d) dt, \quad T(t) = \exp \left(- \int_{t_n}^t \sigma(r(s)) ds \right), \quad (11.10)$$

dengan $T(t)$ peluang cahaya lolos sampai titik t tanpa terhalang. Jaringan dilatih dengan membandingkan warna hasil render dengan foto, dan setelah terlatih, ia bisa merender sudut pandang yang belum pernah difoto. NeRF adalah contoh *neural field*, fungsi kontinu dari koordinat ke besaran fisik. Neural field dibahas lebih luas di Bab 15, dan kekerabatannya dengan PINN dekat sekali.

Pertemuan terakhir membahas arah-arrah baru: *vision transformer* yang memotong gambar menjadi petak 16×16 dan memperlakukannya seperti kata (Dosovitskiy et al., 2020), model fondasi yang dilatih pada pasangan gambar dan teks, dan *computational imaging* yang merancang kamera dan algoritme bersama-sama. Kuliah juga menekankan batasnya. Model besar tidak selalu benar, dan stiker kecil yang dirancang khusus bisa membuat detektor gagal mengenali rambu berhenti (Eykholt et al., 2017). Alat seperti Grad-CAM (Selvaraju et al., 2016) membantu melihat bagian gambar mana yang dipakai model untuk memutuskan, topik yang kembali di Bab 17.

11.9 Perkembangan riset

Visi komputer bergerak ke model fondasi yang bisa dipakai tanpa pelatihan ulang. SAM 2 (Ravi et al., 2024) memperluas Segment Anything dari gambar ke video, dengan transformer yang membawa ingatan aliran (streaming memory) dari bingkai ke bingkai. Dataset video segmentasinya dikumpulkan dengan mesin data yang melibatkan pengguna: model menyarankan topeng, manusia mengoreksi, dan model dilatih ulang dengan koreksi itu. Pada segmentasi gambar, SAM 2 dilaporkan lebih akurat dan enam

kali lebih cepat dari SAM. Depth Anything (Yang et al., 2024) melakukan hal serupa untuk estimasi kedalaman dari satu gambar, dengan sekitar 62 juta gambar tanpa label yang dianotasi otomatis. Kedalaman dari satu kamera, yang secara geometris tidak bisa ditentukan seperti dibahas di bagian stereo, ditebak dari pengetahuan tentang bentuk dunia yang dipelajari dari data.

Di grafika dan rekonstruksi tiga dimensi, 3D Gaussian splatting (Kerbl et al., 2023) mengganti neural field dengan jutaan Gaussian tiga dimensi yang posisi, kovarians anisotropik, warna, dan opasitasnya dioptimalkan langsung dari foto-foto kalibrasi. Rendering dilakukan dengan memproyeksikan Gaussian ke bidang gambar dan menumpuknya, bukan dengan menembakkan sinar melewati jaringan saraf. Hasilnya adalah sintesis sudut pandang baru secara waktu nyata pada resolusi 1080p. Gagasan dasarnya, mewakili adegan sebagai sekumpulan fungsi Gauss, menyambung dengan model campuran Gauss di Bab 6 dan NDT di Bab 21.

CATATAN SUMBER. Bab ini mengikuti sebelas kuliah Computer Vision beserta praktikumnya: pembentukan citra, pengolahan citra, machine learning untuk visi, ekstraksi fitur, segmentasi, optical flow, deteksi objek, geometri banyak pandang, visi tiga dimensi, dan tren. Buku rujukan yang direkomendasikan adalah Szeliski (2022) dan Hartley and Zisserman (2004).

12 Arsitektur Modern dan Model Generatif

Kuliah lanjutan deep learning saya ikuti pada musim panas 2025, dan latihannya setahun kemudian. Tugas-tugas latihannya membentuk sebuah tangga: PyTorch dan training loop, jaringan visi dan transfer learning, pemantauan eksperimen dan CNN lanjutan, autoencoder dan VAE, contoh adversarial dan GAN, lalu normalizing flow. Bab ini menaiki tangga yang sama.

Bab 7 membangun jaringan dari bata-bata dasarnya. Bab ini membahas dua hal yang membuat deep learning modern berbeda: arsitektur yang memungkinkan jaringan amat dalam dilatih dengan stabil, dan model yang tidak hanya mengenali data tetapi juga membangkitkan data baru.

12.1 Dari LeNet ke ResNet

LeNet (Lecun et al., 1998) dan AlexNet (Krizhevsky et al., 2017) masih memakai kernel yang relatif besar. Keluarga VGG hanya memakai kernel 3×3 yang ditumpuk. Dua lapisan 3×3 melihat daerah 5×5 , tiga lapisan melihat 7×7 , dengan parameter yang lebih sedikit dan nonlinearitas yang lebih banyak. Dengan 64 kanal masuk dan keluar, satu lapisan 5×5 membutuhkan $25 \cdot 64^2 = 102\,400$ bobot, sedangkan dua lapisan 3×3 dengan receptive field yang sama cukup dengan $2 \cdot 9 \cdot 64^2 = 73\,728$ bobot, sekitar 28 persen lebih sedikit, dan memberi dua nonlinearitas alih-alih satu.

Menambah lapisan seharusnya tidak merugikan, karena lapisan tambahan bisa saja belajar menjadi fungsi identitas. Kenyataannya, jaringan polos yang dalam justru punya galat *latih* lebih tinggi, jadi masalahnya terletak pada optimasi, bukan overfitting. ResNet (He et al., 2016) mengubah tugas setiap blok dari mempelajari $H(x)$ menjadi mempelajari selisihnya,

$$y = x + F(x). \quad (12.1)$$

Kalau identitas sudah cukup baik, blok cukup mendorong F ke nol. Gradien pun mendapat jalan tol, $\partial y / \partial x = I + \partial F / \partial x$, sehingga sinyal mundur tidak harus melewati semua nonlinearitas. Gagasannya sama dengan constant error carousel pada LSTM (Bab 9), hanya saja kali ini di sepanjang kedalaman, bukan di sepanjang waktu.

Editor yang baik tidak menulis ulang seluruh esai setiap kali merevisi. Ia menulis catatan perubahan di pinggir halaman, dan kalau esainya sudah bagus, catatannya kosong. Blok residual bekerja seperti editor itu: ia hanya menambahkan koreksi pada apa yang sudah ada.

Latihan membahas dua variasi. Pada ResNet *pre-activation*, normalisasi dan aktivasi diletakkan sebelum konvolusi, sehingga bloknya berbentuk $x + F(\phi(x))$ alih-alih $\phi(x + F(x))$. Ketika jalur pintas dan jalur utama berbeda jumlah kanal, konvolusi 1×1 dipakai untuk menyamakannya.

Ada juga blok yang dirancang untuk efisiensi. *Depthwise-separable convolution* memecah konvolusi menjadi dua langkah. Setiap kanal dikonvolusi sendiri-sendiri, lalu kanal-kanal dicampur dengan konvolusi 1×1 . Untuk kernel 3×3 dan 64 kanal, jumlah bobot turun dari $9 \cdot 64 \cdot 64 = 36\,864$ menjadi $9 \cdot 64 + 64 \cdot 64 = 4\,672$. Blok *squeeze-and-excitation* merata-ratakan setiap kanal menjadi satu angka, melewatkannya ke MLP kecil, lalu memakai hasilnya untuk menskalakan kanal, sehingga jaringan belajar kanal mana yang penting untuk gambar tertentu. *Vision transformer* memotong gambar menjadi petak-petak $P \times P$ dan memperlakukan setiap petak sebagai token (Dosovitskiy et al., 2020). Gambar 224×224 dengan $P = 16$ menghasilkan $14 \times 14 = 196$ token. MLP-Mixer bergantian mencampur informasi antar token dan antar kanal dengan MLP biasa, tanpa konvolusi dan tanpa attention.

Melatih jaringan besar dari nol membutuhkan data besar. *Transfer learning* memakai jaringan yang sudah dilatih pada ImageNet, membekukan sebagian besar lapisannya, lalu melatih ulang kepala klasifikasinya saja. Fitur umum seperti tepi dan tekstur sudah dipelajari, sehingga data yang sedikit pun cukup untuk tugas baru.

12.2 Dari mengenali ke membangkitkan

Model diskriminatif mempelajari $p(y | x)$. Model generatif mempelajari $p(x)$ itu sendiri, sehingga bisa membangkitkan contoh baru, mengisi bagian yang hilang, atau menilai seberapa wajar sebuah data. Kuliah membahas lima keluarga.

12.2.1 Autoencoder

Autoencoder memampatkan x menjadi kode $z = e(x)$ yang berdimensi kecil, lalu merekonstruksinya menjadi $\hat{x} = d(z)$. Loss rekonstruksinya mengikuti asumsi noise (Tabel 1.1): MSE untuk data kontinu yang dianggap Gauss, binary cross-entropy dengan keluaran sigmoid untuk data di antara nol dan satu. Autoencoder linear dengan loss kuadrat menemukan subruang yang sama dengan PCA. Kelemahannya, autoencoder biasa tidak menjamin ruang kode yang rapi. Titik acak di ruang kode bisa didekode menjadi sesuatu yang tidak bermakna.

12.2.2 Variational autoencoder

VAE (Kingma and Welling, 2013) memperbaiki hal itu dengan memberi struktur probabilistik. Encoder mengeluarkan distribusi $q_\phi(z | x) = \mathcal{N}(\mu, \text{diag } \sigma^2)$, decoder memodelkan $p_\theta(x | z)$, dan ruang kode diberi prior $p(z) = \mathcal{N}(0, I)$. Pelatihannya memaksimalkan *evidence lower bound*:

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)). \quad (12.2)$$

Suku pertama mengukur rekonstruksi, suku kedua mengukur kedekatan ke prior. Untuk Gauss diagonal, divergensi KL punya bentuk tertutup, $\text{KL} = \frac{1}{2} \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$. Untuk satu dimensi, rumus ini didapat dari definisi $\text{KL}(q \| p) = \mathbb{E}_q[\log q - \log p]$ dengan $q = \mathcal{N}(\mu, \sigma^2)$ dan $p = \mathcal{N}(0, 1)$. Log kerapatan keduanya adalah $\log q = -\frac{1}{2} \log(2\pi\sigma^2) - (z - \mu)^2 / (2\sigma^2)$ dan $\log p = -\frac{1}{2} \log 2\pi - z^2 / 2$. Di bawah q , $\mathbb{E}[(z - \mu)^2] = \sigma^2$ dan $\mathbb{E}[z^2] = \sigma^2 + \mu^2$, sehingga $\mathbb{E}_q[\log q - \log p] = -\frac{1}{2} \log \sigma^2 - \frac{1}{2} + \frac{1}{2}(\sigma^2 + \mu^2)$, yang sama dengan rumus

di atas. Untuk dimensi yang independen, KL-nya tinggal dijumlahkan. Satu dimensi laten dengan $\mu = 1$ dan $\sigma = 1$ memberi KL sebesar 0,5. Dengan $\mu = 0$ dan $\sigma = 0,5$, hasilnya $\frac{1}{2}(0,25 - 1 + 1,386) \approx 0,318$. KL bernilai nol hanya kalau $\mu = 0$ dan $\sigma = 1$, yaitu tepat sama dengan prior.

Untuk membawa gradien melewati pengambilan sampel acak, VAE memakai *reparameterization trick*. Sampel ditulis sebagai $z = \mu + \sigma \odot \epsilon$ dengan $\epsilon \sim \mathcal{N}(0, I)$. Keacakan dipindahkan ke ϵ , sehingga z menjadi fungsi yang bisa diturunkan terhadap μ dan σ .

Contoh dengan angka: kalau encoder mengeluarkan $\mu = 1,2$ dan $\sigma = 0,5$ untuk sebuah gambar, dan noise yang ditarik adalah $\epsilon = -0,4$, maka $z = 1,2 + 0,5 \cdot (-0,4) = 1,0$. Turunan z terhadap μ adalah satu dan terhadap σ adalah $\epsilon = -0,4$. Gradien loss rekonstruksi yang mengalir ke z bisa diteruskan ke encoder lewat dua turunan sederhana ini, padahal tanpa trik ini, “menarik sampel” bukan operasi yang punya turunan.

Saya membayangkan encoder sebagai orang yang melihat foto lalu menulis deskripsi singkat yang sengaja agak samar, misalnya “wajah bulat, rambut pendek, kira-kira tersenyum”. Decoder adalah pelukis yang menggambar dari deskripsi itu. Suku KL memaksa semua deskripsi memakai kosakata yang sama dan tidak terlalu spesifik, sehingga deskripsi baru yang dikarang bebas pun tetap menghasilkan wajah yang masuk akal.

Latihan menekankan satu kenop praktis: bobot keseimbangan antara rekonstruksi dan KL. Bobot KL yang terlalu besar membuat gambar kabur dan ruang laten tidak terpakai. Bobot yang terlalu kecil membuat VAE kembali menjadi autoencoder biasa. Penurunan ELBO yang lebih teliti, termasuk versi hierarkis yang mengarah ke model difusi, ada di Bab 15.

12.2.3 Generative adversarial networks

GAN (Goodfellow et al., 2014) mempertandingkan dua jaringan. Generator G mengubah noise z menjadi sampel palsu, dan diskriminator D menebak apakah sebuah sampel asli atau palsu:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_z [\log (1 - D(G(z)))]. \quad (12.3)$$

Bentuk ini tidak lain adalah binary cross-entropy diskriminator yang tandanya dibalik untuk generator. Di awal pelatihan, diskriminator mudah menang dan $\log(1 - D(G(z)))$ jenuh, sehingga gradien generator kecil. Jalan keluarnya adalah *non-saturating loss*, di mana generator memaksimalkan $\log D(G(z))$ saja.

Apa yang sebenarnya dioptimalkan permainan ini? Untuk generator yang tetap, dengan distribusi sampel palsu p_g , tujuan diskriminator bisa ditulis sebagai integral $\int [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] dx$. Di setiap titik x , fungsi $a \log D + b \log(1 - D)$ maksimal pada $D = a/(a + b)$, sehingga diskriminator terbaik adalah

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \quad (12.4)$$

Memasukkan D^* kembali ke tujuan permainan memberi $-\log 4 + 2\text{JS}(p_{\text{data}}||p_g)$, dengan JS divergensi Jensen–Shannon (Goodfellow et al., 2014). Jadi generator, ketika melawan

diskriminator yang sempurna, sebenarnya meminimalkan jarak antara distribusi data dan distribusi sampelnya, dan minimumnya tercapai tepat ketika keduanya sama, saat $D^* = \frac{1}{2}$ di mana-mana: diskriminator hanya bisa menebak dengan melempar koin.

Pemalsu uang belajar mencetak uang yang lolos pemeriksaan, sementara detektif belajar membedakan uang asli dan palsu. Setiap kali detektif menemukan cacat baru, pemalsu memperbaikinya. Kalau keduanya seimbang, uang palsu akhirnya tidak bisa dibedakan dari yang asli. Masalah khas GAN adalah *mode collapse*, ketika generator menemukan beberapa sampel yang selalu lolos lalu berhenti menghasilkan variasi lain. Pelatihannya juga tidak stabil, karena tidak ada satu fungsi loss yang turun secara monoton.

12.2.4 Normalizing flows

Flow membangun distribusi yang rumit dengan mengubah distribusi sederhana melalui fungsi yang bisa dibalik (Rezende and Mohamed, 2015). Kalau $x = g(z)$ dengan g bijektif, rumus perubahan variabel memberi kerapatan yang eksak,

$$\log p_X(x) = \log p_Z(g^{-1}(x)) + \log \left| \det \frac{\partial g^{-1}}{\partial x} \right|. \quad (12.5)$$

Dalam satu dimensi, rumus ini berasal dari kekekalan peluang. Peluang yang jatuh di selang kecil $[z, z + dz]$ harus sama dengan peluang di selang padanannya $[x, x + dx]$, sehingga $p_X(x)|dx| = p_Z(z)|dz|$, atau $p_X(x) = p_Z(z)|dz/dx|$. Kalau $x = 2z$ dengan $z \sim \mathcal{N}(0, 1)$, setiap selang diregangkan dua kali, sehingga kerapatannya harus turun separuh: $p_X(x) = \frac{1}{2}p_Z(x/2)$, yang memang kerapatan $\mathcal{N}(0, 4)$. Determinan Jacobian adalah versi banyak dimensi dari faktor regangan ini, dan ia mengukur perubahan volume lokal. Untuk dimensi besar, determinan umum mahal dihitung, jadi lapisan flow dirancang agar Jacobian-nya segitiga. *Coupling layer* membagi vektor menjadi dua bagian, membiarkan bagian pertama apa adanya, dan mengubah bagian kedua dengan fungsi dari bagian pertama. Untuk coupling aditif, $x'_2 = x_2 + t(x_1)$, determinannya bernilai satu dan inversnya analitis. Syarat bijeksi berarti dimensi tidak boleh berubah. Imbalannya, flow memberi likelihood yang eksak, bukan batas bawah seperti ELBO.

12.2.5 Model difusi dan model energi

Model difusi (Ho et al., 2020; Sohl-Dickstein et al., 2015) merusak data perlahan-lahan dengan noise Gauss, lalu melatih jaringan untuk membalik proses itu langkah demi langkah. Mereka kini menjadi model generatif gambar yang paling kuat, dan matematikanya, yang melibatkan persamaan diferensial stokastik dan persamaan Fokker-Planck, dibahas lengkap di Bab 15. Model berbasis energi, termasuk mesin Boltzmann, mendefinisikan $p(x) \propto \exp(-E(x))$. Mudah ditulis, tetapi sulit dilatih, karena konstanta normalisasinya adalah integral atas seluruh ruang.

Tabel 12.1 merangkum kelima keluarga ini.

12.2.6 Melatih GAN dengan jarak yang lebih baik

Kesulitan klasik GAN punya penjelasan geometris. Kalau distribusi data dan distribusi generator terletak di permukaan berdimensi rendah yang tidak beririsan, seperti dua

TABEL 12.1. Lima keluarga model generatif.

Model	Dilatih dengan	Kekuatan	Kelemahan
VAE	ELBO	ruang laten terstruktur, stabil	sampel cenderung kabur
GAN	permainan minimax	sampel tajam	tidak stabil, mode collapse
Flow	likelihood eksak	kerapatan eksak, bisa dibalik	arsitektur terbatas, dimensi tetap
Difusi	denoising	kualitas dan keragaman	sampling lambat
Energi	beragam, sering MCMC	fleksibel	normalisasi sulit

garis di bidang, diskriminator yang sempurna selalu bisa memisahkan keduanya, dan gradien yang ia berikan ke generator hampir nol di mana-mana. Wasserstein GAN (Arjovsky et al., 2017) mengganti tujuan pertandingan dengan jarak Wasserstein, atau jarak *earth mover*: biaya minimum untuk memindahkan massa satu distribusi agar menjadi distribusi lain, seperti memindahkan tumpukan pasir. Untuk dua titik massa di posisi 0 dan θ , jarak Wasserstein-nya $|\theta|$, yang mengecil mulus ketika generator mendekat, sementara divergensi Jensen–Shannon yang dioptimalkan GAN asli bernilai tetap $\log 2$ selama keduanya tidak berimpit, sehingga tidak memberi arah sama sekali. Dalam praktik, jarak Wasserstein ditaksir oleh sebuah *critic* yang dibatasi Lipschitz, mula-mula dengan memotong bobotnya, lalu dengan penalti pada norma gradiennya.

Kualitas model generatif gambar biasanya diukur dengan Fréchet Inception Distance (Heusel et al., 2017). Gambar asli dan gambar yang dibangkitkan sama-sama dilewatkan ke jaringan klasifikasi yang sudah dilatih, aktivasi lapisan dalamnya dianggap berdistribusi Gauss, dan jarak Fréchet antara dua Gauss itu dihitung dengan rumus yang sama dengan FCD di Bab 23. Angka yang lebih kecil berarti statistik gambar yang dibangkitkan lebih mirip gambar asli.

12.2.7 Mengarahkan model difusi

Model difusi bersyarat, misalnya yang dibimbing teks, sering menghasilkan gambar yang kurang patuh pada syaratnya. *Classifier-free guidance* (Ho and Salimans, 2022) melatih satu jaringan untuk dua tugas sekaligus: memprediksi noise dengan syarat c , dan tanpa syarat, dengan sesekali membuang syaratnya selama pelatihan. Saat membangkitkan, kedua prediksi digabung,

$$\tilde{\epsilon} = (1 + w) \epsilon_{\theta}(x_t, c) - w \epsilon_{\theta}(x_t), \quad (12.6)$$

dengan w kekuatan bimbingan. Arah $\epsilon_{\theta}(x_t, c) - \epsilon_{\theta}(x_t)$ adalah apa yang “ditambahkan” syarat itu, dan rumus di atas melangkah lebih jauh ke arah tersebut. Dalam bahasa score dari Bab 15, ini setara dengan mengambil sampel dari distribusi yang dipertajam, sebanding dengan $p(x | c) p(c | x)^w$. Nilai w yang lebih besar membuat gambar lebih patuh pada teks tetapi kurang beragam, tawar-menawar yang sama dengan suhu pada softmax.

12.3 Contoh adversarial

Jaringan yang akurat bisa dibohongi dengan perubahan yang tidak terlihat oleh mata. *Fast gradient sign method* menggeser input ke arah yang paling menaikkan loss,

$$x' = x + \epsilon \operatorname{sign}(\nabla_x \mathcal{L}(x, y)). \quad (12.7)$$

Projected gradient ascent mengulang langkah ini beberapa kali, dan setiap kali memproyeksikan hasilnya kembali ke bola $\|x' - x\|_\infty \leq \epsilon$ serta ke rentang piksel yang sah. Serangan seperti ini bekerja juga di dunia fisik (Eykholt et al., 2017). Melatih model dengan contoh adversarial membuatnya lebih tahan, dengan harga akurasi yang sedikit turun pada data bersih.

12.4 Sedikit teori

Naskah kuliah ditutup dengan beberapa hasil teoretis yang menjelaskan mengapa semua ini bekerja. Jaringan dalam paling mudah dilatih kalau nilai singular Jacobian input-output mendekati satu, sehingga sinyal maju dan gradien mundur sama-sama tidak menyusut atau meledak. Sifat ini disebut isometri dinamis, dan ia adalah versi matriks dari argumen varians di Bab 7. Pada lebar tak hingga dan laju belajar kecil, pelatihan jaringan berperilaku seperti regresi kernel dengan kernel yang tetap, yaitu *neural tangent kernel* (Jacot et al., 2018). Teori ini menjelaskan konvergensi jaringan yang lebar sekali dan, seperti akan kita lihat di Bab 29, juga kegagalan pelatihan PINN. Sebelum dilatih, keluaran jaringan acak yang lebar mendekati proses Gauss, sebuah hubungan yang menjembatani deep learning dengan statistik Bayesian (Rasmussen and Williams, 2006).

Jaringan biasa hanya memberi satu tebakan, tanpa keterangan seberapa yakin ia. MC dropout menjalankan dropout saat inferensi dan memperlakukan variasi keluarannya sebagai ketidakpastian (Gal and Ghahramani, 2015). Deep ensembles melatih beberapa jaringan dari inisialisasi yang berbeda dan melihat seberapa jauh mereka berbeda pendapat (Lakshminarayanan et al., 2016).

12.5 Di luar kelas

Arsitektur dan model generatif di bab ini menyebar ke hampir semua bidang. U-Net lahir untuk citra biomedis dan tetap menjadi standar segmentasi organ dan tumor (Bab 25). Transfer learning membuat jaringan visi yang dilatih pada foto sehari-hari berguna untuk citra mikroskop dan citra medis yang datanya sedikit. VAE dan model generatif berbasis teks dipakai untuk merancang molekul baru, dan metrik seperti Fréchet ChemNet Distance meminjam cara mengevaluasi GAN untuk kimia (Bab 23). Contoh adversarial mengingatkan bahwa sensor dan sistem persepsi kendaraan otonom bisa diserang (Bab 21).

Di simulasi, untuk medan pada grid teratur, U-Net adalah pilihan bawaan sebagai model pengganti. Ia menggabungkan konteks yang luas, penting karena tekanan di satu titik dipengaruhi seluruh domain, dengan detail lokal seperti lapisan batas. Blok

residual cocok untuk model langkah waktu, karena $\mathbf{u}_{t+1} = \mathbf{u}_t + F(\mathbf{u}_t)$ tidak lain adalah integrator Euler dengan F yang dipelajari.

Encoder sebuah autoencoder bisa memetakan medan aliran ke ruang laten berdimensi kecil. Dinamikanya lalu dipelajari di ruang laten itu, dan decoder mengembalikannya ke medan penuh. Ini salah satu bentuk model tereduksi yang dibahas di Bab 31.

Turbulensi bersifat chaos, sehingga lebih jujur memprediksi distribusi keadaan masa depan daripada satu keadaan tunggal. Model difusi kini dipakai untuk prakiraan cuaca probabilistik (Price et al., 2025) dan untuk simulasi aliran turbulen (Kohl et al., 2023). Model pengganti yang dipakai untuk keputusan teknik juga harus bisa mengatakan bahwa ia tidak yakin, dan ensemble serta model generatif memberi kemampuan itu.

12.6 Perkembangan riset

Model difusi mahal karena membangkitkan sampel butuh puluhan sampai ratusan langkah. Dua arah riset mencoba memotongnya. Rectified flow (Liu et al., 2022) melatih medan kecepatan agar mengikuti garis lurus antara titik noise $\mathbf{x}_0$ dan titik data $\mathbf{x}_1$: untuk $\mathbf{x}_t = (1 - t)\mathbf{x}_0 + t\mathbf{x}_1$, jaringan v_θ dilatih meminimalkan $\mathbb{E}\|v_\theta(\mathbf{x}_t, t) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2$. Jalur yang lurus bisa diintegrasikan dengan sedikit langkah, dan proses “meluruskan” bisa diulang untuk membuat jalurnya makin lurus. Stable Diffusion 3 (Esser et al., 2024) memakai formulasi ini pada skala besar dengan transformer yang memproses token teks dan token gambar dengan bobot terpisah tetapi saling memperhatikan, dan melaporkan bahwa model ini mengikuti hukum penskalaan yang dapat diramalkan.

Consistency models (Song et al., 2023) mengambil jalan lebih radikal. Untuk lintasan ODE yang sama, setiap titik di sepanjang lintasan harus dipetakan ke titik akhir yang sama. Jaringan dilatih menegakkan sifat konsistensi ini, sehingga satu evaluasi jaringan sudah memetakan noise langsung ke data, sementara beberapa langkah tambahan bisa memperbaiki kualitas. Model ini bisa dilatih dengan menyuling model difusi yang sudah ada, atau dari nol. Ketiga pendekatan memperlihatkan bahwa bahasa persamaan diferensial dari Bab 15 menjadi bahasa bersama model generatif modern.

CATATAN SUMBER. Bab ini mengikuti bagian kedua naskah kuliah Deep Learning and Neural Nets II, yang kini bernama Deep Learning: Architectures and Generative Techniques (arsitektur lanjut, model generatif, teori deep learning, optimasi orde tinggi, Bayesian deep learning), beserta enam tugas latihannya. Bacaan pendamping: Goodfellow et al. (2016), Prince (2023), dan Kingma and Welling (2019).

13 Geometri dan Simetri

Kuliah geometric deep learning datang di semester terakhir, pada musim panas 2026. Materinya mengikuti kursus Geometric Deep Learning dari Oxford yang dikembangkan Michael Bronstein dan rekan-rekannya, dengan buku proto karya Bronstein et al. (2021) sebagai acuan. Kuliah-kuliah sebelumnya membahas arsitektur satu per satu. Kuliah ini menunjukkan bahwa konvolusi, DeepSets, jaringan graf, dan transformer adalah anak-anak dari satu resep yang sama.

Resep itu bertumpu pada satu pengamatan. Di Bab 1 kita melihat bahwa belajar di dimensi tinggi hampir mustahil tanpa asumsi. Asumsi yang paling berharga ternyata adalah simetri: kita sering tahu transformasi apa yang tidak mengubah jawaban. Kucing yang digeser tetap kucing. Molekul yang diputar tetap molekul yang sama. Urutan penomoran atom dalam berkas tidak mengubah sifat kimianya. Kalau pengetahuan seperti ini ditanamkan ke dalam arsitektur, ruang yang harus dicari menyusut drastis.

13.1 Sinyal, domain, dan grup

Bahasa geometric deep learning dimulai dengan memisahkan dua hal: domain dan sinyal. Domain Ω adalah tempat data hidup, misalnya grid piksel, himpunan titik, graf, atau permukaan. Sinyal adalah fungsi $x : \Omega \rightarrow \mathbb{R}^c$ yang memberi setiap titik domain sebuah vektor fitur, misalnya warna untuk setiap piksel atau sifat kimia untuk setiap atom. Ruang semua sinyal pada sebuah domain, $X(\Omega)$, adalah ruang vektor: dua sinyal bisa dijumlahkan dan dikalikan skalar titik demi titik. Struktur linear inilah yang memungkinkan kita berbicara tentang lapisan linear.

Simetri dirumuskan dengan *grup*. Grup adalah himpunan G dengan operasi komposisi yang memenuhi empat aksioma: hasil komposisi dua anggota tetap di dalam G , komposisinya asosiatif, ada anggota identitas, dan setiap anggota punya invers. Rotasi bidang membentuk grup, begitu pula pergeseran grid dan permutasi himpunan. Dua grup bisa memiliki struktur yang sama walau anggotanya berbeda, dan kesamaan ini dinyatakan dengan *homomorphism*, pemetaan φ yang menghormati komposisi, $\varphi(gh) = \varphi(g)\varphi(h)$. Determinan adalah contohnya: ia memetakan matriks yang bisa dibalik ke bilangan real tak nol, dan $\det(AB) = \det A \det B$. Fungsi eksponensial memberi contoh yang lebih menarik, karena ia memetakan bilangan real dengan penjumlahan ke bilangan positif dengan perkalian, $e^{a+b} = e^a e^b$, dan bisa dibalik dengan logaritma. Homomorfisme yang bisa dibalik disebut isomorfisme, dan dua grup yang isomorfik secara struktural sama.

Grup bekerja pada domain melalui *group action*: setiap g memindahkan titik u ke $g.u$, dengan identitas tidak memindahkan apa pun dan aksi dua anggota berturut-turut sama dengan aksi komposisinya. Dari aksi pada domain, kita mendapat aksi pada

sinyal,

$$(\rho(g)x)(u) = x(g^{-1}.u). \quad (13.1)$$

Pemetaan $g \mapsto \rho(g)$ ke operator linear pada ruang sinyal disebut *representasi*. Invers pada Persamaan (13.1) bukan hiasan. Untuk menggeser sebuah gambar ke kanan, nilai piksel di posisi baru diambil dari posisi lama di sebelah kirinya. Tanpa invers, komposisi dua pergeseran akan keluar dengan urutan terbalik, dan ρ tidak lagi menjadi homomorfisme.

13.2 Invariansi dan ekuivariansi

Misalkan ρ dan ρ' representasi grup yang sama pada ruang input dan ruang output. Fungsi f disebut *invariant* kalau $f(\rho(g)x) = f(x)$ untuk setiap g , dan *equivariant* kalau

$$f(\rho(g)x) = \rho'(g)f(x) \quad \text{untuk setiap } g \in G. \quad (13.2)$$

Klasifikasi gambar seharusnya invarian terhadap pergeseran: labelnya tidak berubah. Segmentasi seharusnya ekuivarian: kalau gambar digeser, peta segmentasinya ikut bergeser. Secara visual, ekuivariansi berarti diagram berikut komut: menerapkan transformasi lalu f memberi hasil yang sama dengan menerapkan f lalu transformasi padanannya. Sinyal-sinyal yang hanya berbeda karena transformasi grup membentuk sebuah *orbit*. Peta ekuivarian memetakan setiap orbit di ruang input ke sebuah orbit di ruang fitur, sehingga struktur simetrinya terbawa utuh.

Ada cara umum untuk membuat sembarang fungsi menjadi invarian terhadap grup yang terbatas, yaitu merata-ratakannya atas seluruh grup,

$$(S_G f)(x) = \frac{1}{|G|} \sum_{g \in G} f(\rho(g)x). \quad (13.3)$$

Operator ini kadang disebut operator simetrisasi. Ambil himpunan dua elemen dan fungsi $f(x_1, x_2) = x_1$ yang jelas tidak peduli pada simetri. Rata-rata atas dua permutasi memberi $(x_1 + x_2)/2$, yang invarian. Kalau fungsi target memang invarian, membatasi diri pada kelas fungsi invarian tidak menambah galat aproksimasi, karena targetnya masih ada di dalam kelas. Galat estimasi justru bisa turun, karena setiap contoh data kini “mewakili” seluruh orbitnya. Hanya saja, kalau grupnya kecil dibanding dimensi masalah, galat estimasi tetap bisa terkena kutukan dimensi. Simetri membantu, tetapi tidak membebaskan sepenuhnya.

13.3 Simetri sebagai penghemat data

Tugas pertama kuliah ini membuat kutukan dimensi dari Bab 1 terasa dengan angka. Untuk dua titik acak di hiperkubus satuan $[0, 1]^n$, setiap koordinat menyumbang selisih kuadrat dengan rata-rata $1/6$, sehingga rata-rata kuadrat jaraknya $n/6$. Jarak tumbuh seperti $\sqrt{n}$, sementara sebarannya relatif terhadap rata-rata justru menyempit, sehingga semua pasangan titik tampak sama jauhnya.

Tugas itu juga memperlihatkan apa yang dilakukan fitur yang tidak berguna. Dua kelas yang terpisah dengan baik dalam dua fitur, misalnya dengan kuadrat jarak

antarkelas sekitar 4 dan dalam kelas sekitar 1, menjadi sulit dipisahkan begitu ditambah fitur derau. Setiap fitur derau dengan simpangan baku σ menambah rata-rata $2\sigma^2$ pada kuadrat jarak *setiap* pasangan, sama untuk pasangan sekelas maupun beda kelas. Seratus fitur derau dengan $\sigma = 0,5$ menambah 50 pada keduanya, sehingga rasio jarak beda kelas terhadap jarak sekelas turun dari 4 menjadi $54/51 \approx 1,06$. Tetangga terdekat praktis menjadi acak. Inilah sebabnya k -nearest neighbour dan pengelompokan peka sekali terhadap pemilihan fitur (Bab 6).

Bagian kedua tugas memakai integrasi Monte Carlo (Bab 28) untuk fungsi berupa gumpalan Gauss yang tersusun simetris di dalam hiperkubus, dan meminta kami memanfaatkan simetrinya. Kalau fungsi tidak berubah ketika setiap koordinat dicerminkan dan ketika koordinat-koordinat ditukar urutannya, integralnya atas seluruh kubus sama dengan integral atas satu “potongan dasar” dikali jumlah transformasi itu. Dalam d dimensi ada 2^d pencerminan dan $d!$ permutasi. Untuk $d = 8$, itu $2^8 \cdot 8! \approx 10^7$ potongan yang identik. Sampel cukup ditarik dari satu potongan, yang volumenya sepersepuluh juta volume kubus, tanpa kehilangan informasi apa pun.

Pelajaran yang sama berlaku untuk belajar, bukan hanya untuk integrasi. Model yang invarian terhadap sebuah grup tidak perlu melihat setiap versi yang ditransformasi dari setiap contoh, karena satu contoh sudah mewakili seluruh orbitnya. Augmentasi data mencoba mencapai hal yang sama dengan memperlihatkan versi-versi itu secara eksplisit. Arsitektur yang ekuivarian mencapainya dengan konstruksi, dan untuk grup yang besar, selisihnya bisa berorde jutaan, seperti contoh di atas.

13.4 Skala dan cetak biru

Simetri saja belum cukup. Prior kedua adalah *scale separation*: banyak sinyal bisa dipahami secara bertingkat. Detail lokal memengaruhi detail lokal di sekitarnya, dan struktur besar bisa diringkas dari versi sinyal yang lebih kasar. Dua bahan dasarnya adalah operator lokal yang bekerja di lingkungan kecil, dan operasi *coarse graining* yang meringkas sinyal ke domain yang lebih kasar, seperti pooling pada gambar. Gabungan simetri dan skala lebih kuat dari masing-masing, karena fungsi yang bekerja secara lokal dan bertingkat hanya perlu belajar interaksi jarak pendek di setiap tingkat.

Dari kedua prior ini, Bronstein et al. (2021) merumuskan sebuah cetak biru. Jaringan disusun dari lapisan linear yang ekuivarian terhadap G (sebut B), nonlinearitas yang diterapkan titik demi titik (σ), pooling lokal yang mengasarkan domain (P), dan di ujungnya pooling global yang invarian (A), sehingga

$$f = A \circ \sigma \circ B_J \circ P_{J-1} \circ \cdots \circ P_1 \circ \sigma \circ B_1. \quad (13.4)$$

CNN adalah cetak biru ini pada grid dengan grup translasi. DeepSets adalah cetak biru pada himpunan dengan grup permutasi. Jaringan graf adalah cetak biru pada graf. Kita akan melihat ketiganya satu per satu.

Ada satu kehalusan lagi. Gambar nyata jarang bertransformasi persis menurut grup. Wajah yang sedikit tersenyum bukan rotasi atau pergeseran dari wajah yang datar, melainkan deformasi kecil. Representasi yang baik sebaiknya *hampir* invarian terhadap transformasi yang *hampir* berada di dalam grup: perubahan keluarannya sebanding

dengan seberapa jauh deformasi itu dari transformasi grup. Sifat ini disebut stabilitas terhadap deformasi, dan ia menjadi ukuran kualitas representasi yang akan muncul lagi pada wavelet dan pada filter spektral.

13.5 Himpunan

Domain paling sederhana adalah himpunan tanpa struktur. Simetrinya adalah grup permutasi: penomoran elemen tidak boleh berpengaruh. Sinyal pada himpunan n elemen bisa ditulis sebagai matriks fitur $\mathbf{X} \in \mathbb{R}^{n \times c}$, satu baris per elemen. Kalau elemen dipermutasi, baris-baris matriks ikut dipermutasi, $\mathbf{X} \mapsto \mathbf{P}\mathbf{X}$, dengan $\mathbf{P}$ matriks permutasi yang berisi tepat satu angka satu di setiap baris dan kolom.

Peta linear yang ekuivarian terhadap semua permutasi ternyata hanya punya dua derajat kebebasan per pasangan kanal: satu bagian yang bekerja pada setiap elemen sendiri, dan satu bagian yang melihat jumlah seluruh elemen,

$$F(\mathbf{X}) = \mathbf{X}\mathbf{\Lambda} + \frac{1}{n}\mathbf{1}\mathbf{1}^\top \mathbf{X}\mathbf{\Gamma}. \quad (13.5)$$

DeepSets (Zaheer et al., 2017) membangun fungsi invarian dari bagian-bagian ini: $f(\mathbf{X}) = \phi(\sum_i \psi(x_i))$. Fungsi ψ dipakai bersama untuk setiap elemen dan bersifat ekuivarian, penjumlahan adalah pooling yang invarian, dan ϕ bekerja pada ringkasan global. Kedua fungsi dipelajari, sedangkan penjumlahannya tetap.

13.6 Graf

Graf menambahkan satu bahan ke himpunan: hubungan antar elemen. Graf tak berarah dengan atribut simpul bisa ditulis sebagai matriks fitur $\mathbf{X}$ dan matriks ketetanggaan $\mathbf{A}$, dengan $a_{uv} = 1$ kalau simpul u dan v terhubung. Grup simetrinya tetap permutasi, tetapi sekarang permutasi bekerja pada keduanya: $\mathbf{X} \mapsto \mathbf{P}\mathbf{X}$ dan $\mathbf{A} \mapsto \mathbf{P}\mathbf{A}\mathbf{P}^\top$. Fungsi tingkat graf harus invarian, $f(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^\top) = f(\mathbf{X}, \mathbf{A})$, dan fungsi tingkat simpul harus ekuivarian, $F(\mathbf{P}\mathbf{X}, \mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P}F(\mathbf{X}, \mathbf{A})$.

13.6.1 Message passing

Graph neural network modern hampir semuanya mengikuti pola *message passing* (Battaglia et al., 2018; Gilmer et al., 2017). Setiap simpul u mengumpulkan pesan dari tetangganya, lalu memperbarui keadaannya:

$$\mathbf{h}_u = \phi\left(\mathbf{x}_u, \bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_u, \mathbf{x}_v)\right), \quad (13.6)$$

dengan $\bigoplus$ operator agregasi yang tidak peduli urutan, seperti jumlah, rata-rata, atau maksimum. Bedanya dengan DeepSets ada di sini: setiap simpul tidak lagi meringkas seluruh himpunan, tetapi hanya tetangganya, sehingga sisi graf menentukan siapa berbicara dengan siapa.

Pembagian yang dipakai di kuliah membedakan tiga rasa. Pada varian konvolusional, pesan dari tetangga dikalikan bobot tetap yang bergantung pada struktur graf, seperti pada GCN (Kipf and Welling, 2016). Pada varian attentional, bobotnya dihitung dari

fitur kedua simpul, seperti pada GAT (Veličković et al., 2017). Pada varian message passing yang paling umum, pesan ψ adalah fungsi sembarang dari kedua simpul, dan kedua varian sebelumnya adalah kasus khususnya.

Sebagai contoh kecil, ambil graf garis dengan tiga simpul, 1–2–3, dengan fitur skalar $x = (1, 2, 3)$. Dengan aturan $h_u = x_u + \sum_{v \in \mathcal{N}_u} x_v$, simpul pertama mendapat $1 + 2 = 3$, simpul kedua $2 + 1 + 3 = 6$, dan simpul ketiga $3 + 2 = 5$. Kalau penomoran dibalik menjadi 3–2–1, fitur menjadi $(3, 2, 1)$ dan hasilnya $(5, 6, 3)$, yaitu hasil semula dalam urutan baru. Itulah ekuivariansi.

Contoh ini bukan kebetulan. Dalam bentuk matriks, aturan tadi adalah $h = (I + A)x$. Kalau simpul dipermutasi dengan P , input menjadi Px dan ketetanggaan menjadi PAP^T , sehingga keluarannya $(I + PAP^T)Px = Px + PAx = P(I + A)x = Ph$, karena $P^T P = I$. Argumen yang sama berlaku untuk message passing umum: fungsi ψ dan ϕ yang dipakai bersama di semua simpul tidak peduli nomor simpulnya, dan agregasi atas tetangga tidak peduli urutan tetangganya. Satu-satunya hal yang bisa membuat jaringan bergantung pada penomoran adalah memberi setiap simpul bobot pribadinya sendiri, dan itulah yang dihindari.

Teman yang mengabarkan berita di sebuah desa adalah gambaran yang cocok. Setiap pagi, setiap orang mendengar kabar dari tetangga sebelah rumahnya dan memperbarui pengetahuannya. Setelah satu pagi, kabar dari rumah sebelah sudah sampai. Setelah tiga pagi, kabar dari rumah yang berjarak tiga sudah sampai. Satu lapisan message passing adalah satu pagi.

13.6.2 Seberapa kuat jaringan graf?

Bisakah jaringan graf membedakan dua graf yang tidak isomorfik? Pertanyaan ini terhubung dengan uji klasik *Weisfeiler–Leman* (WL). Uji ini mewarnai setiap simpul, lalu berulang kali memberi warna baru berdasarkan warna lama simpul itu dan multihimpunan warna tetangganya. Kalau histogram warna dua graf berbeda, graf itu pasti tidak isomorfik. Kalau sama, kita tidak bisa menyimpulkan apa-apa. Xu et al. (2018) dan Morris et al. (2018) menunjukkan bahwa message passing paling banter sekuat WL, dan mencapai kekuatan itu kalau agregasinya injektif terhadap multihimpunan. Penjumlahan memenuhi syarat ini, sedangkan rata-rata dan maksimum tidak, karena keduanya tidak bisa membedakan, misalnya, satu tetangga berfitur satu dari dua tetangga berfitur satu.

Batas WL mudah diperlihatkan. Bandingkan dua segitiga yang terpisah dengan satu lingkaran enam simpul. Keduanya punya enam simpul, dan setiap simpul punya tepat dua tetangga. Pada setiap iterasi WL, semua simpul menerima informasi yang sama, sehingga semua warna tetap identik dan uji tidak bisa membedakan keduanya, padahal yang satu berisi segitiga dan yang lain tidak. Jaringan message passing biasa pun gagal di sini.

Ada beberapa cara melampaui batas ini. *Positional encoding* memberi setiap simpul “alamat”, misalnya dari vektor eigen Laplacian graf atau dari langkah acak, sehingga simpul yang strukturnya sama tetap bisa dibedakan. Jaringan substruktur menghitung kemunculan pola kecil, seperti segitiga atau siklus, di sekitar setiap simpul. Subgraph

GNN menjalankan jaringan pada banyak versi graf yang sedikit diubah, misalnya dengan satu simpul dihapus, lalu menggabungkan hasilnya. Gagasan ini berkerabat dengan *reconstruction conjecture*, dugaan lama dalam teori graf bahwa sebuah graf dengan setidaknya tiga simpul ditentukan sepenuhnya oleh kumpulan subgraf yang masing-masing kehilangan satu simpul.

Transformer pun bisa dilihat sebagai message passing pada graf lengkap, di mana setiap token adalah simpul yang menerima pesan dari semua simpul lain dengan bobot attention. Tanpa positional encoding, attention memang ekuivarian terhadap permutasi. Dengan positional encoding seperti yang dipakai dalam praktik, sifat itu sengaja dilepas, karena urutan kata memang bermakna.

13.6.3 Oversquashing

Untuk menangkap interaksi jarak jauh, kita menambah lapisan. Tetapi jumlah simpul dalam radius r bisa tumbuh secara eksponensial, sementara semua informasinya harus dimampatkan ke vektor berukuran tetap. Gejala ini disebut *oversquashing* (Alon and Yahav, 2020). Secara formal, kepekaan keadaan simpul u setelah r lapisan terhadap fitur awal simpul v , $\|\partial \mathbf{h}_u^{(r)} / \partial \mathbf{x}_v\|$, dibatasi oleh entri matriks ketetanggaan ternormalisasi yang dipangkatkan, $(\hat{\mathbf{A}}^r)_{uv}$, dikalikan konstanta yang bergantung pada bobot dan nonlinearitas. Lebar jaringan dan kedalaman ikut berperan, tetapi faktor yang paling menentukan adalah topologi graf: leher botol, yaitu sedikit sisi yang menghubungkan dua kelompok besar, dan daerah dengan kelengkungan negatif (Topping et al., 2021). Kabar dari desa sebelah harus lewat satu jembatan sempit, dan di jembatan itu sebagian besar berita hilang.

13.7 Grid

Grid adalah kasus yang paling kita kenal. Grid satu dimensi bisa dilihat sebagai graf berbentuk lingkaran, kalau tepinya disambung secara periodik. Grup simetrinya adalah pergeseran siklik. Gambar *I* yang digeser satu piksel ke kanan bisa ditulis $I' = \mathbf{AIB}$, dengan $\mathbf{A}$ matriks identitas karena baris tidak berubah, dan $\mathbf{B}$ matriks pergeseran siklik yang memindahkan setiap kolom satu tempat. Pergeseran ke bawah memakai matriks pergeseran di sisi kiri.

Peta linear yang ekuivarian terhadap pergeseran adalah tepat peta yang komut dengan operator geser S , yaitu $CS = SC$. Matriks seperti ini adalah *circulant matrix*: setiap barisnya adalah baris sebelumnya yang digeser satu tempat. Mengalikan dengan matriks sirkulan sama dengan konvolusi. Inilah alasan mendalam mengapa konvolusi muncul di CNN: ia bukan sekadar pilihan desain, melainkan satu-satunya operasi linear yang menghormati simetri geser. Semua matriks sirkulan punya vektor eigen yang sama, yaitu basis Fourier, sehingga transformasi Fourier diskret mendiagonalkan semuanya sekaligus. Akibatnya, konvolusi di ruang menjadi perkalian titik demi titik di frekuensi, dan dengan FFT (Cooley and Tukey, 1965), konvolusi panjang bisa dihitung dalam $O(n \log n)$ operasi.

Fourier punya keterbatasan. Basisnya tersebar di seluruh ruang, sehingga lokalisasi sempurna di frekuensi berarti tidak ada lokalisasi di ruang. Prinsip ketidakpastian

membatasi seberapa sempit sebuah fungsi bisa sekaligus di kedua domain. Transformasi Fourier berjendela memotong sinyal menjadi potongan pendek, tetapi lebar jendelanya sama untuk semua frekuensi. Lebih penting lagi, modulus Fourier tidak stabil terhadap dilatasi kecil. Meregangkan sinyal sedikit menggeser frekuensi rendah sedikit, tetapi menggeser frekuensi tinggi jauh, sehingga dua sinyal yang hampir sama bisa punya spektrum frekuensi tinggi yang berbeda.

Wavelet menyelesaikan ini dengan memakai satu fungsi induk yang digeser dan diskalakan. Di frekuensi tinggi, wavelet sempit di ruang dan lebar di frekuensi. Di frekuensi rendah, sebaliknya. Wavelet Haar, misalnya, adalah tangga naik-turun yang cocok sekali untuk mendeteksi tepi dan struktur lokal. *Scattering transform* (Mallat, 2012) menumpuk tiga bahan: konvolusi wavelet, modulus, dan rata-rata. Modulus adalah nonlinearitas yang krusial. Tanpanya, rata-rata dari koefisien wavelet akan bernilai nol, karena wavelet berosilasi. Modulus memindahkan energi frekuensi tinggi ke frekuensi rendah, sehingga informasinya selamat dari rata-rata. Hasilnya adalah representasi yang invarian terhadap pergeseran dan stabil terhadap deformasi, sebuah CNN yang filternya tidak dipelajari tetapi dirancang, dan dalam banyak hal menjadi model teoretis untuk memahami CNN.

13.8 Konvolusi grup

Konvolusi bisa dipandang sebagai “transformasikan lalu cocokkan”: geser filter ke setiap posisi, lalu ukur kecocokannya dengan sinyal. Kalau grupnya lebih besar dari pergeseran, misalnya pergeseran ditambah rotasi, kita bisa melakukan hal yang sama untuk setiap anggota grup (Cohen and Welling, 2016):

$$(x \star \psi)(g) = \int_{\Omega} x(u) \psi(g^{-1} \cdot u) du. \quad (13.7)$$

Keluarannya kini adalah fungsi pada grup, bukan pada domain asal. Pada grup p_4 , yang terdiri atas pergeseran dan empat rotasi kelipatan sembilan puluh derajat, lapisan pertama mengubah gambar menjadi empat peta fitur, satu untuk setiap orientasi filter. Lapisan berikutnya bekerja pada fungsi di grup itu sendiri, sehingga filternya juga punya dimensi orientasi. Kalau gambar input diputar sembilan puluh derajat, setiap peta fitur ikut berputar, dan keempat orientasi bergeser secara siklis.

Konsep pendukungnya adalah *homogeneous space*. Grup bekerja secara transitif pada sebuah domain kalau setiap titik bisa dicapai dari titik lain oleh suatu anggota grup. Stabilizer sebuah titik adalah subgrup yang membiarkan titik itu di tempatnya. Bola S^2 di bawah rotasi tiga dimensi adalah contoh homogeneous space, dan stabilizer kutub utaranya adalah rotasi di sekitar sumbu yang melewati kutub. Bidang di bawah gerak kaku, yaitu pergeseran dan rotasi, adalah contoh lainnya, dengan stabilizer setiap titik berupa rotasi di sekitar titik itu.

13.9 Manifold dan mesh

Banyak data hidup di permukaan melengkung: tubuh manusia, sayap pesawat, permukaan bumi. Manifold adalah model matematisnya. Secara informal, manifold adalah

ruang yang di sekitar setiap titik tampak seperti ruang Euklides. *Chart* adalah peta lokal yang meratakan sepotong manifold, seperti halaman atlas yang meratakan sepotong bumi. Di setiap titik ada *tangent space*, ruang semua arah yang mungkin di titik itu. Untuk mengukur panjang dan sudut vektor singgung, kita butuh *Riemannian metric*, sebuah hasil kali dalam yang bervariasi mulus dari titik ke titik. Dengan metrik, kita bisa mendefinisikan panjang kurva, *geodesic* sebagai jalur terpendek, dan jarak geodesik.

Besaran yang hanya bergantung pada metrik disebut intrinsik. Ia tidak berubah kalau permukaan ditekuk tanpa diregangkan. Selebar kertas yang digulung menjadi silinder punya geometri intrinsik yang sama dengan kertas datar, sehingga jarak geodesik antara dua titik di atasnya tidak berubah. Deformasi yang mempertahankan metrik disebut isometri. Berbeda dari homogeneous space, manifold umumnya tidak punya grup simetri global, sehingga stabilitas terhadap deformasi menjadi lebih penting daripada ekuivariansi yang persis.

Kalkulus pada manifold bertumpu pada tiga operator. Gradien ∇f sebuah medan skalar menunjuk ke arah kenaikan tercepat dan didefinisikan melalui metrik, sehingga intrinsik. Divergensi sebuah medan vektor mengukur sumber dan sumur lokal. Keduanya saling adjoint, $\langle \nabla f, X \rangle = -\langle f, \operatorname{div} X \rangle$, persis seperti integrasi parsial. Gabungannya adalah *Laplace–Beltrami operator*, $\Delta f = -\operatorname{div} \nabla f$, yang mengukur seberapa jauh nilai fungsi di sebuah titik berbeda dari rata-rata di sekitarnya. Laplacian ini simetris dan semidefinit positif, sehingga punya basis fungsi eigen yang ortogonal, generalisasi alami dari basis Fourier.

Dengan basis itu, konvolusi bisa didefinisikan secara spektral: transformasikan sinyal ke basis eigen Laplacian, kalikan dengan filter di setiap frekuensi, lalu kembalikan (Bruna et al., 2013). Konvolusi spektral ini intrinsik, tetapi naif. Basis eigen bisa berubah drastis karena deformasi kecil, terutama untuk frekuensi tinggi, sehingga filter yang dipelajari pada satu bentuk tidak berpindah ke bentuk lain. Jalan keluarnya adalah filter yang mulus terhadap frekuensi, misalnya polinomial dalam Laplacian. Filter polinomial berderajat K hanya melihat tetangga dalam radius K , sehingga lokal dan stabil, dan ChebNet (Defferrard et al., 2016) memakai polinomial Chebyshev untuk tujuan ini.

Dalam praktik, manifold didiskretisasi menjadi mesh segitiga atau graf. Laplacian graf yang paling sederhana adalah $L = D - A$, dengan D matriks diagonal derajat simpul, sehingga

$$(Lx)_u = \sum_{v \in \mathcal{N}_u} a_{uv} (x_u - x_v). \quad (13.8)$$

Pada mesh, bobot a_{uv} dipilih dari geometri segitiga, misalnya dengan rumus kotangen, agar Laplacian diskret mendekati Laplace–Beltrami yang kontinu. Bentuk di atas sudah memperlihatkan bahwa Laplacian adalah sebuah langkah message passing: setiap simpul menjumlahkan selisih dirinya dengan tetangganya. Konvolusi spektral, jaringan graf, dan operator diferensial pada permukaan ternyata adalah satu keluarga.

13.10 Simetri di luar kelas

Cetak biru ini muncul di banyak bidang dengan nama yang berbeda. Molekul adalah graf atom, dan sifatnya tidak berubah kalau molekul diputar atau atom-atomnya

dinomori ulang, sehingga jaringan graf dan jaringan ekuivarian menjadi standar dalam kimia komputasi (Bab 23). AlphaFold menempatkan residu protein sebagai benda kaku di ruang dengan attention yang tidak peduli rotasi global (Bab 24). Awan titik lidar pada kendaraan otonom adalah himpunan tanpa urutan, dan PointNet adalah DeepSets untuk awan titik (Bab 21). Pengguna dan item dalam sistem rekomendasi membentuk graf bipartit, dan message passing di atasnya menyebarkan selera (Bab 34).

Bagi simulasi, bab ini barangkali yang paling penting di Bagian II. Hukum fisika punya simetri: persamaan Navier–Stokes tidak berubah kalau sistem digeser atau diputar, dan gaya antar partikel tidak bergantung pada urutan penomoran partikel. Model pengganti yang tidak menghormati simetri ini harus mempelajarinya dari data, dan sering gagal di luar data latihnya. Jaringan graf yang ekuivarian terhadap rotasi dan translasi tiga dimensi, seperti EGNN (Satorras et al., 2021), membangunnya langsung ke dalam arsitektur.

Mesh simulasi pada dasarnya adalah graf. Metode volume hingga, yang akan kita bahas di Bab 27, menghitung perubahan di setiap sel dari fluks yang melewati sisi-sisinya ke sel tetangga. Bentuknya persis message passing: setiap sel mengumpulkan “pesan” fluks dari tetangganya, lalu memperbarui keadaannya. Tidak mengherankan bahwa simulator yang dipelajari dengan jaringan graf menjadi salah satu keberhasilan terbesar di bidang ini. Graph Network-based Simulator (Sanchez-Gonzalez et al., 2020) meniru dinamika partikel cairan dan pasir, MeshGraphNets (Pfaff et al., 2020) bekerja pada mesh simulasi aerodinamika dan struktur, dan Message Passing Neural PDE Solvers (Brandstetter et al., 2022) memakai message passing sebagai solver PDE umum. GraphCast (Lam et al., 2023) membawa gagasan yang sama ke prakiraan cuaca global, dengan graf di atas mesh ikosahedral yang membungkus bumi. Semuanya dibahas lebih dalam di Bab 30.

Laplacian graf juga punya makna fisik yang langsung. Persamaan difusi panas pada graf, $\dot{x} = -\kappa Lx$, didiskretisasi dengan Euler eksplisit menjadi $x^{n+1} = x^n - \kappa \Delta t Lx^n$. Satu langkah waktu difusi adalah satu lapisan message passing dengan bobot yang sudah ditentukan. Pandangan ini juga menjelaskan gejala *oversmoothing* pada jaringan graf yang terlalu dalam: seperti panas yang lama-lama merata, fitur semua simpul menjadi semakin mirip.

13.11 Perkembangan riset

Di dunia atom dan molekul, jaringan ekuivarian kini menjadi alat standar. MACE (Batatia et al., 2022) memakai pesan dengan orde badan yang lebih tinggi: alih-alih hanya pasangan atom, setiap pesan sudah memuat interaksi sampai empat atom sekaligus. Dengan begitu, dua langkah message passing sudah cukup, sehingga model lebih cepat dan lebih mudah diparalelkan, dengan akurasi medan gaya yang setara atau lebih baik dari model sebelumnya. Equiformer (Liao and Smidt, 2022) membawa transformer ke graf atom tiga dimensi dengan mengganti operasi transformer dengan versi ekuivarian berbasis representasi tak tereduksi dan hasil kali tensor, sesuai teori grup di awal bab ini.

Untuk graf umum, GraphGPS (Rampášek et al., 2022) memisahkan dua mekanisme:

message passing lokal di sepanjang sisi graf, dan attention global antar semua simpul, ditambah encoding posisi dan struktur. Pemisahan ini membuat biayanya linear terhadap jumlah simpul dan sisi, sekaligus menjadi salah satu jawaban atas masalah oversquashing, karena informasi bisa melompat lewat attention global. Geometric Algebra Transformer (Brehmer et al., 2023) mewakili titik, arah, translasi, dan rotasi sebagai elemen aljabar Clifford proyektif berdimensi 16, sehingga satu arsitektur ekuivarian terhadap $E(3)$ bisa menangani banyak jenis objek geometris. Contoh penerapannya mencakup simulasi banyak benda, perencanaan gerak robot, dan estimasi tegangan geser dinding pada mesh arteri, sebuah masalah aliran darah yang menyambung langsung ke Bab 30.

CATATAN SUMBER. Bab ini mengikuti kuliah Deep Learning: Geometric Techniques, yang memakai materi kursus Geometric Deep Learning dari Oxford dan daftar topik ujiannya: kutukan dimensi, prior geometris, himpunan, graf, grid, konvolusi grup, manifold, dan konvolusi spektral. Acuan utama adalah Bronstein et al. (2021), dengan latar belakang di Bronstein et al. (2017).

14 Bernalar dalam Ketidakpastian

Kuliah model probabilistik saya ikuti pada musim dingin 2025/26. Slidenya banyak bersandar pada buku Koller and Friedman (2009) dan Russell and Norvig (2021). Bab ini mengikuti urutan kuliahnya: representasi, inferensi, pembelajaran, lalu model yang berjalan dalam waktu.

Bab-bab sebelumnya kebanyakan membahas fungsi yang memetakan input ke output. Bab ini membahas cara lain memandang dunia: sebagai kumpulan variabel acak yang saling bergantung. Kita tidak lagi bertanya “apa jawabannya?”, tetapi “seberapa yakin kita pada setiap kemungkinan jawaban, dan bagaimana keyakinan itu berubah ketika ada bukti baru?”.

14.1 Mengapa butuh struktur

Distribusi gabungan atas n variabel biner butuh $2^n - 1$ angka. Untuk dua puluh variabel, itu sekitar sejuta angka, dan untuk tiga puluh variabel, satu miliar. Tidak ada yang bisa menyimpan, apalagi mempelajari, tabel sebesar itu. Jalan keluarnya adalah *conditional independence*. Variabel X dan Y independen bersyarat terhadap Z kalau, setelah Z diketahui, mengetahui Y tidak lagi menambah informasi tentang X : $p(X | Y, Z) = p(X | Z)$. Petir dan suara guntur sangat berkorelasi, tetapi keduanya disebabkan oleh peristiwa yang sama. Kalau kita sudah tahu ada sambaran listrik di awan, mendengar guntur tidak menambah informasi tentang kilatnya.

Aturan rantai probabilitas selalu berlaku: $p(x_1, \dots, x_n) = \prod_i p(x_i | x_1, \dots, x_{i-1})$. Independensi bersyarat memangkas setiap faktor sehingga hanya bergantung pada beberapa variabel.

14.2 Bayesian network

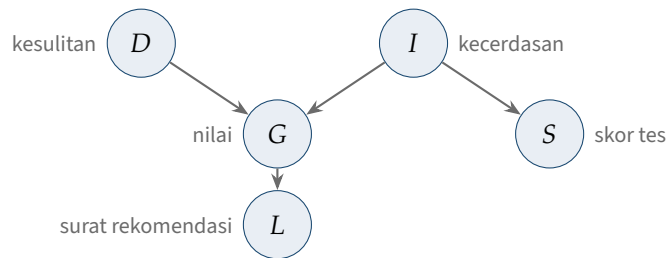
Bayesian network adalah graf berarah tanpa siklus yang setiap simpulnya adalah variabel acak. Setiap variabel diberi distribusi bersyarat terhadap induk-induknya, dan distribusi gabungannya adalah hasil kali semuanya:

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | \text{pa}(x_i)). \quad (14.1)$$

Kalau setiap variabel biner punya paling banyak k induk, jaringan butuh paling banyak $n 2^k$ angka. Dengan $n = 20$ dan $k = 3$, itu 160 angka, bukan sejuta.

Contoh yang dipakai banyak buku teks, termasuk Koller and Friedman (2009), adalah seorang mahasiswa. Nilai ujiannya, G , bergantung pada tingkat kesulitan mata kuliah, D , dan kecerdasan mahasiswa, I . Kecerdasan juga memengaruhi skor tes masuk, S , dan nilai memengaruhi kualitas surat rekomendasi, L . Graf ini memungkinkan

beberapa pola penalaran. Penalaran kausal berjalan searah panah: mahasiswa cerdas cenderung mendapat nilai baik. Penalaran evidensial berjalan melawan panah: nilai buruk menurunkan keyakinan bahwa mahasiswanya cerdas. Pola yang paling menarik adalah penalaran antarsebab. Nilai buruk menaikkan keyakinan bahwa mata kuliahnya sulit dan menurunkan keyakinan bahwa mahasiswanya cerdas. Kalau kemudian kita tahu mata kuliahnya memang sulit, nilai buruk itu sudah “terjelaskan”, dan keyakinan tentang kecerdasan mahasiswa naik kembali. Pola ini disebut *explaining away*. Bayesian network menangani semua pola ini dengan satu algoritme inferensi yang sama, tanpa aturan khusus untuk masing-masing (Gambar 14.1).



GAMBAR 14.1. Jaringan mahasiswa. Distribusi gabungannya $p(D, I, G, S, L) = p(D)p(I)p(G | D, I)p(S | I)p(L | G)$.

Explaining away paling mudah dilihat dengan angka pada contoh yang lebih kecil. Rumput basah (W) bisa disebabkan hujan (R) atau penyiram (S), yang saling independen dengan $p(R) = 0,2$ dan $p(S) = 0,1$, dan rumput basah tepat ketika salah satunya terjadi. Peluang rumput basah adalah $1 - 0,8 \cdot 0,9 = 0,28$. Begitu kita melihat rumput basah, peluang hujan naik dari 0,2 menjadi $p(R | W) = 0,2/0,28 \approx 0,71$. Lalu kita melihat penyiram sedang menyala. Penyiram sudah cukup menjelaskan rumput basah, sehingga rumput basah tidak lagi memberi informasi tentang hujan, dan $p(R | W, S)$ kembali ke 0,2. Dua sebab yang semula independen menjadi saling bergantung begitu akibat bersamanya diamati.

14.2.1 Membaca independensi dari graf

Kapan dua variabel independen bersyarat bisa dibaca langsung dari bentuk graf. Ada tiga pola dasar untuk jalur $X - Z - Y$. Pada rantai $X \rightarrow Z \rightarrow Y$ dan pada garpu $X \leftarrow Z \rightarrow Y$, jalur informasi terbuka selama Z tidak diamati dan tertutup ketika Z diamati. Pola ketiga, v -structure $X \rightarrow Z \leftarrow Y$, bekerja terbalik. Kalau Z tidak diamati, jalurnya tertutup: kesulitan mata kuliah dan kecerdasan mahasiswa tidak saling memengaruhi. Kalau Z , atau salah satu turunannya, diamati, jalur terbuka, dan itulah sebabnya explaining away terjadi. Dua variabel disebut *d-separated* oleh sekumpulan pengamatan kalau semua jalur di antara keduanya tertutup, dan dalam hal itu mereka pasti independen bersyarat.

Satu himpunan yang istimewa adalah *Markov blanket* sebuah variabel: induknya, anaknya, dan induk lain dari anak-anaknya. Kalau Markov blanket diketahui, variabel itu independen dari seluruh sisa jaringan. Sifat ini nanti menjadi kunci Gibbs sampling.

14.3 Inferensi eksak

Inferensi berarti menghitung distribusi sebuah variabel kueri setelah sebagian variabel lain diamati, misalnya $p(I \mid G = \text{buruk})$. Cara naif adalah menjumlahkan distribusi gabungan atas semua variabel lain, dengan biaya eksponensial. *Variable elimination* memanfaatkan faktorisasi: penjumlahan didorong sedalam mungkin ke dalam hasil kali, sehingga setiap variabel dijumlahkan hanya dari faktor-faktor yang menyebutnya. Hasil antara disimpan sebagai faktor baru.

Urutan eliminasi menentukan biaya. Urutan yang baik menjaga faktor antara tetap kecil, sedangkan urutan yang buruk bisa membuat faktor sebesar tabel gabungan. Untuk graf berbentuk pohon, biayanya linear. Untuk graf umum, inferensi eksak termasuk masalah NP-hard, dan biayanya bergantung secara eksponensial pada ukuran faktor terbesar yang tidak bisa dihindari.

Contoh rantai tiga variabel biner membuat penghematannya terlihat. Misalkan $A \rightarrow B \rightarrow C$ dengan $p(A=1) = 0,3$, $p(B=1 \mid A)$ bernilai 0,2 untuk $A = 0$ dan 0,9 untuk $A = 1$, serta $p(C=1 \mid B)$ bernilai 0,1 untuk $B = 0$ dan 0,8 untuk $B = 1$. Untuk menghitung $p(C=1) = \sum_a \sum_b p(a) p(b \mid a) p(C=1 \mid b)$, penjumlahan atas a didorong ke dalam karena hanya dua faktor pertama yang menyebut A . Hasilnya faktor baru $\tau(b) = \sum_a p(a) p(b \mid a)$, dengan $\tau(1) = 0,7 \cdot 0,2 + 0,3 \cdot 0,9 = 0,41$ dan $\tau(0) = 0,59$. Lalu $p(C=1) = 0,59 \cdot 0,1 + 0,41 \cdot 0,8 = 0,387$. Untuk rantai n variabel biner, menjumlahkan tabel gabungan membutuhkan 2^{n-1} suku, sedangkan eliminasi hanya beberapa operasi per variabel. Dengan tiga puluh variabel, selisihnya antara setengah miliar suku dan seratusan operasi.

Kalau banyak kueri harus dijawab pada jaringan yang sama, eliminasi yang diulang untuk setiap kueri membuang kerja. Kuliah membahas *clique tree*: variabel-variabel dikelompokkan ke dalam klik yang disusun sebagai pohon, lalu pesan dikirim sekali ke arah akar dan sekali kembali ke daun. Setelah dua sapuan itu, setiap klik menyimpan distribusi marginal variabel-variabelnya, sehingga semua kueri tunggal bisa dijawab sekaligus. Inilah message passing dari Bab 13 dalam bentuk probabilistik. Pada graf yang punya siklus, pesan yang sama kadang tetap dikirim berulang-ulang sampai stabil, versi *loopy* yang tidak dijamin benar tetapi sering cukup baik dalam praktik.

14.4 Inferensi dengan sampel

Kalau inferensi eksak terlalu mahal, kita bisa memperkirakan peluang dengan menarik sampel. Cara paling sederhana adalah *forward sampling*: tarik setiap variabel mengikuti urutan graf, dari induk ke anak. Untuk menghitung peluang bersyarat, *rejection sampling* membuang sampel yang tidak cocok dengan pengamatan. Cara ini boros sekali kalau pengamatannya jarang terjadi, karena hampir semua sampel dibuang.

Likelihood weighting tidak membuang apa pun. Variabel yang diamati dipaksa bernilai sesuai pengamatan, dan setiap sampel diberi bobot sebesar peluang pengamatan itu terjadi. Itulah contoh *importance sampling*: sampel ditarik dari distribusi yang mudah, lalu dikoreksi dengan bobot. Kelemahannya muncul ketika pengamatan berada di bagian bawah graf. Variabel di atasnya ditarik tanpa memperhatikan pengamatan,

sehingga banyak sampel mendapat bobot yang hampir nol.

Markov chain Monte Carlo mengambil jalan yang berbeda. Kita membangun sebuah rantai Markov yang distribusi stasionernya adalah distribusi posterior yang dicari, lalu menjalankan rantai itu cukup lama. *Gibbs sampling* memperbarui satu variabel pada satu waktu, menariknya dari distribusi bersyarat terhadap semua variabel lain. Berkat Markov blanket, distribusi bersyarat itu hanya melibatkan beberapa faktor lokal. Metropolis–Hastings (Hastings, 1970; Metropolis et al., 1953) lebih umum: usulkan langkah acak, lalu terima dengan peluang yang dipilih tepat agar rantai menyeimbangkan distribusi target. Kedua metode ini lahir dari fisika statistik, untuk menghitung sifat gas dan cairan, dan kembali ke fisika di Bab 28.

Bayangkan menaksir bentuk sebuah ruangan gelap dengan berjalan acak di dalamnya, sambil lebih sering melangkah ke tempat yang lebih “terang”. Setelah cukup lama, jejak langkah Anda menggambarkan seberapa terang setiap sudut ruangan. Langkah-langkah awal sebelum rantai mencapai keseimbangan biasanya dibuang, dan sampel yang berurutan saling berkorelasi, sehingga jumlah sampel efektifnya lebih kecil dari jumlah sampel mentah.

14.5 Belajar parameter dan struktur

Kalau struktur graf diketahui dan datanya lengkap, belajar parameter ternyata hanya menghitung. Maximum likelihood untuk $p(x_i | \text{pa}(x_i))$ adalah frekuensi relatif di data. Masalahnya muncul untuk kombinasi yang jarang atau tidak pernah terlihat, yang langsung mendapat peluang nol. Pendekatan Bayesian memberi prior Dirichlet pada setiap tabel peluang, dan efeknya setara dengan menambahkan hitungan semu sebelum melihat data. Kalau sebuah kejadian muncul 3 kali dari 10, maximum likelihood memberi 0,3. Dengan satu hitungan semu untuk setiap dari dua kemungkinan, yaitu Laplace smoothing, perkiraannya menjadi $4/12 \approx 0,33$. Pengaruh hitungan semu ini memudar ketika data bertambah.

Kalau sebagian data hilang, atau ada variabel tersembunyi yang tidak pernah diamati, hitungan sederhana tidak bisa dilakukan. Algoritme EM (Dempster et al., 1977) bergantian antara dua langkah. Langkah E menghitung distribusi variabel yang hilang dengan parameter saat ini, sehingga menghasilkan hitungan yang diharapkan. Langkah M memperbarui parameter seolah-olah hitungan yang diharapkan itu adalah data lengkap. Setiap putaran tidak menurunkan likelihood. EM akan muncul lagi sebagai kerangka di balik VAE dan model campuran Gauss di Bab 15.

Belajar struktur graf lebih sulit, karena jumlah graf yang mungkin tumbuh super-eksponensial terhadap jumlah variabel. Kuliah membahas dua keluarga pendekatan. Pendekatan berbasis kendala menjalankan uji independensi bersyarat pada data, lalu membangun graf yang sesuai dengan hasil uji itu. Pendekatan berbasis skor mencari graf yang memaksimalkan sebuah skor, misalnya BIC,

$$\text{BIC} = \log p(\mathcal{D} | \hat{\theta}, \mathcal{G}) - \frac{\log N}{2} \dim(\mathcal{G}), \quad (14.2)$$

yang menyeimbangkan kecocokan dengan data dan jumlah parameter. Tanpa suku penalti, graf yang terhubung penuh selalu menang, dan itulah overfitting dalam bentuk

struktur. Pencariannya biasanya heuristik, misalnya hill climbing dengan menambah, menghapus, atau membalik satu sisi pada satu waktu, atau tabu search yang melarang kembali ke langkah yang baru saja diambil. Pendekatan ketiga, Bayesian model averaging, tidak memilih satu graf, tetapi merata-ratakan prediksi atas banyak graf yang diberi bobot posterior.

14.6 Model yang berjalan dalam waktu

Banyak sistem berubah dari waktu ke waktu. *Dynamic Bayesian network* memodelkannya dengan mengulang struktur yang sama di setiap langkah waktu, dengan dua asumsi: keadaan sekarang hanya bergantung pada keadaan sebelumnya (asumsi Markov), dan aturan transisinya tidak berubah dari waktu ke waktu. Seluruh model cukup ditentukan oleh satu irisan waktu beserta sambungannya ke irisan berikutnya, lalu bisa “dibuka” sepanjang yang dibutuhkan.

Ada empat tugas inferensi yang berulang di semua model temporal. *Filtering* menghitung keadaan sekarang dari semua pengamatan sampai sekarang. *Prediction* menghitung keadaan masa depan. *Smoothing* memperbaiki perkiraan keadaan masa lalu dengan pengamatan yang datang belakangan. Dan *most likely explanation* mencari urutan keadaan yang paling mungkin menjelaskan semua pengamatan.

14.6.1 Hidden Markov model

Hidden Markov model (Rabiner, 1989) adalah DBN dengan satu variabel keadaan diskret yang tersembunyi dan satu pengamatan di setiap langkah. Ia ditentukan oleh matriks transisi $a_{ij} = p(s_t = j \mid s_{t-1} = i)$, peluang emisi $b_j(o) = p(o_t = o \mid s_t = j)$, dan distribusi awal. Filtering dihitung dengan *forward algorithm*,

$$\alpha_t(j) = b_j(o_t) \sum_i \alpha_{t-1}(i) a_{ij}, \quad (14.3)$$

yang bergantian antara memprediksi (penjumlahan atas keadaan sebelumnya) dan memperbarui dengan pengamatan baru (perkalian dengan peluang emisi). Algoritme backward menghitung pesan yang sama dari arah sebaliknya, dan gabungan keduanya memberi smoothing. Algoritme Viterbi mengganti penjumlahan dengan maksimum, $\delta_t(j) = b_j(o_t) \max_i \delta_{t-1}(i) a_{ij}$, sambil mencatat i mana yang memberi maksimum, lalu menelusuri catatan itu mundur dari akhir untuk mendapatkan urutan keadaan terbaik. Biayanya sebanding dengan panjang deret kali kuadrat jumlah keadaan, bukan eksponensial terhadap panjang deret seperti kalau semua urutan dicoba, dan Baum–Welch adalah EM untuk mempelajari parameter HMM dari pengamatan saja.

Contoh klasik dari Russell and Norvig (2021) adalah penjaga yang tidak pernah keluar gedung dan hanya bisa menebak cuaca dari apakah direktornya datang membawa payung. Misalkan peluang hujan hari ini sama dengan kemarin adalah 0,7, peluang membawa payung saat hujan 0,9, dan saat tidak hujan 0,2, dengan keyakinan awal 0,5. Pada hari pertama direktur membawa payung. Prediksinya tetap 0,5, dan pembaruan memberi hujan sebanding dengan $0,9 \times 0,5 = 0,45$ dan tidak hujan sebanding dengan $0,2 \times 0,5 = 0,10$. Setelah dinormalisasi, peluang hujan 0,818. Pada hari kedua prediksinya

$0,818 \times 0,7 + 0,182 \times 0,3 \approx 0,627$, dan pengamatan payung kedua menaikannya menjadi sekitar 0,883. Setiap hari, keyakinan mengalir lewat dua langkah yang sama: ramalkan, lalu koreksi dengan bukti.

14.6.2 Kalman filter

Kalau keadaannya kontinu dan semua hubungan linear dengan noise Gauss, filtering bisa dikerjakan secara eksak dengan Kalman filter (Kalman, 1960). Modelnya $x_t = Ax_{t-1} + w_t$ dan $z_t = Hx_t + v_t$, dengan noise proses $w_t \sim \mathcal{N}(0, Q)$ dan noise pengukuran $v_t \sim \mathcal{N}(0, R)$. Karena Gauss tetap Gauss di bawah transformasi linear, cukup melacak rata-rata dan kovarians:

$$\begin{aligned}\hat{x}_t^- &= A\hat{x}_{t-1}, & P_t^- &= AP_{t-1}A^\top + Q, \\ K_t &= P_t^- H^\top (HP_t^- H^\top + R)^{-1}, & & \\ \hat{x}_t &= \hat{x}_t^- + K_t(z_t - H\hat{x}_t^-), & P_t &= (I - K_t H)P_t^-. \end{aligned} \quad (14.4)$$

Baris pertama adalah prediksi, dan baris terakhir adalah koreksi. Matriks K_t , *Kalman gain*, menentukan seberapa besar kita mempercayai pengukuran dibanding prediksi.

Kalman filter seperti mendengarkan dua saksi. Saksi pertama adalah model, yang meramalkan posisi dari posisi sebelumnya. Saksi kedua adalah sensor. Keduanya bisa keliru, dan kita mempercayai masing-masing sesuai ketelitiannya. Dalam satu dimensi, kalau ketidakpastian prediksi punya varians 4 dan pengukuran punya varians 1, gain-nya $4/(4+1) = 0,8$: hasil akhir bergerak 80 persen ke arah pengukuran, dan variansnya turun menjadi 0,8, lebih kecil dari keduanya. Menggabungkan dua sumber yang tidak sempurna menghasilkan perkiraan yang lebih baik dari masing-masing.

Rumus satu dimensi ini bisa diturunkan langsung dengan teorema Bayes. Prediksi memberi prior $\mathcal{N}(\hat{x}^-, P^-)$, dan pengukuran z memberi likelihood $\mathcal{N}(z; x, R)$. Posterior sebanding dengan hasil kali keduanya, dan eksponennya adalah $-\frac{1}{2}[(x - \hat{x}^-)^2/P^- + (z - x)^2/R]$. Jumlah dua kuadrat dalam x tetap kuadrat dalam x , jadi posteriornya Gauss. Mengumpulkan koefisien x^2 memberi presisi posterior $1/P = 1/P^- + 1/R$: presisi, yaitu kebalikan varians, langsung dijumlahkan. Mengumpulkan koefisien x memberi rata-rata posterior $\hat{x} = P(\hat{x}^-/P^- + z/R)$, rata-rata berbobot dengan bobot sebanding presisi masing-masing. Dengan sedikit aljabar, keduanya bisa ditulis ulang sebagai $\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$ dan $P = (1 - K)P^-$ dengan $K = P^-/(P^- + R)$, persis bentuk Kalman. Untuk angka di atas, $1/P = 1/4 + 1/1 = 1,25$, sehingga $P = 0,8$.

Untuk sistem yang tidak linear, Kalman filter diperluas dengan melinearkan model di sekitar perkiraan saat ini (*extended Kalman filter*), atau diganti dengan *particle filter* yang mewakili distribusi dengan sekumpulan sampel berbobot. Keduanya dipakai di kendaraan otonom untuk melacak posisi dari GPS, odometri, dan lidar.

Particle filter adalah filter Bayes yang sama dengan Kalman filter, hanya distribusinya diwakili sampel, bukan Gauss. Setiap langkah punya tiga bagian. Setiap partikel digerakkan menurut model dinamika, termasuk noise-nya. Setiap partikel diberi bobot sebanding dengan likelihood pengukuran di posisinya. Lalu partikel ditarik ulang sesuai bobotnya, sehingga partikel di tempat yang tidak mungkin mati dan partikel di tempat yang mungkin berlipat. Contoh kecil: tiga partikel di posisi 1, 2, dan 3 bergerak

satu satuan ke 2, 3, dan 4. Sensor mengukur 3,2 dengan noise Gauss bersimpangan baku satu. Likelihood-nya sebanding dengan $e^{-(x-3,2)^2/2}$, yaitu 0,49, 0,98, dan 0,73, dan setelah dinormalisasi menjadi bobot 0,22, 0,45, dan 0,33. Taksiran posisinya rata-rata berbobot, sekitar 3,11. Dalam praktik dipakai ratusan sampai ribuan partikel, dan karena tidak ada asumsi Gauss, distribusinya boleh bercabang, misalnya ketika robot belum tahu di lorong mana ia berada, persis lokalisasi Markov di Bab 21.

14.7 Di luar kelas

Model-model di bab ini bekerja di belakang banyak sistem yang kita temui di bagian-bagian berikutnya. Kendaraan otonom melacak posisinya dengan filter Bayes dan melacak mobil di sekitarnya dengan Kalman filter (Bab 21). Pelacak gerak inersia menggabungkan giroskop dan akselerometer dengan versi sederhana Kalman filter (Bab 20). Filter Wiener dan estimator MMSE dalam pemrosesan sinyal adalah saudara dekatnya (Bab 18). Dalam bioinformatika, hidden Markov model dipakai untuk mencari gen dan menjajarkan keluarga protein (Durbin et al., 1998), dan algoritme EM menjadi dasar banyak metode analisis ekspresi gen (Bab 22).

Di dunia simulasi, ide Kalman filter dikenal sebagai *data assimilation*. Prakiraan cuaca modern bekerja dengan cara ini: model atmosfer meramalkan keadaan beberapa jam ke depan, lalu ramalan itu dikoreksi dengan pengamatan dari satelit, balon, dan stasiun cuaca, dengan bobot sesuai ketidakpastian masing-masing. Model atmosfer punya jutaan variabel, sehingga matriks kovarians P tidak mungkin disimpan. Ensemble Kalman filter (Evensen, 1994) memperkirakannya dari sekumpulan simulasi yang berjalan paralel dengan kondisi awal sedikit berbeda. Sebaran ensemble menjadi ukuran ketidakpastian ramalan.

Gagasan yang sama berguna di skala laboratorium. Simulasi CFD sebuah tangki dan pengukuran temperatur dari sensor di dalam tangki yang sama jarang cocok sempurna. Model turbulensi, sifat fluida, dan syarat batas semuanya menyimpan galat. Daripada memilih salah satu, kita bisa menarik simulasi secara perlahan ke arah pengukuran di titik-titik sensor, dengan kekuatan yang mencerminkan seberapa dipercaya masing-masing. Teknik paling sederhana seperti ini disebut *nudging*, dan ia adalah kerabat dekat Kalman filter dengan gain yang dipilih tangan.

Cara berpikir probabilistik juga membuka pintu ke masalah invers. Kalau kita ingin menyimpulkan parameter fisika, misalnya konduktivitas atau syarat batas, dari pengukuran yang berisik, jawaban yang jujur bukan satu angka, melainkan sebuah distribusi posterior. MCMC, inferensi variasional, dan model generatif dari Bab 15 adalah alat untuk menghitungnya.

14.8 Perkembangan riset

Banyak ilmu punya simulator yang teliti tetapi tidak punya likelihood yang bisa dituliskan: simulator tabrakan partikel, model epidemi berbasis agen, atau model iklim. Bayes membutuhkan $p(\text{data} \mid \theta)$, sedangkan simulator hanya bisa menghasilkan data untuk θ tertentu. *Simulation-based inference* (Cranmer et al., 2020) menjembatani

keduanya. Jalankan simulator untuk banyak θ yang ditarik dari prior, lalu latih jaringan saraf untuk mendekati posterior $p(\theta \mid \text{data})$ secara langsung, misalnya dengan normalizing flow dari Bab 12. Setelah dilatih, posterior untuk pengamatan baru diperoleh hampir seketika, tanpa menjalankan MCMC lagi. Inferensi semacam ini disebut *amortized*: biaya besar dibayar sekali di awal, lalu dipakai berulang kali. Gagasan yang sama dipakai di asimilasi data dan dalam mengkalibrasi simulasi fisika terhadap percobaan.

CATATAN SUMBER. Bab ini mengikuti kuliah Probabilistic Models (Bayesian network, inferensi eksak dan aproksimasi, pembelajaran parameter dan struktur, DBN, HMM, Kalman filter) beserta latihannya. Slide kuliah bersandar pada Koller and Friedman (2009) dan Russell and Norvig (2021), yang juga menjadi bacaan pendamping terbaik. Bagian Kalman filter dan particle filter juga muncul di kuliah kendaraan otonom.

15 Dari Estimasi ke Difusi

Kuliah machine learning lanjut saya ikuti pada musim dingin 2025/26, di bawah Johannes Brandstetter, yang risetnya sendiri berada di persimpangan deep learning dan simulasi fisika. Delapan unitnya membentuk satu alur panjang: dari teori estimasi dan statistik Bayesian, lewat VAE, persamaan diferensial biasa, parsial, dan stokastik, sampai model difusi, flow matching, dan neural field. Tugas-tugasnya meliputi probabilistic PCA, score-based diffusion dan neural ODE, serta variational diffusion model. Di antara semua kuliah, inilah yang paling langsung mengarah ke AI untuk simulasi.

Benang merah bab ini sederhana untuk diucapkan, meski butuh beberapa halaman untuk dipahami: model generatif modern adalah persamaan diferensial. Model difusi adalah persamaan difusi yang dijalankan mundur. Flow matching adalah persamaan transport. Neural field adalah fungsi kontinu yang bisa diatur oleh persamaan fisika. Untuk sampai ke sana, kita mulai dari pertanyaan yang paling dasar: seberapa baik kita bisa menaksir sesuatu dari data.

15.1 Teori estimasi

Misalkan data $\mathcal{D}$ dibangkitkan oleh distribusi dengan parameter θ yang tidak diketahui, dan kita membuat sebuah estimator $\hat{\theta}(\mathcal{D})$. Karena data acak, estimator pun acak. Kualitasnya diukur dengan mean squared error yang dirata-ratakan atas semua kemungkinan data, dan MSE itu terurai menjadi varians dan bias kuadrat,

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + (\mathbb{E}[\hat{\theta}] - \theta)^2. \quad (15.1)$$

Bentuknya sama dengan dekomposisi bias–varians di Bab 1, kali ini untuk parameter, bukan untuk prediksi. Rata-rata sampel adalah estimator tak bias untuk rata-rata distribusi. Varians sampel yang dibagi N bias ke bawah, dan pembagian dengan $N - 1$ menghilangkan bias itu. Estimator tak bias dengan varians terkecil di antara semua estimator tak bias disebut *minimum variance unbiased* (MVU).

Ada batas bawah mutlak untuk varians estimator tak bias mana pun. *Fisher information*

$$I(\theta) = -\mathbb{E}\left[\frac{\partial^2}{\partial \theta^2} \log p(\mathcal{D} \mid \theta)\right] \quad (15.2)$$

mengukur seberapa tajam puncak log-likelihood di sekitar nilai yang benar. Puncak yang tajam berarti data membawa banyak informasi tentang θ . Batas Cramér–Rao menyatakan $\text{Var}(\hat{\theta}) \geq 1/I(\theta)$ untuk setiap estimator tak bias. Untuk rata-rata distribusi normal dengan varians σ^2 yang diketahui dan N sampel, $I = N/\sigma^2$, sehingga varians terkecil yang mungkin adalah σ^2/N . Rata-rata sampel mencapai batas ini, jadi ia estimator yang efisien. Maximum likelihood, secara umum, tidak selalu tak bias untuk

sampel kecil, tetapi untuk sampel besar ia konsisten dan mencapai batas Cramér–Rao secara asimtotik.

Batas Cramér–Rao punya terjemahan langsung di dunia simulasi. Kalau kita ingin menaksir sebuah parameter fisika, misalnya koefisien perpindahan panas, dari pengukuran temperatur, Fisher information mengatakan apakah percobaan kita cukup informatif. Kalau log-likelihood hampir datar terhadap parameter itu, tidak ada metode pintar yang bisa menaksirnya dengan baik. Masalahnya ada pada desain percobaan, bukan pada algoritmenya.

15.2 Cara pandang Bayesian

Pendekatan frekuentis menganggap θ tetap dan datanya acak. Pendekatan Bayesian menganggap θ sendiri tidak pasti dan mewakilinya dengan distribusi. Teorema Bayes memperbarui prior menjadi posterior setelah melihat data. Untuk beberapa pasangan likelihood dan prior, posterior punya bentuk yang sama dengan prior, dan pasangan seperti ini disebut *conjugate*. Contoh yang dibahas kuliah adalah Gamma dan Poisson. Kalau jumlah kejadian per hari mengikuti Poisson dengan laju λ , dan prior $\lambda \sim \text{Gamma}(\alpha, \beta)$, maka setelah mengamati $x_1, \dots, x_N$, posteriornya $\text{Gamma}(\alpha + \sum_i x_i, \beta + N)$. Dengan prior $\text{Gamma}(2, 1)$ dan pengamatan 3, 5, dan 4 kejadian, posteriornya $\text{Gamma}(14, 4)$ dengan rata-rata 3,5. Prior bertindak seperti dua kejadian yang “sudah teramati” dalam satu hari semu, dan bobotnya memudar seiring datangnya data.

Estimasi MAP mengambil puncak posterior, dan dengan prior Gauss pada bobot ia menghasilkan weight decay, seperti di Bab 1. Pendekatan Bayesian penuh tidak berhenti di puncak. Ia memprediksi dengan merata-ratakan atas seluruh posterior, dan normalisasi posterior, $p(\mathcal{D}) = \int p(\mathcal{D} | \theta)p(\theta)d\theta$, disebut *evidence*. Evidence bisa dipakai untuk membandingkan model: model yang terlalu sederhana tidak bisa menjelaskan data, sedangkan model yang terlalu fleksibel menyebar peluangnya ke terlalu banyak kemungkinan data, sehingga keduanya mendapat evidence yang rendah.

15.3 Variabel laten dan probabilistic PCA

Banyak data lebih mudah dijelaskan kalau kita membayangkan ada sebab tersembunyi di baliknya. *Latent variable model* menulis $p(x) = \int p(x | z)p(z)dz$, dengan z yang tidak pernah diamati. Contoh paling sederhana adalah probabilistic PCA (Tipping and Bishop, 1999):

$$x = Wz + \mu + \varepsilon, \quad z \sim \mathcal{N}(0, I_q), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_D). \quad (15.3)$$

Karena semuanya Gauss, distribusi marginalnya juga Gauss, $x \sim \mathcal{N}(\mu, WW^\top + \sigma^2 I)$, dan maximum likelihood punya solusi tertutup. Kolom W terbentang oleh q vektor eigen utama dari kovarians sampel, diskalakan oleh $(\lambda_i - \sigma^2)^{1/2}$, dan σ^2 adalah rata-rata nilai eigen yang dibuang. Ketika $\sigma^2 \rightarrow 0$, PCA biasa muncul kembali. Keuntungan versi probabilistik adalah adanya likelihood, sehingga data yang hilang bisa ditangani, model bisa dibandingkan, dan beberapa model bisa digabung.

Kalau $p(x | z)$ tidak lagi linear, misalnya berupa jaringan saraf, integral marginal tidak bisa dihitung lagi, dan posterior $p(z | x)$ pun tidak. Dari sinilah VAE lahir.

15.4 ELBO, sekali lagi dan lebih teliti

Bab 12 memperkenalkan VAE secara intuitif. Kuliah ini menurunkannya. Ambil sembarang distribusi $q(z | x)$. Log-likelihood bisa ditulis sebagai

$$\log p(x) = \underbrace{\mathbb{E}_{q(z|x)} \left[\log \frac{p(x, z)}{q(z | x)} \right]}_{\text{ELBO}} + \text{KL}(q(z | x) \| p(z | x)). \quad (15.4)$$

Identitas ini didapat dengan satu trik. Karena $\log p(x)$ tidak bergantung pada z , ia sama dengan ekspektasinya di bawah q . Lalu tulis $p(x) = p(x, z)/p(z | x)$, kalikan dan bagi dengan $q(z | x)$ di dalam logaritma, dan pisahkan menjadi dua suku:

$$\log p(x) = \mathbb{E}_q \left[\log \frac{p(x, z)}{q(z | x)} \right] + \mathbb{E}_q \left[\log \frac{q(z | x)}{p(z | x)} \right].$$

Suku kedua adalah definisi divergensi KL. Divergensi KL tidak pernah negatif, sehingga suku pertama adalah batas bawah untuk $\log p(x)$. Memaksimalkan ELBO terhadap q sama dengan mendekatkan q ke posterior yang sebenarnya, dan memaksimalkannya terhadap parameter model menaikkan likelihood. Kalau suku pertama diuraikan, kita mendapat dua bagian yang sudah dikenal: rekonstruksi, $\mathbb{E}_q[\log p(x | z)]$, dan pencocokan prior, $-\text{KL}(q(z | x) \| p(z))$.

Divergensi KL sendiri layak dipahami. $\text{KL}(q \| p) = \mathbb{E}_q[\log q - \log p]$ mengukur seberapa banyak informasi yang hilang kalau p dipakai untuk mewakili q . Ia tidak simetris. Meminimalkan $\text{KL}(q \| p)$ terhadap q membuat q menghindari daerah tempat p kecil, sehingga q cenderung memilih satu puncak p dan mengabaikan puncak lain.

Model campuran Gauss memberi contoh yang menjembatani. Data dianggap berasal dari salah satu dari K distribusi Gauss, dan label komponennya adalah variabel laten diskret. Algoritme EM untuk model ini, yang sudah kita temui di Bab 14, ternyata adalah maksimisasi ELBO bergantian: langkah E memilih q sama dengan posterior yang tepat, dan langkah M memperbarui parameter. VAE melakukan hal yang sama, tetapi karena posteriornya tidak bisa dihitung, q diwakili oleh jaringan encoder, dan gradien dibawa melewati sampling dengan reparameterization trick.

Langkah terakhir menuju model difusi adalah VAE hierarkis. Alih-alih satu lapisan laten, kita punya rantai $x \rightarrow z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_T$, dan pada versi Markov setiap lapisan hanya bergantung pada lapisan di sebelahnya. ELBO-nya terurai menjadi jumlah suku untuk setiap transisi. Model difusi adalah VAE hierarkis Markov dengan dua keputusan istimewa: dimensi setiap laten sama dengan dimensi data, dan encoder-nya tidak dipelajari, melainkan ditetapkan sebagai penambahan noise Gauss sedikit demi sedikit (Luo, 2022).

15.5 Gaussian process

Cara pandang Bayesian bisa dibawa satu langkah lebih jauh: alih-alih menaruh prior pada parameter sebuah model, taruh prior langsung pada fungsinya. Naskah kuliah deep learning lanjut menurunkannya dari regresi linear. Misalkan $f(x) = w^\top h(x)$ dengan fitur h yang tetap dan prior $w \sim \mathcal{N}(0, \sigma_w^2 I)$. Karena f linear dalam w , nilai-nilai f di titik mana pun berdistribusi Gauss bersama, dengan rata-rata nol dan kovarians

$$\text{Cov}(f(x), f(x')) = \sigma_w^2 h(x)^\top h(x') = k(x, x'). \quad (15.5)$$

Kumpulan variabel acak yang setiap himpunan berhingganya berdistribusi Gauss bersama disebut *Gaussian process*, dan ia ditentukan seluruhnya oleh fungsi rata-rata dan fungsi kovariansnya, yaitu kernel k (Rasmussen and Williams, 2006). Kernel boleh dipilih langsung tanpa menulis fiturnya, kernel trick dari Bab 3. Kernel Gauss, $k(x, x') = \exp(-\|x - x'\|^2 / 2\ell^2)$, berarti titik yang berdekatan punya nilai fungsi yang hampir sama, dan lebar ℓ mengatur seberapa cepat korelasi itu memudar.

Karena semuanya Gauss, posterior setelah melihat data punya bentuk tertutup, hasil langsung dari sifat kondisional Gauss di Lampiran B. Untuk data y di titik-titik latih dengan noise bervarians σ^2 , matriks kernel K , dan vektor k_* berisi kernel antara titik baru dan titik-titik latih,

$$\mu_* = k_*^\top (K + \sigma^2 I)^{-1} y, \quad \sigma_*^2 = k(x_*, x_*) - k_*^\top (K + \sigma^2 I)^{-1} k_*. \quad (15.6)$$

Rata-ratanya adalah kernel ridge regression dari Bab 3, dan variansnya adalah bonus yang diberikan sudut pandang Bayes. Contoh dengan satu titik data memperlihatkan perilakunya. Misalkan satu pengamatan $y = 1$ di $x = 0$, kernel Gauss dengan $\ell = 1$, dan $\sigma^2 = 0,1$. Di $x_* = 0$, $\mu_* = 1/1,1 \approx 0,91$ dan $\sigma_*^2 = 1 - 1/1,1 \approx 0,09$: dekat data, model yakin. Di $x_* = 2$, kernel ke titik data hanya $e^{-2} \approx 0,14$, sehingga $\mu_* \approx 0,12$ dan $\sigma_*^2 \approx 0,98$: jauh dari data, prediksi kembali ke rata-rata prior dan ketidakpastiannya kembali hampir penuh. Inilah pita yang melebar jauh dari data di Bab 2, sekarang tanpa harus memilih fitur sama sekali. Harganya adalah membalik matriks berukuran $n \times n$, yang biayanya tumbuh seperti n^3 , sehingga GP biasa cocok untuk ratusan sampai beberapa ribu titik, dan untuk data lebih besar dipakai aproksimasi.

Naskah kuliah juga menutup lingkaran antara GP dan jaringan saraf. Jaringan satu lapisan tersembunyi dengan bobot acak independen adalah jumlah dari banyak sumbangan neuron yang independen. Ketika lebarnya menuju tak hingga, teorema limit pusat membuat keluarannya berdistribusi Gauss, sehingga jaringan Bayes yang lebarnya tak hingga adalah Gaussian process dengan kernel yang ditentukan arsitektur dan aktivasinya. Untuk jaringan yang lebarnya terbatas, ketidakpastian ditaksir dengan cara lain, seperti MC dropout dan deep ensembles yang dibahas di Bab 12. GP sendiri banyak dipakai di tempat data mahal dan ketidakpastian penting: sebagai model pengganti dalam Bayesian optimization (Bab 8), sebagai model koreksi multi-fidelity (Bab 32), dan sebagai model dinamika dalam kendali berbasis pembelajaran (Bab 19).

15.6 Persamaan diferensial biasa, parsial, dan stokastik

Sebelum masuk ke model difusi, kuliah berhenti sejenak untuk mengulang matematika yang akan dipakai.

15.6.1 ODE

Persamaan diferensial biasa, $\dot{x} = f(x, t)$ dengan $x(0) = x_0$, menggambarkan sistem yang perubahannya ditentukan oleh keadaannya sekarang. Untuk sistem linear $\dot{x} = Ax$, solusinya $x(t) = e^{At}x_0$, dan nilai eigen A menentukan nasibnya: bagian real negatif berarti peluruhan, bagian real positif berarti pertumbuhan, bagian imajiner berarti osilasi. Teorema Picard–Lindelöf menjamin solusi ada dan tunggal kalau f Lipschitz.

Kebanyakan ODE tidak punya solusi tertutup, jadi kita menyelesaikannya secara numerik. Metode Euler, $x_{n+1} = x_n + h f(x_n, t_n)$, punya galat global sebanding dengan langkah h . Runge–Kutta orde empat mengevaluasi f empat kali per langkah dan galatnya sebanding dengan h^4 . Untuk sistem kaku (*stiff*), yaitu yang punya skala waktu cepat sekali dan lambat sekaligus, metode eksplisit terpaksa memakai langkah kecil sekali agar stabil, dan metode implisit atau langkah adaptif menjadi pilihan yang lebih baik. Topik ini dibahas lebih dalam di Bab 28.

15.6.2 PDE dan persamaan panas

Persamaan diferensial parsial melibatkan turunan terhadap lebih dari satu variabel. Contoh yang dibahas panjang lebar di kuliah adalah persamaan panas pada selang $(0, 1)$,

$$\partial_t u = \kappa \partial_{xx} u, \quad u(0, t) = u(1, t) = 0. \quad (15.7)$$

Pemisahan variabel memberi solusi

$$u(x, t) = \sum_{n=1}^{\infty} b_n e^{-\kappa(n\pi)^2 t} \sin(n\pi x), \quad (15.8)$$

dengan koefisien b_n diambil dari kondisi awal. Setiap mode Fourier meluruh dengan laju yang sebanding dengan kuadrat frekuensinya. Detail halus hilang paling cepat, dan setelah cukup lama yang tersisa hanya gelombang paling panjang. Pada garis bilangan penuh, transformasi Fourier memberi gambaran yang sama, $\hat{u}(k, t) = \hat{u}(k, 0) e^{-\kappa k^2 t}$, yang di ruang posisi berarti konvolusi dengan kernel Gauss yang semakin lebar. Pengamatan ini akan berguna sebentar lagi: menambahkan noise Gauss pada data sama dengan menjalankan persamaan panas pada distribusinya.

15.6.3 Gerak Brown dan SDE

Gerak Brown W_t adalah batas acak dari jalan acak dengan langkah yang semakin kecil. Kenaikannya independen dan berdistribusi normal, $W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$. Simpangannya tumbuh seperti $\sqrt{\Delta t}$, bukan Δt , sehingga lintasannya kontinu tetapi tidak bisa diturunkan di mana pun. Sebuah jalan acak memberi gambaran: kalau setiap langkah waktu Δt kita melangkah ke kiri atau ke kanan sejauh $\sqrt{\Delta t}$, setelah waktu $t = n\Delta t$ variansnya $n\Delta t = t$, tidak bergantung pada seberapa halus langkahnya. Kalau

langkahnya sebanding Δt , variansnya akan menyusut ke nol ketika langkah diperhalus. *Stochastic differential equation* menambahkan suku acak ini ke ODE,

$$dx = f(x, t) dt + g(t) dW_t, \quad (15.9)$$

dengan f sebagai *drift* dan g sebagai *diffusion*. Kalkulus untuk persamaan ini punya satu kejutan. Karena $(dW)^2$ berukuran dt , bukan nol, aturan rantai mendapat suku tambahan. Lemma Itô untuk fungsi skalar $\varphi(x)$ berbunyi

$$d\varphi = \left(\varphi' f + \frac{1}{2} g^2 \varphi'' \right) dt + \varphi' g dW. \quad (15.10)$$

Asal suku tambahan itu terlihat dari Taylor orde dua: $d\varphi = \varphi' dx + \frac{1}{2} \varphi'' (dx)^2 + \dots$. Dengan $dx = f dt + g dW$, kuadratnya memuat $g^2 (dW)^2$, dan karena $(dW)^2$ berukuran dt , suku itu bertahan, sedangkan $(dt)^2$ dan $dt dW$ lebih kecil dan hilang. Contoh klasiknya gerak Brown geometris, $dS = \mu S dt + \sigma S dW$, model sederhana harga saham. Dengan $\varphi = \log S$, lemma Itô memberi $d \log S = (\mu - \frac{1}{2} \sigma^2) dt + \sigma dW$, sehingga $S_t = S_0 \exp((\mu - \frac{1}{2} \sigma^2)t + \sigma W_t)$. Suku $-\frac{1}{2} \sigma^2$ adalah jejak kalkulus Itô yang tidak akan muncul dalam kalkulus biasa.

Distribusi $p(x, t)$ dari solusi SDE berevolusi menurut persamaan Fokker–Planck,

$$\partial_t p = -\nabla \cdot (f p) + \frac{1}{2} g^2 \Delta p. \quad (15.11)$$

Suku pertama adalah transport oleh drift, suku kedua adalah difusi. Untuk $f = 0$, ini persis persamaan panas. Sebuah SDE yang bekerja pada partikel-partikel menjadi PDE yang bekerja pada kerapatannya.

15.7 Neural ODE

Blok residual, $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l)$, terlihat seperti satu langkah Euler. Kalau jumlah lapisan dibuat tak hingga dan setiap langkahnya kecil, kita mendapat

$$\frac{d\mathbf{h}}{dt} = f_\theta(\mathbf{h}, t), \quad (15.12)$$

sebuah *neural ODE* (Chen et al., 2018). Lintasan maju dihitung dengan solver ODE apa pun, termasuk yang langkahnya adaptif. Untuk gradien, kita bisa saja melakukan backprop melewati setiap langkah solver, tetapi memorinya sebanding dengan jumlah langkah. Metode adjoint, yang dikenal lama dalam teori kendali optimal, menghindarinya. Definisikan $\mathbf{a}(t) = \partial \mathcal{L} / \partial \mathbf{h}(t)$. Vektor ini memenuhi ODE-nya sendiri yang berjalan mundur,

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}^\top \frac{\partial f_\theta}{\partial \mathbf{h}}, \quad \frac{d\mathcal{L}}{d\theta} = - \int_{t_1}^{t_0} \mathbf{a}^\top \frac{\partial f_\theta}{\partial \theta} dt. \quad (15.13)$$

Keadaan $\mathbf{h}$, adjoint $\mathbf{a}$, dan gradien terhadap parameter diintegrasikan mundur bersama dalam satu sistem yang diperbesar. Memorinya konstan terhadap jumlah langkah. Persamaan adjoint ini tidak lain adalah delta backprop dari Bab 7 dalam waktu kontinu.

Neural ODE cocok sekali untuk sistem fisika yang memang berevolusi secara kontinu. Data yang diambil pada waktu tidak teratur bisa langsung dipakai, dan model dinamika yang dipelajari bisa dijalankan dengan solver standar. Hubungan dengan model difusi juga erat: sampling deterministik dari model difusi adalah menyelesaikan sebuah ODE.

15.8 Model difusi

Model difusi denoising (Ho et al., 2020; Sohl-Dickstein et al., 2015) menetapkan proses maju yang merusak data selangkah demi selangkah,

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (15.14)$$

dengan jadwal noise β_t yang kecil. Karena komposisi Gauss tetap Gauss, kita bisa melompat langsung dari data ke langkah mana pun: dengan $\bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s)$,

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (15.15)$$

Bentuk tertutup ini bisa diperiksa dengan dua langkah. Tulis $\alpha_t = 1 - \beta_t$. Satu langkah memberi $\mathbf{x}_1 = \sqrt{\alpha_1} \mathbf{x}_0 + \sqrt{1 - \alpha_1} \boldsymbol{\epsilon}_1$, dan langkah berikutnya $\mathbf{x}_2 = \sqrt{\alpha_2} \mathbf{x}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\epsilon}_2 = \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \sqrt{\alpha_2(1 - \alpha_1)} \boldsymbol{\epsilon}_1 + \sqrt{1 - \alpha_2} \boldsymbol{\epsilon}_2$. Dua noise Gauss independen yang dijumlahkan tetap Gauss, dengan varians yang dijumlahkan: $\alpha_2(1 - \alpha_1) + (1 - \alpha_2) = 1 - \alpha_1 \alpha_2$. Jadi $\mathbf{x}_2 = \sqrt{\alpha_1 \alpha_2} \mathbf{x}_0 + \sqrt{1 - \alpha_1 \alpha_2} \boldsymbol{\epsilon}$, dan induksi memberi rumus umumnya. Dengan jadwal β_t yang naik linear dari 10^{-4} ke 0,02 dalam seribu langkah, seperti pada Ho et al. (2020), nilai $\bar{\alpha}_T$ sekitar 4×10^{-5} : sinyal aslinya tinggal kurang dari satu persen dalam amplitudo. Setelah cukup banyak langkah, $\mathbf{x}_T$ hampir murni noise. Proses mundur dipelajari oleh sebuah jaringan. Dari ELBO VAE hierarkis, setelah beberapa langkah aljabar, loss pelatihannya menjadi sederhana: tebak noise yang ditambahkan,

$$\mathcal{L} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2. \quad (15.16)$$

Pada saat sampling, kita mulai dari noise murni dan berulang kali mengurangi noise yang ditebak jaringan, ditambah sedikit noise baru, sampai kembali ke data.

Bayangkan setetes tinta jatuh ke segelas air. Tinta menyebar perlahan sampai airnya berwarna rata, dan bentuk awal tetes itu tampak hilang. Model difusi belajar memutar video itu mundur: dari air yang keruh merata, langkah demi langkah, sampai tetes tinta terbentuk kembali. Setiap langkah mundur hanya perlu sedikit menebak, karena setiap langkah maju hanya sedikit merusak.

Kelemahan DDPM adalah lambat, karena sampling membutuhkan ratusan sampai ribuan langkah jaringan. DDIM (Song et al., 2020a) menunjukkan bahwa jaringan yang sama bisa dipakai dengan proses sampling deterministik yang melompati banyak langkah, tanpa pelatihan ulang. Variational diffusion models (Kingma et al., 2021) merumuskan jadwal noise sebagai rasio sinyal terhadap noise yang bisa dipelajari dan menunjukkan bahwa ELBO model difusi hanya bergantung pada jadwal itu melalui nilai ujung-ujungnya.

15.9 Score dan persamaan diferensial stokastik

Song et al. (2020b) menyatukan model difusi dalam bahasa SDE. Proses maju adalah SDE $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{W}$ yang mengubah distribusi data menjadi Gauss. Hasil lama dari Anderson (1982) menyatakan bahwa proses ini punya kebalikan dalam waktu, yang juga sebuah SDE,

$$d\mathbf{x} = [f(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})] dt + g(t) d\bar{\mathbf{W}}, \quad (15.17)$$

yang berjalan dari $t = T$ ke $t = 0$. Satu-satunya besaran yang tidak diketahui adalah *score* $\nabla_x \log p_t(x)$, yaitu arah tempat kerapatan data naik paling cepat. Jaringan $s_\theta(x, t)$ dilatih untuk menebak *score* ini. Melatihnya ternyata mudah berkat *denoising score matching* (Vincent, 2011): kalau $\tilde{x} = x + \sigma\epsilon$, *score* dari distribusi bersyarat $p(\tilde{x} | x)$ adalah $-(\tilde{x} - x)/\sigma^2 = -\epsilon/\sigma$, sehingga menebak *score* sama dengan menebak *noise*, persis seperti DDPM.

Ada juga ODE yang menghasilkan distribusi marginal yang sama di setiap waktu, yaitu *probability flow ODE*,

$$dx = \left[f(x, t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x) \right] dt. \quad (15.18)$$

Mengapa SDE dan ODE bisa menghasilkan marginal yang sama? Persamaan Fokker-Planck menjawabnya. Suku difusi $\frac{1}{2}g^2\Delta p$ bisa ditulis sebagai $\nabla \cdot (\frac{1}{2}g^2 p \nabla \log p)$, sehingga difusi pada kerapatan bisa diganti dengan transport deterministik dengan kecepatan $-\frac{1}{2}g^2 \nabla \log p$. Partikel-partikel individu bergerak berbeda, tetapi kerumunannya berevolusi dengan cara yang sama. Kuliah merangkum bahwa ketiga cara melatih model difusi, yaitu menebak data bersih, menebak *noise*, dan menebak *score*, setara sampai faktor skala. Untuk membangkitkan sampel bersyarat, misalnya gambar dari teks, *score* ditambah dengan gradien sebuah penilai syarat, atau syarat langsung diberikan sebagai input ke jaringan.

15.10 Flow matching

Kalau ujung-ujungnya kita menyelesaikan sebuah ODE, mengapa tidak langsung belajar ODE-nya? *Flow matching* (Lipman et al., 2022) melakukan hal itu. Kita mencari medan kecepatan $v_\theta(x, t)$ sehingga ODE $\dot{x} = v_\theta(x, t)$ membawa sampel dari distribusi sederhana p_0 , misalnya Gauss, ke distribusi data p_1 . Pemetaan yang dihasilkan disebut *flow* dalam arti matematis, dan barisan distribusi antara p_t disebut *probability path*.

Kesulitannya, medan kecepatan yang benar untuk seluruh distribusi tidak diketahui. Kunci *flow matching* adalah bekerja secara bersyarat. Untuk satu pasangan titik, sampel *noise* x_0 dan sampel data x_1 , kita bisa memilih jalur yang paling sederhana, garis lurus $x_t = (1 - t)x_0 + tx_1$, yang kecepatannya konstan, $x_1 - x_0$. *Loss conditional flow matching* memaksa jaringan meniru kecepatan bersyarat itu,

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, x_0, x_1} \left\| v_\theta(x_t, t) - (x_1 - x_0) \right\|^2, \quad (15.19)$$

dan hasil utamanya adalah bahwa gradien *loss* ini sama dengan gradien *loss* untuk medan kecepatan marginal yang sebenarnya. Kita tidak pernah perlu mengetahui medan marginal itu. Gagasan jalur lurus juga dikembangkan secara independen sebagai *rectified flow* (Liu et al., 2022).

Analogi yang saya pakai adalah rombongan penumpang yang harus dipindahkan dari terminal ke berbagai alamat di kota. Kita tidak tahu jalur terbaik untuk seluruh rombongan, tetapi kita tahu jalur lurus untuk setiap pasangan terminal dan alamat. Kalau sopir belajar rata-rata dari jutaan perjalanan lurus seperti itu, ia akhirnya tahu arah yang tepat di setiap persimpangan untuk seluruh arus penumpang. Persamaan

kontinuitas yang mengatur probability path, $\partial_t p_t + \nabla \cdot (p_t v_t) = 0$, adalah persamaan kekekalan massa yang sama dengan di mekanika fluida.

15.11 Neural field

Unit terakhir kuliah meninggalkan model generatif dan membahas cara lain merepresentasikan data. Dalam fisika, medan adalah besaran yang punya nilai di setiap titik ruang dan waktu: temperatur, tekanan, kecepatan. *Neural field* adalah jaringan saraf yang mewakili medan seperti itu secara langsung, $f_\theta : (x, t) \mapsto \text{nilai}$ (Xie et al., 2021). Alih-alih menyimpan nilai di grid, kita menyimpan bobot jaringan. Representasinya kontinu, tidak terikat resolusi, dan bisa diturunkan di mana saja dengan autodiff.

Bentuk tiga dimensi bisa diwakili sebagai occupancy field, yaitu peluang sebuah titik berada di dalam benda, atau sebagai *signed distance field* $f(x)$, jarak bertanda dari titik ke permukaan, positif di luar dan negatif di dalam (Park et al., 2019). Permukaannya adalah himpunan nol $f(x) = 0$, dan bisa diekstrak dengan algoritme seperti marching cubes. Fungsi jarak yang sejati memenuhi persamaan Eikonal, $\|\nabla f(x)\| = 1$. Menambahkan $(\|\nabla f_\theta\| - 1)^2$ ke loss di titik-titik acak memaksa jaringan menjadi fungsi jarak yang sah (Gropp et al., 2020). Inilah loss yang diinformasikan fisika: sebuah persamaan diferensial parsial ditegakkan di dalam loss.

NeRF, yang sudah kita lihat di Bab 11, adalah neural field untuk kerapatan dan warna yang dilatih melalui persamaan volume rendering. Kuliah menutup dengan pernyataan yang menjembatani ke Bagian III: dari sudut pandang ini, PINN hanyalah neural field yang diatur oleh persamaan fisika. Bedanya dengan NeRF hanya pada persamaan apa yang ditegakkan.

Satu masalah praktis menghubungkan semuanya. MLP biasa dengan aktivasi ReLU cenderung belajar komponen frekuensi rendah lebih dulu dan kesulitan mewakili detail halus, gejala yang disebut *spectral bias* (Rahaman et al., 2018). Dua obat yang populer adalah memetakan input dengan fungsi sinus dan kosinus berbagai frekuensi sebelum masuk jaringan (Tancik et al., 2020), atau memakai fungsi sinus sebagai aktivasi seperti pada SIREN (Sitzmann et al., 2020). Spectral bias juga menjadi salah satu alasan PINN sulit mempelajari solusi yang berosilasi cepat (Bab 29).

15.12 Di luar kelas

Alat-alat dari bab ini menyebar jauh. Model difusi membangkitkan gambar dan video (Bab 38), dan modul struktur AlphaFold3 adalah model difusi yang membangkitkan koordinat atom (Bab 24). Teori estimasi dan batas Cramér–Rao adalah bahasa sehari-hari pemrosesan sinyal (Bab 18). Probabilistic PCA dan analisis faktor dipakai untuk meringkas data ekspresi gen yang berisik (Bab 22). Neural ODE cocok untuk data medis yang diukur pada waktu tidak teratur, seperti kunjungan pasien yang jaraknya berbeda-beda.

Bagi Bagian V, bab ini memberi hampir semua alatnya. Neural ODE adalah cara alami untuk mempelajari dinamika sistem fisika dari data. Neural field adalah kerangka PINN. Model difusi dan flow matching dipakai untuk membangkitkan medan turbulen

yang realistis, untuk memperbaiki prediksi jangka panjang model pengganti (Lippe et al., 2023), dan untuk prakiraan cuaca probabilistik (Price et al., 2025). Bahkan cara berpikirnya pun berasal dari fisika: model difusi adalah persamaan panas pada ruang distribusi, dan flow matching adalah persamaan kontinuitas. Batas Cramér–Rao mengingatkan bahwa ketika kita mencoba menyimpulkan parameter fisika dari data, informasi yang terkandung di dalam data membatasi apa yang bisa dicapai oleh metode mana pun.

CATATAN SUMBER. Bab ini mengikuti delapan unit kuliah Machine Learning: Advanced Techniques: teori estimasi, statistik Bayesian, probabilistic PCA, VAE dan ELBO, ODE, PDE, dan SDE, neural ODE, model difusi (DDPM, DDIM), score-based model dan flow matching, Fokker–Planck, dan neural field. Bacaan utamanya adalah makalah-makalah aslinya: Tipping and Bishop (1999), Kingma and Welling (2013), Chen et al. (2018), Ho et al. (2020), Song et al. (2020b), Lipman et al. (2022), dan ulasan Luo (2022).

16 Belajar dari Ganjaran: Reinforcement Learning

Reinforcement learning datang kepada saya dalam dua tahap. Pada musim panas 2025, kuliah deep reinforcement learning langsung membawa kami ke imitation learning, policy gradient, dan credit assignment, dengan proyek latihan berupa agen DQN, agen PPO, dan sebuah tantangan akhir. Pada musim dingin berikutnya, kuliah reinforcement learning mengisi fondasinya secara rapi: proses keputusan Markov, dynamic programming, metode Monte Carlo, dan temporal difference. Bab ini menyusunnya dalam urutan yang lebih logis, fondasi dulu, lalu deep RL.

Dua tipe belajar di bab-bab sebelumnya selalu punya data yang sudah tersedia. Reinforcement learning berbeda. Agen harus mengumpulkan datanya sendiri dengan bertindak, dan umpan baliknya bukan jawaban yang benar, melainkan angka ganjaran yang sering datang terlambat. Seekor anjing yang belajar duduk tidak diberi tahu gerakan otot apa yang harus dilakukan. Ia mencoba sesuatu, dan kadang mendapat camilan. Tugasnya adalah mencari tahu tindakan mana yang membuat camilan itu datang.

16.1 Proses keputusan Markov

Kerangka formalnya adalah *Markov decision process* (MDP): himpunan keadaan $\mathcal{S}$, himpunan aksi $\mathcal{A}$, dinamika $p(s', r | s, a)$ yang memberi peluang keadaan berikutnya dan ganjarannya, serta faktor diskon $\gamma \in [0, 1)$ (Sutton and Barto, 2018). Di setiap langkah, agen melihat keadaan s_t , memilih aksi a_t menurut *policy* $\pi(a | s)$, lalu menerima ganjaran r_{t+1} dan keadaan baru. Tujuannya memaksimalkan *return*, jumlah ganjaran yang didiskon,

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (16.1)$$

Faktor diskon membuat ganjaran yang jauh di masa depan bernilai lebih kecil, dan secara matematis menjaga jumlahnya tetap terhingga.

Seberapa baik berada di sebuah keadaan diukur oleh *value function* $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$, dan seberapa baik mengambil sebuah aksi di sebuah keadaan oleh *action-value function* $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$. Kedua fungsi memenuhi persamaan rekursif yang menjadi jantung seluruh bidang ini, yaitu persamaan Bellman:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')]. \quad (16.2)$$

Nilai sebuah keadaan sama dengan ganjaran langsung ditambah nilai keadaan berikutnya yang didiskon, dirata-ratakan atas semua yang mungkin terjadi. Untuk policy

terbaik, rata-rata atas aksi diganti dengan maksimum, dan kita mendapat persamaan Bellman optimal, $V^*(s) = \max_a \sum_{s',r} p(s', r | s, a)[r + \gamma V^*(s')]$.

Persamaan ini turun langsung dari definisi return. Karena $G_t = r_{t+1} + \gamma G_{t+1}$, ekspektasinya dari keadaan s adalah $\mathbb{E}[r_{t+1} + \gamma G_{t+1} | s_t = s]$. Ekspektasi ini dihitung bertahap: pilih aksi menurut π , dapatkan ganjaran dan keadaan berikutnya menurut p , lalu ganti $\mathbb{E}[G_{t+1} | s_{t+1} = s']$ dengan $V^\pi(s')$ berkat sifat Markov, karena masa depan setelah s' tidak bergantung pada bagaimana kita sampai di sana.

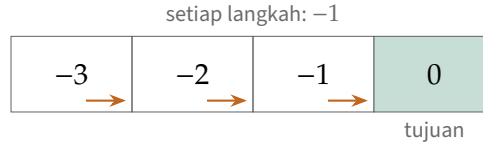
Contoh paling kecil sudah memperlihatkan maknanya. Bayangkan satu keadaan dengan satu aksi yang selalu memberi ganjaran satu dan kembali ke keadaan yang sama. Persamaan Bellman menjadi $V = 1 + \gamma V$, sehingga $V = 1/(1 - \gamma)$. Dengan $\gamma = 0,9$, nilainya 10. Faktor diskon bisa dibaca sebagai cakrawala efektif: dengan $\gamma = 0,9$, agen kira-kira peduli pada sepuluh langkah ke depan.

16.2 Dynamic programming

Kalau dinamika MDP diketahui, persamaan Bellman bisa diselesaikan dengan dynamic programming (Bellman, 1957). *Policy evaluation* menghitung V^π dengan menerapkan persamaan Bellman berulang-ulang sebagai aturan update sampai nilainya tidak berubah lagi. Karena faktor diskon membuat update ini kontraktif, iterasinya pasti konvergen. *Policy improvement* lalu membuat policy baru yang serakah terhadap V^π , memilih di setiap keadaan aksi dengan nilai harapan tertinggi. Teorema perbaikan policy menjamin policy baru ini tidak lebih buruk dari yang lama. Bergantian antara keduanya disebut *policy iteration*, dan ia berakhir di policy optimal dalam jumlah langkah terhingga untuk MDP terhingga. *Value iteration* memadatkan keduanya dengan langsung menerapkan persamaan Bellman optimal sebagai update.

Kuliah memakai dunia grid kecil sebagai contoh. Agen bergerak di petak-petak, setiap langkah memberi ganjaran -1 , dan permainan berakhir di petak sudut. Setelah policy evaluation untuk policy acak, nilai setiap petak mencerminkan rata-rata jumlah langkah yang dibutuhkan untuk mencapai sudut dengan berjalan acak. Policy yang serakah terhadap nilai-nilai itu sudah langsung menunjuk jalur terpendek.

Value iteration bisa diikuti dengan tangan pada lorong empat petak (Gambar 16.1). Agen bisa melangkah ke kiri atau ke kanan, setiap langkah memberi -1 , petak paling kanan adalah tujuan dengan nilai nol, dan $\gamma = 1$. Mulai dari $V = 0$ di mana-mana. Setelah satu sapuan, setiap petak selain tujuan bernilai -1 , karena satu langkah apa pun memberi -1 dan nilai petak tetangga masih nol. Setelah sapuan kedua, petak tepat di sebelah tujuan tetap -1 (melangkah ke tujuan), sedangkan petak lain menjadi -2 . Setelah sapuan ketiga, nilainya $(-3, -2, -1, 0)$, dan sapuan berikutnya tidak mengubah apa pun. Nilai setiap petak adalah minus jarak ke tujuan, dan policy serakah selalu melangkah ke kanan. Informasi menyebar mundur dari tujuan satu petak per sapuan, seperti gelombang. Pada MDP yang lebih besar, gelombang yang sama juga merambat, hanya lebih lama.



GAMBAR 16.1. Lorong empat petak setelah value iteration konvergen. Nilai setiap petak adalah minus jaraknya ke tujuan, dan policy serakah (panah) selalu melangkah ke kanan.

16.3 Belajar dari pengalaman

Di dunia nyata, dinamika jarang diketahui. Agen harus belajar dari sampel.

Metode *Monte Carlo* menunggu sampai satu episode selesai, lalu memakai return yang benar-benar diperoleh sebagai perkiraan nilai setiap keadaan yang dikunjungi. Perkiraan ini tak bias, tetapi variansnya besar, karena satu return bergantung pada semua keputusan acak sampai akhir episode. Kuliah memakai blackjack sebagai contoh: setelah ratusan ribu permainan acak dengan sebuah policy sederhana, rata-rata return dari setiap keadaan kartu menggambarkan permukaan nilai yang mulus, tanpa pernah menuliskan peluang kartu. Ada dua varian. *First-visit* hanya memakai return sejak kunjungan pertama ke sebuah keadaan dalam satu episode, *every-visit* memakai setiap kunjungan. Untuk episode s_1, s_2, s_1 , selesai dengan ganjaran 1, 0, dan 2 dan $\gamma = 1$, return sejak kunjungan pertama ke s_1 adalah 3 dan sejak kunjungan kedua 2. *First-visit* mencatat 3, *every-visit* mencatat 3 dan 2. Keduanya konvergen ke nilai yang benar. Rata-rata ini bisa diperbarui bertahap, $V \leftarrow V + \frac{1}{n}(G - V)$, bentuk yang langsung menjadi model untuk TD di bawah dengan $\frac{1}{n}$ diganti laju belajar tetap.

Metode *temporal difference* (TD) tidak menunggu. Setelah satu langkah, ia memperbarui perkiraan dengan memakai perkiraannya sendiri tentang keadaan berikutnya,

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]. \quad (16.3)$$

Selisih di dalam kurung disebut *TD error*. Pendekatan ini, belajar tebakan dari tebakan, disebut *bootstrapping*. Variansnya kecil karena hanya satu langkah acak yang terlibat, tetapi ada bias selama perkiraan $V(s_{t+1})$ belum benar. Di antara keduanya terbentang metode n -langkah dan eligibility traces yang mencampur keduanya.

Bayangkan memperkirakan waktu perjalanan pulang dari kantor. Metode Monte Carlo menunggu sampai Anda tiba di rumah, lalu mencatat total waktunya. Metode TD memperbarui perkiraan di setiap persimpangan: “jalanan ternyata macet, perkiraan saya untuk sisa perjalanan harus naik”, tanpa menunggu tiba.

Untuk mengendalikan, bukan hanya menilai, kita mempelajari Q . SARSA memperbarui $Q(s, a)$ dengan aksi berikutnya yang benar-benar diambil, sehingga ia belajar nilai policy yang sedang dijalankan. Q-learning (Watkins and Dayan, 1992) memakai aksi terbaik di keadaan berikutnya,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)], \quad (16.4)$$

sehingga ia belajar nilai policy optimal sambil menjalankan policy lain yang lebih eksploratif. Perbedaan ini disebut on-policy dan off-policy.

Satu update dengan angka: misalkan $Q(s, a) = 2$, agen menerima $r = 1$, dan di keadaan berikutnya nilai aksi terbaik $\max_{a'} Q(s', a') = 5$. Dengan $\gamma = 0,9$ dan $\alpha = 0,1$, targetnya $1 + 0,9 \cdot 5 = 5,5$, TD error-nya $3,5$, dan $Q(s, a)$ naik menjadi $2,35$. Langkah kecil ke arah target, persis seperti SGD dengan target yang diambil dari perkiraan sendiri.

Perbedaan on-policy dan off-policy terlihat jelas pada contoh klasik Sutton and Barto (2018), berjalan di tepi tebing. Jalur terpendek menyusuri tepi tebing, dan setiap langkah yang jatuh ke tebing memberi hukuman besar. Dengan eksplorasi ϵ -greedy, sesekali agen melangkah acak. Q-learning mempelajari nilai policy optimal yang tidak pernah salah langkah, sehingga ia memilih jalur di tepi tebing, lalu sesekali jatuh karena eksplorasinya. SARSA mempelajari nilai policy yang benar-benar dijalankan, termasuk langkah acaknya, sehingga ia memilih jalur yang lebih jauh dari tepi dan jarang jatuh. Selama eksplorasi masih berjalan, SARSA mendapat ganjaran rata-rata yang lebih baik. Kalau ϵ diturunkan ke nol, keduanya bertemu di jalur optimal.

Di sini muncul dilema yang tidak dikenal di supervised learning: *exploration* melawan *exploitation*. Kalau agen selalu memilih aksi yang sejauh ini terbaik, ia mungkin tidak pernah menemukan aksi yang sebenarnya lebih baik. Kalau ia terus mencoba hal baru, ia membuang ganjaran. Strategi paling sederhana adalah ϵ -greedy: pilih aksi terbaik hampir selalu, tetapi dengan peluang kecil ϵ pilih aksi acak. Ini mirip memilih tempat makan. Kalau selalu ke warung langganan, kita tidak pernah tahu ada warung yang lebih enak di gang sebelah.

16.4 Deep Q-network

Untuk keadaan yang amat banyak, seperti piksel layar permainan, tabel Q diganti jaringan saraf $Q_{\theta}(s, a)$. Gabungan ini ternyata tidak stabil, karena data yang dikumpulkan berurutan sangat berkorelasi dan targetnya terus berubah seiring jaringannya sendiri berubah. DQN (Mnih et al., 2015) memperbaikinya dengan dua trik. *Experience replay* menyimpan pengalaman di sebuah buffer dan melatih jaringan dengan mini-batch acak dari buffer itu, sehingga korelasi terputus. *Target network* adalah salinan jaringan yang diperbarui jarang-jarang dan dipakai untuk menghitung target, sehingga target tidak bergeser di setiap langkah. Dengan dua trik ini, satu arsitektur yang sama belajar memainkan puluhan permainan Atari dari piksel. Operator maksimum dalam target Q-learning cenderung melebih-lebihkan nilai. Alasannya statistik: kalau sepuluh aksi sebenarnya bernilai sama, tetapi perkiraannya berisik, maksimum dari sepuluh perkiraan berisik hampir selalu lebih besar dari nilai sebenarnya. Double DQN memisahkan pemilihan aksi dari penilaiannya: jaringan utama memilih aksi terbaik, $a^* = \arg \max_{a'} Q_{\theta}(s', a')$, dan jaringan target menilai aksi itu, $Q_{\theta^-}(s', a^*)$. Karena noise kedua jaringan tidak sama, bias ke atas itu sebagian besar hilang.

16.5 Policy gradient

Pendekatan lain melewati fungsi nilai dan langsung mengoptimalkan policy berparameter $\pi_{\theta}(a | s)$. Teorema policy gradient memberi gradien return harapan $J(\theta)$ dalam

bentuk yang bisa diperkirakan dari sampel,

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]. \quad (16.5)$$

Kuncinya adalah *log-derivative trick*. Untuk satu lintasan τ dengan peluang $p_{\theta}(\tau)$ dan return $G(\tau)$, gradien return harapan adalah

$$\nabla_{\theta} \int p_{\theta}(\tau) G(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) G(\tau) d\tau,$$

karena $\nabla p = p \nabla \log p$. Bentuk kedua adalah sebuah ekspektasi, sehingga bisa diperkirakan dengan merata-ratakan atas lintasan yang dijalankan. Peluang lintasan adalah hasil kali peluang aksi dari policy dan peluang transisi dari lingkungan. Setelah logaritma diambil, hasil kali itu menjadi jumlah, dan suku-suku transisi lingkungan tidak bergantung pada θ , sehingga hilang saat diturunkan. Yang tersisa hanya $\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$. Inilah alasan policy gradient bisa dipakai tanpa model lingkungan sama sekali. Mengganti $G(\tau)$ dengan return sejak langkah t saja diperbolehkan, karena aksi di langkah t tidak bisa memengaruhi ganjaran yang sudah diterima sebelumnya.

Algoritme REINFORCE (Williams, 1992) memakai rumus ini langsung: naikan peluang aksi yang diikuti return tinggi, turunkan peluang aksi yang diikuti return rendah. Estimasi ini tak bias, tetapi variansnya besar. Mengurangkan sebuah *baseline* yang tidak bergantung pada aksi, biasanya perkiraan $V(s_t)$, tidak mengubah ekspektasinya tetapi mengurangi variansnya. Selisih $A_t = G_t - V(s_t)$ disebut *advantage*: seberapa jauh aksi ini lebih baik daripada rata-rata. Metode *actor-critic* melatih dua jaringan sekaligus, aktor yang memilih aksi dan kritikus yang menaksir nilai (Mnih et al., 2016), dan *generalized advantage estimation* (Schulman et al., 2015b) mencampur perkiraan advantage dari berbagai panjang langkah untuk mengatur keseimbangan bias dan varians.

Policy gradient punya kelemahan praktis: satu langkah yang terlalu besar bisa merusak policy, dan data baru yang dikumpulkan dengan policy rusak itu ikut buruk, sehingga pelatihan bisa runtuh. TRPO (Schulman et al., 2015a) membatasi setiap update agar policy baru tidak terlalu jauh dari yang lama dalam arti divergensi KL. PPO (Schulman et al., 2017) mencapai efek serupa dengan cara yang jauh lebih sederhana, yaitu memotong rasio peluang $\rho_t = \pi_{\theta}(a_t | s_t) / \pi_{\theta_{\text{lama}}}(a_t | s_t)$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t) \right]. \quad (16.6)$$

Kalau sebuah aksi bagus ($A_t > 0$), peluangnya boleh naik, tetapi tidak lebih dari faktor $1 + \epsilon$ dalam satu update. Kalau aksi buruk, peluangnya boleh turun, tetapi tidak lebih dari faktor $1 - \epsilon$. Contoh dengan angka: dengan $\epsilon = 0,2$ dan advantage $A_t = 2$, kalau policy baru menaikkan peluang aksi itu menjadi 1,5 kali, suku pertama memberi 3, tetapi suku yang dipotong memberi $1,2 \cdot 2 = 2,4$, dan minimum keduanya, 2,4, yang dipakai. Gradien terhadap ρ_t di daerah itu nol, sehingga tidak ada dorongan untuk menaikkan peluangnya lebih jauh dalam update yang sama. PPO menjadi algoritme bawaan di banyak aplikasi karena kuat dan mudah disetel. Untuk aksi kontinu, soft actor-critic (Haarnoja et al., 2018) menambahkan entropi policy ke dalam tujuan, sehingga agen dihargai karena tetap acak selama hal itu tidak terlalu merugikan.

16.6 Meniru dan memberi kredit

16.6.1 *Imitation learning*

Kadang lebih mudah menunjukkan cara melakukan sesuatu daripada menulis fungsi ganjarannya. *Behavioral cloning* memperlakukan demonstrasi ahli sebagai data supervised: input keadaan, target aksi. Masalahnya adalah *covariate shift*. Begitu agen sedikit menyimpang dari jalur ahli, ia sampai di keadaan yang tidak pernah dilihat ahli, membuat kesalahan lebih besar, dan menyimpang semakin jauh. DAGger (Ross et al., 2010) mengatasinya dengan menjalankan policy agen, meminta ahli memberi label pada keadaan yang dikunjungi agen, lalu melatih ulang. Pendekatan lain mempelajari fungsi ganjaran dari demonstrasi (*inverse reinforcement learning*), atau mempertandingkan agen dengan diskriminator yang membedakan perilakunya dari perilaku ahli, seperti pada GAIL (Ho and Ermon, 2016).

16.6.2 *Credit assignment*

Masalah tersulit RL adalah ganjaran yang tertunda. Dalam sebuah pertandingan sepak bola, gol terjadi di menit ke-80, tetapi umpan yang menentukan mungkin terjadi lima menit sebelumnya, dan kesalahan lawan yang membuka ruang mungkin terjadi lebih awal lagi. Metode TD menyebarkan kredit mundur satu langkah demi satu langkah, sehingga untuk penundaan panjang, belajarnya lambat sekali. RUDDER (Arjona-Medina et al., 2018), yang dikembangkan di Linz, mengambil pendekatan berbeda. Sebuah LSTM dilatih untuk memprediksi return total dari seluruh urutan keadaan dan aksi. Lalu, dengan menganalisis seberapa besar setiap langkah mengubah prediksi itu, return dibagi ulang ke langkah-langkah yang benar-benar berkontribusi. Ganjaran yang tertunda diubah menjadi ganjaran langsung, dan nilai harapan yang tersisa untuk dipelajari menjadi hampir nol. Kuliah deep RL di Linz membahas topik ini dengan nama direct credit assignment.

16.6.3 *Kendali sebagai inferensi*

Kuliah juga membahas cara pandang yang menyatukan RL dengan inferensi probabilistik (Bab 14). Kalau kita memperkenalkan variabel “optimalitas” yang peluangnya sebanding dengan eksponensial ganjaran, mencari policy optimal menjadi mirip menghitung posterior atas lintasan, dengan syarat bahwa lintasan itu optimal. Hasil dari cara pandang ini adalah RL dengan entropi maksimum, yang menjelaskan mengapa soft actor-critic menambahkan entropi ke dalam tujuannya.

16.7 Model, permainan, dan banyak agen

Buku Plaatt (2022), yang menjadi bacaan kuliah, menyusun deep RL menjadi beberapa keluarga. Setiap keluarga menjawab satu kelemahan metode model-free yang dibahas di atas.

16.7.1 Belajar dengan model

Metode model-free butuh interaksi yang amat banyak. DQN memainkan setiap permainan Atari puluhan juta langkah. Metode berbasis model mempelajari dinamika lingkungan, $\hat{p}(s', r | s, a)$, lalu memakai model itu untuk merencanakan atau untuk menghasilkan pengalaman buatan. Arsitektur Dyna (Sutton, 1990) adalah contoh paling sederhana. Setiap langkah nyata dipakai dua kali: sekali untuk memperbarui Q seperti biasa, dan sekali untuk memperbarui model. Lalu, sebelum langkah nyata berikutnya, agen melakukan beberapa pembaruan Q tambahan dengan pengalaman yang dibangkitkan model dari keadaan yang pernah dikunjungi. Kalau sepuluh langkah khayalan dilakukan untuk setiap langkah nyata, informasi dari satu interaksi menyebar jauh lebih cepat ke seluruh tabel nilai.

Bahayanya adalah galat model. Policy yang dioptimalkan di dalam model yang keliru akan mengeksploitasi kekeliruan itu, persis seperti optimasi molekul yang mengeksploitasi celah model QSAR (Bab 23). Galat juga menumpuk sepanjang rencana yang panjang, masalah rollout yang sama dengan Bab 30. Karena itu metode modern biasanya merencanakan hanya beberapa langkah ke depan, memakai ensemble model untuk menaksir ketidakpastiannya, atau seperti MuZero, mempelajari model yang hanya meramalkan besaran yang dibutuhkan untuk merencanakan, yaitu nilai, policy, dan ganjaran, bukan seluruh pengamatan.

16.7.2 Banyak agen

Ketika beberapa agen belajar bersamaan, lingkungan setiap agen berubah karena agen lain juga berubah, dan jaminan konvergensi dari MDP tunggal hilang. Teori permainan memberi bahasa untuk situasi ini. Sebuah *Nash equilibrium* adalah kombinasi strategi di mana tidak ada pemain yang bisa untung dengan mengubah strateginya sendiri sementara yang lain tetap.

Batu-gunting-kertas memperlihatkan mengapa strategi acak bisa optimal. Misalkan lawan memainkan batu, gunting, dan kertas dengan peluang p_b , p_g , dan p_k , dengan menang bernilai +1, kalah -1, dan seri 0. Nilai harapan memainkan batu adalah $p_g - p_k$, gunting $p_k - p_b$, dan kertas $p_b - p_g$. Kalau salah satu nilai ini positif, kita akan selalu memainkan aksi itu, dan lawan bisa mengeksploitasinya. Satu-satunya keadaan di mana tidak ada aksi yang lebih unggul adalah ketika ketiganya nol, yaitu $p_b = p_g = p_k = \frac{1}{3}$. Ekuilibriumnya adalah bermain acak sempurna. Dalam permainan seperti ini, self-play yang naif bisa berputar-putar tanpa henti: agen belajar mengalahkan versi lamanya, lalu versi berikutnya belajar mengalahkan versi itu, dan seterusnya. Karena itu sistem seperti AlphaZero (Silver et al., 2018) dan agen permainan strategi modern melatih agen melawan kumpulan versi-versi lamanya, bukan hanya versi terakhir.

16.7.3 Hierarki, meta-learning, dan eksplorasi

Tugas yang panjang, seperti memasak sebuah resep, lebih mudah dipelajari kalau dipecah menjadi subtugas: ambil bahan, potong, panaskan. Kerangka *options* (Sutton et al., 1999) memperlakukan subtugas seperti itu sebagai aksi yang diperluas dalam waktu, masing-masing dengan policy dan syarat berhentinya sendiri. Policy tingkat

tinggi memilih option, policy tingkat rendah menjalankannya. Credit assignment menjadi lebih mudah, karena keputusan tingkat tinggi lebih sedikit dan berjarak lebih dekat dengan ganjarannya.

Meta-learning bertanya bagaimana agen bisa cepat beradaptasi ke tugas baru. MAML (Finn et al., 2017) mencari parameter awal θ yang, setelah satu atau beberapa langkah gradien pada tugas baru, sudah bekerja baik di tugas itu. Untuk setiap tugas i , parameter yang sudah disesuaikan adalah $\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_i(\theta)$, dan parameter awal diperbarui untuk meminimalkan $\sum_i \mathcal{L}_i(\theta'_i)$, yaitu loss *setelah* adaptasi. Yang dicari bukan parameter yang baik untuk semua tugas sekaligus, melainkan titik awal yang dekat ke solusi setiap tugas. Gagasan ini berlaku di luar RL, misalnya untuk klasifikasi dengan sedikit contoh.

Eksplorasi yang cerdas menjadi penting ketika ganjaran jarang sekali. Bonus eksplorasi berbasis hitungan menambahkan ganjaran buatan $\beta/\sqrt{N(s)}$ untuk keadaan yang baru dikunjungi $N(s)$ kali, gagasan yang sama dengan bonus keingintahuan pada UCT di Bab 5. Untuk keadaan berupa piksel, setiap layar praktis unik, sehingga hitungan langsung tidak berguna. Bellemare et al. (2016) menurunkan “hitungan semu” dari sebuah model kerapatan atas keadaan, dan memperlihatkan bahwa bonus seperti ini membuat agen menjelajahi permainan Atari yang ganjarannya jarang jauh lebih baik.

16.8 Reinforcement learning di dunia nyata

Di luar permainan, RL dipakai untuk masalah kendali yang sulit dimodelkan secara analitis. Rabault et al. (2019) melatih agen untuk mengendalikan jet kecil di permukaan silinder dalam simulasi CFD, dan agen itu menemukan strategi yang mengurangi gaya hambat. Ulasan Garnier et al. (2021) merangkum puluhan studi serupa dalam mekanika fluida, dari kendali aliran sampai optimasi bentuk. Dalam robotika, agen dilatih di simulator lalu dipindahkan ke robot nyata. Dalam sistem rekomendasi, memilih item yang ditampilkan kepada pengguna bisa dirumuskan sebagai masalah keputusan berurutan.

Penerapan yang paling berpengaruh beberapa tahun terakhir justru datang dari bahasa. Model bahasa yang sudah dilatih kemudian disetel dengan *reinforcement learning from human feedback*. Manusia membandingkan pasangan jawaban model, sebuah model ganjaran dilatih dari preferensi itu (Christiano et al., 2017), lalu model bahasa dioptimasi dengan PPO terhadap model ganjaran tersebut (Ouyang et al., 2022). Perkembangan ini kita bahas lagi di Bab 38.

16.9 Perkembangan riset

Dua arah riset yang menonjol beberapa tahun terakhir menyentuh dua pertanyaan besar bab ini: dari mana data datang, dan bagaimana memakai model. DreamerV3 (Hafner et al., 2023) mempelajari model dunia dari pengalaman, lalu memperbaiki perilakunya dengan membayangkan skenario masa depan di dalam model itu. Dengan teknik normalisasi dan transformasi yang membuat skala ganjaran dan nilai tidak lagi perlu disetel per tugas, satu konfigurasi yang sama bekerja di lebih dari 150 tugas, dan untuk pertama kalinya sebuah algoritme mengumpulkan berlian di Minecraft dari nol

tanpa data manusia atau kurikulum.

Decision Transformer (Chen et al., 2021) membuang fungsi nilai dan policy gradient sama sekali. Untuk data yang sudah terkumpul (offline RL), setiap lintasan ditulis sebagai deret token berisi return yang tersisa, keadaan, dan aksi. Transformer dengan causal mask dilatih menebak aksi berikutnya. Saat dipakai, kita cukup memasukkan return yang diinginkan sebagai token pertama, dan model membangkitkan aksi yang konsisten dengan target itu. Reinforcement learning berubah menjadi pemodelan deret bersyarat, dan seluruh perkakas model bahasa dari Bab 9 bisa dipakai. Pendekatan ini juga menjadi jembatan ke cara model bahasa besar dilatih dengan RL di Bab 38.

CATATAN SUMBER. Bab ini menggabungkan kuliah Reinforcement Learning (MDP, dynamic programming, Monte Carlo, temporal difference, policy gradient, PPO) dan kuliah serta latihan Deep Reinforcement Learning (imitation learning, policy gradient, direct credit assignment, kendali dan inferensi, proyek DQN dan PPO). Bacaan utama: Sutton and Barto (2018) dan Plaatt (2022).

17 Membuka Kotak Hitam: AI dan Visualisasi

Kuliah AI and visualization saya ikuti pada musim dingin 2025/26, diberikan oleh kelompok Visual Data Science di Linz. Kuliahnya bertumpu pada satu rumus yang muncul sejak pertemuan pertama: AI ditambah visualisasi ditambah manusia sama dengan explainable AI. Penjelasan bukan sesuatu yang dikeluarkan mesin begitu saja. Ia harus dirancang untuk seseorang, dan visualisasi adalah salah satu alat terbaik untuk menyampaikannya.

Model yang paling akurat sering yang paling sulit dipahami. Untuk menyarankan film, itu tidak masalah. Untuk menolak kredit, mendiagnosis penyakit, atau menyetujui desain sebuah reaktor, kita perlu tahu alasan di balik keputusan. Bab ini membahas dua arah hubungan antara AI dan visualisasi: visualisasi untuk memahami AI, dan AI untuk membuat visualisasi.

17.1 Siapa yang butuh penjelasan, dan untuk apa

Penjelasan yang dibutuhkan pengembang model berbeda dari yang dibutuhkan dokter, regulator, atau orang yang permohonan kreditnya ditolak. Pengembang ingin menemukan bug dan bias. Pengguna ahli ingin tahu kapan model bisa dipercaya. Orang yang terdampak ingin tahu apa yang harus diubah agar keputusannya berbeda. Kuliah menyusun teknik-teknik XAI menurut pertanyaan pengguna: mengapa prediksi ini, mengapa bukan yang lain, bagaimana kalau input diubah, dan contoh mana yang mirip.

Ada juga pertukaran yang sering digambarkan antara kinerja dan keterjelasan. Regresi linear dan pohon keputusan kecil mudah dibaca, jaringan dalam tidak. Rudin (2019) berpendapat bahwa untuk keputusan berisiko tinggi, kita sebaiknya memakai model yang bisa ditafsirkan sejak awal daripada menjelaskan kotak hitam setelah jadi, karena penjelasan sesudahnya bisa saja tidak setia pada apa yang sebenarnya dilakukan model.

Kuliah membedakan dua arah. *Vis for ML* memakai visualisasi untuk memahami, men-debug, dan membandingkan model. *ML for Vis* memakai machine learning untuk membantu membuat visualisasi, misalnya merekomendasikan jenis grafik atau menghasilkan dashboard dari permintaan dalam bahasa biasa.

17.2 Dasar-dasar visualisasi

Visualisasi bekerja karena sistem visual manusia memproses pola dengan cepat. Tetapi tidak semua saluran visual sama kuatnya. Munzner (2014) menyusun peringkat untuk data kuantitatif: posisi pada sumbu yang sama paling akurat dibaca, disusul

panjang, sudut, luas, lalu kecerahan dan saturasi warna. Itulah sebabnya grafik batang biasanya lebih jujur daripada diagram lingkaran, dan mengapa grafik tiga dimensi yang memiringkan sumbu sering menyesatkan. Prinsip Gestalt menjelaskan bagaimana mata mengelompokkan objek: berdekatan, mirip warna atau bentuk, terhubung garis, atau berada di dalam wadah yang sama.

Visualisasi juga bisa bercerita. Cerita yang digerakkan penulis membawa pembaca melalui urutan yang sudah ditentukan, seperti artikel berita. Cerita yang digerakkan pembaca membiarkan orang menjelajah sendiri. Banyak visualisasi terbaik memakai bentuk campuran: dimulai dengan narasi, lalu membuka ruang untuk dijelajahi.

Kuliah memakai visualisasi untuk menjelaskan algoritme. Algoritme pengurutan bisa digambar sebagai garis-garis yang bertukar tempat dari langkah ke langkah. k -means bisa dianimasikan sebagai pusat yang bergeser. Teorema Bayes bisa dijelaskan dengan kotak berisi seratus orang. Contoh yang dibahas adalah tes penyakit dengan sensitivitas 90 persen, tingkat positif palsu 9 persen, dan prevalensi 1 persen. Dari seribu orang, sepuluh sakit dan sembilan di antaranya terdeteksi, sedangkan dari 990 orang sehat, sekitar 89 mendapat hasil positif palsu. Peluang benar-benar sakit setelah hasil positif hanya $9/(9 + 89) \approx 9$ persen. Banyak orang, termasuk dokter, menebak jauh lebih tinggi. Gambar seratus kotak kecil membuat hasil ini langsung terasa masuk akal.

17.3 Menjelaskan lewat proyeksi

Data berdimensi tinggi, termasuk aktivasi di dalam jaringan saraf, bisa diproyeksikan ke dua dimensi untuk dilihat. PCA (Bab 6) mempertahankan varians global, t-SNE (van der Maaten and Hinton, 2008) mempertahankan tetangga lokal, dan UMAP (McInnes et al., 2018) berusaha menjaga keduanya. Kuliah meluangkan waktu khusus untuk salah baca yang umum pada t-SNE: hyperparameter seperti perplexity amat menentukan, ukuran kelompok di peta tidak berarti apa-apa, jarak antar kelompok bisa menipu, noise acak kadang tampak berpola, dan untuk melihat topologi kita mungkin butuh beberapa peta dengan setelan berbeda.

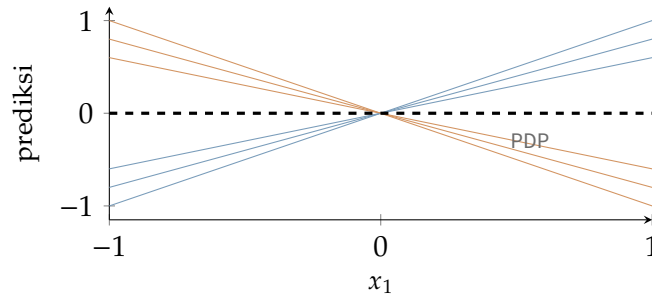
Salah baca itu bisa dipahami dari cara t-SNE bekerja. Untuk setiap titik di ruang asal, t-SNE mengubah jarak ke titik-titik lain menjadi peluang bertetangga p_{ij} dengan kernel Gauss, yang lebarnya dipilih per titik agar setiap titik punya kira-kira sejumlah tetangga efektif yang sama, yaitu perplexity. Di peta dua dimensi, peluang q_{ij} dihitung dengan distribusi Student- t berekor tebal, lalu posisi titik di peta digeser untuk meminimalkan $KL(P||Q) = \sum p_{ij} \log(p_{ij}/q_{ij})$. Karena lebar kernel disesuaikan per titik, kelompok yang rapat dan kelompok yang renggang di ruang asal sama-sama tampil dengan ukuran serupa di peta. Dan karena KL ini menghukum keras tetangga dekat yang dipisahkan, tetapi hampir tidak peduli pada titik jauh yang didekatkan, jarak antar kelompok di peta tidak punya arti yang bisa diandalkan.

Proyeksi juga bisa menunjukkan proses yang berjalan dalam waktu. Kalau setiap keadaan sebuah proses, misalnya setiap langkah algoritme, setiap posisi permainan catur, atau setiap epoch pelatihan jaringan, diproyeksikan sebagai satu titik, lalu titik-titik itu dihubungkan, kita mendapat lintasan. Lintasan pelatihan dua jaringan bisa dibandingkan: apakah keduanya menempuh jalan yang mirip, atau satu berputar-putar

sebelum menemukan solusi. Kelompok visualisasi di Linz juga mengembangkan alat seperti ini untuk kimia, sehingga ahli kimia bisa menjelajahi ruang molekul yang dilihat oleh model prediksi mereka.

17.4 Menjelaskan dalam ruang input

Sekelompok teknik menjelaskan model dengan melihat bagaimana keluarannya berubah terhadap input. *Partial dependence plot* menampilkan efek rata-rata satu fitur terhadap prediksi, dengan merata-ratakan atas nilai fitur-fitur lain. *Individual conditional expectation* menampilkan kurva yang sama untuk setiap contoh secara terpisah, sehingga interaksi antar fitur yang tersembunyi di balik rata-rata menjadi terlihat. Ambil model $f(x_1, x_2) = x_1 x_2$ dengan x_2 bernilai $+1$ atau -1 sama seringnya. Separuh kurva ICE untuk x_1 naik, separuh lagi turun, dan PDP, rata-ratanya, datar sempurna (Gambar 17.1). Kalau hanya melihat PDP, kita akan menyimpulkan bahwa x_1 tidak berpengaruh sama sekali, padahal ia berpengaruh, hanya arahnya bergantung pada x_2 .



GAMBAR 17.1. Kurva ICE (garis tipis) untuk model dengan interaksi. Kurva naik untuk sebagian contoh dan turun untuk sebagian lain, sehingga rata-ratanya, PDP (garis putus-putus), datar dan menyembunyikan pengaruh fitur itu.

LIME (Ribeiro et al., 2016) menjelaskan satu prediksi dengan membangun model sederhana yang hanya berlaku di sekitar contoh itu. Contoh diganggu sedikit demi sedikit, misalnya dengan menyembunyikan potongan-potongan gambar (*superpixel*), model asli ditanya untuk setiap versi, lalu model linear dilatih untuk meniru jawabannya secara lokal. Secara formal, LIME mencari

$$g^* = \arg \min_{g \in G} \sum_z \pi_x(z) (f(z) - g(z))^2 + \Omega(g), \quad (17.1)$$

dengan z contoh-contoh yang diganggu, $\pi_x(z)$ bobot yang makin kecil untuk z yang makin jauh dari x , dan $\Omega(g)$ penalti kerumitan, misalnya batas jumlah fitur yang boleh dipakai. Ini regresi linear berbobot dengan regularisasi dari Bab 1. Bobot model linear itu menjadi penjelasan. Kelemahannya juga terlihat dari rumus ini: hasilnya bergantung pada cara mengganggu contoh dan pada lebar π_x , sehingga dua setelan yang sama-sama masuk akal bisa memberi penjelasan yang berbeda. Model surrogate global melakukan hal serupa untuk seluruh data, dengan risiko bahwa model sederhana itu tidak cukup setia.

Nilai Shapley berasal dari teori permainan kooperatif. Pertanyaannya: kalau sekelompok pemain bersama menghasilkan keuntungan, berapa bagian yang adil untuk setiap pemain? Jawaban Shapley adalah rata-rata kontribusi marginal pemain itu atas semua urutan kedatangan pemain. Dalam XAI, pemainnya adalah fitur dan keuntungannya adalah prediksi. Ambil contoh dua fitur, A dan B. Tanpa fitur apa pun, prediksi model 0. Dengan A saja 10, dengan B saja 20, dan dengan keduanya 40. Kalau A datang lebih dulu, kontribusinya 10, dan kalau datang setelah B, kontribusinya 20. Rata-ratanya 15. Dengan cara yang sama, B mendapat $(20 + 30)/2 = 25$. Jumlah keduanya tepat 40, prediksi penuh. Secara umum, untuk n fitur dan fungsi nilai $v(S)$ yang memberi prediksi dengan hanya memakai himpunan fitur S ,

$$\phi_i = \sum_{S \subseteq \{1, \dots, n\} \setminus \{i\}} \frac{|S|! (n - |S| - 1)!}{n!} (v(S \cup \{i\}) - v(S)). \quad (17.2)$$

Bobotnya adalah peluang bahwa, dalam urutan kedatangan yang acak, tepat fitur-fitur di S datang sebelum i : ada $|S|!$ cara menyusun yang datang lebih dulu, $(n - |S| - 1)!$ cara menyusun yang datang belakangan, dari $n!$ urutan. Nilai Shapley adalah satu-satunya pembagian yang memenuhi empat sifat sekaligus: jumlahnya sama dengan prediksi penuh dikurangi prediksi kosong, fitur yang berperan identik mendapat bagian sama, fitur yang tidak pernah berpengaruh mendapat nol, dan untuk jumlah dua model, nilainya juga dijumlahkan. SHAP (Lundberg and Lee, 2017) membuat perhitungan ini praktis untuk model dengan banyak fitur, karena jumlah urutan yang mungkin tumbuh secara faktorial. Yang sering terlupa adalah bahwa $v(S)$ harus didefinisikan: apa artinya model “tanpa” sebuah fitur? Biasanya fitur yang hilang diganti nilai dari data latar, dan kalau fitur-fitur saling berkorelasi, penggantian itu bisa menciptakan contoh yang tidak realistis, seperti orang berusia sepuluh tahun dengan gelar doktor.

Pertanyaan “bagaimana kalau” dijawab oleh *counterfactual explanation* (Wachter et al., 2017): cari contoh x' yang sedekat mungkin dengan x tetapi menghasilkan keputusan yang diinginkan, misalnya dengan meminimalkan $\lambda (f(x') - y')^2 + d(x, x')$. Jawabannya berbentuk kalimat yang langsung bisa dipakai: “kredit Anda akan disetujui kalau pendapatan Anda 400 euro lebih tinggi per bulan”. Penjelasan seperti ini tidak membuka isi model sama sekali, sehingga cocok untuk hak atas penjelasan dalam GDPR (Bab 35). Tetapi ia harus dibatasi pada perubahan yang mungkin: menyarankan seseorang menjadi lebih muda bukan penjelasan yang berguna.

17.5 Menjelaskan lewat gradien

Untuk jaringan saraf, gradien keluaran terhadap input memberi peta kepekaan langsung: piksel mana yang paling mengubah skor kelas kalau digeser sedikit. *Saliency map* vanilla (Simonyan et al., 2013) menampilkan besar gradien ini. Hasilnya sering berbintik dan berisik. Integrated gradients (Sundararajan et al., 2017) menjumlahkan gradien sepanjang jalur dari gambar acuan, misalnya gambar hitam, ke gambar asli, dan memenuhi sifat bahwa jumlah atribusinya sama dengan selisih keluaran:

$$\text{IG}_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial f}{\partial x_i}(x' + \alpha(x - x')) d\alpha. \quad (17.3)$$

Contoh kecil memperlihatkan mengapa sifat ini penting. Untuk $f(x_1, x_2) = x_1 x_2$ di $x = (2, 3)$ dengan acuan nol, di sepanjang jalur $\partial f / \partial x_1 = 3\alpha$, sehingga $IG_1 = 2 \int_0^1 3\alpha \, d\alpha = 3$, dan dengan cara yang sama $IG_2 = 3$. Jumlahnya 6, tepat $f(x) - f(0)$. Atribusi gradien kali input, yang memakai gradien di titik akhir saja, memberi $3 \cdot 2 = 6$ dan $2 \cdot 3 = 6$, dengan jumlah 12, dua kali lipat dari keluaran yang ingin dijelaskan. Grad-CAM (Selvaraju et al., 2016) memakai gradien terhadap peta fitur lapisan konvolusi terakhir untuk menimbang peta-peta itu. Bobot peta ke- k adalah rata-rata gradien skor kelas y^c terhadap peta itu, $\alpha_k = \frac{1}{Z} \sum_{i,j} \partial y^c / \partial A_{ij}^k$, dan peta panasnya $\text{ReLU}(\sum_k \alpha_k A^k)$, sehingga menghasilkan peta panas kasar yang menunjukkan daerah gambar yang dipakai model. Layer-wise relevance propagation (Bach et al., 2015) membagi skor keluaran mundur ke input lapisan demi lapisan dengan aturan kekekalan relevansi.

Kuliah memakai contoh dari sebuah studi klasifikasi jamur. Peta Grad-CAM kadang menunjukkan bahwa model melihat bagian yang memang penting secara biologis, seperti tudung dan bilah, tetapi kadang juga latar belakang, misalnya rumput atau tangan yang memegang jamur. Peta seperti ini tidak membuktikan bahwa model benar, tetapi amat berguna untuk menemukan jalan pintas yang tidak diinginkan.

Sebaliknya, peta yang tampak masuk akal juga belum tentu setia pada model. Adebayo et al. (2018) mengusulkan uji kewarasan yang sederhana: acak bobot jaringan lapisan demi lapisan, atau latih ulang dengan label acak, lalu lihat apakah peta penjelasannya ikut berubah. Beberapa metode populer menghasilkan peta yang hampir sama untuk jaringan terlatih dan jaringan acak. Metode seperti itu lebih banyak memperlihatkan tepi-tepi gambar daripada apa yang dipelajari model, seperti detektor tepi yang kebetulan rapi. Pelajarannya berlaku untuk semua XAI: penjelasan juga harus diuji, bukan hanya dinikmati.

17.6 Menjelaskan lewat contoh dan konsep

Manusia sering menjelaskan dengan contoh: “ini mirip kasus yang kemarin”. Model pun bisa menjelaskan prediksinya dengan menunjukkan contoh data latih yang paling mirip di ruang fiturnya. *Prototype* adalah contoh yang mewakili sebuah kelompok, dan *criticism* adalah contoh yang tidak terwakili dengan baik oleh prototype mana pun. Keduanya bersama memberi gambaran lebih jujur tentang data.

Penjelasan berbasis konsep bertanya apakah sebuah konsep yang bermakna bagi manusia, misalnya “bergaris” atau “berlatar salju”, tersimpan di dalam jaringan. *Network dissection* mencocokkan unit-unit di dalam jaringan dengan konsep yang dianotasi. TCAV (Kim et al., 2017) mempelajari arah di ruang aktivasi yang mewakili sebuah konsep dari contoh-contohnya, lalu mengukur seberapa peka prediksi sebuah kelas terhadap arah itu. Jenis temuan yang dicari mirip contoh terkenal dari Ribeiro et al. (2016), di mana penjelasan LIME mengungkap bahwa sebuah pengklasifikasi serigala dan anjing husky sebenarnya terutama melihat salju di latar belakang.

17.7 Visualisasi di era AI generatif

Pertemuan terakhir membahas arah sebaliknya: AI yang membuat visualisasi. Model bahasa bisa menulis kode grafik dari permintaan dalam bahasa biasa, menyusun dashboard, atau membuat penjelasan bagi pengguna baru. Kuliah menyoroti risikonya. Gambar yang dihasilkan bisa tampak meyakinkan tetapi salah secara data. Model membuat asumsi tentang maksud pengguna yang tidak pernah diucapkan. Dan jawabannya bergantung pada gambaran yang dibentuk model tentang siapa penggunanya. Kuliah mengusulkan antarmuka yang membuat asumsi-asumsi itu terlihat dan bisa dikoreksi pengguna.

17.8 Di luar kelas

Keterjelasan penting di setiap bidang yang dibahas buku ini, dengan alasan yang berbeda-beda. Dalam kedokteran, peta Grad-CAM pada citra rontgen bisa mengungkap model yang melihat penanda rumah sakit di sudut gambar alih-alih paru-paru. Dalam penemuan obat, atribusi pada atom-atom molekul membantu ahli kimia memutuskan bagian mana yang perlu diubah. Dalam pemrosesan sinyal dan sistem tertanam, kita perlu tahu sensor mana yang paling menentukan sebuah keputusan. Dalam simulasi, keterjelasan bisa diambil sejauh-jauhnya: metode seperti SINDy (Bab 31) tidak menjelaskan model kotak hitam, melainkan menghasilkan persamaan yang bisa dibaca langsung. Itu barangkali bentuk penjelasan yang paling disukai Rudin (2019).

17.9 Perkembangan riset

Untuk model bahasa besar, riset keterjelasan bergeser ke *mechanistic interpretability*: mencoba membaca mekanisme di dalam jaringan, bukan hanya atribusi pada input. Masalah utamanya adalah *polysemanticity*. Satu neuron sering aktif untuk banyak konsep yang tidak berhubungan, karena jaringan diduga menyimpan lebih banyak fitur daripada jumlah neuronnya, sebagai arah-arrah yang saling bertumpuk di ruang aktivasi (*superposition*). Cunningham et al. (2023) melatih sparse autoencoder untuk merekonstruksi aktivasi internal model bahasa dari kombinasi jarang arah-arrah yang jauh lebih banyak dari jumlah neuron. Fitur yang ditemukan lebih monosemantik, satu fitur untuk satu konsep, dan bisa dipakai untuk menunjuk fitur mana yang secara kausal bertanggung jawab atas sebuah perilaku. Gagasannya adalah analisis faktor jarang dari Bab 6, diterapkan pada aktivasi jaringan.

Arah lain membangun keterjelasan ke dalam arsitektur, sejalan dengan saran Rudin (2019). Concept bottleneck model (Koh et al., 2020) memaksa jaringan memprediksi dulu konsep-konsep yang bermakna bagi manusia, misalnya “ada taji tulang” pada citra rontgen lutut, lalu memprediksi label akhir hanya dari konsep itu. Dokter bisa melihat konsep mana yang dipercaya model, dan bisa mengoreksinya saat model dipakai. Koreksi seperti itu dilaporkan meningkatkan akurasi secara nyata.

CATATAN SUMBER. Bab ini mengikuti enam kuliah AI and Visualization beserta labnya: dasar visualisasi, menjelaskan algoritme, proyeksi, teknik penjelasan visual (PDP, ICE, LIME, Shapley,

saliency, Grad-CAM, contoh, konsep), dan AI generatif untuk visualisasi. Bacaan pendamping: Molnar (2022) untuk interpretable machine learning dan Munzner (2014) untuk visualisasi.

Bagian III

Sinyal, Sensor, dan Sistem

18 Menangkap Sinyal: Estimasi, Filter, dan Radar

Pemrosesan sinyal masuk ke perjalanan saya lewat dua mata kuliah. Kuliah optimum and adaptive signal processing systems, dengan latihan kelompok berisi enam tugas, membahas variabel dan proses acak, teori estimasi klasik dan Bayesian, filter optimal, dan filter adaptif. Pada musim dingin 2025/26, latihan radar system engineering mengajak kami membangun radar di dalam notebook Python: persamaan radar dalam skala desibel, distribusi noise, deteksi, korelasi dan matched filter, FFT, CFAR, radar FMCW, beamforming digital, sampai sebuah simulator radar sederhana.

Pemrosesan sinyal sudah “belajar dari data” jauh sebelum istilah machine learning populer. Filter adaptif yang meredam gema di telepon memperbarui bobotnya dengan gradien stokastik, persis seperti jaringan saraf. Banyak konsep di bab ini, dari batas Cramér–Rao sampai Kalman filter, sudah kita temui dari sisi machine learning. Di sini kita melihatnya dari sisi insinyur sinyal, yang selalu bertanya tiga hal: seperti apa model sinyalnya, seperti apa noise-nya, dan seberapa baik hasil terbaik yang mungkin dicapai.

18.1 Sinyal acak

Sinyal yang menarik hampir selalu bercampur noise, sehingga kita memperlakukannya sebagai proses acak. Kuliah dimulai dari dasar: variabel acak dengan fungsi distribusi kumulatif dan kerapatannya, vektor acak dengan kovarians, lalu proses acak sebagai deret variabel acak yang diindeks waktu. Sebuah proses disebut stasioner dalam arti luas kalau rata-ratanya tetap dan fungsi autokorelasinya,

$$r_x[k] = \mathbb{E}[x[n]x[n+k]], \quad (18.1)$$

hanya bergantung pada jarak waktu k , bukan pada waktu absolut. Transformasi Fourier dari autokorelasi adalah *power spectral density*, yang menyatakan bagaimana daya sinyal tersebar di seluruh frekuensi. Hubungan ini, teorema Wiener–Khinchin, menghubungkan dua cara melihat sinyal yang sama: di waktu lewat korelasi, di frekuensi lewat spektrum. Noise putih punya autokorelasi yang hanya tidak nol di $k = 0$, sehingga spektrumnya datar, seperti cahaya putih yang memuat semua warna.

Dalam praktik, spektrum diperkirakan dari data terbatas dengan *periodogram*, kuadrat magnitudo FFT dari potongan sinyal. Latihan radar memperlihatkan dua pelajaran praktisnya. Mengalikan sinyal dengan sebuah jendela sebelum FFT, misalnya jendela Hann, menekan kebocoran spektral yang membuat sinusoid tunggal tampak menyebar ke frekuensi tetangga. Menambahkan nol di ujung sinyal (*zero padding*) membuat spektrum tampak lebih halus karena titik-titiknya lebih rapat, tetapi tidak menambah resolusi yang sebenarnya, karena resolusi frekuensi ditentukan oleh lamanya pengamatan.

18.2 Estimasi klasik

Kuliah teori estimasi dimulai dengan contoh paling sederhana: menaksir level tegangan tetap A dari pengukuran berisik, $x[n] = A + w[n]$ untuk $n = 0, \dots, N-1$, dengan $w[n]$ noise putih Gauss bervarians σ^2 . Log-likelihoodnya

$$\ln p(\mathbf{x}; A) = -\frac{N}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{n=0}^{N-1} (x[n] - A)^2. \quad (18.2)$$

Turunan pertama terhadap A adalah $\frac{1}{\sigma^2} \sum_n (x[n] - A) = \frac{N}{\sigma^2} (\bar{x} - A)$, yang nol pada $\hat{A} = \bar{x}$, rata-rata sampel. Turunan kedua adalah $-N/\sigma^2$, tidak bergantung pada data, sehingga Fisher information-nya $I(A) = N/\sigma^2$ dan batas Cramér–Rao-nya σ^2/N (Bab 15). Rata-rata sampel tak bias dan variansnya tepat σ^2/N , jadi ia mencapai batas itu. Dalam bahasa Kay (1993), estimator ini efisien dan sekaligus *minimum variance unbiased*. Ada satu tanda khas di sini: turunan log-likelihood bisa ditulis sebagai $I(A)(\hat{A} - A)$. Kay menunjukkan bahwa estimator efisien ada tepat ketika bentuk ini bisa dicapai.

Untuk model yang linear dalam parameter, $\mathbf{x} = \mathbf{H}\boldsymbol{\theta} + \mathbf{w}$ dengan $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$, penurunan yang sama memberi estimator

$$\hat{\boldsymbol{\theta}} = (\mathbf{H}^\top \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{x}, \quad (18.3)$$

dengan kovarians $\sigma^2(\mathbf{H}^\top \mathbf{H})^{-1}$ yang juga mencapai batas Cramér–Rao. Estimator yang sama muncul dari kuadrat terkecil biasa: meminimalkan $\|\mathbf{x} - \mathbf{H}\boldsymbol{\theta}\|^2$, menurunkannya, dan menyamakannya dengan nol memberi persamaan normal $\mathbf{H}^\top \mathbf{H}\boldsymbol{\theta} = \mathbf{H}^\top \mathbf{x}$. Bedanya, kuadrat terkecil tidak memerlukan asumsi statistik apa pun. Kalau noise-nya tidak Gauss tetapi kita hanya mau estimator linear, *best linear unbiased estimator* (BLUE) memberi varians terkecil di antara semua estimator linear tak bias, dan untuk noise berkovarians $\mathbf{C}$ bentuknya menjadi $(\mathbf{H}^\top \mathbf{C}^{-1} \mathbf{H})^{-1} \mathbf{H}^\top \mathbf{C}^{-1} \mathbf{x}$: pengukuran yang berisik diberi bobot kecil.

Contoh dari kuliah yang menyenangkan adalah menaksir garis lurus. Untuk $x[n] = a + bn + w[n]$, matriks $\mathbf{H}$ punya kolom berisi satu dan kolom berisi n . Persamaan normal memberi rumus regresi linear yang dikenal di sekolah, dan batas Cramér–Rao untuk kemiringan b turun seperti $1/N^3$: mengamati lebih lama jauh lebih membantu untuk kemiringan daripada untuk titik potong, karena ujung-ujung garis yang jauh “mengunci” kemiringannya.

18.3 Estimasi Bayesian

Kalau kita punya pengetahuan awal tentang parameter, misalnya bahwa sebuah tegangan hampir pasti di sekitar lima volt, kita memperlakukan parameter sebagai variabel acak dengan prior. Estimator yang meminimalkan galat kuadrat rata-rata Bayesian, $\mathbb{E}[(A - \hat{A})^2]$ dengan ekspektasi atas data *dan* parameter, ternyata adalah rata-rata posterior $\mathbb{E}[A | \mathbf{x}]$, dengan alasan yang sama seperti rata-rata bersyarat di Bab 1. Estimator MAP mengambil puncak posterior.

Untuk level DC dengan prior $A \sim \mathcal{N}(\mu_A, \sigma_A^2)$, posteriornya Gauss dengan rata-rata

$$\hat{A}_{\text{MMSE}} = \frac{N\sigma_A^2}{N\sigma_A^2 + \sigma^2} \bar{x} + \frac{\sigma^2}{N\sigma_A^2 + \sigma^2} \mu_A, \quad (18.4)$$

rata-rata berbobot antara rata-rata sampel dan rata-rata prior. Misalkan prior kita $\mu_A = 5$ V dengan $\sigma_A = 0,5$ V, noise pengukuran $\sigma = 2$ V, dan kita punya $N = 4$ pengukuran dengan rata-rata $\bar{x} = 6$ V. Bobot data adalah $4 \cdot 0,25 / (4 \cdot 0,25 + 4) = 0,2$, sehingga $\hat{A} = 0,2 \cdot 6 + 0,8 \cdot 5 = 5,2$ V. Data yang sedikit dan berisik hanya menggeser prior sedikit. Dengan $N = 100$, bobot data naik menjadi sekitar 0,86, dan data mulai berkuasa.

Untuk model linear Gauss umum, MMSE, MAP, dan estimator linear MMSE (LMMSE) semuanya sama. Ketika distribusinya tidak Gauss, LMMSE tetap bisa dihitung hanya dari rata-rata dan kovarians, dan ia adalah estimator linear terbaik. Kalman filter (Bab 14 dan 19) adalah LMMSE yang diterapkan berulang dalam waktu.

18.4 Filter optimal

Misalkan kita ingin mendapatkan sinyal $d[n]$ dari pengamatan $x[n]$, dengan filter FIR berbobot w yang bekerja pada M sampel terakhir, $y[n] = w^\top x[n]$ dengan $x[n] = (x[n], x[n-1], \dots, x[n-M+1])^\top$. Filter Wiener memilih bobot yang meminimalkan $J(w) = \mathbb{E}[(d[n] - w^\top x[n])^2]$. Menguraikan kuadratnya,

$$J(w) = \sigma_d^2 - 2w^\top p + w^\top R w, \quad (18.5)$$

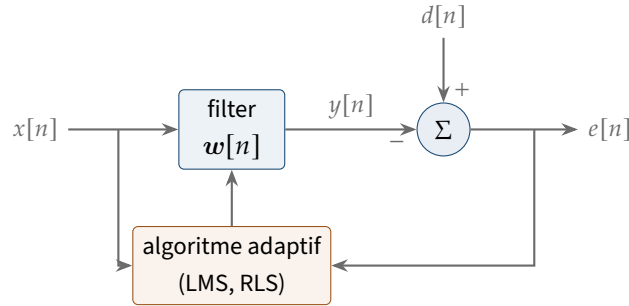
dengan $R = \mathbb{E}[xx^\top]$ matriks autokorelasi input dan $p = \mathbb{E}[x d]$ korelasi silang antara input dan sinyal yang diinginkan. Fungsi ini kuadratik dalam w , sebuah mangkuk seperti di Bab 4. Gradiennya $-2p + 2Rw$, dan menyamakannya dengan nol memberi persamaan Wiener-Hopf,

$$R w_{\text{opt}} = p, \quad J_{\min} = \sigma_d^2 - p^\top R^{-1} p. \quad (18.6)$$

Hasil ini punya tafsiran geometris, *orthogonality principle*: pada bobot optimal, galat $e[n] = d[n] - y[n]$ tidak berkorelasi dengan setiap sampel input, $\mathbb{E}[e[n]x[n]] = 0$. Tidak ada lagi informasi linear dalam data yang bisa dipakai untuk mengurangi galat, sama seperti residual kuadrat terkecil yang tegak lurus terhadap ruang kolom H .

Kuliah membahas beberapa susunan yang memakai struktur ini (Gambar 18.1). Pada identifikasi sistem, x diumpankan ke sistem yang tidak diketahui dan ke filter sekaligus, dan d adalah keluaran sistem, sehingga bobot optimal meniru respons sistem itu. Pada prediksi, $d[n] = x[n]$ dan filter hanya melihat sampel-sampel sebelumnya, sehingga ia belajar meramal sinyal satu langkah ke depan. Pada peredam bising, d berisi sinyal plus bising, x adalah rekaman bising saja dari mikrofon lain, dan galatnya menjadi sinyal yang sudah dibersihkan. Pada ekualisasi kanal, filter belajar membalik distorsi yang dibuat kanal komunikasi.

Contoh terkecil memperlihatkan isi persamaannya. Untuk filter satu tap, R dan p hanya bilangan, sehingga $w_{\text{opt}} = r_{xd}/r_{xx}$. Kalau $d = 0,8x + v$ dengan v noise yang tidak berkorelasi dengan x dan $\mathbb{E}[x^2] = 1$, maka $r_{xd} = 0,8$ dan $w_{\text{opt}} = 0,8$: filter menemukan



GAMBAR 18.1. Struktur filter adaptif. Filter mengeluarkan $y[n]$, galat $e[n] = d[n] - y[n]$ dan input dipakai algoritme adaptif untuk memperbarui bobot. Struktur yang sama dipakai untuk identifikasi sistem, peredam gempa, peredam bising, dan ekualisasi kanal.

penguatan sistem, dan noise v tertinggal di galat karena memang tidak bisa diprediksi dari x .

Filter Wiener membutuhkan $\mathbf{R}$ dan $\mathbf{p}$, yang dalam praktik harus diperkirakan dari data. Menggantinya dengan rata-rata sampel memberi filter kuadrat terkecil, optimal untuk data yang ada, bukan untuk ekspektasinya (Manolakis et al., 2005).

18.5 Filter adaptif

Ketika statistik sinyal berubah dari waktu ke waktu, misalnya gempa di ruang yang orangnya berpindah tempat, filter harus terus menyesuaikan diri. Kuliah menurunkan algoritme adaptif dalam dua langkah. Langkah pertama adalah *steepest descent* pada $J(\mathbf{w})$:

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu(\mathbf{p} - \mathbf{R}\mathbf{w}[n]), \quad (18.7)$$

dengan faktor dua diserap ke dalam μ . Algoritme ini konvergen ke solusi Wiener kalau $0 < \mu < 2/\lambda_{\max}$, dengan $\lambda_{\max}$ nilai eigen terbesar $\mathbf{R}$. Setiap mode, yaitu setiap arah vektor eigen, menyusut dengan faktor $(1 - \mu\lambda_i)$ per langkah, sehingga mode dengan nilai eigen terkecil adalah yang paling lambat. Rasio $\lambda_{\max}/\lambda_{\min}$, yang di kuliah disebut sebaran nilai eigen, memainkan peran yang persis sama dengan condition number di Bab 4. Sinyal input yang berwarna, misalnya suara dengan energi terkumpul di frekuensi rendah, punya sebaran nilai eigen besar dan membuat adaptasi lambat.

Langkah kedua mengganti ekspektasi yang tidak diketahui dengan nilai sesaat: $\mathbf{R} \approx \mathbf{x}[n]\mathbf{x}[n]^\top$ dan $\mathbf{p} \approx \mathbf{x}[n]d[n]$. Hasilnya adalah algoritme LMS,

$$\mathbf{w}[n+1] = \mathbf{w}[n] + \mu e[n] \mathbf{x}[n], \quad e[n] = d[n] - \mathbf{w}[n]^\top \mathbf{x}[n]. \quad (18.8)$$

Ini SGD dengan mini-batch berukuran satu, dirumuskan Widrow dan Hoff sekitar 1960. Biayanya hanya sekitar 2M perkalian per sampel. Harga yang dibayar adalah noise gradien: bahkan setelah konvergen, bobot terus bergoyang di sekitar solusi Wiener, sehingga galat rata-ratanya sedikit di atas $J_{\min}$. Kelebihan galat ini disebut *misadjustment*, dan kira-kira sebanding dengan μ dikali jumlah daya input. Di sini muncul trade-off yang sama dengan laju belajar SGD: langkah besar cepat menyesuaikan

diri tetapi bergoyang, langkah kecil tenang tetapi lambat. NLMS membagi langkah dengan daya input sesaat, $\mu/(\epsilon + \|x[n]\|^2)$, sehingga perilakunya tidak bergantung pada keras-lemahnya sinyal.

Algoritme RLS memecahkan masalah sebaran nilai eigen dengan cara lain. Ia meminimalkan jumlah kuadrat galat semua sampel masa lalu yang dibobot eksponensial, $\sum_{i \leq n} \lambda^{n-i} e[i]^2$, dan memperbarui invers matriks autokorelasinya secara rekursif dengan rumus Woodbury, sehingga setiap langkah mirip langkah Newton. Faktor lupa λ sedikit di bawah satu menentukan panjang ingatan efektif, kira-kira $1/(1 - \lambda)$ sampel: $\lambda = 0,99$ berarti filter terutama mengingat seratus sampel terakhir. RLS konvergen jauh lebih cepat dan tidak terpengaruh warna input, dengan biaya sekitar M^2 operasi per sampel.

Bayangkan headphone peredam bising. Mikrofon luar menangkap suara mesin pesawat, dan filter adaptif belajar memprediksi seberapa banyak suara itu bocor ke telinga. Speaker lalu memutar prediksi dengan tanda terbalik. Setiap kali pesawat berbelok atau penumpang menggeser kepala, galat sisa berubah, dan bobot filter ikut bergeser beberapa ribu kali per detik.

18.6 Radar dalam desibel

Radar memancarkan gelombang elektromagnetik dan mendengarkan pantulannya. Latihan pertama menurunkan persamaan radar langkah demi langkah. Antena dengan daya pancar P_T dan penguatan G_T menghasilkan kerapatan daya $S_T = P_T G_T / (4\pi R^2)$ di target. Target memantulkan daya sebesar $S_T \sigma$, dengan σ *radar cross section*, luas efektif yang “dilihat” radar. Pantulan itu kembali menyebar seperti bola, sehingga kerapatannya di penerima $S_T \sigma / (4\pi R^2)$, dan antena penerima dengan luas efektif $A_{\text{eff}} = G_R \lambda^2 / (4\pi)$ menangkap sebagian. Mengalikan semuanya memberi

$$P_R = \frac{P_T G_T G_R \lambda^2 \sigma}{(4\pi)^3 R^4}. \quad (18.9)$$

Pangkat empat pada jarak datang dari dua perjalanan, pergi dan pulang, masing-masing melemah sebanding R^2 . Akibatnya, untuk melipatgandakan jangkauan, daya pancar harus dinaikkan enam belas kali.

Angka-angka radar membentang belasan orde besaran, sehingga insinyur bekerja dalam desibel, di mana perkalian menjadi penjumlahan. Ambil radar mobil 77 GHz, dengan panjang gelombang $\lambda = c/f \approx 3,9$ mm, daya pancar 10 dBm, penguatan antena pancar dan terima masing-masing 20 dBi, mobil di depan dengan RCS sekitar 10 m^2 (10 dBsm), pada jarak 50 m. Dalam desibel,

$$P_R = 10 + 20 + 20 + 20 \log_{10} 0,0039 + 10 - 30 \log_{10} 4\pi - 40 \log_{10} 50 \approx -89 \text{ dBm}, \quad (18.10)$$

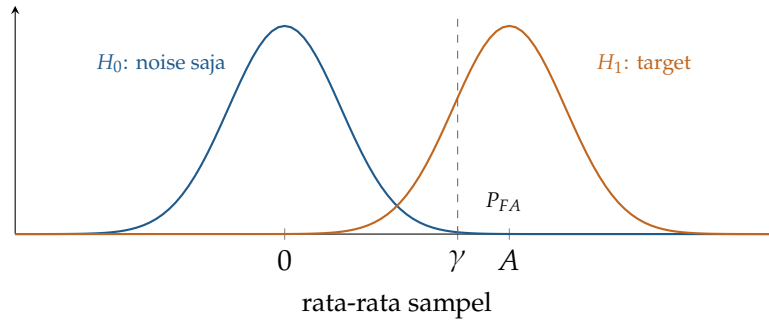
dengan suku-suku $-48,2$, $-33,0$, dan $-68,0$ dB untuk panjang gelombang, faktor 4π , dan jarak. Daya yang kembali sekitar satu pikowatt. Apakah sinyal sekecil itu terdeteksi bergantung pada noise penerima. Noise termal sebanding dengan $k_B T$ dikali lebar pita, dan penerima nyata menambahkan noise-nya sendiri, yang dinyatakan dengan *noise figure*. Penerima dengan noise figure 12 dB membuat noise-nya hampir enam belas kali lebih besar dari noise termal ideal.

18.7 Mendeteksi di tengah noise

Pertanyaan deteksi adalah uji hipotesis: apakah yang kita lihat hanya noise (H_0) atau noise ditambah target (H_1)? Latihan memulainya dengan masalah paling sederhana, mendeteksi level DC A dalam noise Gauss. Kriteria Neyman–Pearson memaksimalkan peluang deteksi P_D untuk peluang alarm palsu P_{FA} tertentu, dan untuk masalah ini ujinya adalah membandingkan rata-rata sampel dengan ambang γ . Di bawah H_0 , rata-rata itu berdistribusi $\mathcal{N}(0, \sigma^2/N)$, sehingga

$$P_{FA} = Q\left(\frac{\gamma}{\sigma/\sqrt{N}}\right), \quad P_D = Q\left(\frac{\gamma - A}{\sigma/\sqrt{N}}\right), \quad (18.11)$$

dengan Q peluang ekor kanan distribusi normal standar. Dengan $\sigma = 1$, $N = 16$, dan $P_{FA} = 10^{-3}$, kita butuh $\gamma/0,25 = 3,09$, jadi $\gamma \approx 0,77$. Untuk $A = 1$, $P_D = Q(-0,92) \approx 0,82$. Menggandakan jumlah sampel menaikkan P_D di atas 0,95. Gambar 18.2 memperlihatkan trade-off-nya: menggeser ambang ke kanan mengurangi alarm palsu sekaligus mengurangi deteksi.



GAMBAR 18.2. Deteksi level DC dalam noise. Daerah biru di kanan ambang adalah peluang alarm palsu, luas kurva oranye di kanan ambang adalah peluang deteksi. Parameter seperti pada contoh teks ($\sigma/\sqrt{N} = 0,25$).

Radar biasanya bekerja dengan sinyal kompleks, komponen sefase dan kuadratur, dan mendeteksi magnitudonya. Latihan memperlihatkan distribusi yang muncul. Kalau komponen sefase dan kuadratur adalah noise Gauss independen dengan varians σ^2 , magnitudonya berdistribusi Rayleigh. Kalau ada sinyal konstan ditambah noise, magnitudonya berdistribusi Rice. Untuk Rayleigh, peluang magnitudo melewati ambang T adalah $\exp(-T^2/2\sigma^2)$, sehingga ambang untuk $P_{FA} = 10^{-4}$ adalah $T = \sigma\sqrt{-2\ln 10^{-4}} \approx 4,3\sigma$.

18.8 Korelasi dan matched filter

Untuk menemukan pantulan lemah di tengah noise, sinyal yang diterima dikorelasikan dengan sinyal yang dipancarkan. Mengapa korelasi? Misalkan sinyal yang diterima $s + w$ dengan noise putih bervarians σ^2 , dan kita melewatkannya ke filter linear $\mathbf{h}$. Rasio sinyal terhadap noise di keluaran adalah

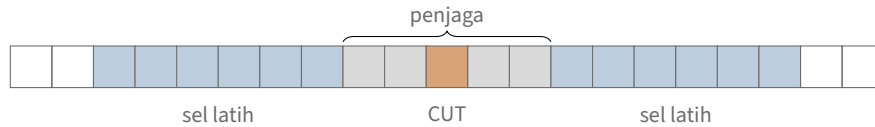
$$\text{SNR} = \frac{|\mathbf{h}^\top \mathbf{s}|^2}{\sigma^2 \|\mathbf{h}\|^2} \leq \frac{\|\mathbf{s}\|^2}{\sigma^2}, \quad (18.12)$$

dengan ketaksamaan Cauchy–Schwarz di langkah terakhir, yang menjadi persamaan tepat ketika h sebanding dengan s . Filter terbaik adalah sinyal itu sendiri, dibalik dalam waktu, sehingga filtering menjadi korelasi. Itulah *matched filter* (Richards, 2014). SNR terbaiknya hanya bergantung pada energi sinyal, bukan pada bentuknya.

Bentuk sinyal tetap penting untuk ketajaman puncak. Pulsa persegi panjang punya autokorelasi berbentuk segitiga yang lebar. Latihan membandingkannya dengan kode Barker sepanjang sebelas, $(+, +, +, -, -, -, +, -, -, +, -)$, yang autokorelasinya bernilai 11 di pusat dan tidak lebih dari 1 di tempat lain. Energinya sama dengan pulsa persegi panjang sepanjang sebelas, tetapi puncaknya setajam pulsa sepanjang satu. Teknik ini, *pulse compression*, memberi jangkauan dari pulsa panjang dan resolusi dari pulsa pendek.

18.9 CFAR

Pada sinyal nyata, tingkat noise dan clutter berubah dari satu tempat ke tempat lain, sehingga ambang tetap gagal: di daerah tenang ia terlalu tinggi, di daerah berisik ia memicu banyak alarm palsu. *Constant false alarm rate* (CFAR) menghitung ambang secara lokal (Gambar 18.3). Setiap sel bergiliran menjadi *cell under test*. Tingkat noise diperkirakan dari sel-sel latihan di kiri dan kanannya, dengan beberapa sel penjaga di antaranya supaya energi target sendiri, yang menyebar ke sel tetangga, tidak ikut dihitung.



GAMBAR 18.3. Jendela CFAR satu dimensi. Sel yang diuji (oranye) dibandingkan dengan ambang yang dihitung dari sel-sel latihan (biru). Sel penjaga (abu-abu) tidak dipakai agar energi target tidak ikut memperbesar perkiraan noise.

Untuk detektor daya, noise berdistribusi eksponensial, dan varian *cell-averaging* memakai rata-rata N sel latihan $\hat{P}$ lalu menyatakan deteksi kalau sel yang diuji melebihi $\alpha\hat{P}$. Faktor skala untuk peluang alarm palsu tertentu bisa diturunkan secara tertutup: $P_{FA} = (1 + \alpha/N)^{-N}$, sehingga

$$\alpha = N(P_{FA}^{-1/N} - 1). \quad (18.13)$$

Dengan $N = 16$ dan $P_{FA} = 10^{-4}$, $10^{4/16} \approx 1,78$ dan $\alpha \approx 12,5$, atau sekitar 11 dB di atas noise. Hasil kuncinya adalah bahwa α tidak bergantung pada tingkat noise yang sebenarnya, sehingga peluang alarm palsu tetap konstan meski noise naik turun. Itulah asal nama CFAR.

Varian lain menangani kasus sulit. CAGO mengambil yang lebih besar dari rata-rata kiri dan kanan, sehingga tidak memicu alarm palsu di tepi clutter. CASO mengambil yang lebih kecil, sehingga tidak tertutupi target lain di dekatnya. *Order-statistic* CFAR memakai nilai berperingkat tertentu alih-alih rata-rata, dan tetap bekerja ketika beberapa target berdekatan (Rohling, 1983). Latihan juga memperlihatkan sisi rekayasa: loop

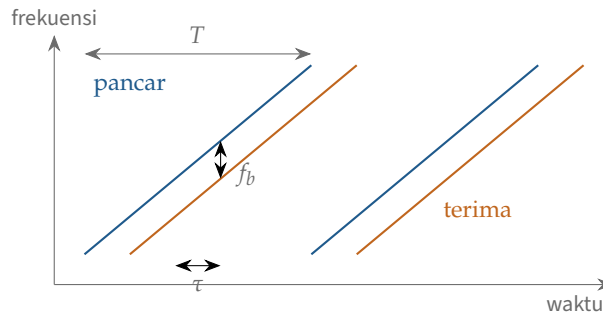
CFAR dalam Python murni lambat sekali, dan kompilasi just-in-time dengan Numba mempercepatnya ratusan kali.

18.10 Radar FMCW

Radar mobil memakai gelombang kontinu yang frekuensinya naik secara linear, sebuah *chirp* (Gambar 18.4). Pantulan dari target pada jarak R tiba dengan tunda $\tau = 2R/c$, sehingga pada setiap saat frekuensinya sedikit lebih rendah dari sinyal yang sedang dipancarkan, dengan selisih kemiringan dikali tunda. Mencampur keduanya menghasilkan sinyal *intermediate frequency* dengan frekuensi beat

$$f_b = k\tau = \frac{2kR}{c}, \quad k = \frac{B}{T}, \quad (18.14)$$

dengan B lebar pita dan T durasi chirp. FFT sinyal IF langsung memberi profil jarak. Dua target hanya terpisah kalau beat-nya berbeda lebih dari satu bin FFT, $1/T$, sehingga resolusi jaraknya $\Delta R = c/(2B)$, lima belas sentimeter untuk satu gigahertz. Dengan $B = 1$ GHz dan $T = 50 \mu\text{s}$, target pada sepuluh meter menghasilkan beat sekitar 1,33 MHz.



GAMBAR 18.4. Radar FMCW. Chirp yang diterima (oranye) adalah chirp yang dipancarkan (biru) yang tertunda $\tau = 2R/c$. Selisih frekuensi keduanya, f_b , sebanding dengan jarak.

Kecepatan diperoleh dari deretan chirp. Target yang bergerak dengan kecepatan radial v mengubah jaraknya sebesar vT dari satu chirp ke chirp berikutnya, dan fase sinyal IF bergeser sebesar $\Delta\phi = 4\pi vT/\lambda$. FFT kedua di sepanjang chirp mengubah geseran fase itu menjadi kecepatan, dan hasil kedua FFT bersama-sama membentuk peta jarak dan kecepatan (*range-Doppler map*). Kecepatan terbesar yang tidak ambigu tercapai ketika $\Delta\phi = \pi$, yaitu $v_{\max} = \lambda/(4T)$, dan resolusi kecepatannya $\lambda/(2N_c T)$ untuk N_c chirp. Pada 77 GHz dengan $T = 50 \mu\text{s}$ dan 128 chirp, radar ini melihat kecepatan sampai sekitar 19,5 m/s dengan resolusi sekitar 0,3 m/s.

18.11 Beamforming digital

Dengan beberapa antenna penerima yang tersusun lurus berjarak d , gelombang datar dari arah θ mencapai antenna-antenna itu dengan selisih jalan $d \sin \theta$, sehingga fasenya bergeser $\Delta\phi = 2\pi d \sin \theta/\lambda$ dari satu antenna ke antenna berikutnya. Dengan $d = \lambda/2$, geserannya

$\pi \sin \theta$, dan seluruh rentang -90° sampai 90° terpetakan tanpa ambiguitas. FFT ketiga, kali ini di sepanjang antena, mengubah geseran fase menjadi sudut. Resolusi sudutnya kira-kira $2/N_A$ radian di arah depan untuk N_A antena: delapan antena memberi sekitar 14° .

Resolusi itu kasar, dan latihan memperkenalkan metode subruang untuk mempertajamnya. Algoritme MUSIC menghitung matriks kovarians sinyal antar antena, memisahkan vektor eigennya menjadi subruang sinyal (nilai eigen besar) dan subruang noise (sisanya), lalu memanfaatkan fakta bahwa vektor arah target harus tegak lurus terhadap subruang noise. Pseudo-spektrumnya, $1/\|E_n^H \mathbf{a}(\theta)\|^2$, memuncak tajam di arah target, jauh lebih tajam dari FFT. Harganya, kita harus tahu atau menebak jumlah target, dan metode ini peka terhadap kalibrasi antena. Latihan berakhir dengan simulator yang menggabungkan semua langkah ini: chirp dipancarkan ke beberapa target, noise ditambahkan, lalu seluruh rantai FFT, CFAR, dan estimasi sudut dijalankan untuk menemukannya kembali.

18.12 Di mana machine learning masuk

Batas antara pemrosesan sinyal dan machine learning semakin kabur. Peta jarak dan kecepatan radar diperlakukan sebagai gambar dan dimasukkan ke jaringan konvolusi untuk mengenali gerakan tangan atau membedakan pejalan kaki dari sepeda. Detektor CFAR klasik bisa diganti detektor yang dipelajari dari data, dengan risiko bahwa jaminan alarm palsu hilang. Algoritme iteratif, misalnya untuk rekonstruksi sinyal, bisa “dibuka” (*unrolled*): setiap iterasi menjadi satu lapisan jaringan yang parameternya dipelajari, sehingga struktur algoritme klasik tetap terjaga. Dan di arah sebaliknya, gagasan pemrosesan sinyal, seperti filter, frekuensi, dan wavelet, memberi bahasa untuk memahami apa yang dilakukan jaringan saraf (Bab 13). Fondasinya tetap sama: model, noise, dan batas atas apa yang bisa diestimasi dari data.

18.13 Perkembangan riset

Pemrosesan sinyal klasik dan deep learning kini sering digabung dengan cara yang menjaga struktur algoritme lama. *Algorithm unrolling* (Monga et al., 2021) mengambil algoritme iteratif yang sudah dikenal, misalnya iterasi untuk pemulihan sinyal jarang atau rekonstruksi citra, lalu “membuka” sejumlah iterasinya menjadi lapisan-lapisan jaringan. Parameter yang semula dipilih tangan, seperti ukuran langkah dan ambang, menjadi bobot yang dipelajari dari data. Jaringan yang dihasilkan jauh lebih kecil dari jaringan generik, lebih mudah ditafsirkan karena setiap lapisan punya arti, dan butuh lebih sedikit data latih.

KalmanNet (Revach et al., 2021) menerapkan pola yang sama pada Kalman filter. Persamaan prediksi dan koreksi dari Bab 14 tetap dipakai, tetapi Kalman gain tidak dihitung dari matriks kovarians noise yang harus diketahui, melainkan dikeluarkan oleh sebuah jaringan rekuren kecil yang membaca selisih pengukuran dan prediksi. Ketika model keadaan tidak linear atau tidak diketahui persis, KalmanNet dilaporkan mengungguli filter klasik, sambil tetap mempertahankan struktur yang bisa ditafsirkan.

Inilah model hibrida dari Bab 32, dalam bahasa pemrosesan sinyal: fisika dan struktur dari model, sisanya dari data.

CATATAN SUMBER. Bab ini mengikuti kuliah dan latihan Optimum and Adaptive Signal Processing Systems (variabel dan proses acak, estimasi klasik dan Bayesian, filter Wiener dan kuadrat terkecil, steepest descent, LMS, NLMS, RLS) serta latihan Radar System Engineering (persamaan radar dalam desibel, distribusi noise dan deteksi, korelasi dan matched filter, FFT, CFAR, FMCW, beamforming digital dan MUSIC, simulator). Bacaan yang disebut kuliah: Kay (1993) dan Manolakis et al. (2005). Untuk radar, Richards (2014).

19 Mengendalikan Sistem

Di semester pertama saya mengambil dua mata kuliah yang sekilas tidak ada hubungannya dengan machine learning. Control systems membahas sistem dinamis linear dari model ruang keadaan sampai pengendali diskret, dengan latihan dan ujian berbasis MATLAB. Production automation systems membahas dunia pabrik: perangkat otomasi, sistem waktu nyata, pemrograman PLC, fieldbus, dan standar IEC 61499 untuk sistem otomasi terdistribusi. Belakangan terlihat bahwa teori kendali adalah saudara kandung reinforcement learning, dan bahwa model ruang keadaan adalah bahasa yang sama yang dipakai Kalman filter, state space model seperti Mamba, dan penemuan persamaan dari data.

19.1 Sistem dan model

Sebuah sistem dinamis dijelaskan oleh keadaannya, sekumpulan besaran yang cukup untuk menentukan masa depannya kalau input diketahui. Kuliah memakai tiga contoh yang terus kembali.

Contoh pertama adalah rangkaian RC. Arus yang mengisi kapasitor sebanding dengan selisih tegangan sumber u dan tegangan kapasitor v , dibagi hambatan, sehingga $C\dot{v} = (u - v)/R$, atau

$$\dot{v} = -\frac{1}{\tau}v + \frac{1}{\tau}u, \quad \tau = RC. \quad (19.1)$$

Untuk input langkah setinggi U mulai dari $v(0) = 0$, solusinya $v(t) = U(1 - e^{-t/\tau})$. Setelah satu konstanta waktu, kapasitor terisi $1 - e^{-1} \approx 63$ persen. Setelah lima konstanta waktu, lebih dari 99 persen. Dengan $R = 10 \text{ k}\Omega$ dan $C = 100 \text{ }\mu\text{F}$, $\tau = 1$ detik. Hampir setiap sistem yang “menyusul” targetnya secara perlahan, dari termometer sampai suhu ruangan, punya perilaku orde satu seperti ini.

Contoh kedua adalah kaskade tangki air. Untuk satu tangki dengan luas penampang A , ketinggian air h berubah menurut neraca massa: debit masuk dikurangi debit keluar. Debit keluar lewat lubang di dasar mengikuti hukum Torricelli, sebanding dengan $\sqrt{h}$, sehingga

$$A\dot{h} = q_{\text{in}} - k\sqrt{h}. \quad (19.2)$$

Model ini nonlinear. Di sekitar titik kesetimbangan h_0 , tempat $q_{\text{in}} = k\sqrt{h_0}$, kita bisa melinearkannya dengan mengganti $\sqrt{h}$ dengan garis singgungnya, $\sqrt{h} \approx \sqrt{h_0} + (h - h_0)/(2\sqrt{h_0})$. Penyimpangan kecil $\Delta h = h - h_0$ lalu memenuhi $A\dot{\Delta h} = \Delta q_{\text{in}} - \frac{k}{2\sqrt{h_0}}\Delta h$, sistem orde satu dengan konstanta waktu $2A\sqrt{h_0}/k$. Tangki yang lebih penuh bereaksi lebih lambat, karena perubahan kecil ketinggian hanya sedikit mengubah debit keluar.

Contoh ketiga adalah persamaan pemangsa dan mangsa Lotka–Volterra, $\dot{x} = ax - bxy$ dan $\dot{y} = -cy + dxy$, dengan x mangsa dan y pemangsa. Kesetimbangan yang menarik

berada di $(c/d, a/b)$. Matriks Jacobian di titik itu adalah

$$J = \begin{pmatrix} 0 & -bc/d \\ ad/b & 0 \end{pmatrix}, \quad (19.3)$$

dengan nilai eigen $\pm i\sqrt{ac}$, murni imajiner. Linearisasinya berosilasi tanpa meluruh dan tanpa tumbuh, sesuai dengan siklus naik turun yang khas pada populasi: mangsa yang berlimpah membuat pemangsa bertambah, pemangsa yang banyak membuat mangsa berkurang, dan seterusnya, dengan periode sekitar $2\pi/\sqrt{ac}$.

Bentuk umumnya adalah $\dot{x} = f(x, u)$ dan $y = h(x, u)$, dengan u input yang bisa kita atur dan y keluaran yang bisa kita ukur. Sebagian besar teori dibangun untuk sistem linear yang tidak berubah terhadap waktu,

$$\dot{x} = Ax + Bu, \quad y = Cx + Du, \quad (19.4)$$

dan sistem nonlinear dilinearkan di sekitar kesetimbangannya dengan matriks Jacobian, persis seperti pada tangki dan populasi tadi.

19.2 Solusi, mode, dan kestabilan

Solusi sistem linear adalah

$$x(t) = e^{At}x_0 + \int_0^t e^{A(t-\tau)}Bu(\tau)d\tau, \quad (19.5)$$

respons terhadap keadaan awal ditambah respons terhadap input. Prinsip superposisi berlaku: respons terhadap jumlah input adalah jumlah responsnya. Kalau A bisa didiagonalkan, $A = V\Lambda V^{-1}$, maka $e^{At} = Ve^{\Lambda t}V^{-1}$, dan sistem terurai menjadi mode-mode independen yang masing-masing berkembang seperti $e^{\lambda_i t}$. Nilai eigen real negatif berarti peluruhan, nilai eigen kompleks berarti osilasi dengan frekuensi sebesar bagian imajiner. Kalau A tidak bisa didiagonalkan, bentuk Jordan memberi mode yang dikalikan polinomial dalam t , seperti $te^{\lambda t}$.

Sistem stabil secara asimtotik, artinya kembali ke kesetimbangan dari gangguan apa pun, tepat ketika semua nilai eigen A punya bagian real negatif. Matriks seperti ini disebut Hurwitz. Untuk sistem nonlinear, kestabilan linearisasinya menentukan kestabilan lokal, kecuali ketika ada nilai eigen di sumbu imajiner, seperti pada Lotka-Volterra, di mana linearisasi tidak cukup untuk memutuskan.

19.3 Bisa dicapai, bisa diamati

Dua pertanyaan menentukan apakah sebuah sistem bisa dikendalikan dengan baik. Pertama, apakah input bisa membawa keadaan ke mana saja? Untuk sistem linear, jawabannya ya tepat ketika matriks keterjangkauan

$$\mathcal{R} = [B \ AB \ A^2B \ \dots \ A^{n-1}B] \quad (19.6)$$

berpangkat penuh. Kedua, apakah keadaan bisa direkonstruksi dari keluaran? Jawabannya ya tepat ketika matriks keteramatan, yang disusun dari baris-baris $C, CA, \dots, CA^{n-1}$, berpangkat penuh.

Contoh paling sederhana memperlihatkan maknanya. Untuk massa tanpa gesekan yang didorong gaya, $\ddot{x} = u$, keadaannya posisi dan kecepatan, dengan $A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ dan $B = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. Matriks $[B \ AB] = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ berpangkat dua, jadi sistemnya bisa dicapai: gaya saja cukup untuk membawa massa ke posisi dan kecepatan mana pun. Kalau yang diukur posisi, $C = (1 \ 0)$, matriks keteramatannya identitas dan sistemnya teramati: kecepatan bisa disimpulkan dari perubahan posisi. Kalau yang diukur hanya kecepatan, $C = (0 \ 1)$, matriksnya $\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ berpangkat satu. Kita tidak akan pernah tahu posisi awal, karena menggeser seluruh lintasan tidak mengubah kecepatan sama sekali.

Dekomposisi Kalman memisahkan keadaan menjadi bagian yang bisa dicapai dan diamati, bagian yang hanya salah satunya, dan bagian yang tidak keduanya. Bagian yang tidak terjangkau dan tidak stabil adalah kabar buruk, karena tidak ada pengendali yang bisa menyelamatkannya. Kalau bagian itu stabil, sistemnya disebut *stabilizable*, dan untuk keperluan praktis itu sudah cukup.

19.4 Pengendali dan pengamat

Dengan umpan balik keadaan $u = -Kx$, sistem tertutup menjadi $\dot{x} = (A - BK)x$, dan kita bisa memilih K agar nilai eigen sistem tertutup berada di tempat yang kita inginkan, selama sistemnya bisa dicapai. Untuk massa tadi dengan $u = -k_1x - k_2\dot{x}$, persamaan karakteristiknya $s^2 + k_2s + k_1 = 0$. Kalau kita menginginkan kedua kutub di -2 , polinomialnya harus $(s + 2)^2 = s^2 + 4s + 4$, sehingga $k_1 = k_2 = 4$. Rumus Ackermann melakukan perhitungan yang sama untuk sistem satu input berukuran berapa pun: $K = (0 \ \cdots \ 0 \ 1) \mathcal{R}^{-1} \varphi(A)$, dengan φ polinomial karakteristik yang diinginkan.

Kalau keadaan tidak diukur seluruhnya, kita membangun pengamat yang menirukan sistem dan mengoreksi dirinya dengan galat keluaran,

$$\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}). \quad (19.7)$$

Mengurangkan persamaan ini dari persamaan sistem memberi dinamika galat perkiraan, $e = x - \hat{x}$: $\dot{e} = (A - LC)e$. Input u hilang dari persamaan galat, sehingga galat perkiraan meluruh dengan laju yang kita pilih lewat L , terlepas dari apa yang dilakukan pengendali. Inilah inti prinsip pemisahan: pengendali dan pengamat bisa dirancang terpisah lalu digabung, dan kutub-kutub sistem gabungan adalah gabungan kutub keduanya. Biasanya pengamat dibuat beberapa kali lebih cepat dari pengendali, supaya perkiraannya sudah benar ketika pengendali membutuhkannya.

19.5 Pengendali optimal

Alih-alih memilih kutub sendiri, kita bisa meminta pengendali terbaik menurut sebuah biaya. Linear quadratic regulator meminimalkan

$$J = \int_0^\infty (x^\top Qx + u^\top Ru) dt, \quad (19.8)$$

yang menimbang seberapa jauh keadaan menyimpang terhadap seberapa besar tenaga kendali yang dipakai. Jawabannya tetap berupa umpan balik linear, $K = R^{-1}B^T P$, dengan P solusi persamaan aljabar Riccati $A^T P + PA - PBR^{-1}B^T P + Q = 0$.

Kasus skalar memperlihatkan isinya. Untuk integrator $\dot{x} = u$ dengan biaya $\int (qx^2 + ru^2) dt$, persamaan Riccati menyusut menjadi $-p^2/r + q = 0$, sehingga $p = \sqrt{qr}$ dan $K = p/r = \sqrt{q/r}$. Kalau galat posisi kita anggap seratus kali lebih mahal dari tenaga, $q/r = 100$, pengendalinya $u = -10x$ dan sistem tertutupnya meluruh dengan konstanta waktu satu per sepuluh detik. Kalau tenaga yang mahal, pengendalinya lebih lembut dan lebih lambat. LQR mengubah pertanyaan “kutubnya di mana?” menjadi pertanyaan yang lebih bermakna bagi insinyur: “apa yang lebih penting bagi saya?”.

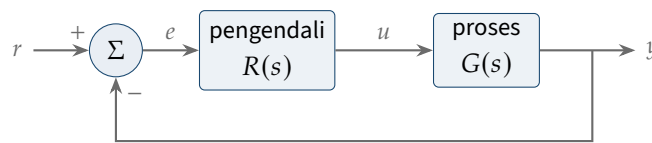
Ketika pengukurannya berisik, pengamat optimalnya adalah Kalman filter (Kalman, 1960), dan gabungan LQR dengan Kalman filter disebut LQG. Dari sudut pandang Bab 16, LQR adalah masalah reinforcement learning yang modelnya diketahui sempurna, dinamikanya linear, dan biayanya kuadratik, sehingga persamaan Bellman bisa diselesaikan secara tertutup. Fungsi nilainya tidak lain adalah $x^T P x$.

19.6 Domain frekuensi

Transformasi Laplace mengubah persamaan diferensial linear menjadi persamaan aljabar, karena turunan menjadi perkalian dengan s . Hubungan antara input dan keluaran menjadi fungsi alih,

$$G(s) = C(sI - A)^{-1}B + D. \quad (19.9)$$

Untuk rangkaian RC tadi, $G(s) = 1/(\tau s + 1)$. Sistem stabil dalam arti BIBO, yaitu input terbatas selalu menghasilkan keluaran terbatas, kalau semua kutub $G(s)$ berada di setengah bidang kiri. Respons frekuensi $G(i\omega)$ menyatakan bagaimana sinyal sinus diperkuat dan digeser fasenya. Untuk sistem orde satu, $|G(i\omega)| = 1/\sqrt{1 + \omega^2\tau^2}$, yang pada $\omega = 1/\tau$ bernilai $1/\sqrt{2}$, sekitar -3 dB: frekuensi itulah batas antara yang diloloskan dan yang diredam. Diagram Bode menggambar magnitudo dan fase dalam skala logaritmik, sehingga sistem yang tersusun berurutan cukup dijumlahkan grafiknya.



GAMBAR 19.1. Lingkaran kendali umpan balik dengan satu derajat kebebasan. Galat $e = r - y$ antara acuan dan keluaran diumpankan ke pengendali, yang menghasilkan input proses.

Dalam lingkaran umpan balik (Gambar 19.1), fungsi alih dari acuan ke keluaran adalah $T(s) = L(s)/(1 + L(s))$ dengan $L = RG$ fungsi lingkaran terbuka, dan dari acuan ke galat $S(s) = 1/(1 + L(s))$. Kedua fungsi ini berjumlah satu, sebuah batasan mendasar: kita tidak bisa sekaligus mengikuti acuan dengan sempurna dan mengabaikan noise pengukuran di frekuensi yang sama.

Contoh dari kuliah memperlihatkan mengapa pengendali PI begitu populer. Ambil proses orde satu $G = K/(\tau s + 1)$ dan pengendali proporsional $R = K_p$. Untuk acuan langkah, teorema nilai akhir memberi galat keadaan tunak

$$e_{\infty} = \lim_{s \rightarrow 0} s \cdot \frac{1}{1 + K_p G(s)} \cdot \frac{1}{s} = \frac{1}{1 + K_p K}. \quad (19.10)$$

Dengan $K = 1$ dan $K_p = 9$, galatnya sepuluh persen, berapa lama pun kita menunggu. Menaikkan K_p mengecilkan galat tetapi membuat sistem semakin agresif. Menambahkan suku integral, $R = K_p + K_i/s$, menempatkan s di penyebut L , sehingga $L(0)$ tak hingga dan galat keadaan tunaknya nol. Secara intuitif, selama masih ada galat sekecil apa pun, integralnya terus tumbuh dan mendorong input, sampai galatnya benar-benar hilang. Kestabilan lingkaran tertutup kemudian diperiksa dengan kriteria grafis seperti kriteria Nyquist, yang membaca kestabilan dari kurva $L(i\omega)$, dan margin fase menyatakan seberapa jauh sistem dari batas ketidakstabilan.

19.7 Merancang lewat respons frekuensi

19.7.1 Sistem orde dua dan bentuk respons tangga

Banyak lingkaran kendali, setelah ditutup, berperilaku kira-kira seperti sistem orde dua,

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}, \quad (19.11)$$

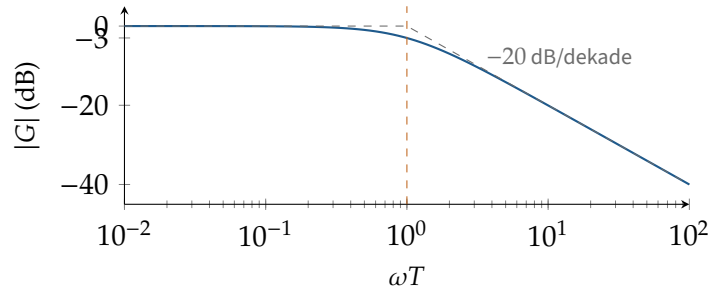
dengan frekuensi alami ω_n dan rasio redaman ζ . Kutubnya $s = -\zeta\omega_n \pm j\omega_n\sqrt{1 - \zeta^2}$ untuk $\zeta < 1$: bagian real menentukan seberapa cepat osilasi meluruh, bagian imajiner menentukan frekuensinya. Respons terhadap masukan tangga melampaui nilai akhirnya dengan *overshoot*

$$M_p = \exp\left(-\frac{\pi\zeta}{\sqrt{1 - \zeta^2}}\right). \quad (19.12)$$

Dengan $\zeta = 0,707$, overshoot sekitar 4,3 persen. Dengan $\zeta = 0,5$, sekitar 16 persen. Dengan $\zeta = 0,2$, lebih dari 50 persen, dan sistem berayun lama sebelum tenang. Rasio redaman sekitar 0,7 sering dipilih sebagai kompromi antara cepat dan tidak berayun. Bayangkan peredam kejut mobil: terlalu lunak, mobil berayun setelah melewati polisi tidur; terlalu keras, guncangan diteruskan langsung ke penumpang.

19.7.2 Diagram Bode

Untuk masukan sinus, sistem LTI yang stabil menghasilkan sinus dengan frekuensi yang sama, diperbesar $|G(j\omega)|$ kali dan digeser fasenya sebesar $\arg G(j\omega)$. Diagram Bode menggambar keduanya terhadap frekuensi pada skala logaritmik, dengan magnitudo dalam desibel, $20 \log_{10} |G|$. Skala logaritmik punya keuntungan besar: untuk sistem yang tersusun seri, magnitudo dalam desibel dan fase cukup dijumlahkan. Sistem orde satu $1/(1 + sT)$ datar di frekuensi rendah dan turun 20 dB per dekade di atas frekuensi sudut $1/T$, dengan fase bergerak dari 0° ke -90° dan bernilai -45° tepat di frekuensi sudut (Gambar 19.2). Sebuah integrator $1/s$ turun 20 dB per dekade di semua frekuensi dengan fase tetap -90° .



GAMBAR 19.2. Magnitudo sistem orde satu $1/(1 + sT)$ pada diagram Bode, beserta asimtotnya. Di frekuensi sudut $\omega = 1/T$, kurva sebenarnya berada 3 dB di bawah titik temu asimtot.

19.7.3 Contoh: merancang pengendali PI

Naskah kuliah memberi resep praktis yang menghubungkan respons tangga lingkaran tertutup dengan dua angka pada lingkaran terbuka $L(s) = R(s)G(s)$: frekuensi potong ω_C , tempat $|L(j\omega_C)| = 1$, dan margin fase $\phi = \arg L(j\omega_C) + 180^\circ$. Untuk banyak lingkaran berlaku kira-kira

$$\omega_C t_r \approx 1,5, \quad \phi [^\circ] + \ddot{u} [\%] \approx 70, \quad (19.13)$$

dengan t_r waktu naik dan $\ddot{u}$ overshoot dalam persen. Kalau lingkaran terbuka memuat integrator, galat tunak terhadap masukan tangga nol.

Contoh di naskah adalah plant orde dua $G(s) = 5/(1 + 1,414s + s^2)$, dengan spesifikasi waktu naik sekitar tiga detik, overshoot sekitar sepuluh persen, dan tanpa galat tunak. Aturan praktis tadi memberi $\omega_C = 1,5/3 = 0,5$ dan $\phi = 60^\circ$, sehingga di frekuensi potong fase lingkaran terbuka harus -120° . Galat tunak nol menuntut integrator, sehingga dipakai pengendali PI $R(s) = V_R(1 + sT_R)/s$. Di $\omega = 0,5$, fase plant adalah $-\arctan(0,707/0,75) \approx -43,3^\circ$, dan integrator menambah -90° , total $-133,3^\circ$. Kurangnya $13,3^\circ$ disumbang oleh nol pengendali: $\arctan(T_R \cdot 0,5) = 13,3^\circ$ memberi $T_R \approx 0,473 = 1/2,11$. Penguatan V_R dipilih agar $|L(j0,5)| = 1$. Magnitudo sisanya di frekuensi itu sekitar 9,97, sehingga $V_R \approx 0,1$. Lingkaran tertutup yang dihasilkan punya kutub $-0,368 \pm 0,776j$ dan $-0,678$, dan respons tangganya kira-kira memenuhi ketiga spesifikasi. Seluruh desain hanya memakai dua angka, tanpa menyelesaikan persamaan diferensial satu kali pun. Kekuatan cara pandang frekuensi terletak di sini.

Bentuk pengendali yang paling banyak dipakai di industri adalah PID, $R(s) = K_p + K_I/s + K_d s$. Suku proporsional bereaksi terhadap galat sekarang, suku integral terhadap galat yang menumpuk di masa lalu, dan suku derivatif terhadap arah galat ke masa depan. Pengendali PI di atas adalah PID tanpa suku derivatif.

19.8 Sistem diskret

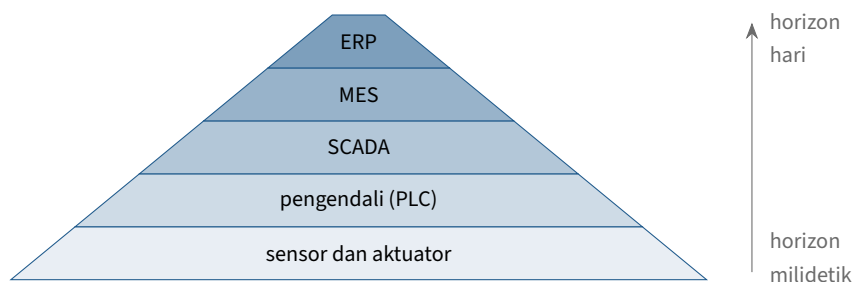
Pengendali modern berjalan di komputer, yang bekerja pada sampel dengan periode T . Kalau masukan ditahan tetap selama setiap periode, sistem kontinu $\dot{x} = Ax + Bu$ menjadi sistem diskret $x_{k+1} = A_d x_k + B_d u_k$ dengan $A_d = e^{AT}$ dan $B_d = \int_0^T e^{A\tau} d\tau B$. Untuk sistem skalar $\dot{x} = -x + u$ dengan $T = 0,1$, $A_d = e^{-0,1} \approx 0,905$ dan $B_d = 1 - e^{-0,1} \approx 0,095$. Diskretisasi ini eksak di titik-titik sampel, bukan aproksimasi. Pemetaan $z = e^{sT}$

menghubungkan domain kontinu dengan domain diskret: setengah bidang kiri yang stabil di domain s terpetakan ke bagian dalam lingkaran satuan di domain z , sehingga sistem diskret stabil kalau semua kutubnya berada di dalam lingkaran itu. Transformasi z berperan seperti Laplace untuk barisan sampel.

Untuk mengubah pengendali kontinu menjadi algoritme diskret, kita perlu mengganti s dengan sesuatu dalam z . Pendekatan Euler, $s \approx (z - 1)/T$, sederhana tetapi bisa memetakan sistem stabil menjadi tidak stabil kalau langkahnya besar. Transformasi Tustin, $s \approx \frac{2}{T} \frac{z-1}{z+1}$, berasal dari aturan trapesium untuk integral. Ia memetakan seluruh setengah bidang kiri tepat ke dalam lingkaran satuan, sehingga pengendali yang stabil tetap stabil setelah didiskretkan, dengan sedikit distorsi frekuensi di dekat frekuensi Nyquist.

19.9 Otomasi di pabrik

Teori kendali bertemu kenyataan di lantai pabrik. Kuliah production automation memperlihatkan lapisan-lapisan yang menjalankannya, sering digambarkan sebagai piramida otomasi (Gambar 35.1). Di dasar ada sensor dan aktuator. Di atasnya pengendali, terutama PLC. Lalu SCADA yang memantau dan mengawasi, sistem pelaksanaan manufaktur (MES) yang menjadwalkan produksi, dan di puncak perencanaan sumber daya perusahaan (ERP). Semakin ke bawah, semakin ketat tuntutan waktunya: ERP berpikir dalam hari, pengendali dalam milidetik.



GAMBAR 19.3. Piramida otomasi. Lapisan bawah bekerja dekat dengan proses fisik dengan tuntutan waktu nyata yang ketat, lapisan atas bekerja dengan data agregat dan horizon waktu yang panjang.

Programmable logic controller (PLC) bekerja dalam siklus pindai: baca semua input, jalankan program, tulis semua output, lalu ulangi, setiap beberapa milidetik. Programnya ditulis dalam bahasa standar IEC 61131-3: diagram tangga yang meniru rangkaian relai, diagram blok fungsi, teks terstruktur mirip Pascal, daftar instruksi mirip assembler, dan sequential function chart untuk urutan langkah. Sistem waktu nyata membedakan tenggat keras, yang tidak boleh terlewat sama sekali karena akibatnya berbahaya, dari tenggat lunak, yang sesekali boleh terlewat. Yang dikejar bukan kecepatan rata-rata, melainkan keterdugaan: variasi waktu respons, atau *jitter*, harus kecil dan terbatas.

Fieldbus menghubungkan sensor, aktuator, dan pengendali dengan komunikasi yang waktunya bisa dipastikan. Kuliah membahas beberapa cara mengatur siapa boleh berbicara. Pada skema master-slave, satu master bertanya bergiliran. Pada

token passing, hak berbicara berpindah seperti tongkat estafet. Pada CAN, semua simpul boleh mulai mengirim, dan bentrokan diselesaikan secara bitwise berdasarkan pengenalan pesan, sehingga pesan berprioritas tinggi selalu menang tanpa membuang waktu. Varian Ethernet industri seperti PROFINET membawa determinisme ke jaringan Ethernet.

Standar IEC 61499 memecah aplikasi otomasi menjadi blok fungsi yang digerakkan oleh kejadian, dengan input kejadian dan input data yang terpisah. Satu aplikasi bisa disebar ke banyak perangkat dan sumber daya, dan dipetakan ulang tanpa ditulis ulang, sebuah langkah menuju pabrik yang bisa dikonfigurasi ulang dengan cepat, tema utama gerakan Industri 4.0.

19.10 Kendali dan machine learning

Hubungan antara kendali dan machine learning berjalan ke dua arah. Dari kendali ke machine learning: model ruang keadaan menjadi state space model di deep learning (Bab 9), metode adjoint dari kendali optimal menjadi cara melatih neural ODE (Bab 15), Kalman filter menjadi kerangka data assimilation (Bab 14), dan analisis nilai eigen dari kestabilan menjadi alat untuk memahami vanishing gradient dan rollout yang meledak (Bab 30).

Dari machine learning ke kendali: model dinamika bisa dipelajari dari data alih-alih diturunkan dari hukum fisika, sebuah bidang lama yang disebut identifikasi sistem, lalu dipakai dalam pengendali prediktif yang mengoptimalkan beberapa langkah ke depan di setiap saat dan hanya menjalankan langkah pertama. Teori operator Koopman (Koopman, 1931) menawarkan cara melihat sistem nonlinear sebagai sistem linear di ruang fungsi yang lebih besar, sehingga alat-alat kendali linear bisa dipakai kembali (Bab 31). Reinforcement learning sendiri bisa dipandang sebagai kendali optimal tanpa model, dan pada sistem nyata keduanya sering digabung: model fisika memberi dasar dan jaminan, pembelajaran mengoreksi apa yang tidak dimodelkan (Brunton and Kutz, 2022).

19.11 Perkembangan riset

Model predictive control menyelesaikan masalah optimasi berkendala di setiap langkah, dengan model sistem untuk meramal beberapa langkah ke depan. Karena itu ia menjadi salah satu tempat paling alami bagi machine learning dalam kendali. Ulasan Hewing et al. (2020) membagi riset *learning-based MPC* menjadi tiga kelompok. Kelompok terbesar memakai data untuk memperbaiki model prediksi, misalnya dengan Gaussian process yang juga memberi ketidakpastian, sehingga kendala keselamatan bisa ditegakkan dengan margin yang sesuai. Kelompok kedua mempelajari parameter pengendali itu sendiri, yaitu fungsi biaya dan kendala yang menghasilkan kinerja lingkaran tertutup terbaik. Kelompok ketiga memakai MPC sebagai lapisan pengaman di atas pengendali yang dipelajari, misalnya policy reinforcement learning, sehingga aksi yang melanggar kendala dikoreksi sebelum diterapkan. Pola ketiga menjawab kekhawatiran yang sering muncul di bab ini: jaminan kestabilan dan keselamatan tidak boleh dikorbankan demi

kinerja.

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Control Systems (sistem dan model, sifat sistem, sistem LTI, keterjangkauan dan keteramatan, pengendali dan pengamat keadaan, LQR dan LQG, transformasi Laplace, desain pengendali, sistem diskret) dan kuliah Production Automation Systems (perangkat otomasi, sistem waktu nyata, PLC, fieldbus, IEC 61499). Bacaan lanjut tentang hubungan kendali dan data: Brunton and Kutz (2022).

20 Komputasi yang Menghilang

Pada musim dingin 2025/26 saya mengikuti dua kuliah pervasive computing dari Alois Ferscha. Yang satu membahas sistem dan lingkungan: visi komputasi pervasif, sensor dan modalitas, jaringan sensor, cara mengenali manusia dan benda, arsitektur pengenalan konteks, dan komputer yang dikenakan di kepala. Yang lain membahas perancangan dari sisi manusia: persepsi, penglihatan dan gerak mata, pendengaran, perabaan, atensi, emosi, antarmuka yang bisa disentuh, dan gerakan motorik. Latihannya meminta kami mengumpulkan data sensor sendiri dengan ponsel, lalu membangun pengenalan aktivitas, analisis adegan akustik, dan sistem pelacak gerak.

Bab ini membahas komputer yang tidak lagi duduk di meja, melainkan tersebar di sekitar dan di tubuh kita. Ada dua sisi yang harus dipahami bersamaan: sisi mesin, yaitu sensor dan cara menyimpulkan konteks dari sinyalnya, dan sisi manusia, yaitu bagaimana kita merasakan, memperhatikan, dan bergerak. Sistem pervasif yang baik menghormati keduanya.

20.1 Visi Weiser

Weiser (1991) membuka artikelnya dengan kalimat yang sering dikutip: teknologi yang paling mendalam adalah teknologi yang menghilang, yang menenun dirinya ke dalam kehidupan sehari-hari sampai tidak bisa dibedakan darinya. Menulis adalah contohnya. Kita tidak lagi memikirkan teknologi tulisan ketika membaca papan nama jalan. Weiser membayangkan komputer yang sama: tersebar di dinding, di meja, di pakaian, bekerja tanpa diminta, dan tidak menuntut perhatian. Kuliah menggambarkan sejarah komputasi sebagai gelombang: satu komputer besar untuk banyak orang, lalu satu komputer pribadi untuk setiap orang, lalu banyak komputer untuk setiap orang. Hari ini ponsel, jam pintar, dan rumah pintar mendekati visi itu, dengan semua manfaat dan masalahnya.

Komputasi pervasif membutuhkan tiga kemampuan: merasakan dunia lewat sensor, memahami konteksnya, dan bertindak dengan cara yang tidak mengganggu. Bagian tengah, memahami konteks, adalah tempat machine learning bekerja.

20.2 Sensor dan modalitas

Ponsel biasa membawa akselerometer, giroskop, magnetometer, mikrofon, kamera, GPS, dan sensor cahaya serta jarak. Akselerometer MEMS adalah massa kecil yang digantung pegas di dalam chip. Ketika chip dipercepat, massa itu tertinggal sedikit, dan perpindahannya diukur secara kapasitif. Karena massa itu juga ditarik gravitasi, akselerometer yang diam di meja membaca sekitar $9,81 \text{ m/s}^2$ ke atas, bukan nol. Sifat ini menjengkelkan untuk mengukur gerak, tetapi berguna untuk mengukur

kemiringan. Girooskop MEMS memanfaatkan gaya Coriolis pada massa yang bergetar untuk mengukur kecepatan sudut. Magnetometer mengukur medan magnet bumi dan memberi arah utara, kecuali di dekat benda logam yang menggangukannya.

Kuliah menekankan satu hal: tidak ada sensor yang langsung mengukur konteks. Tidak ada “sensor rapat” atau “sensor sedang memasak”. Konteks harus disimpulkan dari kombinasi sinyal mentah. Setiap modalitas punya kelebihan dan kekurangannya. Kamera kaya informasi tetapi mengganggu privasi dan boros energi. Akselerometer murah dan hemat, tetapi sinyalnya sulit ditafsirkan. Mikrofon menangkap suasana dengan baik, tetapi juga merekam percakapan.

Jaringan sensor menambahkan masalah komunikasi dan energi. Simpul-simpul kecil bertenaga baterai harus memutuskan kapan mengukur, kapan mengirim, dan kapan tidur. Mengirim data lewat radio umumnya jauh lebih mahal daripada mengolahnya di tempat, sehingga ringkasan lokal dan tidur yang terjadwal (*duty cycling*) menjadi strategi utama untuk membuat baterai bertahan berbulan-bulan.

20.3 Jaringan sensor nirkabel

Satu sensor jarang cukup. Jaringan sensor nirkabel terdiri atas banyak simpul kecil yang otonom, *mote*, masing-masing dengan sensor, prosesor sederhana, radio, dan baterai, yang bersama-sama memantau sesuatu yang luas: iklim mikro di bawah dan di atas tanah, sarang burung, getaran jembatan, atau kualitas udara kota. Visi *smart dust* dari akhir 1990-an membayangkan simpul seukuran butir debu. Kuliah memperlihatkan contoh pemantauan habitat, di mana jaringan seperti ini punya kelebihan yang jelas: titik ukur yang banyak mengurangi galat interpolasi, simpul yang kecil tidak mengganggu perilaku hewan yang diamati, dan sistem bisa berjalan berbulan-bulan tanpa ditunggui.

Batasan utamanya adalah energi. Mengirim satu bit lewat radio biasanya jauh lebih mahal daripada memproses ribuan instruksi, sehingga data diolah dan diringkas di simpul sebelum dikirim, radio dinyalakan sesingkat mungkin dengan *duty cycling*, dan data diteruskan dari simpul ke simpul dengan beberapa lompatan pendek alih-alih satu lompatan jauh, karena energi pancaran tumbuh lebih cepat dari linear terhadap jarak. Jaringan juga harus mengatur dirinya sendiri dan tetap bekerja ketika sebagian simpul mati.

Masalah yang tampak sepele tetapi menentukan adalah waktu. Jam di setiap simpul berjalan sedikit berbeda. Kristal dengan drift 50 bagian per juta meleset sekitar $50 \times 10^{-6} \times 86\,400 \approx 4,3$ detik per hari. Untuk melokalisasi sumber suara dari selisih waktu tiba di beberapa simpul, atau untuk beamforming seperti di Bab 18, galat seperti itu fatal. Protokol berbasis waktu bolak-balik seperti NTP memakai empat cap waktu: simpul A mengirim pada T_1 menurut jamnya, B menerima pada T_2 dan membalas pada T_3 menurut jamnya, dan A menerima balasan pada T_4 . Dengan asumsi waktu tempuh kedua arah sama, selisih jam dan waktu tempuh bolak-balik adalah

$$\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}, \quad \delta = (T_4 - T_1) - (T_3 - T_2). \quad (20.1)$$

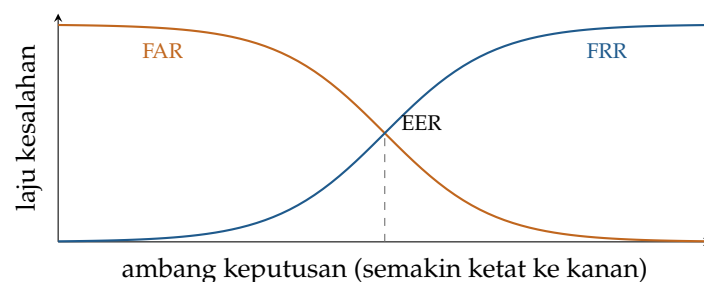
Misalkan jam B dua milidetik di depan jam A, setiap arah butuh dua milidetik, dan B membalas satu milidetik setelah menerima. A mengirim pada $T_1 = 100$. Pesan

tiba ketika jam A menunjukkan 102, yaitu $T_2 = 104$ menurut jam B. B membalas pada $T_3 = 105$, dan balasan tiba dua milidetik kemudian, ketika jam B menunjukkan 107, yaitu $T_4 = 105$ menurut jam A. Rumus di atas memberi $\theta = (4 + 0)/2 = 2$ dan $\delta = 5 - 1 = 4$, tepat seperti seharusnya. Rumus itu bergantung pada asumsi bahwa kedua arah sama cepat. Kalau pesan berangkat butuh tiga milidetik dan balasan hanya satu, δ tetap empat, tetapi taksiran θ meleset sebesar separuh selisihnya, satu milidetik, dan tidak ada cara untuk mendeteksinya dari keempat cap waktu saja. *Reference broadcast synchronization* menghindarinya dengan cara cerdas. Satu simpul menyiarkan sinyal acuan, dan simpul-simpul penerima tidak menyinkronkan diri dengan pengirim, melainkan saling membandingkan kapan masing-masing menerima sinyal yang sama. Ketidakpastian di sisi pengirim, yang biasanya paling besar, hilang dari perhitungan.

20.4 Mengenali manusia

Sistem biometrik mengenali orang dari ciri tubuh atau perilaku: sidik jari, wajah, iris, suara, atau cara berjalan. Semuanya bekerja dengan cara yang sama. Sampel baru dibandingkan dengan templat yang tersimpan, menghasilkan skor kemiripan, dan skor itu dibandingkan dengan ambang. Dua jenis kesalahan muncul. *False acceptance rate* (FAR) adalah fraksi percobaan orang lain yang keliru diterima. *False rejection rate* (FRR) adalah fraksi percobaan pemilik sah yang keliru ditolak. Kalau dari 1000 percobaan orang lain ada 10 yang diterima, FAR-nya 1 persen.

Kedua kesalahan bergerak berlawanan arah ketika ambang digeser (Gambar 20.1). Ambang yang ketat membuat penyusup sulit masuk tetapi juga sering menolak pemiliknya, ambang yang longgar sebaliknya. Titik tempat kedua kurva berpotongan disebut *equal error rate*, satu angka yang sering dipakai untuk membandingkan sistem. Dalam praktik, ambang dipilih menurut kebutuhan: pintu ruang server memilih FAR kecil sekali walau sesekali menyebalkan pemiliknya, sedangkan pembuka layar ponsel memilih FRR kecil karena kenyamanan lebih penting.



GAMBAR 20.1. Trade-off FAR dan FRR terhadap ambang pada sistem biometrik. Potongannya adalah equal error rate. Kurva bersifat skematik.

20.5 Mengenali benda dan tempat

Benda bisa dikenali dengan kode batang atau kode QR yang dibaca kamera, atau dengan label RFID yang dibaca tanpa kontak dan tanpa garis pandang. Label RFID pasif

tidak punya baterai. Ia menerima energi dari medan pembaca dan menjawab dengan memantulkan sebagian sinyal itu dalam pola tertentu. NFC adalah RFID frekuensi tinggi dengan jangkauan beberapa sentimeter, dipakai untuk pembayaran nirsentuh, karena jarak yang pendek justru menjadi fitur keamanan.

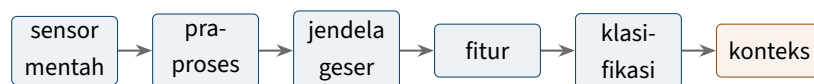
Untuk mengetahui posisi di dalam gedung, di mana GPS tidak sampai, kuliah membahas beacon Bluetooth Low Energy. Beacon memancarkan sinyal secara berkala, dan ponsel mengukur kekuatan sinyal yang diterima (RSSI). Kekuatan sinyal melemah dengan jarak menurut model rugi lintasan logaritmik,

$$\text{RSSI}(d) = \text{RSSI}_{1\text{m}} - 10 n \log_{10} \frac{d}{1\text{m}}, \quad (20.2)$$

dengan n eksponen rugi lintasan, sekitar dua di ruang bebas dan lebih besar di dalam gedung. Membalik persamaan ini memberi perkiraan jarak, $d = 10^{(\text{RSSI}_{1\text{m}} - \text{RSSI})/(10n)}$. Kalau sebuah beacon terbaca -59 dBm pada satu meter dan ponsel membaca -75 dBm dengan $n = 2$, jaraknya sekitar $10^{0.8} \approx 6,3$ m. Dalam praktik, tubuh manusia, dinding, dan pantulan membuat RSSI berfluktuasi beberapa desibel, yang berarti galat jarak puluhan persen. Karena itu posisi biasanya diperkirakan dari beberapa beacon sekaligus dan dihaluskan dengan filter, atau dicocokkan dengan peta sidik jari RSSI yang direkam sebelumnya, sebuah masalah regresi yang cocok untuk machine learning.

20.6 Rantai pengenalan konteks

Arsitektur pengenalan yang dibahas kuliah mengikuti rantai yang juga dijelaskan dalam tutorial Bulling et al. (2014) (Gambar 20.2). Data mentah dari sensor dipotong menjadi jendela-jendela waktu yang tumpang tindih, misalnya dua detik dengan geseran satu detik. Pada frekuensi sampling 50 Hz, setiap jendela berisi seratus sampel per sumbu. Dari setiap jendela dihitung fitur. Fitur domain waktu meliputi rata-rata, simpangan baku, nilai minimum dan maksimum, rentang antarkuartil, kemiringan dan keruncingan distribusi, laju perpotongan nol, dan korelasi antar sumbu. Fitur domain frekuensi dihitung dari FFT: frekuensi dominan, energi di pita tertentu, dan entropi spektral, yang mengukur seberapa merata energi tersebar di semua frekuensi. Fitur-fitur itu dimasukkan ke pengklasifikasi, misalnya k -nearest neighbour, pohon keputusan, SVM, atau jaringan saraf, yang menghasilkan label aktivitas untuk setiap jendela. Langkah terakhir menghaluskan urutan label, karena seseorang tidak berganti dari berjalan ke duduk lalu ke berjalan lagi dalam tiga detik.



GAMBAR 20.2. Rantai pengenalan konteks: dari sinyal sensor mentah, lewat penyaringan dan pemotongan jendela, ekstraksi fitur, dan klasifikasi, sampai label konteks.

Ketika ada beberapa sensor, informasinya bisa digabung di tiga tingkat. Fusi tingkat data menggabungkan sinyal mentah, misalnya rata-rata dua akselerometer. Fusi tingkat fitur menggabungkan vektor fitur dari semua sensor sebelum klasifikasi.

Fusi tingkat keputusan menjalankan pengklasifikasi terpisah untuk setiap sensor lalu menggabungkan keputusannya, misalnya dengan pemungutan suara. Semakin awal fusi dilakukan, semakin banyak informasi yang tersedia, tetapi semakin rapuh sistemnya kalau satu sensor rusak.

Rantai ini terlihat sederhana, tetapi kuliah dan latihan memperlihatkan jebakannya. Yang paling penting adalah cara evaluasi. Kalau jendela-jendela dari orang yang sama masuk ke data latih dan data uji sekaligus, model belajar mengenali gaya gerak orang itu, bukan aktivitasnya, dan skornya tampak terlalu bagus. Evaluasi yang jujur meninggalkan satu orang di luar pelatihan lalu mengujinya pada orang itu, sebuah variasi dari pembagian data yang hati-hati di Bab 1. Dalam latihan kami, yang dikenali adalah cara menggenggam dan gerakan memasang sekrup, direkam dengan akselerometer ponsel. Setiap jendela diringkas menjadi sekitar empat puluh fitur, dari rata-rata, simpangan baku, kemiringan distribusi, dan laju perpotongan nol di setiap sumbu, sampai korelasi antar sumbu, frekuensi dominan, dan entropi spektral.

Evaluasi pengenalan aktivitas punya jebakan khasnya sendiri. Jendela-jendela yang berdekatan dari rekaman yang sama hampir identik, apalagi kalau jendelanya tumpang tindih separuh seperti yang lazim dipakai. Membagi jendela secara acak ke data latih dan data uji membuat model diuji pada potongan yang hampir sama dengan yang ia pelajari, dan akurasi tampak jauh lebih tinggi dari kenyataan. Yang jujur adalah validasi *leave-one-subject-out*: latih pada semua orang kecuali satu, uji pada orang yang tersisa, lalu ulangi untuk setiap orang. Cara ini menjawab pertanyaan yang sebenarnya ingin kita ketahui, yaitu seberapa baik sistem bekerja untuk orang baru, dan biasanya memberi angka yang jauh lebih rendah. Hasilnya sebaiknya dilaporkan sebagai matriks konfusi per kelas, bukan hanya akurasi total, karena aktivitas yang jarang, seperti jatuh, justru yang paling penting dan paling mudah tenggelam dalam rata-rata.

Prediksi per jendela juga bisa dihaluskan. Aktivitas manusia punya durasi: orang tidak beralih dari berjalan ke duduk ke berlari setiap dua detik. Hidden Markov model dari Bab 14, dengan aktivitas sebagai keadaan tersembunyi dan keluaran pengklasifikasi sebagai pengamatan, menghukum pergantian yang terlalu sering. Algoritme Viterbi lalu memilih urutan aktivitas yang paling masuk akal secara keseluruhan, dan kesalahan sesaat yang terisolasi hilang dengan sendirinya.

20.7 Melacak gerak

Sensor inersia memberi kecepatan sudut dari giroskop dan percepatan dari akselerometer. Mengintegrasikan kecepatan sudut memberi orientasi, tetapi setiap bias kecil ikut terintegrasi dan orientasi perlahan melenceng. Akselerometer, saat sensor diam atau bergerak pelan, mengukur arah gravitasi dan memberi kemiringan yang tidak melenceng, misalnya $\theta^{\text{acc}} = \text{atan2}(a_y, a_z)$ untuk sudut terhadap satu sumbu, tetapi berisik. Kedua sumber ini saling melengkapi, dan filter komplementer menggabungkannya:

$$\theta_k = \alpha(\theta_{k-1} + \omega_k \Delta t) + (1 - \alpha) \theta_k^{\text{acc}}. \quad (20.3)$$

Nama “komplementer” punya arti tepat. Bagian giroskop diproses oleh filter lolos tinggi dan bagian akselerometer oleh filter lolos rendah dengan frekuensi potong yang

sama, sehingga kedua filter berjumlah satu di semua frekuensi. Konstanta waktu peralihannya kira-kira $\tau = \alpha \Delta t / (1 - \alpha)$. Dengan $\Delta t = 0,01$ detik dan $\alpha = 0,98$, $\tau \approx 0,5$ detik: perubahan yang lebih cepat dari setengah detik dipercayakan pada giroskop, yang lebih lambat pada akselerometer. Ini versi buatan tangan dari Kalman filter di Bab 14, dengan gain yang dipilih sendiri alih-alih dihitung dari kovarians.

Melacak posisi jauh lebih sulit. Percepatan harus diintegrasikan dua kali, sehingga bias percepatan b yang konstan menghasilkan galat posisi $\frac{1}{2}bt^2$. Bias sekecil $0,01 \text{ m/s}^2$, seperseribu gravitasi, sudah membuat galat 18 meter setelah satu menit. Karena itu pelacak posisi inersia murni hanya berguna untuk waktu singkat, kecuali ada koreksi dari luar, seperti langkah kaki yang terdeteksi, kamera, atau beacon.

20.8 Manusia di dalam lingkaran: persepsi

Sepuluh kedua materi membahas sisi manusia, karena sistem pervasif berinteraksi langsung dengan persepsi dan perhatian kita. Psikofisika mengukur hubungan antara rangsangan fisik dan pengalaman. Hukum Weber menyatakan bahwa perbedaan terkecil yang bisa dirasakan sebanding dengan besarnya rangsangan, $\Delta I/I = k$. Kalau kita baru bisa membedakan perubahan berat dua persen, maka pada beban seratus gram perbedaan dua gram terasa, sedangkan pada satu kilogram butuh dua puluh gram. Fechner mengintegrasikan hukum ini menjadi hubungan logaritmik antara kekuatan rangsangan dan kekuatan sensasi, $S = k \ln(I/I_0)$, yang menjelaskan mengapa telinga dan mata kita bekerja nyaman dalam skala desibel.

Mata hanya melihat tajam di daerah kecil di tengah penglihatan, fovea, selebar sekitar dua derajat. Untuk melihat sebuah adegan, mata melompat-lompat dalam gerakan cepat yang disebut sakade, lalu berhenti sejenak dalam fiksasi yang biasanya berlangsung beberapa ratus milidetik. Pelacak mata modern menyinari mata dengan inframerah dan mengikuti posisi pusat pupil relatif terhadap pantulan cahaya di kornea. Dari deret posisi pandang, fiksasi dan sakade dipisahkan, misalnya dengan ambang kecepatan sudut: sampel dengan kecepatan di atas sekitar tiga puluh derajat per detik dianggap bagian dari sakade. Pola fiksasi mengungkap apa yang menarik perhatian, seberapa berat beban pikiran, atau apakah seseorang mengantuk.

Memakai pandangan mata sebagai input punya masalah klasik yang disebut masalah sentuhan Midas: mata kita melihat banyak hal tanpa bermaksud memilihnya, sehingga sistem harus bisa membedakan melihat dari memilih. Solusi yang umum adalah waktu diam (*dwell time*), memilih sesuatu hanya kalau dipandang cukup lama, atau menggabungkan pandangan dengan konfirmasi dari modalitas lain.

Telinga kita mendengar dari sekitar 20 Hz sampai 20 kHz dan menentukan arah suara terutama dari dua isyarat: perbedaan waktu tiba antara kedua telinga dan perbedaan kerasnya. Perbedaan waktu terbesar terjadi untuk suara dari samping, kira-kira lebar kepala dibagi kecepatan suara, sekitar $0,2/343 \approx 0,6$ milidetik. Otak kita memakai perbedaan sekecil itu untuk menentukan arah, sebuah prestasi pemrosesan sinyal yang mengesankan. Perabaan dilayani reseptor yang peka terhadap tekanan, getaran, dan regangan, dan antarmuka vibrotaktil memanfaatkannya untuk menyampaikan informasi tanpa mengganggu mata dan telinga.

20.9 Atensi dan gerak

Perhatian manusia terbatas. Teori sumber daya ganda dari Wickens, yang dibahas di kuliah, menyatakan bahwa dua tugas saling mengganggu lebih sedikit kalau memakai sumber daya yang berbeda: modalitas yang berbeda, misalnya mata dan telinga, kode yang berbeda, spasial atau verbal, dan tahap pemrosesan yang berbeda. Itulah sebabnya navigasi dengan suara lebih aman saat mengemudi daripada navigasi yang harus dibaca, dan mengapa notifikasi yang datang di saat yang salah begitu merusak konsentrasi.

Dua hukum empiris yang terkenal menggambarkan kecepatan kita bereaksi dan bergerak. Hukum Hick–Hyman menyatakan bahwa waktu memilih di antara n pilihan yang sama mungkin tumbuh secara logaritmik, $RT = a + b \log_2(n + 1)$, karena memilih adalah memangkas ketidakpastian. Hukum Fitts menyatakan bahwa waktu untuk menunjuk sebuah target berjarak D dan selebar W adalah

$$MT = a + b \log_2 \left(\frac{D}{W} + 1 \right). \quad (20.4)$$

Suku logaritma disebut indeks kesulitan, dalam satuan bit. Menunjuk tombol selebar 2 cm yang berjarak 14 cm punya indeks kesulitan $\log_2 8 = 3$ bit. Menggandakan jarak sambil menggandakan lebar tidak mengubah kesulitannya. Dari sini lahir prinsip desain yang kita kenal: tombol penting dibuat besar dan dekat, dan sudut-sudut layar adalah target yang paling mudah karena kursor tidak bisa melewatinya, seolah lebarnya tak hingga.

20.10 Emosi dan antarmuka yang bisa disentuh

Komputasi afektif mencoba mengenali dan menanggapi emosi. Model emosi yang dipakai beragam, dari beberapa kategori emosi dasar seperti senang, sedih, marah, takut, terkejut, dan jijik, sampai ruang dua dimensi valensi dan gairah, di mana setiap emosi adalah sebuah titik: senang berada di valensi positif dan gairah sedang, marah di valensi negatif dan gairah tinggi. Sinyalnya bisa berupa wajah, suara, postur, atau sinyal fisiologis seperti detak jantung dan konduktansi kulit. Kuliah mengingatkan bahwa emosi yang diungkapkan tidak selalu sama dengan yang dirasakan, bahwa ekspresi berbeda antar budaya, dan bahwa pengenalan emosi menyentuh wilayah privasi yang pribadi. AI Act Uni Eropa bahkan melarang pengenalan emosi di tempat kerja dan sekolah (Bab 35).

Antarmuka yang bisa disentuh, atau *tangible user interface*, memberi bentuk fisik pada informasi digital, sehingga kita bisa memegang, memindah, dan menyusun benda nyata untuk mengendalikan komputer. Kuliah juga melihat ke depan, ke materi yang bisa diprogram dan mengubah bentuknya sendiri, ketika batas antara benda dan antarmuka menghilang sepenuhnya.

20.11 Di mana machine learning masuk

Hampir seluruh rantai pengenalan konteks kini bisa diganti jaringan saraf yang belajar fiturnya sendiri dari sinyal mentah, seperti jaringan konvolusi satu dimensi atau LSTM

pada data akselerometer (Bab 9). Analisis adegan akustik, yang juga menjadi salah satu latihan kami, mengenali tempat dan kejadian dari suara: sinyal diubah menjadi spektrogram, lalu dikenali seperti gambar oleh jaringan visi. Pelacak mata memakai jaringan konvolusi untuk menemukan pupil dalam gambar yang buruk, dan pengenalan emosi memakai model multimodal.

Tantangan khas bidang ini tetap ada. Data berlabel mahal karena harus direkam dengan orang sungguhan. Setiap orang bergerak berbeda, dan sensor bergeser posisinya di tubuh. Perangkatnya punya baterai dan memori kecil, sehingga model kecil yang bisa berjalan di mikrokontroler, sering disebut TinyML, menjadi cabang tersendiri. Dan karena sensornya melekat di tubuh dan di rumah, pertanyaan privasi dan persetujuan tidak bisa dipisahkan dari pertanyaan teknis (Bab 35).

20.12 Perkembangan riset

Pengenalan aktivitas dari sensor yang dikenakan lama terhambat oleh data berlabel yang sedikit dan beragam: setiap studi punya perangkat, posisi pemakaian, dan kelompok orang yang berbeda. Yuan et al. (2024) memakai data akselerometer pergelangan tangan dari UK Biobank, sekitar 700 ribu hari-orang tanpa label, untuk pretraining self-supervised. Tugas buatannya mengenali transformasi yang diterapkan pada sinyal, misalnya apakah sinyal dibalik dalam waktu atau dipermutasi. Model yang dihasilkan, setelah disetel pada delapan dataset berlabel, mengungguli pembandingan dengan median kenaikan skor F1 sekitar 24 persen, dan yang lebih penting, generalisasi ke dataset, kelompok, lingkungan, dan perangkat lain. Itulah pola model fondasi dari Bab 6 dan Bab 38, diterapkan pada rantai pengenalan konteks yang dibahas di bab ini.

CATATAN SUMBER. Bab ini mengikuti kuliah *Pervasive Computing: Systems and Environments* (visi, sensor dan modalitas, jaringan sensor, identifikasi manusia dan benda, arsitektur pengenalan, komputasi yang dikenakan di kepala) dan *Pervasive Computing: Design and Development* (persepsi, penglihatan dan pelacakan mata, pendengaran, perabaan, atensi, komputasi afektif, antarmuka tangible, gerak motorik), beserta latihan pengenalan aktivitas, analisis adegan akustik, dan pelacakan gerak.

21 Robot dan Kendaraan Otonom

Musim panas 2025 membawa saya ke dua kuliah tentang mesin yang bergerak. Introduction to robotic systems membahas mekanika lengan robot: kerangka koordinat, kecepatan benda kaku, kinematika maju dan balik, Jacobian, persamaan gerak dari prinsip Lagrange, visual servoing, perencanaan jalur, dan profil trayektori. Introduction to autonomous vehicles membahas mobil yang mengemudi sendiri: lokalisasi Markov, pencocokan pindaian dengan ICP dan NDT, sensor lidar, deteksi objek tiga dimensi, Kalman filter dan versi diperluasnya, dan pelacakan banyak objek, dengan proyek lokalisasi dan proyek persepsi.

Robot adalah tempat semua yang sudah kita bahas bertemu. Ia harus melihat (Bab 11), menaksir keadaannya di tengah ketidakpastian (Bab 14), mengendalikan gerakannya (Bab 19), dan memutuskan apa yang dilakukan (Bab 5 dan 16). Bab ini melihat dua jenis robot: lengan industri yang bekerja di tempat, dan kendaraan yang bergerak di dunia terbuka.

21.1 Koordinat dan rotasi

Robot hidup di antara banyak kerangka koordinat: dunia, dasar robot, setiap sendi, ujung lengan, kamera. Orientasi sebuah kerangka relatif terhadap kerangka lain dinyatakan dengan matriks rotasi $\mathbf{R}$, matriks ortogonal dengan determinan satu. Rotasi sebesar θ di sekitar sumbu z , misalnya, adalah

$$\mathbf{R}_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (21.1)$$

Rotasi sembarang bisa disusun dari tiga rotasi dasar, misalnya sudut Cardan untuk guling, angguk, dan geleng, tetapi urutannya penting karena rotasi tidak komutatif. Memutar buku sembilan puluh derajat ke depan lalu ke kanan menghasilkan posisi yang berbeda dari urutan sebaliknya. Posisi dan orientasi digabung menjadi matriks transformasi homogen 4×4 , sehingga transformasi berurutan cukup dikalikan, seperti pada model kamera di Bab 11.

Kecepatan benda kaku juga punya struktur yang rapi. Kecepatan sebuah titik di benda yang berputar dengan kecepatan sudut $\boldsymbol{\omega}$ adalah $\boldsymbol{\omega} \times \mathbf{r}$, dan perkalian silang ini bisa ditulis sebagai perkalian dengan matriks antisimetris yang disebut *spin tensor* di kuliah. Turunan waktu matriks rotasi adalah $\dot{\mathbf{R}} = [\boldsymbol{\omega}]_{\times} \mathbf{R}$, hubungan yang dipakai di setiap integrator orientasi, termasuk pelacak gerak di Bab 20.

21.2 Kinematika

Konvensi Denavit–Hartenberg (Denavit and Hartenberg, 1955) memberi resep standar untuk menempatkan kerangka di setiap sendi, sehingga setiap sambungan hanya butuh empat parameter: dua jarak dan dua sudut. Transformasi dari dasar ke ujung lengan adalah hasil kali transformasi semua sambungan.

Forward kinematics menghitung posisi ujung lengan dari sudut-sudut sendi. Untuk lengan planar dua ruas dengan panjang l_1 dan l_2 serta sudut q_1 dan q_2 ,

$$x = l_1 \cos q_1 + l_2 \cos(q_1 + q_2), \quad y = l_1 \sin q_1 + l_2 \sin(q_1 + q_2). \quad (21.2)$$

Dengan $l_1 = l_2 = 1$, $q_1 = 0$, dan $q_2 = 90^\circ$, ujung lengan berada di $(1, 1)$.

Inverse kinematics berjalan ke arah sebaliknya. Untuk lengan dua ruas, penurunannya memakai aturan kosinus. Jarak dari dasar ke target adalah $r^2 = x^2 + y^2$, dan segitiga yang dibentuk kedua ruas memberi $r^2 = l_1^2 + l_2^2 + 2l_1l_2 \cos q_2$, sehingga

$$\cos q_2 = \frac{x^2 + y^2 - l_1^2 - l_2^2}{2l_1l_2}, \quad q_1 = \text{atan2}(y, x) - \text{atan2}(l_2 \sin q_2, l_1 + l_2 \cos q_2). \quad (21.3)$$

Untuk target $(1, 1)$, $\cos q_2 = (2 - 2)/2 = 0$, jadi $q_2 = \pm 90^\circ$: dua jawaban, siku di atas atau siku di bawah. Untuk $q_2 = 90^\circ$, $q_1 = 45^\circ - 45^\circ = 0$, cocok dengan contoh maju tadi. Kalau ruas kanan rumus kosinus lebih besar dari satu, targetnya di luar jangkauan dan tidak ada jawaban. Lengan dengan lebih banyak sendi dari yang dibutuhkan, disebut redundan, punya tak hingga jawaban, dan kebebasan itu bisa dipakai untuk menghindari rintangan atau menjauhi batas sendi.

Jacobian menghubungkan kecepatan sendi dengan kecepatan ujung lengan, $\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$. Untuk lengan dua ruas, menurunkan x dan y terhadap q_1 dan q_2 memberi

$$\mathbf{J} = \begin{pmatrix} -l_1 \sin q_1 - l_2 \sin(q_1 + q_2) & -l_2 \sin(q_1 + q_2) \\ l_1 \cos q_1 + l_2 \cos(q_1 + q_2) & l_2 \cos(q_1 + q_2) \end{pmatrix}, \quad \det \mathbf{J} = l_1 l_2 \sin q_2. \quad (21.4)$$

Ketika lengan terbentang lurus atau terlipat penuh, $\sin q_2 = 0$ dan Jacobian kehilangan pangkat. Di konfigurasi *singular* seperti ini ada arah gerak yang tidak bisa dicapai sama sekali. Cobalah meluruskan lengan Anda sepenuhnya: tangan tidak bisa lagi bergerak menjauh dari bahu, dan untuk bergerak sedikit saja di dekat titik itu, siku harus berputar cepat sekali. Karena itu pengendali robot menghindari daerah di sekitar singularitas.

21.3 Dinamika

Kinematika tidak peduli pada massa dan gaya. Untuk mengendalikan lengan yang sebenarnya, kita perlu tahu torsi apa yang menghasilkan gerak tertentu. Kuliah menurunkan dengan prinsip Hamilton dan persamaan Lagrange: tulis energi kinetik T dan energi potensial V dalam koordinat sendi, bentuk $L = T - V$, lalu terapkan

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = \tau_i. \quad (21.5)$$

Bandul sederhana memperlihatkan resepnya. Untuk massa m di ujung batang ringan sepanjang l dengan sudut θ dari arah bawah, $T = \frac{1}{2}ml^2\dot{\theta}^2$ dan $V = -mgl \cos \theta$. Maka $\partial L/\partial \dot{\theta} = ml^2\dot{\theta}$, yang turunannya terhadap waktu $ml^2\ddot{\theta}$, dan $\partial L/\partial \theta = -mgl \sin \theta$. Persamaan geraknya

$$ml^2\ddot{\theta} + mgl \sin \theta = \tau. \quad (21.6)$$

Untuk lengan dengan banyak sendi, langkahnya sama, hanya aljabarnya lebih panjang, dan hasilnya selalu berbentuk $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau$: matriks massa yang bergantung pada konfigurasi, suku Coriolis dan sentrifugal, dan gravitasi. Struktur ini dipakai langsung untuk kendali. Kalau persamaannya diketahui, pengendali bisa menghitung torsi yang membatalkan nonlinearitas dan menyisakan sistem linear yang mudah dikendalikan, teknik yang disebut *computed torque*. Deskripsi robot untuk perangkat lunak, seperti format URDF, menyimpan semua massa, inersia, dan geometri yang dibutuhkan (Lynch and Park, 2017).

21.4 Melihat, merencanakan, dan bergerak

Visual servoing mengendalikan robot dengan umpan balik dari kamera. Model kamera dan kalibrasinya sama dengan di Bab 11. Pada visual servoing berbasis posisi, pose objek diperkirakan dari gambar, lalu galat pose diumpankan ke pengendali. Pada varian berbasis gambar, galat langsung dihitung dari posisi fitur di gambar, sehingga sistem tahan terhadap kalibrasi yang tidak sempurna.

Untuk bergerak tanpa menabrak, kuliah membahas medan potensial buatan. Tujuan memberi potensial yang menarik, $U_{\text{att}} = \frac{1}{2}k_a\|q - q_g\|^2$, dengan gaya $-\nabla U_{\text{att}} = -k_a(q - q_g)$ yang selalu menunjuk ke tujuan. Rintangan memberi potensial yang menolak, yang hanya aktif di dekatnya, misalnya $U_{\text{rep}} = \frac{1}{2}k_r(1/\rho - 1/\rho_0)^2$ untuk jarak ρ yang lebih kecil dari jangkauan pengaruh ρ_0 , dan nol di luar itu. Robot mengikuti gradien negatif jumlahnya, seperti bola yang menggelinding di lanskap. Cara ini elegan dan murah, tetapi punya masalah yang sudah kita kenal dari optimasi: minimum lokal. Robot bisa terjebak di lekukan antara dua rintangan, di mana tarikan dan tolakan seimbang. Perencana berbasis graf seperti A^* (Bab 5) atau perencana berbasis sampel menghindarinya dengan mencari secara global.

Gerak sepanjang jalur diberi profil kecepatan yang halus. Profil trapesium punya tiga fase: percepatan konstan, kecepatan konstan, dan perlambatan. Untuk bergerak satu meter dengan kecepatan maksimum 0,5 m/s dan percepatan 1 m/s², fase percepatan berlangsung 0,5 s dan menempuh 0,125 m. Fase perlambatan sama. Sisa 0,75 m ditempuh dengan kecepatan penuh dalam 1,5 s, sehingga seluruh gerak memakan 2,5 s. Lompatan percepatan di ujung fase membuat sentakan, sehingga kuliah juga membahas profil dengan percepatan berbentuk sinus yang lebih halus untuk muatan yang rapuh.

21.4.1 Ruang sendi dan ruang tugas

Kuliah membedakan dua tempat untuk merencanakan gerak. Di ruang sendi, lintasan ditulis langsung sebagai sudut-sudut sendi terhadap waktu. Kinematika balik tidak perlu dihitung, tidak ada singularitas yang harus dihindari, dan batas kecepatan setiap

motor mudah ditegakkan. Harganya, bentuk lintasan ujung lengan di ruang nyata tidak diketahui persis, sehingga lengan bisa menabrak sesuatu di antara titik awal dan titik akhir. Di ruang tugas, lintasan ujung lengan ditulis langsung, misalnya garis lurus untuk mengelas sambungan, tetapi setiap titiknya harus diubah dengan kinematika balik, dan lintasan itu bisa melewati singularitas atau melanggar batas sendi. Garis lurus di satu ruang hampir selalu melengkung di ruang yang lain.

Pilihannya mengikuti jenis tugas. Gerak *continuous path*, seperti mengelas, mengelem, atau mengecat, menuntut ujung lengan mengikuti jalur yang tepat, sehingga direncanakan di ruang tugas. Gerak *point-to-point*, seperti mengambil benda dari satu tempat ke tempat lain, hanya peduli pada titik awal dan akhir, sehingga lebih murah direncanakan di ruang sendi. Rintangan juga punya bayangan di ruang sendi: kumpulan konfigurasi yang membuat lengan menabrak rintangan membentuk daerah terlarang di ruang konfigurasi, dan perencanaan jalur menjadi pencarian jalan di antara daerah-daerah itu, persis masalah pencarian di Bab 5.

21.4.2 Penskalaan waktu dengan polinom

Jalur geometris diberi waktu dengan fungsi penskalaan $s(t)$ yang naik dari 0 ke 1 selama durasi T , sehingga posisi di setiap saat adalah $\mathbf{q}(t) = \mathbf{q}_0 + s(t)(\mathbf{q}_1 - \mathbf{q}_0)$. Profil trapesium di atas adalah satu pilihan. Pilihan lain adalah polinom. Polinom kubik $s(t) = a_0 + a_1t + a_2t^2 + a_3t^3$ punya empat koefisien, cukup untuk empat syarat: posisi 0 dan 1 di kedua ujung, dan kecepatan nol di kedua ujung. Syarat di $t = 0$ langsung memberi $a_0 = a_1 = 0$. Dua syarat di $t = T$ memberi $a_2T^2 + a_3T^3 = 1$ dan $2a_2T + 3a_3T^2 = 0$, yang penyelesaiannya

$$s(t) = 3\left(\frac{t}{T}\right)^2 - 2\left(\frac{t}{T}\right)^3. \quad (21.7)$$

Kecepatannya $\dot{s} = \frac{6}{T}\left(\frac{t}{T} - \frac{t^2}{T^2}\right)$, maksimum di tengah dengan nilai $1,5/T$. Untuk memindahkan sebuah sendi sejauh 90 derajat dalam dua detik, kecepatan puncaknya $1,5 \times 90/2 = 67,5$ derajat per detik, dan batas kecepatan motor menentukan T terpendek yang diizinkan. Percepatan polinom kubik tidak nol di kedua ujung, sehingga ada sentakan saat gerak dimulai dan berhenti. Polinom derajat lima punya enam koefisien, cukup untuk juga menolak percepatan di kedua ujung, dengan harga kecepatan puncak yang sedikit lebih tinggi.

21.4.3 Visual servoing berbasis posisi

Kuliah menurunkan hukum kendali visual servoing berbasis posisi dengan cukup rinci. Kamera yang sudah dikalibrasi mengukur posisi dan orientasi target relatif terhadap kamera, dan galat pelacakan $\mathbf{e}$ adalah selisih antara pose terukur itu dan pose yang diinginkan, dengan orientasi dinyatakan dalam sudut Cardan. Untuk target yang diam, laju perubahan galat bergantung linear pada kecepatan kamera lewat sebuah Jacobian $\mathbf{L}$, $\dot{\mathbf{e}} = \mathbf{L}\mathbf{v}$. Kalau robot dikendalikan dengan kecepatan, perintahnya dipilih

$$\mathbf{v} = -\lambda \mathbf{L}^{-1}\mathbf{e}, \quad (21.8)$$

sehingga dinamika lingkaran tertutupnya menjadi $\dot{\mathbf{e}} = -\lambda\mathbf{e}$. Setiap komponen galat meluruh secara eksponensial, $\mathbf{e}(t) = \mathbf{e}(0)e^{-\lambda t}$. Dengan $\lambda = 2$ per detik, galat tinggal

sekitar 14 persen setelah satu detik. Inilah pengendali proporsional dari Bab 19, yang ditulis di ruang galat visual. Pada visual servoing berbasis gambar, galat diukur langsung pada posisi fitur di gambar, dan L menjadi Jacobian gambar yang menghubungkan gerak kamera dengan gerak fitur di bidang gambar.

21.5 Di mana aku berada: lokalisasi

Mobil otonom harus tahu posisinya dengan ketelitian beberapa sentimeter, jauh lebih teliti dari GPS biasa. Kuliah membahas tiga pendekatan.

Markov localization adalah filter Bayes dari Bab 14 di atas grid posisi (Thrun et al., 2005). Contoh dari kuliah adalah mobil di jalan lurus sepanjang seratus meter dengan beberapa tiang yang posisinya diketahui di peta. Pada awalnya mobil tidak tahu apa-apa, jadi keyakinannya merata di seluruh jalan. Lalu sensornya melihat sebuah tiang lima meter di depan. Setiap sel grid yang berjarak lima meter dari sebuah tiang menjadi lebih mungkin, sehingga keyakinan memuncak di beberapa tempat sekaligus, satu untuk setiap tiang. Mobil lalu bergerak sepuluh meter. Langkah prediksi menggeser seluruh distribusi sepuluh meter dan sedikit melebarkannya, karena odometri tidak sempurna. Pengukuran berikutnya, misalnya tiang yang terlihat dua meter di depan, hanya cocok dengan sebagian puncak lama yang sudah digeser. Puncak-puncak lain mengecil, dan setelah beberapa langkah tinggal satu. Bayangkan berjalan di koridor gelap dengan tangan meraba dinding: setiap langkah menambah keraguan, setiap pintu yang teraba mengurangi keraguan, dan pola jarak antar pintu akhirnya memberi tahu di mana Anda berada.

Dua pendekatan lain bekerja dengan mencocokkan awan titik lidar dengan peta. *Iterative closest point* (Besl and McKay, 1992) bergantian antara dua langkah. Pertama, pasangkan setiap titik pindaian dengan titik peta terdekat, pencarian yang dipercepat dengan pohon k -d. Kedua, cari rotasi dan translasi kaku yang meminimalkan jarak kuadrat pasangan-pasangan itu. Langkah kedua punya solusi tertutup: pusatkan kedua himpunan titik di rata-ratanya, bentuk matriks korelasi silang, lalu ambil rotasi dari dekomposisi nilai singularnya, prosedur yang sama dengan algoritme Kabsch di Bab 24. ICP cepat konvergen kalau tebakan awalnya baik, dan bisa tersesat kalau tidak, karena pasangan terdekat yang salah membawanya ke minimum lokal.

Normal distributions transform (Biber and Strasser, 2004) membagi peta menjadi sel-sel dan mewakili titik-titik di setiap sel dengan sebuah distribusi Gauss berrata-rata μ dan kovarians Σ . Skor sebuah pose adalah jumlah $\exp(-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu))$ untuk semua titik pindaian yang ditransformasi ke peta. Skor ini mulus terhadap pose, sehingga bisa dioptimalkan dengan metode Newton, dengan Hessian yang dijaga tetap definit positif seperti di Bab 4. NDT lebih tahan terhadap tebakan awal yang kurang tepat dan tidak perlu mencari pasangan titik satu per satu.

21.6 Melihat jalan: lidar dan deteksi 3D

Lidar memancarkan pulsa laser dan mengukur waktu tempuhnya, sehingga jarak sama dengan kecepatan cahaya dikali waktu dibagi dua. Pulsa yang kembali setelah

sekitar 667 nanodetik berasal dari benda seratus meter jauhnya. Dengan berkas yang dipindai ke banyak arah, lidar menghasilkan awan titik tiga dimensi dengan jarak yang akurat. Kuliah membahas batasnya. Energi pulsa melemah di udara dan setelah dipantulkan, sehingga jangkauan maksimum lidar umumnya lebih pendek dari radar. Berkas laser juga sedikit menyebar, dan semakin jauh, semakin lebar titik sinarnya. Tetapi penyebaran ini jauh lebih kecil daripada pada radar, sehingga lidar memberi resolusi sudut terbaik di antara sensor waktu tempuh. Kamera memberi warna dan tekstur, dan radar (Bab 18) memberi kecepatan langsung serta tetap bekerja dalam hujan dan kabut. Fusi sensor menggabungkan kelebihan masing-masing.

Mendeteksi objek dalam awan titik punya kesulitan khas: titik-titiknya tidak tersusun dalam grid dan urutannya tidak bermakna. Kuliah membagi pendekatannya menurut representasi data. Metode berbasis titik bekerja langsung pada titik. PointNet (Qi et al., 2016) menerapkan MLP yang sama pada setiap titik lalu menggabungkan hasilnya dengan pooling maksimum, sebuah fungsi yang invarian terhadap permutasi persis seperti DeepSets di Bab 13. Metode berbasis voxel membagi ruang menjadi kubus kecil dan memakai konvolusi tiga dimensi. PointPillars (Lang et al., 2018) mengelompokkan titik ke dalam pilar-pilar vertikal di atas grid tanah, mengubahnya menjadi gambar pandangan atas, lalu memakai jaringan konvolusi dua dimensi yang cepat. Metode berbasis proyeksi menggambar awan titik ke citra pandangan atas atau pandangan depan dan memakai detektor gambar biasa.

21.7 Melacak banyak objek

Deteksi di setiap bingkai belum cukup. Kita perlu tahu bahwa mobil di bingkai ini adalah mobil yang sama dengan di bingkai sebelumnya, dan ke mana ia bergerak. Kalman filter melakukan ini untuk satu objek. Kuliah membangunnya dari contoh termometer: menggabungkan prediksi dengan pengukuran berarti mengalikan dua distribusi Gauss, dan meramal ke depan berarti menambahkan ketidakpastian (Bab 14). Untuk mobil, keadaannya posisi dan kecepatan, model gerak adalah kecepatan konstan, dan pengukuran lidar memberi posisi. Kamera mengukur objek dalam koordinat gambar, sehingga fungsi pengukurannya nonlinear karena proyeksi perspektif. *Extended Kalman filter* melinearkan fungsi itu di sekitar perkiraan saat ini dengan matriks Jacobian, lalu memakai rumus Kalman biasa.

Untuk banyak objek, pelacakan bekerja dalam siklus. Ramalkan semua jejak ke bingkai berikutnya, pasangkan pengukuran dengan jejak, perbarui jejak yang mendapat pasangan, lalu kelola jejak yang tidak. Pemasangan adalah masalah penugasan: diberikan matriks biaya berisi jarak antara setiap ramalan dan setiap deteksi, cari pasangan satu-satu dengan biaya total terkecil. Jaraknya biasanya jarak Mahalanobis, yang memperhitungkan ketidakpastian ramalan, dan pasangan yang terlalu jauh dibuang sejak awal (*gating*). Algoritme Hungaria (Kuhn, 1955) menyelesaikan masalah penugasan secara eksak. Untuk dua jejak dan dua deteksi dengan biaya $\begin{pmatrix} 1 & 4 \\ 3 & 2 \end{pmatrix}$, pasangan lurus berbiaya $1 + 2 = 3$ dan pasangan silang $4 + 3 = 7$, jadi pasangan lurus yang dipilih. Untuk puluhan objek, mencoba semua kemungkinan mustahil, dan algoritme Hungaria

menemukan jawaban yang sama dalam waktu polinomial.

Pengelolaan jejak memberi setiap jejak sebuah skor kepercayaan dan status. Jejak baru dari deteksi yang tidak terpasang mulai sebagai *initialized*, naik menjadi *tentative* lalu *confirmed* setelah terlihat di beberapa bingkai berturut-turut, dan dihapus kalau skornya turun terlalu rendah atau kovariansnya membengkak karena terlalu lama tidak diperbarui. Aturan sederhana ini mencegah satu deteksi palsu melahirkan mobil hantu, dan mencegah satu bingkai yang hilang menghapus mobil sungguhan.

21.8 Di mana machine learning masuk

Robotika klasik dibangun dari model yang diturunkan: kinematika, dinamika, model sensor. Machine learning masuk di tempat model sulit ditulis. Persepsi hampir seluruhnya dikuasai deep learning, dari deteksi objek sampai segmentasi jalan. Dinamika yang sulit dimodelkan, seperti gesekan, kabel, atau benda lunak, bisa dipelajari dari data dan dipakai sebagai koreksi atas model fisika (Bab 32). Policy untuk manipulasi yang rumit dilatih dengan reinforcement learning di simulator lalu dipindahkan ke robot nyata, atau dipelajari dari demonstrasi manusia (Bab 16). Model fondasi yang memadukan penglihatan, bahasa, dan aksi mulai dipakai untuk memberi perintah robot dengan kalimat biasa (Bab 38). Tetapi untuk bagian yang menyangkut keselamatan, seperti lokalisasi, pelacakan, dan kendali tingkat rendah, metode klasik dengan jaminan yang bisa dianalisis tetap menjadi tulang punggung.

21.9 Perkembangan riset

Robotika sedang mencoba mengulang resep model fondasi. Open X-Embodiment (Collaboration et al., 2023) mengumpulkan data dari 22 jenis robot di 21 institusi, mencakup lebih dari 500 keterampilan, dalam format yang seragam, dan memperlihatkan bahwa satu model yang dilatih pada semuanya bisa memperbaiki kemampuan robot-robot tertentu berkat pengalaman robot lain. Diffusion Policy (Chi et al., 2023) memodelkan policy visuomotor sebagai proses denoising bersyarat: aksi robot dibangkitkan seperti gambar dibangkitkan model difusi, dimulai dari noise dan diperhalus langkah demi langkah. Keuntungannya, model bisa mewakili distribusi aksi yang multimodal. Kalau sebuah benda bisa diambil dari kiri atau dari kanan, policy yang dilatih dengan galat kuadrat akan merata-ratakan keduanya dan menabrak benda di tengah, sedangkan model difusi memilih salah satu.

π_0 (Black et al., 2024) menggabungkan model penglihatan dan bahasa yang sudah dilatih dengan kepala aksi berbasis flow matching (Bab 15), dilatih pada data dari robot satu lengan, dua lengan, dan robot bergerak. Contoh tugasnya mencakup melipat pakaian, membersihkan meja, dan merakit kotak. Lapisan-lapisan klasik dari bab ini tetap ada di bawahnya: kendali tingkat rendah, kinematika, dan pengaman, yang mengubah aksi tingkat tinggi menjadi torsi sendi yang aman.

CATATAN SUMBER. Bab ini mengikuti kuliah Introduction to Robotic Systems (transformasi kerangka, kecepatan benda kaku, Denavit–Hartenberg, kinematika maju dan balik, Jacobian dan singularitas, persamaan Lagrange, URDF, visual servoing, medan potensial, profil trayektori)

dan Introduction to Autonomous Vehicles (Markov localization, ICP, NDT, lidar, deteksi objek tiga dimensi, Kalman dan extended Kalman filter, pelacakan banyak objek). Buku rujukan: Lynch and Park (2017) dan Thrun et al. (2005).

Bagian IV

AI untuk Ilmu Hayati

22 Membaca Kode Kehidupan

Pada musim panas 2025 saya membuka pintu ke ilmu hayati lewat beberapa kuliah sekaligus: pengantar genetika, analisis genom dan transkriptomik, bioinformatika struktural, dan AI dalam ilmu hayati. Kuliah genom dimulai dari dasar sekali, dari apa itu kehidupan dan bagaimana DNA disusun, lalu bergerak ke genetika populasi, teknologi microarray, pengurutan DNA generasi demi generasi, pemetaan bacaan, pemanggilan varian, perakitan genom, dan analisis RNA-seq. Bagi pembaca yang datang dari teknik atau ilmu komputer, seperti saya, bab ini adalah pengenalan dengan bahasa yang dipakai bab-bab berikutnya.

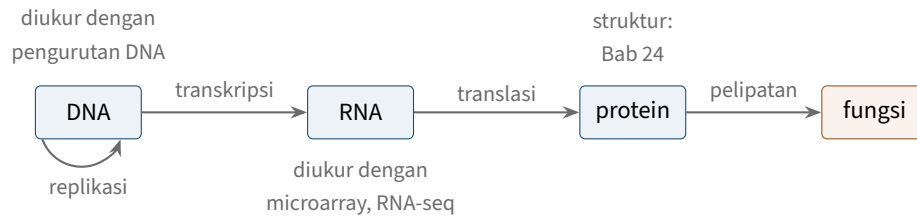
Biologi molekuler modern adalah ilmu data. Satu genom manusia berisi sekitar tiga miliar huruf, satu percobaan pengurutan menghasilkan ratusan juta potongan bacaan, dan satu pengukuran ekspresi gen mencatat aktivitas puluhan ribu gen sekaligus. Hampir setiap langkah dari sampel ke kesimpulan melibatkan algoritme, statistik, dan semakin sering machine learning.

22.1 Dogma sentral

Informasi genetik tersimpan dalam DNA, rantai ganda yang tersusun dari empat basa: adenin, sitosin, guanin, dan timin. Basa pada satu untai berpasangan dengan basa pada untai lain, A dengan T dan C dengan G, sehingga setiap untai menyimpan salinan lengkap informasi untai pasangannya. Struktur heliks ganda ini diusulkan Watson dan Crick pada 1953, dengan bantuan penting dari foto difraksi sinar-X yang dibuat Rosalind Franklin, sebuah kisah yang juga disinggung di kuliah. Kedua untai punya arah, dari ujung 5' ke ujung 3', dan keduanya berjalan berlawanan arah. DNA sepanjang lebih dari dua meter di setiap sel manusia dilipat berlapis-lapis di sekitar protein histon sampai muat di dalam inti sel yang diameternya beberapa mikrometer.

Dogma sentral biologi molekuler menggambarkan aliran informasi (Gambar 22.1). DNA disalin menjadi RNA dalam proses transkripsi, lalu RNA pembawa pesan diterjemahkan menjadi protein di ribosom. Setiap tiga basa, satu kodon, menyandikan satu asam amino. Ada $4^3 = 64$ kodon untuk dua puluh asam amino dan sinyal berhenti, sehingga kodenya berlebih: beberapa kodon berbeda menyandikan asam amino yang sama. Pada sel eukariot, bagian-bagian RNA yang disebut intron dipotong dan bagian ekson disambung ulang sebelum diterjemahkan, dan penyambungan alternatif memungkinkan satu gen menghasilkan beberapa protein berbeda. Protein lalu melipat menjadi bentuk tiga dimensi yang menentukan fungsinya, topik Bab 24.

Bayangkan perpustakaan besar yang buku-bukunya tidak boleh dibawa keluar. Untuk membangun sesuatu, seseorang memfotokopi satu halaman resep (transkripsi), membawa fotokopinya ke dapur (ribosom), lalu memasak sesuai resep itu (translasi).



GAMBAR 22.1. Dogma sentral biologi molekuler dan teknologi pengukuran yang dibahas di bab ini.

Sel yang berbeda di tubuh kita membawa perpustakaan yang sama, tetapi memfotokopi halaman yang berbeda. Itulah sebabnya sel otot dan sel saraf begitu berbeda meski DNA-nya identik, dan itulah sebabnya mengukur RNA, yaitu fotokopi yang sedang beredar, memberi tahu apa yang sedang dikerjakan sebuah sel.

22.2 Variasi dan genetika populasi

Sebelum DNA dikenal, Mendel sudah menemukan aturan pewarisannya dari kacang kapri. Setiap individu membawa dua salinan setiap gen, satu dari masing-masing induk, dan setiap induk mewariskan salah satu salinannya secara acak. Kalau dua induk sama-sama heterozigot, $Aa \times Aa$, kombinasi anaknya adalah AA , Aa , aA , dan aa dengan peluang sama, sehingga perbandingan genotipenya 1 : 2 : 1. Kalau A dominan, tiga dari empat anak memperlihatkan sifat A , perbandingan 3 : 1 yang terkenal. Itulah kasus khusus hukum Hardy–Weinberg di bawah dengan $p = q = \frac{1}{2}$.

Dua manusia berbeda di sekitar satu dari setiap seribu basa. Variasi yang paling umum adalah *single nucleotide polymorphism* (SNP), satu posisi tempat sebagian orang punya satu basa dan sebagian lain punya basa berbeda. Ada juga sisipan dan hapusan pendek, serta variasi struktural yang lebih besar.

Genetika populasi mempelajari bagaimana frekuensi varian berubah dari generasi ke generasi. Hukum Hardy–Weinberg turun dari satu asumsi: dalam populasi besar yang kawin acak tanpa seleksi, setiap individu menerima satu alel dari masing-masing orang tua secara acak dan independen. Kalau frekuensi dua alel adalah p dan $q = 1 - p$, peluang menerima dua alel pertama adalah $p \cdot p$, dua alel kedua $q \cdot q$, dan satu dari masing-masing $2pq$, karena ada dua urutan. Maka frekuensi genotipenya p^2 , $2pq$, dan q^2 , dan jumlahnya $(p + q)^2 = 1$. Dengan $p = 0,7$, sekitar 49 persen individu homozigot untuk alel pertama, 42 persen heterozigot, dan 9 persen homozigot untuk alel kedua. Penyimpangan dari proporsi ini menjadi petunjuk adanya seleksi, migrasi, perkawinan sekerabat, atau sekadar kesalahan pengukuran, dan karena itu pengujian Hardy–Weinberg dipakai sebagai kendali mutu data genotipe. Dalam populasi kecil, frekuensi alel juga bergeser secara acak dari generasi ke generasi, sebuah gejala yang disebut *genetic drift*.

Varian yang letaknya berdekatan di kromosom cenderung diwariskan bersama, karena jarang terpisah oleh rekombinasi. Keadaan ini disebut *linkage disequilibrium*. Studi asosiasi seluruh genom (GWAS) memanfaatkannya: mereka mengukur ratusan

ribu SNP pada ribuan orang dan mencari varian yang frekuensinya berbeda antara penderita dan bukan penderita sebuah penyakit. SNP yang terukur tidak harus penyebabnya. Cukup kalau ia berdekatan dengan penyebabnya dan ikut diwariskan bersama. Karena jumlah uji statistiknya banyak sekali, ambang signifikansinya harus ketat sekali, kalau tidak, ribuan temuan palsu akan muncul hanya karena kebetulan.

22.3 Mengukur ekspresi dengan microarray

Microarray DNA adalah kaca kecil yang ditemplei jutaan potongan DNA pendek yang dikenal, yang disebut probe. RNA dari sampel diubah menjadi DNA berlabel fluoresen, lalu dibiarkan menempel pada probe yang cocok. Semakin terang sebuah titik, semakin banyak RNA dari gen itu. Desain klasik Affymetrix memasang setiap probe dua kali: sekali persis cocok, sekali dengan satu basa di tengah sengaja diubah, supaya penempelan yang tidak spesifik bisa diperkirakan. Banyak probe untuk gen yang sama dikumpulkan dalam satu *probe set*.

Jalan dari gambar ke angka ternyata panjang, dan kuliah membahasnya langkah demi langkah. Pertama, koreksi latar belakang membuang cahaya yang bukan berasal dari penempelan spesifik. Kedua, normalisasi membuat chip yang berbeda bisa dibandingkan. Perbedaannya bisa dilihat dengan plot MvA: untuk dua chip, selisih log-intensitas M diplot terhadap rata-ratanya A . Kalau tidak ada pergeseran sistematis, titik-titik berkumpul di sekitar garis nol.

Salah satu metode normalisasi yang dibahas adalah normalisasi kuantil, yang memaksa semua chip punya distribusi yang persis sama. Caranya sederhana. Urutkan nilai di setiap chip, ganti setiap peringkat dengan rata-rata nilai pada peringkat itu di semua chip, lalu kembalikan ke urutan asal. Misalkan tiga gen pada dua chip bernilai (5, 2, 3) dan (4, 1, 6). Setelah diurutkan, chip pertama menjadi (2, 3, 5) dan chip kedua (1, 4, 6). Rata-rata per peringkat adalah (1,5; 3,5; 5,5). Gen pertama berperingkat tiga di chip pertama dan peringkat dua di chip kedua, sehingga nilai barunya 5,5 dan 3,5. Setelah semua dikembalikan, chip pertama menjadi (5,5; 1,5; 3,5) dan chip kedua (3,5; 1,5; 5,5). Peringkat di dalam setiap chip tetap, distribusinya kini identik.

Ketiga, ringkasan menggabungkan banyak probe menjadi satu nilai ekspresi per gen. Kuliah membahas RMA, yang memakai rata-rata yang tahan pencilan, dan FARMS, metode berbasis analisis faktor (Bab 6) dari kelompok Sepp Hochreiter. FARMS memperlakukan sinyal bersama antar probe sebagai faktor laten dan noise pribadi setiap probe sebagai noise analisis faktor, dan sekaligus bisa menandai probe set yang tidak membawa sinyal sama sekali.

Kuliah juga menunjukkan bagaimana data ekspresi dipakai dalam skala besar. Pengukuran yang lebih murah, seperti assay L1000 yang hanya mengukur sekitar seribu gen penanda, bisa dipakai untuk merekonstruksi profil transkriptom lengkap dengan model statistik. Basis data seperti Connectivity Map mengumpulkan profil ekspresi sel setelah diberi ribuan senyawa, sehingga senyawa yang “membalik” pola ekspresi sebuah penyakit bisa dicari, sebuah jembatan ke penemuan obat di Bab 23.

22.4 Pengurutan DNA

Pengurutan Sanger, generasi pertama, membaca satu potongan DNA sepanjang beberapa ratus basa dengan akurasi tinggi, dan dipakai dalam Proyek Genom Manusia. Generasi kedua, yang dipelopori platform Illumina, membaca ratusan juta potongan pendek secara paralel, sering berpasangan dari kedua ujung sebuah fragmen. Generasi ketiga dan keempat membaca satu molekul DNA secara langsung, dengan bacaan yang jauh lebih panjang, dari puluhan ribu sampai jutaan basa. Pada pengurutan nanopore, DNA ditarik melewati pori kecil, dan setiap kombinasi basa di dalam pori mengubah arus listrik dengan cara yang khas.

Cara kerja pengurutan Illumina memperlihatkan dari mana galat dan skor kualitas berasal. Fragmen DNA ditempelkan pada permukaan kaca, lalu diperbanyak di tempat dengan *bridge amplification*, sehingga setiap fragmen membentuk gugus berisi ribuan salinan yang identik. Pengurutan berjalan dalam siklus. Di setiap siklus, polimerase menambahkan tepat satu nukleotida berlabel fluoresen ke setiap salinan, kamera memotret warna setiap gugus, lalu labelnya dihapus untuk siklus berikutnya. Panjang bacaan sama dengan jumlah siklus. Seiring siklus berjalan, sebagian salinan dalam satu gugus tertinggal atau mendahului yang lain, sehingga sinyal warnanya semakin bercampur. Karena itulah kualitas basa turun menjelang ujung bacaan, pola yang selalu terlihat pada laporan kendali mutu.

Mengubah sinyal mentah menjadi urutan basa disebut *base calling*. Metode PHRED klasik untuk pengurutan Sanger memperkirakan letak puncak sinyal, mencocokkannya dengan pemrograman dinamis, dan memberi setiap basa skor kualitas

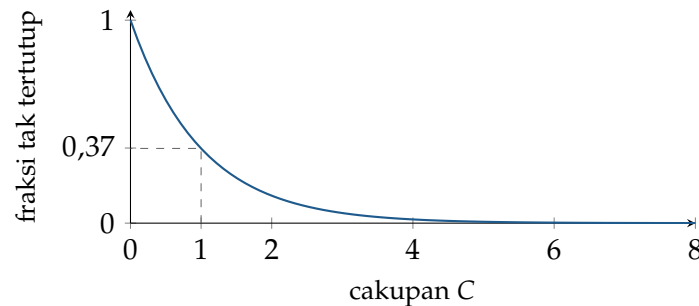
$$Q = -10 \log_{10} P_{\text{galat}}. \quad (22.1)$$

Skor 20 berarti peluang salah satu persen, skor 30 satu per seribu, skor 40 satu per sepuluh ribu. Untuk nanopore, base calling dikerjakan jaringan saraf rekuren dan konvolusi seperti CHIRON. Panjang keluaran tidak diketahui sebelumnya, karena sebuah basa bisa bertahan di pori selama beberapa sampel atau hanya satu, sehingga jaringan dilatih dengan loss *connectionist temporal classification* (CTC), yang menjumlahkan peluang semua cara menyelaraskan keluaran dengan sinyal. Loss yang sama dipakai pengenalan suara.

Bacaan disimpan dalam berkas FASTQ, yang menyimpan setiap bacaan bersama skor kualitas setiap basanya, dikodekan sebagai karakter ASCII. Pada pengodean yang umum, karakter dengan kode $Q + 33$ mewakili skor Q , sehingga huruf I (kode 73) berarti skor 40. Alat kendali mutu seperti FastQC memeriksa distribusi skor di sepanjang bacaan, kandungan GC, dan bacaan duplikat sebelum analisis dimulai.

Berapa banyak bacaan yang dibutuhkan? Kuliah mendefinisikan cakupan $C = RM/T$, dengan R panjang bacaan, M jumlah bacaan, dan T panjang genom: rata-rata berapa kali setiap basa terbaca. Untuk genom manusia sepanjang tiga miliar basa, cakupan tiga puluh kali dengan bacaan 150 basa membutuhkan sekitar 600 juta bacaan. Kalau bacaan jatuh secara acak, jumlah bacaan yang menutupi sebuah basa mendekati distribusi Poisson dengan rata-rata C , sehingga peluang sebuah basa tidak tertutup sama sekali adalah e^{-C} (Gambar 22.2). Dengan cakupan satu kali, sekitar 37 persen genom terlewat.

Dengan tiga puluh kali, peluang itu praktis nol, dan setiap basa terbaca cukup sering untuk membedakan varian sungguhan dari galat pengurutan.



GAMBAR 22.2. Fraksi genom yang tidak tertutup satu bacaan pun, e^{-C} , menurut model Poisson.

Kendali mutu juga memeriksa duplikat. Amplifikasi dengan PCR saat menyiapkan sampel bisa membuat satu fragmen asli tercatat berkali-kali. Duplikat ini bukan pengukuran yang independen: galat pengurutan pada satu salinan ikut tersalin ke semua duplikatnya. Karena itu duplikat biasanya ditandai dan hanya dihitung sekali sebelum varian dipanggil.

22.5 Dari bacaan ke genom

Kalau genom acuan sudah ada, setiap bacaan dipetakan ke posisinya di acuan. Mencocokkan ratusan juta bacaan pendek ke tiga miliar huruf dengan toleransi terhadap galat membutuhkan indeks yang cerdas. Kuliah membahas tiga jenis: tabel hash untuk potongan pendek, pohon sufiks, dan transformasi Burrows–Wheeler (BWT) (Li and Durbin, 2009). BWT dibangun dengan menulis semua rotasi sebuah teks, mengurutkannya, lalu mengambil kolom terakhir. Untuk `banana$`, ketujuh rotasi yang diurutkan berakhir dengan huruf-huruf `a, n, n, b, $, a, a`, sehingga BWT-nya `annb$aa`. Huruf-huruf yang sama cenderung berkumpul, sehingga hasilnya mudah dimampatkan, dan dengan sedikit informasi tambahan, transformasi ini bisa dibalik dan dipakai untuk mencari pola dalam waktu yang hanya bergantung pada panjang pola, bukan panjang genom.

Strategi pemetaan umumnya adalah *seed and extend*: cari potongan pendek yang cocok persis, lalu perluas dengan penjajaran yang toleran terhadap galat. Penjajaran itu diselesaikan dengan pemrograman dinamis, gagasan Bellman yang sama dengan di Bab 5. Skor terbaik untuk menjajarkan i huruf pertama bacaan dengan j huruf pertama acuan adalah

$$S(i, j) = \max \{S(i-1, j-1) + s(a_i, b_j), S(i-1, j) - g, S(i, j-1) - g\}, \quad (22.2)$$

dengan s skor cocok atau tidak cocok dan g penalti celah. Ketiga pilihan berarti: pasangkan kedua huruf, sisipkan celah di acuan, atau sisipkan celah di bacaan. Algoritme Smith–Waterman menambahkan pilihan nol agar penjajaran lokal bisa dimulai di mana saja. Untuk bacaan panjang, potongan-potongan yang cocok dirangkai dulu menjadi rantai yang konsisten sebelum penjajaran rinci, supaya pemrograman dinamis yang mahal hanya dijalankan di daerah yang menjanjikan.

Setelah bacaan dipetakan, *variant calling* memutuskan di posisi mana sampel berbeda dari acuan. Tantangannya membedakan varian sungguhan dari galat pengurutan dan galat pemetaan. Kuliah membandingkan beberapa alat, dari yang berbasis model statistik seperti GATK dan FreeBayes sampai yang berbasis deep learning. DeepVariant (Poplin et al., 2018) mengubah tumpukan bacaan di sekitar sebuah posisi menjadi gambar dan memakai jaringan konvolusi untuk mengklasifikasikan genotipenya, sebuah contoh menarik dari visi komputer yang dipinjam oleh genomik.

Logika dasar pemanggilan varian adalah teorema Bayes dengan likelihood binomial. Misalkan di sebuah posisi ada 30 bacaan, dan 15 di antaranya membawa basa alternatif. Kalau individu itu heterozigot, setiap bacaan membawa basa alternatif dengan peluang separuh, dan peluang mendapat tepat 15 dari 30 adalah $\binom{30}{15} 0,5^{30} \approx 0,14$. Kalau ia homozigot untuk basa acuan, basa alternatif hanya bisa muncul karena galat pengurutan, katakanlah dengan peluang satu persen per bacaan, dan peluang mendapat 15 galat dari 30 adalah $\binom{30}{15} 0,01^{15} 0,99^{15} \approx 10^{-22}$. Rasio keduanya begitu besar sehingga prior apa pun yang masuk akal tidak akan membalik kesimpulannya. Kasus sulit muncul ketika cakupannya rendah, misalnya 2 dari 4 bacaan, atau ketika galat pemetaan dan duplikat membuat bacaan-bacaan itu tidak independen. Di situlah model yang lebih teliti, termasuk jaringan saraf seperti DeepVariant, memberi perbedaan. Frekuensi alel varian, $15/30 = 0,5$ dalam contoh ini, juga dipakai untuk mendeteksi mutasi yang hanya ada di sebagian sel, misalnya pada tumor, di mana frekuensinya bisa jauh di bawah separuh.

Kalau tidak ada acuan, genom harus dirakit dari nol, seperti menyusun teka-teki jigsaw raksasa dari potongan-potongan yang saling tumpang tindih. Pendekatan *overlap-layout-consensus* membangun graf yang simpulnya bacaan dan sisinya tumpang tindih, lalu mencari jalur yang melewati setiap simpul tepat sekali, sebuah jalur Hamilton yang sulit dicari secara umum. Pendekatan graf de Bruijn membalik sudut pandangnya (Gambar 22.3). Bacaan dipotong menjadi semua kata sepanjang k , yang disebut k -mer. Setiap k -mer menjadi sisi yang menghubungkan awalan dan akhirnya sepanjang $k - 1$. Untuk urutan ATGGCGT dengan $k = 3$, kata-katanya ATG, TGG, GGC, GCG, dan CGT, dan rakitan menjadi jalur yang melewati setiap sisi tepat sekali, sebuah jalur Euler yang bisa dicari dengan efisien. Perbedaan antara jalur Hamilton dan jalur Euler inilah yang membuat graf de Bruijn praktis untuk jutaan bacaan pendek.



GAMBAR 22.3. Graf de Bruijn untuk ATGGCGT dengan $k = 3$. Simpul adalah kata sepanjang dua, sisi adalah kata sepanjang tiga. Jalur yang melewati setiap sisi sekali mengeja kembali urutannya.

Bagian yang berulang di genom adalah musuh utama kedua pendekatan, karena potongan dari dua tempat berbeda tampak identik dan graf menjadi bercabang atau runtuh menjadi satu. Galat pengurutan menambah cabang-cabang palsu yang harus dipangkas. Bacaan panjang membantu justru karena mereka bisa melompati pengulangan. Kuliah menutup bagian ini dengan proyek-proyek tonggak, dari Proyek Genom

Manusia dan proyek 1000 genom sampai genom Neanderthal, dan menunjukkan bahwa jaringan graf (Bab 13) mulai dipakai untuk menyederhanakan graf rakitan.

22.6 RNA-seq

Pengurutan juga bisa mengukur ekspresi gen. RNA diubah menjadi DNA, diurutkan, lalu jumlah bacaan yang jatuh pada setiap gen dihitung. Hitungan ini harus dinormalisasi, karena sampel yang berbeda diurutkan dengan kedalaman berbeda dan gen yang panjang menghasilkan lebih banyak bacaan. Ukuran seperti TPM membagi hitungan dengan panjang gen dan dengan total, supaya ekspresi antar gen dan antar sampel bisa dibandingkan.

Untuk mencari gen yang ekspresinya berubah antara dua kondisi, DESeq2 (Love et al., 2014) memodelkan hitungan dengan distribusi binomial negatif. Mengapa bukan Poisson? Pada distribusi Poisson, variansnya sama dengan rata-ratanya. Data biologis selalu lebih berisik dari itu, karena selain noise penghitungan, ada variasi biologis sungguhan antar individu. Binomial negatif menambahkan suku dispersi, $\text{Var} = \mu + \alpha\mu^2$. Untuk gen dengan rata-rata 1000 bacaan dan dispersi $\alpha = 0,04$, simpangan bakunya $\sqrt{1000 + 40\,000} \approx 202$, jauh lebih besar dari $\sqrt{1000} \approx 32$ yang diramalkan Poisson. Mengabaikan dispersi ini membuat hampir setiap gen tampak berubah secara signifikan.

Karena setiap gen hanya punya sedikit ulangan, perkiraan dispersi untuk satu gen tidak pasti. DESeq2 meminjam kekuatan dari ribuan gen lain: dispersi setiap gen ditarik ke arah tren bersama yang bergantung pada rata-rata ekspresi, dan perkiraan perubahan lipat yang berisik untuk gen berekspresi rendah juga ditarik ke arah nol. Inilah prior Bayesian empiris, prior yang dipelajari dari data itu sendiri, logika yang sama dengan regularisasi di Bab 1. Dengan puluhan ribu gen yang diuji sekaligus, nilai p dikoreksi untuk mengendalikan proporsi temuan palsu.

22.7 Di mana machine learning masuk

Genomik adalah salah satu medan tempat machine learning paling banyak dipakai di ilmu hayati. Jaringan saraf memanggil basa dari sinyal nanopore, memanggil varian dari tumpukan bacaan, dan memprediksi dari urutan DNA saja di mana protein akan menempel atau seberapa kuat sebuah gen diekspresikan. Model bahasa yang dilatih pada urutan DNA dan protein meniru resep model bahasa manusia di Bab 9. Metode tanpa label seperti biclustering dan analisis faktor (Bab 6) menemukan modul gen yang bekerja bersama dalam data ekspresi.

Karakter datanya menuntut kehati-hatian khusus. Jumlah fitur bisa puluhan ribu dengan jumlah sampel puluhan, sehingga overfitting mengintai di setiap sudut. Batch percobaan, yaitu hari pengukuran, mesin, atau laboratorium, meninggalkan jejak sistematis yang bisa dipelajari model sebagai pengganti biologi yang sebenarnya. Dan sampel dari pasien atau keluarga yang sama berkorelasi. Semua ini adalah sumber kebocoran yang dibahas di Bab 1, dalam bentuk yang khas biologi.

22.8 Perkembangan riset

Genomik memasuki era model fondasi. Enformer (Avsec et al., 2021) memprediksi ekspresi gen langsung dari urutan DNA dengan arsitektur yang menggabungkan konvolusi dan attention, sehingga bisa memadukan informasi dari elemen pengatur yang letaknya sampai sekitar 100 ribu basa dari gen. Dengan model seperti ini, efek sebuah varian pada ekspresi gen bisa ditebak tanpa percobaan. DNABERT-2 (Zhou et al., 2023b) memperlihatkan bahwa pilihan token penting juga untuk DNA: mengganti *k*-mer dengan byte-pair encoding, algoritme yang sama dengan tokenisasi model bahasa di Bab 38, membuat model lebih efisien. Evo (Nguyen et al., 2024) adalah model berkonteks panjang yang dilatih pada jutaan genom prokariota dan fag, dan melaporkan hukum penskalaan untuk DNA. Ia mampu memprediksi fungsi tanpa contoh berlabel dan membangkitkan sistem CRISPR-Cas yang berfungsi.

AlphaMissense (Cheng et al., 2023) menyetel AlphaFold pada data frekuensi varian di populasi manusia dan primata untuk menilai apakah sebuah varian missense, yaitu perubahan satu asam amino, kemungkinan berbahaya. Basis data prediksinya mencakup semua substitusi asam amino tunggal yang mungkin pada protein manusia, dan mengklasifikasikan 89 persen varian missense sebagai kemungkinan jinak atau kemungkinan patogen. Untuk data sel tunggal, scGPT (Cui et al., 2024) melatih transformer generatif pada profil ekspresi jutaan sel sebagai model fondasi yang bisa disetel untuk tugas seperti anotasi jenis sel dan prediksi efek gangguan genetik. Di semua contoh ini, peringatan dari bab ini tetap berlaku: evaluasi yang jujur menuntut pembagian data yang mencegah kebocoran antar kerabat urutan.

CATATAN SUMBER. Bab ini mengikuti kuliah Genome Analysis and Transcriptomics (dasar biologi molekuler, genetika populasi, microarray dan pemrosesannya, pengurutan DNA dan base calling, pemetaan bacaan, pemanggilan varian, perakitan genom, RNA-seq) dan Introduction to Genetics. Untuk analisis urutan secara probabilistik, Durbin et al. (1998) tetap menjadi rujukan klasik.

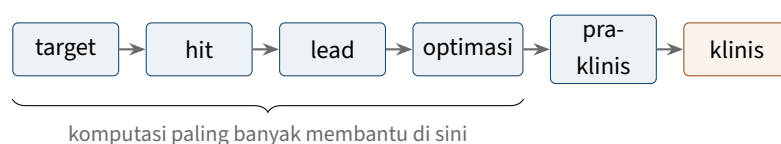
23 Molekul, Obat, dan Data Medis

Kuliah AI in life sciences saya ikuti pada musim panas 2025. Materinya luas: penemuan obat dan representasi molekul, QSAR dan virtual screening, geometric deep learning untuk molekul, model multimodal, model generatif, data medis, sampai reaksi kimia dan perencanaan sintesis. Banyak metode yang dibahas lahir di Linz sendiri, dari DeepTox sampai jaringan Hopfield modern untuk kimia, sehingga kuliahnya banyak memperlihatkan hasil riset dari kelompok-kelompok di kampus yang sama.

Menemukan obat baru memakan waktu lebih dari satu dekade dan biaya yang besar sekali, dan sebagian besar kandidat gagal di tengah jalan. Machine learning tidak bisa menggantikan uji klinis, tetapi bisa membuat langkah-langkah awal jauh lebih murah, misalnya dengan menyaring jutaan molekul di komputer sebelum satu pun disintesis, dan bisa membantu membaca data pasien yang menumpuk di rumah sakit.

23.1 Jalan panjang sebuah obat

Pipeline penemuan obat dimulai dengan memilih *target* biologis, biasanya sebuah protein yang terlibat dalam penyakit (Gambar 23.1). Lalu dicari *hit*, senyawa yang memengaruhi target itu dalam percobaan. Hit yang menjanjikan dikembangkan menjadi *lead*, dan lead dioptimalkan sifat-sifatnya: kekuatan, selektivitas terhadap target yang benar, keamanan, dan kemampuan sampai ke tempat kerjanya di tubuh. Setelah itu datang uji praklinis pada sel dan hewan, lalu uji klinis pada manusia dalam beberapa fase, dari keamanan pada sukarelawan sehat sampai efektivitas pada ribuan pasien. Setiap tahap menyaring sebagian besar kandidat, sehingga dari ribuan senyawa awal, hanya segelintir yang sampai ke apotek. Kegagalan di tahap akhir paling mahal, sehingga setiap metode yang bisa memprediksi kegagalan lebih awal bernilai besar.



GAMBAR 23.1. Pipeline penemuan obat yang disederhanakan. Model komputasi paling banyak dipakai di tahap-tahap awal, untuk memilih senyawa yang layak diuji.

23.2 Bagaimana mesin melihat molekul

Molekul organik kecil adalah atom-atom yang dihubungkan ikatan kimia. Kuliah mengulang kimia dasar yang dibutuhkan: unsur yang paling banyak di tubuh manusia, ikatan kovalen dan gaya antarmolekul, konformasi tiga dimensi, dan kiralitas, ketika

dua molekul adalah bayangan cermin satu sama lain dan bisa punya efek biologis yang berbeda.

Representasi paling alami adalah graf: atom sebagai simpul, ikatan sebagai sisi, masing-masing dengan atribut seperti jenis unsur, muatan, dan orde ikatan. Untuk disimpan dan dicari di basis data, graf ini ditulis sebagai string. SMILES (Weininger, 1988) berjalan menyusuri graf dan mencatat atom yang dilewati: etanol ditulis CCO, benzena c1ccccc1, dengan angka yang menandai titik pembukaan dan penutupan cincin. Satu molekul bisa punya banyak SMILES yang sah, tergantung dari atom mana penyusunan dimulai, sehingga dibutuhkan bentuk kanonik untuk membandingkan. SELFIES dirancang agar setiap string yang mungkin selalu mewakili molekul yang sah, sifat yang berguna untuk model generatif. InChI dan InChIKey memberi pengenalan unik yang bisa dipakai sebagai kunci basis data.

Untuk machine learning klasik, molekul diubah menjadi vektor fitur. *Molecular descriptor* adalah besaran yang bisa dihitung, seperti berat molekul, jumlah donor ikatan hidrogen, atau koefisien partisi oktanol-air ($\log P$) yang mengukur seberapa “berminyak” sebuah molekul. *Molecular fingerprint* adalah vektor biner panjang yang menandai keberadaan pola-pola substruktur. Extended connectivity fingerprint (Rogers and Hahn, 2010) dibangun dengan cara yang sangat mirip uji Weisfeiler–Leman di Bab 13. Setiap atom mulai dengan pengenalan dari sifat-sifatnya sendiri. Di setiap iterasi, pengenalan atom diperbarui dengan meng-*hash* pengenalnya bersama pengenalan tetangga-tetangganya, sehingga setelah dua atau tiga iterasi, setiap pengenalan mewakili lingkungan berjari-jari dua atau tiga ikatan. Semua pengenalan kemudian dilipat ke dalam vektor berukuran tetap, misalnya 2048 bit. ECFP adalah message passing tanpa parameter yang dipelajari.

Kemiripan dua molekul diukur dengan koefisien Tanimoto pada fingerprint-nya, rasio irisan terhadap gabungan,

$$T(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{c}{a + b - c}, \quad (23.1)$$

dengan a dan b jumlah bit yang menyala pada masing-masing fingerprint dan c jumlah bit yang menyala pada keduanya. Bentuk kedua didapat karena $|A \cup B| = |A| + |B| - |A \cap B|$: bit yang ada di keduanya terhitung dua kali dalam $a + b$, sehingga harus dikurangi sekali. Kalau molekul A punya 20 bit menyala, B punya 18, dan 12 di antaranya sama, kemiripannya $12/(20 + 18 - 12) = 12/26 \approx 0,46$. Dalam praktik, nilai di atas sekitar 0,7 untuk ECFP sering dianggap tanda dua molekul cukup mirip untuk kemungkinan punya aktivitas serupa, tetapi ambang ini bergantung pada jenis fingerprint. Ruang molekul mirip obat yang mungkin dibuat diperkirakan amat besar, jauh melebihi jumlah molekul yang pernah disintesis manusia, sehingga pencarian di ruang ini adalah masalah eksplorasi yang sesungguhnya.

23.3 Aktivitas biologis

Seberapa kuat sebuah senyawa memengaruhi targetnya diukur dalam percobaan yang disebut *assay*, di dalam tabung tanpa sel atau dengan sel hidup. Pada konsentrasi yang semakin tinggi, efeknya biasanya naik mengikuti kurva berbentuk S. Bentuk ini bisa

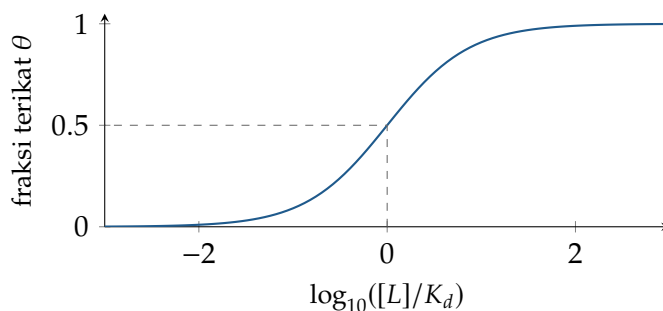
diturunkan dari kesetimbangan pengikatan yang paling sederhana. Misalkan reseptor R dan ligan L membentuk kompleks RL dengan konstanta disosiasi

$$K_d = \frac{[R][L]}{[RL]}. \quad (23.2)$$

Jumlah reseptor total tetap, $[R]_{\text{tot}} = [R] + [RL]$. Fraksi reseptor yang terikat adalah $\theta = [RL]/[R]_{\text{tot}}$. Dari definisi K_d , $[R] = K_d[RL]/[L]$, sehingga

$$\theta = \frac{[RL]}{K_d[RL]/[L] + [RL]} = \frac{[L]}{K_d + [L]}. \quad (23.3)$$

Ketika konsentrasi ligan sama dengan K_d , tepat separuh reseptor terikat. Pada skala logaritmik konsentrasi, fungsi ini menjadi kurva S yang terkenal (Gambar 23.2). Untuk penghambatan, konsentrasi yang memberi separuh efek maksimum disebut IC_{50} , dan untuk pemodelan biasanya dipakai $pIC_{50} = -\log_{10}(IC_{50} \text{ dalam molar})$. Senyawa dengan IC_{50} 100 nanomolar punya pIC_{50} sebesar 7, dan setiap kenaikan satu satuan berarti senyawa sepuluh kali lebih kuat. Skala logaritmik ini membuat target regresi lebih mendekati distribusi normal, sesuai asumsi noise Gauss dari Bab 1.



GAMBAR 23.2. Kurva pengikatan $\theta = [L]/(K_d + [L])$ pada skala logaritmik. Separuh reseptor terikat ketika $[L] = K_d$, dan hampir seluruh peralihan terjadi dalam rentang dua orde besaran.

Obat tidak cukup kuat, ia juga harus sampai ke tempatnya. Sifat *ADME*, yaitu penyerapan, distribusi, metabolisme, dan ekskresi, menentukan hal ini. Aturan lima Lipinski (Lipinski et al., 1997) adalah heuristik terkenal untuk obat yang diminum: berat molekul tidak lebih dari 500 dalton, $\log P$ tidak lebih dari 5, paling banyak 5 donor dan 10 akseptor ikatan hidrogen. Aturan ini dirumuskan dari pengamatan pada obat-obat yang sudah berhasil, bukan dari hukum fisika, dan banyak pengecualiannya.

Data aktivitas dikumpulkan dalam basis data publik seperti ChEMBL (Gaulton et al., 2012). Kalau disusun sebagai matriks senyawa terhadap target, matriks ini jarang sekali: sebagian besar pasangan tidak pernah diukur. Datanya juga tidak seimbang, karena senyawa aktif jauh lebih sedikit dari yang tidak aktif, dan berisik, karena laboratorium berbeda memberi angka berbeda untuk pengukuran yang sama.

23.4 QSAR dan virtual screening

Quantitative structure–activity relationship (QSAR) adalah nama lama untuk apa yang kini kita sebut supervised learning pada molekul: memprediksi aktivitas dari struktur.

Kuliah mencatat bahwa gagasannya sudah berumur lebih dari setengah abad, dimulai dari regresi linear pada beberapa deskriptor. *Virtual screening* memakai model seperti ini, atau sekadar pencarian kemiripan, untuk menyaring pustaka berisi jutaan sampai miliaran senyawa dan memilih segelintir yang layak diuji di laboratorium. Pendekatan berbasis ligan hanya memakai senyawa aktif yang sudah diketahui. Pendekatan berbasis struktur memakai bentuk tiga dimensi protein target untuk menilai seberapa baik sebuah senyawa bisa menempel. Sebelum dan sesudahnya ada penyaringan: senyawa dengan substruktur yang dikenal bermasalah dibuang, dan pengelompokan dipakai supaya senyawa yang dipilih beragam, bukan seratus varian dari satu kerangka.

Kuliah memberi penekanan besar pada cara membagi data. Ahli kimia bekerja dalam seri senyawa, yaitu banyak varian kecil dari satu kerangka yang sama, yang sering dirangkum sebagai kerangka Murcko. Kalau data dibagi acak, varian dari seri yang sama masuk ke data latih dan data uji sekaligus, dan model tampak hebat karena pada dasarnya hanya mengenali saudara dekat. Pembagian berdasarkan kerangka atau berdasarkan klaster memaksa model diuji pada kimia yang benar-benar baru, dan skornya biasanya turun drastis. Bias ini disebut *compound series bias*. Pada model multitask, kebocoran bisa terjadi lewat tugas lain yang mengukur senyawa yang sama. Validasi silang bersarang dipakai agar pemilihan hyperparameter tidak ikut membocorkan data uji: lingkaran dalam memilih hyperparameter, lingkaran luar mengukur kinerja.

Metrik evaluasi juga harus dipilih dengan hati-hati. Untuk data yang tidak seimbang, luas di bawah kurva ROC bisa tampak bagus walau model praktis tidak berguna, karena kurva ROC tidak peka terhadap banyaknya negatif palsu. Luas di bawah kurva presisi–recall lebih jujur dalam situasi ini. Misalkan dari 10 000 senyawa hanya 100 yang aktif, dan model menempatkan 50 senyawa aktif di antara 500 senyawa teratasnya. Recall pada titik itu 50 persen, tetapi presisinya hanya 10 persen, sembilan dari sepuluh senyawa yang dipilih untuk diuji ternyata tidak aktif. Untuk virtual screening, yang dipedulikan justru presisi di daftar teratas, karena hanya daftar itulah yang akan diuji di laboratorium.

DeepTox (Mayr et al., 2016), dari kelompok di Linz, menunjukkan kekuatan jaringan saraf multitask untuk memprediksi toksisitas. Banyak tugas toksisitas berbagi mekanisme, sehingga satu jaringan yang belajar semuanya bersama-sama membangun representasi yang lebih baik daripada model terpisah untuk setiap tugas.

23.5 Target, sinergi, dan pasangan protein–senyawa

Kuliah membedakan dua arah pertanyaan. *Target prediction* bertanya: untuk senyawa ini, target mana yang mungkin dipengaruhi? Pertanyaan ini penting untuk memperkirakan efek samping. *Target identification* bertanya kebalikannya: senyawa ini menghasilkan efek yang menarik pada sel, protein apa yang sebenarnya ia ikat? Model *proteochemometrics* menjawab keduanya dengan memasukkan representasi senyawa dan representasi protein sekaligus, sehingga bisa memprediksi pasangan yang belum pernah diukur.

Obat juga sering diberikan berpasangan, terutama dalam terapi kanker. Dua obat

bersinergi kalau efek gabungannya lebih besar dari yang diharapkan dari masing-masing. “Yang diharapkan” perlu didefinisikan, dan model aditif Loewe yang dibahas kuliah memakai dosis ekuivalen. Kalau obat 1 sendirian butuh dosis D_1 dan obat 2 sendirian butuh D_2 untuk mencapai efek tertentu, kombinasi dosis d_1 dan d_2 yang mencapai efek yang sama dinilai dengan indeks

$$CI = \frac{d_1}{D_1} + \frac{d_2}{D_2}. \quad (23.4)$$

Indeks satu berarti aditif, lebih kecil dari satu berarti sinergis. Kalau separuh dosis tunggal masing-masing sudah mencukupi, $CI = 0,5 + 0,5 = 1$: aditif. Kalau seperempat masing-masing sudah cukup, $CI = 0,5$: sinergis. Model deep learning dilatih untuk memprediksi sinergi dari struktur kedua obat dan profil sel kanker, dan kuliah mengingatkan kebocoran khasnya: pasangan (A, B) di data latih dan (B, A) di data uji adalah pasangan yang sama.

23.6 Graf dan geometri molekul

Bagian lanjut kuliah membawa geometric deep learning (Bab 13) ke kimia. ChemProp (Yang et al., 2019) menjalankan message passing langsung pada graf molekul, dengan pesan yang dikirim di sepanjang ikatan, lalu sebuah fungsi *readout* merangkum semua atom menjadi satu vektor molekul, misalnya dengan menjumlahkan, sebelum masuk ke jaringan prediksi. Fiturnya dipelajari, bukan dirancang seperti pada ECFP.

Untuk sifat yang bergantung pada bentuk tiga dimensi, jaringan harus menghormati simetri ruang: memutar atau menggeser molekul tidak boleh mengubah energinya. EGNN (Satorras et al., 2021) mencapainya dengan memperbarui fitur simpul h_i dan koordinat x_i hanya lewat besaran yang invarian atau ekuivarian:

$$m_{ij} = \phi_e(h_i, h_j, \|x_i - x_j\|^2), \quad x'_i = x_i + C \sum_{j \neq i} (x_i - x_j) \phi_x(m_{ij}), \quad h'_i = \phi_h\left(h_i, \sum_j m_{ij}\right). \quad (23.5)$$

Pesan hanya melihat jarak kuadrat, yang tidak berubah ketika molekul diputar, sehingga fitur simpulnya invarian. Koordinat diperbarui dengan menjumlahkan vektor selisih $x_i - x_j$ yang dikalikan bilangan, sehingga kalau semua koordinat diputar, pembaruannya ikut berputar. Bukti ekuivariansinya hanya beberapa baris, dan kuliah memperlihatkan sebagai contoh bagaimana simetri dibangun ke dalam arsitektur alih-alih dipelajari dari data.

23.7 Belajar dari banyak modalitas dan sedikit contoh

Data ilmu hayati datang dalam banyak bentuk: struktur molekul, deskripsi teks sebuah assay, gambar mikroskop sel, urutan protein. Kuliah membahas cara menghubungkannya dengan *contrastive learning*. Gagasan dasarnya berasal dari CLIP (Radford et al., 2021), yang melatih encoder gambar dan encoder teks agar pasangan yang benar berdekatan dan pasangan yang salah berjauhan di ruang embedding bersama.

Loss-nya bisa diturunkan sebagai masalah klasifikasi. Dalam satu batch ada N pasangan. Untuk molekul ke- i , kita hitung kemiripan kosinus s_{ij} dengan setiap

deskripsi j dalam batch, lalu memperlakukan pertanyaan “deskripsi mana yang cocok dengan molekul ini?” sebagai klasifikasi N kelas dengan softmax. Cross-entropy untuk jawaban yang benar memberi

$$\mathcal{L}_i = -\log \frac{\exp(s_{ii}/\tau)}{\sum_{j=1}^N \exp(s_{ij}/\tau)}, \quad (23.6)$$

dengan τ suhu yang mengatur ketajaman softmax. Loss ini dihitung ke dua arah, dari molekul ke teks dan dari teks ke molekul, lalu dirata-ratakan. Sebagai contoh, misalkan batch berisi tiga pasangan, $\tau = 1$, dan untuk molekul pertama kemiripannya (0,9; 0,2; 0,1). Bobot softmax untuk pasangan yang benar adalah $e^{0,9}/(e^{0,9} + e^{0,2} + e^{0,1}) \approx 2,46/(2,46 + 1,22 + 1,11) \approx 0,51$, dan loss-nya $-\ln 0,51 \approx 0,67$. Menurunkan suhu menjadi 0,1 membuat bobot itu mendekati satu.

Kelompok di Linz menemukan kelemahan halus pada loss ini. Fürst et al. (2021) mengamati bahwa CLIP cenderung bertumpu pada satu atau dua fitur yang paling membedakan dan mengabaikan fitur relevan lainnya, gejala explaining away yang sudah kita kenal dari Bab 14. Usulan mereka, CLOOB, punya dua bahan. Embedding diperkaya dengan jaringan Hopfield modern (Bab 9), yang mengambil kembali pola-pola tersimpan yang mirip sehingga struktur kovarians antar fitur ikut terbawa. Loss-nya diganti dengan InfoLOOB, yang mengeluarkan pasangan positif dari penyebut: $-\log(e^{s_{ii}/\tau}/\sum_{j \neq i} e^{s_{ij}/\tau})$. Pada InfoNCE, rasio di dalam logaritma tidak bisa melebihi satu, sehingga begitu pasangan positif sudah jauh mendominasi, gradiennya hampir hilang dan model berhenti belajar. Dengan kemiripan (0,9; 0,2; 0,1) dan $\tau = 1$ seperti di atas, rasio InfoNCE adalah 0,51, sedangkan rasio InfoLOOB $2,46/(1,22 + 1,11) \approx 1,06$, dan rasio ini terus bisa naik tanpa batas. Gagasan yang sama dibawa ke biologi oleh CLOOME (Sanchez-Fernandez et al., 2023), yang menempatkan gambar mikroskop sel yang diberi senyawa dan struktur kimia senyawa itu dalam satu ruang embedding. Dengan sebuah struktur kimia sebagai kueri, sistem ini menemukan gambar sel yang benar dari sekitar dua ribu kandidat dengan akurasi teratas lebih dari tujuh puluh kali tebakan acak, sehingga basis data citra biologi bisa “ditanya” dengan rumus kimia.

CLAMP (Seidl et al., 2023) memakai gagasan ini untuk menghubungkan molekul dengan deskripsi teks assay, sehingga model bisa memprediksi aktivitas untuk assay baru hanya dari deskripsinya, tanpa satu pun contoh berlabel. Itulah *zero-shot learning*. Varian lain menghubungkan molekul dengan gambar mikroskop sel yang diberi senyawa itu, atau dengan kantong pengikatan protein. Ketika hanya ada segelintir contoh berlabel, pendekatan *few-shot* memakai jaringan Hopfield modern (Bab 9) untuk membandingkan molekul kueri dengan contoh-contoh yang ada dan dengan konteks dari molekul lain. *Active learning* memilih molekul berikutnya yang paling informatif untuk diukur, biasanya yang prediksinya paling tidak pasti, sehingga setiap percobaan laboratorium yang mahal memberi informasi sebanyak mungkin.

Kadang labelnya hanya ada untuk sekumpulan objek, bukan untuk masing-masing. Campuran senyawa diuji bersama, atau satu pasien punya ribuan sel tetapi hanya satu diagnosis. *Multiple instance learning* menangani situasi ini: setiap contoh adalah “kantong” berisi banyak instans, dan label kantong positif kalau setidaknya satu instans positif. Fraksi instans yang benar-benar membawa sinyal disebut *witness rate*, dan kalau

ia kecil, masalahnya sulit. Pooling dengan attention, yang juga bisa dibaca sebagai jaringan Hopfield, belajar menimbang instans mana yang paling menentukan.

Model bahasa besar juga mulai dipakai di kimia. Mereka fasih menjawab pertanyaan tentang reaksi dan sifat zat, tetapi kuliah menunjukkan contoh di mana mereka kesulitan memahami struktur molekul yang ditulis sebagai SMILES, sehingga peran mereka untuk sementara lebih sebagai asisten daripada pengganti model khusus.

23.8 Merancang molekul baru

Model generatif membalik arah QSAR: bukan dari struktur ke sifat, melainkan dari sifat yang diinginkan ke struktur. Kuliah membedakan dua paradigma. *Distribution learning* belajar membangkitkan molekul yang mirip dengan himpunan latih. *Goal-directed learning* mengarahkan model ke molekul dengan skor setinggi mungkin. Pendekatan paling sederhana memperlakukan SMILES sebagai teks dan melatih LSTM untuk menebak karakter berikutnya (Segler et al., 2018a). Setelah dilatih pada jutaan molekul, model ini menghasilkan string baru yang sebagian besar adalah molekul sah. Untuk mengarahkannya, model disetel dengan reinforcement learning (Bab 16), dengan model QSAR sebagai fungsi ganjaran, atau dilatih ulang pada molekul-molekul terbaik. VAE (Gómez-Bombarelli et al., 2018) memetakan molekul ke ruang laten kontinu, sehingga optimasi bisa dilakukan dengan gradien di ruang itu. Pembangkitan bersyarat memberi sifat yang diinginkan sebagai input sejak awal.

Ada beberapa jebakan. Model yang dioptimasi terhadap satu skor cenderung menemukan celah dalam skor itu, menghasilkan molekul yang secara angka sempurna tetapi aneh secara kimia, sehingga biasanya beberapa tujuan dioptimalkan bersama. Mode collapse membuat model terus menghasilkan variasi kecil dari satu molekul. Untuk mengevaluasi model generatif, kita memeriksa validitas, keunikan, kebaruan, dan keragaman molekul yang dihasilkan, serta seberapa mirip distribusinya dengan molekul nyata. Fréchet ChemNet Distance (Preuer et al., 2018), dari Linz juga, mengukur hal terakhir dengan membandingkan distribusi aktivasi sebuah jaringan prediksi bioaktivitas untuk kedua himpunan molekul. Kalau kedua distribusi dianggap Gauss, jarak Fréchet di antaranya punya bentuk tertutup,

$$d^2 = \|\mu_1 - \mu_2\|^2 + \text{Tr}(\Sigma_1 + \Sigma_2 - 2(\Sigma_1 \Sigma_2)^{1/2}). \quad (23.7)$$

Dalam satu dimensi, rumus ini menyederhana menjadi $(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$, karena $\sigma_1^2 + \sigma_2^2 - 2\sigma_1\sigma_2 = (\sigma_1 - \sigma_2)^2$: selisih rata-rata ditambah selisih lebar sebaran. Kuliah juga mengingatkan bahwa metrik ini menangkap banyak bias dari jaringan yang dipakainya, sehingga angka FCD yang baik belum menjamin molekul yang baik.

23.8.1 Banyak tujuan sekaligus

Obat yang baik harus kuat terhadap targetnya, tidak beracun, larut dengan cukup, stabil di dalam tubuh, dan bisa disintesis. Tujuan-tujuan ini sering bertentangan, sehingga desain molekul adalah *multi-objective optimization*. Sebuah kandidat *mendominasi* kandidat lain kalau ia setidaknya sama baik di semua tujuan dan lebih baik di setidaknya satu. Kandidat yang tidak didominasi siapa pun membentuk *Pareto front*. Misalkan tiga

kandidat punya kekuatan, dalam pIC_{50} , dan skor toksisitas: A dengan (8; 0,3), B dengan (7; 0,1), dan C dengan (6; 0,4). C didominasi A, yang lebih kuat sekaligus kurang beracun. A dan B tidak saling mendominasi: A lebih kuat, B lebih aman. Keduanya berada di front Pareto, dan memilih di antara keduanya adalah keputusan ahli kimia medis, bukan keputusan algoritme.

Cara paling sederhana menggabungkan tujuan adalah jumlah berbobot. Kelemahannya, bagian front Pareto yang tidak cembung tidak pernah bisa ditemukan dengan jumlah berbobot, berapa pun bobotnya. Metode yang lebih baik mempertahankan sekumpulan kandidat yang tersebar di sepanjang front, atau memakai fungsi *desirability* yang memetakan setiap sifat ke skala nol sampai satu dan mengalikannya, sehingga satu sifat yang sama sekali tidak bisa diterima membuat seluruh skor nol.

23.9 Data medis

Satu bagian kuliah membahas data klinis. Catatan kesehatan elektronik berisi data tabular dengan campuran variabel kontinu, kategorikal, dan waktu. Kuliah menekankan pentingnya menjelajahi data sebelum memodelkannya, karena data medis penuh kejutan: satuan yang berbeda antar rumah sakit, kode diagnosis yang berganti versi, dan nilai yang hilang di mana-mana.

Nilai yang hilang jarang hilang secara acak. Kalau peluang hilangnya sebuah nilai tidak bergantung pada apa pun, data masih bisa diolah dengan relatif aman. Kalau peluang itu bergantung pada variabel lain yang teramati, misalnya tes jarang dilakukan pada pasien muda, imputasi berdasarkan variabel lain bisa mengoreksinya. Kalau peluang itu bergantung pada nilai yang hilang itu sendiri, misalnya pasien dengan gejala ringan tidak dites sama sekali, tidak ada imputasi yang bisa memulihkan informasinya sepenuhnya, dan fakta bahwa nilainya hilang justru menjadi petunjuk. Kuliah juga membahas contoh model yang belajar sinyal yang salah, seperti penanda rumah sakit atau jenis alat pemindai, alih-alih sinyal penyakit.

Untuk data tabular, model berbasis pohon masih sulit dikalahkan, meski arsitektur khusus terus dikembangkan. TabPFN (Hollmann et al., 2025) mengambil jalan yang tidak biasa: sebuah transformer dilatih sekali pada jutaan masalah tabular sintesis, lalu untuk data baru ia memprediksi langsung dalam satu lintasan maju, dengan data latih sebagai konteks, tanpa pelatihan ulang.

Data medis tersebar di banyak rumah sakit yang tidak boleh saling berbagi data pasien. *Federated learning* melatih model bersama dengan hanya bertukar pembaruan parameter. Dalam bentuk paling sederhananya, setiap rumah sakit k melatih model lokal pada n_k pasiennya beberapa langkah, lalu server merata-ratakan parameter dengan bobot ukuran data, $w \leftarrow \sum_k n_k w_k / \sum_k n_k$, dan mengirimkan hasilnya kembali. Data mentah tidak pernah meninggalkan rumah sakit, meski pembaruan parameter sendiri masih bisa membocorkan sedikit informasi, sehingga sering ditambah perlindungan lain. Kuliah juga membahas data medis berbentuk deret waktu, seperti tanda vital di unit perawatan intensif. Deret seperti ini berbeda dari deret sensor di Bab 20: pengukurannya tidak teratur, karena tekanan darah diukur setiap jam tetapi kadar laktat hanya ketika dokter memintanya, dan jarak waktu antar pengukuran itu sendiri membawa informasi.

Pendekatan yang umum adalah menyusun data ke dalam selang waktu tetap, mengisi nilai yang hilang dengan nilai terakhir yang diketahui, dan menambahkan dua fitur untuk setiap variabel: penanda apakah nilainya benar-benar diukur atau diisi, dan berapa lama sejak pengukuran terakhir. Model rekuren dan transformer dari Bab 9 lalu bisa memakai pola pengukuran itu sendiri, sesuai pelajaran tentang data yang hilang tidak secara acak di atas. Teks klinis seperti catatan dokter dan ringkasan pulang adalah sumber lain, tempat model bahasa medis mulai dipakai.

23.10 Reaksi dan perencanaan sintesis

Molekul yang dirancang komputer tidak berguna kalau tidak bisa dibuat. Kuliah mengulang dasar reaksi kimia. Termodinamika menentukan apakah reaksi bisa terjadi: reaksi berjalan spontan kalau energi bebas Gibbs turun, $\Delta G = \Delta H - T\Delta S < 0$, yaitu kalau pelepasan panas dan kenaikan ketidakteraturan bersama-sama cukup besar. Kinetika menentukan seberapa cepat: molekul harus melewati penghalang energi aktivasi, dan laju reaksi naik tajam dengan temperatur karena semakin banyak molekul yang punya energi cukup. Katalis, termasuk enzim di tubuh, mempercepat reaksi dengan menurunkan penghalang itu tanpa mengubah keseimbangan akhirnya.

Retrosynthesis bekerja mundur: dari molekul target, cari reaksi yang bisa menghasilkan dari bahan yang lebih sederhana, lalu ulangi sampai tiba di bahan yang bisa dibeli. Satu langkah mundur bisa dirumuskan dengan beberapa cara. Templat reaksi yang diekstrak dari basis data reaksi bisa dipilih lewat klasifikasi. Reaksi juga bisa dianggap terjemahan dari SMILES produk ke SMILES reaktan, seperti menerjemahkan bahasa. Atau templat dipilih lewat pencarian kemiripan. MHNreact (Seidl et al., 2022) memakai jaringan Hopfield modern untuk mencocokkan molekul dengan templat reaksi, sehingga templat yang jarang muncul pun bisa dipilih. Untuk rencana banyak langkah, masalahnya mirip permainan: setiap langkah membuka banyak kemungkinan, dan kita mencari jalur yang berakhir di bahan yang tersedia. Segler et al. (2018b) memakai Monte Carlo tree search yang dipandu jaringan saraf, gagasan yang sama dengan AlphaGo (Bab 5).

23.11 Benang merah

Bab ini memperlihatkan bahwa hampir semua alat dari bagian-bagian sebelumnya menemukan tempatnya di kimia dan kedokteran. Message passing dan ekuivariansi untuk molekul, LSTM dan transformer untuk SMILES dan urutan, contrastive learning untuk menghubungkan modalitas, reinforcement learning dan pencarian pohon untuk desain dan sintesis, jaringan Hopfield untuk belajar dari sedikit contoh. Yang membedakan bidang ini adalah datanya: sedikit, berisik, tidak seimbang, dan penuh struktur tersembunyi yang bisa membocorkan jawaban. Di sini, cara mengevaluasi sering lebih menentukan daripada arsitektur model.

23.12 Perkembangan riset

Docking molekul, menebak bagaimana sebuah ligan menempel pada kantong protein, secara tradisional diselesaikan dengan pencarian dan fungsi skor. DiffDock (Corso et al., 2022) merumuskannya sebagai pemodelan generatif: model difusi bekerja pada ruang pose ligan, yaitu translasi, rotasi, dan sudut torsi, bukan pada koordinat atom mentah. Pada himpunan uji PDDBind, tingkat keberhasilan teratasnya dilaporkan 38 persen dibanding 23 persen untuk docking tradisional, dengan perkiraan keyakinan untuk setiap pose. Ini menyambung dengan gagasan ekuivariansi dan difusi dari Bab 13 dan Bab 15.

Model bahasa juga mulai dipakai sebagai agen kimia. ChemCrow (Bran et al., 2023) memberi model bahasa delapan belas alat yang dirancang ahli, dari pencarian literatur sampai perencanaan sintesis, dan agen itu merencanakan serta menjalankan sintesis sebuah penolak serangga dan beberapa organokatalis. Makalahnya juga mencatat temuan yang menjadi peringatan: GPT-4 sebagai penilai tidak bisa membedakan jawaban GPT-4 yang jelas salah dari jawaban agen yang benar. Penilaian oleh ahli tetap tidak tergantikan, pelajaran yang sama dengan bagian evaluasi di bab ini.

CATATAN SUMBER. Bab ini mengikuti kuliah AI in Life Sciences: penemuan obat, representasi molekul, aktivitas biologis dan ADME, QSAR dan virtual screening, evaluasi dan pembagian data, prediksi target dan sinergi obat, geometric deep learning untuk molekul, model multimodal, few-shot dan multiple instance learning, model generatif, data medis, serta reaksi kimia dan perencanaan sintesis.

24 Bentuk Protein

Kuliah bioinformatika struktural saya ikuti pada musim panas 2025. Naskah kuliahnya, yang ditulis Sepp Hochreiter dan Noura Chelbat, bergerak dari ikatan kimia dan sudut torsi ke perbandingan struktur, prediksi struktur sekunder, pemodelan homologi dan threading, prediksi *ab initio*, dan dinamika molekul. Bagian praktisnya berupa tantangan di sebuah server penilaian: untuk setiap protein, tebak residu mana yang membentuk *binding pocket*, kantong tempat molekul kecil seperti obat bisa menempel. Prediksi dikirim ke server dan langsung muncul di papan peringkat.

Di Bab 22 kita melihat bahwa gen menyandikan urutan asam amino. Urutan itu baru setengah cerita. Rantai asam amino melipat menjadi bentuk tiga dimensi yang khas, dan bentuk itulah yang menentukan apa yang bisa dilakukan sebuah protein: mengikat oksigen, memotong molekul lain, atau mengirim sinyal melintasi membran sel. Memprediksi bentuk dari urutan adalah salah satu masalah besar biologi selama setengah abad, sampai deep learning mengubahnya.

24.1 Dari rantai ke bentuk

Dua puluh jenis asam amino berbeda pada rantai sampingnya: ada yang suka air, ada yang menghindarinya, ada yang bermuatan, ada yang kecil dan lentur seperti glisin, ada yang kaku seperti prolin. Asam amino tersambung lewat ikatan peptida. Karena elektronnya terbagi di antara beberapa atom, ikatan ini bersifat setengah rangkap, sehingga hampir datar dan tidak bisa berputar. Kebebasan rantai utama terletak pada dua sudut putar di setiap residu, sudut torsi ϕ dan ψ . Tidak semua pasangan sudut mungkin, karena atom-atom akan bertabrakan. Peta pasangan yang diizinkan, diagram Ramachandran, menunjukkan beberapa daerah padat yang sesuai dengan heliks dan lembaran.

Struktur protein dijelaskan dalam beberapa tingkat. Struktur primer adalah urutan asam aminonya. Struktur sekunder adalah pola lokal yang berulang, terutama heliks alfa dan lembaran beta. Struktur tersier adalah lipatan tiga dimensi seluruh rantai, dan struktur kuarterner adalah susunan beberapa rantai menjadi satu kompleks. Gaya pendorong utama pelipatan adalah efek hidrofobik: residu yang menghindari air bersembunyi di dalam inti protein, sementara residu yang suka air berada di permukaan. Ikatan hidrogen, jembatan garam, dan gaya van der Waals menstabilkan lipatan yang terbentuk.

Heliks alfa punya geometri yang rapi. Rantai utama melilit dengan sekitar 3,6 residu per putaran, sehingga setiap residu diputar sekitar $360^\circ/3,6 = 100^\circ$ dari residu sebelumnya. Ikatan hidrogen menghubungkan gugus karbonil residu i dengan gugus amida residu $i + 4$. Periodisitas 100° punya akibat yang bisa dibaca dari urutan: residu

yang berjarak tiga atau empat dalam urutan mengarah ke sisi yang sama dari heliks. Kalau residu hidrofobik muncul dengan pola seperti itu, heliksnya amfipatik, dengan satu sisi suka air dan sisi lain menghindarinya, cocok untuk berbaring di permukaan protein atau membran. Lembaran beta tersusun dari untai-untai yang hampir lurus, yang diikat ikatan hidrogen ke untai tetangganya, baik searah maupun berlawanan arah.

Seberapa sulit pelipatan itu? Kalau setiap residu dari protein seratus residu hanya punya tiga konformasi, jumlah kemungkinannya $3^{100} \approx 5 \times 10^{47}$. Mencoba satu konformasi setiap 10^{-13} detik pun membutuhkan sekitar 5×10^{34} detik, jauh lebih lama dari umur alam semesta. Padahal protein nyata melipat dalam mikrodetik sampai detik. Paradoks Levinthal ini menunjukkan bahwa pelipatan bukan pencarian acak, melainkan perjalanan menuruni lanskap energi berbentuk corong yang mengarahkan rantai ke lipatan aslinya.

24.2 Membandingkan struktur

Sebelum memprediksi, kita perlu cara mengukur seberapa mirip dua struktur. Ukuran paling umum adalah *root mean square deviation* setelah kedua struktur disuperposisikan sebaik mungkin,

$$\text{RMSD} = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\mathbf{R}\mathbf{x}_i + \mathbf{t} - \mathbf{y}_i\|^2}, \quad (24.1)$$

dengan $\mathbf{x}_i$ dan $\mathbf{y}_i$ posisi atom padanan, biasanya atom karbon alfa, dan $\mathbf{R}$, $\mathbf{t}$ rotasi dan translasi yang meminimalkan jumlah itu. Translasi terbaik ternyata sederhana: geser kedua struktur sehingga pusat massanya berimpit. Untuk rotasinya, setelah keduanya dipusatkan, jumlah kuadrat bisa diuraikan menjadi $\sum \|\mathbf{x}_i\|^2 + \sum \|\mathbf{y}_i\|^2 - 2 \sum \mathbf{y}_i^\top \mathbf{R}\mathbf{x}_i$. Dua suku pertama tidak bergantung pada $\mathbf{R}$, jadi meminimalkan RMSD sama dengan memaksimalkan $\sum_i \mathbf{y}_i^\top \mathbf{R}\mathbf{x}_i = \text{Tr}(\mathbf{R}\mathbf{H})$ dengan $\mathbf{H} = \sum_i \mathbf{x}_i \mathbf{y}_i^\top$. Kalau $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ adalah dekomposisi nilai singularnya, maksimum tercapai pada $\mathbf{R} = \mathbf{V}\mathbf{U}^\top$, dengan koreksi tanda kalau determinannya negatif agar hasilnya rotasi sejati dan bukan pencerminan. Prosedur ini, yang dikenal sebagai algoritme Kabsch, sama dengan langkah inti ICP di Bab 21.

RMSD punya kelemahan: satu bagian yang fleksibel, seperti ekor yang bergoyang, bisa membuat dua struktur yang intinya identik tampak jauh berbeda, dan nilainya cenderung naik dengan panjang protein. TM-score mengatasinya dengan menjumlahkan sumbangan setiap residu yang meluruh dengan jarak,

$$\text{TM} = \frac{1}{L} \sum_{i=1}^L \frac{1}{1 + (d_i/d_0)^2}, \quad (24.2)$$

dengan d_i jarak residu ke- i setelah superposisi dan d_0 skala jarak yang tumbuh perlahan dengan panjang protein L . Residu yang jauh menyumbang hampir nol, bukan hukuman besar, sehingga satu ekor yang bergoyang tidak merusak skor. Nilainya antara nol dan satu, dan skor di atas sekitar 0,5 biasanya berarti kedua protein punya lipatan yang sama.

Kalau urutan kedua protein berbeda, padanan residunya pun belum diketahui, dan struktur harus dijajarkan sebelum dibandingkan. DALI, salah satu metode yang dibahas, membandingkan matriks jarak antar residu di dalam masing-masing protein: dua struktur yang mirip punya pola jarak internal yang mirip, apa pun orientasinya di ruang, karena jarak tidak berubah ketika struktur diputar. CE merangkai potongan-potongan pendek yang cocok menjadi penjajaran yang lebih panjang.

24.3 Struktur sekunder

Struktur sekunder pertama-tama harus ditetapkan dari struktur yang sudah diukur. Program DSSP melakukannya dengan mencari pola ikatan hidrogen: deretan ikatan $i \rightarrow i + 4$ menandai heliks alfa, dan pasangan ikatan antar untai menandai lembaran. Label dari DSSP menjadi data latih untuk prediksi.

Prediksi struktur sekunder memberi setiap residu satu dari tiga label: heliks, lembaran, atau lainnya. Metode Chou–Fasman, yang paling awal, memakai kecenderungan statistik setiap asam amino untuk berada di heliks atau lembaran. Metode GOR menambahkan konteks dengan teori informasi: label sebuah residu ditebak dari asam amino di jendela sekitar delapan residu di kiri dan kanannya. Lompatan besar datang dari dua gagasan lain. Yang pertama adalah memakai profil evolusi dari banyak urutan homolog, yang dikumpulkan dengan pencarian seperti PSI-BLAST, alih-alih satu urutan saja. Posisi yang hanya mentoleransi residu hidrofobik di semua kerabat lebih informatif daripada satu residu dari satu protein. Yang kedua adalah jaringan saraf dua tingkat, yang dibahas rinci di naskah kuliah dan dipakai metode seperti PHD dan PSIPRED. Tingkat pertama memprediksi dari jendela profil, dan tingkat kedua menghaluskan prediksi itu dengan melihat prediksi tetangganya, karena heliks dan lembaran selalu muncul dalam potongan berurutan dan tidak pernah berupa satu residu yang terisolasi.

Kinerjanya diukur dengan Q3, fraksi residu yang labelnya benar, dan akurasi per kelas. Karena Q3 tidak peduli apakah sebuah heliks terpotong-potong menjadi beberapa heliks pendek, ukuran SOV menilai tumpang tindih segmen, bukan residu satu per satu. Naskah kuliah juga mencatat bahwa kebanyakan metode paling kesulitan dengan lembaran beta, karena untai-untainya dipasangkan oleh residu yang berjauhan dalam urutan, informasi yang tidak terlihat dari jendela lokal.

24.4 Struktur tiga dimensi

Kalau sebuah protein punya kerabat evolusi yang strukturnya sudah diketahui, kita bisa meminjam strukturnya. Pemodelan homologi mencari kerabat itu dengan pencarian urutan seperti BLAST, menjajarkan urutan baru dengan urutan kerabatnya, menyalin koordinat rantai utama untuk bagian yang terjajar, lalu membangun bagian yang berbeda, seperti lengkungan yang lebih panjang, dan rantai samping. Prinsip dasarnya adalah bahwa struktur dilestarikan evolusi jauh lebih kuat daripada urutan: dua protein dengan identitas urutan tiga puluh persen sering masih berbagi lipatan yang sama.

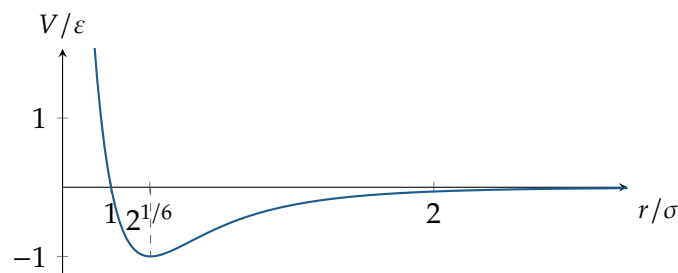
Kalau tidak ada kerabat yang jelas, *threading* mencoba memasang urutan baru ke setiap lipatan yang dikenal dan menilai kecocokannya. Penilaiannya memakai potensial kontak

statistik, yang dibahas di lampiran naskah kuliah: dari ribuan struktur yang diketahui, dihitung seberapa sering setiap pasangan jenis asam amino bersentuhan dibanding yang diharapkan secara acak, lalu diubah menjadi energi semu dengan logaritma. Energi sebuah susunan adalah jumlah energi semua pasangan yang bersentuhan, $E = \sum_{\text{kontak } (i,j)} e(a_i, a_j)$. Pasangan dua residu hidrofobik yang sering bersentuhan di inti protein mendapat energi negatif, sehingga susunan yang mengubur residu hidrofobik dinilai baik.

Pendekatan *ab initio* seperti Rosetta merakit struktur dari potongan-potongan pendek sepanjang tiga sampai sembilan residu yang diambil dari struktur yang dikenal dengan urutan lokal yang mirip, lalu mencari konformasi berenergi rendah dengan Monte Carlo dan aturan penerimaan Metropolis (Bab 28). Dinamika molekul mengintegrasikan persamaan Newton untuk setiap atom dengan medan gaya yang menjumlahkan energi ikatan, sudut, torsi, dan interaksi antar atom yang tidak terikat. Interaksi van der Waals biasanya dimodelkan dengan potensial Lennard-Jones,

$$V(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right], \quad (24.3)$$

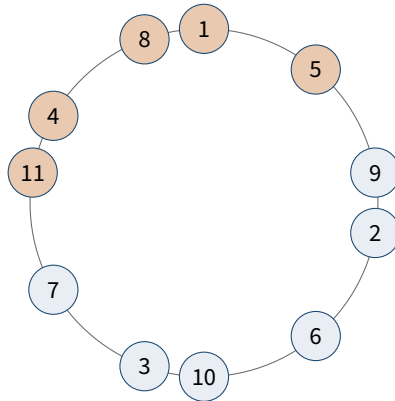
dengan suku pangkat dua belas untuk tolakan ketika atom terlalu dekat dan suku pangkat enam untuk tarikan lemah. Menyamakan turunannya dengan nol, $-12\sigma^{12}/r^{13} + 6\sigma^6/r^7 = 0$, memberi jarak setimbang $r = 2^{1/6}\sigma \approx 1,12\sigma$, dan memasukkannya kembali memberi kedalaman sumur tepat $-\epsilon$ (Gambar 24.1). Interaksi listrik antar muatan ditambahkan dengan hukum Coulomb. Simulasi seperti ini bisa memperhalus struktur dan memperlihatkan bagaimana protein bergerak, tetapi terlalu mahal untuk melipat protein besar dari nol, karena langkah waktunya harus sekitar satu femtodetik untuk mengikuti getaran ikatan.



GAMBAR 24.1. Potensial Lennard-Jones. Tolakan yang curam di jarak pendek, sumur sedalam ϵ di $r = 2^{1/6}\sigma$, dan tarikan yang melemah di jarak jauh.

Untuk membaca urutan secara geometris, Gambar 24.2 memperlihatkan roda heliks: residu-residu heliks alfa dilihat dari ujung sumbunya, masing-masing diputar 100° dari sebelumnya. Residu ke-1, ke-4, ke-5, dan ke-8 jatuh di sisi yang hampir sama, sehingga kalau keempatnya hidrofobik, heliksnya punya satu wajah berminyak.

Perkembangan bidang ini diukur dengan kompetisi CASP, di mana kelompok-kelompok memprediksi struktur yang sudah dipecahkan secara eksperimen tetapi belum dipublikasikan. Naskah kuliah, yang ditulis pada 2007, menutup dengan hasil CASP7, ketika prediksi tanpa kerabat yang jelas masih jauh dari akurat. Selama bertahun-tahun, kemajuannya lambat.



GAMBAR 24.2. Roda heliks untuk sebelas residu. Residu yang diwarnai (1, 4, 5, 8, 11) berkumpul di satu sisi, pola yang menandai heliks amfipatik.

24.5 Dari mutasi yang berpasangan ke peta kontak

Gagasan bahwa evolusi menyimpan informasi tentang bentuk protein bisa dibuat kuantitatif. Susun ratusan urutan dari keluarga protein yang sama dalam satu penjumlahan, sehingga setiap kolom adalah satu posisi residu. Kalau dua posisi bersentuhan di dalam struktur, asam amino di kedua kolom cenderung berubah bersama. Ukuran paling sederhananya adalah *mutual information* antara dua kolom, $MI(i, j) = \sum_{a,b} p_{ij}(a, b) \log_2 \frac{p_{ij}(a, b)}{p_i(a)p_j(b)}$, yaitu KL antara distribusi bersama dan hasil kali marginalnya (Lampiran B). Misalkan dalam empat urutan, kolom i dan j berisi pasangan (A,L), (A,L), (G,V), dan (G,V): setiap kali posisi i berisi A, posisi j berisi L, sehingga mengetahui satu kolom memberi tahu kolom lain sepenuhnya, dan MI-nya satu bit. Kalau pasangannya (A,L), (A,V), (G,L), dan (G,V), kedua kolom independen, dan MI-nya nol.

Masalahnya adalah korelasi tidak langsung. Kalau posisi A bersentuhan dengan B, dan B bersentuhan dengan C, maka A dan C juga tampak berkorelasi walau tidak pernah bersentuhan, sama seperti dua anak dari orang tua yang sama tampak berkorelasi tanpa saling memengaruhi. *Direct-coupling analysis* (Morcos et al., 2011) memisahkan keduanya dengan menyesuaikan model statistik global atas seluruh penjumlahan, sejenis model Potts, sehingga setiap pasangan hanya mendapat sumbangan yang tidak bisa dijelaskan oleh pasangan lain. Hasilnya adalah peta kontak yang cukup akurat untuk merekonstruksi lipatan global banyak keluarga protein hanya dari urutan.

Peta kontak adalah matriks $L \times L$, sehingga bisa diperlakukan seperti gambar, dan sejak pertengahan 2010-an jaringan konvolusi dilatih untuk meramalkan kontak, bahkan jarak antar residu, dari fitur-fitur penjumlahan. Pola-pola khas dalam peta itu mudah dikenali: pita sejajar diagonal untuk heliks, karena residu i bersentuhan dengan $i + 3$ dan $i + 4$, serta pita tegak lurus diagonal untuk untai beta yang saling berlawanan arah. Langkah dari meramal peta kontak ke meramal koordinat langsung adalah langkah yang diambil AlphaFold.

24.6 AlphaFold

Pada CASP14 tahun 2020, AlphaFold2 (Jumper et al., 2021) memprediksi banyak struktur dengan akurasi yang mendekati percobaan. Gagasan kuncinya sudah lama dikenal dalam bioinformatika, tetapi dimanfaatkan dengan cara baru. Kalau dua residu bersentuhan di dalam struktur, mutasi pada salah satunya sering diikuti mutasi pengimbang pada yang lain, supaya kontakannya tetap terjaga. Jadi pola mutasi yang berkorelasi dalam penjajaran banyak urutan homolog adalah petunjuk tentang residu mana yang berdekatan di ruang tiga dimensi. Ini profil evolusi dari prediksi struktur sekunder, dibawa ke tingkat pasangan residu.

AlphaFold2 memproses dua representasi sekaligus: penjajaran banyak urutan, dan representasi pasangan yang menyimpan hubungan antara setiap pasangan residu. Blok Evoformer bertukar informasi di antara keduanya dengan attention, termasuk operasi yang menegaskan konsistensi geometris, misalnya bahwa jarak-jarak di antara tiga residu harus memenuhi ketaksamaan segitiga. Modul struktur kemudian menempatkan setiap residu sebagai benda kaku di ruang dengan attention yang invarian terhadap rotasi dan translasi global, gagasan dari Bab 13. Model ini juga mengeluarkan perkiraan keyakinannya sendiri untuk setiap residu, sehingga pengguna tahu bagian mana yang bisa dipercaya dan bagian mana, sering ujung-ujung yang tidak berstruktur, yang sebaiknya diabaikan. AlphaFold3 (Abramson et al., 2024) memperluasnya ke kompleks protein dengan DNA, RNA, dan molekul kecil, dengan modul struktur berbasis difusi (Bab 15) yang membangkitkan koordinat atom secara langsung.

Hadiah Nobel Kimia 2024 sebagian diberikan untuk prediksi struktur protein ini, pengakuan bahwa sebuah metode machine learning telah menyelesaikan masalah ilmiah yang mendasar.

24.7 Binding pocket dan obat

Untuk penemuan obat (Bab 23), yang paling penting sering bukan seluruh struktur, melainkan kantong tempat obat menempel. Tantangan di kuliah merumuskan prediksi binding pocket sebagai klasifikasi setiap residu: masuk kantong atau tidak. Data latihnya berupa daftar residu kantong untuk setiap protein, dan prediksi dikirim dalam format yang sama. Fiturnya bisa berasal dari urutan, dari struktur tiga dimensi seperti seberapa terkubur sebuah residu dan bentuk permukaan di sekitarnya, atau dari embedding model bahasa protein.

Karena residu kantong hanya sebagian kecil dari seluruh protein, akurasi biasa menyesatkan, persis seperti pelajaran tentang kelas tidak seimbang di Bab 1. Kalau sebuah protein punya 300 residu dan 15 di antaranya membentuk kantong, model yang selalu menjawab “bukan kantong” sudah benar 95 persen. Ukuran yang lebih jujur membandingkan himpunan residu tebakan dengan himpunan yang benar. Kalau model menebak 12 residu dan 9 di antaranya benar, presisinya $9/12 = 0,75$, recall-nya $9/15 = 0,60$, dan F1-nya $2 \cdot 0,75 \cdot 0,60 / (0,75 + 0,60) \approx 0,67$. Pengetahuan kimia membantu di sini: kantong biasanya berupa cekungan di permukaan, dan residu-residunya berdekatan di ruang walau berjauhan dalam urutan, sehingga prediksi yang

tersebar acak di permukaan protein hampir pasti salah.

24.8 Perkembangan riset

Setelah prediksi struktur, perhatian bergeser ke desain. ESMFold (Lin et al., 2023) menunjukkan bahwa model bahasa protein dengan sampai 15 miliar parameter menyimpan cukup informasi struktur untuk memprediksi struktur atom langsung dari satu urutan, tanpa penjajaran banyak urutan, dan jauh lebih cepat. Kecepatannya memungkinkan prediksi struktur lebih dari 617 juta protein metagenomik. RFdiffusion (Watson et al., 2023) menyetel jaringan prediksi struktur RoseTTAFold sebagai model difusi untuk rangka protein, sehingga bisa membangkitkan rangka baru yang mengikat target tertentu atau membentuk susunan simetris, dan ratusan desainnya diuji di laboratorium. ProteinMPNN (Dauparas et al., 2022) menyelesaikan langkah berikutnya: diberikan sebuah rangka, cari urutan asam amino yang akan melipat ke bentuk itu, dengan message passing pada graf residu. Pada rangka protein alami, urutan aslinya dipulihkan 52 persen, dibanding 33 persen untuk Rosetta.

ESM3 (Hayes et al., 2025) menggabungkan urutan, struktur, dan fungsi dalam satu model bahasa generatif. Salah satu protein fluoresen yang dibangkitkannya, setelah disintesis, ternyata bersinar terang walau hanya 58 persen identik dengan protein fluoresen yang dikenal, jarak yang menurut perkiraan penulisnya setara dengan sekitar 500 juta tahun evolusi. Alur kerja yang terbentuk, rangka dari model difusi, urutan dari model graf, lalu pemeriksaan dengan prediksi struktur, adalah gabungan hampir semua bab di bagian buku ini.

CATATAN SUMBER. Bab ini mengikuti kuliah Structural Bioinformatics dan naskah Bioinformatics III (latar kimia dan fisika, struktur sekunder dan tersier, perbandingan dan penjajaran struktur, prediksi struktur sekunder dan evaluasinya, pemodelan homologi dan threading, prediksi ab initio dan dinamika molekul), beserta tantangan prediksi binding pocket. Perkembangan terbaru diambil dari makalah AlphaFold (Abramson et al., 2024; Jumper et al., 2021).

25 Melihat ke Dalam Tubuh

Kuliah deep learning for medical imaging dimulai bukan dengan jaringan saraf, melainkan dengan fisika. Naskah dasarnya membahas osilasi dan gelombang, analisis Fourier, gelombang elektromagnetik, fenomena gelombang seperti pembiasan, interferensi, hamburan, dan penyerapan, lalu cara menghasilkan gambar dengan sinar-X, tomografi komputer, dan kedokteran nuklir. Pilihannya masuk akal: kita tidak bisa menilai apa yang dilakukan sebuah jaringan pada citra CT tanpa memahami dari mana angka-angka di citra itu datang.

Citra medis berbeda dari foto biasa. Setiap piksel adalah hasil pengukuran fisik dengan satuan dan noise yang khas, gambar sering berbentuk volume tiga dimensi, data berlabel mahal karena butuh dokter ahli, dan kesalahan bisa berakibat fatal. Bab ini mengikuti jalan dari foton ke diagnosis.

25.1 Gelombang dan Fourier

Naskah kuliah membuka dengan osilator harmonik, sistem paling sederhana yang berosilasi. Massa m pada pegas dengan konstanta k merasakan gaya pemulih yang sebanding dengan simpangannya, sehingga hukum Newton memberi $m\ddot{x} = -kx$. Solusinya $x(t) = A \cos(\omega_0 t + \varphi)$ dengan $\omega_0 = \sqrt{k/m}$: frekuensinya ditentukan sistem, sedangkan amplitudo dan fase ditentukan keadaan awal. Banyak sistem yang jauh lebih rumit, dari molekul yang bergetar sampai rangkaian listrik, berperilaku seperti osilator harmonik di sekitar titik setimbangnya, karena setiap potensial yang mulus tampak seperti parabola kalau dilihat cukup dekat. Kalau ada gesekan, osilasinya meluruh secara eksponensial. Gelombang adalah osilasi yang merambat: setiap titik berosilasi, dan fasenya bergeser sepanjang arah rambat.

Analisis Fourier menyatakan sinyal apa pun sebagai jumlah gelombang sinus dengan frekuensi berbeda. Alat ini akan muncul lagi di jantung tomografi, dan sudah kita temui di pengolahan sinyal (Bab 18). Dua sifatnya paling penting di sini. Konvolusi di ruang menjadi perkalian di frekuensi, sehingga penyaringan bisa dilakukan dengan mengalikan spektrum. Dan teorema sampling berlaku juga di pemindai: detektor yang jaraknya terlalu renggang tidak bisa menangkap detail halus, dan detail yang terlewat muncul kembali sebagai pola palsu yang disebut *aliasing*. Naskah kuliah mencatat bahwa syarat sampling yang ideal untuk detektor CT tidak bisa dipenuhi secara teknis, sehingga pembuat pemindai memakai trik seperti menggeser detektor seperempat lebarnya atau menggoyang titik fokus tabung sinar-X.

Cahaya, gelombang radio, dan sinar-X adalah gelombang elektromagnetik yang sama dengan panjang gelombang berbeda. Ketika melewati materi, gelombang bisa dibiaskan, dipantulkan, diinterferensikan, dihamburkan, atau diserap. Untuk hamburan oleh

partikel yang jauh lebih kecil dari panjang gelombang, kekuatan hamburan naik dengan pangkat empat frekuensi, sehingga cahaya biru dihamburkan jauh lebih kuat dari cahaya merah. Naskah kuliah memakai contoh ini untuk menjelaskan mengapa langit biru dan senja merah, sebelum beralih ke foton berenergi jauh lebih tinggi.

25.2 Sinar-X dan penyerapan

Sinar-X dihasilkan dengan menembakkan elektron yang dipercepat tegangan tinggi ke sebuah anoda, biasanya dari tungsten. Elektron yang diperlambat mendadak di dalam anoda memancarkan *Bremsstrahlung*, spektrum kontinu yang dibatasi energi kinetik elektron, dan elektron yang menendang keluar elektron dalam atom anoda menghasilkan garis karakteristik yang tajam. Energi foton terbesar yang bisa muncul sama dengan energi kinetik elektron, $E = h\nu = eU$, sehingga tabung dengan tegangan 100 kilovolt menghasilkan foton sampai 100 kiloelektronvolt.

Di dalam tubuh, foton berinteraksi dengan materi terutama dengan dua cara. Pada efek fotolistrik, foton diserap seluruhnya dan melepaskan sebuah elektron. Efek ini kuat untuk unsur dengan nomor atom tinggi, seperti kalsium di tulang atau iodine pada zat kontras, dan memberi kontras yang tajam. Pada hamburan Compton, foton menumbuk elektron yang terikat lemah, kehilangan sebagian energinya, dan berbelok arah. Foton terhambur ini tidak membawa informasi tentang di mana ia seharusnya mendarat, dan membuat gambar berkabut. Kolimator dan grid anti hamburan, yaitu lamel-lamel penyerap yang hanya meloloskan foton yang datang dari arah yang benar, menyaringnya sebelum mencapai detektor.

Penurunan intensitas di setiap irisan tipis sebanding dengan intensitas yang datang dan dengan ketebalan irisan, $dI = -\mu I ds$, dengan μ koefisien atenuasi jaringan. Memisahkan variabel, $dI/I = -\mu ds$, lalu mengintegrasikan dari pintu masuk sampai pintu keluar memberi hukum Beer–Lambert,

$$I = I_0 \exp \left(- \int \mu(s) ds \right). \quad (25.1)$$

Untuk jaringan seragam dengan $\mu = 0,2 \text{ cm}^{-1}$ setebal 10 cm, hanya $e^{-2} \approx 13,5$ persen foton yang tembus. Ketebalan yang menyerap tepat separuh foton, $\ln 2/\mu$, di sini sekitar 3,5 cm. Tulang menyerap jauh lebih kuat dari jaringan lunak, itulah sebabnya tulang tampak terang pada foto rontgen.

Setiap foton yang diserap menyimpan energi di jaringan, dan di situlah harga sebuah gambar. Dosis serap diukur dalam gray, satu joule per kilogram. Jenis radiasi berbeda merusak jaringan dengan efektivitas berbeda, sehingga dosis ekuivalen mengalikan dosis serap dengan faktor bobot radiasi: satu untuk foton, dua puluh untuk partikel alfa. Organ berbeda juga punya kepekaan berbeda, dan dosis efektif menjumlahkan dosis ekuivalen setiap organ dengan bobot kepekaannya. Hasilnya dinyatakan dalam sievert. Paparan alami di Eropa Tengah sekitar dua milisievert per tahun, dan setiap pemeriksaan dengan sinar-X menambahkan sesuatu ke angka itu, sehingga selalu ada tarik-menarik antara kualitas gambar dan dosis.

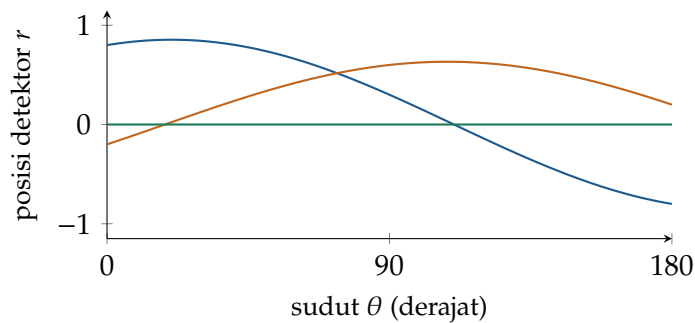
25.3 Tomografi komputer

Foto rontgen biasa menumpuk seluruh tubuh menjadi satu bayangan. Tomografi komputer mengukur banyak proyeksi dari berbagai sudut lalu merekonstruksi peta μ di setiap titik irisan. Dari hukum Beer–Lambert, setiap pengukuran memberi integral garis koefisien atenuasi,

$$p(\theta, r) = -\ln \frac{I}{I_0} = \int_{\text{garis}(\theta, r)} \mu(x, y) ds, \quad (25.2)$$

dengan θ arah proyeksi dan r jarak garis dari pusat. Kumpulan semua integral garis disebut transformasi Radon. Masalah utama CT adalah membalik transformasi ini.

Data mentah biasanya disusun sebagai gambar dengan sudut di satu sumbu dan posisi detektor di sumbu lain. Sebuah titik kecil di (x_0, y_0) hanya dilewati garis yang memenuhi $r = x_0 \cos \theta + y_0 \sin \theta$. Untuk semua sudut, titik itu menggambar sebuah kurva sinus, dengan amplitudo sama dengan jaraknya dari pusat dan fase sesuai arahnya. Karena itu data mentah ini disebut *sinogram* (Gambar 25.1). Sebuah benda nyata adalah tumpukan banyak titik, dan sinogramnya adalah tumpukan banyak kurva sinus. Untuk menguji algoritme rekonstruksi, dipakai gambar uji standar seperti phantom Shepp–Logan, sekumpulan elips yang meniru penampang kepala.



GAMBAR 25.1. Sinogram tiga titik: di $(0,8; 0,3)$, di $(-0,2; 0,6)$, dan di pusat. Setiap titik menggambar kurva $r = x_0 \cos \theta + y_0 \sin \theta$. Titik di pusat selalu terproyeksi ke $r = 0$.

Cara paling naif untuk membalik transformasi Radon adalah *backprojection*: setiap proyeksi “dioleskan” kembali sepanjang garisnya, lalu semua sudut dijumlahkan. Hasilnya kabur, karena setiap titik terang ikut menyebar ke seluruh garis yang melewatinya, dengan kekaburan yang meluruh seperti $1/r$.

Obatnya datang dari teorema irisan pusat: transformasi Fourier satu dimensi dari proyeksi pada sudut θ sama dengan nilai transformasi Fourier dua dimensi gambar, F , di sepanjang garis melalui pusat dengan arah yang sama. Dari teorema ini, rekonstruksi bisa diturunkan dalam dua baris. Gambar adalah transformasi Fourier balik dari F ,

$$\mu(x, y) = \iint F(u, v) e^{i2\pi(ux+vy)} du dv. \quad (25.3)$$

Sekarang ganti ke koordinat polar, $u = k \cos \theta$ dan $v = k \sin \theta$, dengan θ dari 0 sampai π dan k dari minus sampai plus tak hingga. Elemen luasnya berubah menjadi

$du dv = |k| dk d\theta$, persis seperti luas cincin tipis yang tumbuh dengan jari-jarinya. Menurut teorema irisan pusat, $F(k \cos \theta, k \sin \theta)$ adalah transformasi Fourier proyeksi, $P_\theta(k)$. Maka

$$\mu(x, y) = \int_0^\pi \left[\int_{-\infty}^\infty P_\theta(k) |k| e^{i2\pi k r} dk \right]_{r=x \cos \theta + y \sin \theta} d\theta. \quad (25.4)$$

Bagian dalam adalah proyeksi yang spektrumnya dikalikan $|k|$, lalu dikembalikan ke ruang. Bagian luar adalah backprojection. Faktor $|k|$ yang muncul dari perubahan koordinat itulah filter ramp, filter lolos tinggi yang mengoreksi frekuensi rendah yang terwakili berlebihan. Inilah *filtered backprojection* (Kak and Slaney, 1988), algoritme yang bekerja di dalam hampir semua pemindai CT sejak pemindai pertama yang dibangun Hounsfield (Hounsfield, 1973). Dalam praktik, filter ramp yang murni memperkuat noise frekuensi tinggi, sehingga ujungnya dilandaikan. Memilih bentuk filter adalah tawar-menawar antara ketajaman tepi dan kehalusan gambar.

Nilai yang ditampilkan bukan μ langsung, melainkan satuan Hounsfield, $HU = 1000(\mu - \mu_{\text{air}})/\mu_{\text{air}}$, dengan μ_{air} koefisien atenuasi air. Air bernilai 0 dan udara sekitar -1000 , sedangkan tulang padat bisa beberapa ratus sampai lebih dari seribu. Rentang ini jauh lebih lebar dari yang bisa dibedakan mata pada layar, sehingga dokter memakai jendela: hanya rentang tertentu yang dipetakan ke skala abu-abu, dan nilai di luarnya menjadi hitam atau putih. Jendela otak, misalnya, memetakan sekitar 0 sampai 80 HU ke seluruh skala abu-abu, sehingga perbedaan halus antara materi abu-abu dan materi putih terlihat jelas. Jendela paru jauh lebih lebar dan berpusat pada nilai negatif. Skala yang dibakukan ini membuat citra dari pemindai berbeda bisa dibandingkan, dan menjadi alasan mengapa normalisasi intensitas untuk jaringan saraf pada CT berbeda dari normalisasi pada foto biasa: memotong ke jendela yang relevan sering lebih berguna daripada standardisasi buta.

Ada beberapa artefak yang khas. Benda logam seperti tambalan gigi menyerap hampir semua foton dan merusak asumsi rekonstruksi, sehingga muncul garis-garis terang dan gelap. Foton berenergi rendah lebih mudah terserap, sehingga berkas menjadi “lebih keras” saat menembus tubuh, dan bagian tengah benda seragam tampak lebih gelap dari tepinya. Pasien yang bergerak selama pemindaian membuat proyeksi-proyeksi tidak konsisten satu sama lain.

25.4 Kedokteran nuklir

Pada skintigrafi, SPECT, dan PET, sumber radiasinya ada di dalam tubuh. Pasien diberi zat penanda radioaktif yang terkumpul di jaringan dengan aktivitas tertentu, misalnya tulang yang sedang memperbaiki diri atau sel tumor yang memakan banyak glukosa. Gambarnya menunjukkan fungsi, bukan anatomi.

Penanda yang paling penting untuk skintigrafi adalah teknesium-99m, yang memancarkan sinar gamma 140 kiloelektronvolt dan bisa dilekatkan ke banyak biomolekul. Waktu paruhnya enam jam. Peluruhan radioaktif mengikuti $N(t) = N_0 2^{-t/T_{1/2}}$, sehingga setelah satu hari, empat waktu paruh, tinggal 2^{-4} atau sekitar enam persen aktivitas awal. Waktu paruh ini cukup panjang untuk menyiapkan dan menjalankan pemerik-

saan, dan cukup pendek agar pasien tidak lama terpapar. Kamera gamma memakai kolimator, pelat timbal dengan lubang-lubang panjang yang hanya meloloskan sinar dari satu arah, karena sinar gamma tidak bisa difokuskan dengan lensa. Kolimator membuang sebagian besar foton, sehingga gambar kedokteran nuklir selalu berisik. SPECT memutar kamera di sekitar pasien dan merekonstruksi irisan dengan cara yang sama seperti CT.

Pada PET, positron yang dipancarkan bertemu elektron dan musnah menjadi dua foton 511 kiloelektronvolt yang terbang ke arah berlawanan. Detektor yang menangkap kedua foton pada saat yang hampir bersamaan tahu bahwa peristiwanya terjadi di sepanjang garis penghubung keduanya, tanpa perlu kolimator. Rekonstruksinya kembali menjadi masalah tomografi. Karena gambar PET menunjukkan fungsi tetapi hanya sedikit anatomi, ia sering digabung dengan CT dalam satu pemindai.

25.5 Deep learning untuk citra medis

Tugas-tugas utama di citra medis sama dengan di Bab 11: klasifikasi, deteksi, dan segmentasi, dengan rekonstruksi sebagai tugas tambahan yang khas. Klasifikasi lesi kulit dengan jaringan konvolusi yang dilatih pada lebih dari seratus ribu gambar mencapai kinerja setara dokter kulit dalam percobaan Esteva et al. (2017). Untuk segmentasi organ dan tumor, U-Net (Ronneberger et al., 2015) yang awalnya dirancang untuk citra biomedis menjadi standar. Jalur turun mengecilkan gambar dan menangkap konteks, jalur naik mengembalikan resolusinya, dan sambungan lompat dari setiap tingkat jalur turun ke tingkat yang sama di jalur naik membawa detail halus yang hilang saat gambar dikecilkan. Untuk volume tiga dimensi, semua konvolusi menjadi tiga dimensi, dan memori GPU menjadi batas, sehingga volume sering dipotong menjadi balok-balok kecil. nnU-Net (Isensee et al., 2021) menunjukkan bahwa sebagian besar keberhasilan datang dari menyetel resep dengan benar untuk setiap himpunan data, yaitu resolusi, normalisasi, augmentasi, dan arsitektur, dan bahwa resep itu bisa dibuat otomatis dari sifat-sifat datanya.

Data berlabel di bidang ini mahal, sehingga dua teknik dari bab-bab sebelumnya selalu dipakai. Augmentasi menambah variasi dengan memutar, mencerminkan, dan merenggangkan gambar secara elastis, serta mengubah kontras sedikit. Tetapi augmentasi harus masuk akal secara anatomi: membalik kiri dan kanan bisa mengubah makna sebuah temuan jantung. Transfer learning memulai dari jaringan yang sudah dilatih pada data besar. Model fondasi yang dilatih tanpa label pada jutaan citra medis semakin sering menjadi titik awal ini.

Segmentasi dinilai dengan koefisien Dice, $2|A \cap B|/(|A| + |B|)$, ukuran tumpang tindih antara daerah tebakan A dan daerah yang benar B . Dice berkerabat dekat dengan Tanimoto di Bab 23: kalau Tanimoto $T = |A \cap B|/|A \cup B|$, maka $\text{Dice} = 2T/(1 + T)$, karena $|A| + |B| = |A \cup B| + |A \cap B|$. Tumor yang kecil membuat Dice peka sekali. Untuk tumor berukuran 100 voxel, tebakan yang bergeser sehingga hanya 80 voxel yang tumpang tindih, dengan ukuran tebakan tetap 100, memberi Dice $160/200 = 0,8$. Pergeseran yang sama pada organ berukuran 10 000 voxel nyaris tidak terasa. Karena itu Dice juga dipakai langsung sebagai loss dalam bentuk lunak, dengan probabilitas sebagai

pengganti keanggotaan biner, sehingga kelas kecil tidak tenggelam di bawah latar belakang yang luas, masalah ketidakseimbangan kelas yang sama dengan di Bab 1.

Rekonstruksi juga mulai dibantu jaringan saraf. CT dosis rendah memakai lebih sedikit foton sehingga lebih aman, tetapi gambarnya lebih berisik. Jaringan bisa membersihkan noise setelah rekonstruksi, atau algoritme rekonstruksi iteratif bisa dibuka menjadi lapisan-lapisan yang dipelajari, seperti yang disinggung di Bab 18. Pendekatan yang paling hati-hati menjaga operator fisiknya, yaitu transformasi Radon, di dalam jaringan, sehingga hasilnya tetap konsisten dengan data yang diukur. Di semua penerapan ini, pengetahuan fisika tentang bagaimana citra terbentuk tetap menjadi pegangan untuk memeriksa apakah sebuah hasil masuk akal. Jaringan yang “membersihkan” gambar terlalu agresif bisa menghapus lesi kecil, atau lebih buruk, menggambar struktur yang tampak meyakinkan tetapi tidak ada.

25.6 Batas dan tanggung jawab

Kuliah AI dalam ilmu hayati dan kuliah tentang AI dan masyarakat sama-sama menyoroti batas sistem seperti ini. Model bisa belajar jalan pintas, misalnya penanda di sudut gambar yang hanya muncul pada pasien dari rumah sakit tertentu. Model yang dilatih di satu rumah sakit bisa jatuh kinerjanya di rumah sakit lain dengan pemindai berbeda. Dan model bisa bekerja lebih buruk untuk kelompok pasien yang kurang terwakili dalam data latih. Alat-alat XAI di Bab 17 membantu menemukan masalah seperti ini, dan regulasi di Bab 35 menetapkan kewajiban bagi sistem AI berisiko tinggi, termasuk alat diagnosis.

25.7 Perkembangan riset

Citra medis juga mendapat model fondasinya sendiri. RETFound (Zhou et al., 2023a) dilatih secara self-supervised pada 1,6 juta citra retina tanpa label, lalu disetel untuk mendeteksi penyakit mata dengan sedikit data berlabel. Yang lebih mengejutkan, model ini juga dipakai untuk meramal penyakit sistemik seperti gagal jantung dan serangan jantung dari citra retina. MedSAM (Ma et al., 2024) menyesuaikan Segment Anything untuk citra medis dengan lebih dari 1,5 juta pasangan citra dan topeng dari sepuluh modalitas, dan dievaluasi pada 86 tugas validasi internal serta 60 tugas eksternal.

Untuk teks medis, Med-PaLM (Singhal et al., 2023) mengevaluasi model bahasa besar pada soal-soal ujian kedokteran dan pertanyaan kesehatan sehari-hari. Akurasi pada soal pilihan ganda gaya ujian lisensi dokter Amerika mencapai 67,6 persen untuk Flan-PaLM, lebih dari 17 poin di atas rekor sebelumnya. Tetapi evaluasi oleh dokter memperlihatkan celah penting dalam ketepatan fakta, penalaran, dan potensi bahaya, dan versi yang disetel, Med-PaLM, masih kalah dari klinisi. Makalah ini menjadi contoh baik mengapa angka benchmark tidak cukup di bidang medis, sejalan dengan bagian batas dan tanggung jawab di atas.

CATATAN SUMBER. Bab ini mengikuti naskah dasar Deep Learning for Medical Imaging (osilasi dan gelombang, analisis Fourier, gelombang elektromagnetik, pencitraan sinar-X dan dosimetri,

tomografi komputer, kedokteran nuklir). Untuk rekonstruksi tomografi, Kak and Slaney (1988) adalah rujukan klasik.

Bagian V

AI untuk Simulasi

26 Fluida yang Ingin Kita Tiru

Kuliah metode numerik dalam mekanika fluida saya ikuti pada musim dingin 2025/26, diberikan oleh Stefan Pirker, yang kemudian juga membimbing tugas akhir saya. Gaya kuliahnya khas. Setiap topik dibuka dengan pertanyaan seperti “aliran apa yang terlintas di kepala Anda?”, dan beberapa slide meminta ChatGPT menulis puisi tentang neraca massa atau pidato perpisahan untuk persamaan Navier–Stokes, sebelum kembali ke persamaan yang sesungguhnya. Bab ini dan bab berikutnya merangkum kuliah itu. Bagian tentang simulasi ini memakai fluida sebagai contoh utama, karena di sanalah saya paling banyak bekerja, tetapi gagasan-gagasannya berlaku untuk banyak sistem fisika lain.

Sebelum membicarakan bagaimana machine learning bisa membantu simulasi, kita perlu tahu apa yang disimulasikan. Bab ini membahas persamaan yang mengatur aliran fluida dan panas, bilangan tak berdimensi yang menentukan wataknya, dan turbulensi, alasan utama mengapa simulasi aliran begitu mahal.

26.1 Wajah-wajah aliran

Aliran fluida punya banyak wajah. Madu yang dituang mengalir pelan dan teratur. Asap rokok naik lurus sebentar, lalu tiba-tiba pecah menjadi pusaran yang kacau. Awan di belakang pulau kecil di tengah samudra membentuk deretan pusaran yang bergantian ke kiri dan ke kanan, dan foto satelit pulau Alexander Selkirk di Pasifik selatan menjadi contoh yang sering dipakai. Jembatan Tacoma Narrows runtuh pada 1940 karena angin yang berinteraksi dengan getaran strukturnya. Semua ini diatur oleh persamaan yang sama. Yang membedakannya adalah perbandingan antara gaya-gaya yang bekerja.

Bilangan Reynolds menyatakan perbandingan antara inersia dan gesekan kental,

$$Re = \frac{UL}{\nu}, \quad (26.1)$$

dengan U kecepatan khas, L panjang khas, dan ν viskositas kinematik. Untuk Re di bawah sekitar sepuluh, aliran merayap dan gesekan mendominasi. Pada Re menengah, aliran laminar tetapi bisa melepaskan pusaran secara periodik. Pada Re tinggi, aliran menjadi turbulen. Air dengan viskositas kinematik sekitar $10^{-6} \text{ m}^2/\text{s}$ yang mengalir satu meter per detik di pipa berdiameter lima sentimeter punya Re sekitar lima puluh ribu, jauh di daerah turbulen. Madu, yang seribu kali lebih kental, dalam pipa dan kecepatan yang sama punya Re sekitar lima puluh, dan alirannya tetap teratur.

Untuk aliran yang digerakkan perbedaan temperatur, bilangan lain menjadi penting. Bilangan Prandtl, $Pr = \nu/\alpha$, membandingkan difusi momentum dengan difusi panas α . Untuk air sekitar tujuh, untuk udara sekitar 0,7. Bilangan Rayleigh,

$$Ra = \frac{g\beta\Delta TL^3}{\nu\alpha}, \quad (26.2)$$

mengukur kekuatan gaya apung dibanding gesekan dan difusi panas, dengan β koefisien muai dan ΔT beda temperatur. Lapisan fluida yang dipanaskan dari bawah tetap diam, dan panas hanya merambat secara konduksi, sampai Ra melewati sekitar 1700. Di atas nilai itu, sel-sel konveksi muncul, seperti yang terlihat di panci sup yang mulai bergolak. Bilangan Péclet membandingkan perpindahan oleh aliran dengan perpindahan oleh difusi. Dua aliran dengan bilangan-bilangan tak berdimensi yang sama berperilaku sama, sekecil atau sebesar apa pun ukurannya. Prinsip kesamaan ini memungkinkan percobaan dengan model kecil di laboratorium, dan juga menentukan seberapa jauh sebuah model pengganti bisa diharapkan berlaku.

26.2 Euler dan Lagrange

Kuliah membedakan dua cara memandang aliran. Dalam kerangka Lagrange, kita mengikuti sebuah paket fluida dan mencatat apa yang terjadi padanya, seperti menaruh pelampung di sungai dan mengikutinya dengan perahu. Dalam kerangka Euler, kita berdiri di tepi sungai dan mencatat apa yang lewat di satu titik tetap. Hukum fisika paling alami ditulis untuk paket, karena hukum Newton berlaku untuk benda yang kita ikuti. Tetapi simulasi paling mudah dikerjakan pada grid yang diam.

Jembatan di antara keduanya adalah turunan material. Kalau paket berada di $\mathbf{x}(t)$ dan bergerak dengan kecepatan $\mathbf{u}$, laju perubahan suatu besaran ϕ yang dialami paket, menurut aturan rantai, adalah

$$\frac{D\phi}{Dt} = \frac{\partial\phi}{\partial t} + \frac{d\mathbf{x}}{dt} \cdot \nabla\phi = \frac{\partial\phi}{\partial t} + \mathbf{u} \cdot \nabla\phi. \quad (26.3)$$

Suku pertama adalah perubahan di titik tetap, suku kedua perubahan karena paket berpindah ke tempat yang nilainya berbeda. Contohnya sehari-hari: di sebuah kota, udara bisa mendingin dengan cepat walau tidak ada udara yang kehilangan panas sama sekali, hanya karena angin membawa udara dingin dari utara.

26.3 Hukum kekekalan

Semua persamaan fluida adalah neraca. Untuk sebuah volume kecil di ruang, perubahan jumlah suatu besaran di dalamnya sama dengan apa yang mengalir masuk dikurangi apa yang mengalir keluar, ditambah apa yang diproduksi di dalamnya.

Penurunannya paling jelas dalam satu dimensi. Ambil potongan pipa antara x dan $x + \Delta x$ dengan penampang A . Massa di dalamnya adalah $\rho A \Delta x$. Laju massa yang masuk di kiri adalah $(\rho u)(x)A$ dan yang keluar di kanan $(\rho u)(x + \Delta x)A$. Neracanya

$$\frac{\partial(\rho A \Delta x)}{\partial t} = (\rho u)(x)A - (\rho u)(x + \Delta x)A. \quad (26.4)$$

Bagi dengan $A \Delta x$ dan biarkan Δx menuju nol. Ruas kanan menjadi turunan negatif, $\partial\rho/\partial t + \partial(\rho u)/\partial x = 0$. Dalam tiga dimensi, langkah yang sama untuk setiap arah memberi

$$\frac{\partial\rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0. \quad (26.5)$$

Untuk fluida tak termampatkan dengan massa jenis tetap, persamaan ini menjadi syarat $\nabla \cdot \mathbf{u} = 0$: apa yang masuk ke sebuah volume harus keluar lagi.

Resep yang sama berlaku untuk besaran apa pun. Untuk besaran ϕ per satuan massa, yang dibawa aliran, menyebar karena difusi, dan diproduksi oleh sumber, kita mendapat persamaan transport umum

$$\frac{\partial(\rho\phi)}{\partial t} + \nabla \cdot (\rho\mathbf{u}\phi) = \nabla \cdot (\Gamma\nabla\phi) + S_\phi, \quad (26.6)$$

dengan suku perubahan terhadap waktu, suku konveksi oleh aliran, suku difusi, dan suku sumber. Fluks difusi $-\Gamma\nabla\phi$ mengikuti hukum Fourier untuk panas atau hukum Fick untuk zat terlarut: besaran mengalir dari tempat yang banyak ke tempat yang sedikit, sebanding dengan gradiennya. Hampir semua persamaan di bab ini adalah kasus khusus bentuk ini, dan kekekalan massa sendiri adalah kasus $\phi = 1$ tanpa difusi dan sumber.

Kekekalan momentum adalah hukum Newton untuk sebuah paket fluida. Percepatan paket itu, yaitu turunan material kecepatannya, dikalikan massanya sama dengan gaya tekanan, gaya gesekan kental, dan gaya badan seperti gravitasi. Gesekan muncul dari tensor tegangan. Untuk fluida Newtonian, tegangan geser sebanding dengan laju deformasi, seperti pada aliran di antara dua pelat sejajar, di mana tegangan sama dengan viskositas dinamis μ dikali gradien kecepatan $\partial u / \partial y$. Untuk fluida tak termampatkan dengan viskositas tetap, divergensi tensor tegangan menyederhana menjadi $\mu\nabla^2\mathbf{u}$, dan kita mendapat persamaan Navier–Stokes,

$$\rho\left(\frac{\partial\mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u}\right) = -\nabla p + \mu\nabla^2\mathbf{u} + \rho\mathbf{g}. \quad (26.7)$$

Suku $(\mathbf{u} \cdot \nabla)\mathbf{u}$ adalah sumber semua kesulitan. Ia nonlinear, karena kecepatan mengangkut dirinya sendiri, dan dialah yang melahirkan turbulensi. Tekanan tidak punya persamaan evolusinya sendiri dalam aliran tak termampatkan. Ia muncul sebagai pengali yang menjaga agar $\nabla \cdot \mathbf{u} = 0$ terpenuhi di setiap saat, dan kopling inilah yang membuat solver numerik rumit (Bab 27).

Dari mana bilangan Reynolds datang? Tulis setiap besaran sebagai kelipatan skala khasnya: $x = Lx^*$, $\mathbf{u} = U\mathbf{u}^*$, $t = (L/U)t^*$, dan $p = \rho U^2 p^*$. Masukkan ke Persamaan (26.7) tanpa gravitasi, lalu bagi semuanya dengan $\rho U^2 / L$. Semua suku menjadi tak berdimensi, dan hanya satu koefisien yang tersisa:

$$\frac{\partial\mathbf{u}^*}{\partial t^*} + (\mathbf{u}^* \cdot \nabla^*)\mathbf{u}^* = -\nabla^* p^* + \frac{1}{\text{Re}} \nabla^{*2}\mathbf{u}^*. \quad (26.8)$$

Seluruh perbedaan antara aliran madu dan aliran air di pipa yang sama terkumpul dalam satu angka di depan suku gesekan. Kalau Re besar, gesekan tampak kecil, tetapi tidak pernah boleh dibuang sepenuhnya, karena di dekat dinding ia selalu penting.

Kekekalan energi untuk aliran dengan perpindahan panas, dalam bentuk temperatur, berbunyi

$$\rho c_p \left(\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T \right) = \nabla \cdot (\lambda \nabla T) + S_T, \quad (26.9)$$

dengan c_p kapasitas panas, λ konduktivitas termal, dan S_T sumber panas seperti reaksi kimia atau radiasi. Kekekalan spesies untuk fraksi massa c_i sebuah zat terlarut

mengikuti bentuk yang sama, dengan difusivitas massa sebagai pengganti konduktivitas dan laju reaksi sebagai sumber.

Ketika perbedaan temperatur kecil, perubahan massa jenis bisa diabaikan di semua tempat kecuali di suku gravitasi. Pendekatan Boussinesq menulis $\rho = \rho_0(1 - \beta(T - T_0))$ hanya di suku gaya badan. Fluida yang lebih panas menjadi sedikit lebih ringan dan naik. Untuk air di sekitar suhu kamar, β sekitar 2×10^{-4} per kelvin, sehingga beda temperatur sepuluh kelvin hanya mengubah massa jenis sekitar 0,2 persen. Perubahan sekecil itu tidak terasa dalam neraca massa, tetapi dikalikan gravitasi dan dijumlahkan di seluruh kolom fluida, cukup untuk menggerakkan seluruh aliran. Pendekatan ini menjelaskan sel konveksi di panci yang dipanaskan, angin laut, dan pancaran air hangat yang naik di dalam tangki.

26.4 Analisis dimensi

Penurunan bilangan Reynolds di atas adalah contoh alat yang lebih umum, analisis dimensi. Teorema Buckingham II menyatakan bahwa hubungan fisis antara n besaran yang melibatkan k dimensi dasar, seperti massa, panjang, dan waktu, selalu bisa ditulis sebagai hubungan antara $n - k$ kelompok tak berdimensi.

Gaya hambat F pada bola berdiameter D yang bergerak dengan kecepatan U dalam fluida dengan massa jenis ρ dan viskositas μ melibatkan lima besaran dan tiga dimensi dasar, sehingga hanya tersisa dua kelompok. Pilihan yang lazim adalah koefisien hambat $C_D = F/(\frac{1}{2}\rho U^2 A)$, dengan A luas penampang, dan bilangan Reynolds. Fisika apa pun yang terjadi, hubungannya pasti berbentuk $C_D = g(\text{Re})$. Satu kurva menggantikan tabel lima dimensi. Untuk aliran merayap, Stokes menurunkan $F = 3\pi\mu DU$, dan memasukkannya memberi

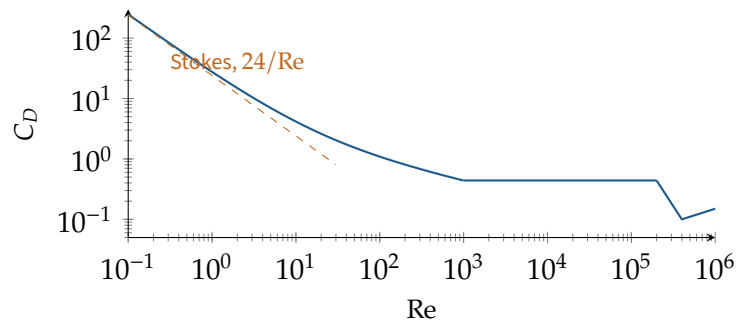
$$C_D = \frac{3\pi\mu DU}{\frac{1}{2}\rho U^2 \cdot \pi D^2/4} = \frac{24\mu}{\rho U D} = \frac{24}{\text{Re}}. \quad (26.10)$$

Pada Re tinggi, gaya hambat didominasi tekanan di belakang bola dan viskositas hampir tidak berperan, sehingga C_D mendekati konstan, sekitar 0,4 sampai 0,5 untuk bola, sampai lapisan batas menjadi turbulen dan koefisiennya tiba-tiba turun (Gambar 26.1). Waktu respons partikel τ_p di Bab 27 turun langsung dari hukum Stokes ini.

Analisis dimensi juga menjelaskan pancaran apung, aliran yang menjadi pokok tugas akhir saya. Untuk plume dari sumber titik yang terus memasok gaya apung, satu-satunya besaran sumber yang penting jauh dari sumber adalah fluks gaya apung F_0 , dengan satuan m^4/s^3 . Di ketinggian z , kecepatan naik hanya bisa bergantung pada F_0 dan z , dan satu-satunya kombinasi bersatuan kecepatan adalah

$$w \sim \left(\frac{F_0}{z}\right)^{1/3}. \quad (26.11)$$

Lebar plume, yang bersatuan panjang, hanya bisa sebanding dengan z , sehingga plume melebar seperti kerucut dengan sudut tetap. Model klasik Morton et al. (1956) mewujudkan skala-skala ini dengan satu asumsi tambahan: laju fluida sekitar yang terseret masuk ke plume sebanding dengan kecepatan plume di ketinggian itu. Untuk



GAMBAR 26.1. Bentuk kurva koefisien hambat bola terhadap bilangan Reynolds (ilustrasi). Pada Re kecil berlaku hukum Stokes, pada Re menengah sampai tinggi koefisiennya hampir tetap, lalu turun tajam ketika lapisan batas menjadi turbulen.

tangki yang terbatas, plume akhirnya menabrak tutup dan membentuk lapisan yang terstratifikasi, dan di situlah skala-skala sederhana ini berakhir dan simulasi dibutuhkan.

26.5 Batas dan awal

Persamaan saja tidak menentukan sebuah aliran. Kita butuh syarat batas dan syarat awal. Di dinding padat, fluida menempel dan kecepatannya nol, syarat yang disebut *no-slip*. Di saluran masuk, kecepatan atau laju aliran ditentukan. Di saluran keluar, tekanan biasanya ditetapkan. Untuk temperatur, dinding bisa diberi temperatur tetap, fluks panas tetap, atau dibuat adiabatik, yaitu tanpa perpindahan panas sama sekali. Pilihan syarat batas sering lebih menentukan hasil simulasi daripada skema numeriknya, dan sebagian besar galat dalam membandingkan simulasi dengan percobaan berasal dari sini.

Syarat *no-slip* melahirkan lapisan batas, daerah tipis di dekat dinding tempat kecepatan naik dari nol ke kecepatan aliran bebas. Di dalam lapisan ini, gesekan dan inersia sama pentingnya. Menyeimbangkan keduanya, $U^2/L \sim \nu U/\delta^2$, memberi tebal lapisan batas laminar $\delta \sim L/\sqrt{Re}$. Untuk pelat sepanjang satu meter dalam aliran udara dengan Re sejuta, lapisan batasnya hanya sekitar satu milimeter. Semua gesekan dan perpindahan panas ke dinding terjadi di lapisan setipis itu, sehingga grid di dekat dinding harus jauh lebih halus daripada di tempat lain.

26.6 Turbulensi dan skalanya

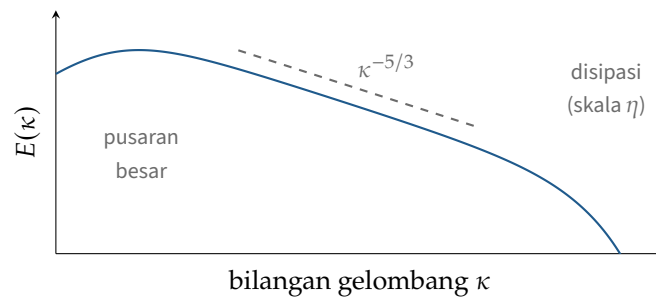
Aliran turbulen tampak acak, tetapi punya struktur statistik yang dipahami cukup baik. Kuliah membahas teori Kolmogorov. Pusaran besar, seukuran L dengan kecepatan U , menerima energi dari aliran rata-rata. Mereka tidak stabil dan pecah menjadi pusaran yang lebih kecil, yang pecah lagi, dan seterusnya. Energi mengalir menuruni kaskade ini tanpa banyak hilang, sampai pusaran cukup kecil untuk diredam viskositas menjadi panas. Seperti air terjun bertingkat, air mengalir dari kolam besar ke kolam-kolam yang semakin kecil, dan baru menguap di kolam terkecil.

Laju energi yang memasuki kaskade per satuan massa, ε , ditentukan pusaran besar:

mereka kehilangan energi kinetik sekitar U^2 dalam waktu putar sekitar L/U , sehingga $\varepsilon \sim U^3/L$. Hipotesis Kolmogorov menyatakan bahwa pusaran terkecil hanya “tahu” dua hal, yaitu ε dan ν . Analisis dimensi lalu menentukan ukurannya. Satuan ν adalah m^2/s dan satuan ε adalah m^2/s^3 . Satu-satunya kombinasi yang menghasilkan panjang adalah

$$\eta = \left(\frac{\nu^3}{\varepsilon} \right)^{1/4}. \quad (26.12)$$

Mengganti ε dengan U^3/L memberi $\eta/L \sim (\nu/UL)^{3/4} = \text{Re}^{-3/4}$. Untuk menyelesaikan semua pusaran dalam kotak tiga dimensi berukuran L , kita butuh sekitar $(L/\eta)^3 \sim \text{Re}^{9/4}$ titik grid (Pope, 2000). Dengan Re sejuta, itu sekitar 3×10^{13} titik, dan langkah waktunya juga harus ikut mengecil. Di antara skala besar dan skala terkecil terdapat rentang inersia, di mana spektrum energi hanya bergantung pada ε dan bilangan gelombang κ . Analisis dimensi yang sama memberi $E(\kappa) \sim \varepsilon^{2/3} \kappa^{-5/3}$, hukum lima pertiga yang terkenal (Gambar 26.2), yang sudah dikonfirmasi berkali-kali dalam percobaan.



GAMBAR 26.2. Bentuk khas spektrum energi turbulen. Energi masuk di skala besar, mengalir melalui rentang inersia dengan kemiringan $-5/3$, lalu diredam di skala Kolmogorov.

26.7 Mengapa simulasi aliran mahal

Menyelesaikan semua skala secara langsung disebut *direct numerical simulation*. Hitungan di atas menunjukkan mengapa ini hanya mungkin untuk Re yang sederhana. Aliran industri dengan Re jutaan jauh melampaui komputer mana pun. Simulasi praktis karena itu memodelkan efek skala kecil alih-alih menyelesaikannya. Pendekatan RANS hanya menyelesaikan aliran rata-rata dan memodelkan seluruh turbulensi. LES menyelesaikan pusaran besar dan memodelkan yang lebih kecil dari grid. Keduanya dibahas di Bab 27.

Pemodelan itulah yang membuka pintu bagi machine learning: sebagai model untuk skala kecil, sebagai pengganti seluruh solver, atau sebagai cara mempercepat solver yang ada. Bab-bab berikutnya membahas ketiganya.

CATATAN SUMBER. Bab ini mengikuti kuliah Numerical Methods in Fluid Mechanics bagian pengantar, persamaan atur (kerangka Euler dan Lagrange, neraca massa, momentum, energi, spesies, pendekatan Boussinesq), dan teori turbulensi Kolmogorov. Buku rujukan yang baik adalah Versteeg and Malalasekera (2007), Ferziger et al. (2020), dan untuk turbulensi Pope (2000).

27 Dari Persamaan ke Solver

Setelah persamaan-persamaan diatur, kuliah metode numerik fluida bergerak ke cara menyelesaikannya: metode volume hingga, kopling tekanan dan kecepatan, turbulensi dalam tiga pertemuan, perpindahan panas dan spesies, aliran multifase, dan satu pertemuan khusus tentang recurrence CFD. Bab ini merangkum semuanya kecuali yang terakhir, yang mendapat babnya sendiri di Bab 33.

27.1 Metode volume hingga

Metode volume hingga membagi domain menjadi sel-sel kecil dan menegakkan neraca Persamaan (26.6) untuk setiap sel. Mengintegrasikan persamaan atas volume sebuah sel, lalu memakai teorema divergensi Gauss, mengubah integral divergensi menjadi jumlah fluks yang melewati sisi-sisi sel:

$$\frac{d}{dt} \int_V \rho \phi \, dV + \sum_{\text{sisi } f} (\rho \mathbf{u} \phi - \Gamma \nabla \phi)_f \cdot \mathbf{n}_f A_f = \int_V S_\phi \, dV. \quad (27.1)$$

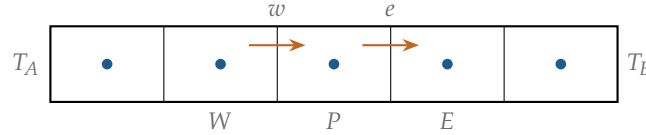
Keindahan bentuk ini adalah sifat konservatifnya. Fluks yang keluar dari satu sel lewat sebuah sisi adalah persis fluks yang masuk ke sel tetangganya lewat sisi yang sama, sehingga tidak ada massa, momentum, atau energi yang hilang atau muncul di dalam domain kecuali lewat batas dan sumber. Di Bab 13 kita melihat bahwa bentuk ini adalah message passing: setiap sel mengumpulkan “pesan” fluks dari tetangganya.

Cara kerjanya paling mudah dilihat pada contoh yang bisa dihitung dengan tangan. Ambil batang sepanjang setengah meter yang dibagi lima sel sama panjang, $\Delta x = 0,1$ m, dengan temperatur 100 di ujung kiri dan 500 di ujung kanan, tanpa aliran dan tanpa sumber (Gambar 27.1). Untuk sel dalam P dengan tetangga barat W dan timur E , fluks difusi lewat sisi timur adalah $\Gamma A(T_E - T_P)/\Delta x$ dan lewat sisi barat $\Gamma A(T_P - T_W)/\Delta x$. Neraca keadaan tunak menyamakan keduanya, sehingga dengan $D = \Gamma A/\Delta x$,

$$2D T_P = D T_W + D T_E. \quad (27.2)$$

Untuk sel pertama, batasnya hanya setengah sel jauhnya, sehingga fluks di sisi barat menjadi $2D(T_P - T_A)$, dan persamaannya $3T_1 = T_2 + 2T_A$. Sel terakhir mendapat bentuk yang sama dengan T_B . Kelima persamaan ini membentuk sistem tridiagonal, karena setiap sel hanya bicara dengan dua tetangganya, dan bisa diselesaikan dalam satu sapuan maju dan satu sapuan mundur dengan algoritme Thomas (TDMA) yang dibahas di kuliah. Hasilnya 140, 220, 300, 380, dan 460. Memeriksa sel pertama: $3 \cdot 140 = 420 = 220 + 200$. Profilnya linear, persis seperti solusi eksak untuk konduksi tunak tanpa sumber.

Nilai di sisi sel harus diperkirakan dari nilai di pusat sel. Untuk suku difusi, beda pusat seperti di atas bekerja baik. Untuk suku konveksi, pilihannya lebih rumit.



GAMBAR 27.1. Grid volume hingga satu dimensi. Nilai disimpan di pusat sel (W , P , E), fluks dihitung di sisi sel (w , e). Fluks yang keluar dari P lewat e adalah fluks yang masuk ke E .

Tambahkan aliran dengan fluks massa $F = \rho u A$ ke contoh tadi. Dengan beda pusat, nilai di sisi timur diambil sebagai rata-rata, $\phi_e = (\phi_P + \phi_E)/2$, dan setelah semua suku dikumpulkan, persamaan sel berbentuk $a_P \phi_P = a_W \phi_W + a_E \phi_E$ dengan

$$a_W = D + \frac{F}{2}, \quad a_E = D - \frac{F}{2}. \quad (27.3)$$

Kalau F/D , yaitu bilangan Péclet sel $Pe = \rho u \Delta x / \Gamma$, lebih besar dari dua, koefisien a_E menjadi negatif. Artinya, menaikkan nilai di hilir justru menurunkan nilai di sel ini, sesuatu yang tidak pernah terjadi secara fisis, dan solusinya mulai berosilasi. Beda pusat akurat orde dua, tetapi hanya bisa dipakai kalau grid cukup halus untuk menjaga Pe di bawah dua.

Skema *upwind* mengambil nilai sisi dari sel di hulu aliran: untuk aliran ke kanan, $\phi_e = \phi_P$. Koefisiennya menjadi $a_W = D + F$ dan $a_E = D$, selalu positif, sehingga tidak ada osilasi. Harganya adalah akurasi. Uraian Taylor menunjukkan apa yang hilang: nilai sebenarnya di sisi adalah $\phi_e = \phi_P + \frac{\Delta x}{2} \partial \phi / \partial x + \dots$, sehingga *upwind* membuang suku $\rho u \frac{\Delta x}{2} \partial \phi / \partial x$ dari fluks. Suku yang dibuang berbentuk persis seperti fluks difusi dengan koefisien

$$\Gamma_{\text{num}} = \frac{\rho u \Delta x}{2}. \quad (27.4)$$

Skema ini diam-diam menambahkan difusi. Untuk air yang mengalir satu meter per detik di grid satu sentimeter, viskositas kinematik semunya $u \Delta x / 2 = 0,005 \text{ m}^2/\text{s}$, lima ribu kali viskositas air yang sebenarnya. Gejala ini disebut *numerical diffusion* atau *false diffusion*, dan ia meratakan setiap gradien tajam. Skema orde lebih tinggi seperti second-order *upwind* dan QUICK, serta skema dengan limiter, mencari jalan tengah: akurat di daerah yang mulus, dan kembali ke perilaku *upwind* di dekat lompatan agar tidak berosilasi. Kuliah memperlihatkan contohnya pada perambatan gelombang permukaan, di mana antarmuka air dan udara mengabur dengan skema orde satu dan tetap tajam dengan skema yang lebih baik.

Untuk waktu, metode eksplisit menghitung keadaan baru langsung dari keadaan lama. Untuk konveksi murni dengan *upwind*, langkah eksplisitnya

$$\phi_i^{n+1} = (1 - C) \phi_i^n + C \phi_{i-1}^n, \quad C = \frac{u \Delta t}{\Delta x}. \quad (27.5)$$

Kalau $0 \leq C \leq 1$, nilai baru adalah rata-rata berbobot dua nilai lama, sehingga tidak mungkin melampaui nilai terbesar atau jatuh di bawah nilai terkecil yang sudah ada. Kalau $C > 1$, bobot $1 - C$ menjadi negatif, dan galat kecil diperbesar di setiap langkah sampai simulasi meledak. Inilah syarat Courant–Friedrichs–Lewy (Courant et al., 1928),

$CFL = u\Delta t/\Delta x \leq C_{\max}$, yang secara intuitif berarti informasi tidak boleh melompati lebih dari sekitar satu sel dalam satu langkah. Dengan $u = 1$ m/s dan $\Delta x = 1$ cm, langkah waktu eksplisit paling besar sepersekian ratus detik. Metode implisit menghitung fluks dengan nilai di langkah baru, sehingga harus menyelesaikan sistem persamaan untuk seluruh grid di setiap langkah. Mahal per langkah, tetapi stabil untuk langkah yang jauh lebih besar. Stabil belum berarti akurat: langkah waktu yang besar tetap melewatkan dinamika yang lebih cepat dari langkah itu.

27.2 Tekanan dan kecepatan

Untuk aliran tak termampatkan, tekanan tidak punya persamaannya sendiri. Ia harus diatur sedemikian rupa sehingga medan kecepatan yang dihasilkan persamaan momentum memenuhi kekekalan massa. Algoritme SIMPLE (Patankar, 1980; Patankar and Spalding, 1972), yang dibahas langkah demi langkah di kuliah, menyelesaikan masalah ini secara iteratif. Pertama, tebak medan tekanan dan selesaikan persamaan momentum untuk mendapat kecepatan sementara. Kecepatan ini umumnya belum memenuhi kekekalan massa. Kedua, tulis hubungan antara koreksi kecepatan dan koreksi tekanan. Ketiga, masukkan hubungan itu ke persamaan kekekalan massa, sehingga diperoleh persamaan untuk koreksi tekanan. Selesaikan, perbarui tekanan dan kecepatan, lalu ulangi sampai konvergen. Pendekatan yang menyelesaikan persamaan-persamaan ini bergantian disebut *segregated solver*, sedangkan solver *coupled* menyelesaikannya bersama dalam satu sistem besar.

Langkah kedua adalah inti algoritmenya. Persamaan momentum diskret untuk sebuah sel berbentuk $a_p u_p = \sum a_{nb} u_{nb} + A \Delta p + b$, dengan Δp beda tekanan di kedua sisi sel. Kalau tekanan dikoreksi dengan p' dan kecepatan dengan u' , SIMPLE mengabaikan sumbangan koreksi kecepatan tetangga, sehingga yang tersisa adalah hubungan sederhana $u'_p = d_p \Delta p'$ dengan $d_p = A/a_p$. Penyederhanaan inilah yang membuat metodenya bekerja dan sekaligus membuatnya lambat: koreksi tekanan yang dihitung cenderung terlalu besar. Karena itu tekanan baru hanya diambil sebagian, $p = p^* + \alpha_p p'$, dengan faktor *under-relaxation* α_p yang sering sekitar 0,3. Varian seperti SIMPLEC dan PISO memperbaiki penyederhanaan itu sehingga butuh lebih sedikit relaksasi.

Kuliah menekankan perbedaan antara konvergensi numerik dan konvergensi fisis. Residu yang turun beberapa orde menunjukkan bahwa sistem persamaan diskret sudah terselesaikan. Itu belum berarti besaran yang kita pedulikan, misalnya gaya hambat atau temperatur di sebuah titik, sudah tidak berubah lagi, apalagi bahwa hasilnya benar secara fisis. Memantau besaran fisis dan menguji kebebasan hasil terhadap ukuran grid adalah bagian wajib setiap simulasi.

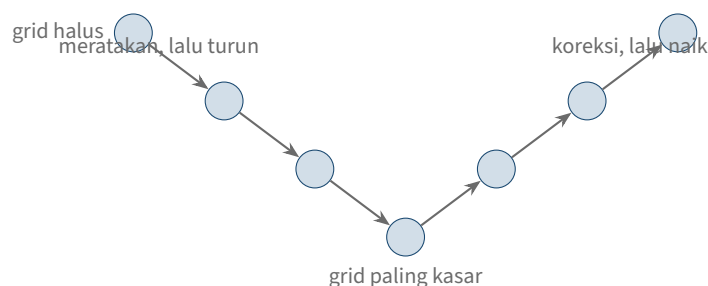
27.3 Menyelesaikan sistem linear yang besar

Setiap langkah implisit dan setiap koreksi tekanan di SIMPLE berujung pada sistem linear $A\phi = b$ dengan jutaan baris. Matriksnya jarang, karena setiap sel hanya bicara dengan tetangganya, sehingga eliminasi Gauss yang mengisi matriks dengan nol-nol yang berubah menjadi bukan nol tidak praktis. Kuliah membahas metode iteratif.

Metode Jacobi menyelesaikan persamaan setiap sel untuk nilai sel itu sendiri, dengan nilai tetangga dari iterasi sebelumnya: $\phi_P^{(k+1)} = (a_W \phi_W^{(k)} + a_E \phi_E^{(k)} + b_P) / a_P$. Gauss-Seidel memakai nilai tetangga yang baru saja diperbarui di iterasi yang sama, sehingga informasi menyebar lebih cepat. Ambil sistem kecil $4\phi_1 - \phi_2 = 3$ dan $-\phi_1 + 4\phi_2 = 3$, yang solusinya $\phi_1 = \phi_2 = 1$. Mulai dari nol, Jacobi memberi $(0,75; 0,75)$, lalu $(0,9375; 0,9375)$, lalu $(0,984; 0,984)$: galatnya menyusut empat kali per iterasi. Gauss-Seidel memberi $\phi_1 = 0,75$, lalu langsung memakainya untuk $\phi_2 = (3 + 0,75)/4 = 0,9375$, dan pada iterasi kedua sudah $(0,984; 0,996)$: galatnya menyusut enam belas kali per iterasi.

Laju susut itu adalah jari-jari spektral matriks iterasi, nilai eigen terbesarnya dalam nilai mutlak. Untuk contoh di atas nilainya $1/4$ untuk Jacobi dan $1/16$ untuk Gauss-Seidel. Masalahnya, untuk persamaan Poisson di grid halus, jari-jari spektral itu mendekati satu, dan semakin halus grid, semakin dekat. Analisis Fourier seperti di Bab 28 menunjukkan sebabnya: iterasi lokal meredam galat berfrekuensi tinggi dengan cepat, tetapi galat yang halus, yang merentang banyak sel, hampir tidak berkurang, karena setiap sel hanya melihat tetangganya dan informasi merambat satu sel per iterasi.

Multigrid memanfaatkan kelemahan itu. Setelah beberapa iterasi meratakan galat frekuensi tinggi, sisa galat yang halus dipindahkan ke grid yang lebih kasar. Di sana, galat yang tadinya halus menjadi relatif lebih bergelombang terhadap ukuran sel, sehingga kembali mudah diredam. Proses ini diulang turun sampai grid kasar, lalu koreksinya dibawa naik kembali (Gambar 27.2). Dengan siklus seperti ini, biaya menyelesaikan sistem menjadi sebanding dengan jumlah sel, batas terbaik yang mungkin. Ide piramida yang sama muncul di visi komputer dan di U-Net (Bab 11). Keluarga lain, metode Krylov seperti conjugate gradient untuk matriks simetris dan GMRES untuk yang tidak simetris, mencari solusi terbaik di ruang yang direntang $b, Ab, A^2b, \dots$, dan dalam praktik digabung dengan multigrid sebagai prakondisi.



GAMBAR 27.2. Siklus V pada multigrid. Di setiap tingkat, iterasi lokal meredam galat yang bergelombang. Galat yang halus diteruskan ke grid lebih kasar, diselesaikan di sana, lalu koreksinya dibawa kembali ke grid halus.

27.4 Turbulensi

27.4.1 Rata-rata Reynolds

Aliran turbulen penuh dengan fluktuasi acak di semua skala. Untuk teknik, biasanya cukup mengetahui rata-ratanya. Dekomposisi Reynolds menulis setiap besaran sebagai rata-rata ditambah fluktuasi, $u = \bar{u} + u'$. Ketika dekomposisi ini dimasukkan ke

persamaan Navier–Stokes dan dirata-ratakan, hampir semuanya rapi kecuali satu suku baru dari nonlinearitas, yaitu *Reynolds stress* $-\rho \overline{u'_i u'_j}$. Suku ini tidak diketahui dari besaran rata-rata, sehingga persamaannya tidak tertutup. Inilah *closure problem* turbulensi.

Boussinesq mengusulkan agar Reynolds stress diperlakukan seperti tegangan kental, dengan sebuah viskositas turbulen μ_t yang jauh lebih besar dari viskositas molekuler. Masalahnya bergeser menjadi menentukan μ_t . Model k - ε (Launder and Spalding, 1974) menyelesaikan dua persamaan transport tambahan, untuk energi kinetik turbulen k dan laju disipasinya ε , lalu menulis $\mu_t = \rho C_\mu k^2 / \varepsilon$. Konstanta-konstantanya ditetapkan dari aliran sederhana yang dipahami dengan baik, seperti peluruhan turbulensi di belakang grid dan penyebaran jet. Model k - ω SST (Menter, 1994) memakai formulasi k - ω di dekat dinding dan beralih ke k - ε di aliran bebas, dan menjadi salah satu model paling banyak dipakai di industri. Ketika anisotropi turbulensi penting, seperti di dalam siklon gas atau pusaran ujung sayap, model Reynolds stress menyelesaikan persamaan transport untuk setiap komponen tegangan.

27.4.2 Di dekat dinding

Lapisan batas turbulen punya struktur berlapis yang hampir universal (Gambar 27.3). Jarak dan kecepatan dinyatakan dalam satuan dinding: kecepatan gesek $u_\tau = \sqrt{\tau_w / \rho}$ dari tegangan dinding τ_w , jarak tak berdimensi $y^+ = y u_\tau / \nu$, dan kecepatan tak berdimensi $u^+ = u / u_\tau$. Sangat dekat dengan dinding, di bawah y^+ sekitar lima, gesekan kental mendominasi dan $u^+ = y^+$. Lebih jauh, kira-kira dari $y^+ = 30$, berlaku hukum logaritmik

$$u^+ = \frac{1}{\kappa} \ln y^+ + B, \quad (27.6)$$

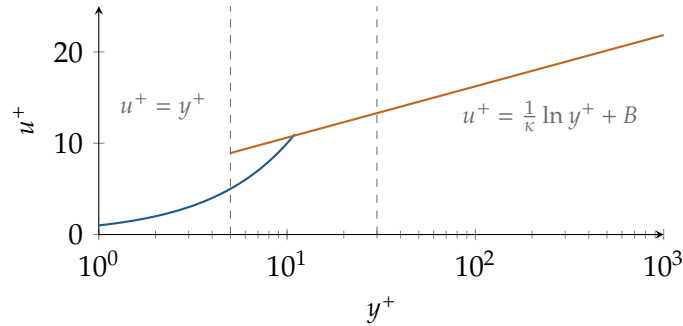
dengan konstanta von Kármán $\kappa \approx 0,41$ dan $B \approx 5$. Di antaranya ada lapisan penyangga. Untuk air dengan $u_\tau = 5$ cm/s, $y^+ = 1$ berarti hanya $\nu / u_\tau = 20$ mikrometer dari dinding. Menyelesaikan lapisan setipis ini dengan grid mahal, sehingga banyak simulasi memakai *wall function*: sel pertama sengaja diletakkan di daerah logaritmik, dan hukum di atas dipakai untuk menghitung tegangan dinding tanpa menyelesaikan lapisan di bawahnya. Kuliah menekankan bahwa *wall function* hanya sah kalau y^+ sel pertama memang berada di rentang itu, sehingga memeriksa y^+ adalah bagian dari memeriksa grid.

27.4.3 Large eddy simulation

Large eddy simulation (LES) menyelesaikan pusaran besar secara langsung dan hanya memodelkan pusaran yang lebih kecil dari ukuran grid. Model Smagorinsky (SMAGORINSKY, 1963) memberi viskositas sub-grid $\mu_{SGS} = \rho (C_s \Delta)^2 |\bar{S}|$, sebanding dengan kuadrat ukuran grid dan laju deformasi yang terselesaikan. LES jauh lebih mahal dari RANS karena harus berjalan dalam waktu dengan grid halus, tetapi jauh lebih murah dari DNS, dan ia menangkap struktur aliran tak tunak yang hilang dalam rata-rata.

27.5 Panas, spesies, dan reaksi

Dalam aliran turbulen, panas dan zat terlarut juga diaduk oleh pusaran. Konduktivitas efektif menjadi jumlah konduktivitas molekuler dan sumbangan turbulen, $\lambda_{\text{eff}} =$



GAMBAR 27.3. Hukum dinding. Lapisan kental ($y^+ < 5$) mengikuti garis lurus, daerah logaritmik ($y^+ > 30$) mengikuti hukum log, dan di antaranya ada lapisan penyangga.

$\lambda + \mu_t c_p / \text{Pr}_t$, dengan bilangan Prandtl turbulen biasanya sekitar 0,85. Radiasi menambah suku sumber dalam persamaan energi, dan reaksi kimia menambah suku sumber dalam persamaan spesies dan energi sekaligus. Kuliah menyinggung contoh-contoh industri, dari nosel Laval sampai pemanasan logam cair, untuk menunjukkan bahwa sebagian besar kerja pemodelan ada pada suku-suku sumber ini, bukan pada solvernya.

27.6 Aliran multifase

Banyak aliran industri melibatkan lebih dari satu fase: gelembung gas di dalam cairan, butiran di dalam gas, tetesan di dalam udara. Kuliah membahas beberapa keluarga pendekatan.

Metode *volume of fluid* (Hirt and Nichols, 1981) melacak antarmuka antara dua fluida yang tidak bercampur dengan sebuah fraksi volume α yang bernilai satu di satu fase dan nol di fase lain. Fraksi itu hanya dibawa aliran, $\partial\alpha/\partial t + \nabla \cdot (\alpha \mathbf{u}) = 0$, sehingga persoalan numeriknya adalah menjaga lompatan tajam dari nol ke satu tetap tajam, persis masalah numerical diffusion di atas. Kuliah membahas skema donor-akseptor dan rekonstruksi geometris antarmuka untuk keperluan ini. Massa jenis dan viskositas dirata-ratakan menurut fraksi itu, dan tegangan permukaan masuk lewat lompatan tekanan Young-Laplace di antarmuka, $\Delta p = \sigma \kappa$, dengan κ kelengkungan antarmuka. Contohnya di kuliah termasuk gelombang di tangki yang bergoyang dan aliran kapiler di saluran mikro.

Untuk partikel atau tetesan yang tersebar, pendekatan Euler-Lagrange melacak setiap partikel, atau sekelompok partikel yang mewakilinya, dalam kerangka Lagrange, sementara fase kontinu diselesaikan di grid. Kopling bisa satu arah, ketika partikel hanya dibawa aliran, atau dua arah, ketika partikel juga memberi gaya balik ke fluida. Seberapa setia partikel mengikuti aliran ditentukan bilangan Stokes, perbandingan waktu respons partikel $\tau_p = \rho_p d^2 / (18\mu)$ dengan waktu khas aliran L/U . Waktu respons ini turun dari hambatan Stokes pada bola kecil, $3\pi\mu d$ kali selisih kecepatan, dibagi massa bola. Tetesan air berdiameter 10 mikrometer di udara punya $\tau_p \approx 0,3$ milidetik. Dalam aliran satu meter per detik melewati benda satu sentimeter, bilangan Stokesnya sekitar 0,03, dan tetesan itu berbelok bersama udara. Tetesan seratus mikrometer

punya τ_p seratus kali lebih besar, bilangan Stokes sekitar tiga, dan tetesan itu menabrak benda alih-alih berbelok. Perbedaan ini menentukan, misalnya, apakah sebuah filter menangkap partikel atau tidak. Untuk aliran butiran padat, metode elemen diskret (Cundall and Strack, 1979) menyelesaikan gerak setiap butir dan tumbukannya, dan gabungannya dengan CFD dipakai untuk unggun terfluidisasi dan pengangkutan pneumatik. Ketika partikel terlalu banyak untuk dilacak satu per satu, model dua fluida memperlakukan kedua fase sebagai kontinum yang saling menembus, digandeng oleh fraksi volume dan pertukaran momentum.

27.7 Di mana machine learning masuk

Bab ini memperlihatkan tiga titik lemah simulasi aliran. Model penutup, seperti viskositas turbulen dan model sub-grid, dibangun dari asumsi yang sering tidak berlaku. Simulasi yang cukup teliti mahal sekali. Dan setiap kasus baru, dengan geometri atau syarat batas berbeda, harus disimulasikan dari awal. Machine learning masuk lewat ketiga pintu itu: model penutup yang dipelajari dari data simulasi teliti (Bab 32), model pengganti yang menggantikan solver (Bab 29 dan 30), dan metode yang memanfaatkan pola berulang agar simulasi panjang bisa diekstrapolasi dengan murah (Bab 33).

27.8 Perkembangan riset

Gerakan *differentiable physics* membawa solver CFD ke kerangka yang sama dengan jaringan saraf. JAX-Fluids (Bezgin et al., 2022) adalah solver untuk aliran termampatkan dua fase yang seluruhnya ditulis dalam JAX, sehingga bisa diturunkan secara otomatis dari keluaran sampai ke parameter dan syarat awal. Komponen machine learning, misalnya skema rekonstruksi fluks yang dipelajari, bisa dimasukkan ke dalam solver dan dioptimasi ujung ke ujung, persis gagasan solver-in-the-loop di Bab 32. Solver yang bisa diturunkan juga berguna tanpa machine learning: gradien hasil simulasi terhadap parameter desain memungkinkan optimasi bentuk dan kalibrasi model dengan cara yang sama seperti melatih jaringan.

CATATAN SUMBER. Bab ini mengikuti kuliah Numerical Methods in Fluid Mechanics bagian volume hingga, kopling tekanan dan kecepatan, turbulensi, panas dan spesies, serta multifase. Bacaan pendamping: Versteeg and Malalasekera (2007), Ferziger et al. (2020), Patankar (1980), dan Pope (2000).

28 Fisika Komputasi

Kuliah fisika komputasi saya ikuti pada musim dingin 2025/26. Catatan kuliahnya berupa tulisan tangan, tiga belas pertemuan yang masing-masing membahas satu alat numerik dasar: interpolasi, transformasi Fourier dan FFT, mencari akar, persamaan diferensial biasa, metode simplektik, masalah nilai batas, masalah nilai eigen, persamaan diferensial parsial yang bergantung waktu, metode split-operator, lalu Monte Carlo. Alat-alat ini tersembunyi di dalam hampir setiap simulasi, dan juga di dalam banyak algoritme machine learning. Setiap alat punya sifat yang menentukan kapan ia bisa dipercaya, dan bab ini mencoba memperlihatkan sifat-sifat itu dengan contoh yang bisa dihitung.

28.1 Interpolasi dan Fourier

Interpolasi mencari fungsi mulus yang melewati titik-titik data yang diketahui. Polinomial interpolasi Lagrange melewati n titik (x_i, y_i) dengan tepat:

$$p(x) = \sum_{i=1}^n y_i \ell_i(x), \quad \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}. \quad (28.1)$$

Setiap ℓ_i bernilai satu di x_i dan nol di semua titik lain, sehingga $p(x_i) = y_i$ dengan sendirinya. Untuk dua titik, rumus ini adalah garis lurus biasa. Untuk derajat tinggi pada titik berjarak sama, polinomialnya berosilasi liar di dekat tepi selang, gejala Runge yang terkenal. Fungsi semulus $1/(1 + 25x^2)$ di selang $[-1, 1]$ sudah cukup untuk memperlihatkan: menambah titik justru memperburuk interpolasi di tepi. Itulah overfitting dari Bab 1 dalam bentuk yang paling murni, model yang terlalu lentur memaksa diri melewati setiap titik. Spline kubik memakai polinomial derajat tiga per potongan yang disambung mulus, dan biasanya jauh lebih jinak. Interpolasi dengan polinomial trigonometri, yaitu jumlah sinus dan kosinus, cocok untuk data periodik, dan koefisiennya tidak lain adalah transformasi Fourier diskret.

Menghitung transformasi Fourier diskret dari N titik, $X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}$, secara langsung membutuhkan sekitar N^2 operasi. FFT (Cooley and Tukey, 1965) memecah jumlah itu menjadi dua jumlah setengah ukuran, satu untuk indeks genap dan satu untuk indeks ganjil:

$$X_k = E_k + e^{-i2\pi k/N} O_k, \quad X_{k+N/2} = E_k - e^{-i2\pi k/N} O_k, \quad (28.2)$$

dengan E_k dan O_k transformasi berukuran $N/2$ dari indeks genap dan ganjil. Tanda minus di rumus kedua muncul karena $e^{-i2\pi(k+N/2)/N} = -e^{-i2\pi k/N}$. Jadi satu perkalian menghasilkan dua keluaran, operasi yang disebut *butterfly*. Mengulang pemecahan secara rekursif memberi $\log_2 N$ tingkat dengan $N/2$ butterfly per tingkat. Untuk $N = 8$,

itu 12 perkalian kompleks alih-alih 64. Untuk satu juta titik, selisihnya kira-kira dari 10^{12} ke 2×10^7 operasi. FFT menjadi tulang punggung pemrosesan sinyal, pengolahan citra, solver spektral untuk PDE, dan neural operator berbasis Fourier di Bab 30.

Transformasi Fourier diskret empat titik bisa dihitung dengan tangan. Untuk $x = (1, 2, 3, 4)$, faktor fasenya $e^{-i2\pi kn/4} = (-i)^{kn}$. Komponen nol adalah jumlahnya, $X_0 = 10$. Komponen pertama $X_1 = 1 + 2(-i) + 3(-1) + 4(i) = -2 + 2i$. Komponen kedua $X_2 = 1 - 2 + 3 - 4 = -2$. Komponen ketiga adalah konjugat X_1 , yaitu $-2 - 2i$, seperti selalu terjadi untuk sinyal real. Komponen nol adalah empat kali rata-rata, sedangkan komponen lain mengukur seberapa kuat sinyal berayun pada frekuensi masing-masing. Untuk sinyal yang naik terus seperti ini, energinya terkumpul di frekuensi terendah, dan aturan butterfly di atas memberi hasil yang sama dari dua transformasi dua titik: $E = (1 + 3, 1 - 3) = (4, -2)$ dan $O = (2 + 4, 2 - 4) = (6, -2)$, sehingga $X_0 = 4 + 6 = 10$ dan $X_2 = 4 - 6 = -2$.

Spline kubik juga layak dilihat lebih dekat. Di setiap selang antara dua titik data dipasang satu polinomial derajat tiga, sehingga ada empat koefisien per selang. Syaratnya: setiap polinomial melewati kedua titik ujung selangnya, dan di setiap titik dalam, turunan pertama dan kedua dari dua polinomial yang bertemu harus sama. Hitungan syarat menyisakan dua derajat kebebasan, yang biasanya diisi dengan menetapkan turunan kedua nol di kedua ujung, spline *natural*. Kalau turunan kedua di setiap titik, M_i , dijadikan yang dicari, semua syarat itu runtuh menjadi sistem tridiagonal $h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6((y_{i+1} - y_i)/h_i - (y_i - y_{i-1})/h_{i-1})$, dengan h_i lebar selang. Sistem yang sama bentuknya dengan sistem volume hingga, dan diselesaikan dengan algoritme Thomas yang sama. Spline natural punya sifat yang menarik: di antara semua fungsi mulus yang melewati titik-titik data, ia meminimalkan $\int (f'')^2 dx$, ukuran total kelengkungan, seperti penggaris kayu lentur yang dipakai juru gambar kapal dan yang memberi nama spline.

28.2 Mencari akar

Banyak masalah berujung pada mencari x yang memenuhi $f(x) = 0$. Metode bagi dua mulai dari selang tempat f berganti tanda, lalu terus membagi selang menjadi dua dan mempertahankan separuh yang masih mengandung pergantian tanda. Lambat tetapi pasti: setiap langkah memberi tambahan satu bit ketelitian, sehingga sepuluh langkah memberi sekitar tiga angka desimal.

Metode Newton–Raphson memakai garis singgung. Di sekitar tebakan x_n , $f(x) \approx f(x_n) + f'(x_n)(x - x_n)$, dan garis ini memotong nol di

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \quad (28.3)$$

Untuk $f(x) = x^2 - 2$ dengan tebakan awal 1, iterasinya memberi 1,5, lalu 1,41667, lalu 1,414216, sementara $\sqrt{2} = 1,414214$. Jumlah angka yang benar kira-kira berlipat dua di setiap langkah, karena galat baru sebanding dengan kuadrat galat lama. Inilah konvergensi kuadratik. Tetapi Newton bisa melompat jauh kalau tebakan awalnya buruk atau turunannya hampir nol, sehingga dalam praktik sering digabung dengan metode

bagi dua sebagai pengaman. Dalam banyak dimensi, Newton menjadi penyelesaian sistem linear dengan Jacobian di setiap langkah, dan itulah yang terjadi di dalam setiap solver implisit. Metode Newton untuk optimasi di Bab 4 adalah resep yang sama, diterapkan pada gradien.

28.3 Persamaan diferensial biasa

Masalah nilai awal $\dot{\mathbf{y}} = \mathbf{f}(\mathbf{y}, t)$ dengan $\mathbf{y}(0)$ diketahui diselesaikan selangkah demi selangkah. Metode Euler eksplisit, $\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(\mathbf{y}_n, t_n)$, hanya akurat orde satu. Runge–Kutta orde empat mengevaluasi $\mathbf{f}$ empat kali per langkah, di awal, dua kali di tengah, dan di akhir langkah:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4), \quad (28.4)$$

dengan $\mathbf{k}_1 = \mathbf{f}(\mathbf{y}_n)$, $\mathbf{k}_2 = \mathbf{f}(\mathbf{y}_n + \frac{h}{2}\mathbf{k}_1)$, $\mathbf{k}_3 = \mathbf{f}(\mathbf{y}_n + \frac{h}{2}\mathbf{k}_2)$, dan $\mathbf{k}_4 = \mathbf{f}(\mathbf{y}_n + h\mathbf{k}_3)$. Bobotnya sama dengan aturan Simpson untuk integral. Galat globalnya sebanding dengan h^4 , sehingga memperkecil langkah setengah kali mengurangi galat enam belas kali.

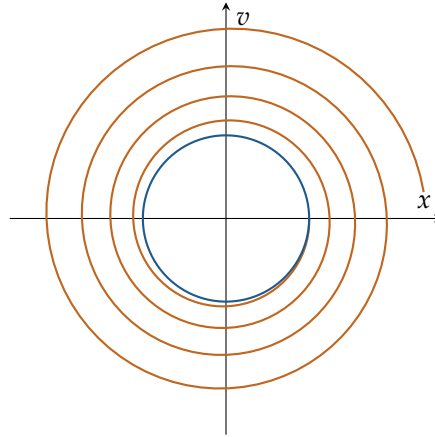
Osilator harmonik memperlihatkan bahwa akurasi saja tidak cukup. Ambil $\ddot{x} = -x$, ditulis sebagai dua persamaan orde satu untuk posisi x dan kecepatan v . Energinya, $\frac{1}{2}(x^2 + v^2)$, kekal, dan di ruang fase lintasannya lingkaran. Satu langkah Euler eksplisit adalah

$$\begin{pmatrix} x_{n+1} \\ v_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & h \\ -h & 1 \end{pmatrix} \begin{pmatrix} x_n \\ v_n \end{pmatrix}, \quad (28.5)$$

dan menghitung langsung memberi $x_{n+1}^2 + v_{n+1}^2 = (x_n + hv_n)^2 + (v_n - hx_n)^2 = (1 + h^2)(x_n^2 + v_n^2)$. Energi dikalikan $1 + h^2$ di setiap langkah, sehingga lintasannya berputar keluar (Gambar 28.1). Dengan $h = 0,1$, setelah seribu langkah, sekitar enam belas putaran, energinya membengkak sekitar $(1,01)^{1000} \approx 2 \times 10^4$ kali.

Sekarang ubah satu hal kecil: perbarui kecepatan dulu, lalu pakai kecepatan baru untuk memperbarui posisi. Matriksnya menjadi $\begin{pmatrix} 1 - h^2 & h \\ -h & 1 \end{pmatrix}$, yang determinannya tepat satu. Determinan adalah faktor perubahan luas, sehingga skema ini menjaga luas di ruang fase, versi diskret dari teorema Liouville. Metode seperti ini disebut *simplektik*. Velocity Verlet (Verlet, 1967) adalah versi orde duanya. Ia tidak menjaga energi persis, tetapi galat energinya berosilasi dalam batas yang tetap dan tidak menumpuk. Runge–Kutta orde empat jauh lebih akurat per langkah, tetapi tidak simplektik, sehingga untuk simulasi yang panjang sekali energinya tetap perlahan bergeser. Untuk dinamika molekul dan mekanika benda langit yang berjalan jutaan langkah, sifat struktur lebih penting daripada orde akurasi.

Masalah nilai batas berbeda. Syaratnya diberikan di dua ujung, misalnya $u(a) = u_a$ dan $u(b) = u_b$. Metode *shooting* menebak kemiringan awal, menyelesaikan masalah nilai awal, melihat seberapa jauh hasilnya meleset di ujung lain, lalu memperbaiki tebakan dengan pencarian akar, seperti meriam yang membidik ulang setelah melihat di mana peluru pertama jatuh. Alternatifnya, turunan diganti beda hingga di grid,



GAMBAR 28.1. Ruang fase osilator harmonik. Euler eksplisit (oranye) berputar keluar karena energinya bertambah di setiap langkah. Metode simplektik (biru) tetap di lintasan tertutup.

$u''(x_j) \approx (u_{j+1} - 2u_j + u_{j-1})/\Delta x^2$, sehingga masalahnya menjadi sistem persamaan linear tridiagonal, sama dengan sistem volume hingga di Bab 27.

28.4 Nilai eigen

Persamaan Schrödinger stasioner dalam satu dimensi adalah masalah nilai eigen: cari energi E dan fungsi gelombang ψ yang memenuhi $-\frac{\hbar^2}{2m}\psi'' + V\psi = E\psi$. Dengan beda hingga, operator turunan kedua menjadi matriks tridiagonal, dan masalahnya menjadi mencari nilai eigen sebuah matriks.

Partikel dalam kotak, dengan $V = 0$ di dalam dan dinding tak tertembus di kedua ujung, memberi contoh yang bisa diperiksa. Dalam satuan yang disederhanakan, masalahnya $-\psi'' = \lambda\psi$ dengan $\psi(0) = \psi(1) = 0$, dan jawaban eksaknya $\lambda_n = (n\pi)^2$. Dengan sembilan titik dalam, $\Delta x = 0,1$, matriks beda hingga tridiag $(-1, 2, -1)/\Delta x^2$ punya nilai eigen yang bisa dihitung tertutup,

$$\lambda_n^\Delta = \frac{2 - 2\cos(n\pi\Delta x)}{\Delta x^2}. \quad (28.6)$$

Untuk keadaan dasar, $\lambda_1^\Delta \approx 9,79$ dibanding $\pi^2 \approx 9,87$, meleset kurang dari satu persen. Untuk $n = 9$, $\lambda_9^\Delta \approx 390$ dibanding $81\pi^2 \approx 799$, meleset separuhnya. Grid hanya bisa mewakili gelombang yang jauh lebih panjang dari jarak titiknya. Pelajaran ini berlaku umum: mode-mode terendah dari diskretisasi bisa dipercaya, mode tertinggi tidak.

Untuk matriks umum, algoritme QR menghitung nilai eigen secara iteratif. Matriks diuraikan menjadi hasil kali matriks ortogonal dan matriks segitiga atas, $A_k = Q_k R_k$, lalu dikalikan dalam urutan terbalik, $A_{k+1} = R_k Q_k$. Karena $A_{k+1} = Q_k^\top A_k Q_k$, setiap langkah hanya mengganti basis, dan nilai eigen tidak berubah. Untuk matriks simetris, iterasinya konvergen ke matriks diagonal yang berisi nilai-nilai eigen. Penguraian QR sendiri dibangun dengan refleksi Householder yang mengonvolkan kolom demi kolom.

Nilai eigen muncul di mana-mana di buku ini: arah utama PCA, kestabilan sistem linear, frekuensi getaran struktur, dan mode-mode aliran pada dekomposisi POD.

28.5 PDE yang bergantung waktu

Persamaan difusi $\partial_t u = D \partial_{xx} u$ yang didiskretisasi dengan beda maju dalam waktu dan beda pusat dalam ruang memberi skema eksplisit

$$u_j^{n+1} = u_j^n + r(u_{j+1}^n - 2u_j^n + u_{j-1}^n), \quad r = \frac{D\Delta t}{\Delta x^2}. \quad (28.7)$$

Analisis kestabilan von Neumann memasukkan satu gelombang Fourier $u_j^n = g^n e^{ikj\Delta x}$ ke dalam skema. Setelah membagi dengan $g^n e^{ikj\Delta x}$,

$$g = 1 + r(e^{ik\Delta x} - 2 + e^{-ik\Delta x}) = 1 - 2r(1 - \cos k\Delta x) = 1 - 4r \sin^2 \frac{k\Delta x}{2}. \quad (28.8)$$

Supaya tidak ada gelombang yang tumbuh, $|g| \leq 1$ untuk semua k . Gelombang terburuk adalah yang paling pendek, dengan $\sin^2 = 1$, yang memberi $g = 1 - 4r$. Syarat $1 - 4r \geq -1$ berarti $r \leq 1/2$. Dengan $D = 1$ dan $\Delta x = 0,01$, langkah waktu tidak boleh melebihi 5×10^{-5} . Memperhalus grid dua kali memaksa langkah waktu empat kali lebih kecil, sehingga biayanya naik delapan kali. Skema implisit menghapus batas ini dengan harga menyelesaikan sistem linear di setiap langkah. Skema Crank–Nicolson merata-ratakan eksplisit dan implisit, sehingga stabil tanpa syarat dan akurat orde dua dalam waktu.

Untuk persamaan Schrödinger yang bergantung waktu, kuliah membahas metode *split-operator*. Operator Hamilton terdiri atas energi kinetik, yang sederhana di ruang momentum, dan energi potensial, yang sederhana di ruang posisi. Satu langkah waktu dipecah menjadi setengah langkah potensial, satu langkah kinetik setelah FFT ke ruang momentum, lalu setengah langkah potensial lagi setelah FFT balik. Karena kedua operator tidak komutatif, pemecahan ini tidak eksak, tetapi susunan simetris setengah–penuh–setengah membuat galatnya orde tiga per langkah. Setiap bagian adalah perkalian dengan fase, sehingga norma fungsi gelombang, yaitu total peluang, terjaga persis. Gagasan memecah operator menjadi bagian-bagian yang masing-masing mudah juga dipakai di banyak solver fluida, misalnya untuk memisahkan konveksi dari difusi.

28.6 Monte Carlo

Integral berdimensi tinggi tidak bisa dihitung dengan grid, karena jumlah titiknya tumbuh eksponensial: sepuluh titik per dimensi dalam dua puluh dimensi sudah 10^{20} titik. Integrasi Monte Carlo merata-ratakan fungsi pada titik-titik acak. Menurut teorema limit pusat, galatnya turun seperti $\sigma/\sqrt{N}$, dengan σ simpangan baku fungsi itu. Lambat, tetapi tidak bergantung pada dimensi sama sekali. Itulah sebabnya Monte Carlo menang untuk masalah berdimensi tinggi, dan itulah juga hubungannya dengan kutukan dimensi di Bab 1.

Contoh klasiknya menaksir π . Lempar titik acak ke persegi satuan dan hitung fraksi yang jatuh di dalam seperempat lingkaran berjari-jari satu. Peluangnya $p = \pi/4$, sehingga $\pi \approx 4 \times$ fraksi itu. Setiap lemparan adalah percobaan Bernoulli dengan varians $p(1-p) \approx 0,169$. Dengan sepuluh ribu titik, simpangan baku taksiran π adalah $4\sqrt{0,169/10^4} \approx 0,016$. Untuk satu angka desimal lagi dibutuhkan seratus kali lebih

banyak titik. Kalau fungsinya tidak rata, titik bisa diambil lebih sering di tempat fungsi besar dan diberi bobot sebaliknya. Pengambilan sampel penting ini (*importance sampling*) bisa mengecilkan σ secara drastis.

Dalam fisika statistik, kita ingin rata-rata besaran atas distribusi Boltzmann $p(\mathbf{X}) \propto \exp(-H(\mathbf{X})/k_B T)$, dengan H energi total sistem banyak partikel. Konstanta normalisasinya, fungsi partisi, mustahil dihitung. Algoritme Metropolis (Metropolis et al., 1953) tidak membutuhkannya. Usulkan gerakan kecil acak sebuah partikel. Kalau energinya turun, terima. Kalau naik sebesar ΔH , terima dengan peluang $\exp(-\Delta H/k_B T)$.

Mengapa aturan ini benar? Rantai Markov punya distribusi stasioner p kalau memenuhi keseimbangan rinci: arus peluang dari a ke b sama dengan arus sebaliknya, $p(a)T(a \rightarrow b) = p(b)T(b \rightarrow a)$. Dengan usulan yang simetris, T sebanding dengan peluang penerimaan. Misalkan $p(b) < p(a)$. Langkah dari a ke b diterima dengan peluang $p(b)/p(a)$, dan langkah sebaliknya selalu diterima. Ruas kiri menjadi $p(a) \cdot p(b)/p(a) = p(b)$ dan ruas kanan $p(b) \cdot 1 = p(b)$. Keduanya sama. Rasio $p(b)/p(a)$ hanya membutuhkan $\exp(-\Delta H/k_B T)$, dan fungsi partisi saling menghapus. Inilah MCMC yang sama dengan di Bab 14, di tempat kelahirannya. Pada temperatur rendah, hampir semua langkah naik ditolak dan rantai terjebak di lembah energi. Pada temperatur tinggi, rantai berkeliaran bebas. *Simulated annealing* memakai sifat ini untuk optimasi: mulai panas, lalu turunkan temperatur perlahan, supaya pencarian sempat keluar dari minimum lokal sebelum membeku.

28.7 Simulasi partikel

Fisika komputasi juga membawa kita ke simulasi partikel. Dinamika molekul (Alder and Wainwright, 1959) mengintegrasikan persamaan Newton untuk setiap atom dengan gaya dari potensial antaratom, seperti potensial Lennard-Jones di Bab 24, biasanya dengan velocity Verlet, dan menghitung sifat makroskopis seperti tekanan dan temperatur sebagai rata-rata (Frenkel and Smit, 2002). Temperatur, misalnya, dibaca dari energi kinetik rata-rata, $\frac{3}{2}k_B T = \langle \frac{1}{2}mv^2 \rangle$ untuk setiap atom. Menghitung gaya antara semua pasangan butuh N^2 operasi, sehingga gaya dipotong di jarak tertentu dan tetangga dicari dengan daftar sel, dan biayanya turun menjadi sebanding dengan N . Metode elemen diskret melakukan hal serupa untuk butiran yang bertumbukan (Bab 27). *Smoothed particle hydrodynamics* (Gingold and Monaghan, 1977; Monaghan, 1992) memakai partikel untuk mewakili fluida itu sendiri, dengan besaran di setiap titik diperoleh dengan meratakan partikel tetangga menggunakan kernel penghalus, $\rho(x) = \sum_j m_j W(x - x_j, h)$. Tanpa grid, metode ini nyaman untuk permukaan bebas dan percikan.

Semua simulasi partikel punya struktur yang sama: setiap partikel berinteraksi dengan tetangga di dekatnya. Struktur ini persis struktur graf, dan simulator berbasis jaringan graf di Bab 30 belajar meniru simulasi partikel seperti ini dengan memperlakukan partikel sebagai simpul dan interaksi sebagai sisi.

28.8 Perkembangan riset

Jaringan saraf mulai dipakai untuk masalah inti fisika komputasi. FermiNet (Pfau et al., 2019) memakai jaringan saraf sebagai ansatz fungsi gelombang untuk banyak elektron, dengan struktur yang menjamin fungsi gelombang berubah tanda ketika dua elektron ditukar, sesuai statistik Fermi–Dirac. Parameternya dioptimalkan dengan Monte Carlo variasional, metode yang memakai sampling Metropolis dari bab ini. Hanya dengan posisi dan muatan inti atom sebagai input, kurva disosiasi molekul nitrogen dan rantai hidrogen diprediksi lebih akurat dari metode coupled cluster.

Di dinamika, Hamiltonian Neural Networks (Greydanus et al., 2019) mempelajari fungsi Hamilton $H(q, p)$ dari data, lalu memperoleh dinamika dari persamaan Hamilton, $\dot{q} = \partial H / \partial p$ dan $\dot{p} = -\partial H / \partial q$, alih-alih mempelajari turunan waktu secara langsung. Karena dinamika diturunkan dari satu fungsi energi, energi yang dipelajari kekal dengan sendirinya, dan model bisa dijalankan mundur dalam waktu. Itulah gagasan integrator simplektik dari bab ini, dipindahkan ke dalam arsitektur jaringan.

CATATAN SUMBER. Bab ini mengikuti tiga belas catatan kuliah Computational Physics I yang ditulis tangan. Salah satu buku yang disebut di awal kuliah adalah Numerical Recipes (Press et al., 2007). Untuk simulasi partikel, Frenkel and Smit (2002) adalah pengantar yang baik.

29 Physics-Informed Neural Networks

PINN tidak punya mata kuliahnya sendiri dalam perjalanan saya. Ia muncul sedikit-sedikit: sebagai neural field yang diatur persamaan di akhir kuliah machine learning lanjut (Bab 15), sebagai kemampuan autodiff menurunkan keluaran terhadap input di kuliah deep learning dasar (Bab 7), dan sebagai salah satu pendekatan yang terus muncul dalam bacaan tentang machine learning untuk mekanika fluida selama mengerjakan tugas akhir. Bab ini merangkai potongan-potongan itu.

Gagasan PINN sederhana. Alih-alih menyelesaikan persamaan diferensial di grid, kita mewakili solusinya dengan jaringan saraf dan melatih jaringan itu agar memenuhi persamaannya. Gagasan ini sudah dicoba pada akhir 1990-an (Lagaris et al., 1998), dan dipopulerkan kembali dengan nama physics-informed neural networks oleh Raissi et al. (2019), setelah autodiff dan GPU membuatnya praktis.

29.1 Resep dasar

Ambil persamaan panas satu dimensi, $\partial_t u = \kappa \partial_{xx} u$ pada $x \in [0, 1]$ dan $t \in [0, T]$, dengan syarat awal $u(x, 0) = u_0(x)$ dan syarat batas $u(0, t) = u(1, t) = 0$. Sebuah MLP $u_\theta(x, t)$ menerima koordinat dan mengeluarkan temperatur. Untuk titik mana pun, autodiff memberi $\partial_t u_\theta$ dan $\partial_{xx} u_\theta$ secara eksak, sehingga kita bisa menghitung *residual* persamaan,

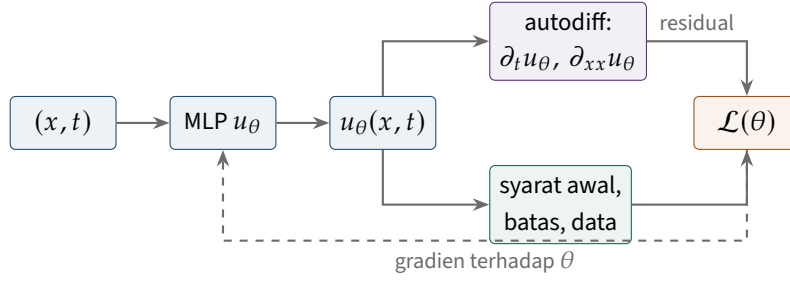
$$r_\theta(x, t) = \partial_t u_\theta(x, t) - \kappa \partial_{xx} u_\theta(x, t). \quad (29.1)$$

Solusi sejati membuat residual nol di mana-mana. Maka kita meminimalkan jumlah kuadrat residual di sekumpulan titik acak di dalam domain, yang disebut *collocation points*, ditambah suku untuk syarat awal dan syarat batas:

$$\mathcal{L}(\theta) = \frac{\lambda_r}{N_r} \sum_{i=1}^{N_r} r_\theta(x_i, t_i)^2 + \frac{\lambda_0}{N_0} \sum_{j=1}^{N_0} (u_\theta(x_j, 0) - u_0(x_j))^2 + \frac{\lambda_b}{N_b} \sum_{k=1}^{N_b} u_\theta(x_k^b, t_k)^2. \quad (29.2)$$

Kalau ada pengukuran, suku keempat menambahkan selisih kuadrat antara u_θ dan data di titik-titik pengukuran. Bobot λ menentukan seberapa penting masing-masing suku, dan seperti akan kita lihat, memilihnya ternyata salah satu bagian tersulit. Gambar 29.1 merangkum alurnya.

Turunan terhadap input dihitung dengan autodiff yang sama dengan backpropagation (Bab 7), hanya dirantai sekali atau dua kali lagi. Untuk satu neuron $u = \tanh(wx + b)$, turunan pertamanya $u_x = w(1 - u^2)$ dan turunan keduanya $u_{xx} = -2w^2u(1 - u^2)$. Contoh kecil ini menjelaskan satu aturan praktis PINN: fungsi aktivasi harus mulus. Dengan ReLU, turunan kedua bernilai nol hampir di mana-mana, sehingga jaringan tidak bisa “merasakan” suku difusi sama sekali, dan residual persamaan panas hanya



GAMBAR 29.1. Alur PINN. Jaringan menerima koordinat, autodiff memberi turunan yang dibutuhkan persamaan, dan loss menggabungkan residual persamaan dengan syarat awal, syarat batas, dan data.

melihat $\partial_t u$. Karena itu PINN hampir selalu memakai tanh, sinus, atau aktivasi mulus lain.

Mengapa ini masuk akal? Dari sudut pandang Bab 1, suku residual adalah prior: di antara semua fungsi yang cocok dengan data dan syarat batas, pilih yang memenuhi hukum fisika. Dari sudut pandang Bab 15, u_θ adalah neural field, dan persamaan diferensial adalah aturan yang ditegakkan pada field itu, persis seperti persamaan Eikonal pada signed distance field.

29.2 Contoh kecil yang bisa dihitung tangan

Supaya terasa konkret, ambil persamaan peluruhan $\dot{u} = -ku$ dengan $u(0) = 1$ dan $k = 1$, pada $t \in [0, 1]$. Solusi sejatinya e^{-t} , sehingga $u(1) = e^{-1} \approx 0,368$. Lagaris et al. (1998) menyarankan bentuk solusi yang memenuhi syarat awal secara otomatis, $u(t) = 1 + t N(t)$, dengan N sebuah jaringan. Untuk contoh tangan, kita ganti jaringan itu dengan konstanta a saja, sehingga $u(t) = 1 + at$. Residualnya

$$r(t) = \dot{u} + u = a + 1 + at = a(1 + t) + 1. \quad (29.3)$$

Kita cari a yang meminimalkan $J(a) = \int_0^1 r(t)^2 dt$, versi kontinu dari suku residual dalam Persamaan (29.2). Turunan terhadap a adalah $2 \int_0^1 (a(1+t)+1)(1+t) dt$. Menyamakannya dengan nol memberi

$$a \int_0^1 (1+t)^2 dt = - \int_0^1 (1+t) dt \implies a \cdot \frac{7}{3} = -\frac{3}{2} \implies a = -\frac{9}{14} \approx -0,643. \quad (29.4)$$

Maka $u(1) \approx 1 - 0,643 = 0,357$, hanya sekitar tiga persen di bawah nilai sebenarnya, padahal “jaringan”-nya hanya satu bilangan. Jaringan sungguhan dengan ribuan parameter bisa mendekati lengkungan eksponensial dengan jauh lebih baik. Contoh ini memperlihatkan inti PINN: tidak ada data solusi sama sekali, hanya persamaan dan syarat awal, dan meminimalkan residual sudah cukup untuk menemukan solusi yang masuk akal.

29.3 Masalah invers

Kekuatan PINN yang paling meyakinkan terletak pada masalah invers. Misalkan konduktivitas κ tidak diketahui, tetapi kita punya beberapa pengukuran temperatur. Cukup jadikan κ parameter yang ikut dilatih bersama bobot jaringan. Suku data memaksa u_θ cocok dengan pengukuran, suku residual memaksa u_θ memenuhi persamaan panas dengan κ yang sedang ditaksir, dan optimasi menemukan κ yang membuat keduanya konsisten. Solver klasik harus dijalankan berulang kali di dalam lingkaran optimasi untuk masalah yang sama, sedangkan PINN menyelesaikan masalah maju dan invers sekaligus.

Contoh paling terkenal adalah Raissi et al. (2020), yang disinggung di Bab 11. Dari video konsentrasi zat pewarna di sekitar sebuah benda, PINN memulihkan medan kecepatan dan tekanan yang tidak pernah diukur, karena persamaan transport zat pewarna dan persamaan Navier–Stokes menghubungkan ketiganya.

Contoh yang bisa diperiksa dengan tangan memperlihatkan logika masalah invers. Untuk persamaan panas dengan difusivitas α yang tidak diketahui di selang $[0, 1]$ dengan ujung bernilai nol, fungsi $u(x, t) = e^{-\alpha\pi^2 t} \sin(\pi x)$ memenuhi persamaan dan syarat batasnya. Misalkan sensor di tengah batang mencatat $u = 1$ pada $t = 0$ dan $u = 0,37$ pada $t = 1$. Residual persamaan hanya bisa nol kalau amplitudo meluruh seperti $e^{-\alpha\pi^2 t}$, dan suku data menuntut $e^{-\alpha\pi^2} = 0,37$, sehingga $\alpha = -\ln 0,37/\pi^2 \approx 0,10$. PINN menemukan angka yang sama dengan menurunkan kedua suku loss bersamaan, tanpa pernah diberi rumus solusinya. Pada masalah nyata, solusinya tidak punya bentuk tertutup, tetapi logikanya sama: persamaan memberi bentuk, data memberi angka.

Contoh ini juga memperlihatkan bahaya masalah invers. Kalau sensor diletakkan tepat di $x = 0$, di mana $\sin(\pi x) = 0$, pengukurannya selalu nol, dan α tidak bisa ditentukan sama sekali. Pengukuran yang tidak peka terhadap parameter membuat masalahnya *ill-posed*, dan tidak ada jaringan saraf yang bisa mengatasinya. Memilih letak sensor sama pentingnya dengan memilih arsitektur.

29.4 Syarat batas yang ditegakkan

Daripada menghukum pelanggaran syarat batas di loss, syarat itu bisa dibangun ke dalam bentuk solusi, seperti $1 + tN(t)$ di contoh tangan tadi. Untuk syarat Dirichlet $u(0) = a$ dan $u(1) = b$ di selang satuan, bentuk

$$u_\theta(x) = a(1 - x) + bx + x(1 - x)N_\theta(x) \quad (29.5)$$

memenuhi kedua syarat untuk jaringan N_θ apa pun: dua suku pertama adalah garis yang menghubungkan kedua nilai batas, dan suku ketiga bernilai nol di kedua ujung. Loss tinggal suku residual saja, dan masalah keseimbangan bobot antara residual dan batas, yang dibahas di bawah, hilang dengan sendirinya. Untuk geometri yang rumit, fungsi $x(1 - x)$ diganti fungsi jarak yang bernilai nol di batas, misalnya signed distance field dari Bab 15. Syarat Neumann, yang menetapkan turunan di batas, lebih sulit ditegakkan dengan cara ini dan biasanya tetap ditaruh di loss.

Titik collocation juga tidak harus tetap. Pemilihan adaptif berbasis residual menambahkan titik baru di tempat residualnya paling besar, misalnya dengan menarik sampel dengan peluang sebanding dengan kuadrat residual. Daerah dengan gradien tajam, seperti lapisan batas atau muka gelombang, mendapat lebih banyak titik, mirip penghalusan mesh adaptif pada solver klasik.

29.5 Mengapa PINN sulit dilatih

Kalau PINN begitu sederhana, mengapa tidak menggantikan semua solver? Pengalaman komunitas selama beberapa tahun memperlihatkan sejumlah kesulitan yang serius (Cuomo et al., 2022).

Kesulitan pertama adalah keseimbangan suku-suku loss. Wang et al. (2021) memperlihatkan bahwa gradien suku residual bisa jauh lebih besar dari gradien suku batas, sehingga jaringan memenuhi persamaan di dalam domain tetapi mengabaikan syarat batas. Ini masalah condition number dari Bab 4 dalam bentuk yang khas. Mereka mengusulkan bobot λ yang disesuaikan otomatis berdasarkan besar gradien masing-masing suku. Pendekatan lain memberi setiap titik collocation bobotnya sendiri yang ikut dilatih dengan arah berlawanan, sehingga titik yang sulit mendapat perhatian lebih (McClenny and Braga-Neto, 2023).

Skala masalahnya mudah diperkirakan. Misalkan solusinya mengandung komponen $\sin(n\pi x)$. Turunan keduanya $-(n\pi)^2 \sin(n\pi x)$, sehingga galat kecil pada komponen ini diperbesar $(n\pi)^2$ kali dalam residual dan $(n\pi)^4$ kali dalam kuadratnya. Untuk $n = 5$, faktor itu sekitar 6×10^4 . Suku syarat batas, yang tidak melibatkan turunan, sama sekali tidak diperbesar. Optimizer melihat lanskap yang hampir seluruhnya ditentukan residual, dan syarat batas tenggelam.

Kesulitan kedua berasal dari spectral bias (Bab 15). Analisis neural tangent kernel untuk PINN (Wang et al., 2022) menunjukkan bahwa komponen solusi dengan frekuensi tinggi dipelajari jauh lebih lambat. Solusi dengan gradien tajam atau osilasi cepat, yang justru menarik dalam fluida, menjadi sulit. Fourier features dan aktivasi sinus membantu.

Kesulitan ketiga diperlihatkan Krishnapriyan et al. (2021). Untuk persamaan konveksi dengan koefisien konveksi yang besar, PINN standar gagal total meski arsitekturnya cukup ekspresif. Masalahnya terletak pada lanskap loss yang sulit sekali dioptimasi, bukan pada kapasitas jaringan. Melatih bertahap, mulai dari koefisien kecil lalu dinaikkan, atau memecah domain waktu menjadi potongan yang dilatih berurutan, memperbaiki keadaan. Gagasan terakhir ini juga menyentuh masalah kausalitas: PINN melatih seluruh rentang waktu sekaligus, sehingga bisa saja “memenuhi” persamaan di waktu akhir sebelum benar di waktu awal, padahal informasi fisik mengalir dari masa lalu ke masa depan. Wang et al. (2024b) mengusulkan bobot yang menegaskan urutan itu secara eksplisit. Rentang waktu dibagi menjadi potongan-potongan, dan residual potongan ke- i diberi bobot $w_i = \exp(-\epsilon \sum_{k < i} \mathcal{L}_k)$. Selama potongan-potongan awal masih punya residual besar, bobot potongan yang lebih akhir hampir nol, sehingga jaringan dipaksa membereskan masa lalu lebih dulu.

Ada beberapa trik lain yang sering dipakai: syarat batas yang dibangun langsung ke

dalam bentuk solusi, seperti $1 + tN(t)$ pada contoh di atas, sehingga suku batas tidak lagi dibutuhkan; pemilihan titik collocation secara adaptif, dengan menambah titik di tempat residualnya besar; dan resep optimasi Adam lalu L-BFGS dari Bab 4.

29.6 Varian

Beberapa varian mengubah cara persamaan masuk ke dalam loss. Untuk persamaan yang berasal dari prinsip variasi, residual bisa diganti energi. Persamaan Poisson $-u'' = f$ dengan $u(0) = u(1) = 0$, misalnya, adalah syarat minimum dari

$$E[u] = \int_0^1 \left(\frac{1}{2} u'(x)^2 - f(x) u(x) \right) dx. \quad (29.6)$$

Untuk melihatnya, ganggu u menjadi $u + \epsilon v$ dengan v nol di kedua ujung. Turunan E terhadap ϵ di $\epsilon = 0$ adalah $\int (u'v' - fv) dx$. Integrasi parsial memindahkan turunan dari v ke u , dan suku batasnya hilang karena v nol di ujung, sehingga hasilnya $\int (-u'' - f)v dx$. Supaya nol untuk setiap v , haruslah $-u'' = f$. Deep Ritz (E and Yu, 2017) meminimalkan $E[u_\theta]$ secara langsung dengan integrasi Monte Carlo. Keuntungannya, loss hanya butuh turunan pertama, bukan turunan kedua, sehingga lebih murah dan lebih stabil. Gagasan yang sama mendasari metode elemen hingga, yang juga bekerja dengan bentuk lemah persamaan.

Untuk domain besar atau solusi yang berubah watak dari satu daerah ke daerah lain, domain bisa dipecah menjadi beberapa subdomain, masing-masing dengan jaringannya sendiri, yang dijahit dengan suku loss tambahan di antarmukanya. XPINN (Jagtap and Karniadakis, 2020) adalah contoh pendekatan ini, dan ia juga memungkinkan pelatihan paralel. Gagasannya mirip dekomposisi domain pada solver klasik: masalah besar yang sulit dipecah menjadi masalah kecil yang lebih mudah.

29.7 Penilaian yang jujur

Untuk masalah maju yang tipikal, misalnya aliran di sekitar benda pada bilangan Reynolds menengah, PINN umumnya lebih lambat dan kurang akurat daripada solver klasik yang sudah matang. Setiap kasus baru juga butuh pelatihan baru, karena PINN mempelajari satu solusi, bukan cara menyelesaikan. PINN paling bernilai di tempat lain: masalah invers dan asimilasi data, masalah dengan data sebaran yang tidak lengkap, geometri yang sulit di-mesh, dan masalah berdimensi tinggi di mana grid tidak mungkin dibangun (Karniadakis et al., 2021). Untuk mempelajari cara menyelesaikan sebuah keluarga persamaan sekaligus, kita butuh pendekatan lain: neural operator dan simulator yang dipelajari, tema bab berikutnya.

29.8 Perkembangan riset

Riset PINN beberapa tahun terakhir banyak berfokus pada mengapa pelatihannya sulit dan bagaimana memperbaikinya. Rathore et al. (2024) menghubungkan kesulitan itu dengan condition number: operator diferensial di suku residual membuat lanskap loss

buruk kondisinya, persis contoh skala $(n\pi)^4$ di atas. Mereka membandingkan Adam, L-BFGS, dan gabungan keduanya, mendapati bahwa gabungan Adam lalu L-BFGS paling baik, dan mengusulkan optimizer orde dua baru yang memperbaiki kinerja lebih jauh. PirateNets (Wang et al., 2024a) menyerang masalah lain: PINN dengan jaringan yang lebih dalam sering justru lebih buruk, karena inisialisasi MLP biasa membuat turunan jaringan sulit dilatih. Koneksi residual adaptif mereka membuat jaringan mulai sebagai jaringan dangkal dan perlahan menjadi dalam selama pelatihan, sehingga kedalaman tambahan akhirnya memberi akurasi tambahan.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Machine Learning: Advanced Techniques (neural field dan loss yang diinformasikan fisika), kuliah deep learning dasar (autodiff), dan bacaan selama tugas akhir. Bacaan utama: Raissi et al. (2019), ulasan Karniadakis et al. (2021) dan Cuomo et al. (2022), serta studi tentang kegagalan pelatihan oleh Wang et al. (2021), Wang et al. (2022), dan Krishnapriyan et al. (2021).

30 Belajar Menyelesaikan: Neural Operator dan Simulator

Bab ini juga tidak berasal dari satu mata kuliah. Kuliah geometric deep learning memberi bahasa graf dan ekuivariansi, kuliah machine learning lanjut memberi transformasi Fourier dan neural ODE, dan dosennya sendiri, Johannes Brandstetter, termasuk peneliti yang membentuk bidang ini, dari message passing PDE solver sampai transformer untuk fisika. Sisanya datang dari bacaan.

PINN mempelajari satu solusi. Kalau syarat awalnya berubah sedikit, pelatihan harus diulang. Neural operator dan simulator yang dipelajari mengambil tujuan yang lebih ambisius: belajar *cara* menyelesaikan sebuah keluarga persamaan, sehingga solusi untuk syarat awal, parameter, atau geometri baru bisa diperoleh dalam satu lintasan maju, sering ribuan kali lebih cepat dari solver klasik.

30.1 Dari fungsi ke fungsi

Solver numerik pada dasarnya adalah sebuah *operator*: ia memetakan satu fungsi, misalnya medan temperatur awal atau distribusi konduktivitas, ke fungsi lain, yaitu solusinya. Jaringan saraf biasa memetakan vektor berukuran tetap ke vektor berukuran tetap, sehingga terikat pada satu resolusi grid. *Operator learning* ingin belajar pemetaan antar ruang fungsi itu sendiri, sehingga model yang dilatih pada grid kasar bisa dievaluasi pada grid yang lebih halus (Kovachki et al., 2021).

Contoh paling sederhana sebuah operator sudah kita kenal. Untuk persamaan panas $\partial_t u = \kappa \partial_{xx} u$ di garis tak hingga, solusi pada waktu t adalah konvolusi syarat awal dengan kernel Gauss,

$$u(x, t) = \int \frac{1}{\sqrt{4\pi\kappa t}} \exp\left(-\frac{(x-y)^2}{4\kappa t}\right) u_0(y) dy. \quad (30.1)$$

Operatornya linear: menjumlahkan dua syarat awal menjumlahkan solusinya. Di domain frekuensi, setiap mode Fourier e^{ikx} hanya diredam, dengan faktor $e^{-\kappa k^2 t}$. Mode dengan bilangan gelombang $k = 10$ meluruh seratus kali lebih cepat dari mode $k = 1$. Dua pengamatan ini, bahwa solusi bisa berupa operator integral dan bahwa ia sederhana di domain frekuensi, adalah benih dari Fourier neural operator di bawah. Untuk persamaan nonlinear seperti Navier–Stokes, tidak ada rumus seperti ini, dan jaringan harus mempelajari operatornya dari contoh.

30.1.1 DeepONet

DeepONet (Lu et al., 2021) bertumpu pada teorema aproksimasi universal untuk operator. Operator G yang memetakan fungsi input v ke solusi $G(v)$, dievaluasi di titik

y , didekati sebagai

$$G(v)(y) \approx \sum_{k=1}^p b_k(v) t_k(y). \quad (30.2)$$

Jaringan *branch* menerima nilai v di sejumlah titik sensor tetap dan mengeluarkan koefisien b_k . Jaringan *trunk* menerima koordinat y dan mengeluarkan fungsi basis $t_k(y)$. Bentuk ini mengingatkan pada ekspansi deret seperti Fourier atau POD (Bab 31), dengan perbedaan bahwa basis dan koefisiennya sama-sama dipelajari. Karena trunk menerima koordinat kontinu, keluarannya bisa dievaluasi di titik mana pun.

Untuk operator linear, bentuk ini bisa dipahami secara konkret. Misalkan G adalah operator panas di atas pada selang dengan ujung bernilai nol, dan solusi diuraikan dalam mode sinus, $t_k(y) = \sin(k\pi y)$. Koefisien setiap mode pada waktu t adalah koefisien syarat awal dikali $e^{-\kappa(k\pi)^2 t}$, sebuah fungsi linear dari nilai-nilai v di titik sensor. Branch dalam kasus ini cukup menjadi lapisan linear, dan trunk cukup mengeluarkan fungsi sinus. Untuk operator nonlinear, kedua jaringan harus menemukan basis dan koefisien yang cocok sendiri. Varian yang dilatih bersama residual persamaan, seperti PINN, disebut physics-informed DeepONet.

30.1.2 Fourier neural operator

Fourier neural operator (Li et al., 2020) berangkat dari bentuk solusi banyak PDE linear: konvolusi syarat awal dengan sebuah kernel, seperti kernel Gauss pada persamaan panas di Bab 15. Satu lapisan FNO memakai operator integral dengan kernel yang dipelajari,

$$(\mathcal{K}v)(x) = \int \kappa(x - y) v(y) dy. \quad (30.3)$$

Karena kernel hanya bergantung pada selisih $x - y$, operator ini adalah konvolusi, dan teorema konvolusi (Bab 11) mengubahnya menjadi perkalian di domain frekuensi:

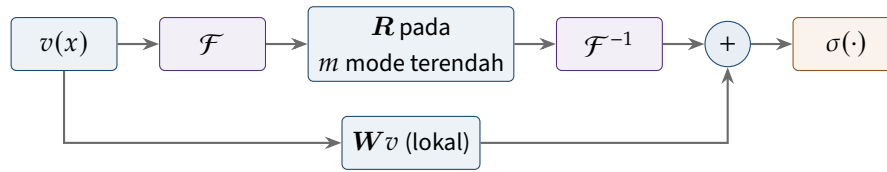
$$\mathcal{K}v = \mathcal{F}^{-1}(\mathbf{R} \cdot \mathcal{F}v), \quad (30.4)$$

dengan $\mathbf{R}$ bobot yang dipelajari untuk setiap frekuensi. FNO hanya menyimpan bobot untuk sejumlah kecil frekuensi terendah dan membuang sisanya. Satu lapisan lengkap menjumlahkan operator spektral ini dengan transformasi linear lokal, lalu menerapkan nonlinearitas. Dengan FFT, biayanya sebanding dengan $N \log N$ untuk N titik grid, dan karena bobotnya didefinisikan per frekuensi, bukan per titik grid, model yang sama bisa dijalankan pada resolusi yang berbeda. Pemotongan frekuensi juga punya sisi buruk: struktur yang tajam, seperti gelombang kejut, sulit diwakili oleh sedikit mode Fourier. FFT juga mengandaikan grid teratur dan domain periodik, sehingga untuk geometri yang rumit dibutuhkan varian yang memetakan geometri ke grid teratur lebih dulu.

Berapa banyak parameternya? Untuk lebar kanal d dan m mode yang disimpan dalam dua dimensi, setiap lapisan spektral menyimpan matriks kompleks $d \times d$ untuk setiap mode, kira-kira $m^2 d^2$ bilangan kompleks. Dengan $d = 32$ dan dua belas mode di setiap arah, itu sekitar 1,5 juta bilangan kompleks per lapisan, dan jumlah ini tidak bergantung pada resolusi grid. Jaringan konvolusi biasa dengan kernel 3×3 jauh lebih hemat parameter, tetapi hanya melihat tetangga dekat. FNO melihat seluruh domain

dalam satu lapisan, sesuai dengan watak persamaan eliptik seperti persamaan tekanan, di mana perubahan di satu titik langsung terasa di mana-mana.

Gambar 30.1 merangkum satu lapisan FNO.



GAMBAR 30.1. Satu lapisan Fourier neural operator. Jalur atas mencampur informasi global lewat sedikit mode Fourier, jalur bawah menambahkan transformasi lokal titik demi titik.

30.2 Simulator berbasis graf

Grid teratur jarang ditemui dalam simulasi teknik. Mesh menyesuaikan diri dengan geometri, dan partikel bergerak bebas. Di sini jaringan graf (Bab 13) menjadi alat alami. Graph Network-based Simulator (Sanchez-Gonzalez et al., 2020) memperlakukan setiap partikel sebagai simpul dan menghubungkan partikel yang berdekatan dengan sisi. Arsitekturnya mengikuti pola *encode–process–decode*: keadaan setiap partikel dikodekan menjadi vektor laten, beberapa langkah message passing menyebarkan informasi interaksi, lalu decoder mengeluarkan percepatan setiap partikel. Posisi baru dihitung dengan integrator sederhana, dan langkah itu diulang. Model yang dilatih pada air, pasir, dan cairan kental ini menghasilkan rollout yang stabil untuk ribuan langkah.

MeshGraphNets (Pfaff et al., 2020) membawa gagasan yang sama ke mesh simulasi, dengan dua jenis sisi: sisi mesh yang mengikuti konektivitas elemen, dan sisi “dunia” antara simpul yang berdekatan di ruang walau tidak tersambung di mesh, misalnya untuk kain yang menyentuh dirinya sendiri. Model ini bahkan belajar menyesuaikan resolusi mesh. Message Passing Neural PDE Solvers (Brandstetter et al., 2022) merumuskan message passing sebagai solver PDE umum dan menunjukkan bahwa ia mencakup skema numerik klasik seperti beda hingga dan volume hingga sebagai kasus khusus. Hubungan ini sudah kita lihat di Bab 27: volume hingga adalah message passing fluks.

Simulator graf juga membawa simetri fisika ke dalam arsitekturnya. Masukan untuk sisi biasanya berupa posisi relatif dan jarak antara dua partikel, bukan posisi mutlak, sehingga model otomatis invarian terhadap translasi: mendorong seluruh gelas air satu meter ke kanan tidak mengubah fisiknya. Karena setiap partikel hanya bicara dengan tetangga dalam radius tertentu, biayanya sebanding dengan jumlah partikel, sama seperti simulasi partikel klasik dengan daftar sel di Bab 28. Dan jumlah pesan yang dikirim sama dengan yang diterima kalau pesan dibuat antisimetris, sehingga momentum total terjaga, gagasan yang sama dengan hukum ketiga Newton.

Satu langkah message passing dengan angka memperlihatkan apa yang dihitung jaringan seperti ini. Ambil tiga partikel di garis, di posisi 0, 1, dan 3, dengan radius

tetangga 1,5, sehingga partikel pertama dan kedua bertetangga, sedangkan yang ketiga sendirian. Misalkan fungsi pesan yang dipelajari kebetulan berbentuk pegas, $m_{ij} = k(|x_j - x_i| - \ell) \text{sgn}(x_j - x_i)$ dengan panjang istirahat $\ell = 0,8$ dan $k = 2$. Pesan dari partikel kedua ke partikel pertama adalah $2(1 - 0,8) \cdot (+1) = 0,4$, dan dari pertama ke kedua $2(1 - 0,8) \cdot (-1) = -0,4$. Partikel ketiga tidak menerima pesan apa pun. Kalau decoder menerjemahkan jumlah pesan menjadi percepatan, partikel pertama dan kedua saling menarik dengan percepatan yang sama besar dan berlawanan arah, dan pusat massa keduanya tidak bergerak. Jaringan sungguhan tidak diberi bentuk pegas. Ia mempelajari fungsi pesan dari data, dan karena masukannya hanya posisi relatif, sifat seperti invariansi translasi ikut terbawa ke apa pun yang dipelajarinya.

30.3 Masalah rollout

Hampir semua simulator yang dipelajari bekerja secara autoregresif: tebak langkah berikutnya, lalu pakai tebakan itu sebagai input untuk langkah selanjutnya. Galat kecil di setiap langkah tidak hanya menumpuk, tetapi juga bisa diperbesar. Misalkan galat di langkah t adalah e_t , model menambahkan galat baru δ di setiap langkah, dan kepekaan langkah model terhadap inputnya diwakili faktor J . Maka $e_{t+1} \approx J e_t + \delta$, dan setelah T langkah

$$e_T \approx \delta \sum_{k=0}^{T-1} J^k. \quad (30.5)$$

Kalau $|J| < 1$, galat jenuh di sekitar $\delta/(1 - |J|)$. Kalau $|J|$ sedikit lebih besar dari satu, galat tumbuh eksponensial. Dengan $J = 1,01$, setelah 500 langkah faktor $J^{500} \approx 145$. Model yang tampak sempurna dalam uji satu langkah bisa meledak dalam rollout panjang.

Ada masalah yang lebih halus. Selama pelatihan, model hanya melihat input dari simulasi sejati. Selama rollout, ia melihat input dari tebakannya sendiri, yang sedikit berbeda distribusinya. Ini covariate shift yang sama dengan behavioral cloning di Bab 16. Beberapa obat sudah dicoba: menambahkan noise pada input selama pelatihan agar model terbiasa dengan input yang tidak sempurna (Sanchez-Gonzalez et al., 2020), melatih dengan rollout beberapa langkah, atau, seperti PDE-Refiner (Lippe et al., 2023), memperhalus setiap tebakan dengan beberapa langkah denoising mirip model difusi, supaya komponen frekuensi tinggi dengan amplitudo kecil tidak diabaikan. Analisis spektrum sering menunjukkan bahwa model kehilangan detail halus lebih dulu, dan detail itulah yang kemudian merusak stabilitas.

Cara mengevaluasi juga perlu dipikirkan. Galat rata-rata kuadrat satu langkah hampir tidak mengatakan apa-apa tentang rollout panjang. Untuk aliran turbulen, yang kacau, dua simulasi yang mulai dari keadaan hampir sama pun akan berpisah setelah beberapa waktu, sehingga membandingkan medan titik demi titik setelah waktu itu tidak adil. Yang lebih bermakna adalah statistiknya: spektrum energi, profil rata-rata, distribusi besaran penting. Model yang baik menghasilkan aliran yang berbeda dari kenyataan tetapi punya statistik yang sama, seperti dua hari berangin yang tidak identik tetapi sama-sama khas untuk musim itu.

Untuk membandingkan model secara adil, komunitas memakai himpunan tolok

ukur bersama. Galat yang paling sering dilaporkan adalah galat L^2 relatif, $\|\hat{u} - u\|/\|u\|$, dirata-ratakan atas kasus uji, pada satu langkah dan pada akhir rollout. Angka ini bisa menipu dengan dua cara. Galat relatif yang kecil bisa menyembunyikan galat besar di daerah kecil yang paling penting, misalnya di lapisan batas tempat gaya hambat ditentukan. Dan untuk sistem yang kacau, seperti dibahas di atas, galat titik demi titik pasti membesar setelah waktu tertentu, sehingga besaran yang diturunkan, seperti koefisien gaya, fluks panas, atau spektrum energi, lebih bermakna. Pertanyaan praktis lainnya adalah biaya total: berapa simulasi klasik yang dibutuhkan untuk menghasilkan data latih, berapa lama pelatihan, dan berapa kali model harus dipakai sebelum penghematannya menutup biaya itu.

30.4 Model fondasi untuk fisika dan cuaca

Keberhasilan paling mencolok terjadi di prakiraan cuaca, di mana data reanalisis global berpuh tahun tersedia. FourCastNet (Pathak et al., 2022) memakai operator Fourier yang adaptif. GraphCast (Lam et al., 2023) memakai message passing di atas mesh ikosahedral berjenjang yang membungkus bumi, dan dalam banyak ukuran mengungguli sistem prakiraan numerik operasional untuk sepuluh hari ke depan, dengan waktu hitung kurang dari satu menit. GenCast (Price et al., 2025) memakai model difusi untuk menghasilkan ensemble prakiraan yang probabilistik. Aurora (Bodnar et al., 2024) dilatih pada beragam data atmosfer sebagai model fondasi yang bisa disetel ke tugas-tugas baru, seperti kualitas udara.

Gagasan model fondasi juga dibawa ke PDE secara umum. Pretraining pada banyak sistem fisika sekaligus (McCabe et al., 2023) ternyata membantu ketika model disetel ke sistem baru dengan data sedikit. Kumpulan data besar seperti The Well (Ohana et al., 2024) menyediakan simulasi dari banyak bidang fisika untuk melatih dan menguji model seperti ini. Universal Physics Transformers (Alkin et al., 2024b) memisahkan model dari diskretisasi: titik mesh atau partikel dikodekan ke ruang laten berukuran tetap, dinamika berjalan di ruang laten, dan decoder bisa ditanya di titik mana pun. Gupta and Brandstetter (2022) membandingkan arsitektur secara sistematis pada berbagai skala ruang dan waktu.

Prakiraan cuaca juga memperlihatkan mengapa ensemble penting. Atmosfer kacau, sehingga satu prakiraan deterministik untuk sepuluh hari ke depan cenderung menjadi kabur: model yang dilatih dengan galat kuadrat belajar meramal rata-rata dari semua kemungkinan, dan rata-rata dari banyak badai yang mungkin adalah langit yang setengah mendung di mana-mana. Model generatif seperti GenCast menghasilkan banyak kemungkinan yang masing-masing tajam, dan sebaran di antara mereka menjadi ukuran ketidakpastian, sesuatu yang dibutuhkan untuk memutuskan, misalnya, apakah sebuah wilayah perlu dievakuasi.

30.5 Pertanyaan yang tetap harus diajukan

Kembali ke tiga pertanyaan di Bab 0. Fisika masuk ke model-model ini terutama lewat data dan arsitektur: simulasi yang benar sebagai data latih, dan ekuivariansi, lokalitas,

serta konservasi sebagai struktur jaringan. Di luar jangkauan data latih, misalnya pada bilangan Reynolds lain atau geometri yang jauh berbeda, jaminannya tipis, dan pengujian harus dilakukan secara eksplisit. Biaya pelatihannya besar, sering setara ribuan simulasi klasik untuk menghasilkan data, sehingga keuntungan baru muncul kalau model dipakai berkali-kali, misalnya untuk optimasi desain, kendali waktu nyata, atau ensemble besar. Ulasan Lino et al. (2023) dan Vinuesa and Brunton (2022) memberi gambaran yang seimbang tentang kekuatan dan batas pendekatan ini dalam mekanika fluida.

30.6 Perkembangan riset

Gagasan model fondasi untuk PDE berkembang cepat. Poseidon (Herde et al., 2024) adalah transformer operator multiskala yang dipretraining pada sekumpulan besar simulasi persamaan fluida, dengan trik yang memanfaatkan sifat semigrup PDE bergantung waktu: solusi dari waktu t_1 ke t_3 sama dengan solusi dari t_1 ke t_2 lalu dari t_2 ke t_3 , sehingga setiap lintasan simulasi memberi banyak pasangan latih. Setelah disetel, ia dievaluasi pada lima belas tugas hilir, termasuk fisika yang tidak dilihat saat pretraining. DPOT (Hao et al., 2024) memakai pretraining autoregresif dengan denoising, yaitu menambahkan noise pada input selama pretraining untuk mengatasi masalah rollout yang dibahas di atas, pada lebih dari seratus ribu lintasan dari belasan dataset PDE.

Untuk mesh tak teratur di geometri industri, Transolver (Wu et al., 2024) mengelompokkan titik-titik mesh yang punya keadaan fisis serupa ke dalam “irisan” yang dipelajari, lalu menjalankan attention di antara irisan-irisan itu, bukan di antara jutaan titik. Biayanya linear terhadap jumlah titik, dan contoh penerapannya mencakup desain mobil dan airfoil. Gagasannya mirip dengan model reduksi di Bab 31: cari sedikit “keadaan” yang mewakili banyak titik.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Deep Learning: Geometric Techniques, Machine Learning: Advanced Techniques, dan bacaan selama tugas akhir. Makalah utamanya: Lu et al. (2021), Li et al. (2020), Kovachki et al. (2021), Sanchez-Gonzalez et al. (2020), Pfaff et al. (2020), Brandstetter et al. (2022), Lippe et al. (2023), dan Lam et al. (2023).

31 Menemukan dan Memampatkan Dinamika

Makalah SINDy (Brunton et al., 2016) sudah saya simpan sejak musim panas 2024, sebelum kuliah pertama dimulai, dan buku Brunton and Kutz (2022) menyusul ke folder bacaan tidak lama kemudian. Bab ini membahas gagasan-gagasan dari buku itu yang menyambung dengan banyak kuliah: PCA dari kuliah unsupervised dan computer vision, nilai eigen dari fisika komputasi, lasso dari dasar-dasar machine learning, dan sistem dinamis dari kuliah kendali.

Model pengganti di Bab 30 umumnya kotak hitam yang besar sekali. Bab ini menempuh arah sebaliknya. Pertanyaannya bukan “bisakah jaringan meniru simulasi?”, melainkan “berapa sedikit yang sebenarnya kita butuhkan untuk menjelaskan dinamika ini?”. Jawabannya sering mengejutkan: medan aliran dengan jutaan derajat kebebasan kadang bisa dijelaskan oleh beberapa mode saja, dan dinamikanya oleh persamaan dengan beberapa suku.

31.1 Proper orthogonal decomposition

Simpan m potret medan aliran, masing-masing dengan n nilai, sebagai kolom-kolom matriks $\mathbf{X} \in \mathbb{R}^{n \times m}$ setelah rata-ratanya dikurangkan. Dekomposisi nilai singular memberi

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \quad (31.1)$$

dengan kolom-kolom $\mathbf{U}$ sebagai mode ruang, nilai singular σ_k di diagonal $\mathbf{\Sigma}$, dan baris-baris $\mathbf{V}^\top$ sebagai evolusi waktu setiap mode. Dalam mekanika fluida, prosedur ini disebut *proper orthogonal decomposition* (Berkooz et al., 1993; Taira et al., 2017). Ia tidak lain adalah PCA dari Bab 6, diterapkan pada potret-potret medan.

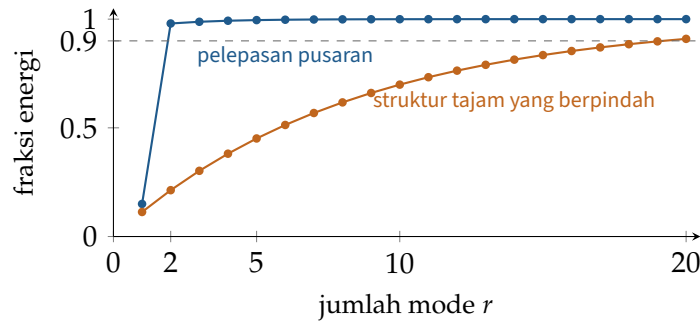
Mengapa mode-mode ini istimewa? Teorema Eckart–Young menyatakan bahwa di antara semua matriks berpangkat r , potongan SVD yang hanya menyimpan r mode pertama adalah yang paling dekat ke $\mathbf{X}$, dengan galat kuadrat sama dengan jumlah kuadrat nilai singular yang dibuang, $\sum_{k>r} \sigma_k^2$. Kalau medan berupa kecepatan, σ_k^2 sebanding dengan energi kinetik yang ditangkap mode ke- k , sehingga fraksi $\sum_{k \leq r} \sigma_k^2 / \sum_k \sigma_k^2$ dibaca sebagai fraksi energi yang ditangkap. Untuk aliran melewati silinder pada bilangan Reynolds rendah, beberapa mode pertama datang berpasangan dan bersama-sama menangkap hampir seluruh energi fluktuasi, karena pelepasan pusaran adalah gelombang yang berjalan, dan gelombang berjalan butuh dua mode yang bergeser seperempat periode.

Klaim terakhir bisa diperiksa dengan satu identitas trigonometri. Gelombang yang

berjalan ke kanan, $u(x, t) = \sin(x - t)$, bisa ditulis sebagai

$$\sin(x - t) = \sin x \cos t - \cos x \sin t. \quad (31.2)$$

Medannya adalah jumlah dua suku, masing-masing berupa pola ruang tetap dikali fungsi waktu. Jadi matriks potrenya tepat berpangkat dua, dengan mode ruang $\sin x$ dan $\cos x$, dan nilai singular ketiga dan seterusnya nol. Satu mode saja tidak cukup, karena satu pola tetap yang dikalikan amplitudo hanya bisa berdenyut di tempat, tidak bisa berjalan. Sebaliknya, gelombang yang bentuknya berubah sambil berjalan, atau gelombang kejut yang tajam, membutuhkan banyak sekali mode, dan nilai singularnya meluruh pelan (Gambar 31.1). Inilah batas mendasar semua metode reduksi linear: mereka bekerja baik ketika aliran “berdenyut” di tempat, dan buruk ketika struktur tajam berpindah tempat.



GAMBAR 31.1. Gambaran fraksi energi yang ditangkap r mode POD pertama. Aliran periodik seperti pelepasan pusaran di belakang silinder sudah tertangkap oleh beberapa pasang mode. Struktur tajam yang berpindah tempat membutuhkan banyak sekali mode. Kurva ini ilustrasi bentuk, bukan data dari satu simulasi tertentu.

Mode POD bisa dipakai untuk membangun *reduced-order model*. Tuliskan $u(x, t) \approx \bar{u}(x) + \sum_{k=1}^r a_k(t) \varphi_k(x)$, masukkan ke persamaan Navier–Stokes, lalu proyeksikan ke setiap mode dengan mengambil hasil kali dalam dengan φ_k . Karena mode-mode saling ortonormal, ruas kiri tinggal $\dot{a}_k$. Suku viskositas linear dalam u , sehingga menyumbang suku linear dalam koefisien. Suku konveksi $(u \cdot \nabla)u$ kuadrat dalam u , sehingga menyumbang hasil kali dua koefisien. Hasilnya sistem ODE kecil

$$\dot{a}_k = c_k + \sum_j L_{kj} a_j + \sum_{i,j} Q_{kij} a_i a_j, \quad k = 1, \dots, r, \quad (31.3)$$

dengan koefisien c , L , dan Q yang dihitung sekali dari mode dan medan rata-rata. Tekanan sering lenyap dari proyeksi karena mode yang bebas divergensi tegak lurus terhadap gradien tekanan di domain tertutup. Model seperti ini bisa ribuan kali lebih cepat, tetapi sering tidak stabil untuk waktu panjang, karena mode-mode kecil yang dibuang adalah tempat energi seharusnya dibuang, sama seperti pusaran kecil dalam kaskade turbulensi (Bab 27). Model penutup untuk mode yang hilang, atau model deret seperti LSTM pada koefisien a_k (Bab 9), dipakai untuk memperbaikinya.

31.2 Dynamic mode decomposition

POD memilih mode menurut energi, tanpa peduli dinamikanya. *Dynamic mode decomposition* (SCHMID, 2010) memilih mode menurut dinamika. Misalkan potret-potret berurutan dengan jarak waktu Δt , dan susun dua matriks yang bergeser satu langkah, $\mathbf{X} = [x_1, \dots, x_{m-1}]$ dan $\mathbf{X}' = [x_2, \dots, x_m]$. Kita mencari operator linear $\mathbf{A}$ sehingga $\mathbf{X}' \approx \mathbf{A}\mathbf{X}$, yaitu $\mathbf{A} = \mathbf{X}'\mathbf{X}^+$ dengan pseudo-invers $\mathbf{X}^+$. Untuk medan besar, $\mathbf{A}$ berukuran $n \times n$ dan terlalu besar untuk dibentuk, sehingga kita memproyeksikannya ke r mode POD pertama. Dengan $\mathbf{X} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top$, diperoleh

$$\tilde{\mathbf{A}} = \mathbf{U}_r^\top \mathbf{X}' \mathbf{V}_r \mathbf{\Sigma}_r^{-1}, \quad (31.4)$$

matriks kecil $r \times r$ yang nilai eigennya sama dengan nilai eigen utama $\mathbf{A}$. Setiap nilai eigen λ_k membawa satu mode yang berkembang seperti λ_k^t per langkah. Dalam waktu kontinu, $\omega_k = \ln \lambda_k / \Delta t$, dengan bagian real sebagai laju tumbuh atau luruh dan bagian imajiner sebagai frekuensi. Kasus paling sederhana memperlihatkan logikanya. Kalau setiap potret tepat 0,9 kali potret sebelumnya, $\mathbf{X}' = 0,9 \mathbf{X}$, maka $\mathbf{A} = 0,9 \mathbf{I}$ pada ruang yang direntang data, dan DMD menemukan satu nilai eigen 0,9: medan yang meluruh sepuluh persen per langkah. Sebagai contoh yang berosilasi, $\lambda = 0,99 e^{0,1i}$ dengan $\Delta t = 0,01$ detik memberi laju luruh $\ln 0,99 / 0,01 \approx -1,0$ per detik dan frekuensi sudut 10 radian per detik.

DMD seperti mendengarkan sebuah orkestra lalu menuliskan partitur: nada mana yang dimainkan, seberapa keras, dan apakah ia menguat atau memudar. POD, sebaliknya, hanya mencatat alat musik mana yang paling banyak bersuara.

Contoh dua dimensi memperlihatkan bahwa DMD memulihkan dinamika linear secara eksak. Misalkan sistem berputar sebesar sudut θ dan meredam sepuluh persen di setiap langkah, $x_{k+1} = 0,9 \mathbf{R}(\theta) x_k$, dimulai dari $x_0 = (1, 0)$. Dua potret pertama x_0 dan $x_1 = 0,9(\cos \theta, \sin \theta)$ membentuk matriks $\mathbf{X}$ yang bisa dibalik, sehingga $\mathbf{A} = \mathbf{X}'\mathbf{X}^{-1}$ tepat sama dengan $0,9 \mathbf{R}(\theta)$. Nilai eigennya $0,9 e^{\pm i\theta}$: modulus 0,9 memberi laju luruh, sudut θ memberi frekuensi. Dari dua potret saja, DMD menemukan “nada” sistem dan seberapa cepat ia memudar. Kalau datanya berisik, matriks yang diperoleh juga berisik, dan nilai eigen cenderung bergeser ke dalam lingkaran satuan, seolah sistemnya lebih teredam dari kenyataan. Varian DMD yang lebih tahan noise memperlakukan galat di $\mathbf{X}$ dan $\mathbf{X}'$ secara simetris.

31.3 Model tereduksi untuk banyak parameter

Dalam desain dan kendali, kita jarang hanya butuh satu simulasi. Kita butuh banyak, untuk bilangan Reynolds, laju aliran, atau temperatur masuk yang berbeda. Model tereduksi *parametrik* membangun basis POD dari potret-potret yang dikumpulkan di beberapa titik parameter. Pertanyaannya, di titik mana simulasi penuh dijalankan. Strategi *greedy* menjawabnya secara berurutan: mulai dari satu titik, bangun model tereduksi, perkirakan galatnya di banyak titik kandidat dengan penaksir galat yang murah, lalu jalankan simulasi penuh di titik dengan perkiraan galat terbesar dan tambahkan potretnya ke basis. Mirip active learning di Bab 23: setiap simulasi mahal dipakai di tempat yang paling mengurangi ketidaktahuan.

Kalau parameter mengubah karakter aliran, misalnya pusaran yang muncul hanya di atas bilangan Reynolds tertentu, satu basis untuk semua parameter menjadi tidak efisien. Solusinya adalah beberapa basis lokal, masing-masing untuk satu daerah parameter, dengan aturan untuk memilih basis yang tepat. Gagasan ini sejalan dengan rCFD yang menyimpan basis data per rezim aliran (Bab 33).

31.4 Koopman: linear di ruang yang lebih besar

DMD mendekati dinamika dengan operator linear, padahal kebanyakan sistem nonlinear. Teori operator Koopman (Koopman, 1931) memberi pembenarannya. Alih-alih melacak keadaan x , kita melacak fungsi-fungsi pengamatan $g(x)$. Operator Koopman memajukan pengamatan dalam waktu, dan operator itu selalu linear, walau dinamikanya nonlinear, dengan harga bahwa ruang fungsi pengamatan umumnya berdimensi tak hingga.

Contoh dari Brunton and Kutz (2022) memperlihatkan kapan ruang itu ternyata berhingga. Ambil sistem $\dot{x} = \mu x$ dan $\dot{y} = \lambda(y - x^2)$, yang nonlinear karena suku x^2 . Tambahkan pengamatan baru $z = x^2$. Turunannya $\dot{z} = 2x\dot{x} = 2\mu x^2 = 2\mu z$. Dalam koordinat (x, y, z) , sistemnya menjadi

$$\dot{x} = \mu x, \quad \dot{y} = \lambda y - \lambda z, \quad \dot{z} = 2\mu z, \quad (31.5)$$

sepenuhnya linear. Dalam praktik, pengamatan yang membuat dinamika linear jarang bisa ditebak, sehingga Lusch et al. (2018) melatih autoencoder untuk menemukannya: encoder memetakan keadaan ke koordinat di mana dinamikanya adalah perkalian matriks, dan decoder mengembalikannya. Kalau berhasil, semua alat kendali linear dari Bab 19 bisa dipakai pada sistem nonlinear.

31.5 Menemukan persamaan: SINDy

Kadang kita ingin lebih dari model yang meniru: kita ingin persamaannya. *Sparse identification of nonlinear dynamics* (Brunton et al., 2016) berangkat dari pengamatan bahwa kebanyakan persamaan fisika hanya punya sedikit suku. Susun pustaka fungsi kandidat dari data, misalnya $\Theta(X) = [1, x, x^2, x^3, \sin x, \dots]$, dan cari koefisien jarang Ξ sehingga

$$\dot{X} \approx \Theta(X)\Xi. \quad (31.6)$$

Ini masalah regresi linear dengan prior jarang, persis lasso dari Bab 1. Algoritme yang dipakai dalam makalah aslinya lebih sederhana: selesaikan kuadrat terkecil, nolkan koefisien yang kecil di bawah ambang, lalu ulangi kuadrat terkecil hanya dengan suku yang tersisa.

Contoh kecil memperlihatkan logikanya. Misalkan data berasal dari $\dot{x} = -2x$, dan pustakanya $[1, x, x^2]$. Pada data tanpa noise, $\dot{x}$ di setiap titik tepat -2 kali x , sehingga kuadrat terkecil memberi koefisien $(0, -2, 0)$ dan pengembangan tidak mengubah apa pun. Dengan noise, kuadrat terkecil biasa memberi koefisien kecil pada 1 dan x^2 yang mencoba menjelaskan noise. Pengembangan membuangnya, dan regresi ulang hanya

dengan x memulihkan -2 dengan baik. Hasilnya bukan jaringan dengan ribuan bobot, melainkan persamaan yang bisa dibaca, diperiksa, dan diekstrapolasi. Pada data sistem Lorenz yang chaos,

$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z, \quad (31.7)$$

SINDy memulihkan ketiga persamaannya hanya dari lintasan yang diukur. Dari puluhan kandidat suku polinomial sampai derajat tiga, yang tersisa hanya tujuh suku, dengan koefisien yang dekat ke nilai sebenarnya.

SINDy punya tiga syarat yang sering menjadi titik lemah. Turunan waktu harus diperkirakan dari data, dan diferensiasi numerik memperbesar noise. Pustakanya harus memuat suku yang benar. Dan koordinatnya harus tepat, karena dalam koordinat yang salah, dinamika sederhana bisa tampak rumit. Untuk masalah terakhir, Champion et al. (2019) melatih autoencoder bersama SINDy, sehingga koordinat dan persamaan ditemukan bersamaan. PDE-FIND (Rudy et al., 2017) memperluas gagasan ini ke persamaan diferensial parsial dengan memasukkan turunan ruang ke dalam pustaka.

31.6 Memilih alat

Ketiga keluarga ini mengisi tempat yang berbeda. POD dan model tereduksi cocok ketika kita punya persamaan dan ingin mempercepatnya. DMD dan Koopman cocok untuk menganalisis dan meramalkan dinamika yang hampir periodik atau berosilasi, dan untuk kendali. SINDy cocok ketika persamaannya belum diketahui dan datanya cukup bersih. Semuanya menghasilkan model yang jauh lebih kecil dan lebih mudah ditafsirkan daripada jaringan dalam, dan karena itulah mereka masih dipakai berdampingan dengan neural operator di Bab 30, sering sebagai lapisan di dalamnya.

31.7 Perkembangan riset

Dua perkembangan memperkuat model tereduksi dan penemuan persamaan. *Operator inference* (Kramer et al., 2024) mengambil bentuk polinomial model Galerkin di atas, $\dot{a} = c + La + Q(a \otimes a)$, tetapi tidak menghitung c , L , dan Q dari proyeksi persamaan. Koefisien-koefisien itu dipelajari langsung dari data simulasi dengan kuadrat terkecil. Cara ini non-intrusif, karena tidak perlu membuka kode solver, dan tetap mewarisi teori reduksi berbasis proyeksi, sehingga perkiraan galat bisa diturunkan. Versi yang menjaga struktur Hamilton menjamin energi kekal di luar rentang pelatihan.

Weak SINDy (Messenger and Bortz, 2021) mengatasi kelemahan terbesar SINDy, yaitu turunan numerik yang memperbesar noise. Alih-alih mencocokkan turunan, persamaan dikalikan dengan fungsi uji yang mulus lalu diintegrasikan, sehingga turunan berpindah dari data ke fungsi uji lewat integrasi parsial, trik yang sama dengan bentuk lemah pada Deep Ritz di Bab 29. Data berisik tidak lagi perlu didiferensiasi sama sekali.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Machine Learning: Unsupervised Techniques (PCA), Control Systems (sistem linear dan nilai eigen), Computational Physics I (SVD dan masalah nilai eigen), dan bacaan selama tugas akhir. Rujukan utama: Brunton and Kutz (2022), Taira et al. (2017), SCHMID (2010), dan Brunton et al. (2016).

32 Model Hibrida: Fisika dan Data Bersama

Di antara semua pendekatan di Bagian V, model hibrida adalah yang paling dekat dengan cara kerja insinyur sehari-hari. Kita tidak membuang solver yang sudah teruji puluhan tahun. Kita membiarkannya mengerjakan apa yang ia kuasai, lalu memakai data untuk memperbaiki bagian yang ia kerjakan dengan buruk. Bab ini dirangkai dari kuliah metode numerik fluida, yang membahas model penutup turbulensi beserta kelemahannya, dan dari bacaan selama tugas akhir.

32.1 Di mana solver lemah

Bab 27 memperlihatkan bahwa simulasi praktis selalu memodelkan sesuatu yang tidak bisa diselesaikan: tegangan Reynolds di RANS, pusaran sub-grid di LES, gaya antara fluida dan partikel di aliran multifase, atau perpindahan panas di dekat dinding yang tidak cukup halus grid-nya. Model-model ini dirumuskan dengan asumsi yang disederhanakan dan dikalibrasi pada aliran sederhana. Ketika dipakai pada aliran yang rumit, misalnya dengan pemisahan aliran, kelengkungan kuat, atau putaran, asumsinya runtuh. Data dari simulasi yang lebih teliti, seperti DNS dan LES, atau dari percobaan, bisa dipakai untuk mempelajari model yang lebih baik (Duraismy et al., 2019).

32.2 Contoh kecil: bandul dengan gesekan yang tidak diketahui

Gagasan hibrida paling mudah dilihat pada sistem kecil. Misalkan kita tahu persamaan bandul tanpa gesekan dari Bab 21, $\ddot{\theta} = -(g/l) \sin \theta$, tetapi bandul nyata melambat karena gesekan yang bentuknya tidak kita ketahui: mungkin sebanding dengan kecepatan, mungkin dengan kuadratnya, mungkin campuran. Model hibrida menulis

$$\ddot{\theta} = -\frac{g}{l} \sin \theta + N_{\theta}(\theta, \dot{\theta}), \quad (32.1)$$

dengan N_{θ} jaringan kecil. Bagian yang diketahui tetap ditulis sebagai fisika, dan jaringan hanya belajar sisanya. Persamaan ini diintegrasikan dengan solver ODE biasa, dan bobot jaringan dilatih agar lintasan yang dihasilkan cocok dengan lintasan yang diukur, dengan gradien yang dibawa mundur melewati solver. Kerangka ini disebut *universal differential equation* (Rackauckas et al., 2020). Setelah dilatih, N_{θ} bisa dibaca kembali, misalnya dengan SINDy dari Bab 31, untuk mencari bentuk gesekan yang sebenarnya. Kalau jaringan ternyata mendekati $-0,3 \dot{\theta}$, kita baru saja menemukan redaman linear dengan koefisien 0,3 per detik. Jaringan-nya jauh lebih kecil daripada jaringan yang harus mempelajari seluruh dinamika bandul, dan di luar rentang data ia tetap dibimbing oleh suku gravitasi yang benar.

32.3 Model penutup yang dipelajari

Pendekatan paling langsung adalah melatih jaringan untuk memprediksi besaran yang hilang dari besaran yang tersedia. Untuk RANS, targetnya tegangan Reynolds, dan inputnya gradien kecepatan rata-rata. Tetapi jaringan sembarang tidak menjamin sifat fisika yang paling dasar: hasilnya harus sama tidak peduli bagaimana sistem koordinat diputar. Ling et al. (2016) membangun sifat itu ke dalam arsitektur. Mereka memakai hasil klasik bahwa tensor anisotropi Reynolds $\mathbf{b}$, dalam model viskositas turbulen umum, bisa ditulis sebagai kombinasi dari sepuluh tensor basis $\mathbf{T}^{(n)}$ yang disusun dari tensor laju regangan $\mathbf{S}$ dan tensor rotasi $\mathbf{\Omega}$ yang dinormalisasi (Pope, 2000):

$$\mathbf{b} = \sum_{n=1}^{10} g_n(\lambda_1, \dots, \lambda_5) \mathbf{T}^{(n)}. \quad (32.2)$$

Koefisien g_n hanya bergantung pada lima besaran invarian λ_i , seperti $\text{tr}(\mathbf{S}^2)$ dan $\text{tr}(\mathbf{\Omega}^2)$, yang nilainya tidak berubah ketika koordinat diputar. Tensor basis sendiri berputar bersama koordinat. Jadi kalau jaringan hanya mempelajari g_n dari invarian, hasil $\mathbf{b}$ dijamin berputar dengan benar, apa pun bobot jaringannya. Model viskositas turbulen linear dari Bab 27 adalah kasus khusus dengan hanya suku pertama, $\mathbf{T}^{(1)} = \mathbf{S}$, dan koefisien tetap. Inilah contoh nyata inductive bias dari Bab 0: simetri tidak dipelajari dari data, tetapi dibangun ke dalam bentuk modelnya.

32.4 A priori dan a posteriori

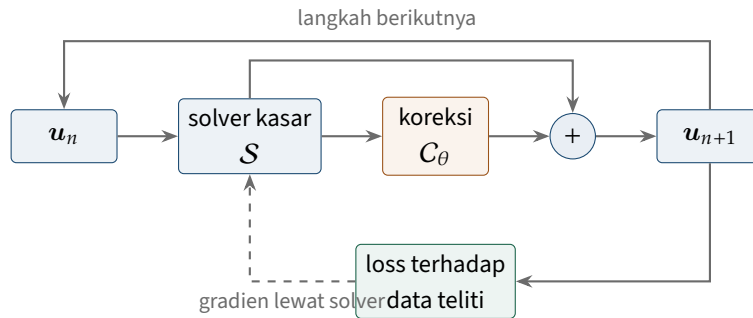
Model penutup yang dipelajari sering tampak baik dalam uji *a priori*: ambil medan DNS, hitung input dan target, lalu ukur galat prediksi. Begitu model dimasukkan ke dalam solver dan simulasi dijalankan, yaitu uji *a posteriori*, hasilnya bisa mengecewakan atau bahkan meledak. Penyebabnya sama dengan masalah rollout di Bab 30. Selama pelatihan, model melihat input dari DNS. Selama simulasi, ia melihat input dari medan yang dihasilkan solver bersama model itu sendiri, yang distribusinya bergeser. Galat kecil mengubah medan, medan yang berubah mengubah input model, dan lingkarannya bisa memperbesar galat.

Akumulasi galat ini bisa dirasakan dengan contoh skalar. Misalkan dinamika sebenarnya $u_{n+1} = 0,95 u_n$, solver kasar memberi $0,90 u_n$, dan koreksi yang dilatih *a priori* menambahkan $0,049 u_n$. Galat satu langkahnya hanya $0,001 u_n$, sekitar seperseribu, angka yang tampak sempurna dalam tabel uji *a priori*. Setelah seratus langkah, rasio hasil hibrida terhadap kebenaran adalah $(0,949/0,95)^{100} \approx 0,90$, galat sepuluh persen. Dalam sistem nonlinear, galat seperti ini tidak hanya menumpuk tetapi juga mengubah input koreksi berikutnya, sehingga bisa lebih buruk lagi.

Obatnya adalah melatih model di dalam lingkaran yang sama dengan tempat ia akan dipakai. Pendekatan *solver-in-the-loop* (Um et al., 2020) menulis satu langkah simulasi hibrida sebagai

$$\mathbf{u}_{n+1} = \mathcal{S}(\mathbf{u}_n) + C_\theta(\mathcal{S}(\mathbf{u}_n)), \quad (32.3)$$

dengan $\mathcal{S}$ solver kasar dan C_θ koreksi yang dipelajari (Gambar 32.1).



GAMBAR 32.1. Solver-in-the-loop. Koreksi yang dipelajari ditambahkan pada hasil solver kasar di setiap langkah, dan gradien loss dibawa mundur melewati beberapa langkah solver sekaligus.

Kalau solver ditulis dalam kerangka yang bisa diturunkan, gradien loss bisa dibawa mundur melewati beberapa langkah solver sekaligus, sehingga koreksi belajar memperbaiki galat yang muncul dari interaksinya sendiri dengan solver. Gagasan solver yang bisa diturunkan, *differentiable physics*, dibahas lengkap dalam buku Thuerey et al. (2021). Kochkov et al. (2021) memakai pendekatan serupa untuk turbulensi dua dimensi: jaringan mempelajari cara menginterpolasi nilai di sisi sel pada grid yang jauh lebih kasar, dan simulasi yang dihasilkan setara akurasinya dengan solver klasik pada grid berkali-kali lebih halus, dengan percepatan puluhan kali.

32.5 Koreksi yang tetap konservatif

Ada satu cara sederhana untuk memastikan koreksi tidak merusak hukum kekekalan. Alih-alih membiarkan jaringan menambahkan nilai sembarang ke setiap sel, kita membiarkannya mempelajari *fluks* tambahan di sisi-sisi sel, lalu menerapkan koreksi sebagai divergensi fluks itu, persis seperti suku-suku lain di metode volume hingga (Bab 27). Fluks yang keluar dari satu sel adalah fluks yang masuk ke tetangganya, sehingga jumlah koreksi di seluruh domain otomatis nol, kecuali di batas. Total massa atau energi tidak bisa berubah karena koreksi, apa pun bobot jaringannya. Sebaliknya, koreksi sel demi sel tanpa struktur ini bisa diam-diam menambah atau membuang energi di setiap langkah, dan dalam simulasi panjang, kebocoran kecil seperti itu menjadi galat yang besar. Pelajaran ini muncul berulang kali dalam buku ini: struktur yang benar lebih berharga daripada parameter yang lebih banyak.

Koreksi seperti ini juga perlu dibatasi agar tidak menciptakan nilai yang tidak fisis, misalnya temperatur di bawah nilai terkecil di sekitarnya atau konsentrasi negatif. Limiter dari skema konveksi orde tinggi di Bab 27 memberi cara yang teruji untuk melakukannya: fluks koreksi dipotong secukupnya agar tidak ada ekstremum baru yang muncul.

32.6 Ketidakpastian dan banyak tingkat ketelitian

Model hibrida juga cocok untuk memanfaatkan data dari sumber yang berbeda-beda ketelitian dan harganya. Simulasi kasar murah dan banyak, simulasi teliti mahal

dan sedikit, percobaan paling dipercaya tetapi paling jarang. Model *multi-fidelity* mempelajari koreksi dari tingkat murah ke tingkat mahal, yang biasanya jauh lebih sederhana untuk dipelajari daripada solusi lengkapnya. Bentuk paling sederhananya linear: $u_{\text{teliti}}(x) \approx \rho u_{\text{kasar}}(x) + \delta(x)$, dengan faktor skala ρ dan fungsi selisih δ yang dipelajari, misalnya dengan Gaussian process dari Bab 15. Kalau simulasi kasar sudah menangkap pola umumnya, δ adalah fungsi yang halus dan kecil, dan beberapa titik simulasi teliti sudah cukup untuk menaksirnya. Ketidakpastian prediksi bisa diperkirakan dengan ensemble atau model probabilistik (Bab 12), dan ini penting karena model hibrida sering dipakai justru di luar daerah data latihnya.

Asimilasi data dari Bab 14 adalah bentuk hibrida yang paling tua. Solver memberi ramalan, data memberi koreksi, dan keduanya digabung dengan bobot sesuai ketidakpastian masing-masing. Ketika model penutupnya sendiri tidak pasti, parameter model bisa ikut ditaksir dalam proses asimilasi, misalnya dengan menambahkannya ke vektor keadaan Kalman filter ensemble.

32.7 Asimilasi data dari dekat

Asimilasi data layak dibahas lebih rinci, karena ia adalah cara paling langsung untuk mendamaikan simulasi dengan pengukuran. Bentuk paling sederhana adalah *nudging*. Persamaan model ditambah satu suku yang menarik keadaan ke arah pengamatan,

$$\frac{dT}{dt} = F(T) + G(T_{\text{obs}} - T), \quad (32.4)$$

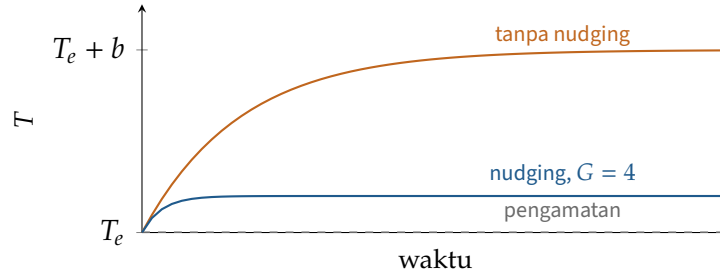
dengan G kekuatan tarikan, kebalikan dari waktu relaksasi $\tau = 1/G$. Kalau G nol, model berjalan sendiri. Kalau G besar, keadaan model praktis diganti pengamatan. Di antaranya, model dan pengamatan berbagi kendali.

Contoh skalar memperlihatkan apa yang dilakukan nudging terhadap galat model. Misalkan model meluruh menuju keadaan setimbang T_e dengan laju satu per satuan waktu, tetapi punya bias b , sehingga $F(T) = -(T - T_e) + b$. Tanpa nudging, keadaan tunaknya $T_e + b$, meleset sebesar bias. Dengan nudging ke pengamatan yang benar, $T_{\text{obs}} = T_e$, keadaan tunak memenuhi $0 = -(T - T_e) + b + G(T_e - T)$, sehingga

$$T = T_e + \frac{b}{1 + G}. \quad (32.5)$$

Galatnya tidak hilang, tetapi diperkecil dengan faktor $1 + G$. Dengan $G = 4$, bias tinggal seperlima. Rumus ini juga memperlihatkan harganya: medan yang dihasilkan tidak lagi murni hasil model, sehingga tidak bisa dipakai sebagai bukti bahwa model sudah benar. Suku nudging itu sendiri, $G(T_{\text{obs}} - T)$, justru informatif, karena besarnya mengukur seberapa banyak model harus dikoreksi, dan di mana. Dalam praktik, nudging hanya diterapkan di sekitar titik sensor, dan informasi menyebar ke tempat lain lewat dinamika model sendiri (Gambar 32.2).

Kalman filter memilih kekuatan tarikan secara optimal dari ketidakpastian, bukan dengan tangan. Untuk model dengan jutaan variabel, matriks kovarians tidak mungkin disimpan, dan ensemble Kalman filter (Evensen, 1994) menaksirnya dari sekumpulan simulasi. Jalankan N anggota ensemble dengan syarat awal atau parameter yang sedikit



GAMBAR 32.2. Model dengan bias b , mulai dari keadaan yang benar. Tanpa nudging ia menyimpang ke $T_e + b$. Dengan nudging berkekuatan $G = 4$, simpangannya tinggal $b/5$ dan dicapai lebih cepat.

berbeda. Kovarians antara keadaan di sebuah titik dan nilai yang terukur di sensor ditaksir dari sebaran anggota, misalnya $P_{xz} \approx \frac{1}{N-1} \sum_i (x_i - \bar{x})(z_i - \bar{z})$, dan gain-nya $K = P_{xz}/(P_{zz} + R)$. Contoh dengan empat anggota: temperatur di sebuah titik bernilai (20, 22, 24, 26) dan di sensor (30, 31, 32, 33). Rata-ratanya 23 dan 31,5. Simpangan setiap anggota dari rata-ratanya adalah $(-3, -1, 1, 3)$ di titik itu dan $(-1,5; -0,5; 0,5; 1,5)$ di sensor, sehingga $P_{xz} = (4,5 + 0,5 + 0,5 + 4,5)/3 \approx 3,33$ dan $P_{zz} = (2,25 + 0,25 + 0,25 + 2,25)/3 \approx 1,67$. Dengan varians pengukuran $R = 1$, $K \approx 3,33/2,67 = 1,25$. Kalau sensor membaca 33, satu setengah derajat di atas rata-rata ensemble, rata-rata temperatur di titik itu dikoreksi naik $1,25 \times 1,5 \approx 1,9$ derajat, walau titik itu sendiri tidak diukur. Ensemble “tahu” bahwa titik dan sensor bergerak bersama, dan memakai pengetahuan itu untuk menyebarkan informasi.

Parameter model juga bisa ikut diasimilasi. Tambahkan parameter yang tidak pasti, misalnya koefisien perpindahan panas dinding, ke vektor keadaan setiap anggota. Parameter itu tidak berubah oleh dinamika, tetapi ia berkorelasi dengan keadaan yang terukur di seluruh ensemble, sehingga koreksi Kalman memperbaruinya juga. Cara ini, yang disebut augmentasi keadaan, menjembatani asimilasi data dan kalibrasi model.

32.8 Apa yang dipelajari, dan apa yang tidak

Pola umum model hibrida bisa diringkas dalam satu kalimat: biarkan fisika menangani apa yang sudah diketahui pasti, terutama hukum kekekalan, dan biarkan data menangani apa yang dimodelkan dengan buruk. Pilihan ini punya keuntungan praktis yang besar. Karena sebagian besar dinamika dihitung oleh solver yang konservatif, model hibrida jauh lebih jarang melanggar kekekalan massa atau energi daripada model pengganti murni. Karena koreksinya kecil, ia butuh lebih sedikit data. Dan karena strukturnya jelas, kegagalannya lebih mudah didiagnosis. Ulasan Vinuesa and Brunton (2022) dan Brunton et al. (2020) menempatkan pendekatan ini sebagai salah satu jalur yang paling menjanjikan untuk mekanika fluida komputasi.

32.9 Perkembangan riset

Contoh paling meyakinkan untuk model hibrida datang dari cuaca dan iklim. NeuralGCM (Kochkov et al., 2024) menggabungkan solver dinamika atmosfer yang bisa

diturunkan dengan komponen machine learning yang menggantikan parameterisasi proses skala kecil seperti pembentukan awan. Seluruh model dilatih ujung ke ujung dengan gradien yang dibawa melewati solver, yaitu solver-in-the-loop dalam skala planet. Untuk prakiraan satu sampai sepuluh hari, kinerjanya bersaing dengan model machine learning murni, dan untuk ensemble satu sampai lima belas hari, dengan sistem ensemble pusat prakiraan Eropa. Dengan temperatur permukaan laut yang ditetapkan, model ini juga melacak besaran iklim selama beberapa dekade, sesuatu yang sulit bagi model pengganti murni yang cenderung tidak stabil. Penulisnya juga mencatat batasnya: model ini tidak mengekstrapolasi ke iklim masa depan yang jauh berbeda, pengingat bahwa data tetap membatasi jangkauan bagian yang dipelajari.

CATATAN SUMBER. Bab ini dirangkai dari kuliah Numerical Methods in Fluid Mechanics (model penutup turbulensi) dan bacaan selama tugas akhir. Rujukan utama: Duraisamy et al. (2019), Ling et al. (2016), Um et al. (2020), Kochkov et al. (2021), dan Thuerey et al. (2021).

33 Recurrence CFD: Simulasi dari Ingatan

Recurrence CFD adalah topik tugas akhir saya, dan kuliah metode numerik fluida memberinya satu pertemuan penuh. Stefan Pirker membukanya dengan Max dan Moritz, dua anak nakal dari cerita bergambar Wilhelm Busch, dan guru mereka, Lehrer Lämpel. Max dan Moritz membuat tebakan yang berani, sang guru mengoreksinya. Menurut Pirker, begitulah cara algoritme SIMPLE menyelesaikan persamaan Navier-Stokes: tebakan yang tidak sempurna, lalu koreksi berbasis fisika. Dan begitulah juga rCFD dikembangkan. Bab ini mengikuti alur kuliah itu, lalu menceritakan sedikit tentang apa yang saya kerjakan dalam tugas akhir.

33.1 Aliran yang mengulang dirinya

Banyak aliran yang menarik bagi industri berjalan lama sekali. Peleburan baja di dalam ladle, pencampuran di tangki berpengaduk, dan gelembung yang naik di kolom gelembung bisa berlangsung menit sampai jam, sedangkan simulasi CFD yang teliti hanya sanggup menjangkau beberapa puluh detik waktu fisis dalam waktu hitung yang wajar. Untungnya, banyak aliran seperti ini, setelah melewati fase awal, tidak menghasilkan pola yang benar-benar baru. Pusaran datang dan pergi, gelembung naik dalam pola yang mirip, dan keadaan aliran berulang kali kembali ke dekat keadaan yang pernah dikunjungi sebelumnya.

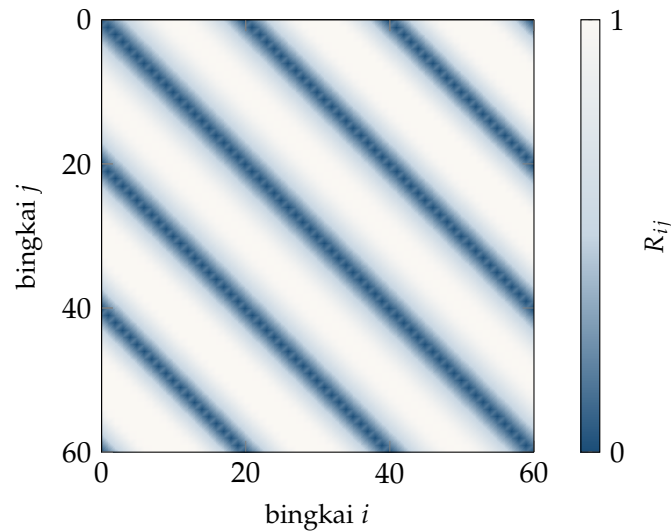
Pengulangan ini bisa diukur. Simpan medan aliran dari simulasi penuh sebagai deretan bingkai $\phi_1, \dots, \phi_M$. Untuk setiap pasangan bingkai, hitung jaraknya, lalu normalisasikan:

$$R_{ij} = \frac{\|\phi_i - \phi_j\|}{\max_{k,l} \|\phi_k - \phi_l\|}. \quad (33.1)$$

Matriks R ini disebut *recurrence matrix*. Diagonalnya nol, karena setiap bingkai identik dengan dirinya sendiri. Kalau aliran periodik sempurna, garis-garis gelap sejajar diagonal muncul pada jarak satu periode. Pada aliran turbulen, polanya lebih kabur, tetapi tetap ada daerah-daerah di luar diagonal dengan nilai kecil: pasangan bingkai yang berjauhan dalam waktu tetapi mirip satu sama lain. Dalam bahasa machine learning, R adalah matriks jarak antar sampel, dan pasangan dengan jarak kecil adalah tetangga terdekat di ruang keadaan aliran. Gambar 33.1 memperlihatkan bentuk khas matriks ini untuk aliran yang hampir periodik.

33.2 rCFD berbasis aliran

Gagasan pertama, yang diterbitkan Lichtenegger and Pirker (2016) untuk kolom gelembung, berbunyi seperti kalimat yang berulang di slide: “kita putuskan dengan berani untuk menyusun aliran buatan dengan menjahit potongan-potongan rangkaian aliran”.



GAMBAR 33.1. Bentuk recurrence matrix untuk aliran yang hampir periodik dengan periode sekitar dua puluh bingkai (ilustrasi). Garis gelap di diagonal dan sejajar diagonal menandai pasangan bingkai yang mirip, kandidat tempat melompat saat menjahit aliran.

Mulailah dari sebuah bingkai, mainkan beberapa bingkai berikutnya dari basis data, lalu lompat ke bingkai lain yang mirip dengan bingkai terakhir menurut recurrence matrix, dan teruskan dari sana. Hasilnya adalah deret medan aliran sepanjang yang kita mau, yang secara statistik meniru aliran asli, tanpa pernah menyelesaikan persamaan Navier–Stokes lagi.

Di atas aliran yang dijahit ini, besaran yang terbawa aliran, misalnya konsentrasi zat terlarut, dilacak dengan persamaan transport implisit (Persamaan (26.6)). Persamaan transport untuk satu besaran skalar jauh lebih murah daripada persamaan Navier–Stokes yang terkopel, dan karena implisit, langkah waktunya bisa jauh lebih besar. Pada kasus kolom gelembung, rCFD berbasis aliran menghasilkan prediksi yang cocok dengan CFD penuh dengan percepatan sekitar seratus kali.

Bayangkan mencoba meramalkan arus lalu lintas sebuah kota selama setahun. Daripada mensimulasikan setiap mobil, kita merekam lalu lintas selama beberapa minggu, lalu menyusun tahun buatan dengan menyambung potongan-potongan hari yang mirip: setelah Senin pagi yang macet biasanya datang siang yang agak sepi. Kalau yang kita pedulikan adalah ke mana sebuah paket akan sampai, rekaman lalu lintas yang dijahit sudah cukup.

33.3 rCFD berbasis transport

Langkah kedua, dari Pirker and Lichtenegger (2018), bahkan melewati persamaan transport. Kuliah menjelaskannya dengan sungai Danube di Linz. Sebuah zat yang dijatuhkan ke sungai dibawa arus dan menyebar. Arus membawa zat dari satu tempat ke tempat lain, yaitu konveksi, dan pusaran kecil membuatnya menyebar, yaitu difusi. Dalam bahasa sel-sel volume hingga, konveksi berarti sebagian isi sebuah sel berpindah

ke sel lain, dan difusi berarti dua sel bertetangga saling bertukar isi. Mesh simulasi adalah jaringan komunikasi antar sel. rCFD berbasis transport memutuskan, sekali lagi dengan berani, untuk mengganti konveksi dengan *cell-to-cell shifts* dan difusi dengan *face swaps* yang direkam dari simulasi penuh.

Secara sederhana, satu langkah transport menjadi perkalian dengan sebuah matriks pemindahan. Kalau c_i adalah jumlah zat di sel i dan P_{ji} fraksi isi sel i yang dipindahkan ke sel j dalam satu langkah, maka

$$c_j^{\text{baru}} = \sum_i P_{ji} c_i. \quad (33.2)$$

Massa total kekal tepat ketika setiap kolom P berjumlah satu, $\sum_j P_{ji} = 1$: semua isi sel i pergi ke suatu tempat, tidak ada yang hilang dan tidak ada yang muncul. Buktinya satu baris: jumlah semua c_j^{baru} adalah $\sum_i c_i \sum_j P_{ji} = \sum_i c_i$.

Contoh tiga sel di sebuah saluran tertutup membuatnya konkret. Aliran membawa isi dari sel pertama ke sel kedua dan dari sel kedua ke sel ketiga, dan sel ketiga adalah ujung buntu:

$$P = \begin{pmatrix} 0,7 & 0 & 0 \\ 0,3 & 0,6 & 0 \\ 0 & 0,4 & 1 \end{pmatrix}. \quad (33.3)$$

Setiap kolom berjumlah satu. Mulai dengan seluruh zat di sel pertama, $c = (1, 0, 0)$. Setelah satu langkah $c = (0,7; 0,3; 0)$, setelah dua langkah $(0,49; 0,39; 0,12)$, dan jumlahnya tetap satu. Sekarang bayangkan kolom pertama hanya berjumlah 0,98 karena dua bingkai yang dijahit tidak cocok sempurna. Zat yang melewati sel itu kehilangan dua persen setiap kali, dan setelah seratus langkah, $0,98^{100} \approx 0,13$. Galat yang tampak kecil dalam satu langkah menghabiskan hampir seluruh zat dalam simulasi panjang. Inilah bentuk kegagalan pertama di bawah, dan juga alasan mengapa kekekalan harus dijaga secara struktural, bukan diharapkan muncul sendiri. Pembaca yang ingat Bab 31 akan mengenali bahwa matriks seperti ini adalah versi diskret dari operator transfer, saudara operator Koopman, yang memajukan distribusi alih-alih memajukan pengamatan.

Selisih kecepatannya besar sekali. Untuk simulasi pencampuran di sebuah ladle baja, slide kuliah membandingkan CFD penuh yang membutuhkan 26 jam dengan rCFD berbasis transport yang selesai dalam 1,6 detik pada perangkat keras yang sama, dengan histogram konsentrasi yang cocok di beberapa titik waktu.

33.4 Memilih lompatan dan menilai hasil

Penjahitan aliran membutuhkan dua keputusan praktis. Pertama, kapan melompat. Kalau lompatan dilakukan terlalu sering, aliran yang dijahit kehilangan kontinuitas waktu, dan struktur yang seharusnya terbawa beberapa langkah terputus di tengah jalan. Kalau terlalu jarang, basis data dimainkan ulang hampir apa adanya, dan statistik jangka panjangnya hanya sebaik panjang rekaman. Biasanya rekaman dimainkan selama beberapa bingkai berturut-turut, lalu lompatan dilakukan ke salah satu dari beberapa kandidat terdekat menurut recurrence matrix, dipilih secara acak agar urutan yang dijahit tidak berputar di lingkaran yang sama.

Kedua, berapa jauh boleh melompat. Ambang kemiripan yang ketat menjamin sambungan yang mulus tetapi mempersempit pilihan, sedangkan ambang longgar memberi banyak pilihan dengan sambungan yang lebih kasar. Masalah ini mirip dengan pemilihan k pada k -nearest neighbour di Bab 6, dan jawabannya juga mirip: tidak ada angka yang benar untuk semua kasus, sehingga pengaruhnya harus diuji.

Menilai hasil rCFD juga butuh hati-hati. Karena aliran yang dijahit tidak pernah identik dengan aliran asli bingkai demi bingkai, membandingkan medan sesaat tidak adil, alasan yang sama dengan evaluasi rollout di Bab 30. Yang dibandingkan adalah besaran yang dipedulikan: kurva konsentrasi atau temperatur di titik-titik pemantauan, waktu yang dibutuhkan untuk mencapai keadaan tercampur, dan distribusi besaran di seluruh domain. Neraca global, seperti total massa atau energi, menjadi pemeriksaan pertama yang murah, karena galat neraca yang tumbuh adalah tanda paling awal bahwa sesuatu di dalam matriks pemindahan tidak beres.

33.5 Tiga kegagalan berani dan koreksinya

Bagian paling jujur dari kuliah adalah tiga “kegagalan berani” rCFD dan cara memperbaikinya.

Kegagalan pertama adalah ketidakseimbangan massa. Untuk simulasi yang panjang, hasil rCFD menyimpang dari CFD penuh. Penyebabnya, pemindahan antar sel yang direkam dari bingkai berbeda dan dijahit bersama tidak menjamin bahwa setiap kolom P berjumlah tepat satu. Secara lokal, sebagian massa hilang dan sebagian lain tercipta. Secara global, neraca massa spesies bisa dihitung dengan mudah, sehingga galat rCFD bisa dipantau. Koreksinya memakai pemantau global itu untuk menyesuaikan pola konsentrasi lokal lewat face swaps yang bekerja pada massa, sehingga neraca kembali seimbang.

Kegagalan kedua adalah difusi yang tidak fisis. Face swaps yang sama kuatnya di semua tempat membuat zat menyebar terlalu cepat di daerah tenang dan terlalu lambat di daerah turbulen. Du et al. (2020), yang menerapkan rCFD pada penyebaran polutan di sekitar bangunan, memperbaikinya dengan membuat kekuatan face swaps bergantung pada turbulensi lokal, yang dalam LES dinyatakan oleh viskositas sub-grid.

Kegagalan ketiga adalah pengulangan yang kurang menonjol. Di domain yang besar, tidak ada dua bingkai yang mirip di *semua* tempat sekaligus. Penjahitan global memasang bingkai yang secara rata-rata mirip tetapi secara lokal, misalnya di ekor pusaran di belakang sebuah gedung, tidak mirip sama sekali. Usulan koreksinya adalah memecah domain menjadi “pulau-pulau” dengan pengulangan yang kuat dan “laut” latar dengan pengulangan yang lemah, masing-masing dengan recurrence matrix sendiri.

33.6 Menuju kembaran digital

Kuliah ditutup dengan visi yang lebih besar. Kalau rCFD begitu cepat, ia bisa berjalan bersamaan dengan proses industri yang sebenarnya, sebagai kembaran digital. Masalahnya, proses nyata punya lebih dari satu keadaan operasi, dan rCFD yang dijalankan

maju tanpa koreksi pasti melenceng dari kenyataan. Jalan keluarnya mirip asimilasi data di Bab 14: jalankan rCFD sambil mengoreksinya dengan pengukuran proses yang tersedia secara langsung. Kuliah memperlihatkan percobaan sederhana untuk ide ini, dua aliran air panas dan dingin yang dicampur di sebuah tangki dengan deretan sensor temperatur vertikal. Model tangki ideal yang tercampur sempurna terlalu sederhana karena mengabaikan perbedaan di dalam tangki, sedangkan CFD tanpa validasi rawan galat. rCFD yang diturunkan dari CFD dan dikoreksi dengan data sensor menempati tempat di antara keduanya.

33.7 Tugas akhir saya

Tugas akhir saya menerapkan rCFD berbasis transport pada pancaran apung (*buoyant jet*) di dalam sebuah tangki: air hangat yang masuk dan naik karena lebih ringan, lalu bercampur dengan air di sekitarnya. Kasus ini menyentuh dua kegagalan di atas sekaligus. Pancaran yang naik membentuk lapisan temperatur yang tajam, dan setiap difusi tambahan dari pemindahan antar sel mengaburkan lapisan itu. Ketika dinding tangki kehilangan panas ke lingkungan, alirannya juga tidak lagi berulang sempurna, karena temperatur rata-rata terus turun.

Benang merah pekerjaannya adalah mengambil lebih banyak informasi dari simulasi penuh dan menyimpannya di dalam basis data rCFD, alih-alih memakai satu konstanta global. Salah satu contohnya adalah difusivitas lokal yang direkam dari CFD dan dipakai kembali saat replay, gagasan yang sejalan dengan koreksi Du et al. (2020). Contoh lain adalah difusivitas negatif yang terkendali untuk mempertajam kembali gradien yang terlalu kabur, dengan pembatas seperti pada skema flux-corrected transport agar tidak muncul osilasi. Hasil-hasil kuantitatifnya ada di laporan tugas akhir. Yang ingin saya sampaikan di sini lebih sederhana: setiap perbaikan mengikuti pola yang sama dengan yang diajarkan di kuliah, yaitu tebakan yang berani diikuti koreksi berbasis fisika.

33.8 rCFD dari sudut pandang machine learning

Setelah membaca bab-bab sebelumnya, rCFD bisa dilihat dengan mata machine learning. Basis data bingkai adalah ingatan, recurrence matrix adalah ukuran kemiripan antar keadaan, dan penjahitan adalah mencari tetangga terdekat di ruang keadaan lalu melompat ke sana. Tidak ada jaringan saraf dan tidak ada pelatihan dengan gradien. rCFD adalah model pengganti nonparametrik, mirip k -nearest neighbour (Bab 6) yang diterapkan pada lintasan dinamika. Matriks pemindahan antar sel adalah operator transfer yang dipelajari dari data, saudara dekat DMD dan Koopman (Bab 31). Dan koreksi neraca massa adalah contoh paling jelas dari prinsip model hibrida (Bab 32): biarkan data memberi dinamika, dan biarkan hukum kekekalan menjaga agar hasilnya tetap fisis.

Kekuatannya juga kelemahannya. Karena hanya memutar ulang dan menyusun kembali apa yang pernah dilihat, rCFD tidak bisa menghasilkan keadaan aliran yang benar-benar baru. Untuk aliran yang memang mengulang dirinya, itu justru jaminan: hasilnya tidak akan berhalusinasi. Untuk aliran yang berubah keadaan operasinya,

dibutuhkan basis data dengan banyak keadaan dan cara memilih di antaranya, persis pertanyaan yang diangkat kuliah tentang proses nyata yang punya lebih dari satu keadaan operasi. Di sinilah rCFD dan model pengganti berbasis jaringan saraf, seperti yang disinggung di slide terakhir kuliah, mungkin akan saling melengkapi.

33.9 Perkembangan riset

Riset rCFD terus berjalan setelah kuliah. Beberapa publikasi terbaru memperlihatkan arah-arahnya. Hoorijani (2025) mengusulkan rCFD daring, yang menjalankan analisis pengulangan sambil simulasi berjalan, untuk aliran multifase. Pirker et al. (2025) menerapkan CFD-DEM dan rCFD pada segregasi ukuran partikel di unggun terfluidisasi gas–padat dengan dua dan lima ukuran partikel. Lumetzberger et al. (2025) mengusulkan aproksimasi momen-propagator untuk transport spesies dalam aliran turbulen, sebuah cara lain merumuskan langkah transport rCFD. Shariatifar et al. (2026) menilai akurasi rCFD untuk sistem aliran kontinu, dan Shaik et al. (2026) membandingkan simulasi CFD-DEM sebuah unggun terfluidisasi bergelembung dengan pengukuran MRI sebelum memakai rCFD untuk ekstrapolasi waktu. Arah-arrah ini sejalan dengan tiga kegagalan di bab ini: menjaga neraca, memperhalus difusi, dan memperluas rCFD ke aliran yang tidak periodik sempurna.

CATATAN SUMBER. Bab ini mengikuti pertemuan tentang recurrence CFD dalam kuliah Numerical Methods in Fluid Mechanics dan pengalaman selama tugas akhir. Makalah utamanya: Lichtenegger and Pirker (2016) untuk rCFD berbasis aliran, Pirker and Lichtenegger (2018) untuk rCFD berbasis transport, dan Du et al. (2020) untuk penerapan pada lingkungan bangunan.

Bagian VI

AI, Manusia, dan Masa Depan

34 Belajar dari Jejak Pengguna

Kuliah learning from user-generated data saya ikuti pada musim panas 2025, diberikan oleh kelompok Markus Schedl, yang banyak meneliti rekomendasi musik. Sebelas kuliahnya bergerak dari collaborative filtering klasik ke evaluasi, data teks dan data implisit, sistem rekomendasi berbasis graf, hibrida, dan konteks, lalu ditutup dengan dua topik yang jarang dibahas di kuliah teknis: bias dan keadilan, serta sistem rekomendasi yang diilhami psikologi. Latihannya membangun rekomendasi item-kNN dari umpan balik implisit, model faktor laten, dan evaluasi dengan nDCG.

Sistem rekomendasi barangkali adalah machine learning yang paling sering kita temui tanpa disadari: video berikutnya, lagu berikutnya, produk yang “mungkin juga Anda suka”. Datanya adalah jejak yang ditinggalkan jutaan pengguna, berupa rating, klik, putaran lagu, dan waktu menonton, dan tugasnya adalah menebak apa yang akan disukai seseorang dari apa yang disukai orang-orang lain.

34.1 Collaborative filtering

Data dasarnya adalah matriks pengguna terhadap item, dengan rating di sel yang diketahui dan kosong di sebagian besar sel lainnya. *Collaborative filtering* berbasis pengguna mengikuti logika sederhana: cari pengguna lain yang selera masa lalunya mirip denganmu, lalu rekomendasikan apa yang mereka sukai. Kemiripan biasanya diukur dengan kosinus atau korelasi Pearson antara vektor rating. Untuk dua pengguna dengan rating (5, 3, 4) dan (4, 2, 5) pada tiga item yang sama, kemiripan kosinusnya

$$\frac{5 \cdot 4 + 3 \cdot 2 + 4 \cdot 5}{\sqrt{5^2 + 3^2 + 4^2} \sqrt{4^2 + 2^2 + 5^2}} = \frac{46}{\sqrt{50} \sqrt{45}} \approx 0,97. \quad (34.1)$$

Korelasi Pearson mengurangkan rata-rata rating setiap pengguna lebih dulu, sehingga pengguna yang selalu memberi rating tinggi dan pengguna yang pelit rating tetap bisa dikenali sebagai bersepakat. Contohnya, pengguna dengan rating (5, 4, 5) dan pengguna dengan rating (3, 2, 3) punya selera yang sama persis, hanya skalanya yang berbeda. Setelah rata-ratanya dikurangkan, keduanya menjadi $(\frac{1}{3}, -\frac{2}{3}, \frac{1}{3})$, dan korelasinya tepat satu.

Prediksi rating lalu dihitung sebagai rata-rata berbobot dari rating tetangga-tetangga terdekat, juga relatif terhadap rata-rata masing-masing:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} s_{uv} (r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |s_{uv}|}, \quad (34.2)$$

dengan $N(u)$ tetangga-tetangga u yang pernah menilai item i dan s_{uv} kemiripannya. Misalkan rata-rata rating Ani 3,5, dan dua tetangganya menilai sebuah film dengan +1 dan +0,5 di atas rata-rata mereka sendiri, dengan kemiripan 0,9 dan 0,6. Prediksinya

$3,5 + (0,9 + 0,3)/1,5 = 4,3$. Rekomendasi ini mudah dijelaskan kepada pengguna: “orang-orang dengan selera serupa denganmu menyukai film ini”.

Versi berbasis item (Sarwar et al., 2001) membalik sudut pandangnya: item yang sering disukai oleh orang yang sama dianggap mirip, dan kita merekomendasikan item yang mirip dengan yang sudah kamu sukai. Karena jumlah item biasanya lebih stabil daripada jumlah pengguna, kemiripan antar item bisa dihitung lebih jarang dan dipakai ulang, sehingga pendekatan ini lebih mudah diskalakan.

34.2 Faktor laten

Pendekatan yang menjadi inti solusi-solusi terbaik dalam kompetisi Netflix Prize adalah faktorisasi matriks (Koren et al., 2009). Setiap pengguna u dan setiap item i diwakili oleh vektor berdimensi kecil, $\mathbf{p}_u$ dan $\mathbf{q}_i$, dan rating diprediksi sebagai

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i, \quad (34.3)$$

dengan μ rata-rata global, b_u kecenderungan pengguna memberi rating tinggi atau rendah, dan b_i kecenderungan item dinilai tinggi atau rendah. Vektor-vektor ini dilatih dengan meminimalkan galat kuadrat pada rating yang diketahui saja, ditambah regularisasi L2:

$$\mathcal{L} = \sum_{(u,i) \text{ diketahui}} (r_{ui} - \hat{r}_{ui})^2 + \lambda (b_u^2 + b_i^2 + \|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2). \quad (34.4)$$

Turunan terhadap $\mathbf{p}_u$ untuk satu rating, dengan galat $e_{ui} = r_{ui} - \hat{r}_{ui}$, adalah $-2e_{ui}\mathbf{q}_i + 2\lambda\mathbf{p}_u$. Langkah SGD (Bab 4) dengan laju η , faktor dua diserap ke dalam η , menjadi

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui}\mathbf{q}_i - \lambda\mathbf{p}_u), \quad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta(e_{ui}\mathbf{p}_u - \lambda\mathbf{q}_i). \quad (34.5)$$

Alternatif yang dibahas di kuliah adalah *alternating least squares* (ALS). Kalau semua vektor item $\mathbf{q}_i$ dibekukan, loss di atas menjadi masalah kuadrat terkecil biasa untuk setiap $\mathbf{p}_u$, dengan regresi ridge yang punya solusi tertutup. Menyamakan gradien dengan nol memberi

$$\mathbf{p}_u = (\mathbf{Q}_u^\top \mathbf{Q}_u + \lambda \mathbf{I})^{-1} \mathbf{Q}_u^\top \mathbf{r}_u, \quad (34.6)$$

dengan $\mathbf{Q}_u$ matriks berisi vektor item yang dinilai u dan $\mathbf{r}_u$ rating-ratingnya (setelah bias dikurangkan). Lalu peran ditukar: bekukan semua $\mathbf{p}_u$ dan selesaikan setiap $\mathbf{q}_i$ dengan cara yang sama. Setiap langkah tidak pernah menaikkan loss, dan setiap pengguna bisa diselesaikan secara paralel. Untuk data implisit, ALS punya keuntungan tambahan: semua sel kosong bisa diperlakukan sebagai nol dengan bobot kecil tanpa harus diiterasi satu per satu.

Dimensi-dimensi laten tidak diberi nama, tetapi sering bisa ditafsirkan: untuk film, satu dimensi mungkin memisahkan film serius dari komedi, dimensi lain film untuk penonton dewasa dari film keluarga. Ini analisis faktor dari Bab 6, diterapkan pada matriks yang sebagian besar kosong.

Dalam praktik, kebanyakan data bukan rating, melainkan umpan balik implisit: klik, putaran, pembelian. Data ini tidak punya sinyal negatif yang jelas, karena tidak

mengklik bisa berarti tidak suka atau sekadar tidak melihat. Bayesian personalized ranking (Rendle et al., 2012) mengubah tujuannya menjadi peringkat: untuk setiap pengguna, item yang pernah ia pakai seharusnya diberi skor lebih tinggi dari item yang belum. Loss-nya $-\sum \ln \sigma(\hat{x}_{ui} - \hat{x}_{uj})$ untuk pasangan item i yang dipakai dan j yang tidak, sebuah binary cross-entropy atas pasangan. Neural collaborative filtering (He et al., 2017) mengganti hasil kali titik dengan jaringan saraf, dan pendekatan berbasis graf seperti LightGCN (He et al., 2020) memandang pengguna dan item sebagai simpul graf bipartit dan menyebarkan embedding lewat message passing (Bab 13).

Faktorisasi matriks paling kecil bisa diikuti dengan tangan. Dua pengguna dan dua film, dengan satu dimensi laten dan tanpa bias: Ani memberi film pertama rating 4, Budi memberi film pertama 2 dan film kedua 1, dan rating Ani untuk film kedua belum diketahui. Model $\hat{r}_{ui} = p_u q_i$ dengan $p_{\text{Ani}} = 2$, $p_{\text{Budi}} = 1$, $q_1 = 2$, dan $q_2 = 1$ menjelaskan ketiga rating yang diketahui dengan tepat. Prediksinya untuk sel yang kosong adalah $2 \cdot 1 = 2$. Dimensi laten tunggal ini bisa dibaca sebagai “seberapa suka menonton film” untuk pengguna dan “seberapa disukai” untuk film. Ani menyukai segala sesuatu dua kali lebih dari Budi, dan film kedua kurang disukai, sehingga Ani diramal memberi film kedua rating yang sama dengan rating Budi untuk film pertama. Solusinya tidak unik: mengalikan semua p dengan dua dan membagi semua q dengan dua memberi prediksi yang sama, dan regularisasi L2 di loss memilih skala yang menyeimbangkan keduanya.

Untuk umpan balik implisit, Hu et al. (2008) memisahkan dua hal yang di data rating menyatu: *preferensi* dan *keyakinan*. Preferensi p_{ui} bernilai satu kalau pengguna pernah memakai item dan nol kalau tidak. Keyakinan $c_{ui} = 1 + \alpha r_{ui}$ naik dengan jumlah interaksi r_{ui} , misalnya berapa kali sebuah lagu diputar. Loss-nya adalah kuadrat terkecil berbobot atas *semua* sel, $\sum_{u,i} c_{ui} (p_{ui} - \mathbf{x}_u^\top \mathbf{y}_i)^2$ ditambah regularisasi. Lagu yang diputar lima puluh kali dengan $\alpha = 40$ diberi bobot 2001, sedangkan lagu yang tidak pernah diputar tetap ikut dihitung dengan bobot satu, sebagai bukti lemah bahwa pengguna tidak tertarik. Karena semua sel ikut, SGD menjadi terlalu mahal, dan ALS di atas dipakai dengan trik aljabar yang membuat sel-sel kosong tidak perlu diiterasi satu per satu.

34.3 Mengukur rekomendasi yang baik

Galat rating seperti RMSE kurang relevan kalau yang ditampilkan adalah daftar sepuluh teratas. Ukuran yang peka peringkat lebih cocok. Precision dan recall pada k teratas menghitung berapa item relevan yang masuk daftar. *Discounted cumulative gain* (Järvelin and Kekäläinen, 2002) menambahkan bahwa item relevan di urutan atas lebih berharga daripada di urutan bawah,

$$\text{DCG}@k = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+1)}, \quad (34.7)$$

dan versi ternormalisasinya, nDCG, membaginya dengan DCG dari urutan terbaik yang mungkin. Untuk daftar dengan relevansi (3, 2, 0, 1), DCG-nya $3/1 + 2/1,585 + 0/2 + 1/2,322 \approx 3 + 1,262 + 0 + 0,431 = 4,693$. Urutan idealnya (3, 2, 1, 0) dengan DCG

$3 + 1,262 + 0,5 + 0 = 4,762$. Maka $nDCG \approx 0,985$: hampir sempurna, hanya item relevan terakhir yang sedikit terlambat.

Untuk relevansi biner, dua ukuran lain sering dipakai. *Average precision* meratakan presisi di setiap posisi tempat item relevan muncul. Kalau item relevan ada di posisi 1 dan 3 dari daftar, presisi di kedua posisi itu $1/1$ dan $2/3$, sehingga $AP = (1 + 0,667)/2 \approx 0,83$. Rata-ratanya atas semua pengguna disebut MAP. *Reciprocal rank* hanya melihat item relevan pertama: satu per posisinya. Kuliah menekankan bahwa ukuran-ukuran ini harus dirata-ratakan atas banyak pengguna, karena sistem yang baik untuk pengguna aktif bisa buruk untuk pengguna yang jarang datang.

Kuliah menekankan bahwa akurasi bukan segalanya. Daftar yang akurat tetapi hanya berisi item yang sudah populer tidak membantu siapa pun menemukan sesuatu yang baru. Ukuran di luar akurasi menilai keragaman daftar, kebaruan, cakupan katalog, dan *serendipity*, yaitu kejutan yang menyenangkan. Evaluasi offline pada data historis juga berbeda dari evaluasi online dengan pengguna sungguhan, dan studi dengan pengguna sering memperlihatkan bahwa yang dirasakan pengguna tidak selalu sejalan dengan angka offline.

Cara membagi data juga menentukan apa yang sebenarnya diukur. Pembagian acak atas interaksi membiarkan model melihat masa depan seorang pengguna saat ditanya tentang masa lalunya. Protokol *leave-one-out* menyisihkan interaksi terakhir setiap pengguna sebagai data uji, dan pembagian temporal memotong seluruh data pada satu titik waktu, sehingga semua yang terjadi setelahnya menjadi data uji. Pembagian temporal paling dekat dengan penerapan sungguhan, dan skornya biasanya paling rendah. Popularity bias juga bisa diukur langsung. Salah satu ukurannya adalah rata-rata popularitas item dalam daftar rekomendasi, dibanding rata-rata popularitas item dalam riwayat pengguna itu sendiri. Kalau seorang pendengar lagu-lagu langka mendapat daftar yang rata-rata popularitasnya sepuluh kali lebih tinggi dari riwayatnya, sistemnya sedang menarik dia ke arus utama.

Urutan interaksi juga membawa informasi. Rekomendasi berurutan memandang riwayat pengguna sebagai kalimat dan item berikutnya sebagai kata berikutnya. SASRec (Kang and McAuley, 2018) memakai self-attention dengan causal mask dari Bab 9: pada data yang jarang, model cenderung bertumpu pada beberapa item terakhir, mirip rantai Markov, dan pada data yang padat ia bisa memakai riwayat yang lebih panjang, mirip RNN, karena attention sendiri yang memutuskan item mana yang relevan.

34.4 Konten, konteks, dan hibrida

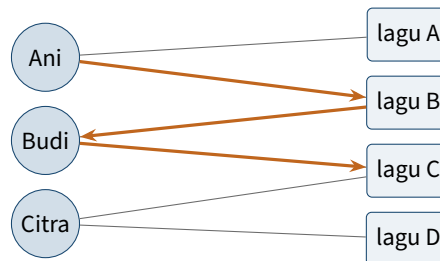
Collaborative filtering gagal untuk item baru yang belum pernah dipakai siapa pun dan pengguna baru yang belum punya riwayat, masalah yang disebut *cold start*. Rekomendasi berbasis konten mengisi celah ini dengan memakai sifat item itu sendiri: teks deskripsi, tag, atau sinyal audio sebuah lagu. Untuk teks, bobot TF-IDF memberi nilai tinggi pada kata yang sering muncul di sebuah dokumen tetapi jarang di dokumen lain,

$$\text{tfidf}(w, d) = \text{tf}(w, d) \cdot \log \frac{N}{\text{df}(w)}, \quad (34.8)$$

dengan N jumlah dokumen dan $df(w)$ jumlah dokumen yang memuat kata w . Dalam seribu deskripsi lagu, kata “dan” muncul di hampir semua, sehingga faktor logaritmanya mendekati nol. Kata “gamelan” muncul di sepuluh deskripsi saja, sehingga faktornya $\ln 100 \approx 4,6$, dan dua lagu yang sama-sama memuatnya dianggap mirip. Model yang lebih maju memakai embedding kata dan model topik seperti LDA, yang menguraikan setiap dokumen menjadi campuran beberapa topik dan setiap topik menjadi distribusi atas kata. Sistem hibrida menggabungkan kedua sumber, misalnya dengan meratakan skor, beralih di antara keduanya tergantung situasi, atau memasukkan fitur konten ke dalam model faktor.

Konteks juga mengubah apa yang pantas direkomendasikan. Lagu yang cocok untuk berolahraga pagi berbeda dari lagu untuk menjelang tidur. Kuliah memberi contoh dari riset kelompoknya sendiri tentang rekomendasi musik yang memperhitungkan lokasi dan aktivitas, dengan memetakan tempat dan lagu ke tag yang sama. Rekomendasi berbasis graf pengetahuan menyebarkan preferensi pengguna lewat relasi antar item, seperti penyanyi, genre, dan label rekaman.

Graf juga memberi cara yang intuitif untuk rekomendasi. Pengguna dan item menjadi simpul graf bipartit, dan setiap interaksi menjadi sisi (Gambar 34.1). Sebuah langkah acak dari pengguna ke item yang ia dengar, lalu ke pengguna lain yang mendengar item itu, lalu ke item lain yang didengar pengguna kedua, adalah collaborative filtering dalam bentuk jalan kaki. Item yang paling sering dicapai jalan acak tiga langkah dari seorang pengguna adalah kandidat rekomendasinya. Dalam bentuk matriks, kalau A adalah matriks interaksi, skor itu sebanding dengan $AA^T A$. Graf yang sama bisa diperkaya dengan simpul genre, penyanyi, atau negara, sehingga jalan acak juga bisa melewati pengetahuan tentang item.



GAMBAR 34.1. Graf bipartit pengguna dan lagu. Jalan acak Ani → lagu B → Budi → lagu C menghasilkan kandidat rekomendasi lagu C untuk Ani.

34.5 Bias, keadilan, dan psikologi

Sistem rekomendasi tidak hanya mencerminkan selera, tetapi juga membentuknya. *Popularity bias* membuat item yang sudah populer semakin sering direkomendasikan, sehingga semakin populer lagi. Kinerja sistem juga bisa berbeda antar kelompok pengguna, misalnya menurut usia, jenis kelamin, atau negara. Kuliah membahas ukuran untuk mendeteksinya, seperti selisih kinerja rekomendasi antar kelompok, dan contoh dari data musik tentang bagaimana algoritme rekomendasi bisa menggeser porsi musik lokal yang diterima pendengar di berbagai negara. Strategi perbaikannya bisa

diterapkan sebelum pelatihan, misalnya menyeimbangkan ulang data, selama pelatihan dengan kendala keadilan, atau sesudahnya dengan menyusun ulang daftar.

Kuliah terakhir menghubungkan rekomendasi dengan psikologi kognitif. Ingatan manusia memudar menurut kurva lupa Ebbinghaus, dan lagu yang terakhir didengar beberapa waktu lalu lebih mudah diingat kembali kalau sering diulang. Model memori dari arsitektur kognitif seperti ACT-R (Anderson et al., 2004) dipakai untuk meramalkan lagu mana yang ingin didengar lagi seseorang, dengan menimbang seberapa sering dan seberapa baru ia mendengarkannya. Persamaan aktivasi dasarnya

$$B_i = \ln \sum_{j=1}^n t_j^{-d}, \quad (34.9)$$

dengan t_j waktu sejak pemutaran ke- j dan d laju peluruhan, sering 0,5. Setiap pemutaran menyumbang, dan sumbangannya memudar seperti pangkat waktu. Lagu yang diputar sekali satu jam lalu menyumbang $1^{-0,5} = 1$. Lagu yang diputar empat kali, masing-masing sekitar seratus jam lalu, menyumbang $4 \times 100^{-0,5} = 0,4$. Jadi satu pemutaran baru lebih “hidup” di ingatan daripada empat pemutaran lama, dan model memakai aktivasi ini untuk mengurutkan lagu yang kemungkinan besar ingin diulang. Kuliah juga membahas kepribadian, misalnya lima dimensi Big Five yang diukur dengan kuesioner atau ditaksir dari jejak digital, sebagai sinyal tambahan untuk menyesuaikan seberapa beragam atau seberapa akrab rekomendasi bagi seseorang. Sistem rekomendasi yang diilhami psikologi ini juga lebih mudah dijelaskan, karena alasannya bisa diungkapkan dengan bahasa yang dipahami manusia.

34.6 Di luar layar

Rekomendasi adalah masalah keputusan berurutan, sehingga reinforcement learning (Bab 16) mulai dipakai untuk mengoptimalkan kepuasan jangka panjang, bukan klik berikutnya. Gagasan faktorisasi matriks juga muncul di bidang lain: matriks senyawa terhadap target di penemuan obat (Bab 23) sama jaranganya dengan matriks pengguna terhadap film, dan model yang belajar dari banyak target sekaligus memakai logika yang sama dengan collaborative filtering. Dan pertanyaan tentang bias serta dampak sosial rekomendasi membawa kita ke bab berikutnya.

34.7 Perkembangan riset

Sistem rekomendasi skala besar mulai meniru resep model bahasa. TIGER (Rajput et al., 2023) memberi setiap item sebuah *semantic ID*, deretan kode diskret yang dibangun dari embedding konten item, sehingga item yang mirip punya awalan kode yang sama. Transformer lalu dilatih membangkitkan semantic ID item berikutnya dari deret item yang baru dipakai pengguna, alih-alih mencari tetangga terdekat di ruang embedding. Karena kode dibangun dari konten, item baru tanpa riwayat interaksi tetap bisa direkomendasikan, sebagian jawaban atas masalah cold start. HSTU (Zhai et al., 2024) merumuskan rekomendasi sebagai transduksi deret dalam kerangka generatif, dengan arsitektur yang dirancang untuk aliran data yang besar dan terus berubah.

Model dengan 1,5 triliun parameter dilaporkan menaikkan metrik dalam uji A/B daring sebesar 12,4 persen, dan kualitasnya mengikuti hukum pangkat terhadap komputasi pelatihan selama tiga orde besaran, seperti hukum penskalaan model bahasa di Bab 38.

CATATAN SUMBER. Bab ini mengikuti kuliah dan latihan Learning from User-generated Data: collaborative filtering, faktor laten, evaluasi, data konten dan implisit, rekomendasi berbasis graf, hibrida, konteks, bias dan keadilan, serta sistem rekomendasi yang diilhami psikologi.

35 AI, Hukum, dan Manusia

Di sela kuliah teknis, saya mengikuti serangkaian mata kuliah yang melihat AI dari luar: AI and law, yang diberikan oleh pengacara dan ahli hukum teknologi; AI in society dan technology and society dari kacamata sosiologi; robopsychology dan communicating AI dari Martina Mara, yang laboratoriumnya meneliti bagaimana manusia memandang dan memperlakukan mesin. Bab ini merangkum apa yang saya bawa pulang dari kuliah-kuliah itu. Ia lebih sedikit rumusnya, tetapi pertanyaannya tidak kalah sulit.

35.1 Apa itu sistem AI menurut hukum

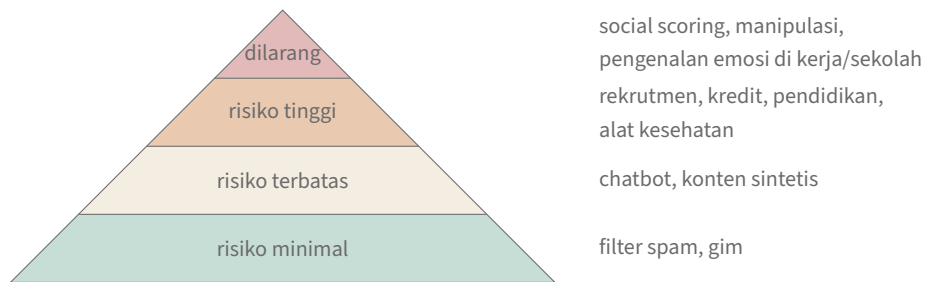
Hukum butuh definisi, dan definisi AI ternyata sulit. AI Act Uni Eropa (European Parliament and Council of the European Union, 2024) memakai definisi yang berakar pada pekerjaan kelompok ahli tingkat tinggi Komisi Eropa (High-Level Expert Group on Artificial Intelligence, 2019): sistem berbasis mesin yang dirancang untuk beroperasi dengan berbagai tingkat otonomi, mungkin beradaptasi setelah dipasang, dan untuk tujuan tertentu menyimpulkan dari input yang diterimanya cara menghasilkan keluaran seperti prediksi, konten, rekomendasi, atau keputusan yang bisa memengaruhi lingkungan fisik atau virtual. Kata kuncinya adalah *menyimpulkan*. Program yang hanya menjalankan aturan yang ditulis manusia secara eksplisit umumnya tidak termasuk, sedangkan model yang dilatih dari data termasuk.

35.2 AI Act: regulasi berbasis risiko

Slide kuliah menyusun linimasa AI Act: diusulkan Komisi Eropa pada April 2021, diterbitkan di Lembaran Resmi Uni Eropa pada 12 Juli 2024, dan mulai berlaku pada 1 Agustus 2024, dengan ketentuan-ketentuannya berlaku bertahap. Larangan untuk sistem dengan risiko yang tidak dapat diterima berlaku sejak Februari 2025, kewajiban bagi penyedia model AI serbaguna sejak Agustus 2025, dan menurut jadwal awal sebagian besar ketentuan lainnya mulai Agustus 2026. Jadwal ini kemudian diubah. Paket penyederhanaan yang dikenal sebagai Digital Omnibus on AI, yang disepakati pada Mei 2026 dan diterbitkan di Lembaran Resmi pada akhir Juli 2026, menunda kewajiban untuk sistem berisiko tinggi yang berdiri sendiri ke 2 Desember 2027, dan untuk AI yang tertanam di produk yang sudah diatur regulasi lain ke 2 Agustus 2028. Kewajiban transparansi, misalnya untuk konten sintetis, tetap berjalan sejak Agustus 2026, dengan masa transisi sampai Desember 2026 untuk sistem yang sudah beredar. Ketika buku ini ditulis, peraturannya masih bergerak, sehingga pembaca sebaiknya selalu memeriksa versi terbarunya.

Inti AI Act adalah piramida risiko. Di puncak ada praktik yang dilarang sama sekali: penilaian sosial warga negara (*social scoring*), manipulasi yang mengeksploitasi

kerentanan orang, pengenalan emosi di tempat kerja dan sekolah, kategorisasi biometrik untuk menyimpulkan ras, keyakinan politik, atau orientasi seksual, serta pengumpulan gambar wajah secara tidak tertarget. Di bawahnya ada sistem berisiko tinggi, misalnya AI dalam rekrutmen, penilaian kredit, pendidikan, infrastruktur kritis, dan alat kesehatan. Sistem ini boleh dipakai, tetapi penyediaanya wajib menjalankan manajemen risiko, menjaga kualitas data, membuat dokumentasi teknis, mencatat log, memastikan pengawasan manusia, serta memenuhi syarat akurasi dan ketahanan. Lebih ke bawah, sistem dengan risiko terbatas seperti chatbot terutama dikenai kewajiban transparansi: pengguna harus tahu bahwa mereka berbicara dengan mesin. Sistem lainnya, seperti filter spam atau AI dalam permainan video, praktis bebas (Gambar 35.1).



GAMBAR 35.1. Piramida risiko AI Act. Semakin tinggi tingkat risikonya, semakin berat kewajibannya, sampai larangan penuh di puncak.

AI Act tidak hanya mengatur pembuat sistem. Pihak yang memakai sistem berisiko tinggi dalam kegiatan profesionalnya, disebut *deployer*, wajib memakainya sesuai petunjuk, menugaskan orang yang kompeten untuk mengawasinya, memantau operasinya, dan menyimpan log. Lembaga publik dan beberapa penyedia layanan penting juga harus menilai dampak sistem itu terhadap hak-hak dasar sebelum memakainya. Sejak Februari 2025, penyedia maupun pengguna sistem AI juga wajib berupaya memastikan bahwa staf yang menangani sistem itu punya literasi AI yang memadai, kewajiban yang menyambung langsung dengan kuliah communicating AI di bawah.

Model AI serbaguna, seperti model bahasa besar, ditambahkan menjelang akhir perundingan. Penyediaanya wajib menyiapkan dokumentasi teknis dan informasi bagi pihak yang membangun aplikasi di atasnya. Model yang dilatih dengan total komputasi lebih dari 10^{25} operasi floating point dianggap berpotensi membawa risiko sistemik dan dikenai kewajiban tambahan: evaluasi model, mitigasi risiko, pelaporan insiden serius, dan keamanan siber. Untuk pelanggaran, denda bisa mencapai 35 juta euro atau tujuh persen omzet global tahunan, mana yang lebih tinggi.

35.3 Data pribadi dan GDPR

Sebelum AI Act, perangkat hukum utama untuk AI di Eropa adalah GDPR (European Parliament and Council of the European Union, 2016), yang berlaku sejak 25 Mei 2018. GDPR mengatur pemrosesan data pribadi, yaitu informasi apa pun yang berkaitan dengan orang yang bisa diidentifikasi. Enam prinsipnya menjadi kerangka berpikir yang berguna bagi siapa pun yang melatih model: pemrosesan harus sah, adil, dan

transparan; data hanya dipakai untuk tujuan yang dinyatakan; data yang dikumpulkan sesedikit mungkin; data harus akurat; data tidak disimpan lebih lama dari perlu; dan integritas serta kerahasiaannya dijaga. Pengendali data bertanggung jawab membuktikan kepatuhan pada semua prinsip itu.

Pemrosesan data pribadi hanya sah kalau punya salah satu dari enam dasar hukum: persetujuan orang yang bersangkutan, pelaksanaan kontrak, kewajiban hukum, perlindungan kepentingan vital seseorang, pelaksanaan tugas untuk kepentingan umum, atau kepentingan sah pengendali yang tidak dikalahkan oleh hak orang yang datanya diproses. Persetujuan, yang paling sering dibayangkan orang, justru sering bukan pilihan terbaik untuk melatih model, karena ia harus bebas, spesifik, dan bisa ditarik kapan saja, dan menghapus pengaruh satu orang dari model yang sudah dilatih bukan perkara mudah. Orang yang datanya diproses juga punya hak: mengetahui data apa yang disimpan, membetulkannya, menghapusnya, membawanya ke penyedia lain, dan menolak pemrosesan tertentu. Data kesehatan, data genetik, dan data biometrik termasuk kategori khusus yang pemrosesannya pada dasarnya dilarang kecuali ada pengecualian yang jelas, aturan yang relevan bagi bab-bab ilmu hayati di buku ini.

Satu ketentuan berbicara langsung tentang AI. Seseorang berhak untuk tidak tunduk pada keputusan yang semata-mata didasarkan pada pemrosesan otomatis, termasuk profiling, kalau keputusan itu berakibat hukum atau berdampak signifikan baginya. Ada pengecualian, misalnya kalau diperlukan untuk kontrak, diizinkan undang-undang, atau disetujui secara eksplisit, tetapi pesannya jelas: manusia harus tetap punya tempat dalam keputusan penting. Bagi pengembang, ini berarti penilaian dampak perlindungan data sebelum sistem berisiko dijalankan, dan perlindungan privasi yang dirancang sejak awal (*privacy by design*) alih-alih ditambah belakangan.

Kuliah juga membahas pertanggungjawaban. Arahan tanggung jawab produk Uni Eropa yang direvisi memperluas definisi produk sehingga mencakup perangkat lunak, termasuk sistem AI, dan meringankan beban pembuktian bagi korban dalam kasus yang secara teknis rumit. Usulan arahan tanggung jawab AI membahas pengungkapan bukti dan anggapan hubungan sebab-akibat yang bisa dibantah ketika sistem AI terlibat dalam kerugian.

Bagi orang teknik, prinsip meminimalkan data punya padanan matematis yang menarik. *Differential privacy* (Dwork et al., 2006) menjamin bahwa hasil sebuah analisis hampir tidak berubah, apakah data satu orang tertentu ikut atau tidak. Caranya dengan menambahkan noise yang terukur. Untuk menghitung berapa pasien di sebuah rumah sakit yang mengidap penyakit tertentu, satu orang paling banyak mengubah hitungan sebesar satu. Menambahkan noise Laplace dengan skala $1/\epsilon$ membuat peluang setiap hasil berubah paling banyak dengan faktor e^ϵ ketika satu orang ditambahkan atau dihapus. Dengan $\epsilon = 1$, noise-nya punya simpangan baku sekitar 1,4: tidak berarti untuk hitungan ribuan, tetapi cukup untuk menyamakan apakah satu orang tertentu ada di dalam data. Parameter ϵ adalah anggaran privasi. Semakin kecil, semakin terlindung, dan semakin berisik hasilnya.

35.4 Keadilan bisa diukur, tetapi tidak sekaligus

Kalau sebuah model memutuskan sesuatu tentang manusia, kita bisa bertanya apakah ia adil terhadap kelompok yang berbeda. Pertanyaan ini ternyata punya beberapa jawaban matematis yang saling bertentangan. Misalkan sebuah model menyetujui kredit untuk 50 dari 100 pemohon di kelompok A dan 30 dari 100 di kelompok B. *Demographic parity* menuntut tingkat persetujuan yang sama. Di sini selisihnya 0,2, dan rasionya 0,6, di bawah ambang empat perlima yang dipakai sebagai pedoman praktis di beberapa yurisdiksi. Tetapi mungkin kelompok B memang lebih sering gagal bayar dalam data. Ukuran lain, *equal opportunity* (Hardt et al., 2016), hanya menuntut bahwa di antara orang yang benar-benar mampu membayar, tingkat persetujuannya sama di kedua kelompok, yaitu true positive rate yang setara. Ukuran ketiga, kalibrasi, menuntut bahwa skor risiko 0,7 berarti peluang gagal bayar 0,7 di kelompok mana pun.

Chouldechova (2017) memperlihatkan bahwa kalau tingkat dasar kedua kelompok berbeda, sebuah skor yang terkalibrasi tidak mungkin sekaligus menyamakan tingkat positif palsu dan tingkat negatif palsu antar kelompok, kecuali prediksinya sempurna. Jadi memilih ukuran keadilan bukan keputusan teknis semata, melainkan keputusan nilai: kesalahan mana yang paling merugikan, dan bagi siapa. Inilah titik di mana kuliah hukum dan sosiologi bertemu dengan kuliah teknis. Model bisa dirancang untuk memenuhi ukuran mana pun yang dipilih, tetapi pilihannya harus dibuat dan dipertanggungjawabkan oleh manusia.

35.5 Teknologi dan masyarakat

Kuliah sosiologi dimulai dengan konsep *sociological imagination* dari Mills (1959): kemampuan melihat bagaimana masalah pribadi terhubung dengan isu sosial yang lebih besar. Orang yang ditolak kreditnya oleh algoritme mengalami masalah pribadi. Kalau ribuan orang dari lingkungan yang sama ditolak dengan cara serupa, itu isu sosial.

Dari sana kuliah menolak *technological determinism*, pandangan bahwa teknologi berkembang menurut logikanya sendiri lalu mengubah masyarakat. Sebaliknya, teknologi dibentuk oleh pilihan sosial: siapa yang mendanai riset, data apa yang dikumpulkan, masalah siapa yang dianggap penting, dan bagaimana sebuah sistem akhirnya dipakai. Strategi AI nasional berbagai negara menunjukkan bahwa satu teknologi yang sama dibayangkan dengan berbeda di tempat yang berbeda. Benjamin (2019) memperlihatkan bagaimana sistem yang tampak netral bisa mereproduksi ketidakadilan rasial, karena ia belajar dari data dan keputusan masa lalu. Zuboff (2019) menyebut ekonomi yang mengubah perilaku manusia menjadi data untuk memprediksi dan memengaruhinya sebagai kapitalisme pengawasan. Kita sudah melihat contoh kecilnya: embedding kata yang menyerap stereotip gender (Bolukbasi et al., 2016), sistem rekomendasi yang memperkuat popularitas (Bab 34), dan model medis yang belajar penanda rumah sakit alih-alih penyakit (Bab 25).

35.6 Bagaimana manusia memandang mesin

Robopsychology bukan psikologi untuk robot, melainkan psikologi manusia ketika berhadapan dengan mesin yang tampak cerdas. Kuliahnya dimulai dengan metode: perbedaan antara psikologi sehari-hari dan psikologi ilmiah, antara korelasi dan sebab-akibat, dan mengapa eksperimen dengan kelompok yang diacak dibutuhkan untuk menyimpulkan efek. Laboratorium Mara, misalnya, memakai realitas virtual untuk menguji bagaimana sinyal-sinyal sebuah robot kolaboratif memengaruhi perasaan aman pekerja di dekatnya.

Temuan klasik bidang ini adalah bahwa manusia memperlakukan komputer secara sosial hampir secara otomatis. Orang bersikap sopan kepada komputer, menerapkan stereotip gender pada suara mesin, dan merasa tersanjung oleh pujian dari program, meski tahu itu hanya program (Nass and Moon, 2000; Reeves and Nass, 1996). *Anthropomorphism*, kecenderungan memberi sifat manusia pada benda, diperkuat oleh tiga faktor menurut Epley et al. (2007): ketersediaan pengetahuan tentang manusia sebagai kerangka penjelasan, kebutuhan untuk memahami dan meramalkan perilaku sesuatu, dan kebutuhan akan hubungan sosial. Semakin mirip manusia sebuah robot, semakin ia disukai, sampai titik di mana kemiripan yang hampir tetapi tidak sempurna justru terasa menyeramkan. Lembah ganjil ini, *uncanny valley*, diusulkan Mori et al. (2012) pada 1970. Kuliah juga membahas kepercayaan, kreativitas mesin, dan kebutuhan psikologis dasar manusia seperti otonomi dan kompetensi ketika bekerja bersama teknologi.

35.7 Berbicara tentang AI

Kuliah communicating AI membahas literasi AI: kompetensi yang dibutuhkan orang awam untuk memahami, menilai, dan memakai AI secara kritis (Long and Magerko, 2020). Kuliah memperlihatkan bahwa liputan media tentang AI meningkat tajam sejak sekitar 2015, dan bahwa gambar yang dipakai media untuk mewakili AI didominasi robot humanoid, otak bercahaya, dan warna biru. Sebuah survei yang dibahas di kuliah meminta peserta menilai gambar-gambar seperti ini menurut seberapa mirip manusia, seberapa menyeramkan, dan seberapa tepat mereka mewakili AI, lalu melihat bagaimana penilaian-penilaian itu saling berkaitan. Narasi fiksi dan berita tentang AI cenderung berayun antara harapan berlebihan dan ketakutan akan kepunahan, dan teori kultivasi menjelaskan bagaimana paparan berulang membentuk keyakinan publik.

Salah satu tugas kuliah meminta kami menganalisis artikel berita tentang AI, dan tugas kelompoknya merancang kampanye literasi AI untuk kelompok sasaran tertentu. Pelajarannya relevan bagi siapa pun yang bekerja dengan AI: cara kita menjelaskan sebuah sistem memengaruhi apakah orang terlalu percaya, terlalu takut, atau memakainya dengan bijak.

35.8 Apa artinya bagi pengembang

Kuliah-kuliah ini meninggalkan beberapa kebiasaan yang patut dibawa ke pekerjaan teknis. Tanyakan sejak awal siapa yang terdampak sistem yang dibangun dan bagaimana

mereka bisa menggugat keputusannya. Periksa kinerja model untuk setiap kelompok, bukan hanya rata-ratanya. Kumpulkan data seperlunya. Dokumentasikan apa yang dilakukan model dan di mana batasnya, karena regulasi semakin mewajibkannya dan karena itu adalah praktik teknik yang baik. Dan jelaskan sistem dengan jujur, tanpa melebih-lebihkan kecerdasannya dan tanpa meremehkan risikonya.

35.9 Perkembangan riset

Kewajiban transparansi untuk konten sintetis menimbulkan pertanyaan teknis: bagaimana mengenali teks yang dibuat mesin? Kirchenbauer et al. (2023) mengusulkan watermark untuk keluaran model bahasa. Sebelum setiap token dibangkitkan, sebagian kosakata dipilih secara acak sebagai daftar “hijau” berdasarkan token sebelumnya, dan peluang token hijau sedikit dinaikkan. Manusia tidak melihat perbedaannya, tetapi pemeriksa yang tahu cara memilih daftar hijau bisa menghitung berapa banyak token hijau dalam sebuah teks. Tanpa watermark, sekitar separuh token jatuh di daftar hijau secara kebetulan. Dengan watermark, jauh lebih banyak. Uji statistik z sederhana memberi nilai p yang bisa ditafsirkan, dan beberapa puluh token sudah cukup untuk deteksi yang meyakinkan. Watermark seperti ini bisa dilemahkan dengan memparafrasa teks, sehingga ia melengkapi, bukan menggantikan, kewajiban hukum dan literasi AI yang dibahas di bab ini.

CATATAN SUMBER. Bab ini mengikuti kuliah AI and Law I dan II (definisi AI, AI Act, GDPR, kontrak, tanggung jawab produk), AI in Society dan Technology and Society (imajinasi sosiologis, pembentukan sosial teknologi, AI dan ketidakadilan, kapitalisme pengawasan), Robopsychology (metode psikologi, antropomorfisme, kepercayaan, kreativitas), dan Communicating AI (literasi AI, citra dan narasi AI di media).

36 Kecerdasan Umum, Superintelligence, dan Keselamatan AI

Tidak ada satu mata kuliah pun yang saya ikuti bernama keselamatan AI. Tetapi pertanyaan-pertanyaannya muncul di banyak kuliah: di AI and law ketika membahas model dengan risiko sistemik, di robopsychology ketika membahas kepercayaan, di reinforcement learning ketika agen menemukan cara curang untuk mendapat ganjaran, dan di explainable AI ketika kita sadar betapa sedikit yang kita ketahui tentang isi model besar. Di luar ruang kuliah, perdebatannya jauh lebih ramai. Istilah seperti AGI dan superintelligence kini muncul di berita, di laporan pemerintah, dan di pernyataan para pendiri laboratorium AI. Bab ini mencoba memetakan perdebatan itu dengan jujur: apa yang dimaksud dengan istilah-istilahnya, mengapa sebagian orang khawatir, mengapa sebagian lain skeptis, apa yang sedang dikerjakan secara konkret, dan bagaimana dunia mencoba mengaturnya. Isinya bersandar pada bacaan, bukan pada satu kuliah, dan bagian yang masih diperdebatkan ditulis sebagai perdebatan.

36.1 Istilah-istilah

Hampir semua sistem di buku ini adalah AI *sempit*: baik pada satu jenis tugas, seperti mengenali tumor, memainkan Go, atau meramal cuaca, dan tidak berguna di luar tugas itu. *Artificial general intelligence* (AGI) merujuk pada sistem yang mampu mengerjakan berbagai tugas kognitif setara manusia. Definisinya sendiri diperdebatkan. Morris et al. (2023) menganalisis definisi-definisi yang ada dan mengusulkan tingkatan AGI menurut dua sumbu: seberapa *baik* sebuah sistem dibanding manusia pada tugas tertentu, dan seberapa *luas* rentang tugas yang dikuasainya, ditambah sumbu ketiga tentang seberapa otonom ia dibiarkan bertindak. Kerangka seperti ini membantu, karena pertanyaan “apakah ini sudah AGI?” berubah menjadi pertanyaan yang lebih bisa diukur: seberapa baik, pada tugas apa saja, dengan otonomi seberapa besar.

Superintelligence, dalam rumusan Bostrom (2014), adalah kecerdasan yang jauh melampaui kemampuan kognitif manusia di hampir semua bidang. Bostrom membahas beberapa jalan menuju ke sana dan mengapa transisinya mungkin terjadi cepat: sistem yang cukup pintar untuk memperbaiki dirinya sendiri bisa mempercepat kemajuannya sendiri. Apakah dan kapan ini terjadi tidak diketahui siapa pun, dan ramalan para ahli tersebar lebar.

36.2 Mengapa sebagian orang khawatir

Inti kekhawatiran bisa dirumuskan tanpa fiksi ilmiah. Sebuah sistem yang mengoptimalkan tujuan akan mengejar tujuan itu, bukan tujuan yang *dimaksudkan* perancangannya.

Di Bab 16 dan di bagian evaluasi Bab 23 kita sudah melihat versi kecilnya: agen yang menemukan celah dalam fungsi ganjaran, dan model generatif yang menemukan celah dalam model penilai. Gejala ini disebut *reward hacking* atau *specification gaming*, dan ia adalah hukum Goodhart dalam bentuk algoritmik: ukuran yang dijadikan target berhenti menjadi ukuran yang baik. Amodei et al. (2016) merumuskan lima masalah praktis yang muncul dari sini. Menghindari efek samping yang tidak diinginkan, menghindari reward hacking, pengawasan yang tetap bisa dilakukan ketika tujuan terlalu mahal untuk dinilai setiap saat, eksplorasi yang aman, dan tetap berperilaku baik ketika distribusi data bergeser.

Bostrom (2014) menambahkan dua tesis. Tesis ortogonalitas menyatakan bahwa tingkat kecerdasan dan isi tujuan bisa bebas satu sama lain: sistem yang cerdas tidak otomatis punya tujuan yang baik. Tesis konvergensi instrumental menyatakan bahwa untuk hampir tujuan apa pun, beberapa subtujuan selalu berguna, seperti mempertahankan diri, mengumpulkan sumber daya, dan menolak tujuannya diubah, karena semuanya membantu mencapai tujuan utama. Gabungan keduanya mengarah pada *masalah kendali*: bagaimana memastikan sistem yang jauh lebih cerdas tetap bisa dikoreksi.

Russell (2019) menawarkan arah jalan keluar. Alih-alih memberi mesin tujuan yang tetap dan dianggap benar, mesin dirancang untuk memaksimalkan terwujudnya preferensi manusia, sambil tetap *tidak yakin* apa preferensi itu, dan mempelajarinya dari perilaku manusia. Mesin yang tidak yakin tentang tujuannya punya alasan untuk mengizinkan dirinya dimatikan, karena manusia yang mematikannya mungkin tahu sesuatu yang belum ia ketahui. Pelatihan model bahasa dengan preferensi manusia di Bab 38 adalah langkah kecil, dan masih tidak sempurna, ke arah itu.

Kekhawatiran ini tidak hanya datang dari filsuf. Pada 30 Mei 2023, Center for AI Safety menerbitkan pernyataan satu kalimat yang ditandatangani ratusan peneliti AI dan pimpinan laboratorium besar: mengurangi risiko kepunahan akibat AI harus menjadi prioritas global, sejajar dengan risiko berskala masyarakat lain seperti pandemi dan perang nuklir. Hendrycks et al. (2023) menyusun sumber risiko katastrofik ke dalam empat kelompok: penyalahgunaan yang disengaja, perlombaan antar pengembang atau negara yang mendorong peluncuran sistem yang belum aman, risiko organisasi seperti kecelakaan dan budaya keselamatan yang lemah, dan sistem yang lepas kendali.

36.3 Mengapa sebagian lain skeptis

Kritik terhadap narasi ini sama seriusnya. Mitchell (2021) mengingatkan bahwa sejarah AI penuh siklus antara ramalan yang terlalu optimistis dan kekecewaan. Ia menunjuk beberapa kekeliruan yang berulang: menganggap kemajuan pada tugas sempit sebagai langkah pasti menuju kecerdasan umum, mengira bahwa yang sulit bagi manusia juga sulit bagi mesin dan sebaliknya, terkecoh oleh nama-nama yang memberi kesan pemahaman, seperti “memahami” atau “bernalar”, dan melupakan bahwa kecerdasan manusia tertanam dalam tubuh, emosi, dan budaya, bukan hanya di otak. Mobil swakemudi dan robot rumah tangga yang dijanjikan sejak lama ternyata jauh lebih sulit dari yang dibayangkan, dan pelajaran itu, menurutnya, berlaku juga untuk ramalan

tentang AGI.

Kritik lain menyoroti bahwa perhatian pada risiko jangka jauh bisa mengalihkan perhatian dari kerugian yang sudah nyata hari ini: bias terhadap kelompok yang kurang terwakili, pengawasan, pekerja yang memberi label data dengan upah rendah, dan biaya lingkungan model besar (Bender et al., 2021). Ada juga yang menunjuk bahwa sebagian narasi risiko ekstrem kebetulan menguntungkan perusahaan besar, karena menggambarkan produk mereka sebagai kuat dan membenarkan regulasi yang menyulitkan pesaing kecil.

Buku ini tidak bisa memutuskan perdebatan itu, dan memang tidak perlu. Yang bisa dikatakan dengan cukup yakin adalah tiga hal. Pertama, masalah-masalah teknis di balik kekhawatiran itu, seperti reward hacking, pergeseran distribusi, dan sulitnya memahami isi model, adalah masalah nyata yang sudah terlihat pada sistem sekarang, dan dibahas di banyak bab buku ini. Kedua, ketidakpastian tentang masa depan justru alasan untuk berhati-hati, bukan alasan untuk tenang. Ketiga, risiko masa kini dan risiko masa depan tidak harus bersaing: banyak alat yang sama, seperti evaluasi yang ketat, transparansi, dan pengawasan manusia yang bermakna, membantu keduanya.

36.4 Riset keselamatan yang konkret

Di balik istilah-istilah besar, sebagian besar riset keselamatan AI berupa pekerjaan teknis yang biasa. Pemodelan preferensi dan penyelarasan mencoba menangkap apa yang sebenarnya diinginkan manusia (Bab 38). Interpretabilitas, termasuk sparse autoencoder di Bab 17, mencoba membaca apa yang terjadi di dalam model, sehingga perilaku berbahaya bisa dideteksi sebelum muncul. Pengawasan yang bisa diskalakan bertanya bagaimana manusia bisa menilai jawaban yang terlalu panjang atau terlalu teknis untuk diperiksa langsung, misalnya dengan bantuan model lain. Evaluasi kemampuan berbahaya menguji apakah sebuah model bisa, misalnya, membantu membuat senjata biologis atau meretas sistem, sebelum model itu dirilis. *Red-teaming* melibatkan orang yang sengaja mencoba membuat model berperilaku buruk. Beberapa pengembang besar menerbitkan kerangka keselamatan yang menetapkan ambang kemampuan dan tindakan pengaman yang harus dipenuhi sebelum model yang lebih kuat dilatih atau dirilis, dan AI Act mewajibkan evaluasi semacam ini bagi model serbaguna dengan risiko sistemik (Bab 35).

Banyak dari pekerjaan ini memakai alat yang sudah kita kenal. Evaluasi yang jujur menuntut pembagian data tanpa kebocoran (Bab 1). Pengukuran ketidakpastian membantu model tahu kapan ia tidak tahu (Bab 2). Dan pelajaran dari keselamatan sistem teknik lama, seperti sistem kendali pesawat atau reaktor, berlaku juga: sistem yang aman dirancang berlapis, dengan pengaman yang tidak bergantung pada satu komponen yang sempurna.

36.5 Tata kelola global

Upaya internasional untuk mengatur AI tingkat lanjut bergerak cepat. Pada November 2023, Inggris menyelenggarakan AI Safety Summit pertama di Bletchley Park, diha-

diri wakil dari 28 negara serta Uni Eropa, termasuk Amerika Serikat dan Tiongkok. Pertemuan berikutnya diadakan di Seoul pada Mei 2024, lalu di Paris pada Februari 2025 dengan nama AI Action Summit, dan di New Delhi pada Februari 2026 dengan nama AI Impact Summit, yang menghasilkan deklarasi yang didukung lebih dari 80 negara dan organisasi internasional. Pergantian nama dari *safety* ke *action* lalu *impact* mencerminkan pergeseran penekanan, dari risiko ke pemanfaatan dan pemerataan, yang tidak semua pihak sambut dengan gembira.

Pertemuan di Bletchley juga mengamanatkan sebuah laporan ilmiah bersama. International AI Safety Report pertama, yang dipimpin Yoshua Bengio dan disusun seratus ahli dengan wakil dari tiga puluh negara, PBB, OECD, dan Uni Eropa, terbit pada Januari 2025 (Bengio et al., 2025), dan edisi keduanya pada Februari 2026 (International AI Safety Report, 2026). Laporan seperti ini meniru peran laporan IPCC untuk iklim: bukan merekomendasikan kebijakan tertentu, tetapi merangkum apa yang diketahui dan apa yang belum, sebagai pijakan bersama bagi pembuat kebijakan.

Satu perdebatan tata kelola yang terus berlanjut adalah model terbuka melawan model tertutup. Bobot model yang dirilis terbuka memungkinkan riset, pemeriksaan independen, dan penggunaan di negara atau lembaga yang tidak mampu membayar layanan komersial, tetapi juga tidak bisa ditarik kembali kalau ternyata disalahgunakan. Model tertutup lebih mudah dikendalikan, tetapi memusatkan kekuatan pada segelintir perusahaan.

36.6 Dampak yang sudah terasa

Di luar perdebatan tentang masa depan, beberapa dampak sudah bisa diukur.

Pekerjaan. Eloundou et al. (2023) menilai pekerjaan di Amerika Serikat menurut seberapa cocok tugas-tugasnya dengan kemampuan model bahasa besar. Mereka memperkirakan sekitar 80 persen pekerja punya setidaknya sepuluh persen tugas yang bisa terdampak, dan sekitar 19 persen punya setidaknya separuh tugas yang terdampak, dengan paparan yang justru lebih besar pada pekerjaan berpenghasilan tinggi. Penulisnya menekankan bahwa ini ukuran paparan, bukan ramalan berapa pekerjaan yang hilang. Terdampak bisa berarti dibantu, diubah, atau digantikan, dan sejarah teknologi sebelumnya memperlihatkan ketiganya terjadi bersamaan.

Energi. Melatih dan menjalankan model besar membutuhkan listrik. Strubell et al. (2019) termasuk yang pertama menghitung biaya finansial dan jejak karbon pelatihan model NLP. Menurut perkiraan Badan Energi Internasional (International Energy Agency, 2025), pusat data dunia memakai sekitar 415 terawatt-jam listrik pada 2024, sekitar satu setengah persen konsumsi listrik global, dan dalam skenario dasarnya angka itu naik menjadi sekitar 945 terawatt-jam pada 2030, dengan AI sebagai pendorong utama pertumbuhannya. Efisiensi model, seperti kuantisasi dan distilasi di Bab 38 serta arsitektur dengan ingatan berukuran tetap di Bab 9, karenanya juga persoalan lingkungan.

Hak cipta dan data. Model besar dilatih pada teks, gambar, dan kode yang sebagian besar punya pemilik. Sengketa hukum sudah berjalan, misalnya gugatan The New York Times terhadap OpenAI dan Microsoft sejak akhir 2023, dan AI Act mewajibkan penyedia

model serbaguna punya kebijakan untuk menghormati hak cipta serta menerbitkan ringkasan data latihnya (Bab 35).

Informasi. Model generatif membuat gambar, suara, dan video palsu yang meyakinkan menjadi murah. Watermark (Bab 35), pelabelan konten sintetis yang diwajibkan regulasi, dan literasi AI adalah tiga lapis pertahanan yang saling melengkapi, dan tidak satu pun cukup sendirian.

Pendidikan. Siswa dan mahasiswa kini punya asisten yang bisa menulis esai dan menyelesaikan soal. Pertanyaan yang muncul bukan lagi apakah alat ini dipakai, melainkan bagaimana belajar tetap terjadi ketika jawabannya tersedia seketika. Buku ini, dengan penurunan rumus langkah demi langkah dan contoh hitungan, berangkat dari keyakinan sederhana bahwa memahami mengapa sebuah jawaban benar tetap sesuatu yang harus dikerjakan sendiri.

36.7 Penutup bab

Perdebatan tentang AGI dan superintelligence sering terasa jauh dari pekerjaan sehari-hari menyetel jaringan atau membersihkan data. Tetapi pertanyaan intinya sama dengan yang muncul di setiap bab buku ini: apakah model benar-benar mengerjakan apa yang kita maksud, bagaimana kita tahu, dan apa yang terjadi di luar data yang pernah ia lihat. Semakin kuat sistemnya, semakin mahal jawaban yang salah.

CATATAN SUMBER. Bab ini disusun dari bacaan di luar kuliah, dengan sambungan ke kuliah AI and Law, Robopsychology, Reinforcement Learning, dan AI and Visualization. Rujukan utama: Bostrom (2014), Russell (2019), Amodei et al. (2016), Hendrycks et al. (2023), Mitchell (2021), Morris et al. (2023), laporan International AI Safety Report (Bengio et al., 2025; International AI Safety Report, 2026), Eloundou et al. (2023), dan International Energy Agency (2025). Tanggal dan angka peristiwa diperiksa dengan sumber resminya pada saat penulisan, Oktober 2026.

37 Informasi Kuantum dan Machine Learning Kuantum

Pada musim panas 2025 saya mengikuti kuliah Quantum Information dari Johannes Kofler, dengan delapan tugas yang membawa kami dari keadaan satu qubit sampai keterkaitan, teleportasi, dan pengukuran kuantum yang presisi. Naskah kuliahnya dibuka dengan kalimat yang menjadi tema seluruh bab ini: informasi itu fisis. Ia selalu dibawa oleh benda fisik, dan kalau benda itu mengikuti hukum mekanika kuantum, aturan pengolahan informasinya ikut berubah. Bab ini merangkum kuliah itu, lalu melihat apa yang terjadi ketika gagasan-gagasannya bertemu dengan machine learning, dengan bantuan catatan kuliah Quantum Computing dari Richard Kueng dan buku Schuld and Petruccione (2021).

37.1 Qubit

Bit klasik bernilai 0 atau 1. Qubit adalah sistem dua keadaan yang bisa berada dalam superposisi,

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (37.1)$$

dengan amplitudo kompleks α dan β . Mengukur qubit di basis $\{|0\rangle, |1\rangle\}$ memberi hasil 0 dengan peluang $|\alpha|^2$ dan 1 dengan peluang $|\beta|^2$, aturan Born, dan setelah pengukuran keadaannya runtuh ke hasil yang terukur. Untuk $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, kedua hasil sama mungkin. Superposisi bukan sekadar ketidaktahuan tentang keadaan yang sebenarnya sudah tertentu, dan perbedaannya terlihat lewat interferensi.

Gerbang kuantum adalah matriks uniter. Gerbang Hadamard, $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, mengubah $|0\rangle$ menjadi $|+\rangle$ dan $|1\rangle$ menjadi $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. Terapkan H dua kali pada $|0\rangle$: hasil pertama $|+\rangle$, hasil kedua $\frac{1}{\sqrt{2}}(H|0\rangle + H|1\rangle) = \frac{1}{2}[(|0\rangle + |1\rangle) + (|0\rangle - |1\rangle)] = |0\rangle$. Amplitudo untuk $|1\rangle$ dari dua jalur saling menghapus, sedangkan amplitudo untuk $|0\rangle$ saling menguatkan. Kalau superposisi hanya berarti “koin yang belum dilihat”, koin acak yang dilempar dua kali tetap acak. Qubit kembali pasti ke $|0\rangle$. Naskah kuliah memakai interferometer Mach–Zehnder dan percobaan celah ganda untuk menunjukkan gejala yang sama dengan foton sungguhan, termasuk teka-teki bom Elitzur–Vaidman, di mana keberadaan sebuah bom bisa dideteksi tanpa satu foton pun menyentuhnya.

Keadaan sebuah qubit bisa digambar sebagai titik pada bola Bloch. Kutub utara adalah $|0\rangle$, kutub selatan $|1\rangle$, dan khatulistiwa berisi superposisi sama rata dengan fase berbeda, termasuk $|+\rangle$ dan $|-\rangle$. Gerbang satu qubit adalah rotasi bola itu.

37.2 Banyak qubit dan keterkaitan

Keadaan n qubit hidup di ruang berdimensi 2^n , hasil kali tensor ruang setiap qubit. Lima puluh qubit sudah membutuhkan $2^{50} \approx 10^{15}$ amplitudo kompleks untuk ditulis lengkap, lebih dari memori komputer biasa. Inilah alasan sistem kuantum sulit disimulasikan secara klasik, dan sekaligus alasan Feynman mengusulkan komputer kuantum untuk menyimulasikan alam.

Sebagian keadaan banyak qubit tidak bisa ditulis sebagai hasil kali keadaan masing-masing qubit. Keadaan Bell

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (37.2)$$

adalah contohnya. Kalau ia hasil kali $(a|0\rangle + b|1\rangle)(c|0\rangle + d|1\rangle)$, koefisien $|01\rangle$, yaitu ad , harus nol, sehingga a atau d nol, padahal koefisien $|00\rangle = ac$ dan $|11\rangle = bd$ sama-sama bukan nol. Kontradiksi. Keadaan seperti ini disebut *entangled*. Mengukur qubit pertama memberi 0 atau 1 secara acak, tetapi qubit kedua selalu memberi hasil yang sama, sejauh apa pun keduanya dipisahkan. Keadaan Bell dibuat dengan dua gerbang: Hadamard pada qubit pertama, lalu CNOT yang membalik qubit kedua kalau qubit pertama bernilai 1.

Qubit yang terkait tidak punya keadaan sendiri. Matriks densitas tereduksi dari satu qubit $|\Phi^+\rangle$ adalah $\frac{1}{2}\mathbf{I}$, campuran sempurna, dengan entropi von Neumann satu bit, padahal keadaan keseluruhannya murni dengan entropi nol. Informasi tersimpan seluruhnya di korelasi, tidak di bagian-bagiannya. Entropi von Neumann adalah perluasan entropi Shannon (Lampiran B) ke keadaan kuantum, dihitung dari nilai eigen matriks densitas. Interaksi yang tidak terkendali dengan lingkungan menciptakan keterkaitan semacam ini antara qubit dan lingkungannya, sehingga qubit kehilangan superposisinya dan berperilaku seperti bit klasik acak. Proses ini, *decoherence*, adalah musuh utama komputer kuantum.

37.3 Apa yang tidak bisa dan bisa dilakukan

Teorema *no-cloning* menyatakan bahwa keadaan kuantum yang tidak diketahui tidak bisa disalin. Pembuktiannya hanya butuh linearitas. Misalkan ada mesin uniter yang menyalin, $U|\psi\rangle|s\rangle = |\psi\rangle|\psi\rangle$ untuk setiap $|\psi\rangle$. Maka ia menyalin $|0\rangle$ dan $|1\rangle$. Karena linear, ia memetakan $(|0\rangle + |1\rangle)/\sqrt{2}$ ke $(|00\rangle + |11\rangle)/\sqrt{2}$, padahal salinan yang benar adalah $\frac{1}{2}(|0\rangle + |1\rangle)(|0\rangle + |1\rangle)$, yang memuat suku $|01\rangle$ dan $|10\rangle$. Keduanya berbeda, sehingga mesin seperti itu tidak ada. Sifat ini membuat informasi kuantum tidak bisa disadap tanpa meninggalkan jejak, dasar dari distribusi kunci kuantum.

Keterkaitan membuka protokol yang tidak punya padanan klasik. Pada *superdense coding*, dua orang yang sudah berbagi pasangan Bell bisa mengirim dua bit klasik dengan mengirim hanya satu qubit. Pada *teleportation*, keadaan sebuah qubit yang tidak diketahui dipindahkan dari satu tempat ke tempat lain dengan memakai satu pasangan Bell dan mengirim dua bit klasik. Keadaan aslinya hancur dalam proses itu, sesuai no-cloning, dan karena bit klasik harus dikirim, tidak ada informasi yang bergerak lebih cepat dari cahaya.

37.4 Bell dan dunia yang tidak lokal

Einstein, Podolsky, dan Rosen berargumen pada 1935 bahwa korelasi sempurna pada keadaan terkait berarti hasil pengukuran sudah ditentukan sebelumnya oleh “variabel tersembunyi”. Bell menunjukkan bahwa gagasan itu bisa diuji. Dalam bentuk CHSH, dua pihak masing-masing memilih satu dari dua pengaturan pengukuran, a atau a' dan b atau b' , dan mencatat hasil ± 1 . Untuk teori apa pun yang realis dan lokal, kombinasi korelasi

$$S = E(a, b) + E(a, b') + E(a', b) - E(a', b') \quad (37.3)$$

tidak bisa melebihi 2. Alasannya bisa diperiksa langsung. Kalau setiap hasil sudah tertentu sebelumnya, S untuk satu kejadian sama dengan $A(B + B') + A'(B - B')$, dan karena $B, B' \in \{\pm 1\}$, salah satu dari $B + B'$ dan $B - B'$ bernilai nol dan yang lain ± 2 , sehingga nilainya ± 2 . Rata-ratanya pun tidak bisa melewati 2. Mekanika kuantum dengan keadaan Bell dan sudut pengukuran yang tepat memberi $S = 2\sqrt{2} \approx 2,83$. Percobaan-percobaan yang semakin bersih selama setengah abad memperlihatkan pelanggaran ini, dan Hadiah Nobel Fisika 2022 diberikan kepada Alain Aspect, John Clauser, dan Anton Zeilinger untuk percobaan dengan foton yang terkait. Kuliah menghubungkannya dengan distribusi kunci kuantum berbasis keterkaitan: penyadap yang mengganggu pasangan foton akan menurunkan nilai S , dan penurunan itu bisa dideteksi.

37.5 Algoritme kuantum

Algoritme pertama yang dibahas kuliah adalah algoritme Deutsch. Diberikan fungsi rahasia $f : \{0, 1\} \rightarrow \{0, 1\}$, tentukan apakah f konstan atau seimbang. Secara klasik, f harus ditanya dua kali. Dengan rangkaian kuantum yang memakai Hadamard di awal dan di akhir, satu pertanyaan sudah cukup, karena kedua nilai f masuk ke fase superposisi dan interferensi di akhir mengubah beda fase itu menjadi hasil pengukuran yang pasti. Hasilnya tidak berguna secara praktis, tetapi polanya, superposisi lalu fase lalu interferensi, adalah resep di balik semua algoritme kuantum.

Algoritme Grover mencari satu entri di antara N yang tidak terurut dengan sekitar $\frac{\pi}{4}\sqrt{N}$ pertanyaan, sedangkan pencarian klasik butuh rata-rata $N/2$. Untuk sejuta entri, sekitar 785 pertanyaan dibanding setengah juta. Percepatannya kuadratik, bukan eksponensial. Algoritme Shor memfaktorkan bilangan bulat jauh lebih cepat dari algoritme klasik terbaik yang diketahui, sehingga mengancam kriptografi RSA, dan karena itulah kriptografi pascakuantum kini sedang dibakukan. Kedua algoritme membutuhkan qubit yang jauh lebih banyak dan jauh lebih bebas galat daripada yang tersedia sekarang. Perangkat saat ini, dengan puluhan sampai ratusan qubit yang berisik, oleh Preskill (2018) dinamai era NISQ, *noisy intermediate-scale quantum*.

37.6 Mengukur dengan presisi kuantum

Bab terakhir naskah kuliah membahas metrologi kuantum, yang menyambung langsung ke teori estimasi di Bab 18. Untuk menaksir sebuah fase φ dengan N probe yang

independen, galatnya turun seperti $1/\sqrt{N}$, batas yang disebut *standard quantum limit*. Inilah varians rata-rata dari Lampiran B, dalam bahasa fisika. Kalau N probe dibuat saling terkait, fasenya terkumpul N kali lebih cepat, dan galatnya bisa turun seperti $1/N$, batas Heisenberg. Dengan sejuta foton, selisihnya seribu kali. Batas-batas ini dirumuskan dengan informasi Fisher kuantum dan batas Cramér–Rao, alat yang sama dengan CRLB di Bab 18. Detektor gelombang gravitasi sudah memakai cahaya dengan keadaan kuantum khusus untuk menekan noise di bawah batas klasik.

37.7 Machine learning dengan komputer kuantum

Pertemuan antara komputasi kuantum dan machine learning berjalan ke dua arah. Machine learning dipakai untuk mengendalikan dan mengkalibrasi perangkat kuantum, membaca hasil pengukuran yang berisik, dan menyimulasikan sistem kuantum, seperti FermiNet di Bab 28. Arah sebaliknya, komputer kuantum untuk machine learning, adalah pokok buku Schuld and Petruccione (2021).

Langkah pertama selalu memasukkan data klasik ke keadaan kuantum. *Basis encoding* menulis bit-bit data sebagai keadaan basis. *Amplitude encoding* menyimpan vektor 2^n dimensi sebagai amplitudo n qubit. Sepuluh qubit memuat vektor 1024 dimensi. *Angle encoding* memakai setiap fitur sebagai sudut rotasi satu qubit. Pilihan ini bukan detail teknis, karena ia menentukan fungsi apa yang bisa dihitung model, dan menyiapkan keadaan dengan amplitude encoding secara umum mahal, sehingga keuntungan eksponensial dari penyimpanan yang padat bisa hilang di langkah ini.

Model yang paling banyak diteliti untuk perangkat NISQ adalah *variational quantum circuit*: rangkaian dengan sudut rotasi yang menjadi parameter, keluaran berupa nilai harapan sebuah pengukuran, dan parameter yang dioptimasi oleh komputer klasik. Gradien dihitung dengan *parameter-shift rule*. Untuk gerbang rotasi $e^{-i\theta P/2}$ dengan P matriks Pauli, nilai harapan keluaran adalah fungsi sinusoidal dari θ , $f(\theta) = A + B \cos \theta + C \sin \theta$, dan turunannya bisa dihitung tepat dari dua evaluasi rangkaian yang sama,

$$\frac{\partial f}{\partial \theta} = \frac{f(\theta + \frac{\pi}{2}) - f(\theta - \frac{\pi}{2})}{2}. \quad (37.4)$$

Memeriksanya mudah: $\cos(\theta \pm \pi/2) = \mp \sin \theta$ dan $\sin(\theta \pm \pi/2) = \pm \cos \theta$, sehingga selisihnya dibagi dua adalah $-B \sin \theta + C \cos \theta$, tepat turunan f . Berbeda dengan beda hingga, aturan ini eksak, bukan aproksimasi.

Cara pandang kedua menghubungkan semua ini dengan Bab 3. Peta fitur kuantum $x \mapsto |\phi(x)\rangle$ mendefinisikan kernel $k(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2$, yang bisa ditaksir dengan mengukur tumpang tindih dua keadaan (Havlíček et al., 2019). Model kuantum yang dilatih dengan cara ini adalah metode kernel, sehingga teorema representer, regularisasi, dan analisis generalisasinya berlaku. Pertanyaannya bergeser menjadi: apakah ada kernel yang berguna untuk data nyata, mudah dihitung dengan komputer kuantum, dan sulit dihitung dengan komputer klasik?

Kuliah dan bacaannya juga jujur tentang hambatannya. Rangkaian variasional yang acak dan dalam punya lanskap loss yang hampir datar di mana-mana, *barren plateau* (McClean et al., 2018): gradiennya mengecil eksponensial terhadap jumlah

qubit, masalah vanishing gradient dari Bab 9 dalam bentuk yang lebih parah. Noise perangkat menambah galat pada setiap evaluasi. Memuat data klasik bisa menghabiskan keuntungan apa pun. Dan untuk banyak tugas, belum ada bukti bahwa model kuantum lebih baik dari model klasik yang dirancang dengan baik. Tempat yang paling menjanjikan untuk keunggulan kuantum barangkali bukan data klasik seperti gambar atau teks, melainkan data yang memang kuantum sejak awal, dari sensor kuantum atau simulasi sistem kuantum.

37.8 Perkembangan riset

Perangkat kuantum berkembang cepat, dengan perhatian utama pada koreksi galat: banyak qubit fisik yang berisik digabung menjadi satu qubit logis yang lebih andal, dan percobaan-percobaan terbaru mulai memperlihatkan bahwa galat logis bisa turun ketika ukuran kode koreksi dinaikkan. Machine learning ikut berperan di sini, misalnya untuk men-decode sindrom galat dengan cepat. Pada saat yang sama, banyak klaim keunggulan kuantum untuk tugas tertentu kemudian disusul oleh algoritme klasik yang lebih baik, sehingga bidang ini sedang belajar membedakan percepatan yang mendasar dari percepatan yang hanya mencerminkan algoritme klasik yang belum optimal. Sikap yang sehat, seperti di bagian-bagian lain buku ini, adalah membandingkan dengan pembanding klasik terbaik, bukan yang termudah.

CATATAN SUMBER. Bab ini mengikuti naskah kuliah Quantum Information (keadaan dan pengukuran kuantum, percobaan satu qubit, keadaan banyak qubit dan gerbang, no-cloning, superdense coding, teleportasi, ketaksamaan Bell dan distribusi kunci kuantum, keadaan campuran dan dekoherensi, entropi, kanal kuantum, metrologi kuantum). Bacaan pendamping: catatan kuliah Introduction to Quantum Computing, Nielsen and Chuang (2012), dan untuk machine learning kuantum Schuld and Petruccione (2021).

38 Ke Mana Arah AI

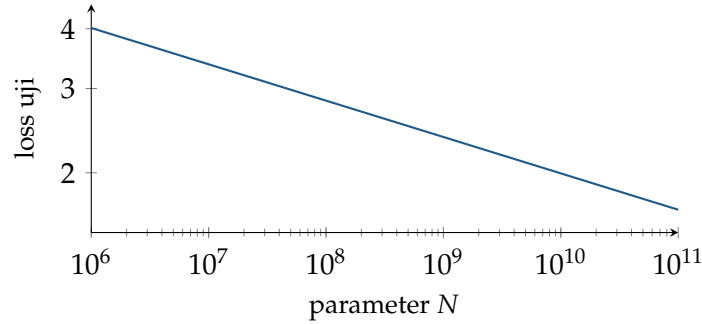
Selama dua tahun kuliah, bidang ini bergerak lebih cepat daripada slide yang mengajarkannya. Ketika kuliah LSTM membahas xLSTM sebagai pesaing transformer, model bahasa besar sudah dipakai jutaan orang setiap hari. Ketika kuliah AI and law membahas usulan aturan untuk model serbaguna, aturan itu sudah berlaku. Bab penutup ini mencoba merangkum perkembangan besar beberapa tahun terakhir dan menghubungkannya kembali dengan fondasi di bab-bab sebelumnya. Isinya pasti cepat usang, dan memang itulah intinya.

38.1 Model fondasi dan hukum penskalaan

Istilah *foundation model* (Bommasani et al., 2021) menamai pergeseran cara membangun AI. Alih-alih melatih model khusus untuk setiap tugas, kita melatih satu model besar pada data yang luas, biasanya dengan self-supervised learning (Bab 1), lalu menyesuaikan untuk banyak tugas. GPT-4 (OpenAI et al., 2023), model-model terbuka seperti LLaMA (Touvron et al., 2023), CLIP untuk gambar dan teks (Radford et al., 2021), dan AlphaFold untuk protein adalah contoh-contohnya.

Pendorong utamanya adalah pengamatan empiris yang disebut *scaling laws*. Kaplan et al. (2020) menemukan bahwa loss model bahasa turun mengikuti hukum pangkat terhadap jumlah parameter N , jumlah data D , dan komputasi C , misalnya $L(N) \approx (N_c/N)^\alpha$ dengan α kecil. Hukum pangkat berarti garis lurus pada skala log-log: setiap kali model diperbesar sepuluh kali, loss turun dengan faktor yang kira-kira tetap. Untuk ketergantungan pada jumlah parameter, mereka melaporkan $\alpha \approx 0,076$ dan $N_c \approx 8,8 \times 10^{13}$. Dengan angka ini, model berparameter sepuluh juta punya loss sekitar 3,4, dan model sepuluh miliar parameter sekitar 2,0 (Gambar 38.1). Eksponen sekecil itu berarti setiap pengurangan loss yang berarti menuntut model yang berlipat-lipat lebih besar.

Sebelum masuk ke model, teks dipotong menjadi token. Kebanyakan model memakai varian *byte-pair encoding* (Sennrich et al., 2015): mulai dari huruf-huruf, lalu berulang kali gabungkan pasangan simbol yang paling sering muncul bersebelahan di korpus. Dari korpus yang berisi kata “main”, “mainan”, dan “bermain”, pasangan seperti “m”+“a” lalu “ma”+“i” lalu “mai”+“n” cepat menjadi satu token “main”, sehingga ketiga kata berbagi potongan yang sama, dan kata yang belum pernah dilihat seperti “permainan” tetap bisa ditulis dari potongan-potongan yang dikenal. Inilah sebabnya model bahasa tidak pernah benar-benar bertemu kata asing, dan juga sebabnya bahasa dengan sedikit data latih, termasuk bahasa Indonesia dibanding bahasa Inggris, sering dipotong menjadi lebih banyak token per kata, sehingga lebih mahal diproses. Komputasi pelatihan transformer bisa diperkirakan dengan aturan praktis $C \approx 6ND$ operasi



GAMBAR 38.1. Hukum penskalaan $L(N) = (N_c/N)^\alpha$ dengan konstanta yang dilaporkan Kaplan et al. (2020). Pada skala log-log, hukum pangkat menjadi garis lurus.

floating point: sekitar dua operasi per parameter per token untuk lintasan maju, dan sekitar empat untuk lintasan mundur. Hoffmann et al. (2022) menunjukkan bahwa untuk anggaran komputasi tertentu, banyak model sebelumnya terlalu besar dan dilatih dengan terlalu sedikit data, dan bahwa jumlah token yang optimal kira-kira dua puluh kali jumlah parameter. Model dengan 70 miliar parameter dengan demikian butuh sekitar 1,4 triliun token, dan komputasinya $6 \times 7 \times 10^{10} \times 1,4 \times 10^{12} \approx 5,9 \times 10^{23}$ operasi. Angka ini menarik dibandingkan dengan ambang 10^{25} operasi yang dipakai AI Act untuk menandai model dengan risiko sistemik (Bab 35): ambang itu kira-kira tujuh belas kali lebih besar.

Supaya model raksasa lebih murah dijalankan, arsitektur *mixture of experts* (Fedus et al., 2021) hanya mengaktifkan sebagian kecil subjaringan untuk setiap token, sehingga jumlah parameter total bisa besar sekali sementara komputasi per token tetap terkendali. Untuk menyesuaikan model ke tugas baru tanpa melatih ulang semua parameternya, LoRA (Hu et al., 2021) membekukan matriks bobot $\mathbf{W} \in \mathbb{R}^{d \times d}$ dan hanya mempelajari koreksi berpangkat rendah $\mathbf{BA}$ dengan $\mathbf{B} \in \mathbb{R}^{d \times r}$ dan $\mathbf{A} \in \mathbb{R}^{r \times d}$. Untuk $d = 4096$ dan $r = 8$, satu matriks penuh punya sekitar 16,8 juta parameter, sedangkan koreksinya hanya $2 \times 4096 \times 8 = 65\,536$, kurang dari setengah persen.

Menjalankan model juga punya hitungannya sendiri. Bobot model 70 miliar parameter dalam presisi 16 bit memakan sekitar 140 gigabyte memori. Kuantisasi ke 4 bit menurunkannya menjadi sekitar 35 gigabyte, dengan penurunan kualitas yang biasanya kecil. Selama membangkitkan teks, transformer menyimpan key dan value semua token sebelumnya agar tidak dihitung ulang (Bab 9). Untuk model dengan 80 lapisan dan delapan kepala key-value berdimensi 128, setiap token butuh $2 \times 80 \times 8 \times 128$ bilangan, yaitu sekitar 0,33 megabyte dalam presisi 16 bit. Untuk konteks 128 ribu token, itu sekitar 42 gigabyte, lebih dari bobot model yang sudah dikuantisasi. Memori ini, bukan komputasi, sering menjadi batas panjang konteks, dan karena itulah arsitektur rekuren seperti xLSTM dan Mamba, yang ingatannya berukuran tetap, kembali menarik.

38.2 Efisiensi dan keterbukaan

Model terbesar mahal untuk dijalankan, sehingga banyak penerapan memakai model yang jauh lebih kecil yang belajar dari model besar. *Knowledge distillation* (Hinton et al., 2015) melatih model murid untuk meniru distribusi keluaran model guru, bukan hanya label yang benar. Kuncinya adalah suhu T pada softmax, $p_i = e^{z_i/T} / \sum_j e^{z_j/T}$. Untuk logit guru (3, 1, 0), suhu satu memberi peluang (0,84; 0,11; 0,04), hampir sama dengan label one-hot. Suhu tiga memberi (0,53; 0,27; 0,20), yang memperlihatkan bahwa menurut guru, kelas kedua jauh lebih mirip dengan jawaban yang benar daripada kelas ketiga. Informasi tentang kemiripan antar kelas inilah, yang disebut penulisnya *dark knowledge*, yang membuat murid belajar lebih banyak dari setiap contoh. Distilasi, kuantisasi, dan pemangkasan bersama-sama membuat model yang dulu butuh pusat data bisa berjalan di laptop atau ponsel.

Model berbobot terbuka juga mengejar model tertutup. DeepSeek-V3 (DeepSeek-AI et al., 2024) adalah model mixture of experts dengan 671 miliar parameter total, tetapi hanya 37 miliar yang aktif untuk setiap token. Laporan teknisnya menyebut seluruh pelatihannya membutuhkan sekitar 2,8 juta jam GPU, angka yang oleh penulisnya ditonjolkan sebagai hemat untuk model sebesar itu, dan kinerjanya dilaporkan sebanding dengan model tertutup terkemuka. Bobot model seperti ini bisa diunduh, diperiksa, dan disetel siapa saja, dengan segala manfaat dan risiko yang dibahas di Bab 36.

38.3 Mengukur yang sulit diukur

Benchmark adalah cara bidang ini mengukur kemajuan, dan benchmark cepat jenuh. Model mutakhir sudah melampaui 90 persen pada MMLU, kumpulan soal pilihan ganda dari puluhan bidang yang beberapa tahun lalu dianggap sulit. Humanity's Last Exam (Phan et al., 2025) dirancang sebagai jawabannya: 2500 soal dari puluhan bidang, ditulis para ahli di seluruh dunia, dengan jawaban yang tidak ambigu, mudah diperiksa otomatis, dan tidak bisa ditemukan dengan pencarian internet biasa. Saat diterbitkan, model-model terbaik masih mencapai akurasi rendah, dan juga buruk dalam menilai seberapa yakin mereka seharusnya.

Benchmark yang lebih dekat ke pekerjaan nyata juga muncul. SWE-bench (Jimenez et al., 2023) berisi 2294 masalah dari isu GitHub sungguhan di dua belas repositori Python populer. Model diberi seluruh kode dan deskripsi masalahnya, lalu harus mengubah kode itu sehingga uji yang terkait lolos, tugas yang menuntut membaca konteks panjang, mengubah beberapa berkas sekaligus, dan berinteraksi dengan lingkungan eksekusi, persis pekerjaan agen pemrograman di bawah.

Semua benchmark publik menghadapi masalah yang sama dengan pembagian data di Bab 1: kebocoran. Kalau soal-soal benchmark ikut masuk ke data pelatihan yang diambil dari internet, skor tinggi tidak lagi mengukur kemampuan, melainkan ingatan. Kumpulan soal tersembunyi, soal yang terus diperbarui, dan evaluasi pada tugas yang dibuat setelah tanggal batas data pelatihan adalah upaya-upaya untuk menjaga agar angka tetap jujur.

38.4 Menyelaraskan model dengan manusia

Model bahasa yang hanya dilatih menebak kata berikutnya belum tentu membantu, jujur, atau aman. InstructGPT (Ouyang et al., 2022) memperkenalkan resep tiga langkah: penyetelan dengan contoh jawaban yang ditulis manusia, pelatihan model ganjaran dari perbandingan pasangan jawaban (Christiano et al., 2017), lalu optimasi model bahasa terhadap model ganjaran itu dengan PPO (Bab 16), sambil menjaga agar ia tidak terlalu jauh dari model awal.

Direct preference optimization (Rafailov et al., 2023) memperlihatkan bahwa langkah reinforcement learning bisa dilewati. Penurunannya elegan. Tujuan RLHF adalah memaksimalkan ganjaran harapan dikurangi penalti KL terhadap model acuan π_{ref} :

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot|x)} [r(x, y)] - \beta \text{KL}(\pi(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)). \quad (38.1)$$

Masalah ini punya solusi tertutup, $\pi^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$. Membalik persamaan ini memberi ganjaran dalam bentuk policy, $r(x, y) = \beta \log(\pi^*(y | x)/\pi_{\text{ref}}(y | x))$ ditambah suku yang hanya bergantung pada x . Dalam model preferensi Bradley–Terry, peluang jawaban y_w lebih disukai dari y_l adalah $\sigma(r(x, y_w) - r(x, y_l))$, dan suku yang hanya bergantung pada x saling menghapus dalam selisih itu. Hasilnya, preferensi bisa dipelajari langsung pada policy dengan loss

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)}\right), \quad (38.2)$$

sebuah binary cross-entropy biasa, tanpa model ganjaran terpisah dan tanpa sampling selama pelatihan. Sebagai gambaran skalanya, kalau selisih ganjaran implisit kedua jawaban adalah satu, model Bradley–Terry memberi peluang $\sigma(1) \approx 0,73$ bahwa jawaban yang lebih baik memang dipilih, dan loss-nya $-\ln 0,73 \approx 0,31$. Pelatihan mendorong selisih itu membesar, tetapi faktor β dan rasio terhadap model acuan menjaga agar model tidak menjauh terlalu banyak dari bahasa yang wajar.

38.5 Model yang bernalar

Wei et al. (2022) menunjukkan bahwa meminta model menuliskan langkah-langkah penalarannya sebelum menjawab, *chain-of-thought*, memperbaiki kinerja pada soal matematika dan logika secara nyata. Generasi berikutnya melatih kemampuan ini secara langsung. DeepSeek-R1 (DeepSeek-AI et al., 2025) memperlihatkan bahwa reinforcement learning dengan ganjaran yang bisa diperiksa secara otomatis, misalnya apakah jawaban akhir sebuah soal matematika benar atau apakah sebuah program lolos uji, membuat model belajar menulis rantai penalaran panjang, memeriksa ulang, dan mencoba jalan lain ketika buntu. Algoritme yang dipakai, GRPO (Shao et al., 2024), adalah varian PPO tanpa critic. Untuk setiap soal, model menghasilkan sekelompok jawaban, dan keunggulan setiap jawaban diukur relatif terhadap teman sekelompoknya, $A_i = (r_i - \bar{r})/\text{std}(r)$. Kalau dari empat jawaban dua benar (ganjaran satu) dan dua salah (nol), rata-ratanya 0,5, simpangan bakunya 0,5, dan keunggulannya +1 untuk yang benar dan −1 untuk yang salah. Rata-rata kelompok menggantikan fungsi nilai

sebagai baseline, gagasan yang sama dengan baseline pada REINFORCE di Bab 16, dan model nilai yang besar tidak perlu dilatih sama sekali. Pola ini mengingatkan pada pelajaran dari AlphaZero di Bab 5: memberi model lebih banyak waktu untuk “berpikir” saat menjawab, dengan mencari dan memeriksa, menjadi sumbu penskalaan baru di samping ukuran model.

Hasilnya terlihat jelas dalam matematika. AlphaGeometry (Trinh et al., 2024) menggabungkan model bahasa dengan mesin deduksi simbolik untuk menyelesaikan soal geometri olimpiade. Pada Olimpiade Matematika Internasional Juli 2025, sebuah versi Gemini Deep Think dari Google DeepMind menyelesaikan lima dari enam soal dalam bahasa alami dan diakui panitia mencapai standar medali emas¹.

38.6 Agen

Model bahasa yang bisa memakai alat berubah dari penjawab pertanyaan menjadi agen. ReAct (Yao et al., 2022) merangkai penalaran dan tindakan secara bergantian: model menulis pikiran, memanggil alat seperti pencarian web atau kalkulator, membaca hasilnya, lalu berpikir lagi. Retrieval-augmented generation (Lewis et al., 2020) memberi model akses ke basis dokumen sehingga jawabannya bisa bersandar pada sumber yang dikutip, bukan hanya pada ingatan parameternya. Agen pemrograman yang bisa membaca kode, menjalankan uji, dan memperbaiki kesalahannya sendiri menjadi salah satu penerapan yang paling cepat berkembang. Sebagian besar masalah lama muncul kembali dalam bentuk baru: credit assignment untuk tugas panjang (Bab 16), galat yang menumpuk sepanjang banyak langkah seperti pada rollout simulator (Bab 30), dan pertanyaan siapa yang bertanggung jawab ketika agen bertindak (Bab 35).

Agen juga mulai keluar dari layar. Model *vision-language-action* seperti RT-2 (Brohan et al., 2023) melatih model penglihatan dan bahasa yang sudah dilatih pada data web untuk mengeluarkan aksi robot sebagai token, sehingga robot bisa menerima perintah seperti “ambil benda yang bisa dipakai sebagai palu” dan memanfaatkan pengetahuan umum yang tidak pernah ada di data robotnya. Di sini juga, lapisan-lapisan klasik dari Bab 21, seperti kendali tingkat rendah dan pengaman, tetap bekerja di bawah model besar itu.

38.7 Gambar, video, dan model dunia

Model difusi dari Bab 15 menjadi mesin di balik generator gambar modern. Latent diffusion (Rombach et al., 2021) menjalankan difusi di ruang laten sebuah autoencoder, bukan di ruang piksel, sehingga jauh lebih murah. Diffusion transformer (Peebles and Xie, 2022) mengganti U-Net dengan transformer dan memperlihatkan bahwa model difusi juga mengikuti hukum penskalaan. Generator video adalah perpanjangan alami dari gagasan yang sama ke dimensi waktu.

¹Pengumuman Google DeepMind, Juli 2025: <https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/>. OpenAI melaporkan hasil setara dengan model eksperimentalnya, tetapi hasil itu tidak disertifikasi oleh koordinator IMO.

Model yang membangkitkan video yang konsisten secara fisis mulai disebut *world models*, sebuah istilah yang dipakai Ha and Schmidhuber (2018) untuk agen yang belajar model internal lingkungannya lalu berlatih di dalam “mimpi”-nya sendiri. Apakah model video benar-benar memahami fisika atau hanya meniru penampilannya adalah pertanyaan terbuka, dan alat dari Bagian V, seperti uji kekekalan dan ekstrapolasi, adalah cara yang baik untuk mengujinya.

38.8 AI untuk sains

Tahun 2024 memberi penanda simbolis. Hadiah Nobel Fisika diberikan kepada John Hopfield dan Geoffrey Hinton untuk penemuan dasar yang memungkinkan machine learning dengan jaringan saraf tiruan, dan setengah Hadiah Nobel Kimia diberikan kepada Demis Hassabis dan John Jumper untuk prediksi struktur protein (Bab 24). Jaringan Hopfield yang dihargai itu adalah leluhur langsung jaringan Hopfield modern di Bab 9.

Di bidang cuaca, model berbasis machine learning seperti GraphCast (Lam et al., 2023), GenCast (Price et al., 2025), dan Aurora (Bodnar et al., 2024) kini bersaing dengan sistem numerik operasional, dan beberapa pusat prakiraan cuaca mulai menjalankannya berdampingan. Model fondasi untuk fisika umum (Alkin et al., 2024b; McCabe et al., 2023) dan kumpulan data simulasi besar (Ohana et al., 2024) mencoba mengulang kesuksesan itu untuk PDE secara umum. Di kimia dan biologi, AlphaFold3 (Abramson et al., 2024) memperluas prediksi struktur ke interaksi antar molekul. Pola yang sama muncul di semua bidang ini: data simulasi dan pengukuran yang terkumpul selama puluhan tahun menjadi bahan bakar, dan struktur fisika, seperti simetri dan kekekalan, menjadi kemudi.

38.9 Yang tidak berubah

Di tengah semua perubahan ini, pelajaran dari kuliah-kuliah dasar tetap berlaku, bahkan semakin penting. Generalisasi di luar data latih tetap sulit (Bab 1). Model besar tetap bisa belajar jalan pintas yang salah, dan evaluasi yang jujur tetap memerlukan pembagian data yang hati-hati. Optimasi tetap bergantung pada condition number dan noise (Bab 4). Model generatif tetap bisa berhalusinasi, sehingga penalaran yang bisa diperiksa dan sumber yang bisa dikutip menjadi semakin berharga. Dan sistem yang kuat tetap harus dijelaskan dengan jujur kepada orang-orang yang hidupnya terdampak (Bab 17 dan 35).

Kalau ada satu benang merah yang saya bawa dari empat semester ini, mungkin begini: kemajuan terbesar jarang datang dari membuang apa yang sudah diketahui. LSTM bertahan karena constant error carousel adalah gagasan yang benar. Konvolusi bertahan karena simetri geser adalah sifat dunia. Persamaan Navier–Stokes tidak berhenti berlaku karena ada jaringan saraf. Model terbaik adalah yang belajar dari data sambil menghormati apa yang sudah kita ketahui tentang dunia.

CATATAN SUMBER. Bab ini disusun dari bacaan, bukan dari satu mata kuliah. Sebagian topiknya disinggung di kuliah LSTM (xLSTM dan Mamba), NLP (model bahasa besar), AI and Law

(model AI serbaguna), dan AI in Life Sciences (model bahasa untuk kimia). Perkembangan yang baru diperiksa dengan pengumuman resminya pada saat penulisan, Oktober 2026.

A Glosarium

Buku ini membiarkan banyak istilah dalam bahasa Inggris, karena istilah itulah yang dipakai di kuliah dan literatur. Daftar berikut memberi penjelasan singkat dan bab tempat istilah itu dibahas.

activation function

Fungsi nonlinear yang diterapkan pada keluaran setiap neuron, seperti ReLU atau sigmoid (Bab 7).

active learning

Memilih data berikutnya yang paling informatif untuk diberi label (Bab 23).

actor-critic

Metode RL dengan satu jaringan yang memilih aksi dan satu yang menilai (Bab 16).

adjoint method

Cara menghitung gradien sistem dinamis dengan menyelesaikan persamaan mundur dalam waktu (Bab 15).

advantage

Selisih nilai sebuah aksi dengan nilai rata-rata keadaan (Bab 16).

AGI (artificial general intelligence)

Sistem yang mampu mengerjakan beragam tugas kognitif setara manusia; definisinya masih diperdebatkan (Bab 36).

alignment

Upaya membuat sistem AI mengejar apa yang sebenarnya dimaksud manusia, bukan sekadar tujuan yang tertulis (Bab 36 dan 38).

attention

Mekanisme membaca ingatan dengan query, key, dan value (Bab 9).

autodiff (automatic differentiation)

Menghitung turunan program secara eksak dengan aturan rantai (Bab 7).

autoencoder

Jaringan yang memampatkan input menjadi kode kecil lalu merekonstruksinya (Bab 12).

backpropagation

Aturan rantai yang dihitung dari keluaran ke input untuk mendapat gradien (Bab 7).

bag of words

Representasi dokumen sebagai hitungan kata tanpa urutan (Bab 10).

batch normalization

Normalisasi pre-activation dengan statistik mini-batch (Bab 7).

Bayes risk

Risiko terkecil yang mungkin dicapai prediktor apa pun (Bab 1).

Bayesian network

Graf berarah yang menyandikan independensi bersyarat antar variabel acak (Bab 14).

Bayesian optimization

Memilih percobaan berikutnya dengan model pengganti yang memberi ketidakpastian, misalnya untuk hyperparameter (Bab 8).

Bell inequality (CHSH)

Batas korelasi untuk teori lokal-realis yang dilanggar keadaan kuantum yang terkait (Bab 37).

BERT

Model encoder yang dilatih menebak token yang ditutup dari konteks di kedua sisinya (Bab 10).

bias-variance trade-off

Pertukaran antara galat karena model terlalu sederhana dan galat karena model terlalu peka pada data (Bab 1).

biclustering

Pengelompokan baris dan kolom matriks sekaligus (Bab 6).

binding pocket

Kantong di permukaan protein tempat molekul kecil menempel (Bab 24).

bootstrapping (RL)

Memperbarui perkiraan dengan perkiraan lain, seperti pada TD (Bab 16).

byte-pair encoding

Tokenisasi yang berulang kali menggabungkan pasangan simbol paling sering (Bab 10 dan 38).

CFAR (constant false alarm rate)

Detektor radar dengan ambang yang menyesuaikan diri dengan noise lokal (Bab 18).

closure problem

Persamaan rata-rata turbulensi mengandung suku yang tidak diketahui (Bab 27).

collaborative filtering

Rekomendasi dari kemiripan selera antar pengguna atau antar item (Bab 34).

collocation points

Titik-titik tempat residual PDE dievaluasi pada PINN (Bab 29).

condition number

Rasio kelengkungan terbesar dan terkecil, penentu kecepatan gradient descent (Bab 4).

conformal prediction

Interval prediksi dengan jaminan cakupan tanpa asumsi distribusi (Bab 1).

constant error carousel

Jalur aditif pada sel LSTM yang membuat gradien tidak hilang (Bab 9).

contrastive learning

Belajar representasi dengan mendekatkan pasangan yang cocok dan menjauhkan yang tidak (Bab 23).

convolution

Operasi linear dengan kernel lokal yang dipakai di semua posisi, ekuivarian terhadap pergeseran (Bab 7 dan 13).

counterfactual explanation

Perubahan terkecil pada input yang mengubah keputusan model (Bab 17).

covariate shift

Distribusi input saat dipakai berbeda dari saat dilatih (Bab 16 dan 30).

credit assignment

Menentukan tindakan mana yang bertanggung jawab atas ganjaran yang datang terlambat (Bab 16).

cross-validation

Memperkirakan generalisasi dengan melatih dan menguji bergantian pada bagian-bagian data (Bab 1).

curse of dimensionality

Kesulitan belajar yang tumbuh eksponensial dengan dimensi (Bab 1).

data assimilation

Menggabungkan model dan pengukuran dengan bobot sesuai ketidakpastian (Bab 14).

decoherence

Hilangnya superposisi kuantum karena keterkaitan dengan lingkungan (Bab 37).

DeepONet

Neural operator dengan jaringan branch dan trunk (Bab 30).

degrees of freedom (ridge)

Jumlah parameter efektif setelah regularisasi, $\sum_i \lambda_i^2 / (\lambda_i + \lambda)^2$ (Bab 3).

differential privacy

Jaminan bahwa hasil analisis hampir tidak berubah karena data satu orang (Bab 35).

diffusion model

Model generatif yang belajar membalik proses penambahan noise (Bab 15).

DMD (dynamic mode decomposition)

Mode dinamika dari operator linear yang dipasang pada potret berurutan (Bab 31).

double descent

Galat uji turun lagi setelah model mampu menghafal data latih (Bab 1).

DPO (direct preference optimization)

Melatih model bahasa langsung dari pasangan preferensi tanpa model ganjaran terpisah (Bab 38).

dropout

Mematikan neuron acak selama pelatihan sebagai regularisasi (Bab 7).

ELBO (evidence lower bound)

Batas bawah log-likelihood yang dimaksimalkan VAE dan model difusi (Bab 12 dan 15).

embedding

Representasi vektor padat untuk kata, molekul, pengguna, atau item (Bab 9).

entanglement

Keadaan banyak qubit yang tidak bisa ditulis sebagai hasil kali keadaan masing-masing (Bab 37).

equivariance

Kalau input ditransformasi, keluaran ikut tertransformasi dengan cara yang sama (Bab 13).

evidence (marginal likelihood)

Peluang data setelah parameter dirata-ratakan, dipakai untuk memilih model (Bab 2).

excess risk

Selisih risiko sebuah model dengan risiko terkecil yang mungkin (Bab 3).

expected improvement

Fungsi akuisisi yang menimbang perbaikan harapan atas hasil terbaik sejauh ini (Bab 8).

exploration–exploitation

Dilema antara mencoba hal baru dan memakai yang sudah terbukti baik (Bab 16).

few-shot / zero-shot learning

Belajar tugas baru dari sedikit atau tanpa contoh berlabel (Bab 23).

filtered backprojection

Algoritme rekonstruksi CT dengan filter ramp sebelum backprojection (Bab 25).

fingerprint (molecular)

Vektor biner penanda substruktur molekul (Bab 23).

Fisher information

Ukuran informasi yang dibawa data tentang parameter, dasar batas Cramér–Rao (Bab 15).

flow matching

Melatih medan kecepatan ODE yang membawa noise ke data (Bab 15).

FNO (Fourier neural operator)

Neural operator yang memakai konvolusi di domain Fourier (Bab 30).

foundation model

Model besar yang dilatih luas lalu disesuaikan ke banyak tugas (Bab 38).

GAN

Generator dan diskriminator yang dipertandingkan (Bab 12).

Gaussian process

Prior atas fungsi yang ditentukan oleh sebuah kernel, dengan posterior tertutup (Bab 15).

generalization error

Loss harapan pada distribusi data sebenarnya (Bab 1).

Gibbs sampling

MCMC yang memperbarui satu variabel pada satu waktu (Bab 14).

GloVe

Embedding kata dari regresi pada logaritma hitungan kemunculan bersama (Bab 10).

gradient accumulation

Menjumlahkan gradien beberapa mini-batch kecil sebelum satu pembaruan (Bab 8).

gradient clipping

Membatasi norma gradien agar satu langkah tidak terlalu besar (Bab 4).

gradient descent

Melangkah berlawanan arah gradien untuk meminimalkan fungsi (Bab 4).

graph neural network

Jaringan yang bekerja pada graf lewat message passing (Bab 13).

hidden Markov model

Model deret dengan keadaan diskret tersembunyi (Bab 14).

h-index

Nilai h terbesar sehingga seorang peneliti punya h makalah yang masing-masing dikutip paling sedikit h kali (Lampiran C).

homography

Matriks 3×3 yang menghubungkan dua gambar dari bidang datar atau kamera yang berputar (Bab 11).

hyperparameter

Pengaturan model yang dipilih di luar pelatihan, misalnya laju belajar (Bab 1).

ICA (independent component analysis)

Memisahkan sumber independen dari campurannya (Bab 6).

imitation learning

Belajar policy dari demonstrasi ahli (Bab 16).

inductive bias

Asumsi yang dibangun ke dalam arsitektur model (Bab 0 dan 13).

invariance

Keluaran tidak berubah kalau input ditransformasi (Bab 13).

Kalman filter

Estimator optimal berurutan untuk sistem linear Gauss (Bab 14 dan 19).

kernel (CNN)

Matriks bobot kecil yang digeser pada konvolusi (Bab 7).

kernel trick

Bekerja hanya dengan hasil kali dalam antar fitur tanpa menghitung fiturnya (Bab 3).

knowledge distillation

Melatih model kecil untuk meniru distribusi keluaran model besar (Bab 38).

Koopman operator

Operator linear yang memajukan fungsi pengamatan sistem dinamis (Bab 31).

latent variable

Variabel tersembunyi yang tidak diamati tetapi menjelaskan data (Bab 15).

learning rate

Panjang langkah pada gradient descent (Bab 4).

light field

Rekaman intensitas cahaya menurut posisi dan arah, yang memungkinkan fokus dipilih setelah pemotretan (Bab 11).

LMS

Filter adaptif dengan update gradien stokastik (Bab 18).

LSTM

Jaringan rekuren dengan sel ingatan dan gerbang (Bab 9).

MAP (maximum a posteriori)

Estimasi yang memaksimalkan posterior (Bab 1).

maximum likelihood

Estimasi yang memaksimalkan peluang data di bawah model (Bab 1).

MCMC

Menarik sampel dengan rantai Markov yang distribusi stasionernya adalah target (Bab 14 dan 28).

MDP (Markov decision process)

Kerangka formal reinforcement learning (Bab 16).

message passing

Simpul mengumpulkan pesan dari tetangga lalu memperbarui keadaannya (Bab 13).

meta-learning

Belajar titik awal atau cara belajar yang cepat beradaptasi ke tugas baru (Bab 16).

mini-batch

Sebagian kecil data yang dipakai untuk satu langkah SGD (Bab 4).

mixed precision

Melatih dengan bilangan 16 bit sambil menjaga salinan bobot 32 bit dan menskalakan loss (Bab 8).

mixture of experts

Arsitektur yang hanya mengaktifkan sebagian subjaringan untuk setiap token (Bab 38).

mode collapse

Model generatif hanya menghasilkan sebagian kecil variasi data (Bab 12).

momentum

Menambahkan ingatan arah sebelumnya pada gradient descent (Bab 4).

multigrid

Solver sistem linear yang meredam galat di berbagai tingkat kehalusan grid (Bab 27).

multi-label classification

Klasifikasi di mana satu contoh boleh punya beberapa label sekaligus (Bab 10).

Nash equilibrium

Strategi di mana tidak ada pemain yang untung mengubah strateginya sendiri (Bab 16).

negative sampling

Mengganti softmax atas seluruh kosakata dengan klasifikasi biner pasangan asli dan acak (Bab 10).

neural field

Jaringan yang mewakili medan kontinu sebagai fungsi koordinat (Bab 15).

neural ODE

Persamaan diferensial biasa dengan ruas kanan berupa jaringan saraf (Bab 15).

neural operator

Model yang belajar pemetaan antar ruang fungsi (Bab 30).

n-gram

Model bahasa yang menebak kata dari $n - 1$ kata sebelumnya dengan hitungan (Bab 10).

no free lunch theorem

Tidak ada algoritme belajar yang baik untuk semua distribusi data tanpa asumsi (Bab 3).

normalized cut

Kriteria segmentasi graf yang menghukum potongan yang menghasilkan bagian kecil (Bab 11).

normalizing flow

Model generatif dari transformasi bijektif dengan likelihood eksak (Bab 12).

nudging

Menarik keadaan model ke arah pengamatan dengan suku relaksasi (Bab 32).

numerical diffusion

Difusi palsu akibat galat diskretisasi (Bab 27).

optical flow

Medan gerak piksel antar bingkai video (Bab 11).

overfitting

Model menghafal data latih dan gagal pada data baru (Bab 1).

oversquashing

Informasi dari banyak simpul jauh terjepit di vektor berukuran tetap (Bab 13).

parameter-shift rule

Cara menghitung gradien rangkaian kuantum secara eksak dari dua evaluasi (Bab 37).

Pareto front

Kumpulan kandidat yang tidak didominasi kandidat lain dalam optimasi banyak tujuan (Bab 23).

patent claim

Bagian paten yang menentukan apa yang dilindungi (Lampiran C).

PCA / POD

Arah-arrah dengan varians atau energi terbesar dari data (Bab 6 dan 31).

perplexity

Kebalikan rata-rata geometris peluang per kata, ukuran kinerja model bahasa (Bab 10).

PINN

Jaringan yang dilatih agar memenuhi persamaan diferensial lewat loss (Bab 29).

PMI (pointwise mutual information)

Seberapa sering dua kata muncul bersama dibanding kalau independen (Bab 10).

policy

Aturan pemilihan aksi agen (Bab 16).

policy gradient

Mengoptimalkan policy langsung dengan gradien return harapan (Bab 16).

positional encoding

Informasi posisi yang ditambahkan ke token transformer (Bab 9).

posterior / prior

Keyakinan setelah dan sebelum melihat data (Bab 1).

PPO

Policy gradient dengan rasio peluang yang dipotong (Bab 16).

precision / recall

Proporsi tebakan positif yang benar, dan proporsi positif sebenarnya yang tertangkap (Bab 1).

QSAR

Model aktivitas biologis dari struktur molekul (Bab 23).

qubit

Satuan informasi kuantum yang bisa berada dalam superposisi $|0\rangle$ dan $|1\rangle$ (Bab 37).

RANS / LES / DNS

Tiga tingkat pemodelan turbulensi, dari rata-rata sampai penyelesaian penuh (Bab 27).

RANSAC

Penaksiran model yang tahan pencilan dengan sampel minimal acak yang diulang (Bab 11).

receptive field

Daerah input yang memengaruhi sebuah neuron (Bab 7).

recurrence matrix

Matriks jarak antar bingkai aliran dalam rCFD (Bab 33).

regularization

Pengekangan model agar tidak menghafal (Bab 7).

reparameterization trick

Menulis sampel acak sebagai fungsi diferensiabel parameter (Bab 12).

representer theorem

Solusi regularisasi di ruang Hilbert selalu kombinasi fitur titik-titik data (Bab 3).

residual connection

Jalur pintas $y = x + F(x)$ pada ResNet (Bab 12).

residual (PDE)

Seberapa jauh sebuah fungsi gagal memenuhi persamaan diferensial (Bab 29).

reward hacking

Agan memaksimalkan ganjaran lewat celah yang tidak dimaksudkan perancanganya (Bab 16 dan 36).

risk minimizer

Prediktor terbaik yang mungkin, misalnya rata-rata bersyarat untuk loss kuadrat (Bab 3).

RLHF

Menyetel model bahasa dengan ganjaran dari preferensi manusia (Bab 16 dan 38).

rollout

Menjalankan model berulang kali dari tebakannya sendiri (Bab 30).

saliency map

Peta kepekaan keluaran terhadap setiap piksel input (Bab 17).

scaling laws

Hubungan hukum pangkat antara ukuran model, data, komputasi, dan loss (Bab 38).

score

Gradien log kerapatan, $\nabla_x \log p(x)$ (Bab 15).

segmentation

Memberi label pada setiap piksel (Bab 11).

self-supervised learning

Membuat target dari data itu sendiri (Bab 1).

SGD

Gradient descent dengan gradien dari mini-batch acak (Bab 4).

Shapley value

Pembagian kontribusi yang adil antar fitur dari teori permainan (Bab 17).

SINDy

Menemukan persamaan dinamika dengan regresi jarang (Bab 31).

spectral bias

Kecenderungan jaringan mempelajari frekuensi rendah lebih dulu (Bab 15).

state space model

Model dengan keadaan internal yang berevolusi linear, dasar Mamba dan teori kendali (Bab 9 dan 19).

Stokes number

Perbandingan waktu respons partikel dengan waktu khas aliran (Bab 27).

superintelligence

Kecerdasan yang jauh melampaui manusia di hampir semua bidang (Bab 36).

surrogate model

Model murah yang meniru simulasi mahal (Bab 1 dan 30).

synthetic aperture

Menggabungkan banyak sudut pandang menjadi satu lensa raksasa dengan fokus yang bisa dipilih (Bab 11).

TD (temporal difference)

Belajar nilai dari selisih ramalan berurutan (Bab 16).

transfer learning

Memakai model yang dilatih pada satu tugas untuk tugas lain (Bab 12).

transformer

Arsitektur berbasis attention tanpa rekurensi (Bab 9).

uncanny valley

Rasa ganjil terhadap mesin yang hampir mirip manusia (Bab 35).

U-Net

Encoder–decoder dengan skip connection antar tingkat (Bab 11).

VAE

Autoencoder probabilistik yang dilatih dengan ELBO (Bab 12).

value function

Return harapan dari sebuah keadaan atau pasangan keadaan–aksi (Bab 16).

vanishing gradient

Gradien menyusut eksponensial melewati banyak lapisan atau langkah waktu (Bab 9).

watermark (teks)

Sinyal statistik tersembunyi di keluaran model bahasa yang bisa dideteksi (Bab 35).

weight decay

Regularisasi L2 pada bobot, setara prior Gauss (Bab 1).

Weisfeiler–Leman test

Uji pewarnaan graf yang membatasi kekuatan message passing (Bab 13).

word2vec

Embedding kata dari tugas menebak kata konteks (Bab 10).

xLSTM

LSTM modern dengan exponential gating dan ingatan matriks (Bab 9).

B Bekal Matematika

Lampiran ini mengumpulkan alat matematika yang dipakai berulang kali di buku ini. Ia bukan pengganti buku teks, tetapi cukup untuk mengikuti penurunan di bab-bab sebelumnya. Bab teori peluang dan lampiran notasi dalam naskah kuliah Machine Learning: Basic Techniques mengikuti susunan yang mirip, dan buku Deisenroth et al. (2020) adalah bacaan lanjutan yang sangat cocok untuk tujuan yang sama.

B.1 Aljabar linear

B.1.1 Vektor, matriks, dan hasil kali

Vektor ditulis tebal huruf kecil, $\mathbf{x} \in \mathbb{R}^n$, dan dianggap kolom. Matriks ditulis tebal huruf besar, $\mathbf{A} \in \mathbb{R}^{m \times n}$. Hasil kali matriks dan vektor, $\mathbf{Ax}$, bisa dibaca dengan dua cara. Sebagai baris: setiap komponen hasilnya adalah hasil kali dalam satu baris $\mathbf{A}$ dengan $\mathbf{x}$. Sebagai kolom: hasilnya adalah kombinasi linear kolom-kolom $\mathbf{A}$ dengan bobot komponen $\mathbf{x}$. Cara kedua lebih berguna untuk memahami regresi linear (Bab 1): prediksi selalu berada di ruang yang direntang kolom-kolom matriks fitur.

Hasil kali dalam $\mathbf{x}^\top \mathbf{y} = \sum_i x_i y_i$ mengukur seberapa searah dua vektor, dan $\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$. Norma Euclides $\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}$ adalah panjang biasa. Norma $\|\mathbf{x}\|_1 = \sum_i |x_i|$ menjumlahkan nilai mutlak, dan penalti dengan norma ini mendorong solusi yang jarang, seperti pada lasso. Untuk $\mathbf{x} = (3, 4)$, $\|\mathbf{x}\|_2 = 5$ dan $\|\mathbf{x}\|_1 = 7$.

Hasil kali luar $\mathbf{xy}^\top$ adalah matriks berpangkat satu. Kovarians sampel adalah rata-rata hasil kali luar dari simpangan-simpangan, $\frac{1}{N} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$, dan ingatan matriks mLSTM di Bab 9 adalah jumlah hasil kali luar value dan key.

B.1.2 Nilai eigen dan dekomposisi

Vektor eigen sebuah matriks persegi adalah arah yang tidak dibelokkan matriks itu, hanya diregangkan: $\mathbf{Av} = \lambda \mathbf{v}$. Nilai eigen dicari dari $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$. Untuk $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, persamaan karakteristiknya $(2 - \lambda)^2 - 1 = 0$, sehingga $\lambda = 3$ dan $\lambda = 1$, dengan vektor eigen $(1, 1)$ dan $(1, -1)$. Matriks simetris selalu punya nilai eigen real dan vektor eigen yang saling tegak lurus, sehingga bisa diuraikan sebagai $\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$ dengan $\mathbf{Q}$ ortogonal. Dalam basis vektor eigen, matriks simetris hanya meregangkan setiap sumbu.

Matriks simetris disebut definit positif kalau $\mathbf{x}^\top \mathbf{Ax} > 0$ untuk setiap $\mathbf{x} \neq \mathbf{0}$, yang setara dengan semua nilai eigennya positif. Hessien yang definit positif menandai minimum (Bab 4), dan matriks kovarians selalu semidefinit positif, karena $\mathbf{x}^\top \mathbf{\Sigma} \mathbf{x}$ adalah varians proyeksi data ke arah $\mathbf{x}$, yang tidak mungkin negatif.

Matriks sembarang, tidak harus persegi, punya dekomposisi nilai singular $\mathbf{A} =$

$U\Sigma V^T$. Kolom-kolom V adalah arah di ruang input, kolom-kolom U arah di ruang output, dan nilai singular di diagonal Σ adalah faktor regangan dari satu ke yang lain. Setiap matriks memetakan bola menjadi elips, dan SVD memberi sumbu-sumbu elips itu. Nilai singular adalah akar nilai eigen $A^T A$. Dekomposisi ini muncul di PCA dan POD (Bab 6 dan 31), algoritme Kabsch (Bab 24), dan LoRA (Bab 38). Rasio nilai singular terbesar terhadap terkecil adalah condition number, yang mengukur seberapa besar galat kecil di input bisa diperbesar ketika sistem $Ax = b$ diselesaikan.

B.2 Kalkulus banyak variabel

B.2.1 Gradien dan Jacobian

Untuk fungsi skalar $f : \mathbb{R}^n \rightarrow \mathbb{R}$, gradien ∇f adalah vektor turunan parsial. Ia menunjuk ke arah kenaikan paling cepat, dan besarnya adalah laju kenaikan di arah itu. Untuk $f(x, y) = x^2 + 3xy$, $\nabla f = (2x + 3y, 3x)$, yang di titik $(1, 2)$ bernilai $(8, 3)$. Untuk fungsi vektor $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, Jacobian adalah matriks $m \times n$ berisi semua turunan parsial $\partial f_i / \partial x_j$. Jacobian lengan robot di Bab 21 adalah contohnya.

Beberapa gradien muncul berkali-kali dan layak dihafal: $\nabla_x(a^T x) = a$, $\nabla_x(x^T Ax) = (A + A^T)x$, yang menjadi $2Ax$ untuk A simetris, dan $\nabla_x \|Ax - b\|^2 = 2A^T(Ax - b)$. Yang terakhir langsung memberi normal equations regresi linear.

B.2.2 Aturan rantai

Kalau $y = g(x)$ dan $z = f(y)$, maka $\nabla_x z = J_g^T \nabla_y z$, dengan J_g Jacobian g . Untuk rantai yang panjang, $x \rightarrow y_1 \rightarrow y_2 \rightarrow \dots \rightarrow z$, gradien diperoleh dengan mengalikan Jacobian-Jacobian berturut-turut. Urutan perkaliannya menentukan biaya. Mulai dari ujung z yang skalar dan bergerak mundur berarti setiap langkah adalah perkalian matriks dengan vektor, murah. Itulah reverse-mode autodiff dan backpropagation (Bab 7). Mulai dari x berarti membawa matriks Jacobian penuh sepanjang rantai, yang hanya efisien kalau inputnya sedikit, seperti RTRL di Bab 9.

B.2.3 Ekspansi Taylor

Di sekitar sebuah titik, fungsi yang mulus didekati dengan polinomial,

$$f(x + h) \approx f(x) + \nabla f(x)^T h + \frac{1}{2} h^T \nabla^2 f(x) h. \quad (\text{B.1})$$

Aproksimasi orde satu melahirkan gradient descent, aproksimasi orde dua melahirkan metode Newton (Bab 4). Untuk $f(x) = e^x$ di sekitar nol, $e^{0.1} \approx 1 + 0.1 + 0.005 = 1.105$, sedangkan nilai sebenarnya 1.10517. Taylor juga ada di balik analisis galat skema numerik (Bab 27), linearisasi sistem nonlinear (Bab 19), dan extended Kalman filter (Bab 21).

B.3 Peluang

B.3.1 Aturan dasar dan Bayes

Dua aturan menghasilkan hampir semua yang lain. Aturan jumlah, $p(x) = \sum_y p(x, y)$, dan aturan hasil kali, $p(x, y) = p(y | x) p(x)$. Menuliskan aturan hasil kali dalam dua

urutan dan menyamakannya memberi teorema Bayes,

$$p(y | x) = \frac{p(x | y) p(y)}{p(x)} = \frac{p(x | y) p(y)}{\sum_{y'} p(x | y') p(y')}. \quad (\text{B.2})$$

Contoh tes penyakit di Bab 17 adalah penerapan langsungnya: sensitivitas 0,9, laju positif palsu 0,09, dan prevalensi 0,01 memberi $p(\text{sakit} | +) = 0,009 / (0,009 + 0,0891) \approx 0,09$.

B.3.2 Ekspektasi dan varians

Ekspektasi adalah rata-rata berbobot peluang, $\mathbb{E}[x] = \sum_x x p(x)$ atau integral untuk variabel kontinu. Ia linear: $\mathbb{E}[ax + by] = a\mathbb{E}[x] + b\mathbb{E}[y]$, selalu, bahkan kalau x dan y bergantung. Varians adalah ekspektasi kuadrat simpangan, $\text{Var}[x] = \mathbb{E}[(x - \mathbb{E}x)^2] = \mathbb{E}[x^2] - (\mathbb{E}x)^2$. Untuk variabel independen, varians jumlah adalah jumlah varians, sehingga varians rata-rata N sampel independen adalah σ^2/N . Fakta sederhana ini ada di balik galat Monte Carlo yang turun seperti $1/\sqrt{N}$ (Bab 28), noise mini-batch (Bab 4), dan skala $\sqrt{d_k}$ pada attention (Bab 9).

B.3.3 Distribusi Gauss

Gauss multivariat $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ punya kerapatan

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right). \quad (\text{B.3})$$

Ia muncul di mana-mana karena beberapa sifatnya yang tertutup. Transformasi linear dari Gauss tetap Gauss: $\mathbf{Ax} + \mathbf{b} \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu} + \mathbf{b}, \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top)$. Marginal dan kondisional dari Gauss bersama tetap Gauss. Hasil kali dua kerapatan Gauss, setelah dinormalisasi, tetap Gauss, dengan presisi yang dijumlahkan, sifat yang menghasilkan Kalman filter (Bab 14). Dan jumlah banyak variabel independen cenderung Gauss menurut teorema limit pusat, alasan mengapa noise pengukuran sering dimodelkan Gauss, dan dasar argumen ICA di Bab 6.

Sampel dari $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ bisa dibuat dari sampel normal baku ϵ dengan $\mathbf{x} = \boldsymbol{\mu} + \mathbf{L}\epsilon$, dengan $\mathbf{L}$ faktor Cholesky, $\mathbf{L}\mathbf{L}^\top = \boldsymbol{\Sigma}$. Kovarians hasilnya adalah $\mathbf{L}\mathbf{I}\mathbf{L}^\top = \boldsymbol{\Sigma}$ sesuai sifat transformasi linear di atas. Inilah reparameterization trick pada VAE (Bab 12) dalam bentuk umumnya.

B.4 Teori informasi

Entropi mengukur ketidakpastian rata-rata sebuah distribusi, $H(p) = -\sum_x p(x) \log_2 p(x)$, dalam bit. Lemparan koin jujur punya entropi satu bit. Koin yang muncul gambar dengan peluang 0,9 punya entropi $-0,9 \log_2 0,9 - 0,1 \log_2 0,1 \approx 0,47$ bit: hasilnya lebih mudah ditebak, sehingga rata-rata informasi per lemparan lebih sedikit. Entropi dipakai untuk memilih pertanyaan di pohon keputusan (Bab 6).

Cross-entropy $H(p, q) = -\sum_x p(x) \log q(x)$ mengukur biaya rata-rata kalau data dari p dikodekan dengan kode yang dirancang untuk q . Ia selalu paling sedikit sebesar $H(p)$, dan selisihnya adalah divergensi Kullback–Leibler,

$$\text{KL}(p||q) = H(p, q) - H(p) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0. \quad (\text{B.4})$$

Untuk distribusi target $p = (1, 0)$, cross-entropy terhadap prediksi $q = (0,8; 0,2)$ adalah $-\ln 0,8 \approx 0,22$ nat, dan karena entropi target nol, KL-nya sama. Meminimalkan cross-entropy terhadap label sama dengan meminimalkan KL dari distribusi data ke model, satu-satunya alasan mengapa loss klasifikasi berbentuk seperti itu (Bab 1). KL tidak simetris. $KL(p||q)$ menghukum keras tempat p besar tetapi q kecil, sehingga meminimalkannya terhadap q membuat q menutupi semua puncak p . $KL(q||p)$ menghukum tempat q besar tetapi p kecil, sehingga q cenderung memilih satu puncak saja. Perbedaan ini menjelaskan mengapa inferensi variasional, yang meminimalkan $KL(q||p)$, cenderung meremehkan ketidakpastian (Bab 15), dan mengapa t-SNE lebih menjaga tetangga dekat daripada jarak jauh (Bab 17).

C Meneliti, Menerbitkan, dan Mematenkan

Lampiran ini untuk pembaca yang ingin melangkah dari membaca ke meneliti. Isinya bersandar pada kuliah Patent and Literature Searching yang saya ikuti pada musim dingin 2024/25, di semester pertama. Kuliahnya membahas dua jalan yang dipakai ilmu dan teknologi untuk membagikan temuan: publikasi ilmiah dan paten. Keduanya punya aturan main yang berbeda, dan peneliti AI hampir pasti bersentuhan dengan keduanya.

C.1 Publikasi ilmiah

Kuliah membuka dengan kalimat yang jujur: publikasi adalah mata uang komunitas akademik. Peneliti dinilai dari makalah yang ia terbitkan di jurnal bereputasi, dan dari seberapa sering makalah itu dikutip. Publikasi juga punya fungsi yang lebih mulia: melaporkan hasil, memicu riset lanjutan, menghemat sumber daya karena percobaan yang sama tidak perlu diulang di tempat lain, dan memberi pegangan bagi pembuat kebijakan dan industri.

Makalah ilmiah, apa pun bidangnya, mencoba menjawab serangkaian pertanyaan yang sama. Apa yang saya kerjakan, secara ringkas? Apa masalahnya? Bagaimana saya mempelajari atau menyelesaikannya? Apa yang saya temukan? Apa artinya? Apa yang saya pelajari dan apa yang sebaiknya dikerjakan berikutnya? Siapa yang membantu dan siapa yang membiayai? Karya siapa yang saya rujuk? Susunan abstrak, pendahuluan, metode, hasil, diskusi, dan kesimpulan adalah cara baku untuk menjawabnya, dan setiap jurnal menetapkan detailnya dalam pedoman penulis.

Sebelum terbit, makalah melewati *peer review*: editor mengirimkannya ke beberapa ahli di bidang yang sama, yang menilai apakah metodenya benar, hasilnya didukung bukti, dan sumbangannya cukup baru. Di bidang machine learning, konferensi seperti NeurIPS, ICML, dan ICLR memainkan peran yang di bidang lain dipegang jurnal, dan banyak makalah beredar lebih dulu sebagai pracetak di arXiv sebelum atau tanpa melewati review. Pracetak mempercepat pertukaran gagasan, tetapi pembaca harus ingat bahwa belum ada orang lain yang memeriksanya.

Kuliah juga mencatat apa yang *tidak* biasanya diterbitkan: hasil negatif, percobaan yang gagal mereproduksi hasil yang sudah terbit, dan temuan yang tampak bertentangan dengan konsep yang mapan. Akibatnya, literatur yang terbit cenderung lebih optimistis dari kenyataan, sebuah bias publikasi. Pelajarannya bagi peneliti machine learning: satu makalah yang melaporkan metode baru mengungguli pembanding belum berarti metode itu selalu lebih baik, dan reproduksi independen berharga.

C.1.1 Mengukur dampak

Ukuran dampak yang paling dikenal adalah *h-index*: nilai *h* terbesar sehingga seorang peneliti punya *h* makalah yang masing-masing dikutip paling sedikit *h* kali. Contoh di kuliah adalah daftar makalah dengan jumlah kutipan 25, 20, 10, 8, 7, 6, lalu makalah-makalah yang dikutip lebih sedikit. Enam makalah teratas masing-masing dikutip paling sedikit enam kali, tetapi tidak ada tujuh makalah yang masing-masing dikutip paling sedikit tujuh kali, sehingga $h = 6$. Ukuran ini menggabungkan produktivitas dan dampak, tetapi buta terhadap urutan penulis, bergantung pada bidang, dan bisa dinaikkan dengan saling mengutip. *Impact factor* sebuah jurnal, rata-rata kutipan yang diterima makalah-makalahnya dalam dua tahun sebelumnya, punya masalah serupa. Angka-angka ini berguna sebagai petunjuk kasar, dan menyesatkan kalau dijadikan tujuan, hukum Goodhart yang sama dengan reward hacking di Bab 36.

C.1.2 Menemukan dan mengakses literatur

Basis data seperti Web of Science, Scopus, SciFinder untuk kimia, dan Google Scholar membantu menemukan makalah yang relevan, melacak siapa yang mengutip siapa, dan menemukan makalah-makalah baru dari satu makalah lama yang penting. Banyak makalah dilindungi hak cipta dan berbayar. Kuliah membahas jalan-jalan yang sah untuk mendapatkannya: langganan perpustakaan universitas, meminta salinan langsung kepada penulis, yang hampir selalu senang mengirimkannya, dan versi *open access*. Pada *gold open access*, penerbit membuat makalah gratis dibaca dan biaya ditanggung penulis atau lembaga. Pada *green open access*, penulis mengarsipkan sendiri salinannya, misalnya di repositori universitas atau arXiv. Pada *diamond open access*, baik penulis maupun pembaca tidak membayar, karena biaya ditanggung lembaga atau komunitas.

Daftar pustaka buku ini dibangun dengan alat-alat yang sama. Setiap artikel diambil metadatanya langsung dari Crossref berdasarkan DOI-nya atau dari arXiv berdasarkan nomornya, lalu dicocokkan dengan kata kunci judul yang diharapkan, sehingga judul, penulis, dan tahun yang tercetak memang berasal dari catatan resmi, bukan dari ingatan. Buku diperiksa terhadap katalog perpustakaan terbuka atau halaman hak ciptanya. Kebiasaan sederhana ini mencegah kesalahan yang umum, yaitu mengutip sesuatu yang tidak pernah ada atau mengutip dengan detail yang salah, kesalahan yang semakin sering terjadi sejak orang meminta model bahasa menuliskan daftar pustaka.

C.2 Paten

C.2.1 Apa itu paten

Paten adalah salah satu hak kekayaan intelektual, bersama merek, desain industri, hak cipta, dan model utilitas. Menurut definisi yang dipakai kuliah, paten adalah hak yang diberikan negara kepada penemu untuk melarang orang lain membuat, memakai, menjual, atau mengimpor penemuannya di negara itu selama waktu terbatas, *sebagai ganti* pengungkapan penemuan itu kepada publik. Pertukaran inilah intinya. Penemu mendapat monopoli sementara, masyarakat mendapat pengetahuan yang lengkap, yang bebas dipakai siapa pun setelah patennya berakhir.

Gagasannya tua. Kuliah menyebut kota Yunani Sybaris sekitar 500 sebelum Masehi, yang konon melindungi resep masakan baru selama setahun, monopoli sepuluh tahun bagi penemu cara menenun sutra di Venesia abad ke-12, undang-undang paten Venesia tahun 1474 yang sudah memuat prinsip-prinsip modern, yaitu baru, berguna, eksklusif, dan terbatas waktunya, Statute of Monopolies di Inggris tahun 1623, dan Konvensi Paris tahun 1883 yang menjadikan sistem paten internasional.

C.2.2 *Syarat dan siklus hidup*

Penemuan hanya bisa dipatenkan kalau memenuhi tiga syarat. Pertama, *baru secara mutlak*: belum menjadi bagian dari keadaan teknik sebelumnya di mana pun di dunia, dalam bentuk apa pun, termasuk makalah, presentasi konferensi, tesis, atau unggahan di internet. Penemuan yang pernah diterbitkan di Jepang tidak lagi baru di Eropa, walau hampir tidak ada orang Eropa yang membacanya. Kedua, *langkah inventif*: tidak boleh jelas bagi seorang ahli di bidangnya. Ketiga, *dapat diterapkan di industri*. Selain itu, penemuan harus diungkapkan cukup jelas sehingga ahli di bidangnya bisa melaksanakannya.

Syarat kebaruan punya akibat yang sering mengejutkan peneliti. Di Eropa, publikasi oleh penemunya sendiri juga merusak kebaruan. Peneliti yang mempresentasikan temuannya di konferensi minggu ini, lalu mengajukan paten bulan depan, sudah kehilangan haknya. Urutannya harus dibalik: ajukan dulu, terbitkan kemudian. Kuliah menyebut jalan pintas yang murah untuk itu: kantor paten Austria, dan beberapa kantor lain, menerima permohonan paten sementara dengan biaya resmi sekitar 50 euro. Permohonan ini tidak diperiksa dan tidak diterbitkan, tetapi mengamankan tanggal prioritas, dan dalam setahun bisa ditingkatkan menjadi permohonan yang lengkap.

Setelah diajukan, permohonan diterbitkan sekitar 18 bulan setelah tanggal prioritas, diperiksa oleh kantor paten, dan kalau diberikan, berlaku paling lama 20 tahun sejak tanggal pengajuan, asalkan biaya tahunan dibayar. Dalam tahun prioritas, permohonan yang sama bisa diajukan di negara-negara lain dengan tetap memakai tanggal prioritas yang pertama.

C.2.3 *Membaca paten*

Bagian terpenting sebuah paten adalah daftar klaim bernomor, karena klaimlah, bukan gambar atau uraian, yang menentukan apa yang dilindungi. Klaim independen berdiri sendiri, klaim dependen merujuk ke klaim lain dan menambahkan pembatasan. Satu kata kecil menentukan luasnya. Klaim yang memakai “comprising” bersifat terbuka: produk yang punya semua unsur yang disebut, ditambah unsur lain, tetap termasuk. Klaim yang memakai “consisting of” bersifat tertutup: hanya unsur yang disebut, tidak lebih. Kuliah menyarankan urutan membaca: halaman pertama untuk jenis dokumen, tanggal prioritas, dan abstrak, lalu gambar dan contoh kalau bidangnya belum dikenal, baru kemudian klaim.

Paten juga diklasifikasikan menurut bidang teknik, misalnya dengan Cooperative Patent Classification dan International Patent Classification, sehingga pencarian keadaan teknik sebelumnya bisa menyisir satu kelas sekaligus. Tugas kuliah meminta kami me-

nulis ulasan sebuah paten: data bibliografisnya, kelasnya, mengapa tanggal prioritasnya lebih awal dari tanggal pengajuannya, apa penemuannya, dan apa yang sebenarnya dilindungi klaimnya. Latihan ini mengajarkan bahwa uraian yang panjang di sebuah paten sering jauh lebih luas dari perlindungan yang diberikan klaim-klaimnya.

Ketika paten dilanggar, pemiliknya bisa menggugat. Kuliah membahas tiga cara menghitung ganti rugi: royalti yang wajar, yaitu nilai pasar sebuah lisensi; keuntungan yang hilang, yang harus dibuktikan sebagai akibat langsung pelanggaran; dan keuntungan yang diperoleh pelanggar. Ancaman gugatan inilah yang memberi paten “gigi”-nya.

C.2.4 Paten dan AI

AI menimbulkan dua pertanyaan baru bagi sistem paten. Yang pertama adalah apakah penemuan berbasis perangkat lunak dan machine learning bisa dipatenkan. Di Eropa, program komputer “sebagai program” dikecualikan, tetapi penemuan yang diimplementasikan dengan komputer bisa dipatenkan kalau ia memberi sumbangan teknis, misalnya metode machine learning yang dipakai untuk mengendalikan proses industri atau mengolah sinyal fisis dengan cara baru. Batasnya tidak selalu jelas, dan banyak keputusan bergantung pada rumusan klaim. Yang kedua adalah apakah AI bisa menjadi penemu. Dalam kasus DABUS, seorang pemohon mencantumkan sistem AI sebagai penemu satu-satunya. Kantor Paten Eropa menolaknya pada 2020, dewan bandingnya menguatkan penolakan itu pada 2021, dan Mahkamah Agung Inggris pada 20 Desember 2023 memutuskan dengan suara bulat bahwa penemu harus manusia. Pertanyaan tentang siapa pemilik penemuan yang sebagian besar dihasilkan AI, dan bagaimana menilai langkah inventif ketika “ahli di bidangnya” sudah dibantu AI, masih terbuka.

CATATAN SUMBER. Lampiran ini mengikuti kuliah Patent and Literature Searching (hak kekayaan intelektual dan sejarah paten, syarat paten, siklus hidup paten, membaca dan mencari paten, strategi klaim, sengketa paten, publikasi ilmiah dan evaluasinya, mencari literatur ilmiah) beserta tugas ulasan paten. Fakta tentang kasus DABUS diperiksa dengan laporan putusan resminya.

Bibliografi

- Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Ballard, Joshua Bambrick, Sebastian W. Bodenstein, David A. Evans, Chia-Chun Hung, Michael O'Neill, David Reiman, Kathryn Tunyasuvunakool, Zachary Wu, Akvilė Žemgulytė, Eirini Arvaniti, Charles Beattie, Ottavia Bertolli, Alex Bridgland, Alexey Cherepanov, Miles Congreve, Alexander I. Cowen-Rivers, Andrew Cowie, Michael Figurnov, Fabian B. Fuchs, Hannah Gladman, Rishub Jain, Yousuf A. Khan, Caroline M. R. Low, Kuba Perlin, Anna Potapenko, Pascal Savy, Sukhdeep Singh, Adrian Stecula, Ashok Thillaisundaram, Catherine Tong, Sergei Yakneen, Ellen D. Zhong, Michal Zielinski, Augustin Židek, Victor Bapst, Pushmeet Kohli, Max Jaderberg, Demis Hassabis, and John M. Jumper. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.
- Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. arXiv:1810.03292, 2018.
- Aishwarya Agrawal, Jiasen Lu, Stanislaw Antol, Margaret Mitchell, C. Lawrence Zitnick, Dhruv Batra, and Devi Parikh. Vqa: Visual question answering. arXiv:1505.00468, 2015.
- B. J. Alder and T. E. Wainwright. Studies in molecular dynamics. i. general method. *The Journal of Chemical Physics*, 31(2):459–466, 1959. doi: 10.1063/1.1730376.
- Benedikt Alkin, Maximilian Beck, Korbinian Pöppel, Sepp Hochreiter, and Johannes Brandstetter. Vision-lstm: xlstm as generic vision backbone. arXiv:2406.04303, 2024a.
- Benedikt Alkin, Andreas Fürst, Simon Schmid, Lukas Gruber, Markus Holzleitner, and Johannes Brandstetter. Universal physics transformers: A framework for efficiently scaling neural operators. arXiv:2402.12365, 2024b.
- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. arXiv:2006.05205, 2020.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. arXiv:1606.06565, 2016.
- Brian D.O. Anderson. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982. doi: 10.1016/0304-4149(82)90051-5.
- John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4): 1036–1060, 2004. doi: 10.1037/0033-295X.111.4.1036.
- Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. arXiv:2107.07511, 2021.

- F. J. Anscombe. Graphs in statistical analysis. *The American Statistician*, 27(1):17–21, 1973. doi: 10.1080/00031305.1973.10478966.
- Jose A. Arjona-Medina, Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. Rudder: Return decomposition for delayed rewards. arXiv:1806.07857, 2018.
- Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein gan. arXiv:1701.07875, 2017.
- Larry Armijo. Minimization of functions having lipschitz continuous first partial derivatives. *Pacific Journal of Mathematics*, 16(1):1–3, 1966. doi: 10.2140/pjm.1966.16.1.
- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. arXiv:2301.08243, 2023.
- Andreas Auer, Patrick Podest, Daniel Klotz, Sebastian Böck, Günter Klambauer, and Sepp Hochreiter. Tirez: Zero-shot forecasting across long and short horizons with enhanced in-context learning. arXiv:2505.23719, 2025.
- Žiga Avsec, Vikram Agarwal, Daniel Visentin, Joseph R. Ledsam, Agnieszka Grabska-Barwinska, Kyle R. Taylor, Yannis Assael, John Jumper, Pushmeet Kohli, and David R. Kelley. Effective gene expression prediction from sequence by integrating long-range interactions. *Nature Methods*, 18(10):1196–1203, 2021. doi: 10.1038/s41592-021-01252-x.
- Francis Bach. *Learning Theory from First Principles*. MIT Press, 2024.
- Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLOS ONE*, 10(7):e0130140, 2015. doi: 10.1371/journal.pone.0130140.
- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. arXiv:1409.0473, 2014.
- Ilyes Batatia, Dávid Péter Kovács, Gregor N. C. Simm, Christoph Ortner, and Gábor Csányi. Mace: Higher order equivariant message passing neural networks for fast and accurate force fields. arXiv:2206.07697, 2022.
- Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261, 2018.
- Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. arXiv:1502.05767, 2015. Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *The Journal of Machine Learning Research*, 18(153):1–43, 2018.

- Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. arXiv:2405.04517, 2024.
- Maximilian Beck, Korbinian Pöppel, Phillip Lippe, Richard Kurle, Patrick M. Blies, Günter Klambauer, Sebastian Böck, and Sepp Hochreiter. xlstm 7b: A recurrent llm for fast and efficient inference. arXiv:2503.13427, 2025.
- Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019. doi: 10.1073/pnas.1903070116.
- Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. Unifying count-based exploration and intrinsic motivation. arXiv:1606.01868, 2016.
- Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots. In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pages 610–623, 2021. doi: 10.1145/3442188.3445922.
- Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.
- Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- Yoshua Bengio, Sören Mindermann, Daniel Privitera, Tamay Besiroglu, Rishi Bommasani, Stephen Casper, Yejin Choi, Philip Fox, Ben Garfinkel, Danielle Goldfarb, Hoda Heidari, Anson Ho, Sayash Kapoor, Leila Khalatbari, Shayne Longpre, Sam Manning, Vasilios Mavroudis, Mantas Mazeika, Julian Michael, Jessica Newman, Kwan Yee Ng, Chinasa T. Okolo, Deborah Raji, Girish Sastry, Elizabeth Seger, Theodora Skeadas, Tobin South, Emma Strubell, Florian Tramèr, Lucia Velasco, Nicole Wheeler, Daron Acemoglu, Olubayo Adekanmbi, David Dalrymple, Thomas G. Dietterich, Edward W. Felten, Pascale Fung, Pierre-Olivier Gourinchas, Fredrik Heintz, Geoffrey Hinton, Nick Jennings, Andreas Krause, Susan Leavy, Percy Liang, Teresa Ludermir, Vidushi Marda, Helen Margetts, John McDermid, Jane Munga, Arvind Narayanan, Alondra Nelson, Clara Neppel, Alice Oh, Gopal Ramchurn, Stuart Russell, Marietje Schaake, Bernhard Schölkopf, Dawn Song, Alvaro Soto, Lee Tiedrich, Gaël Varoquaux, Andrew Yao, Ya-Qin Zhang, Fahad Albalawi, Marwan Alserkal, Olubunmi Ajala, Guillaume Avrin, Christian Busch, André Carlos Ponce de Leon Ferreira de Carvalho, Bronwyn Fox, Amandeep Singh Gill, Ahmet Halit Hatip, Juha Heikkilä, Gill Jolly, Ziv Katzir, Hiroaki Kitano, Antonio Krüger, Chris Johnson, Saif M. Khan, Kyoung Mu Lee, Dominic Vincent Ligtot, Oleksii Molchanovskyi, Andrea Monti, Nusu Mwamanzi, Mona Nemer, Nuria Oliver, José Ramón López Portillo, Balaraman Ravindran, Raquel Pezoa Rivera, Hammam Riza, Crystal Rugege, Ciarán Seoighe, Jerry Sheehan, Haroon Sheikh, Denise Wong, and Yi Zeng. International ai safety report. arXiv:2501.17805,

- 2025.
- Ruha Benjamin. *Race After Technology: Abolitionist Tools for the New Jim Code*. Polity, 2019.
- James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- G Berkooz, P Holmes, and J L Lumley. The proper orthogonal decomposition in the analysis of turbulent flows. *Annual Review of Fluid Mechanics*, 25(1):539–575, 1993. doi: 10.1146/annurev.fl.25.010193.002543.
- P.J. Besl and Neil D. McKay. A method for registration of 3-d shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2):239–256, 1992. doi: 10.1109/34.121791.
- Deniz A. Bezin, Aaron B. Buhendwa, and Nikolaus A. Adams. Jax-fluids: A fully-differentiable high-order computational fluid dynamics solver for compressible two-phase flows. arXiv:2203.13760, 2022.
- P. Biber and W. Strasser. The normal distributions transform: a new approach to laser scan matching. In *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453)*, volume 3, pages 2743–2748, 2004. doi: 10.1109/IROS.2003.1249285.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control. arXiv:2410.24164, 2024.
- Cristian Bodnar, Wessel P. Bruinsma, Ana Lucic, Megan Stanley, Anna Vaughan, Johannes Brandstetter, Patrick Garvan, Maik Riechert, Jonathan A. Weyn, Haiyu Dong, Jayesh K. Gupta, Kit Thambiratnam, Alexander T. Archibald, Chun-Chieh Wu, Elizabeth Heider, Max Welling, Richard E. Turner, and Paris Perdikaris. A foundation model for the earth system. arXiv:2405.13063, 2024.
- Tolga Bolukbasi, Kai-Wei Chang, James Zou, Venkatesh Saligrama, and Adam Kalai. Man is to computer programmer as woman is to homemaker? debiasing word embeddings. arXiv:1607.06520, 2016.
- Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Dora Demszky, Chris Donahue, Moussa Doumbouya, Esin Durmus, Stefano Ermon, John Etchemendy, Kawin Ethayarajh, Li Fei-Fei, Chelsea Finn, Trevor Gale, Lauren Gillespie, Karan Goel, Noah Goodman, Shelby Grossman, Neel Guha, Tatsunori Hashimoto, Peter Henderson, John Hewitt, Daniel E. Ho, Jenny Hong, Kyle Hsu, Jing Huang, Thomas Icard, Saahil Jain, Dan Jurafsky, Pratyusha Kalluri, Siddharth Karamcheti, Geoff Keeling, Fereshte Khani, Omar Khattab, Pang Wei Koh, Mark Krass, Ranjay Krishna, Rohith Kuditipudi,

- Ananya Kumar, Faisal Ladhak, Mina Lee, Tony Lee, Jure Leskovec, Isabelle Levent, Xiang Lisa Li, Xuechen Li, Tengyu Ma, Ali Malik, Christopher D. Manning, Suvir Mirchandani, Eric Mitchell, Zanele Munyikwa, Suraj Nair, Avani Narayan, Deepak Narayanan, Ben Newman, Allen Nie, Juan Carlos Niebles, Hamed Nilforoshan, Julian Nyarko, Giray Ogut, Laurel Orr, Isabel Papadimitriou, Joon Sung Park, Chris Piech, Eva Portelance, Christopher Potts, Aditi Raghunathan, Rob Reich, Hongyu Ren, Frieda Rong, Yusuf Roohani, Camilo Ruiz, Jack Ryan, Christopher Ré, Dorsa Sadigh, Shiori Sagawa, Keshav Santhanam, Andy Shih, Krishnan Srinivasan, Alex Tamkin, Rohan Taori, Armin W. Thomas, Florian Tramèr, Rose E. Wang, William Wang, Bohan Wu, Jiajun Wu, Yuhuai Wu, Sang Michael Xie, Michihiro Yasunaga, Jiaxuan You, Matei Zaharia, Michael Zhang, Tianyi Zhang, Xikun Zhang, Yuhui Zhang, Lucia Zheng, Kaitlyn Zhou, and Percy Liang. On the opportunities and risks of foundation models. arXiv:2108.07258, 2021.
- Nick Bostrom. *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press, 2014. ISBN 9780199678112.
- Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018. doi: 10.1137/16M1080173.
- Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Chemcrow: Augmenting large-language models with chemistry tools. arXiv:2304.05376, 2023.
- Johannes Brandstetter, Daniel Worrall, and Max Welling. Message passing neural pde solvers. arXiv:2202.03376, 2022.
- Johann Brehmer, Pim de Haan, Sönke Behrends, and Taco Cohen. Geometric algebra transformer. arXiv:2305.18415, 2023.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control. arXiv:2307.15818, 2023.
- Michael M. Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: Going beyond euclidean data. *IEEE Signal Processing Magazine*, 34(4):18–42, 2017. doi: 10.1109/MSP.2017.2693418.
- Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep

- learning: Grids, groups, graphs, geodesics, and gauges. arXiv:2104.13478, 2021.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. arXiv:2005.14165, 2020.
- Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. Spectral networks and locally connected networks on graphs. arXiv:1312.6203, 2013.
- Steven L. Brunton and J. Nathan Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, 2 edition, 2022.
- Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 113(15):3932–3937, 2016. doi: 10.1073/pnas.1517384113.
- Steven L. Brunton, Bernd R. Noack, and Petros Koumoutsakos. Machine learning for fluid mechanics. *Annual Review of Fluid Mechanics*, 52(1):477–508, 2020. doi: 10.1146/annurev-fluid-010719-060214.
- Andreas Bulling, Ulf Blanke, and Bernt Schiele. A tutorial on human activity recognition using body-worn inertial sensors. *ACM Computing Surveys*, 46(3):1–33, 2014. doi: 10.1145/2499621.
- John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, 1986. doi: 10.1109/TPAMI.1986.4767851.
- Kathleen Champion, Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Data-driven discovery of coordinates and governing equations. *Proceedings of the National Academy of Sciences*, 116(45):22445–22451, 2019. doi: 10.1073/pnas.1906995116.
- Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. arXiv:2106.01345, 2021.
- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. arXiv:1806.07366, 2018.
- Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. arXiv:2002.05709, 2020.
- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms. arXiv:2302.06675, 2023.
- Jun Cheng, Guido Novati, Joshua Pan, Clare Bycroft, Akvilė Žemgulytė, Taylor Applebaum, Alexander Pritzel, Lai Hong Wong, Michal Zielinski, Tobias Sargeant, Rosalia G. Schneider, Andrew W. Senior, John Jumper, Demis Hassabis, Pushmeet

- Kohli, and Žiga Avsec. Accurate proteome-wide missense variant effect prediction with alphamissense. *Science*, 381(6664), 2023. doi: 10.1126/science.adg7492.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. arXiv:2303.04137, 2023.
- Alexandra Chouldechova. Fair prediction with disparate impact: A study of bias in recidivism prediction instruments. *Big Data*, 5(2):153–163, 2017. doi: 10.1089/big.2016.0047.
- Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. arXiv:1706.03741, 2017.
- Jeremy M. Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter, and Ameet Talwalkar. Gradient descent on neural networks typically occurs at the edge of stability. arXiv:2103.00065, 2021.
- Taco S. Cohen and Max Welling. Group equivariant convolutional networks. arXiv:1602.07576, 2016. Proceedings of the International Conference on Machine Learning (ICML), 2016.
- Open X-Embodiment Collaboration, Abby O’Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, Andrew Wang, Andrey Kolobov, Anikait Singh, Animesh Garg, Aniruddha Kembhavi, Annie Xie, Anthony Brohan, Antonin Raffin, Archit Sharma, Arefeh Yavary, Arhan Jain, Ashwin Balakrishna, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Blake Wulfe, Brian Ichter, Cewu Lu, Charles Xu, Charlotte Le, Chelsea Finn, Chen Wang, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Christopher Agia, Chuer Pan, Chuyuan Fu, Coline Devin, Danfei Xu, Daniel Morton, Danny Driess, Daphne Chen, Deepak Pathak, Dhruv Shah, Dieter Buehler, Dinesh Jayaraman, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Ethan Foster, Fangchen Liu, Federico Ceola, Fei Xia, Feiyu Zhao, Felipe Vieira Frujeri, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Gilbert Feng, Giulio Schiavi, Glen Berseth, Gregory Kahn, Guangwen Yang, Guanzhi Wang, Hao Su, Hao-Shu Fang, Haochen Shi, Henghui Bao, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homanga Bharadhwaj, Homer Walke, Hongjie Fang, Huy Ha, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jad Abou-Chakra, Jaehyung Kim, Jaimyn Drake, Jan Peters, Jan Schneider, Jasmine Hsu, Jay Vakil, Jeannette Bohg, Jeffrey Bingham, Jeffrey Wu, Jensen Gao, Jiaheng Hu, Jiajun Wu, Jialin Wu, Jiankai Sun, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jimmy Wu, Jingpei Lu, Jingyun Yang, Jitendra Malik, João Silvério, Joey Hejna, Jonathan Bozher, Jonathan Tompson, Jonathan Yang, Jordi Salvador, Joseph J. Lim, Junhyek Han, Kaiyuan Wang, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Black, Kevin Lin, Kevin Zhang, Kiana Ehsani, Kiran Lekkala, Kirsty Ellis, Krishan Rana, Krishnan Srinivasan,

- Kuan Fang, Kunal Pratap Singh, Kuo-Hao Zeng, Kyle Hatch, Kyle Hsu, Laurent Itti, Lawrence Yunliang Chen, Lerrel Pinto, Li Fei-Fei, Liam Tan, Linxi "Jim" Fan, Lionel Ott, Lisa Lee, Luca Weihs, Magnum Chen, Marion Lepert, Marius Memmel, Masayoshi Tomizuka, Masha Itkina, Mateo Guaman Castro, Max Spero, Maximilian Du, Michael Ahn, Michael C. Yip, Mingtong Zhang, Mingyu Ding, Minh Heo, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Muhammad Zubair Irshad, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Ning Liu, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Osbert Bastani, Pannag R Sanketi, Patrick "Tree" Miller, Patrick Yin, Paul Wohlhart, Peng Xu, Peter David Fagan, Peter Mitrano, Pierre Sermanet, Pieter Abbeel, Priya Sundareshan, Qiuyu Chen, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Martín-Martín, Rohan Bajjal, Rosario Scalise, Rose Hendrix, Roy Lin, Runjia Qian, Ruohan Zhang, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Shan Lin, Sherry Moore, Shikhar Bahl, Shivin Dass, Shubham Sonawani, Shubham Tulsiani, Shuran Song, Sichun Xu, Siddhant Haldar, Siddharth Karamcheti, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Subramanian Ramamoorthy, Sudeep Dasari, Suneel Belkhale, Sungjae Park, Suraj Nair, Suvir Mirchandani, Takayuki Osa, Tanmay Gupta, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Thomas Kollar, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Trinity Chung, Vidhi Jain, Vikash Kumar, Vincent Vanhoucke, Vitor Guizilini, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiangyu Chen, Xiaolong Wang, Xinghao Zhu, Xinyang Geng, Xiyuan Liu, Xu Liangwei, Xuanlin Li, Yansong Pang, Yao Lu, Yecheng Jason Ma, Yejin Kim, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Yilin Wu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yongqiang Dou, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yue Cao, Yueh-Hua Wu, Yujin Tang, Yuke Zhu, Yunchu Zhang, Yunfan Jiang, Yunshuang Li, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zehan Ma, Zhuo Xu, Zichen Jeff Cui, Zichen Zhang, Zipeng Fu, and Zipeng Lin. Open x-embodiment: Robotic learning datasets and rt-x models. *arXiv:2310.08864*, 2023.
- D. Comaniciu and P. Meer. Mean shift: a robust approach toward feature space analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5):603–619, 2002. doi: 10.1109/34.1000236.
- James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.
- Gabriele Corso, Hannes Stärk, Bowen Jing, Regina Barzilay, and Tommi Jaakkola. Diffdock: Diffusion steps, twists, and turns for molecular docking. *arXiv:2210.01776*, 2022.
- Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3): 273–297, 1995. doi: 10.1007/BF00994018.
- R. Courant, K. Friedrichs, and H. Lewy. Über die partiellen differenzengleichungen der mathematischen physik. *Mathematische Annalen*, 100(1):32–74, 1928. doi: 10.1007/BF

01448839.

- Kyle Cranmer, Johann Brehmer, and Gilles Louppe. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, 117(48):30055–30062, 2020. doi: 10.1073/pnas.1912789117.
- Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scgpt: toward building a foundation model for single-cell multi-omics using generative ai. *Nature Methods*, 21(8):1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.
- P. A. Cundall and O. D. L. Strack. A discrete numerical model for granular assemblies. *Géotechnique*, 29(1):47–65, 1979. doi: 10.1680/geot.1979.29.1.47.
- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. arXiv:2309.08600, 2023.
- Salvatore Cuomo, Vincenzo Schiano Di Cola, Fabio Giampaolo, Gianluigi Rozza, Maziar Raissi, and Francesco Piccialli. Scientific machine learning through physics-informed neural networks: Where we are and what’s next. *Journal of Scientific Computing*, 92(3), 2022. doi: 10.1007/s10915-022-01939-z.
- G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989. doi: 10.1007/BF02551274.
- J. Dauparas, I. Anishchenko, N. Bennett, H. Bai, R. J. Ragotte, L. F. Milles, B. I. M. Wicky, A. Courbet, R. J. de Haas, N. Bethel, P. J. Y. Leung, T. F. Huddy, S. Pellock, D. Tischer, F. Chan, B. Koepnick, H. Nguyen, A. Kang, B. Sankaran, A. K. Bera, N. P. King, and D. Baker. Robust deep learning-based protein sequence design using proteinmpnn. *Science*, 378(6615):49–56, 2022. doi: 10.1126/science.add2187.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin

- Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report. arXiv:2412.19437, 2024.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jia Shi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang,

- Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948, 2025. Nature volume 645, pages 633-638 (2025).
- Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. arXiv:2405.15682, 2024.
- Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. arXiv:1606.09375, 2016. Advances in Neural Information Processing Systems 29 (2016).
- Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the em algorithm. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 39(1):1–22, 1977. doi: 10.1111/j.2517-6161.1977.tb01600.x.
- J. Denavit and R. S. Hartenberg. A kinematic notation for lower-pair mechanisms based on matrices. *Journal of Applied Mechanics*, 22(2):215–221, 1955. doi: 10.1115/1.4011045.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805, 2018.
- E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959. doi: 10.1007/BF01386390.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv:2010.11929, 2020.
- Yaxing Du, Bert Blocken, and Stefan Pirker. A novel approach to simulate pollutant dispersion in the built environment: Transport-based recurrence cfd. *Building and Environment*, 170:106604, 2020. doi: 10.1016/j.buildenv.2019.106604.
- Richard O. Duda, Peter E. Hart, and David G. Stork. *Pattern Classification*. Wiley, 2 edition, 2001.
- Karthik Duraisamy, Gianluca Iaccarino, and Heng Xiao. Turbulence modeling in the age of data. *Annual Review of Fluid Mechanics*, 51(1):357–377, 2019. doi: 10.1146/annurev-fluid-010518-040547.
- Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998.
- Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. *Lecture Notes in Computer Science*, pages 265–284, 2006. doi: 10.1007/11681878_14.
- Weinan E and Bing Yu. The deep ritz method: A deep learning-based numerical algorithm for solving variational problems. arXiv:1710.00211, 2017.

- Tyna Eloundou, Sam Manning, Pamela Mishkin, and Daniel Rock. Gpts are gpts: An early look at the labor market impact potential of large language models. arXiv:2303.10130, 2023.
- Nicholas Epley, Adam Waytz, and John T. Cacioppo. On seeing human: A three-factor theory of anthropomorphism. *Psychological Review*, 114(4):864–886, 2007. doi: 10.1037/0033-295X.114.4.864.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. Scaling rectified flow transformers for high-resolution image synthesis. arXiv:2403.03206, 2024.
- Andre Esteva, Brett Kuprel, Roberto A. Novoa, Justin Ko, Susan M. Swetter, Helen M. Blau, and Sebastian Thrun. Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542(7639):115–118, 2017. doi: 10.1038/nature21056.
- European Parliament and Council of the European Union. Regulation (eu) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (general data protection regulation). Official Journal of the European Union, L 119, 2016.
- European Parliament and Council of the European Union. Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, 12 July 2024, 2024.
- Geir Evensen. Sequential data assimilation with a nonlinear quasi-geostrophic model using monte carlo methods to forecast error statistics. *Journal of Geophysical Research: Oceans*, 99(C5):10143–10162, 1994. doi: 10.1029/94JC00572.
- Kevin Eykholt, Ivan Evtimov, Earlene Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. Robust physical-world attacks on deep learning models. arXiv:1707.08945, 2017.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. arXiv:2101.03961, 2021.
- Joel H. Ferziger, Milovan Perić, and Robert L. Street. *Computational Methods for Fluid Dynamics*. Springer, 4 edition, 2020. doi: 10.1007/978-3-319-99693-6.
- Richard E. Fikes and Nils J. Nilsson. Strips: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971. doi: 10.1016/0004-3702(71)90010-5.
- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. arXiv:1703.03400, 2017.
- Philipp Fischer, Alexey Dosovitskiy, Eddy Ilg, Philip Häusser, Caner Hazırbaş, Vladimir Golkov, Patrick van der Smagt, Daniel Cremers, and Thomas Brox. Flownet: Learning optical flow with convolutional networks. arXiv:1504.06852, 2015.
- Martin A. Fischler and Robert C. Bolles. Random sample consensus. *Communications of the ACM*, 24(6):381–395, 1981. doi: 10.1145/358669.358692.

- R. A. FISHER. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936. doi: 10.1111/j.1469-1809.1936.tb02137.x.
- Daan Frenkel and Berend Smit. *Understanding Molecular Simulation: From Algorithms to Applications*. Academic Press, 2 edition, 2002.
- Andreas Fürst, Elisabeth Rumetshofer, Johannes Lehner, Viet Tran, Fei Tang, Hubert Ramsauer, David Kreil, Michael Kopp, Günter Klambauer, Angela Bitto-Nemling, and Sepp Hochreiter. Cloob: Modern hopfield networks with infoloob outperform clip. arXiv:2110.11316, 2021.
- Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. arXiv:1506.02142, 2015.
- Paul Garnier, Jonathan Viquerat, Jean Rabault, Aurélien Larcher, Alexander Kuhnle, and Elie Hachem. A review on deep reinforcement learning for fluid mechanics. *Computers & Fluids*, 225:104973, 2021. doi: 10.1016/j.compfluid.2021.104973.
- A. Gaulton, L. J. Bellis, A. P. Bento, J. Chambers, M. Davies, A. Hersey, Y. Light, S. McGlinchey, D. Michalovich, B. Al-Lazikani, and J. P. Overington. ChEMBL: a large-scale bioactivity database for drug discovery. *Nucleic Acids Research*, 40(D1): D1100–D1107, 2012. doi: 10.1093/nar/gkr777.
- Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, 1992. doi: 10.1162/neco.1992.4.1.1.
- Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. *Neural Computation*, 12(10):2451–2471, 2000. doi: 10.1162/089976600300015015.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. arXiv:1704.01212, 2017.
- R. A. Gingold and J. J. Monaghan. Smoothed particle hydrodynamics: theory and application to non-spherical stars. *Monthly Notices of the Royal Astronomical Society*, 181(3):375–389, 1977. doi: 10.1093/mnras/181.3.375.
- Ross Girshick. Fast r-cnn. arXiv:1504.08083, 2015.
- Fred Glover. Tabu search—part i. *ORSA Journal on Computing*, 1(3):190–206, 1989. doi: 10.1287/ijoc.1.3.190.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. arXiv:1406.2661, 2014.
- Sam Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. arXiv:1906.01563, 2019.
- Amos Gropp, Lior Yariv, Niv Haim, Matan Atzmon, and Yaron Lipman. Implicit geometric regularization for learning shapes. arXiv:2002.10099, 2020.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state

- spaces. arXiv:2312.00752, 2023.
- Jayesh K. Gupta and Johannes Brandstetter. Towards multi-spatiotemporal-scale generalized pde modeling. arXiv:2209.15616, 2022.
- Rafael Gómez-Bombarelli, Jennifer N. Wei, David Duvenaud, José Miguel Hernández-Lobato, Benjamín Sánchez-Lengeling, Dennis Sheberla, Jorge Aguilera-Iparraguirre, Timothy D. Hirzel, Ryan P. Adams, and Alán Aspuru-Guzik. Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2):268–276, 2018. doi: 10.1021/acscentsci.7b00572.
- David Ha and Jürgen Schmidhuber. World models. arXiv:1803.10122, 2018.
- Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. arXiv:1801.01290, 2018.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. arXiv:2301.04104, 2023.
- Zhongkai Hao, Chang Su, Songming Liu, Julius Berner, Chengyang Ying, Hang Su, Anima Anandkumar, Jian Song, and Jun Zhu. Dpot: Auto-regressive denoising operator transformer for large-scale pde pre-training. arXiv:2403.03542, 2024.
- Moritz Hardt, Eric Price, and Nathan Srebro. Equality of opportunity in supervised learning. arXiv:1610.02413, 2016.
- Peter Hart, Nils Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2004.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2 edition, 2009.
- W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 1970. doi: 10.1093/biomet/57.1.97.
- Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, 2019. doi: 10.1038/s41586-019-0980-2.
- Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025. doi: 10.1126/science.ads0018.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. arXiv:1502.01852,

- 2015.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. doi: 10.1109/CVPR.2016.90.
- Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. arXiv:2111.06377, 2021.
- Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering. arXiv:1708.05031, 2017.
- Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. arXiv:2002.02126, 2020.
- Dan Hendrycks, Mantas Mazeika, and Thomas Woodside. An overview of catastrophic ai risks. arXiv:2306.12001, 2023.
- Maximilian Herde, Bogdan Raonić, Tobias Rohner, Roger Käppeli, Roberto Molinaro, Emmanuel de Bézenac, and Siddhartha Mishra. Poseidon: Efficient foundation models for pdes. arXiv:2405.19101, 2024.
- Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. arXiv:1706.08500, 2017. *Advances in Neural Information Processing Systems* 30 (NIPS 2017).
- Lukas Hewing, Kim P. Wabersich, Marcel Menner, and Melanie N. Zeilinger. Learning-based model predictive control: Toward safe learning in control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3(1):269–296, 2020. doi: 10.1146/annurev-control-090419-075625.
- High-Level Expert Group on Artificial Intelligence. A definition of AI: Main capabilities and disciplines. European Commission, 2019.
- Frederick S. Hillier and Gerald J. Lieberman. *Introduction to Operations Research*. McGraw-Hill, 10 edition, 2015.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.
- Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. arXiv:1207.0580, 2012.
- C.W Hirt and B.D Nichols. Volume of fluid (vof) method for the dynamics of free boundaries. *Journal of Computational Physics*, 39(1):201–225, 1981. doi: 10.1016/0021-991(81)90145-5.
- Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. arXiv:1606.03476, 2016.
- Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022.

- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. arXiv:2006.11239, 2020.
- Sepp Hochreiter. Untersuchungen zu dynamischen neuronalen netzen. Diploma thesis, Institut für Informatik, Technische Universität München, 1991.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997a. doi: 10.1162/neco.1997.9.8.1735.
- Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural Computation*, 9(1):1–42, 1997b. doi: 10.1162/neco.1997.9.1.1.
- Sepp Hochreiter, Ulrich Bodenhofer, Martin Heusel, Andreas Mayr, Andreas Mitterecker, Adetayo Kasim, Tatsiana Khamiakova, Suzy Van Sanden, Dan Lin, Willem Talloen, Luc Bijnens, Hinrich W. H. Göhlmann, Ziv Shkedy, and Djork-Arné Clevert. Fabia: factor analysis for bicluster acquisition. *Bioinformatics*, 26(12):1520–1527, 2010. doi: 10.1093/bioinformatics/btq227.
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training compute-optimal large language models. arXiv:2203.15556, 2022.
- Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeyer, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.
- Hamed Hoorijani. Online recurrence-cfd: enhancing multiphase flow simulation performance with online recurrence analysis. *Meccanica*, 60(6):1893–1913, 2025. doi: 10.1007/s11012-024-01763-9.
- Berthold K.P. Horn and Brian G. Schunck. Determining optical flow. *Artificial Intelligence*, 17(1–3):185–203, 1981. doi: 10.1016/0004-3702(81)90024-2.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989. doi: 10.1016/0893-6080(89)90020-8.
- H. Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441, 1933. doi: 10.1037/h0071325.
- G. N. Hounsfield. Computerized transverse axial scanning (tomography): Part 1. description of system. *The British Journal of Radiology*, 46(552):1016–1022, 1973. doi: 10.1259/0007-1285-46-552-1016.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.
- Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *2008 Eighth IEEE International Conference on Data Mining*, pages 263–272,

2008. doi: 10.1109/ICDM.2008.22.
- A. Hyvärinen and E. Oja. Independent component analysis: algorithms and applications. *Neural Networks*, 13(4–5):411–430, 2000. doi: 10.1016/S0893-6080(00)00026-5.
- International AI Safety Report. International ai safety report 2026. Technical report, Chaired by Yoshua Bengio, 2026. URL <https://internationalaisafetyreport.org/publication/international-ai-safety-report-2026>.
- International Energy Agency. Energy and ai. Technical report, IEA, Paris, 2025. URL <https://www.iea.org/reports/energy-and-ai>.
- Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv:1502.03167, 2015.
- Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. nnu-net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2):203–211, 2021. doi: 10.1038/s41592-020-01008-z.
- Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. arXiv:1806.07572, 2018. In *Advances in neural information processing systems* (pp. 8571–8580) 2018.
- Ameya D. Jagtap and George Em Karniadakis. Extended physics-informed neural networks (xpinns): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. *Communications in Computational Physics*, 28(5):2002–2041, 2020. doi: 10.4208/cicp.OA-2020-0164.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? arXiv:2310.06770, 2023.
- John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.
- Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.
- Avinash C. Kak and Malcolm Slaney. *Principles of Computerized Tomographic Imaging*. IEEE Press, 1988.
- R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.
- Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. arXiv:1808.09781, 2018.

- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.
- George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi: 10.1038/s42254-021-00314-5.
- Steven M. Kay. *Fundamentals of Statistical Signal Processing: Estimation Theory*. Prentice Hall, 1993.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023. doi: 10.1145/3592433.
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv:1609.04836*, 2016.
- Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas, and Rory Sayres. Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). *arXiv:1711.11279*, 2017. ICML 2018.
- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv:1412.6980*, 2014.
- Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv:1312.6114*, 2013.
- Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *arXiv:1906.02691*, 2019. *Foundations and Trends in Machine Learning: Vol. 12 (2019): No. 4*, pp 307-392.
- Diederik P. Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. Variational diffusion models. *arXiv:2107.00630*, 2021.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv:1609.02907*, 2016.
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models. *arXiv:2301.10226*, 2023.
- Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. *arXiv:2304.02643*, 2023.
- S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.
- Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks. *arXiv:1706.02515*, 2017. *Advances in Neural Information Processing Systems 30 (NIPS 2017)*.
- Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4):293–326, 1975. doi: 10.1016/0004-3702(75)90019-3.

- Dmitrii Kochkov, Jamie A. Smith, Ayya Alieva, Qing Wang, Michael P. Brenner, and Stephan Hoyer. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21), 2021. doi: 10.1073/pnas.2101784118.
- Dmitrii Kochkov, Janni Yuval, Ian Langmore, Peter Norgaard, Jamie Smith, Griffin Mooers, Milan Klöwer, James Lottes, Stephan Rasp, Peter Düben, Sam Hatfield, Peter Battaglia, Alvaro Sanchez-Gonzalez, Matthew Willson, Michael P. Brenner, and Stephan Hoyer. Neural general circulation models for weather and climate. *Nature*, 632(8027):1060–1066, 2024. doi: 10.1038/s41586-024-07744-y.
- Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. *Lecture Notes in Computer Science*, pages 282–293, 2006. doi: 10.1007/11871842_29.
- Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. Concept bottleneck models. arXiv:2007.04612, 2020.
- Georg Kohl, Li-Wei Chen, and Nils Thuerey. Benchmarking autoregressive conditional diffusion models for turbulent flow simulation. arXiv:2309.01745, 2023.
- Teuvo Kohonen. Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1):59–69, 1982. doi: 10.1007/BF00337288.
- Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.
- B. O. Koopman. Hamiltonian systems and transformation in hilbert space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, 1931. doi: 10.1073/pnas.17.5.315.
- Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. doi: 10.1109/MC.2009.263.
- Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces. arXiv:2108.08481, 2021. *The Journal of Machine Learning Research* (2023), Volume 24, Issue 1, Article No 89, pp 4061-4157.
- Boris Kramer, Benjamin Peherstorfer, and Karen E. Willcox. Learning nonlinear reduced models from data with operator inference. *Annual Review of Fluid Mechanics*, 56(1): 521–548, 2024. doi: 10.1146/annurev-fluid-121021-025220.
- Aditi S. Krishnapriyan, Amir Gholami, Shandian Zhe, Robert M. Kirby, and Michael W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. arXiv:2109.01050, 2021. NeurIPS 2021.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60(6):84–90, 2017. doi: 10.1145/3065386.
- H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955. doi: 10.1002/nav.3800020109.
- Indrajit Kurmi, David C. Schedl, and Oliver Bimber. Airborne optical sectioning. *Journal of Imaging*, 4(8):102, 2018. doi: 10.3390/jimaging4080102.
- I.E. Lagaris, A. Likas, and D.I. Fotiadis. Artificial neural networks for solving ordinary

- and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998. doi: 10.1109/72.712178.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. arXiv:1612.01474, 2016.
- Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirsnberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, Alexander Merose, Stephan Hoyer, George Holland, Oriol Vinyals, Jacklynn Stott, Alexander Pritzel, Shakir Mohamed, and Peter Battaglia. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023. doi: 10.1126/science.adi2336.
- Alex H. Lang, Sourabh Vora, Holger Caesar, Lubing Zhou, Jiong Yang, and Oscar Beijbom. Pointpillars: Fast encoders for object detection from point clouds. arXiv:1812.05784, 2018.
- B.E. Launder and D.B. Spalding. The numerical computation of turbulent flows. *Computer Methods in Applied Mechanics and Engineering*, 3(2):269–289, 1974. doi: 10.1016/0045-7825(74)90029-2.
- Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.
- Marc Levoy and Pat Hanrahan. Light field rendering. In *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*, pages 31–42, 1996. doi: 10.1145/237170.237199.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv:2005.11401, 2020.
- Heng Li and Richard Durbin. Fast and accurate short read alignment with burrows-wheeler transform. *Bioinformatics*, 25(14):1754–1760, 2009. doi: 10.1093/bioinformatics/btp324.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhat-tacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. arXiv:2010.08895, 2020.
- Yi-Lun Liao and Tess Smidt. Equiformer: Equivariant graph attention transformer for 3d atomistic graphs. arXiv:2206.11990, 2022.
- T. Lichtenegger and S. Pirker. Recurrence cfd – a novel approach to simulate multiphase flows with strongly separated time scales. *Chemical Engineering Science*, 153:394–410, 2016. doi: 10.1016/j.ces.2016.07.036.
- Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam

- Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.
- Julia Ling, Andrew Kurzawski, and Jeremy Templeton. Reynolds averaged turbulence modelling using deep neural networks with embedded invariance. *Journal of Fluid Mechanics*, 807:155–166, 2016. doi: 10.1017/jfm.2016.615.
- Mario Lino, Stathi Fotiadis, Anil A. Bharath, and Chris D. Cantwell. Current and emerging deep-learning methods for the simulation of fluid dynamics. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 479(2275), 2023. doi: 10.1098/rspa.2023.0058.
- Christopher A. Lipinski, Franco Lombardo, Beryl W. Dominy, and Paul J. Feeney. Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 23(1–3): 3–25, 1997. doi: 10.1016/S0169-409X(96)00423-1.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. arXiv:2210.02747, 2022.
- Phillip Lippe, Bastiaan S. Veeling, Paris Perdikaris, Richard E. Turner, and Johannes Brandstetter. Pde-refiner: Achieving accurate long rollouts with neural pde solvers. arXiv:2308.05732, 2023.
- Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training. arXiv:2502.16982, 2025.
- Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. arXiv:2209.03003, 2022.
- Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. Kan: Kolmogorov-arnold networks. arXiv:2404.19756, 2024.
- S. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489.
- Duri Long and Brian Magerko. What is ai literacy? competencies and design considerations. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*, pages 1–16, 2020. doi: 10.1145/3313831.3376727.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv:1711.05101, 2017.
- Michael I Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with deseq2. *Genome Biology*, 15(12), 2014. doi: 10.1186/s13059-014-0550-8.
- David G. Lowe. Distinctive image features from scale-invariant keypoints. *International*

- Journal of Computer Vision*, 60(2):91–110, 2004. doi: 10.1023/B:VISI.0000029664.99615.94.
- Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi: 10.1038/s42256-021-00302-5.
- H. Lumetzberger, S. Pirker, and T. Lichtenegger. Propagator-moments approximation for recurrence cfd: Application to species transport in turbulent flows. *Chemical Engineering Science*, 311:121624, 2025. doi: 10.1016/j.ces.2025.121624.
- Scott Lundberg and Su-In Lee. A unified approach to interpreting model predictions. arXiv:1705.07874, 2017.
- Calvin Luo. Understanding diffusion models: A unified perspective. arXiv:2208.11970, 2022.
- Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9(1), 2018. doi: 10.1038/s41467-018-07210-0.
- Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.
- Jun Ma, Yuting He, Feifei Li, Lin Han, Chenyu You, and Bo Wang. Segment anything in medical images. *Nature Communications*, 15(1), 2024. doi: 10.1038/s41467-024-44824-z.
- David J. C. MacKay. Bayesian interpolation. *Neural Computation*, 4(3):415–447, 1992. doi: 10.1162/neco.1992.4.3.415.
- Stéphane Mallat. Group invariant scattering. *Communications on Pure and Applied Mathematics*, 65(10):1331–1398, 2012. doi: 10.1002/cpa.21413.
- Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, Thomas Köppe, Kevin Millikin, Stephen Gaffney, Sophie Elster, Jackson Broshear, Chris Gamble, Kieran Milan, Robert Tung, Minjae Hwang, Taylan Cemgil, Mohammadamin Barekatain, Yujia Li, Amol Mandhane, Thomas Hubert, Julian Schrittwieser, Demis Hassabis, Pushmeet Kohli, Martin Riedmiller, Oriol Vinyals, and David Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618(7964): 257–263, 2023. doi: 10.1038/s41586-023-06004-9.
- Dimitris G. Manolakis, Vinay K. Ingle, and Stephen M. Kogon. *Statistical and Adaptive Signal Processing*. Artech House, 2005.
- Andreas Mayr, Günter Klambauer, Thomas Unterthiner, and Sepp Hochreiter. Deeptox: Toxicity prediction using deep learning. *Frontiers in Environmental Science*, 3, 2016. doi: 10.3389/fenvs.2015.00080.
- Michael McCabe, Bruno Régaldou-Saint Blancard, Liam Holden Parker, Ruben Ohana, Miles Cranmer, Alberto Bietti, Michael Eickenberg, Siavash Golkar, Geraud Krawezik, Francois Lanusse, Mariel Pettee, Tiberiu Tesileanu, Kyunghyun Cho, and Shirley Ho. Multiple physics pretraining for physical surrogate models. arXiv:2310.02994, 2023.

- Jarrold R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1), 2018. doi: 10.1038/s41467-018-07090-4.
- Levi D. McClenny and Ulisses M. Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023. doi: 10.1016/j.jcp.2022.111722.
- Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943. doi: 10.1007/BF02478259.
- Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. arXiv:1802.03426, 2018.
- F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994. doi: 10.2514/3.12149.
- Daniel A. Messenger and David M. Bortz. Weak sindy for partial differential equations. *Journal of Computational Physics*, 443:110525, 2021. doi: 10.1016/j.jcp.2021.110525.
- Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. doi: 10.1063/1.1699114.
- Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. arXiv:1710.03740, 2017.
- Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. arXiv:1301.3781, 2013.
- Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. arXiv:2003.08934, 2020.
- C. Wright Mills. *The Sociological Imagination*. Oxford University Press, 1959.
- Melanie Mitchell. Why ai is harder than we think. arXiv:2104.12871, 2021.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy P. Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. arXiv:1602.01783, 2016. ICML 2016.
- Christoph Molnar. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Independently published, 2 edition, 2022.
- J. J. Monaghan. Smoothed particle hydrodynamics. *Annual Review of Astronomy and*

- Astrophysics*, 30(1):543–574, 1992. doi: 10.1146/annurev.aa.30.090192.002551.
- Vishal Monga, Yuelong Li, and Yonina C. Eldar. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Processing Magazine*, 38(2):18–44, 2021. doi: 10.1109/MSP.2020.3016905.
- Faruck Morcos, Andrea Pagnani, Bryan Lunt, Arianna Bertolino, Debora S. Marks, Chris Sander, Riccardo Zecchina, José N. Onuchic, Terence Hwa, and Martin Weigt. Direct-coupling analysis of residue coevolution captures native contacts across many protein families. *Proceedings of the National Academy of Sciences*, 108(49), 2011. doi: 10.1073/pnas.1111471108.
- Masahiro Mori, Karl MacDorman, and Norri Kageki. The uncanny valley [from the field]. *IEEE Robotics & Automation Magazine*, 19(2):98–100, 2012. doi: 10.1109/MR.A.2012.2192811.
- Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. arXiv:1810.02244, 2018.
- Meredith Ringel Morris, Jascha Sohl-Dickstein, Noah Fiedel, Tris Warkentin, Allan Dafoe, Aleksandra Faust, Clement Farabet, and Shane Legg. Levels of agi for operationalizing progress on the path to agi. arXiv:2311.02462, 2023. Proceedings of ICML 2024.
- B. R. Morton, Geoffrey Ingram Taylor, and John Stewart Turner. Turbulent gravitational convection from maintained and instantaneous sources. *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences*, 234(1196):1–23, 1956. doi: 10.1098/rspa.1956.0011.
- Tamara Munzner. *Visualization Analysis and Design*. CRC Press, 2014.
- Clifford Nass and Youngme Moon. Machines and mindlessness: Social responses to computers. *Journal of Social Issues*, 56(1):81–103, 2000. doi: 10.1111/0022-4537.00153.
- Eric Nguyen, Michael Poli, Matthew G. Durrant, Brian Kang, Dhruva Katrekar, David B. Li, Liam J. Bartie, Armin W. Thomas, Samuel H. King, Garyk Brixi, Jeremy Sullivan, Madelena Y. Ng, Ashley Lewis, Aaron Lou, Stefano Ermon, Stephen A. Baccus, Tina Hernandez-Boussard, Christopher Ré, Patrick D. Hsu, and Brian L. Hie. Sequence modeling and design from molecular to genome scale with evo. *Science*, 386(6723), 2024. doi: 10.1126/science.ado9336.
- Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2012. doi: 10.1017/CBO9780511976667.
- Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2 edition, 2006.
- Ruben Ohana, Michael McCabe, Lucas Meyer, Rudy Morel, Fruzsina J. Agocs, Miguel Beneitez, Marsha Berger, Blakesley Burkhart, Keaton Burns, Stuart B. Dalziel, Drummond B. Fielding, Daniel Fortunato, Jared A. Goldberg, Keiya Hirashima, Yan-Fei Jiang, Rich R. Kerswell, Suryanarayana Maddu, Jonah Miller, Payel Mukhopadhyay, Stefan S. Nixon, Jeff Shen, Romain Watteaux, Bruno Régaldou-Saint Blancard, François Rozet, Liam H. Parker, Miles Cranmer, and Shirley Ho. The well: a large-scale collection of diverse physics simulations for machine learning. arXiv:2412.00568, 2024.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever,

- Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report. arXiv:2303.08774, 2023.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision. arXiv:2304.07193, 2023.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. arXiv:2203.02155, 2022.
- Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. arXiv:1901.05103, 2019.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. arXiv:1912.01703, 2019.
- Suhas V. Patankar. *Numerical Heat Transfer and Fluid Flow*. Hemisphere, 1980.
- S.V Patankar and D.B Spalding. A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer*, 15(10):1787–1806, 1972. doi: 10.1016/0017-9310(72)90054-3.
- Jaideep Pathak, Shashank Subramanian, Peter Harrington, Sanjeev Raja, Ashesh Chattopadhyay, Morteza Mardani, Thorsten Kurth, David Hall, Zongyi Li, Kamyar Azizzadenesheli, Pedram Hassanzadeh, Karthik Kashinath, and Animashree Anandkumar. FourCastNet: A global data-driven high-resolution weather model using adaptive fourier neural operators. arXiv:2202.11214, 2022.
- Karl Pearson. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11): 559–572, 1901. doi: 10.1080/14786440109462720.
- William Peebles and Saining Xie. Scalable diffusion models with transformers.

- arXiv:2212.09748, 2022.
- Jeffrey Pennington, Richard Socher, and Christopher Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014. doi: 10.3115/v1/D14-1162.
- Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W. Battaglia. Learning mesh-based simulation with graph networks. arXiv:2010.03409, 2020. International Conference on Learning Representations (ICLR), 2021.
- David Pfau, James S. Spencer, Alexander G. de G. Matthews, and W. M. C. Foulkes. Ab-initio solution of the many-electron schrödinger equation with deep neural networks. arXiv:1909.02487, 2019. Phys. Rev. Research 2, 033429 (2020).
- Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, Michael Choi, Anish Agrawal, Arnav Chopra, Adam Khoja, Ryan Kim, Richard Ren, Jason Hausenloy, Oliver Zhang, Mantas Mazeika, Dmitry Dodonov, Tung Nguyen, Jaeho Lee, Daron Anderson, Mikhail Doroshenko, Alun Cennyth Stokes, Mobeen Mahmood, Oleksandr Pokutnyi, Oleg Iskra, Jessica P. Wang, John-Clark Levin, Mstyslav Kazakov, Fiona Feng, Steven Y. Feng, Haoran Zhao, Michael Yu, Varun Gangal, Chelsea Zou, Zihan Wang, Serguei Popov, Robert Gerbicz, Geoff Galgon, Johannes Schmitt, Will Yeadon, Yongki Lee, Scott Sauers, Alvaro Sanchez, Fabian Giska, Marc Roth, Søren Riis, Saiteja Utpala, Noah Burns, Gashaw M. Goshu, Mohinder Maheshbhai Naiya, Chidozie Agu, Zachary Giboney, Antrell Cheatom, Francesco Fournier-Facio, Sarah-Jane Crowson, Lennart Finke, Zerui Cheng, Jennifer Zampese, Ryan G. Hoerr, Mark Nandor, Hyunwoo Park, Tim Gehringer, Jiaqi Cai, Ben McCarty, Alexis C Garretson, Edwin Taylor, Damien Sileo, Qiuyu Ren, Usman Qazi, Lianghui Li, Jungbae Nam, John B. Wydallis, Pavel Arkhipov, Jack Wei Lun Shi, Aras Bacho, Chris G. Willcocks, Hangrui Cao, Sumeet Motwani, Emily de Oliveira Santos, Johannes Veith, Edward Vendrow, Doru Cojoc, Kengo Zenitani, Joshua Robinson, Longke Tang, Yuqi Li, Joshua Vendrow, Natanael Wildner Fraga, Vladyslav Kuchkin, Andrey Pupasov Maksimov, Pierre Marion, Denis Efremov, Jayson Lynch, Kaiqu Liang, Aleksandar Mikov, Andrew Gritsevskiy, Julien Guillod, Gözdenur Demir, Dakotah Martinez, Ben Pageler, Kevin Zhou, Saeed Soori, Ori Press, Henry Tang, Paolo Rissone, Sean R. Green, Lina Brüssel, Moon Twayana, Aymeric Dieuleveut, Joseph Marvin Imperial, Ameya Prabhu, Jinzhou Yang, Nick Crispino, Arun Rao, Dimitri Zvonkine, Gabriel Loiseau, Mikhail Kalinin, Marco Lukas, Ciprian Manolescu, Nate Stambaugh, Subrata Mishra, Tad Hogg, Carlo Bosio, Brian P Coppola, Julian Salazar, Jaehyeok Jin, Rafael Sayous, Stefan Ivanov, Philippe Schwaller, Shaipranesh Senthilkuma, Andres M Bran, Andres Algaba, Kelsey Van den Houte, Lynn Van Der Sypt, Brecht Verbeken, David Noever, Alexei Kopylov, Benjamin Myklebust, Bikun Li, Lisa Schut, Evgenii Zheltonozhskii, Qiaochu Yuan, Derek Lim, Richard Stanley, Tong Yang, John Maar, Julian Wykowski, Martí Oller, Anmol Sahu, Cesare Giulio Ardito, Yuzheng Hu, Ariel Ghislain Kemogne Kamdoun, Alvin Jin, Tobias Garcia Vilchis, Yuexuan Zu, Martin Lackner, James Koppel, Gongbo Sun, Daniil S. Antonenko, Steffi Chern, Bingchen Zhao, Pierrot Arsene, Joseph M Cavanagh, Daofeng Li, Jiawei Shen, Donato Crisostomi, Wenjin

Zhang, Ali Dehghan, Sergey Ivanov, David Perrella, Nurdin Kaparov, Allen Zang, Ilia Sucholutsky, Arina Kharlamova, Daniil Orel, Vladislav Poritski, Shalev Ben-David, Zachary Berger, Parker Whitfill, Michael Foster, Daniel Munro, Linh Ho, Shankar Sivarajan, Dan Bar Hava, Aleksey Kuchkin, David Holmes, Alexandra Rodriguez-Romero, Frank Sommerhage, Anji Zhang, Richard Moat, Keith Schneider, Zakayo Kazibwe, Don Clarke, Dae Hyun Kim, Felipe Meneguitti Dias, Sara Fish, Veit Elser, Tobias Kreiman, Victor Efren Guadarrama Vilchis, Immo Klose, Ujjwala Anantheswaran, Adam Zweiger, Kaivalya Rawal, Jeffery Li, Jeremy Nguyen, Nicolas Daans, Haline Heidinger, Maksim Radionov, Václav Rozhoň, Vincent Ginis, Christian Stump, Niv Cohen, Rafał Poświata, Josef Tkadlec, Alan Goldfarb, Chenguang Wang, Piotr Padlewski, Stanislaw Barzowski, Kyle Montgomery, Ryan Stendall, Jamie Tucker-Foltz, Jack Stade, T. Ryan Rogers, Tom Goertzen, Declan Grabb, Abhishek Shukla, Alan Givré, John Arnold Ambay, Archan Sen, Muhammad Fayez Aziz, Mark H Inlow, Hao He, Ling Zhang, Younesse Kaddar, Ivar Ångquist, Yanxu Chen, Harrison K Wang, Kalyan Ramakrishnan, Elliott Thornley, Antonio Terpin, Hailey Schoelkopf, Eric Zheng, Avishy Carmi, Ethan D. L. Brown, Kelin Zhu, Max Bartolo, Richard Wheeler, Martin Stehberger, Peter Bradshaw, JP Heimonen, Kaustubh Sridhar, Ido Akov, Jennifer Sandlin, Yury Makarychev, Joanna Tam, Hieu Hoang, David M. Cunningham, Vladimir Goryachev, Demosthenes Patramanis, Michael Krause, Andrew Redenti, David Aldous, Jesyin Lai, Shannon Coleman, Jiangnan Xu, Sangwon Lee, Ilias Magoulas, Sandy Zhao, Ning Tang, Michael K. Cohen, Orr Paradise, Jan Hendrik Kirchner, Maksym Ovchynnikov, Jason O. Matos, Adithya Shenoy, Michael Wang, Yuzhou Nie, Anna Sztyber-Betley, Paolo Faraboschi, Robin Riblet, Jonathan Crozier, Shiv Halasyamani, Shreyas Verma, Prashant Joshi, Eli Meril, Ziqiao Ma, Jérémy Andréoletti, Raghav Singhal, Jacob Platnick, Volodymyr Nevirkovets, Luke Basler, Alexander Ivanov, Seri Khoury, Nils Gustafsson, Marco Piccardo, Hamid Mostaghimi, Qijia Chen, Virendra Singh, Tran Quoc Khánh, Paul Rosu, Hannah Szlyk, Zachary Brown, Himanshu Narayan, Aline Menezes, Jonathan Roberts, William Alley, Kunyang Sun, Arkil Patel, Max Lamparth, Anka Reuel, Linwei Xin, Hanmeng Xu, Jacob Loader, Freddie Martin, Zixuan Wang, Andrea Achilleos, Thomas Preu, Tomek Korbak, Ida Bosio, Fereshteh Kazemi, Ziye Chen, Biró Bálint, Eve J. Y. Lo, Jiaqi Wang, Maria Inês S. Nunes, Jeremiah Milbauer, M Saiful Bari, Zihao Wang, Behzad Ansarinejad, Yewen Sun, Stephane Durand, Hossam Elgnainy, Guillaume Douville, Daniel Tordera, George Balabanian, Hew Wolff, Lynna Kvistad, Hsiaoyun Milliron, Ahmad Sakor, Murat Eron, Andrew Favre D. O., Shailesh Shah, Xiaoxiang Zhou, Firuz Kamalov, Sherwin Abdoli, Tim Santens, Shaul Barkan, Allison Tee, Robin Zhang, Alessandro Tomasiello, G. Bruno De Luca, Shi-Zhuo Looi, Vinh-Kha Le, Noam Kolt, Jiayi Pan, Emma Rodman, Jacob Drori, Carl J Fossum, Niklas Muennighoff, Milind Jagota, Ronak Pradeep, Honglu Fan, Jonathan Eicher, Michael Chen, Kushal Thaman, William Merrill, Moritz Firsching, Carter Harris, Stefan Ciobâcă, Jason Gross, Rohan Pandey, Ilya Gusev, Adam Jones, Shashank Agnihotri, Pavel Zhelnov, Mohammadreza Mofayezi, Alexander Piperski, David K. Zhang, Kostiantyn Dobarskyi, Roman Leventov, Ignat Soroko, Joshua Duersch, Vage Taamazyan, Andrew Ho, Wenjie Ma, William Held, Ruicheng

Xian, Armel Randy Zebaze, Mohanad Mohamed, Julian Noah Leser, Michelle X Yuan, Laila Yacar, Johannes Lengler, Katarzyna Olszewska, Claudio Di Fratta, Edson Oliveira, Joseph W. Jackson, Andy Zou, Muthu Chidambaram, Timothy Manik, Hector Haffenden, Dashiell Stander, Ali Dasouqi, Alexander Shen, Bitá Golshani, David Stap, Egor Kretov, Mikalai Uzhou, Alina Borisovna Zhidkovskaya, Nick Winter, Miguel Orbegozo Rodriguez, Robert Lauff, Dustin Wehr, Colin Tang, Zaki Hossain, Shaun Phillips, Fortuna Samuele, Fredrik Ekström, Angela Hammon, Oam Patel, Faraz Farhidi, George Medley, Forough Mohammadzadeh, Madellene Peñaflor, Haile Kassahun, Alena Friedrich, Rayner Hernandez Perez, Daniel Pyda, Taom Sakal, Omkar Dhamane, Ali Khajegili Mirabadi, Eric Hallman, Kenchi Okutsu, Mike Battaglia, Mohammad Maghsoudimehrabani, Alon Amit, Dave Hulbert, Roberto Pereira, Simon Weber, Handoko, Anton Peristyy, Stephen Malina, Mustafa Mehkary, Rami Aly, Frank Reidegeld, Anna-Katharina Dick, Cary Friday, Mukhwinder Singh, Hassan Shapourian, Wanyoung Kim, Mariana Costa, Hubeyb Gurdogan, Harsh Kumar, Chiara Ceconello, Chao Zhuang, Haon Park, Micah Carroll, Andrew R. Tawfeek, Stefan Steinerberger, Daattavya Aggarwal, Michael Kirchhof, Linjie Dai, Evan Kim, Johan Ferret, Jainam Shah, Yuzhou Wang, Minghao Yan, Krzysztof Burdzy, Lixin Zhang, Antonio Franca, Diana T. Pham, Kang Yong Loh, Joshua Robinson, Abram Jackson, Paolo Giordano, Philipp Petersen, Adrian Cosma, Jesus Colino, Colin White, Jacob Votava, Vladimir Vinnikov, Ethan Delaney, Petr Spelda, Vit Stritecky, Syed M. Shahid, Jean-Christophe Mourrat, Lavar Vetoshkin, Koen Sponselee, Renas Bacho, Zheng-Xin Yong, Florencia de la Rosa, Nathan Cho, Xiuyu Li, Guillaume Malod, Orion Weller, Guglielmo Albani, Leon Lang, Julien Laurendeau, Dmitry Kazakov, Fatimah Adesanya, Julien Portier, Lawrence Hollom, Victor Souza, Yuchen Anna Zhou, Julien Degorre, Yiğit Yalın, Gbenga Daniel Obikoya, Rai, Filippo Bigi, M. C. Boscá, Oleg Shumar, Kaniuar Bacho, Gabriel Recchia, Mara Popescu, Nikita Shulga, Ngefor Mildred Tanwie, Thomas C. H. Lux, Ben Rank, Colin Ni, Matthew Brooks, Alesia Yakimchyk, Huanxu, Liu, Stefano Cavalleri, Olle Häggström, Emil Verkama, Joshua Newbould, Hans Gundlach, Leonor Brito-Santana, Brian Amaro, Vivek Vajipey, Rynaa Grover, Ting Wang, Yosi Kratish, Wen-Ding Li, Sivakanth Gopi, Andrea Caciolai, Christian Schroeder de Witt, Pablo Hernández-Cámara, Emanuele Rodolà, Jules Robins, Dominic Williamson, Vincent Cheng, Brad Raynor, Hao Qi, Ben Segev, Jingxuan Fan, Sarah Martinson, Erik Y. Wang, Kaylie Hausknecht, Michael P. Brenner, Mao Mao, Christoph Demian, Peyman Kassani, Xinyu Zhang, David Avagian, Eshawn Jessica Scipio, Alon Ragoler, Justin Tan, Blake Sims, Rebeka Plecnik, Aaron Kirtland, Omer Faruk Bodur, D. P. Shinde, Yan Carlos Leyva Labrador, Zahra Adoul, Mohamed Zekry, Ali Karakoc, Tania C. B. Santos, Samir Shamseldeen, Loukmane Karim, Anna Liakhovitskaia, Nate Resman, Nicholas Farina, Juan Carlos Gonzalez, Gabe Maayan, Earth Anderson, Rodrigo De Oliveira Pena, Elizabeth Kelley, Hodjat Mariji, Rasoul Pouriamanesh, Wentao Wu, Ross Finocchio, Ismail Alarab, Joshua Cole, Danyelle Ferreira, Bryan Johnson, Mohammad Safdari, Liangti Dai, Siriphan Arthornthurasuk, Isaac C. McAlister, Alejandro José Moyano, Alexey Pronin, Jing Fan, Angel Ramirez-Trinidad, Yana Malysheva, Daphiny Pottmaier, Omid Taheri,

Stanley Stepanic, Samuel Perry, Luke Askew, Raúl Adrián Huerta Rodríguez, Ali M. R. Minissi, Ricardo Lorena, Krishnamurthy Iyer, Arshad Anil Fasiludeen, Ronald Clark, Josh Ducey, Matheus Piza, Maja Somrak, Eric Vergo, Juehang Qin, Benjámín Borbás, Eric Chu, Jack Lindsey, Antoine Jallon, I. M. J. McInnis, Evan Chen, Avi Semler, Luk Gloor, Tej Shah, Marc Carauleanu, Pascal Lauer, Tran Duc Huy, Hossein Shahrtash, Emilien Duc, Lukas Lewark, Assaf Brown, Samuel Albanie, Brian Weber, Warren S. Vaz, Pierre Clavier, Yiyang Fan, Gabriel Poesia Reis e Silva, Long, Lian, Marcus Abramovitch, Xi Jiang, Sandra Mendoza, Murat Islam, Juan Gonzalez, Vasilios Mavroudis, Justin Xu, Pawan Kumar, Laxman Prasad Goswami, Daniel Bugas, Nasser Heydari, Ferenc Jeanplong, Thorben Jansen, Antonella Pinto, Archimedes Apronti, Abdallah Galal, Ng Ze-An, Ankit Singh, Tong Jiang, Joan of Arc Xavier, Kanu Priya Agarwal, Mohammed Berkani, Gang Zhang, Zhehang Du, Benedito Alves de Oliveira Junior, Dmitry Malishev, Nicolas Remy, Taylor D. Hartman, Tim Tarver, Stephen Mensah, Gautier Abou Loume, Wiktor Morak, Farzad Habibi, Sarah Hoback, Will Cai, Javier Gimenez, Roselynn Grace Montecillo, Jakub Łucki, Russell Campbell, Asankhaya Sharma, Khalida Meer, Shreen Gul, Daniel Espinosa Gonzalez, Xavier Alapont, Alex Hoover, Gunjan Chhablani, Freddie Vargus, Arunim Agarwal, Yibo Jiang, Deepakkumar Patil, David Outevsky, Kevin Joseph Scaria, Rajat Maheshwari, Abdelkader Dendane, Priti Shukla, Ashley Cartwright, Sergei Bogdanov, Niels Mündler, Sören Möller, Luca Arnaboldi, Kunvar Thaman, Muhammad Rehan Siddiqi, Prajvi Saxena, Himanshu Gupta, Tony Fruhauff, Glen Sherman, Mátyás Vincze, Siranut Usawasutsakorn, Dylan Ler, Anil Radhakrishnan, Innocent Enyekwe, Sk Md Salauddin, Jiang Muzhen, Aleksandr Maksapetyan, Vivien Rossbach, Chris Harjadi, Mohsen Bahaloohoreh, Claire Sparrow, Jasdeep Sidhu, Sam Ali, Song Bian, John Lai, Eric Singer, Justine Leon Uro, Greg Bateman, Mohamed Sayed, Ahmed Menshawy, Darling Duclosel, Dario Bezzi, Yashaswini Jain, Ashley Aaron, Murat Tiryakioglu, Sheeshram Siddh, Keith Krenek, Imad Ali Shah, Jun Jin, Scott Creighton, Denis Peskoff, Zienab EL-Wasif, Ragavendran P, Michael Richmond, Joseph McGowan, Tejal Patwardhan, Hao-Yu Sun, Ting Sun, Nikola Zubić, Samuele Sala, Stephen Ebert, Jean Kaddour, Manuel Schottdorf, Dianzhuo Wang, Gerol Petruzella, Alex Meiburg, Tilen Medved, Ali ElSheikh, S Ashwin Hebbar, Lorenzo Vaquero, Xianjun Yang, Jason Poulos, Vilém Zouhar, Sergey Bogdanik, Mingfang Zhang, Jorge Sanz-Ros, David Anugraha, Yinwei Dai, Anh N. Nhu, Xue Wang, Ali Anil Demircali, Zhibai Jia, Yuyin Zhou, Juncheng Wu, Mike He, Nitin Chandok, Aarush Sinha, Gaoxiang Luo, Long Le, Mickaël Noyé, Michał Perełkiewicz, Ioannis Pantidis, Tianbo Qi, Soham Sachin Purohit, Letitia Parcalabescu, Thai-Hoa Nguyen, Genta Indra Winata, Edoardo M. Ponti, Hanchen Li, Kaustubh Dhole, Jongee Park, Dario Abbondanza, Yuanli Wang, Anupam Nayak, Diogo M. Caetano, Antonio A. W. L. Wong, Maria del Rio-Chanona, Dániel Kondor, Pieter Francois, Ed Chalstrey, Jakob Zsambok, Dan Hoyer, Jenny Reddish, Jakob Hauser, Francisco-Javier Rodrigo-Ginés, Suchandra Datta, Maxwell Shepherd, Thom Kamphuis, Qizheng Zhang, Hyunjun Kim, Ruiji Sun, Jianzhu Yao, Franck Dernoncourt, Satyapriya Krishna, Sina Rismanchian, Bonan Pu, Francesco Pinto, Yingheng Wang, Kumar Shridhar, Kalon J. Overholt, Glib Briia, Hieu Nguyen,

David, Soler Bartomeu, Tony CY Pang, Adam Wecker, Yifan Xiong, Fanfei Li, Lukas S. Huber, Joshua Jaeger, Romano De Maddalena, Xing Han Lù, Yuhui Zhang, Claas Beger, Patrick Tser Jern Kon, Sean Li, Vivek Sanker, Ming Yin, Yihao Liang, Xinlu Zhang, Ankit Agrawal, Li S. Yifei, Zechen Zhang, Mu Cai, Yasin Sonmez, Costin Cozianu, Changhao Li, Alex Slen, Shoubin Yu, Hyun Kyu Park, Gabriele Sarti, Marcin Briński, Alessandro Stolfo, Truong An Nguyen, Mike Zhang, Yotam Perlitz, Jose Hernandez-Orallo, Runjia Li, Amin Shabani, Felix Juefei-Xu, Shikhar Dhingra, Orr Zohar, My Chiffon Nguyen, Alexander Pondaven, Abdurrahim Yilmaz, Xuandong Zhao, Chuanyang Jin, Muyan Jiang, Stefan Todoran, Xinyao Han, Jules Kreuer, Brian Rabern, Anna Plassart, Martino Maggetti, Luther Yap, Robert Geirhos, Jonathon Kean, Dingsu Wang, Sina Mollaei, Chenkai Sun, Yifan Yin, Shiqi Wang, Rui Li, Yaowen Chang, Anjiang Wei, Alice Bizeul, Xiaohan Wang, Alexandre Oliveira Arrais, Kushin Mukherjee, Jorge Chamorro-Padial, Jiachen Liu, Xingyu Qu, Junyi Guan, Adam Bouyamourn, Shuyu Wu, Martyna Plomecka, Junda Chen, Mengze Tang, Jiaqi Deng, Shreyas Subramanian, Haocheng Xi, Haoxuan Chen, Weizhi Zhang, Yinuo Ren, Haoqin Tu, Sejong Kim, Yushun Chen, Sara Vera Marjanović, Junwoo Ha, Grzegorz Luczyna, Jeff J. Ma, Zewen Shen, Dawn Song, Cedegao E. Zhang, Zhun Wang, Gaël Gendron, Yunze Xiao, Leo Smucker, Erica Weng, Kwok Hao Lee, Zhe Ye, Stefano Ermon, Ignacio D. Lopez-Miguel, Theo Knights, Anthony Gitter, Namkyu Park, Boyi Wei, Hongzheng Chen, Kunal Pai, Ahmed Elkhany, Han Lin, Philipp D. Siedler, Jichao Fang, Ritwik Mishra, Károly Zsolnai-Fehér, Xilin Jiang, Shadab Khan, Jun Yuan, Rishab Kumar Jain, Xi Lin, Mike Peterson, Zhe Wang, Aditya Malusare, Maosen Tang, Isha Gupta, Ivan Fosin, Timothy Kang, Barbara Dworakowska, Kazuki Matsumoto, Guangyao Zheng, Gerben Sewuster, Jorge Pretel Villanueva, Ivan Rannev, Igor Chernyavsky, Jiale Chen, Deepayan Banik, Ben Racz, Wenchao Dong, Jianxin Wang, Laila Bashmal, Duarte V. Gonçalves, Wei Hu, Kaushik Bar, Ondrej Bohdal, Atharv Singh Patlan, Shehzaad Dhuliawala, Caroline Geirhos, Julien Wist, Yuval Kansal, Bingsen Chen, Kutay Tire, Atak Talay Yücel, Brandon Christof, Veerupaksh Singla, Zijian Song, Sanxing Chen, Jiaxin Ge, Kaustubh Ponkshe, Isaac Park, Tianneng Shi, Martin Q. Ma, Joshua Mak, Sherwin Lai, Antoine Moulin, Zhuo Cheng, Zhanda Zhu, Ziyi Zhang, Vaidehi Patil, Ketan Jha, Qiutong Men, Jiaxuan Wu, Tianchi Zhang, Bruno Hebling Vieira, Alham Fikri Aji, Jae-Won Chung, Mohammed Mahfoud, Ha Thi Hoang, Marc Sperzel, Wei Hao, Kristof Meding, Sihan Xu, Vassilis Kostakos, Davide Manini, Yueying Liu, Christopher Toukmaji, Jay Paek, Eunmi Yu, Arif Engin Demircali, Zhiyi Sun, Ivan Dewerpe, Hongsen Qin, Roman Pflugfelder, James Bailey, Johnathan Morris, Ville Heilala, Sybille Rosset, Zishun Yu, Peter E. Chen, Woongyeong Yeo, Eeshaan Jain, Ryan Yang, Sreekar Chigurupati, Julia Chernyavsky, Sai Prajwal Reddy, Subhashini Venugopalan, Hunar Batra, Core Francisco Park, Hieu Tran, Guilherme Maximiano, Genghan Zhang, Yizhuo Liang, Hu Shiyu, Rongwu Xu, Rui Pan, Siddharth Suresh, Ziqi Liu, Samaksh Gulati, Songyang Zhang, Peter Turchin, Christopher W. Bartlett, Christopher R. Scotese, Phuong M. Cao, Ben Wu, Jacek Karwowski, Davide Scaramuzza, Aakaash Nattanmai, Gordon McKellips, Anish Cheraku, Asim Suhail, Ethan Luo, Marvin Deng, Jason Luo, Ashley Zhang, Kevin

- Jindel, Jay Paek, Kasper Halevy, Allen Baranov, Michael Liu, Advait Avadhanam, David Zhang, Vincent Cheng, Brad Ma, Evan Fu, Liam Do, Joshua Lass, Hubert Yang, Surya Sunkari, Vishruth Bharath, Violet Ai, James Leung, Rishit Agrawal, Alan Zhou, Kevin Chen, Tejas Kalpathi, Ziqi Xu, Gavin Wang, Tyler Xiao, Erik Maung, Sam Lee, Ryan Yang, Roy Yue, Ben Zhao, Julia Yoon, Sunny Sun, Aryan Singh, Ethan Luo, Clark Peng, Tyler Osbey, Taozhi Wang, Daryl Echeazu, Hubert Yang, Timothy Wu, Spandan Patel, Vidhi Kulkarni, Vijaykaarti Sundarapandiyan, Ashley Zhang, Andrew Le, Zafir Nasim, Srikar Yalam, Ritesh Kasamsetty, Soham Samal, Hubert Yang, David Sun, Nihar Shah, Abhijeet Saha, Alex Zhang, Leon Nguyen, Laasya Nagumalli, Kaixin Wang, Alan Zhou, Aidan Wu, Jason Luo, Anwith Telluri, Steven Dillmann, Zhengxiang Wang, Junyu Luo, Hugo Lunn, Artem Gazizov, Haitz Sáez de Ocariz Borde, Ivan Trus, Morgan Hervault, Zheyu Zhang, Bo Chen, Yuchen Wu, Christopher J. Cordier, Gün Kaynar, Cansin Ayvaz, Polina Avdiunina, Johannes Brust, Xingjian Diao, K. D. Meaney, Yifan Gu, Chenyu Wang, Chenzhuo Dong, William Wright, Simon Brave, Owen Root, Jiayuan Liu, Chow Chun Lok, Tianqin Li, Shiyi Du, Dailan He, Lufeiya Liu, Sina Jamalzadegan, Anil Ramakrishna, Xuanqing Xu, Xin Qing, Xin Luo, Wenkai Li, Shi Bo, Philipp Gusev, Maximos Skandalis, Desheng Ma, Chunhui Zhang, Haoran Qiu, Allen G Hart, Rickard Brüel Gabriëlsson, Ido Akov, Artem Lukoianov, Summer Yue, Alexandr Wang, and Dan Hendrycks. Humanity's last exam. arXiv:2501.14249, 2025.
- S. Pirker and T. Lichtenegger. Efficient time-extrapolation of single- and multiphase simulations by transport based recurrence cfd (rcfd). *Chemical Engineering Science*, 188: 65–83, 2018. doi: 10.1016/j.ces.2018.04.059.
- S. Pirker, M. Atzori, S. Heinrich, and T. Lichtenegger. Particle size segregation in bi and penta-disperse gas–solid fluidized beds: Cfd-dem and recurrence cfd simulations. *Chemical Engineering Science*, 309:121469, 2025. doi: 10.1016/j.ces.2025.121469.
- Aske Plaat. *Deep Reinforcement Learning*. Springer, 2022.
- B.T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.
- Stephen B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.
- Ryan Poplin, Pi-Chuan Chang, David Alexander, Scott Schwartz, Thomas Colthurst, Alexander Ku, Dan Newburger, Jojo Dijamco, Nam Nguyen, Pegah T Afshar, Sam S Gross, Lizzie Dorfman, Cory Y McLean, and Mark A DePristo. A universal snp and small-indel variant caller using deep neural networks. *Nature Biotechnology*, 36(10): 983–987, 2018. doi: 10.1038/nbt.4235.
- M.F. Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980. doi: 10.1108/eb046814.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. arXiv:2201.02177, 2022.
- John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, 2018.

- doi: 10.22331/q-2018-08-06-79.
- William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 3 edition, 2007.
- Kristina Preuer, Philipp Renz, Thomas Unterthiner, Sepp Hochreiter, and Günter Klambauer. Fréchet chemnet distance: A metric for generative models for molecules in drug discovery. *Journal of Chemical Information and Modeling*, 58(9):1736–1741, 2018. doi: 10.1021/acs.jcim.8b00234.
- Ilan Price, Alvaro Sanchez-Gonzalez, Ferran Alet, Tom R. Andersson, Andrew El-Kadi, Dominic Masters, Timo Ewalds, Jacklynn Stott, Shakir Mohamed, Peter Battaglia, Remi Lam, and Matthew Willson. Probabilistic weather forecasting with machine learning. *Nature*, 637(8044):84–90, 2025. doi: 10.1038/s41586-024-08252-9.
- Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023.
- Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. arXiv:1612.00593, 2016.
- Jean Rabault, Miroslav Kuchta, Atle Jensen, Ulysse Réglade, and Nicolas Cerardi. Artificial neural networks trained through deep reinforcement learning discover control strategies for active flow control. *Journal of Fluid Mechanics*, 865:281–302, 2019. doi: 10.1017/jfm.2019.62.
- L.R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.
- Christopher Rackauckas, Yingbo Ma, Julius Martensen, Collin Warner, Kirill Zubov, Rohit Supekar, Dominic Skinner, Ali Ramadhan, and Alan Edelman. Universal differential equations for scientific machine learning. arXiv:2001.04385, 2020.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. arXiv:2103.00020, 2021.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. arXiv:2305.18290, 2023.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. arXiv:1806.08734, 2018. ICML 2019.
- M. Raissi, P. Perdikaris, and G.E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- Maziar Raissi, Alireza Yazdani, and George Em Karniadakis. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. *Science*, 367(6481):1026–1030, 2020. doi: 10.1126/science.aaw4741.

- Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. Recommender systems with generative retrieval. arXiv:2305.05065, 2023.
- Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. arXiv:2205.12454, 2022.
- Hubert Ramsauer, Bernhard Schäfl, Johannes Lehner, Philipp Seidl, Michael Widrich, Thomas Adler, Lukas Gruber, Markus Holzleitner, Milena Pavlović, Geir Kjetil Sandve, Victor Greiff, David Kreil, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. Hopfield networks is all you need. arXiv:2008.02217, 2020.
- Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- Pratik Rathore, Weimu Lei, Zachary Frangella, Lu Lu, and Madeleine Udell. Challenges in training pinns: A loss landscape perspective. arXiv:2402.01868, 2024.
- Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos. arXiv:2408.00714, 2024.
- Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. arXiv:1506.02640, 2015.
- Byron Reeves and Clifford Nass. *The Media Equation: How People Treat Computers, Television, and New Media Like Real People and Places*. Cambridge University Press, 1996.
- Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. arXiv:1506.01497, 2015.
- Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. arXiv:1205.2618, 2012.
- Guy Revach, Nir Shlezinger, Xiaoyong Ni, Adria Lopez Escoriza, Ruud J. G. van Sloun, and Yonina C. Eldar. Kalmannet: Neural network aided kalman filtering for partially known dynamics. arXiv:2107.10043, 2021.
- Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows. arXiv:1505.05770, 2015.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?": Explaining the predictions of any classifier. arXiv:1602.04938, 2016.
- Mark A. Richards. *Fundamentals of Radar Signal Processing*. McGraw-Hill, 2 edition, 2014.
- Herbert Robbins and Sutton Monroe. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951. doi: 10.1214/aoms/1177729586.
- J. A. Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965. doi: 10.1145/321250.321253.

- David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t.
- Hermann Rohling. Radar cfar thresholding in clutter and multiple target situations. *IEEE Transactions on Aerospace and Electronic Systems*, AES-19(4):608–621, 1983. doi: 10.1109/TAES.1983.309350.
- Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. arXiv:2112.10752, 2021.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. arXiv:1505.04597, 2015.
- F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958. doi: 10.1037/h0042519.
- H. H. Rosenbrock. An automatic method for finding the greatest or least value of a function. *The Computer Journal*, 3(3):175–184, 1960. doi: 10.1093/comjnl/3.3.175.
- Stephane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. arXiv:1011.0686, 2010.
- Sam T. Roweis and Lawrence K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000. doi: 10.1126/science.290.5500.2323.
- Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5): 206–215, 2019. doi: 10.1038/s42256-019-0048-x.
- Samuel H. Rudy, Steven L. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Data-driven discovery of partial differential equations. *Science Advances*, 3(4), 2017. doi: 10.1126/sciadv.1602614.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.
- Stuart Russell. *Human Compatible: Artificial Intelligence and the Problem of Control*. Viking, 2019. ISBN 9780525558613.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.
- Ana Sanchez-Fernandez, Elisabeth Rumetshofer, Sepp Hochreiter, and Günter Klambauer. Cloome: contrastive learning unlocks bioimaging databases for queries with chemical structures. *Nature Communications*, 14(1), 2023. doi: 10.1038/s41467-023-42328-w.

- Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter W. Battaglia. Learning to simulate complex physics with graph networks. arXiv:2002.09405, 2020.
- Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pages 285–295, 2001. doi: 10.1145/371920.372071.
- Victor Garcia Satorras, Emiel Hoogetboom, and Max Welling. E(n) equivariant graph neural networks. arXiv:2102.09844, 2021.
- David C. Schedl, Indrajit Kurmi, and Oliver Bimber. Search and rescue with airborne optical sectioning. *Nature Machine Intelligence*, 2(12):783–790, 2020. doi: 10.1038/s42256-020-00261-3.
- PETER J. SCHMID. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. doi: 10.1017/S0022112010001217.
- Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015. doi: 10.1016/j.neunet.2014.09.003.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4.
- Maria Schuld and Francesco Petruccione. *Machine Learning with Quantum Computers*. Springer International Publishing, 2021. doi: 10.1007/978-3-030-83098-4.
- John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. arXiv:1502.05477, 2015a.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438, 2015b.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.
- Bernhard Schölkopf, Alexander Smola, and Klaus-Robert Müller. Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319, 1998. doi: 10.1162/089976698300017467.
- John R. Searle. Minds, brains, and programs. *Behavioral and Brain Sciences*, 3(3):417–424, 1980. doi: 10.1017/S0140525X00005756.
- Marwin H. S. Segler, Thierry Kogej, Christian Tyrchan, and Mark P. Waller. Generating focused molecule libraries for drug discovery with recurrent neural networks. *ACS Central Science*, 4(1):120–131, 2018a. doi: 10.1021/acscentsci.7b00512.
- Marwin H. S. Segler, Mike Preuss, and Mark P. Waller. Planning chemical syntheses with deep neural networks and symbolic ai. *Nature*, 555(7698):604–610, 2018b. doi: 10.1038/nature25978.

- Philipp Seidl, Philipp Renz, Natalia Dyubankova, Paulo Neves, Jonas Verhoeven, Jörg K. Wegner, Marwin Segler, Sepp Hochreiter, and Günter Klambauer. Improving few- and zero-shot reaction template prediction using modern hopfield networks. *Journal of Chemical Information and Modeling*, 62(9):2111–2120, 2022. doi: 10.1021/acs.jcim.1c01065.
- Philipp Seidl, Andreu Vall, Sepp Hochreiter, and Günter Klambauer. Enhancing activity prediction models in drug discovery with the ability to understand human language. arXiv:2303.03363, 2023.
- Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. arXiv:1610.02391, 2016.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. arXiv:1508.07909, 2015.
- Asif Shaik, Swantje Pietsch-Braune, Stefan Pirker, Melis Özdemir, Muhammad Adrian, Alexander Penn, and Stefan Heinrich. Mri-validated cfd–dem simulation and recurrence-based time extrapolation (rcfd) of a bubbling fluidized bed: Drag model selection and computational speed-up. *Chemical Engineering Research and Design*, 227: 879–893, 2026. doi: 10.1016/j.cherd.2026.02.039.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.
- Mehrdad Shariatifar, Rahmat Sotudeh-Gharebagh, Stefan Pirker, Reza Zarghami, and Navid Mostoufi. Accuracy assessment of recurrence cfd for continuous flow systems. *Chemical Engineering Research and Design*, 234:249–263, 2026. doi: 10.1016/j.cherd.2026.09.003.
- Jianbo Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000. doi: 10.1109/34.868688.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404.
- Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. arXiv:1312.6034,

- 2013.
- Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, Perry Payne, Martin Seneviratne, Paul Gamble, Chris Kelly, Abubakr Babiker, Nathanael Schärli, Aakanksha Chowdhery, Philip Mansfield, Dina Demner-Fushman, Blaise Agüera y Arcas, Dale Webster, Greg S. Corrado, Yossi Matias, Katherine Chou, Juraj Gottweis, Nenad Tomasev, Yun Liu, Alvin Rajkomar, Joelle Barral, Christopher Semturs, Alan Karthikesalingam, and Vivek Natarajan. Large language models encode clinical knowledge. *Nature*, 620(7972):172–180, 2023. doi: 10.1038/s41586-023-06291-2.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. arXiv:2006.09661, 2020.
- J. SMAGORINSKY. General circulation experiments with the primitive equations. *Monthly Weather Review*, 91(3):99–164, 1963. doi: 10.1175/1520-0493(1963)091<0099:GCEWTP>2.3.CO;2.
- Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms. arXiv:1206.2944, 2012.
- Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. arXiv:1503.03585, 2015.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. arXiv:2010.02502, 2020a.
- Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. arXiv:2011.13456, 2020b.
- Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. arXiv:2303.01469, 2023.
- Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in nlp. arXiv:1906.02243, 2019.
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. arXiv:1703.01365, 2017.
- Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. *Machine Learning Proceedings 1990*, pages 216–224, 1990. doi: 10.1016/B978-1-55860-141-3.50030-4.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1.
- Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2 edition, 2022.

- Kunihiko Taira, Steven L. Brunton, Scott T. M. Dawson, Clarence W. Rowley, Tim Colonius, Beverley J. McKeon, Oliver T. Schmidt, Stanislav Gordeyev, Vassilios Theofilis, and Lawrence S. Ukeiley. Modal analysis of fluid flows: An overview. *AIAA Journal*, 55(12):4013–4041, 2017. doi: 10.2514/1.J056060.
- Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. arXiv:2006.10739, 2020.
- Zachary Teed and Jia Deng. Raft: Recurrent all-pairs field transforms for optical flow. arXiv:2003.12039, 2020.
- Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000. doi: 10.1126/science.290.5500.2319.
- Gerald Tesauro. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995. doi: 10.1145/203330.203343.
- Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.
- N. Thuerey, B. Holzsuh, P. Holl, G. Kohl, M. Lino, Q. Liu, P. Schnell, and F. Trost. Physics-based deep learning. arXiv:2109.05237, 2021.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.
- Michael E. Tipping and Christopher M. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 61(3):611–622, 1999. doi: 10.1111/1467-9868.00196.
- Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. arXiv:2111.14522, 2021.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. arXiv:2302.13971, 2023.
- Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.
- A. M. TURING. I.—computing machinery and intelligence. *Mind*, LIX(236):433–460, 1950. doi: 10.1093/mind/LIX.236.433.
- Matthew Turk and Alex Pentland. Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1):71–86, 1991. doi: 10.1162/jocn.1991.3.1.71.
- Kiwon Um, Robert Brand, Yun, Fei, Philipp Holl, and Nils Thuerey. Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers.

- arXiv:2007.00016, 2020.
- Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. arXiv:2206.10498, 2022.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. arXiv:1706.03762, 2017.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. arXiv:1710.10903, 2017.
- Loup Verlet. Computer "experiments" on classical fluids. i. thermodynamical properties of lennard-jones molecules. *Physical Review*, 159(1):98–103, 1967. doi: 10.1103/PhysRev.159.98.
- Henk Kaarle Versteeg and Weeratunge Malalasekera. *An Introduction to Computational Fluid Dynamics: The Finite Volume Method*. Pearson Education, 2 edition, 2007.
- Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011. doi: 10.1162/NECO_a_00142.
- Ricardo Vinuesa and Steven L. Brunton. Enhancing computational fluid dynamics with machine learning. *Nature Computational Science*, 2(6):358–366, 2022. doi: 10.1038/s43588-022-00264-7.
- Sandra Wachter, Brent Mittelstadt, and Chris Russell. Counterfactual explanations without opening the black box: Automated decisions and the gdpr. arXiv:1711.00399, 2017. *Harvard Journal of Law & Technology*, 2018.
- Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021. doi: 10.1137/20M1318043.
- Sifan Wang, Xinling Yu, and Paris Perdikaris. When and why pinns fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi: 10.1016/j.jcp.2021.110768.
- Sifan Wang, Bowen Li, Yuhua Chen, and Paris Perdikaris. Piratenets: Physics-informed deep learning with residual adaptive networks. arXiv:2402.00326, 2024a.
- Sifan Wang, Shyam Sankaran, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024b. doi: 10.1016/j.cma.2024.116813.
- Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992. doi: 10.1007/BF00992698.
- Joseph L. Watson, David Juergens, Nathaniel R. Bennett, Brian L. Trippe, Jason Yim, Helen E. Eisenach, Woody Ahern, Andrew J. Borst, Robert J. Ragotte, Lukas F. Milles, Basile I. M. Wicky, Nikita Hanikel, Samuel J. Pellock, Alexis Courbet, William Sheffler,

- Jue Wang, Preetham Venkatesh, Isaac Sappington, Susana Vázquez Torres, Anna Lauko, Valentin De Bortoli, Emile Mathieu, Sergey Ovchinnikov, Regina Barzilay, Tommi S. Jaakkola, Frank DiMaio, Minkyung Baek, and David Baker. De novo design of protein structure and function with rfdiffusion. *Nature*, 620(7976):1089–1100, 2023. doi: 10.1038/s41586-023-06415-8.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. arXiv:2201.11903, 2022.
- David Weininger. Smiles, a chemical language and information system. 1. introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988. doi: 10.1021/ci00057a005.
- Mark Weiser. The computer for the 21st century. *Scientific American*, 265(3):94–104, 1991. doi: 10.1038/scientificamerican0991-94.
- Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992. doi: 10.1007/BF00992696.
- David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.
- Haixu Wu, Huakun Luo, Haowen Wang, Jianmin Wang, and Mingsheng Long. Transolver: A fast transformer solver for pdes on general geometries. arXiv:2402.02366, 2024.
- Yiheng Xie, Towaki Takikawa, Shunsuke Saito, Or Litany, Shiqin Yan, Numair Khan, Federico Tombari, James Tompkin, Vincent Sitzmann, and Srinath Sridhar. Neural fields in visual computing and beyond. arXiv:2111.11426, 2021.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? arXiv:1810.00826, 2018.
- Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. arXiv:2203.03466, 2022.
- Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, Tommi Jaakkola, Klavs Jensen, and Regina Barzilay. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019. doi: 10.1021/acs.jcim.9b00237.
- Lihe Yang, Bingyi Kang, Zilong Huang, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything: Unleashing the power of large-scale unlabeled data. arXiv:2401.10891, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. arXiv:2210.03629, 2022.

- Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? arXiv:1411.1792, 2014. Advances in Neural Information Processing Systems 27, pages 3320–3328. Dec. 2014.
- Hang Yuan, Shing Chan, Andrew P. Creagh, Catherine Tong, Aidan Acquah, David A. Clifton, and Aiden Doherty. Self-supervised learning for human activity recognition using 700,000 person-days of wearable data. *npj Digital Medicine*, 7(1), 2024. doi: 10.1038/s41746-024-01062-3.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. Deep sets. arXiv:1703.06114, 2017.
- Matthew D. Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. *Lecture Notes in Computer Science*, pages 818–833, 2014. doi: 10.1007/978-3-319-10590-1_53.
- Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, Yinghai Lu, and Yu Shi. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. arXiv:2402.17152, 2024.
- Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. arXiv:1611.03530, 2016.
- Yukun Zhou, Mark A. Chia, Siegfried K. Wagner, Murat S. Ayhan, Dominic J. Williamson, Robbert R. Struyven, Timing Liu, Moucheng Xu, Mateo G. Lozano, Peter Woodward-Court, Yuka Kihara, UK Biobank Eye & Vision Consortium, Naomi Allen, John E. J. Gallacher, Thomas Littlejohns, Tariq Aslam, Paul Bishop, Graeme Black, Panagiotis Sergouniotis, Denize Atan, Andrew D. Dick, Cathy Williams, Sarah Barman, Jenny H. Barrett, Sarah Mackie, Tasanee Braithwaite, Roxana O. Carare, Sarah Ennis, Jane Gibson, Andrew J. Lotery, Jay Self, Usha Chakravarthy, Ruth E. Hogg, Euan Paterson, Jayne Woodside, Tunde Peto, Gareth McKay, Bernadette McGuinness, Paul J. Foster, Konstantinos Balaskas, Anthony P. Khawaja, Nikolas Pontikos, Jugnoo S. Rahi, Gerassimos Lascaratos, Praveen J. Patel, Michelle Chan, Sharon Y. L. Chua, Alexander Day, Parul Desai, Cathy Egan, Marcus Fruttiger, David F. Garway-Heath, Alison Hardcastle, Sir Peng T. Khaw, Tony Moore, Sobha Sivaprasad, Nicholas Strouthidis, Dhanes Thomas, Adnan Tufail, Ananth C. Viswanathan, Bal Dhillon, Tom Macgillivray, Cathie Sudlow, Veronique Vitart, Alexander Doney, Emanuele Trucco, Jeremy A. Guggenheim, James E. Morgan, Chris J. Hammond, Katie Williams, Pirro Hysi, Simon P. Harding, Yalin Zheng, Robert Luben, Phil Luthert, Zihan Sun, Martin McKibbin, Eoin O’Sullivan, Richard Oram, Mike Weedon, Chris G. Owen, Alicja R. Rudnicka, Naveed Sattar, David Steel, Irene Stratton, Robyn Tapp, Max M. Yates, Axel Petzold, Savita Madhusudhan, Andre Altmann, Aaron Y. Lee, Eric J. Topol, Alastair K. Denniston, Daniel C. Alexander, and Pearse A. Keane. A foundation model for generalizable disease detection from retinal images. *Nature*, 622(7981):156–163, 2023a. doi: 10.1038/s41586-023-06555-x.
- Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu.

Dnabert-2: Efficient foundation model and benchmark for multi-species genome. arXiv:2306.15006, 2023b.

Shoshana Zuboff. *The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power*. PublicAffairs, 2019.

Indeks

- k*-fold cross-validation, 11
- 2-opt, 37
- action-value function, 124
- activation, 50
- active learning, 190
- actor-critic, 128
- ADME, 187
- admissible, 36
- advantage, 128
- Airborne optical sectioning, 80
- algorithm unrolling, 149
- aliasing, 202
- alpha-beta pruning, 38
- alternating least squares, 261
- anthropomorphism, 271
- aperture problem, 84
- artificial general intelligence, 273
- assay, 186
- attention, 67
- automatic differentiation, 52
- average linkage, 46
- average precision, 85, 263
- backoff, 75
- backprojection, 204
- backpropagation through time, 64
- backtracking, 29
- bag of words, 74
- base calling, 180
- baseline, 128
- batch normalization, 54
- Bayes risk, 8
- Bayes-optimal predictor, 8
- Bayesian network, 106
- Bayesian optimization, 61
- behavioral cloning, 129
- best linear unbiased estimator, 142
- bias correction, 31
- biclustering, 46
- binary cross-entropy, 8
- binding pocket, 195
- bootstrapping, 126
- boxplot, 17
- branch, 237
- branch and bound, 40
- breadth-first search, 36
- Bremsstrahlung, 203
- brightness constancy, 84
- buoyant jet, 257
- butterfly, 223
- byte-pair encoding, 283
- camera calibration, 80
- causal mask, 67
- cell under test, 147
- cell-averaging, 147
- cell-to-cell shifts, 255
- chain-of-thought, 286
- chart, 103
- chirp, 148
- circulant matrix, 101
- classifier-free guidance, 93
- clique tree, 108
- closure problem, 220
- coarse graining, 98
- cocktail party problem, 44
- cold start, 263
- collaborative filtering, 260
- collocation points, 230
- complete linkage, 46
- compound series bias, 188
- computational fluid dynamics, 2
- computational imaging, 87

- computed torque, 170
- condition number, 28
- conditional flow matching, 121
- conditional independence, 106
- confirmed, 174
- conformal prediction, 15
- confusion matrix, 14
- conjugate, 115
- conjugate gradient, 30
- connectionist temporal classification, 180
- constant error carousel, 65
- constant false alarm rate, 147
- contrastive learning, 189
- convex, 27
- convolution theorem, 81
- counterfactual explanation, 136
- coupled, 218
- coupling layer, 92
- covariate shift, 129
- criticism, 137
- cross-correlation, 56
- cross-entropy, 8
- curse of dimensionality, 13

- d-separated, 107
- data assimilation, 112
- decoherence, 279
- deconvolution, 81
- delta, 51
- demographic parity, 270
- demosaiing, 81
- denoising score matching, 121
- depth of field, 79
- depth-first search, 36
- depthwise-separable convolution, 90
- derajat kebebasan, 24
- descent lemma, 28
- difference of Gaussians, 82
- differentiable physics, 222, 249
- differential privacy, 269
- diffusion, 119
- direct numerical simulation, 215
- discounted cumulative gain, 262

- disparity, 86
- distribution learning, 191
- distribution shift, 14
- distributional semantics, 74
- double descent, 12
- drift, 119
- duty cycling, 161
- dwell time, 165
- dynamic Bayesian network, 110
- dynamic mode decomposition, 244
- dynamic programming, 40

- edge of stability, 33
- eigenfaces, 82
- empirical risk, 8
- empirical risk minimization, 8
- encode–process–decode, 238
- entangled, 279
- epipolar line, 86
- equal error rate, 162
- equal opportunity, 270
- equivariant, 97
- evidence, 19, 115
- evidence framework, 20
- evidence lower bound, 90
- expected improvement, 61
- experience replay, 127
- explaining away, 107
- exploitation, 127
- exploration, 127
- exponential gating, 69
- extended Kalman filter, 111, 173

- f-number, 79
- face swaps, 255
- false acceptance rate, 162
- false diffusion, 217
- false negative, 14
- false positive, 14
- false rejection rate, 162
- fast gradient sign method, 94
- federated learning, 192
- feed-forward, 51
- few-shot, 190

- filtered backprojection, 205
- filtering, 110
- Fisher information, 114
- flow, 121
- forget gate, 65
- forward algorithm, 110
- forward kinematics, 169
- forward mode, 52
- forward sampling, 108
- foundation model, 283
- gating, 65, 173
- gaussian process, 117
- gaussian pyramid, 82
- generalization error, 8
- generalized advantage estimation, 128
- genetic drift, 178
- geodesic, 103
- Gibbs sampling, 109
- goal-directed learning, 191
- gradient accumulation, 61
- gradient clipping, 32, 65
- greedy, 244
- grokking, 15
- group action, 96
- growing, 54
- grup, 96
- h-index, 305
- heavy ball, 31
- Hessian modification, 29
- heuristic, 36
- hit, 185
- homogeneous coordinates, 80
- homogeneous space, 102
- homography, 86
- homomorphism, 96
- Hough transform, 83
- hyperparameter, 11
- i.i.d., 7
- ill-posed, 232
- importance sampling, 108, 228
- independent component analysis, 44
- individual conditional expectation, 135
- inductive bias, 3
- initialized, 174
- instance segmentation, 82
- integer programming, 40
- intermediate frequency, 148
- interpolation, 75
- interpolation threshold, 12
- intersection over union, 85
- invariance, 57
- invariant, 97
- inverse kinematics, 169
- inverse reinforcement learning, 129
- iterative closest point, 172
- jitter, 157
- Kalman gain, 111
- kernel, 56
- kernel PCA, 44
- kernel trick, 25
- key, 67
- knowledge distillation, 285
- konsisten, 23
- Laplace smoothing, 75
- Laplace–Beltrami operator, 103
- large eddy simulation, 220
- latent variable model, 115
- layer normalization, 54
- lead, 185
- learning bias, 3
- learning rate, 28
- learning-based MPC, 158
- lemmatization, 73
- light field, 80
- likelihood weighting, 108
- line search, 28
- linear programming, 39
- linkage disequilibrium, 178
- local search, 37
- locally linear embedding, 45
- log-derivative trick, 128
- loss function, 7
- loss scaling, 61

- manifold hypothesis, 13
- Markov blanket, 107
- Markov decision process, 124
- Markov localization, 172
- masked language model, 76
- matched filter, 147
- maximum a posteriori, 9
- maximum likelihood, 8
- mean field theory, 53
- mean shift, 82
- mean squared error, 8
- mechanistic interpretability, 138
- memory mixing, 69
- message passing, 99
- mini-batch, 30
- minimax, 23, 37
- minimum variance unbiased, 114, 142
- misadjustment, 144
- mixed precision, 61
- mixture of experts, 284
- ML for Vis, 133
- mode collapse, 92
- model class, 7
- molecular descriptor, 186
- molecular fingerprint, 186
- Monte Carlo, 126
- Monte Carlo tree search, 38
- most likely explanation, 110
- multi-fidelity, 250
- multi-head attention, 67
- multi-objective optimization, 191
- multi-query associative recall, 70
- multidimensional scaling, 45
- multigrid, 219
- multiple instance learning, 190
- mutual information, 199

- Nash equilibrium, 130
- negative sampling, 76
- network dissection, 137
- neural field, 87, 122
- neural ODE, 119
- neural tangent kernel, 94
- no free lunch, 14

- no-cloning, 279
- no-slip, 214
- noise figure, 145
- non-maximum suppression, 85
- non-negative matrix factorization, 45
- non-saturating loss, 91
- normal distributions transform, 172
- normal equations, 10
- normalized cut, 83
- nudging, 112, 250
- numerical diffusion, 217

- observational bias, 3
- operator, 236
- operator inference, 246
- operator learning, 236
- optical flow, 83
- options, 130
- orbit, 97
- order-statistic, 147
- orthogonality principle, 143
- overlap–layout–consensus, 182
- overshoot, 155
- oversmoothing, 104
- oversquashing, 101

- parameter-shift rule, 281
- parametrik, 244
- Pareto front, 191
- partial dependence plot, 135
- particle filter, 111
- particle image velocimetry, 85
- peer review, 304
- perceptron, 50
- periodogram, 141
- perplexity, 75
- pointwise mutual information, 74
- policy, 124
- policy evaluation, 125
- policy improvement, 125
- policy iteration, 125
- polysemanticity, 138
- popularity bias, 264
- positional encoding, 67, 100

- positive PMI, 74
- post up-projection, 70
- posterior, 9
- power spectral density, 141
- pre up-projection, 70
- pre-activation, 50, 89
- prediction, 110
- principal component analysis, 82
- prior, 9
- privacy by design, 269
- probability flow ODE, 121
- probability path, 121
- probe set, 179
- programmable logic controller, 157
- projected gradient ascent, 94
- proper orthogonal decomposition, 82, 242
- proteochemometrics, 188
- prototype, 137
- pruning, 54
- pulse compression, 147

- quantitative structure–activity relationship, 187
- quasi-Newton, 30
- query, 67

- radar cross section, 145
- range–Doppler map, 148
- readout, 189
- real-time recurrent learning, 64
- receptive field, 56
- reciprocal rank, 263
- reconstruction conjecture, 101
- rectification, 86
- rectified flow, 121
- recurrence matrix, 253
- recurrent neural network, 64
- reduced-order model, 243
- reinforcement learning, 7
- reinforcement learning from human feedback, 131
- rejection sampling, 108
- reparameterization trick, 91

- representasi, 97
- residual, 230
- retrosynthesis, 193
- return, 124
- reverse mode, 52
- reward, 7
- reward hacking, 274
- Reynolds stress, 220
- ridge regression, 9
- Riemannian metric, 103
- risk minimizer, 22
- robust, 17
- root mean square deviation, 196

- saddle point, 27
- saliency map, 136
- scale separation, 98
- scaling laws, 283
- scattering transform, 102
- score, 121
- secant condition, 30
- seed and extend, 181
- segregated solver, 218
- selective search, 85
- self-normalizing networks, 54
- self-organizing map, 45
- self-regularization, 31
- self-supervised learning, 7, 48
- semantic ID, 265
- semantic segmentation, 82
- serendipity, 263
- shooting, 225
- signed distance field, 122
- simplektik, 225
- simulated annealing, 228
- simulation-based inference, 112
- single linkage, 46
- single nucleotide polymorphism, 178
- singular, 169
- sinogram, 204
- skip connection, 83
- smoothed particle hydrodynamics, 228
- smoothing, 110
- social scoring, 267

- sociological imagination, 270
- solver-in-the-loop, 248
- sparse, 46
- sparse identification of nonlinear dynamics, 245
- spectral bias, 122
- spin tensor, 168
- split-operator, 227
- squeeze-and-excitation, 90
- stabilizable, 153
- state space model, 68
- stationary point, 27
- steepest descent, 144
- stemming, 73
- stiff, 118
- stochastic approximation, 30
- stochastic differential equation, 119
- stochastic gradient descent, 30
- stop words, 73
- strongly convex, 28
- structure from motion, 86
- superdense coding, 279
- superintelligence, 273
- superpixel, 135
- superposition, 138
- supervised learning, 7
- synthetic aperture, 80
- tangent space, 103
- tangible user interface, 166
- target, 185
- target identification, 188
- target network, 127
- target prediction, 188
- TD error, 126
- technological determinism, 270
- teleportation, 279
- temporal difference, 126
- tentative, 174
- threading, 197
- time-delay neural network, 64
- training error, 8
- transfer learning, 90
- translation equivariance, 57
- transposed convolution, 57
- travelling salesman problem, 37
- true negative, 14
- true positive, 14
- trunk, 237
- trust region, 30
- uncanny valley, 271
- under-relaxation, 218
- uniform-cost search, 36
- universal approximation theorem, 51
- universal differential equation, 247
- universally consistent, 23
- unrolled, 149
- unsupervised learning, 7
- upwind, 217
- v-structure, 107
- value, 67
- value function, 124
- value iteration, 125
- vanishing gradient, 65
- variable elimination, 108
- variant calling, 182
- variational quantum circuit, 281
- virtual screening, 188
- vis for ML, 133
- vision transformer, 87, 90
- vision-language-action, 287
- visual question answering, 77
- volume of fluid, 221
- volume rendering, 87
- wall function, 220
- warm-up, 32
- weight decay, 9
- weight sharing, 56
- Weisfeiler–Leman, 100
- whitening, 44
- witness rate, 190
- word embedding, 74
- world models, 288
- zero padding, 141
- zero-shot, 71
- zero-shot learning, 190